
DATA SCIENCE

Série 1 — Volume III

Apprentissage automatique

Démarche et validation · Modèles supervisés · Non supervisé
Variables et déséquilibre · Évaluation et interprétabilité

De la démarche aux modèles interprétables,
avec une boutique pour fil rouge.

Table des matières

Introduction : statistique ou apprentissage automatique ?	1
Là où le volume II nous a laissés	1
Expliquer ou prédire ?	1
Deux questions sur les mêmes données	1
Deux cultures qui se rejoignent	2
À qui s'adresse ce volume ?	3
La carte du volume	3
Six idées qui reviennent dans tout le volume	4
Comment travailler avec ce volume	5
Un livre, et son cahier	5
Le rythme d'une section	5
Le code dans ce livre	5
Des données simulées, et un jeu réel	6
Reproductibilité d'un résultat d'apprentissage automatique	6
Les données et l'environnement du volume	7
Les jeux de la boutique	7
Les clients : clients_ml.csv	8
Les commandes en ligne : transactions.csv	9
Les achats : interactions.csv et produits_ml.csv	10
Un jeu réel : credit_default.csv	10
Les jeux embarqués de scikit-learn	11
L'environnement de travail	11
Chapitre 1 : La démarche d'apprentissage automatique	13
Le chemin de ce chapitre	13
Les données du chapitre	14
1.1 Formulation du problème et séparation des données	15
1.1.1 Un exemple minuscule, entièrement à la main	15
1.1.2 Le cadre : prédire, perte, risque	16
1.1.3 La perte n'est pas la métrique	16
1.1.4 L'unité d'analyse et le moment de la prédiction	17
1.1.5 Séparer les données : entraînement, validation, test	18
1.1.6 La fuite d'information	19
1.2 Validation croisée	20
1.2.1 Un seul découpage ne suffit pas	20
1.2.2 La validation croisée à k plis	21
1.2.3 Les variantes : stratifiée, répétée, groupée, temporelle	23
1.2.4 Quelle confiance accorder à l'estimation ?	24

1.2.5 La validation croisée imbriquée : un aperçu	24
1.3 Compromis biais-variance et surapprentissage	25
1.3.1 Deux façons de se tromper	25
1.3.2 Un exemple à la main : estimer une moyenne	26
1.3.3 La décomposition biais-variance	26
1.3.4 Voir le compromis : un polynôme de degré croissant	27
1.3.5 Sous-apprentissage, surapprentissage	28
1.3.6 Les courbes d'apprentissage : diagnostiquer son modèle	28
1.3.7 Que faire quand le modèle ne généralise pas ?	29
1.4 Modèles de référence et rigueur expérimentale	30
1.4.1 Le modèle de référence : l'adversaire à battre	30
1.4.2 Comparer deux modèles : l'écart est-il réel ?	31
1.4.3 Graines, reproductibilité et variabilité	32
1.4.4 Le jeu de test, une seule fois	33
1.4.5 Rapporter honnêtement : une liste de contrôle	34
1.5 ⊕ Pour aller plus loin : le réglage des hyperparamètres	34
1.5.1 Paramètres et hyperparamètres	35
1.5.2 La grille et la recherche aléatoire	35
1.5.3 L'optimisation bayésienne : apprendre des essais précédents	36
1.5.4 L'arrêt précoce	37
1.5.5 Le piège du réglage et la validation imbriquée	37
Bilan du chapitre 1	39
Chapitre 2 : Apprentissage supervisé	41
Le chemin de ce chapitre	41
Le problème qui nous accompagne	41
Ce qu'« apprendre » veut dire	42
2.1 Modèles linéaires et logistiques vus sous l'angle du ML	42
2.1.1 Une perte pour chaque philosophie	43
2.1.2 La régression logistique, une perte et un gradient	44
2.1.3 La descente de gradient : un pas à la main	45
2.1.4 Pourquoi il faut mettre les variables à l'échelle	45
2.1.5 La pénalité : une contrainte déguisée	46
2.1.6 Ce qu'un score linéaire ne sait pas écrire	48
2.2 Arbres de décision	49
2.2.1 Un arbre minuscule, entièrement à la main	49
2.2.2 Mesurer l'impureté	50
2.2.3 Le meilleur découpage : un par un	51
2.2.4 Les arbres de régression	52
2.2.5 Jusqu'où laisser pousser l'arbre ?	52
2.2.6 Ce que les arbres savent écrire : effets non linéaires et interactions	54
2.2.7 Le point faible : l'instabilité	55
2.3 Forêts aléatoires	56
2.3.1 Le bagging : moyenner des arbres entraînés sur des échantillons bootstrap	56
2.3.2 Pourquoi moyenner réduit la variance : la formule	57
2.3.3 La forêt aléatoire : décorrélérer les arbres	57
2.3.4 L'erreur « hors sac » : une validation gratuite	58
2.3.5 Combien d'arbres ? Quels réglages ?	58
2.3.6 Quelles variables comptent ? Importance et ses pièges	59
2.4 Gradient boosting : XGBoost et LightGBM	61
2.4.1 Un exemple à la main : deux tours de boosting	61
2.4.2 Le gradient boosting : une descente de gradient dans l'espace des fonctions	61

2.4.3 XGBoost : un objectif régularisé et une formule fermée	63
2.4.4 LightGBM et le choix d'une stratégie de croissance	63
2.4.5 Régler le boosting : pas, nombre d'arbres et arrêt précoce	64
2.4.6 Valeurs manquantes et catégories : sans artifice	65
2.4.7 Le match : régression logistique, arbre, forêt, boosting	66
2.5 ⊕ Pour aller plus loin : SVM et noyaux, k plus proches voisins, Bayes naïf	68
2.5.1 Les machines à vecteurs de support (SVM)	68
2.5.2 Les k plus proches voisins (k-NN)	70
2.5.3 Bayes naïf	71
2.5.4 Comment choisir entre elles ?	71
2.6 ⊕ Pour aller plus loin : CatBoost, stacking et blending	72
2.6.1 CatBoost et le piège de l'encodage par la cible	72
2.6.2 Combiner des modèles : le blending	73
2.6.3 Le stacking : laisser un modèle apprendre à combiner	74
2.6.4 Quand cela vaut-il la peine ?	74
Bilan du chapitre 2	75
Chapitre 3 : Apprentissage non supervisé et réduction de dimension	77
Le chemin de ce chapitre	77
Les données de ce chapitre	78
3.1 Classification non supervisée et validation	78
3.1.1 Le problème : des clients, aucune étiquette	78
3.1.2 Critères internes : compacité et séparation	79
3.1.3 Y a-t-il seulement des groupes ? La référence sans structure	81
3.1.4 Stabilité : les mêmes groupes si l'on change l'échantillon ?	82
3.1.5 Validation externe : confronter les groupes à une vérité connue	83
3.1.6 Les variables et l'échelle décident des groupes	84
3.1.7 Lire les groupes : de la partition à la décision	85
3.2 Réduction de dimension	86
3.2.1 Pourquoi réduire : la malédiction de la dimension	86
3.2.2 L'ACP en une page, et ce qu'elle garde	87
3.2.3 Quand la structure n'est pas linéaire : l'ACP à noyau	88
3.2.4 SVD tronquée : quand on ne peut pas centrer	90
3.2.5 La factorisation non négative : décrire par parties	90
3.2.6 Les projections aléatoires et le lemme de Johnson–Lindenstrauss	90
3.2.7 Combien de dimensions garder ? Et l'idée de variété	92
3.2.8 Réduire avant de classer : sur les clients de la boutique	93
3.3 ⊕ Pour aller plus loin : DBSCAN, classification hiérarchique, mélanges gaussiens	94
3.3.1 DBSCAN : les groupes comme régions denses	95
3.3.2 La classification hiérarchique, revue par les liens	97
3.3.3 Les mélanges gaussiens : des appartenances « floues »	98
3.4 ⊕ Pour aller plus loin : t-SNE et UMAP	101
3.4.1 t-SNE : conserver les voisinages	101
3.4.2 UMAP : un graphe de voisins, puis un dessin	102
3.4.3 Les chiffres manuscrits, vus par trois méthodes	102
3.4.4 Ce qu'on n'a pas le droit de lire sur un dessin	103
3.4.5 Un dernier regard : les clients de la boutique	104
Bilan du chapitre 3	105
Chapitre 4 : Ingénierie des variables et données déséquilibrées	107
Le chemin de ce chapitre	107
Les données du chapitre	108

4.1 Encodage, mise à l'échelle, transformations	108
4.1.1 Deux familles de modèles, deux besoins	108
4.1.2 Encoder une variable qualitative	109
4.1.3 Mettre les variables à la même échelle	111
4.1.4 Corriger l'asymétrie : logarithme, Box-Cox, Yeo-Johnson	113
4.1.5 Les valeurs manquantes	114
4.1.6 Tout assembler, sans fuite : Pipeline et ColumnTransformer	115
4.2 Création et sélection de variables	117
4.2.1 Penser métier : des variables qui ont du sens	117
4.2.2 Découvrir les seuils à partir des données	117
4.2.3 Fabriquer, puis mesurer	118
4.2.4 Les variables cycliques : l'exemple de l'heure	119
4.2.5 Agréger : passer de plusieurs lignes à une ligne par client	120
4.2.6 Redondance, variance nulle et fuite	120
4.3 Classes déséquilibrées : pondération et rééchantillonnage	121
4.3.1 Pourquoi l'exactitude trompe	121
4.3.2 La pondération des classes	122
4.3.3 Le rééchantillonnage aléatoire	124
4.3.4 Le seuil de décision : le levier oublié	124
4.3.5 La vue par les coûts	125
4.3.6 Comparer honnêtement : et le déséquilibre modéré ?	127
4.4 ⊕ Pour aller plus loin : les méthodes de sélection de variables	128
4.4.1 Trois familles de méthodes	128
4.4.2 Les filtres : information mutuelle et χ^2	128
4.4.3 Les enveloppes : RFE et sélection progressive	129
4.4.4 Les méthodes intégrées : ℓ_1 , importances d'arbre, permutation	129
4.4.5 Le piège : sélectionner avant de valider	130
4.5 ⊕ Pour aller plus loin : SMOTE et ses variantes	131
4.5.1 L'idée : interpoler entre voisins	131
4.5.2 Les variantes	132
4.5.3 Dans un pipeline, et seulement dans le pli d'entraînement	133
4.5.4 Faut-il vraiment rééchantillonner ? Une comparaison honnête	133
4.5.5 Que retenir de SMOTE ?	134
Bilan du chapitre 4	135
Chapitre 5 : Évaluation, calibration et interprétabilité	136
Le chemin de ce chapitre	136
Le fil rouge : un même problème, trois modèles	137
5.1 Métriques	137
5.1.1 Ce que l'on mesure : un score, une décision, une valeur	138
5.1.2 La matrice de confusion et ses dérivés	138
5.1.3 Le seuil change tout	139
5.1.4 La courbe ROC et l'AUC	140
5.1.5 Précision-rappel : ce que la ROC ne voit pas	141
5.1.6 Juger les probabilités : log-loss et score de Brier	141
5.1.7 Choisir le seuil par les coûts	142
5.1.8 Gain et lift : la lecture du marketing	143
5.1.9 Plus de deux classes	144
5.1.10 Régression : choisir sa perte	144
5.2 Calibration	145
5.2.1 Qu'est-ce qu'une probabilité « juste » ?	146
5.2.2 Qui est bien calibré, qui ne l'est pas ?	146

5.2.3 Réparer : la méthode de Platt	148
5.2.4 Réparer : la régression isotonique	148
5.2.5 Calibrer sans tricher	149
5.2.6 Poids de classes et rééchantillonnage : un décalage de prévalence	150
5.3 Interprétabilité	151
5.3.1 Expliquer quoi, et à qui ?	151
5.3.2 L'importance par permutation	152
5.3.3 Effets moyens et individuels : PDP et ICE	152
5.3.4 LIME : un modèle simple autour d'un client	153
5.3.5 Les valeurs de Shapley	154
5.3.6 SHAP : Shapley appliqué à un modèle	155
5.3.7 Les pièges de l'interprétation	157
5.4 ⊕ Pour aller plus loin : équité, biais et éthique	159
5.4.1 Un jeu de données réel, et un avertissement	159
5.4.2 Mesurer : les critères d'équité	159
5.4.3 Supprimer la variable sensible ne suffit pas	160
5.4.4 Pourquoi on ne peut pas tout avoir	161
5.4.5 Corriger : des seuils par groupe	161
5.4.6 L'éthique commence où le calcul s'arrête	163
5.5 ⊕ Pour aller plus loin : quantifier l'incertitude, la prédiction conforme	163
5.5.1 L'objectif : une garantie de couverture	164
5.5.2 La recette : la prédiction conforme « séparée »	164
5.5.3 Pourquoi cela marche	165
5.5.4 Garantie marginale, pas conditionnelle	165
5.5.5 En régression : des intervalles avec garantie	166
5.5.6 Limites et bonnes pratiques	167
Bilan du chapitre 5	168
Chapitre 6 : ⊕ Détection d'anomalies et de fraude	170
Le chemin de ce chapitre	170
Les données et le protocole	171
6.1 Le problème et son évaluation	172
6.1.1 Qu'est-ce qu'une anomalie ?	172
6.1.2 Pourquoi l'apprentissage supervisé ne suffit pas toujours	173
6.1.3 L'exactitude ne veut plus rien dire	173
6.1.4 Un modèle supervisé de référence	174
6.1.5 Les bons outils de mesure	174
6.1.6 Du score à l'argent	176
6.2 Méthodes statistiques, distances et densités	177
6.2.1 Le z-score, et pourquoi il faut parfois le rendre robuste	177
6.2.2 La distance de Mahalanobis	178
6.2.3 Les plus proches voisins	179
6.2.4 Le facteur local d'anomalie (LOF)	182
6.2.5 Ce que voient ces méthodes, et ce qu'elles ratent	184
6.3 La forêt d'isolement	184
6.3.1 L'idée : isoler plutôt que décrire	185
6.3.2 La longueur de chemin et le score d'anomalie	185
6.3.3 De l'arbre à la forêt	186
6.3.4 Le paramètre de contamination et le seuil	187
6.3.5 Sur nos transactions	187
6.4 Autoencodeurs, et comparaison des méthodes	188
6.4.1 L'idée : comprimer, puis reconstruire	188

6.4.2 L'ACP est un autoencodeur linéaire	189
6.4.3 Un autoencodeur non linéaire avec scikit-learn	190
6.4.4 La version PyTorch	191
6.4.5 Comparer honnêtement les méthodes	192
Bilan du chapitre 6	194
Chapitre 7 : ☉ Systèmes de recommandation	195
Le chemin de ce chapitre	195
Les données et le protocole	196
7.1 Données d'interaction et références simples	196
7.1.1 Ce que l'on recommande, et à qui	197
7.1.2 Retours explicites et implicites	197
7.1.3 La matrice d'interactions	197
7.1.4 La similarité cosinus	199
7.1.5 La référence : la popularité	199
7.1.6 Le filtrage par contenu	200
7.2 Filtrage collaboratif : les méthodes de voisinage	201
7.2.1 L'intelligence collective	201
7.2.2 Les voisins de clients	201
7.2.3 Les voisins de produits	202
7.2.4 Rétrécissement et choix du voisinage	203
7.2.5 Premiers résultats et limites	203
7.3 Factorisation matricielle	205
7.3.1 L'idée : des facteurs latents	205
7.3.2 L'objectif : des moindres carrés régularisés	206
7.3.3 Deux algorithmes	206
7.3.4 Une itération à la main	207
7.3.5 Retours implicites : donner aux cases vides un poids faible	207
7.3.6 La SVD tronquée comme référence	208
7.3.7 Choisir k et λ , et ce que valent les facteurs	208
7.4 Évaluation et démarrage à froid	211
7.4.1 Évaluer un classement	211
7.4.2 Un protocole honnête et des comparaisons avec incertitude	212
7.4.3 Pourquoi un découpage aléatoire peut tromper	214
7.4.4 Au-delà de la précision : couverture, nouveauté, biais de popularité	215
7.4.5 Le démarrage à froid	216
7.4.6 Les recommandations changent les données	217
Bilan du chapitre 7	219
Chapitre 8 : ☉ Apprentissage semi-supervisé et actif	220
Le chemin de ce chapitre	221
Les données du chapitre	221
8.1 Pourquoi et hypothèses	222
8.1.1 Le problème de l'étiquette chère	222
8.1.2 Ce que les données non étiquetées apportent	222
8.1.3 Les quatre hypothèses qui permettent d'aider	223
8.1.4 Quand ça aide, quand ça nuit	225
8.1.5 Évaluer à budget d'étiquettes fixé : la courbe d'apprentissage	225
8.2 Auto-apprentissage et propagation d'étiquettes	227
8.2.1 L'auto-apprentissage	227
8.2.2 Pourquoi l'auto-apprentissage peut s'auto-tromper	228
8.2.3 Le co-apprentissage	229

8.2.4 Propager les étiquettes le long d'un graphe	229
8.2.5 La propagation sur les chiffres manuscrits	231
8.2.6 Quand le graphe ne dit rien : les clients de la boutique	233
8.3 Apprentissage actif	234
8.3.1 L'idée et la boucle	234
8.3.2 Mesurer l'incertitude	235
8.3.3 Le comité de modèles	236
8.3.4 Densité et représentativité	237
8.3.5 Comparer les stratégies	237
8.3.6 Le piège du biais d'échantillonnage	239
8.3.7 Le démarrage à froid	240
8.3.8 Combien ça rapporte ?	240
Bilan du chapitre 8	241
Chapitre 9 : ☉ Bases de l'apprentissage par renforcement	243
Le chemin de ce chapitre	243
Ce qui change par rapport à l'apprentissage supervisé	244
Trois petits mondes pour tout le chapitre	244
9.1 Processus de décision markoviens	245
9.1.1 Apprendre en agissant : la boucle agent-environnement	245
9.1.2 Récompense, retour et actualisation	245
9.1.3 États, actions, transitions : le processus de décision markovien	246
9.1.4 Politiques et fonctions de valeur	247
9.1.5 Les équations de Bellman : démonstration	247
9.1.6 L'équation d'optimalité	248
9.1.7 L'itération de la valeur : un exemple calculé à la main	248
9.1.8 Pourquoi l'itération converge : un argument de contraction	250
9.1.9 Le problème de stock	250
9.1.10 Un couloir d'entrepôt : la valeur comme distance au but	251
9.2 Bandits manchots	252
9.2.1 Le dilemme exploration-exploitation	253
9.2.2 Mesurer le coût de l'ignorance : le regret	253
9.2.3 Le test A/B, vu comme une stratégie	253
9.2.4 La stratégie ϵ -glouton	254
9.2.5 UCB : l'optimisme face à l'incertitude	254
9.2.6 L'échantillonnage de Thompson	255
9.2.7 Comparaison des stratégies	256
9.2.8 Quand le meilleur choix dépend du contexte	257
9.2.9 Précautions	257
9.3 Q-learning : apprendre sans connaître les règles	258
9.3.1 Apprendre sans modèle	258
9.3.2 La différence temporelle	259
9.3.3 Le Q-learning	259
9.3.4 SARSA, ou apprendre en tenant compte de ses propres erreurs	260
9.3.5 Explorer pendant l'apprentissage	261
9.3.6 Quand le Q-learning converge	261
9.3.7 Sur le problème de stock	262
9.3.8 Les limites de la table	262
9.4 Du Q-learning à l'apprentissage profond	263
9.4.1 Quand la table explose	263
9.4.2 Approximer la fonction de valeur : le principe du DQN	263
9.4.3 Optimiser directement la politique	264

9.4.4 Pourquoi l'apprentissage devient instable	264
9.4.5 Apprendre à partir de données déjà collectées	264
9.4.6 La récompense, ou ce que l'on demande vraiment	265
9.4.7 Sécurité, éthique et responsabilité	265
9.4.8 Quand utiliser l'apprentissage par renforcement ?	265
Bilan du chapitre 9	266
Points clés	268
Les points clés, chapitre par chapitre	269
Six idées qui traversent tout le volume	270
Et maintenant ?	271

Introduction : statistique ou apprentissage automatique ?

« Un bon modèle n'est pas celui qui explique le mieux le passé, mais celui qui se trompe le moins sur ce qu'il n'a pas encore vu. »

Là où le volume II nous a laissés

Le volume II vous a appris à **construire et interpréter des modèles** : une droite de régression, un modèle logistique, un modèle de survie, une prévision. La question centrale était : *que disent les données du mécanisme qui les a produites ?* Un coefficient avait un sens, un intervalle de confiance en avait un autre, et les hypothèses du modèle se vérifiaient avec des diagnostics.

Ce volume change légèrement de question. Une gérante de boutique ne veut pas seulement savoir *pourquoi* certains clients s'en vont ; elle veut **repérer, ce mois-ci, ceux qui vont partir**, pour leur écrire avant qu'il ne soit trop tard. Elle ne demande pas un coefficient : elle demande une liste, et une raison de lui faire confiance.

Intuition. Expliquer, c'est regarder **en arrière** : comprendre ce qui s'est passé. Prédire, c'est regarder **en avant** : annoncer ce qui n'a pas encore eu lieu. Le premier se juge à la qualité de la compréhension, le second à la qualité des annonces *vérifiées sur des cas que le modèle n'a jamais vus*. Les deux démarches utilisent les mêmes mathématiques ; elles ne posent pas la même exigence.

Expliquer ou prédire ?

Deux questions sur les mêmes données

Prenons les douze mille clients de la boutique (présentés dans la section suivante) et une question : *le client va-t-il partir dans les 90 jours ?* On peut y répondre de deux manières.

- **La démarche d'explication** (volume II, section 2.2) : ajuster un modèle logistique et **lire ses coefficients**. Ici, 30 jours de plus sans commande multiplient les cotes de départ par 1,17 environ (par 1,94 pour un écart-type de récence, soit 124 jours), « toutes choses égales par ailleurs », avec un intervalle de confiance et des hypothèses que l'on vérifie. L'objet important est le coefficient.
- **La démarche de prédiction** : peu importe ce que valent les coefficients. On met de côté un quart des clients, on ajuste un modèle sur les trois autres quarts, puis on **mesure sa qualité**

sur les clients mis de côté. L'objet important est le score sur des cas nouveaux (ici l'AUC, qui vaut 0,5 pour un modèle sans valeur et 1 pour un modèle parfait : section 5.1).

Le résultat de la seconde démarche est instructif. Sur les clients mis de côté, le modèle logistique atteint un AUC de **0,87** et un modèle d'arbres boostés (chapitre 2) **0,90** : les deux sont utiles, mais le second classe mieux les clients. Remarquez aussi ce que le même modèle fait sur les clients qui ont servi à l'ajuster : l'AUC du boosting y monte à 0,99, alors que celui de la logistique reste à 0,87. Le premier chiffre est un auto-compliment ; seul le second (0,90 sur des clients inédits) est une mesure honnête, et l'écart entre les deux est le signe qu'un modèle souple peut en partie **apprendre par cœur** (chapitre 1, section 1.3). Le boosting serait en outre incapable de nous donner « le » coefficient de la récence : ses prédictions sortent de centaines d'arbres, pas d'une formule. Voilà le compromis central de ce volume : **la précision de la prédiction se paie souvent d'une perte de lisibilité**, que nous apprendrons à compenser (chapitre 5, section 5.3).

Deux cultures qui se rejoignent

Les deux démarches sont aussi anciennes l'une que l'autre, mais elles ont grandi dans des cultures différentes. Le tableau ci-dessous est une **caricature utile** : dans la pratique, les bons projets empruntent aux deux.

	Statistique (volume II)	Apprentissage automatique (volume III)
Question typique	Quel est l'effet de cette variable ? Cet écart est-il dû au hasard ?	Quelle valeur ce client aura-t-il demain ? Cette commande est-elle frauduleuse ?
Objectif	comprendre un mécanisme, quantifier une incertitude	prédire de nouveaux cas, le plus justement possible
Hypothèses	un modèle probabiliste explicite (normalité, indépendance, linéarité...)	peu d'hypothèses sur la forme ; on ne suppose que des cas tirés de la même source
Évaluation	tests, intervalles de confiance, diagnostics sur les résidus	score sur des données jamais vues (jeu de test, validation croisée)
Interprétabilité	native : un coefficient a un sens	à reconstruire : importances, explications locales
Données	quelques centaines à milliers de lignes, quelques variables choisies	de milliers à millions de lignes, des dizaines de variables parfois sans sens évident
Risque principal	un modèle mal spécifié, des hypothèses fausses	le surapprentissage et la fuite d'information

Ces deux mondes se rejoignent plus qu'on ne le dit. La régularisation (volume II, section 1.5), la validation croisée (volume II, section 1.4.3) et la régression logistique (volume II, section 2.2) sont déjà des outils d'apprentissage automatique. Réciproquement, un modèle prédictif qu'on n'ose pas expliquer ne sera pas utilisé : l'interprétabilité et l'équité sont une part de l'évaluation, pas un luxe (chapitre 5).

Prédire n'est pas expliquer, et encore moins causer. Un modèle qui prédit bien le départ des clients ne dit pas que *changer* la récence ferait changer le départ. Savoir quelle action déclencherait l'effet voulu est une question causale, traitée au volume II (chapitre 7, facultatif). Nous le rappellerons chaque fois que la tentation sera grande.

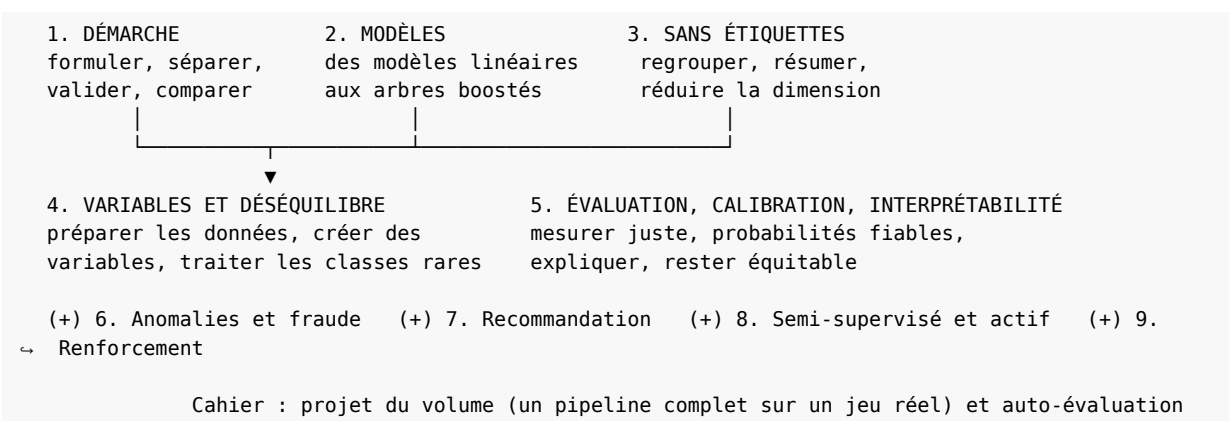
À qui s'adresse ce volume ?

Aux lecteurs des volumes I et II, ou à ceux qui en ont le bagage : statistique de base (intervalles de confiance, tests, bootstrap), régression linéaire et logistique, un peu d'algèbre et de dérivées, et de la pratique de Python. Chaque fois qu'un résultat des volumes précédents est utilisé, nous le citons (« volume II, section 2.2 ») pour que vous puissiez aller le relire.

Ce que vous devez maîtriser	Où le revoir
Gradient, descente de gradient	volume I, section 1.3 (en particulier 1.3.3)
Loi des grands nombres, théorème central limite	volume I, section 2.4
Intervalles de confiance, bootstrap	volume I, section 3.3 (en particulier 3.3.5)
Tests d'hypothèses, p-valeurs, tests multiples	volume I, sections 3.4 et 3.5
pandas, graphiques	volume I, sections 4.4 et 4.5
Régression linéaire, surajustement, critères d'information, validation croisée	volume II, sections 1.1 et 1.4
Régularisation (Ridge, Lasso)	volume II, section 1.5
Régression logistique, ROC et AUC	volume II, section 2.2
ACP, classification non supervisée (k-means)	volume II, sections 3.1 et 3.3

La carte du volume

Le volume contient **cinq chapitres principaux**, qui suivent le cycle de vie d'un projet, et **quatre chapitres complémentaires** facultatifs. Chaque chapitre a son pendant dans le **cahier d'exercices et d'applications** du volume.



Chapitre	Question centrale	Vous saurez...
1. La démarche	Comment savoir qu'un modèle fonctionnera sur des cas nouveaux ?	formuler un problème, séparer les données, valider sans tricher, comparer à un modèle de référence
2. Apprentissage supervisé	Quelle famille de modèles choisir, et pourquoi ?	utiliser modèles linéaires, arbres, forêts et gradient boosting, et comprendre leurs forces et leurs limites
3. Sans étiquettes	Que faire quand il n'y a rien à prédire ?	valider un regroupement, réduire la dimension, visualiser

Chapitre	Question centrale	Vous saurez...
4. Variables et déséquilibre	Comment préparer les données, et que faire quand l'événement est rare ?	encoder, transformer, créer des variables, pondérer ou rééchantillonner
5. Évaluation et interprétabilité	Mon modèle est-il bon, fiable, compréhensible, équitable ?	choisir la bonne métrique, calibrer, expliquer (SHAP, LIME), détecter les biais
⊕ 6. Anomalies et fraude	Comment repérer le très rare, sans toujours savoir à quoi il ressemble ?	utiliser forêt d'isolement, densités, autoencodeurs
⊕ 7. Recommandation	Quel produit proposer à quel client ?	construire et évaluer un système de recommandation
⊕ 8. Semi-supervisé et actif	Comment apprendre quand les étiquettes coûtent cher ?	exploiter des données non étiquetées et choisir quoi étiqueter
⊕ 9. Renforcement	Comment apprendre en agissant ?	raisonner en bandits manchots et en Q-learning
Cahier : projet	Peut-on tout assembler sur un vrai jeu ?	mener un pipeline complet de bout en bout

Les chapitres complémentaires. Les chapitres 6 à 9, ainsi que toutes les sections marquées ⊕, sont **facultatifs** : on peut lire les cinq premiers chapitres sans eux. Ils sont là pour celles et ceux qui veulent aller plus loin.

Six idées qui reviennent dans tout le volume

Avant de commencer, voici six idées qui traversent chaque chapitre. Elles valent mieux que n'importe quelle formule.

1. **La généralisation est le seul but.** Un modèle n'a de valeur que sur des cas qu'il n'a jamais vus. Ajuster parfaitement les données d'apprentissage ne prouve rien, et peut même être un mauvais signe (chapitre 1).
2. **Le jeu de test est sacré.** On le met de côté dès le début, on n'y touche qu'une fois à la fin, et on ne règle rien avec lui. Chaque fois qu'on le regarde avant d'avoir terminé, il devient un peu moins fiable.
3. **La fuite d'information est le piège numéro un.** Un modèle peut obtenir un score magnifique parce qu'il a eu accès, directement ou non, à ce qu'il doit prédire. Le piège est discret et ses effets sont spectaculaires : il se détecte par la méthode, pas par le flair.
4. **Un modèle de référence avant la sophistication.** Avant d'essayer un modèle complexe, on mesure ce que fait une règle bête (toujours prédire la classe majoritaire) et un modèle simple. Un modèle complexe qui ne fait pas mieux n'a pas sa place.
5. **La métrique doit épouser la décision.** Il n'existe pas de « bonne métrique » dans l'absolu : si une fraude manquée coûte cent fois plus qu'une fausse alerte, la mesure doit le refléter (chapitre 5).
6. **Interprétabilité et équité font partie de l'évaluation.** Un modèle précis qu'on ne peut ni expliquer ni contrôler, ou qui traite différemment des groupes de clients sans raison valable, n'est pas un bon modèle (chapitre 5).

Comment travailler avec ce volume

Un livre, et son cahier

Ce volume est composé de **deux ouvrages complémentaires** :

- **le livre** (celui que vous lisez) explique les idées : intuition, exemple calculé à la main, démonstration quand elle éclaire, pièges, résumé ;
- **le cahier d'exercices et d'applications** contient tout ce qui se pratique : exercices corrigés, applications guidées sur données, et le projet de fin de volume avec son auto-évaluation.

Le livre renvoie au cahier par une ligne qui termine les sections concernées :

Pour s'entraîner. Cahier, chapitre 2 : application 2.1, exercices 2.1 à 2.4.

Lisez d'abord la section dans le livre, puis, si vous voulez fixer l'idée, passez au cahier. On peut aussi lire le livre seul : rien d'indispensable à la compréhension n'a été déplacé.

Le rythme d'une section

Pour chaque notion : une **intuition**, un **exemple minuscule calculé à la main**, la **démonstration** quand elle éclaire, les **pièges**, puis un **résumé**. Les encadrés ont des pictogrammes fixes :

Encadré	Rôle
■ Intuition	l'idée en langage courant
■ Démonstration	le raisonnement rigoureux (on peut le sauter à la première lecture)
△ Piège	l'erreur classique et comment l'éviter
■ Expérience / remarque	une simulation ou un commentaire
✓ À retenir	le résumé de la section
■ Repère	un guide de lecture, ou une section optionnelle
■ Pour s'entraîner	le renvoi vers les exercices et applications du cahier

Le code dans ce livre

Ce n'est pas un livre de programmation : le code n'y apparaît que lorsqu'il aide à comprendre, typiquement un appel court qui montre comment demander un modèle à une bibliothèque. Les formules, les démonstrations et les algorithmes sont expliqués en mathématiques et en français. Les figures, les simulations et les vérifications numériques sont bien produites par du code, mais ce code est **caché** dans les sources : il est exécuté à chaque construction du livre, de sorte que **chaque nombre cité est reproductible**. Les versions complètes se trouvent dans le cahier et dans le dépôt du livre.

Des données simulées, et un jeu réel

Les données de la boutique sont **simulées**, comme aux volumes précédents. C'est un choix délibéré : quand on simule, on connaît la **vérité** (quelles variables comptent, comment elles interagissent), ce qui permet de vérifier qu'une méthode la retrouve, ou de comprendre pourquoi elle la retrouve mal. À la fin des études importantes, nous dévoilons ce qui avait été programmé. Mais la réalité est plus rugueuse : c'est pourquoi nous utilisons aussi un **jeu réel et public** (30 000 clients d'une banque, section suivante), sur lequel on ne connaît aucune vérité.

Simulé ne veut pas dire facile. Un jeu simulé est plus *propre* que la réalité. Lorsque vous appliquerez ces méthodes à de vrais jeux, comptez sur davantage de nettoyage et de vérification (volume I, cahier, projet du volume, étape 1).

Reproductibilité d'un résultat d'apprentissage automatique

Les modèles de ce volume font intervenir le hasard (division des données, arbres tirés au sort, initialisations). Nous fixons **toujours** les graines (`random_state=0`) : le livre est ainsi reproductible. Mais deux choses échappent à ce contrôle : les **versions des bibliothèques** (un algorithme d'optimisation qui change, un arrondi) et le **calcul en parallèle**. Il est donc normal que vos résultats diffèrent de quelques unités de la dernière décimale ; une conclusion qui changerait d'une version à l'autre ne serait pas une conclusion. Dans votre propre travail, notez toujours les versions, la graine et la division des données avec les résultats : c'est la condition pour qu'un résultat soit refaisable.

Les données et l'environnement du volume

Les jeux de la boutique

Au volume II, nous avons suivi la boutique pendant dix ans, avec quelques milliers de clients. Pour apprendre à **prédire**, il faut davantage de données : dans ce volume, nous observons la boutique au **31 décembre 2025** à travers douze mille clients, soixante mille commandes en ligne et une matrice d'achats clients × produits. Tous ces jeux sont **simulés** (graines fixes, script `build/donnees3.py`) : vous obtiendrez exactement les mêmes chiffres que dans le livre. Un cinquième jeu est, lui, **réel**.

Fichier	Contenu	Lignes	Utilisé surtout dans
<code>clients_ml.csv</code>	un client par ligne : comportement d'achat, satisfaction, support, trois cibles à prédire	12 000	chapitres 1 à 4, 5, 8
<code>transactions.csv</code>	une commande en ligne par ligne, dont une très petite part de fraudes	60 000	chapitres 4 et 6
<code>interactions.csv</code>	les achats « client × produit » (format long)	27 687	chapitre 7
<code>produits_ml.csv</code>	catégorie, prix et nouveauté des 150 produits	150	chapitre 7
<code>credit_default.csv</code>	jeu réel : clients d'une banque et défaut de paiement	30 000	chapitre 5 et projet du cahier

Chargeons le premier, avec les trois gestes habituels (forme, types, valeurs manquantes) :

```
import pandas as pd
clients = pd.read_csv("donnees/clients_ml.csv")
print(clients.shape)
```

```
(12000, 24)
```

Les valeurs manquantes de `clients_ml.csv` et de `interactions.csv` ne sont pas des accidents : elles sont **voulues**, car la vraie vie en est pleine, et le chapitre 4 apprend à les traiter.

Les clients : clients_ml.csv

Le fichier contient un identifiant, **19 variables d'entrée**, **trois cibles** (ce qu'on voudra prédire) et une colonne de plus dont nous parlerons à la fin de cette section.

Variable	Type	Signification
age	quantitative	âge du client (18 à 78 ans)
ville	qualitative, 20 modalités	« Ville A » à « Ville T »
canal_acquisition	qualitative	Boutique (magasin), Site (site web) ou Réseaux (réseaux sociaux)
appareil	qualitative, manquante	mobile, ordinateur ou tablette
anciennete_mois	quantitative	ancienneté du client, en mois
nb_commandes_12m	comptage	commandes sur les douze derniers mois
panier_moyen	quantitative, manquante	montant moyen d'une commande, en € (manquant si aucune commande)
montant_12m	quantitative ≥ 0	dépense totale sur douze mois, en €
recence_jours	quantitative	jours écoulés depuis la dernière commande (365 si aucune)
nb_retours_12m	comptage	articles retournés sur douze mois
satisfaction_moy	quantitative (1 à 5), manquante	note de satisfaction moyenne
nb_tickets_support_12m	comptage	demandes au service client
programme_fidelite	binaire	1 si le client est inscrit au programme de fidélité
nb_promos_recues_12m	comptage	promotions reçues
part_achats_promo	proportion	part des achats faits en promotion
taux_ouverture_email	proportion	part des courriels de la boutique ouverts
delai_livraison_moy	quantitative, manquante	délai moyen de livraison, en jours
categorie_preferee	qualitative	catégorie de produits la plus achetée (A à D)
revenu_zone	quantitative	indice de revenu de la zone de résidence

Cible	Type	Signification
churn_90j	binaire	1 si le client n'a plus commandé dans les 90 jours suivants (« départ »)
depense_6m	quantitative ≥ 0	dépense, en €, sur les six mois suivants (souvent nulle)

Cible	Type	Signification
segment_vrai	qualitative (0 à 3)	classe latente qui a servi à fabriquer les comportements : elle n'existe que parce que les données sont simulées ; nous ne l'utilisons que pour vérifier les méthodes non supervisées (chapitre 3)

Quelques constats guideront nos choix de méthodes :

- **Le départ est un événement minoritaire** : environ 14 % des clients (1 685 sur 12 000). Une prédiction qui répondrait toujours « il reste » aurait raison 86 fois sur 100 sans rien comprendre : c'est le problème des classes déséquilibrées (chapitres 4 et 5).
- **La dépense à six mois est asymétrique et pleine de zéros** (un peu plus d'un tiers des clients n'achètent rien) : sa moyenne (84,3 €) est le double de sa médiane (42,1 €).
- **Quatre variables ont des valeurs manquantes**, de 12,0 % (delai_livraison_moy) à 15,2 % (appareil). Le panier moyen manque pour les 13,6 % de clients qui n'ont passé aucune commande ; la satisfaction (12,8 % de manquants) manque plus souvent chez les clients peu satisfaits : une absence qui est elle-même une information, comme nous le verrons au chapitre 4.
- **La variable ville a vingt modalités** : un cas d'école pour l'encodage de variables qualitatives à nombreuses catégories (chapitre 4).

Une colonne ne doit pas servir de variable d'entrée. Le fichier contient une vingt-quatrième colonne, qui n'est ni un identifiant, ni une entrée, ni l'une des trois cibles. Le **dictionnaire des données** du cahier (« Mode d'emploi ») précise le statut de chaque colonne, y compris celle-ci. Comprendre pourquoi on ne peut pas l'utiliser pour prédire est l'une des leçons du chapitre 1. Elle n'a rien d'une rareté : c'est l'une des erreurs les plus fréquentes, et les plus coûteuses, des projets réels.

Les commandes en ligne : transactions.csv

Chaque ligne est une commande passée en ligne ; la colonne fraude indique si elle était frauduleuse.

Variable	Signification
montant	montant de la commande, en €
heure, jour_semaine	heure (0 à 23) et jour (0 à 6) de la commande
canal	Site ou Réseaux
appareil_connu	1 si le client a déjà utilisé cet appareil
distance_facturation_livraison_km	écart entre l'adresse de facturation et celle de livraison
nb_commandes_24h	commandes du même compte dans les 24 dernières heures
age_compte_jours	ancienneté du compte, en jours
ip_pays_différent	1 si l'adresse IP ne correspond pas au pays du client
mode_paieement	carte, virement ou portefeuille

Variable	Signification
delai_depuis_derniere_cmd_h	heures écoulées depuis la commande précédente du compte
nb_articles	nombre d'articles
fraude	cible : 1 si la commande est frauduleuse
type_fraude	0 (aucune), 1 ou 2 : deux façons différentes de frauder (utile pour comprendre les détecteurs d'anomalies, chapitre 6)

Sur 60 000 commandes, 486 sont frauduleuses (0,81 %), réparties en 287 fraudes de type 1 et 199 de type 2. Ici, la règle « aucune commande n'est frauduleuse » a raison dans 99,2 % des cas : l'**exactitude** (la proportion de bonnes réponses) est une mesure trompeuse, ce qui motive une partie du chapitre 5.

Les achats : `interactions.csv` et `produits_ml.csv`

`interactions.csv` est au **format long** : une ligne par couple (client, produit) pour lequel il y a eu au moins un achat. Les colonnes sont `id_client`, `id_produit`, `nb_achats` et `note` (une note de 1 à 5, renseignée pour environ un achat sur quatre seulement). `produits_ml.csv` décrit les 150 produits : catégorie (A à D), prix en € et nouveauté (1 pour les produits récents).

Sur 3 000 clients et 150 produits, seules **6,2 %** des cases de la matrice sont remplies (9,2 achats par client en moyenne) : c'est une matrice très **creuse**, caractéristique des systèmes de recommandation (chapitre 7).

Un jeu réel : `credit_default.csv`

Les données de la boutique sont simulées, ce qui nous permet de connaître la vérité. Pour ne pas oublier à quoi ressemble la réalité, le chapitre 5 et le projet du cahier utilisent un **jeu réel et public** : « *Default of Credit Card Clients* ».

Source et licence. Jeu publié par le dépôt de données de l'université de Californie à Irvine (UCI) et disponible sur OpenML, **licence CC0** (domaine public). Référence : I-C. Yeh et C-H. Lien, « The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients », *Expert Systems with Applications*, 36(2), 2009. Il décrit 30 000 titulaires de cartes de crédit d'une banque de Taïwan en 2005. Les montants sont en dollars taïwanais (NT\$). Le fichier a été téléchargé une fois (script `build/telecharger_credit.py`) et est fourni dans `donnees/` : rien n'est téléchargé à l'exécution.

Variable	Signification
limit_bal	montant du crédit accordé, en NT\$
sex	1 = homme, 2 = femme
education	1 = études supérieures, 2 = université, 3 = lycée, 4 = autre
marriage	1 = marié(e), 2 = célibataire, 3 = autre
age	âge, en années

Variable	Signification
pay_1 à pay_6	statut de remboursement, de septembre (pay_1) à avril (pay_6) : -1 = payé à temps, 1 à 9 = nombre de mois de retard
bill_amt1 à bill_amt6	montant de la facture, de septembre à avril
pay_amt1 à pay_amt6	montant payé, de septembre à avril
default	cible : 1 si le client est en défaut de paiement le mois suivant

Sur 30 000 clients, 6 636 (22,1 %) sont en défaut. Le statut du dernier remboursement est très informatif : 50,3 % des clients qui étaient déjà en retard en septembre font défaut, contre 13,8 % des autres. Ce jeu contient aussi des **attributs sensibles** (le sexe, l'état civil, l'âge, le niveau d'études) : c'est pourquoi nous l'utiliserons pour la section facultative sur l'équité (section 5.4), avec toutes les précautions que le sujet demande.

Les jeux embarqués de scikit-learn

Pour quelques illustrations, nous utilisons aussi des jeux **réels et classiques**, fournis avec la bibliothèque scikit-learn (donc disponibles hors ligne) :

Jeu	Contenu	Utilisé dans
load_digits	1 797 images 8 × 8 de chiffres manuscrits	réduction de dimension (chapitre 3), apprentissage semi-supervisé (chapitre 8)
load_breast_cancer	569 tumeurs, 30 mesures, bénigne ou maligne	exemples de classification
load_wine	178 vins, 13 mesures chimiques, 3 cépages	exemples de classification
load_diabetes	442 patients, 10 variables, progression de la maladie	exemples de régression

L'environnement de travail

Ce volume utilise les outils des volumes précédents, avec des bibliothèques d'apprentissage automatique en plus. Voici les commandes d'installation (non exécutées ici : elles installent des paquets sur *votre* machine). Les quatre premiers paquets sont **figés** à la version qui a produit les sorties du livre.

```
python -m venv .venv
source .venv/bin/activate          # sous Windows : .venv\Scripts\activate
pip install numpy==2.5.3 pandas==3.0.6 scipy==1.18.1 scikit-learn==1.9.1
pip install matplotlib seaborn statsmodels xgboost lightgbm catboost shap lime optuna
↪ imbalanced-learn umap-learn
```

Bibliothèque	Pour quoi faire	Chapitres
scikit-learn	l'outil central : pipelines, validation croisée, modèles, métriques	tous
xgboost, lightgbm	gradient boosting (arbres boostés)	2
catboost	boosting avec traitement natif des variables qualitatives	2 (facultatif)
optuna	réglage des hyperparamètres	1 (facultatif)
imbalanced-learn (imblearn)	rééchantillonnage, SMOTE	4
shap, lime	explication des prédictions	5
umap-learn	réduction de dimension non linéaire	3 (facultatif)
statsmodels	comparaisons avec le volume II	quelques sections

Une absence mérite d'être signalée : ce volume **n'utilise pas PyTorch ni TensorFlow**. Les réseaux de neurones apparaissent seulement dans les chapitres facultatifs (autoencodeurs du chapitre 6, aperçu du chapitre 9) : nous les écrivons avec le MLPRegressor de scikit-learn ou à la main, et le code PyTorch équivalent est montré sans être exécuté (« non exécuté »).

Les sorties de ce livre ont été produites avec les versions suivantes :

```

Python      : 3.13.3
numpy       : 2.5.3
pandas      : 3.0.6
scipy       : 1.18.1
scikit-learn : 1.9.1
xgboost     : 3.4.1
lightgbm    : 4.7.0
catboost    : 1.2.10
shap        : 0.52.0
optuna      : 5.0.0
imbalanced-learn : 0.14.2
umap-learn  : 0.5.12

```

Les résultats d'apprentissage automatique varient plus légèrement que ceux de la statistique classique. Un modèle de boosting ou une forêt aléatoire repose sur des tirages aléatoires (nous fixons toujours la graine) et sur des calculs en parallèle dont l'ordre peut changer d'une version à l'autre. Avec d'autres versions, il est normal que vos chiffres s'écartent dans la dernière décimale (un AUC de 0,899 au lieu de 0,900) ; si l'écart dépasse un point, c'est un signal à examiner. Dans ce volume, aucune conclusion ne dépend de la dernière décimale.

Prêt ? Au chapitre 1, nous commençons par la démarche qui rend toutes les suivantes possibles : formuler un problème de prédiction, séparer les données, et comprendre pourquoi un modèle ne se juge jamais sur ce qu'il a déjà vu.

Chapitre 1 : La démarche d'apprentissage automatique

« Un modèle qui n'a jamais été testé sur des données neuves n'est pas un modèle : c'est une promesse. »

Les volumes I et II vous ont appris à **comprendre** des données : décrire, estimer, tester, modéliser pour expliquer. Ce volume change de question. On ne cherche plus d'abord *pourquoi* un client part, mais **lequel** partira, afin d'agir à temps : la gérante veut une liste de cent clients à contacter cette semaine. C'est le terrain de l'**apprentissage automatique** (*machine learning*, ML) : construire des modèles dont la qualité se juge à leur capacité de **prédire des cas qu'ils n'ont jamais vus**.

Ce premier chapitre est le plus important du volume, et pourtant il ne contient presque aucun « algorithme à la mode ». Il enseigne la **démarche**, c'est-à-dire la discipline qui sépare un modèle utile d'un modèle qui n'a l'air bon que sur le papier. Les chapitres suivants fourniront les modèles ; celui-ci fournit les règles du jeu. Si vous ne deviez lire qu'un chapitre de ce volume, ce serait celui-ci.

Le chemin de ce chapitre

- **1.1 Formulation du problème et séparation des données** : qu'est-ce qu'on prédit, avec quelles informations, jugé comment ? Pourquoi on met des données de côté, et le piège numéro un : la **fuite d'information**.
- **1.2 Validation croisée** : exploiter les données au mieux sans tricher, et savoir à quel point l'estimation est incertaine.
- **1.3 Compromis biais-variance et surapprentissage** : pourquoi un modèle trop souple ou trop rigide échoue, démontré proprement.
- **1.4 Modèles de référence et rigueur expérimentale** : battre un modèle simple, comparer deux modèles avec un intervalle d'incertitude, rapporter honnêtement.
- ⊕ **1.5 Réglage des hyperparamètres** : grille, recherche aléatoire, optimisation bayésienne avec Optuna, arrêt précoce, et le piège du réglage.

Pour s'entraîner. Chaque section de ce chapitre renvoie à des applications et à des exercices du **cahier** (chapitre 1). Le livre explique ; le cahier fait pratiquer.

Les données du chapitre

Presque tout le chapitre s'appuie sur un seul tableau : `clients_ml.csv`, **12 000 clients de la boutique observés au 31 décembre 2025**. Il est **simulé** (graine fixe), comme dans les volumes précédents : la vérité est connue de l'auteur, ce qui permettra à plusieurs reprises de vérifier qu'une méthode fait bien ce qu'elle prétend. La question posée est celle de la gérante : **ce client ne commandera-t-il plus dans les 90 jours qui viennent ?** Cette cible s'appelle `churn_90j` (1 : le client est parti ; 0 : il reste).

Famille	Variables
Profil	age, ville (20 modalités), canal_acquisition (Boutique, Site, Réseaux), appareil (parfois manquant), anciennete_mois
Comportement d'achat	nb_commandes_12m, panier_moyen (manquant s'il n'y a aucune commande), montant_12m, recence_jours (jours depuis la dernière commande), nb_retours_12m, part_achats_promo, categorie_preferee
Relation	satisfaction_moy (manquante pour environ 13 % des clients), nb_tickets_support_12m, programme_fidelite, nb_promos_recues_12m, taux_ouverture_email, delai_livraison_moy
Contexte	revenu_zone (indice de la zone de résidence)
À prédire	churn_90j

Le tableau contient aussi trois colonnes qui **ne sont pas des variables d'entrée** : `depense_6m` (la dépense future, utilisée dans d'autres chapitres), `segment_vrai` (une classe cachée, utilisée pour valider les méthodes non supervisées) et `commandes_apres_cible`, qui est un **piège volontaire**. Nous le désamorçons à la section 1.1.6 : retenez seulement, pour l'instant, qu'on ne la donnera jamais au modèle.

Sur les 12 000 clients, **14,0 %** sont partis (environ 1 685) : le problème est **déséquilibré** (nous y reviendrons au chapitre 4), et quatre variables ont des valeurs manquantes, de 12 % à 15 % des clients selon la variable (le panier moyen manque pour 1 630 clients, qui n'ont passé aucune commande). Dans tout le chapitre, deux modèles servent de fil conducteur, volontairement très différents : une **régression logistique** (le modèle du volume II, section 2.2) et un **gradient boosting** (un ensemble d'arbres, étudié en détail à la section 2.4 : ici, on l'utilise comme une boîte performante). Ils jouent le rôle de deux « candidats » à comparer avec rigueur.

Comment lire ce chapitre. Le code qui produit les figures et les chiffres cités est **exécuté mais masqué** : le livre ne montre que de courts extraits quand ils aident à comprendre. Les applications complètes, avec tout leur code, sont dans le **cahier**.

1.1 Formulation du problème et séparation des données

Avant de choisir un algorithme, un projet d'apprentissage automatique commence par trois décisions qui pèsent plus lourd que n'importe quel hyperparamètre : **ce que l'on prédit** (et pour quelle ligne du tableau), **avec quelles informations** (et à quel moment), et **comment on jugera le résultat**. Cette section pose le cadre mathématique, puis montre pourquoi on met des données de côté et comment une erreur de formulation, la **fuite d'information**, peut faire passer un modèle inutilisable pour un excellent modèle.

1.1.1 Un exemple minuscule, entièrement à la main

Huit clients de la boutique, une variable (le nombre de jours depuis la dernière commande), et un modèle qui, pour chaque client, donne une **probabilité de départ** $\hat{p}$:

Client	Récence (jours)	Parti ? (y)	Probabilité prédite $\hat{p}$
A	20	0	0,05
B	35	0	0,10
C	60	0	0,20
D	90	1	0,30
E	120	0	0,40
F	150	1	0,60
G	200	1	0,80
H	300	1	0,90

Comment dire si ce modèle est bon ? Cela dépend de **ce qu'on appelle « bon »**. Voici trois mesures, calculées à la main.

Le taux d'erreur. Prédisons « parti » quand $\hat{p} > 0,5$. Le modèle prédit F, G, H comme partis, les autres comme restants. Il se trompe sur D (parti, prédit restant) seulement : 7 bonnes réponses sur 8, soit une **exactitude** de $7/8 = 0,875$.

La perte logarithmique (*log-loss*). Elle punit une probabilité confiante et fausse bien plus qu'une probabilité prudente et fausse :

$$LL = -\frac{1}{8} \sum_{i=1}^8 [y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)].$$

Les huit termes $-\ln(\text{probabilité attribuée à ce qui s'est réellement passé})$ valent : A $-\ln 0,05 = 0,051$; B $0,105$; C $0,223$; D $-\ln 0,3 = 1,204$; E $-\ln 0,6 = 0,511$; F $-\ln 0,6 = 0,511$; G $0,223$; H $0,105$. Leur somme est 2,934, d'où $LL = 2,934/8 \approx 0,367$. Remarquez que **D à lui seul pèse 41 %** du total : le modèle était assez sûr que D resterait (il lui donnait 30 % de risque) et il est parti.

L'AUC. C'est la probabilité qu'un client parti, tiré au hasard, ait reçu une probabilité **plus élevée** qu'un client resté tiré au hasard. Il y a $4 \times 4 = 16$ paires (parti, resté). Le client D (0,30) bat A, B, C mais pas E : 3 paires gagnées sur 4. F, G et H (0,60 ; 0,80 ; 0,90) battent les quatre clients restés : 12 paires sur 12. Total : $15/16 = 0,9375$.

Trois mesures, trois lectures : 87,5 % de bonnes réponses, une perte de 0,367, une capacité à classer les clients de 0,94. Aucune n'est « la » vérité : elles répondent à des questions différentes (*combien de décisions justes ? les probabilités sont-elles bien dosées ? l'ordre est-il bon ?*). Garder cette idée en tête évite bien des malentendus.

1.1.2 Le cadre : prédire, perte, risque

Formalisons. Chaque observation est une paire (x, y) : x regroupe les **variables d'entrée** (les *features*) et y la **cible**. On suppose que ces paires sont tirées indépendamment d'une même loi inconnue P . Un **prédicteur** est une fonction f qui associe à x une prédiction $f(x)$. Une **fonction de perte** $\ell(y, f(x)) \geq 0$ mesure le coût d'une prédiction (erreur au carré, perte logarithmique, erreur 0-1...).

Le **risque** de f est sa perte moyenne **sur la population entière**, c'est-à-dire sur les clients à venir :

$$R(f) = \mathbb{E}_{(x,y) \sim P}[\ell(y, f(x))].$$

On ne peut pas le calculer, puisque P est inconnue. On dispose seulement d'un échantillon $S = \{(x_i, y_i)\}_{i=1}^n$ et du **risque empirique**

$$\widehat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(x_i)).$$

Apprendre, c'est choisir $\hat{f}$ dans une famille $\mathcal{F}$ de prédicteurs possibles (les droites, les arbres de profondeur 5, etc.) en minimisant $\widehat{R}_S$: c'est la **minimisation du risque empirique**. Ce qui nous intéresse, cependant, est $R(\hat{f})$, le **risque sur des clients que le modèle n'a pas vus**. L'écart $R(\hat{f}) - \widehat{R}_S(\hat{f})$ s'appelle l'**erreur de généralisation**.

Pourquoi l'erreur d'entraînement est optimiste. Deux faits, faciles à démontrer.

(1) Si f ne dépend pas de l'échantillon S , alors $\widehat{R}_S(f)$ est un estimateur **sans biais** de $R(f)$. En effet, par linéarité de l'espérance et parce que chaque (x_i, y_i) suit la loi P :

$$\mathbb{E}[\widehat{R}_S(f)] = \frac{1}{n} \sum_i \mathbb{E}[\ell(y_i, f(x_i))] = R(f).$$

C'est la situation d'un **jeu de test** : un prédicteur déjà figé, évalué sur des données qui ne l'ont pas influencé.

(2) Si $\hat{f}$ est choisi en **minimisant** $\widehat{R}_S$, l'estimateur devient optimiste. Soit f^* le meilleur prédicteur de la famille au sens du vrai risque ($R(f^*) \leq R(f)$ pour tout $f \in \mathcal{F}$). Par construction $\widehat{R}_S(\hat{f}) \leq \widehat{R}_S(f^*)$; en prenant l'espérance et en utilisant le fait (1) pour le prédicteur **fixe** f^* :

$$\mathbb{E}[\widehat{R}_S(\hat{f})] \leq \mathbb{E}[\widehat{R}_S(f^*)] = R(f^*) \leq \mathbb{E}[R(\hat{f})].$$

Donc **en moyenne, l'erreur d'entraînement est inférieure à l'erreur réelle** de $\hat{f}$. Plus la famille $\mathcal{F}$ est riche, plus l'écart peut être grand. ■

Voilà la raison d'être de tout ce chapitre : **on ne peut pas juger un modèle sur les données qui ont servi à le construire**. Il faut des données *neuves*, ou des méthodes qui simulent la nouveauté (section 1.2).

1.1.3 La perte n'est pas la métrique

Deux notions voisines sont à distinguer, parce qu'on les confond tout le temps :

- La **perte** est ce que l'algorithme **minimise** pendant l'apprentissage. Elle doit être commode : dérivable, convexe si possible (la perte logarithmique pour une régression logistique, l'erreur quadratique pour une régression).
- La **métrique** est ce que l'**utilisateur** veut maximiser dans la vie réelle. Elle peut être discontinue, difficile à optimiser, et dépend du métier : « parmi les cent clients que la gérante appellera cette semaine, combien sont vraiment sur le point de partir ? »

Question du métier	Perte typique pour l'apprentissage	Métrique de décision
Quels clients appeler ?	perte logarithmique	précision sur les 100 meilleurs scores
Combien de ventes le mois prochain ?	erreur quadratique	erreur moyenne en € (MAE)
Cette transaction est-elle frauduleuse ?	perte logarithmique pondérée	coût total des fraudes manquées et des fausses alertes

Le chapitre 5 détaille les métriques. Retenons ici un **piège élémentaire** : l'exactitude (*accuracy*) trompe quand les classes sont déséquilibrées. Dans notre tableau, 14,0 % des clients partent ; un « modèle » qui répond toujours « il reste » obtient donc **86,0 % d'exactitude** sans rien avoir appris. Un score de 88 % n'a de sens que comparé à ce repère (section 1.4).

1.1.4 L'unité d'analyse et le moment de la prédiction

Un projet bien posé répond d'abord à quatre questions simples, que l'on s'oblige à écrire :

1. **Que représente une ligne ?** Ici : *un client, vu à une date donnée*. Ce n'est pas « le client » en général, mais son état au 31 décembre 2025.
2. **À quel moment prédit-on ?** Le modèle sera utilisé le 31 décembre pour décider qui appeler en janvier. La date de prédiction t_0 est donc le 31 décembre.
3. **Qu'est-ce qui est connu à t_0 ?** Tout ce qui s'est passé **jusqu'à t_0** : l'historique de commandes, les tickets, la satisfaction déclarée. Rien de ce qui se passera après.
4. **Quand la cible est-elle connue ?** Le départ à 90 jours ne se saura que **le 31 mars** : il y a un **délai d'étiquetage**. Pour construire le jeu d'entraînement, on a dû se placer dans le passé, à une date t_0 assez ancienne pour que les 90 jours suivants soient écoulés.

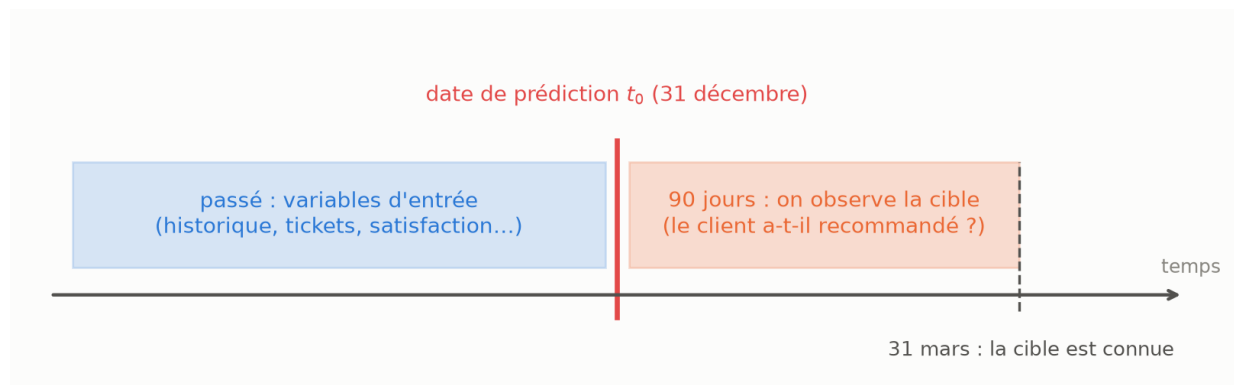


Figure 1: Chronologie d'un problème de prédiction : les variables d'entrée appartiennent au passé de la date de prédiction, la cible est observée pendant les 90 jours suivants et n'est connue que le 31 mars.

La question qui évite les catastrophes. Pour chaque variable d'entrée, demandez-vous : « à la date où j'utiliserai le modèle, cette valeur existe-t-elle déjà ? » Si la réponse est « non » ou « pas sûr », la variable est interdite. Cette seule question aurait suffi à éviter la plupart des projets de ML qui échouent au moment du déploiement.

1.1.5 Séparer les données : entraînement, validation, test

Comme l'erreur d'entraînement est optimiste (section 1.1.2), on garde de côté des données que le modèle ne verra pas pendant sa construction. On distingue **trois rôles**, qui correspondent à trois questions différentes :

Jeu	Rôle	Question	Qui l'utilise
Entraînement (<i>train</i>)	ajuster les paramètres du modèle	« quels coefficients, quels arbres ? »	l'algorithme
Validation	comparer des modèles, régler les hyperparamètres	« lequel choisir ? »	vous , de nombreuses fois
Test	estimer la performance finale du modèle choisi	« que vaudra-t-il en production ? »	vous, une seule fois

L'analogie qui aide : l'entraînement, ce sont les **exercices** que l'on fait en révisant ; la validation, ce sont les **examens blancs**, que l'on peut passer plusieurs fois pour ajuster sa méthode ; le test est l'**examen final**. Si on regarde le sujet de l'examen final pour adapter sa révision, la note ne mesure plus rien. Dès que le jeu de test a influencé une décision (choisir un modèle, régler un seuil, supprimer une variable), il devient un jeu de validation, et il n'y a plus de jeu de test.

Comment découper ? Plusieurs façons, qui ne sont pas interchangeables :

- **Aléatoire**, quand les lignes sont indépendantes : on tire au hasard, par exemple 75 % pour l'entraînement et 25 % pour le test.
- **Stratifié**, quand la cible est rare : on garde la même proportion de clients partis (14 %) dans chaque jeu. Sans cela, un petit jeu de test pourrait n'en contenir que 10 % ou 18 % par hasard.
- **Temporel**, quand le futur doit être prédit à partir du passé : on entraîne sur les périodes anciennes, on teste sur les récentes. Un tirage aléatoire mélangerait passé et futur et serait trop favorable (section 1.2.3).
- **Groupé**, quand plusieurs lignes concernent la même entité (plusieurs commandes d'un même client, plusieurs photos d'un même objet) : toutes les lignes d'une entité vont du même côté, sinon le modèle « reconnaît » l'entité au lieu de généraliser.

Dans tout le chapitre, nous mettons de côté **25 % des clients** (3 000) pour le test final, en conservant la proportion de départs. Le jeu d'entraînement (9 000 clients) sert à toutes les expériences ; le jeu de test ne sera ouvert qu'**une fois**, à la section 1.4.

```
from sklearn.model_selection import train_test_split

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
print(X_tr.shape, X_te.shape, y_tr.mean().round(4), y_te.mean().round(4))
```

```
(9000, 19) (3000, 19) 0.1404 0.1403
```

Le résultat est : 9 000 clients d'entraînement (14,04 % de départs) et 3 000 clients de test (14,03 %). L'argument `random_state=0` fixe la graine : le même découpage sera obtenu à chaque exécution, condition indispensable pour qu'une expérience soit **reproductible**.

Quelle taille pour le jeu de test ? Plus il est petit, plus la note finale est incertaine. Pour une exactitude voisine de 0,9 mesurée sur n clients, l'erreur-type est environ $\sqrt{0,9 \times 0,1/n}$: 0,5 point pour $n = 3\,000$ mais 1,7 point pour $n = 300$ (un intervalle à 95 % est environ deux fois plus large de chaque côté). Sur un problème rare (14 % de positifs), c'est le nombre de **positifs** dans le test qui compte : ici environ 420. Nous mesurerons précisément cette

incertitude à la section 1.4.

Pour s'entraîner. Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.3.

1.1.6 La fuite d'information

La **fuite d'information** (*data leakage*) désigne toute situation où le modèle, pendant sa construction, a accès à une information qu'il n'aura pas au moment de l'utilisation réelle. C'est la cause la plus fréquente de projets dont les résultats brillants s'effondrent en production. Elle se présente sous plusieurs formes :

- **La fuite par la cible** : une variable d'entrée contient, directement ou indirectement, la réponse (un « motif de résiliation » enregistré *après* le départ).
- **La contamination du jeu de test** : un traitement qui *apprend* des données (calcul d'une moyenne, choix de variables, normalisation) est appliqué *avant* le découpage, sur l'ensemble des données.
- **La fuite temporelle** : on prédit le passé avec le futur (section 1.2.3).
- **Les doublons et les entités partagées** : le même client apparaît des deux côtés du découpage.

Un cas concret : la colonne commandes_apres_cible. Notre tableau contient le nombre de commandes passées dans les **trois mois suivant** la date de prédiction. Elle semble anodine : c'est un nombre de commandes comme un autre. Ajoutons-la aux variables d'entrée et mesurons la performance par validation croisée (la méthode de la section 1.2).

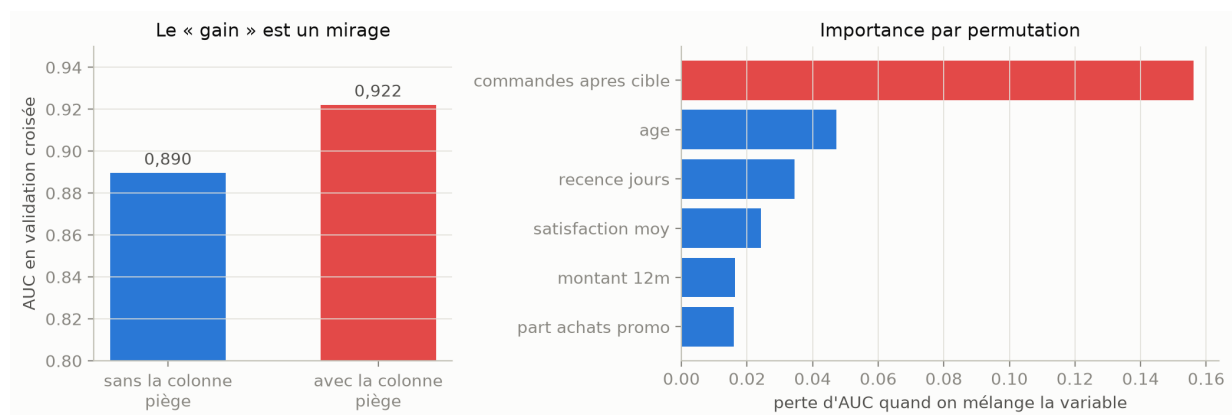


Figure 2: À gauche : l'AUC en validation croisée passe de 0,89 à 0,92 quand on ajoute la colonne des commandes futures. À droite : cette colonne devient la variable la plus « importante » du modèle.

L'AUC passe de 0,890 à 0,922 : un gain de trois points, qui ferait la joie de n'importe quelle équipe. Et la colonne piège devient la variable la plus importante du modèle (figure de droite). **C'est un mirage.** Le nombre de commandes des trois mois suivants est précisément ce que l'on cherche à prédire : un client qui ne commande plus a, par définition, zéro commande après. Au 31 décembre, cette colonne **n'existe pas encore**. Le modèle « avec » ne peut donc pas être utilisé : appliqué en production, il recevrait des valeurs manquantes ou inventées, et sa performance réelle s'effondrerait à celle du modèle « sans », voire pire (il s'est appuyé sur une information qui n'arrive jamais).

Un détecteur de fuite imparfait. On entend souvent qu'une variable « trop belle » trahit la fuite. Ici, elle est discrète : prise seule, la colonne piège a une AUC de 0,76, au **deuxième rang** sur 16 variables, juste derrière le montant des commandes (0,77) ; rien d'aberrant, et un tri des variables par pouvoir prédictif individuel ne l'aurait pas signalée. Ce qui la trahit, c'est (1) **l'audit du calendrier** (« quand cette valeur est-elle connue ? ») et (2) le **bond** de performance quand on l'ajoute, combiné à son importance démesurée. La fuite par la cible est une faute de **raisonnement**, rarement visible dans les chiffres seuls.

Un second cas : la contamination par le choix des variables. Voici une expérience classique. On génère 100 clients fictifs avec 2 000 variables de **pur bruit** et une cible tirée à pile ou face : aucune information à trouver. On retient les 10 variables les mieux corrélées à la cible, puis on estime la performance par validation croisée.

En sélectionnant les variables **sur toutes les données avant** de valider, on obtient une AUC de **0,88** sur un jeu qui ne contient que du bruit. En refaisant la sélection **à l'intérieur de chaque pli** (dans un *pipeline*), on tombe à 0,60, ce qui est, sur 100 observations, la dispersion normale autour du hasard (0,5). La différence entre 0,88 et 0,60 est la mesure de la contamination : les données de validation avaient « voté » pour le choix des variables.

À retenir.

- Un modèle se juge sur des données **qu'il n'a pas vues**, parce que l'erreur d'entraînement est en moyenne **optimiste** (démonstration 1.1.2).
- Une ligne = une unité d'analyse à une **date de prédiction** ; chaque variable doit être **connue à cette date**. L'audit du calendrier est la meilleure défense contre la fuite.
- Trois rôles : **entraînement** (ajuster), **validation** (choisir, souvent), **test** (évaluer, **une seule fois**).
- Tout traitement qui *apprend* des données (imputation, normalisation, sélection, encodage cible) fait partie du modèle : il doit être **entraîné sur le jeu d'entraînement seulement**, dans un pipeline.
- La perte guide l'algorithme, la **métrique** guide la décision ; l'exactitude seule trompe sur les classes déséquilibrées.

1.2 Validation croisée

Mettre de côté un jeu de test (section 1.1) protège de l'optimisme, mais coûte cher : ces clients ne participent pas à l'apprentissage, et la note obtenue dépend du **hasard du découpage**. Quand il s'agit de **choisir** entre modèles ou de les régler, on a besoin de nombreuses évaluations fiables. La **validation croisée** est la réponse standard : elle fait servir chaque observation tantôt à l'entraînement, tantôt à la validation, sans jamais mélanger les deux rôles pour une même évaluation.

1.2.1 Un seul découpage ne suffit pas

Prenons un petit échantillon de 2 000 clients, et mesurons l'AUC d'une régression logistique sur un jeu de validation de 20 % (400 clients, dont 56 partis). Re commençons **200 fois**, avec 200 découpages aléatoires différents. Le modèle, les données et la métrique sont les mêmes ; seul le hasard du découpage change.

Selon le découpage, l'AUC varie de **0,79 à 0,91**, avec un intervalle central à 95 % de 0,82 à 0,91. Deux analystes qui utilisent des graines différentes pourraient publier des conclusions opposées

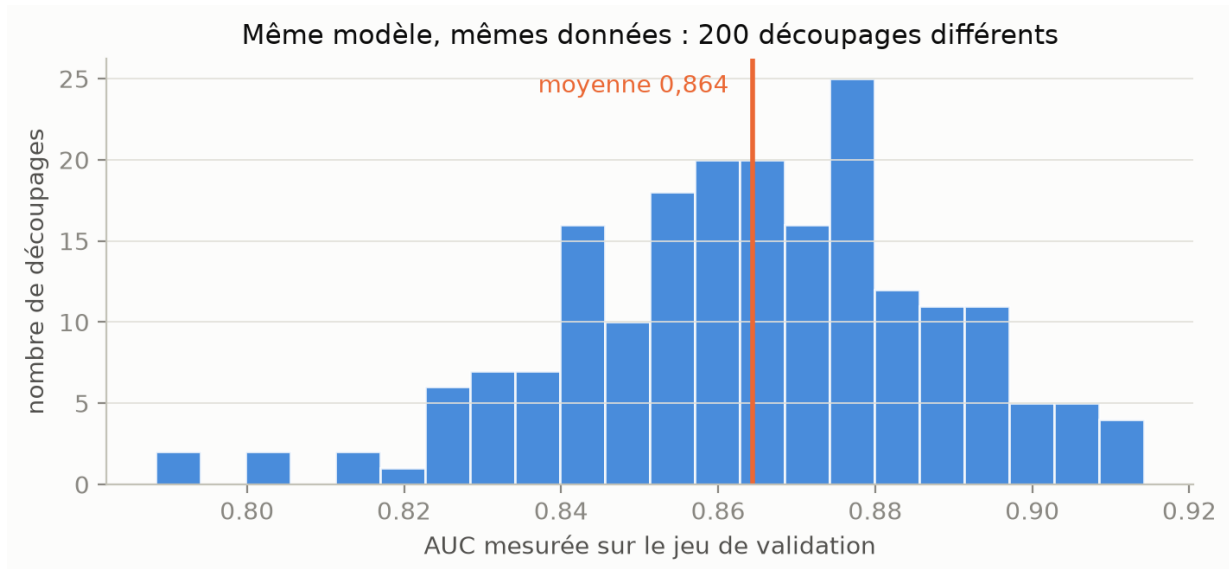


Figure 3: Distribution de l'AUC d'une même régression logistique sur 200 découpages aléatoires différents d'un échantillon de 2 000 clients.

sur le même modèle, et choisir, entre deux modèles proches, celui que le hasard a favorisé. Un découpage unique est un instrument de mesure **bruité**. Avec 56 clients partis dans le jeu de validation, rien d'étonnant : la mesure s'appuie sur un petit nombre d'événements.

1.2.2 La validation croisée à k plis

L'idée : au lieu d'un découpage, on en fait k , **de façon systématique**, pour que chaque client serve exactement une fois à la validation.

1. On mélange les données et on les coupe en k parties de taille égale, les **plis** (*folds*).
2. Pour chaque pli $j = 1, \dots, k$: on **entraîne** le modèle sur les $k - 1$ autres plis, et on mesure sa perte sur le pli j , qui n'a pas servi à l'entraînement.
3. On **moyenne** les k mesures :

$$CV_k = \frac{1}{k} \sum_{j=1}^k \frac{1}{|F_j|} \sum_{i \in F_j} \ell(y_i, \hat{f}^{(-j)}(x_i)),$$

où $\hat{f}^{(-j)}$ est le modèle entraîné **sans** le pli F_j .

Un exemple à la main. Le cas extrême $k = n$ (chaque observation forme un pli) s'appelle la validation croisée « **un seul laissé de côté** » (*leave-one-out*, LOO). Prenons trois montants de commandes $y = (2; 4; 9)$ et un « modèle » très simple : prédire la **moyenne des données d'entraînement**. L'erreur sur les données d'entraînement vaut, avec $\bar{y} = 5$: $\frac{(2-5)^2 + (4-5)^2 + (9-5)^2}{3} = \frac{26}{3} \approx 8,67$. Voyons maintenant la validation LOO :

- on retire 2, on prédit avec la moyenne de (4 ; 9), soit 6,5 : erreur $-4,5$;
- on retire 4, on prédit avec la moyenne de (2 ; 9), soit 5,5 : erreur $-1,5$;
- on retire 9, on prédit avec la moyenne de (2 ; 4), soit 3 : erreur 6.

L'erreur quadratique LOO vaut $\frac{20,25+2,25+36}{3} = 19,5$, soit **plus du double** de l'erreur d'entraînement (8,67). On voit à l'œuvre l'optimisme de la section 1.1.2. Et il existe ici une jolie formule : si l'on retire y_i , la moyenne des autres vaut $\frac{n\bar{y}-y_i}{n-1}$, donc l'erreur est $y_i - \frac{n\bar{y}-y_i}{n-1} = \frac{n}{n-1}(y_i - \bar{y})$; l'erreur LOO est ainsi l'erreur d'entraînement multipliée par $\left(\frac{n}{n-1}\right)^2 = 2,25$, et $2,25 \times 8,67 = 19,5$.

	pli 1	pli 2	pli 3	pli 4	pli 5
essai 1	validation				
essai 2		validation			
essai 3			validation		
essai 4				validation	
essai 5					validation

Figure 4: Validation croisée à 5 plis : à chaque essai, un pli (orange) sert à la validation et les quatre autres (bleu) à l'entraînement.

(Pour la régression linéaire, il existe de même une formule exacte qui évite de réentraîner n fois : le **PRESS**, vu dans le volume II, section 1.4.)

En pratique, sur les données de la boutique, une validation croisée à 5 plis de la régression logistique tient en trois lignes :

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modele_logit(), X_tr, y_tr, cv=5, scoring="roc_auc")
print(scores.round(3), scores.mean().round(3))
```

```
[0.873 0.858 0.87 0.836 0.862] 0.86
```

Les cinq plis donnent des AUC de 0,84 à 0,87 ; l'estimation retenue est leur moyenne, **0,860**. Ici, $cv=5$ découpe en cinq plis *stratifiés* (même proportion de départs dans chacun), comportement par défaut pour un classifieur.

Ce que la validation croisée estime vraiment. Chacun des k modèles est entraîné sur $\frac{k-1}{k}$ des données ; la moyenne estime donc la performance d'un modèle entraîné sur **un peu moins** de données que le jeu complet. C'est une estimation de la *procédure* d'apprentissage (« entraîner ce type de modèle sur ce volume de données »), pas du modèle final particulier que l'on livrera en réentraînant sur tout.

Combien de plis ? Il y a un compromis. Avec peu de plis ($k = 2$), chaque modèle voit la moitié des données : il est moins bon que le modèle final, et la validation est **pessimiste** (biais). Avec beaucoup de plis, le biais disparaît, mais les modèles entraînés sont presque identiques, donc les erreurs sont très corrélées, et le calcul coûte k entraînements. Vérifions sur nos données : on tire 10 sous-échantillons de 1 000 clients ; pour chacun, on compare l'estimation par validation croisée à la **vraie performance** du modèle entraîné sur ces 1 000 clients, mesurée sur les 8 000 clients restants.

Avec $k = 2$, l'estimation est **pessimiste** de 1 point d'AUC et plus dispersée (écart-type 0,021). À partir de $k = 5$, le biais est de l'ordre du millième, plus petit que la dispersion, et augmenter encore k n'apporte rien de mesurable. D'où la règle pratique : $k = 5$ **ou** $k = 10$. Le « un seul laissé de côté » est surtout réservé aux très petits jeux de données ou aux modèles pour lesquels une formule évite les réentraînements.

1.2.3 Les variantes : stratifiée, répétée, groupée, temporelle

Le découpage en plis n'est pas anodin : il doit **reproduire la situation de l'utilisation réelle**. Les variantes suivantes répondent à quatre situations différentes.

Variante	Quand l'utiliser	Idée
Stratifiée	cible rare ou classes déséquilibrées	même proportion de chaque classe dans chaque pli
Répétée	jeu de données petit, mesure instable	on refait la validation m fois avec des découpages différents
Groupée	plusieurs lignes par entité (client, magasin, patient)	toutes les lignes d'une entité sont dans le même pli
Temporelle	prédire l'avenir à partir du passé	on entraîne sur le passé, on valide sur le futur proche

Répétée. Refaire 5 plis 20 fois (avec 20 découpages différents) réduit le bruit dû au découpage lui-même. Sur nos 9 000 clients, il est déjà minuscule : l'écart-type des moyennes entre répétitions vaut seulement 0,0007 d'AUC. La répétition est utile pour de **petits** jeux de données.

Groupée. Supposons qu'on veuille savoir comment le modèle se comportera dans une **ville qu'il n'a jamais vue**. On place alors chaque ville tout entière dans un seul pli.

```
from sklearn.model_selection import GroupKFold

villes = df.loc[X_tr.index, "ville"]
cv_ville = cross_val_score(modele_logit(), X_tr, y_tr, groups=villes, cv=GroupKFold(5),
    ← scoring="roc_auc")
print(cv_ville.mean().round(4))
```

```
0.8577
```

L'AUC « par ville » (0,858) est à peine inférieure à celle de la validation stratifiée (0,860) : dans nos données, l'effet de la ville est modeste. Dans un jeu où le même **client** apparaît sur plusieurs lignes (plusieurs commandes), la différence serait énorme, car le modèle reconnaîtrait le client plutôt que de généraliser.

Temporelle. C'est la variante où l'erreur est la plus courante, et la plus coûteuse. Prenons une série mensuelle simulée sur 11 ans (tendance, saisonnalité, bruit corrélé) et un modèle qui prédit chaque mois à partir des valeurs de 1, 2, 3 et 12 mois plus tôt. Trois estimations de l'erreur (RMSE) :

La validation croisée **aléatoire** annonce une erreur de 3,5 : elle est presque deux fois trop optimiste. La validation **temporelle** annonce 7,3, proche de l'erreur réellement observée sur les 24 derniers mois (6,75). La raison est simple : en mélangeant les mois, le modèle s'entraîne sur des mois **voisins** du mois à prédire (le mois d'avant et le mois d'après se ressemblent), ce qu'il ne pourra jamais faire en production, où l'avenir est inconnu. C'est une **fuite temporelle**.

Règle de décision. Avant de choisir une validation croisée, posez la question : « *en production, qu'est-ce qui sera connu de ce qui m'entoure ?* » Si le futur n'est pas connu, la validation doit respecter l'ordre du temps ; si les lignes d'une même entité arrivent ensemble, elles doivent rester ensemble.

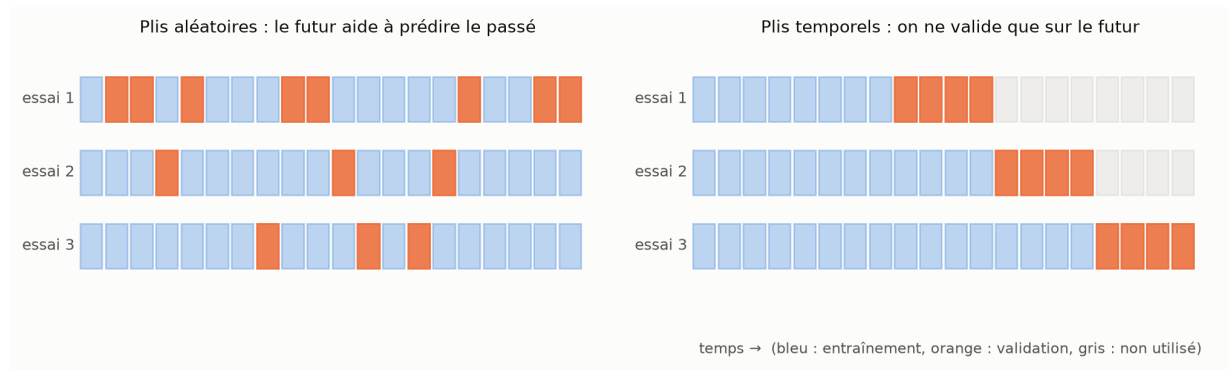


Figure 5: Deux façons de découper une série temporelle : plis aléatoires (à gauche, le modèle voit des mois situés après ceux qu'il doit prédire) et plis temporels (à droite, l'entraînement précède toujours la validation).

1.2.4 Quelle confiance accorder à l'estimation ?

La validation croisée donne un **nombre**, pas une certitude. Deux sources de bruit s'y mélangent : le découpage (que la répétition réduit) et, surtout, **l'échantillon lui-même**, c'est-à-dire le fait qu'on ait observé ces 9 000 clients et pas d'autres. La seconde source est la plus importante, et on ne peut pas la réduire par des calculs.

On est tenté de calculer une erreur-type avec les k scores des plis : $\hat{\sigma} / \sqrt{k}$. **Ce n'est pas valable**. Les k modèles sont entraînés sur des données qui se recouvrent presque entièrement : les k mesures ne sont pas indépendantes, et la formule suppose qu'elles le sont. Un résultat théorique (Bengio et Grandvalet, 2004) montre même qu'il **n'existe pas d'estimateur sans biais universel** de la variance de la validation croisée. Vérifions numériquement. On découpe nos 9 000 clients en 12 blocs disjoints de 750 ; sur chacun, on fait une validation à 5 plis. Les 12 estimations sont indépendantes : leur dispersion est la **vraie** incertitude d'une validation croisée sur 750 clients.

L'incertitude réelle (écart-type 0,0145) diffère de l'erreur-type « naïve » tirée des plis (0,0215) : ici, cette dernière est trop **pessimiste** ; dans d'autres situations, elle est trop optimiste. Le message est clair : **ne présentez jamais $\hat{\sigma} / \sqrt{k}$ comme une barre d'erreur**. Pour comparer deux modèles ou rapporter un intervalle, la section 1.4 présente des outils adaptés (test t corrigé, bootstrap du jeu de test). Retenez surtout un ordre de grandeur : avec quelques centaines à quelques milliers de clients, **des écarts de moins d'un point d'AUC ne sont pas interprétables**.

1.2.5 La validation croisée imbriquée : un aperçu

Un dernier piège guette dès qu'on se sert de la validation croisée pour **choisir** : un modèle parmi dix, un hyperparamètre parmi cent. Le meilleur score obtenu est alors lui-même le résultat d'une **sélection**, donc optimiste (c'est exactement l'argument de la démonstration 1.1.2, appliqué à la validation). La parade est la **validation croisée imbriquée** (*nested cross-validation*) :

1. une **boucle externe** découpe les données en plis ; chaque pli externe est un jeu de test provisoire ;
2. pour chaque pli externe, une **boucle interne** (une autre validation croisée sur les données d'entraînement externes seulement) choisit le meilleur modèle ou réglage ;
3. le modèle ainsi choisi est évalué **sur le pli externe**, que la sélection n'a jamais vu ;
4. la moyenne des évaluations externes estime la performance de **toute la procédure** « choisir puis entraîner ».

Le coût est multiplicatif ($k_{\text{ext}} \times k_{\text{int}} \times$ le nombre de candidats), mais c'est le prix d'une estimation

honnête. Nous la mettrons en œuvre à la section 1.5, où nous verrons sur un jeu de pur bruit à quel point le réglage peut s'auto-persuader.

À retenir.

- Un seul découpage est un instrument **bruité** : sur 400 clients de validation, l'AUC varie de 0,79 à 0,91 selon la graine.
- La **validation croisée à k plis** utilise chaque observation pour valider exactement une fois ; $k = 5$ ou 10 est un bon compromis (biais de l'ordre du millième pour $k \geq 5$, pessimiste pour $k = 2$).
- Le découpage doit **imiter l'usage réel** : stratifié (classes rares), groupé (entités répétées), **temporel** (prédire l'avenir). Une validation aléatoire sur une série temporelle annonce 3,5 d'erreur au lieu de 7.
- L'**écart-type entre plis n'est pas une barre d'erreur** : les plis ne sont pas indépendants.
- Quand la validation croisée sert à **choisir**, son meilleur score est optimiste : on utilise la validation **imbriquée**.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.2 à 1.4, exercices 1.4 à 1.6.

1.3 Compromis biais-variance et surapprentissage

Pourquoi un modèle très souple, capable d'épouser les moindres détails des données d'entraînement, prédit-il souvent **moins bien** qu'un modèle plus simple ? Et pourquoi un modèle trop simple échoue-t-il aussi ? Cette section répond par un résultat mathématique d'une grande élégance, la **décomposition biais-variance**, puis le montre à l'œuvre sur un exemple chiffré, sur des polynômes, et sur les données de la boutique.

1.3.1 Deux façons de se tromper

Imaginez un tireur à l'arc. Il peut mal viser **systématiquement** du même côté de la cible : ses flèches sont groupées, mais loin du centre. C'est un défaut de **biais**. Il peut aussi avoir la main qui tremble : ses flèches sont réparties autour du centre, sans direction privilégiée, mais très dispersées. C'est un défaut de **variance**. Un bon tireur a peu des deux ; entre deux tireurs mauvais, on ne peut pas dire lequel est « moins mauvais » sans savoir ce qu'on veut.

En apprentissage, la « cible » est la vraie relation $f(x)$ entre les variables d'entrée et ce qu'on prédit. Le « tireur » est l'**algorithme**, qui reçoit un jeu de données d'entraînement D tiré au sort et produit un prédicteur $\hat{f}_D$. Si on lui donnait un autre jeu de données (d'autres clients, mêmes caractéristiques générales), il produirait un autre prédicteur. Deux questions se posent alors :

- **En moyenne** sur tous les jeux d'entraînement possibles, le prédicteur vise-t-il juste ? Sinon, il a un **biais**.
- D'un jeu à l'autre, le prédicteur change-t-il beaucoup ? Si oui, il a une grande **variance**.

1.3.2 Un exemple à la main : estimer une moyenne

Le plus petit exemple possible n'a pas de variables d'entrée. On veut estimer une grandeur inconnue $\mu = 2$ (un panier moyen, en dizaines d'euros) à partir de $n = 4$ observations de variance $\sigma^2 = 4$. L'estimateur habituel est la moyenne $\bar{y}$: il est **sans biais** (en moyenne il vaut μ) et sa variance est $\sigma^2/n = 4/4 = 1$. Son erreur quadratique moyenne vaut donc $\text{EQM} = \text{biais}^2 + \text{variance} = 0 + 1 = 1$.

Essayons maintenant un estimateur **volontairement biaisé** : $0,8 \bar{y}$, qui « tire » la moyenne vers zéro.

- Son espérance vaut $0,8\mu = 1,6$: le **biais** est $1,6 - 2 = -0,4$, donc $\text{biais}^2 = 0,16$.
- Sa variance vaut $0,8^2 \times 1 = 0,64$.
- Son erreur quadratique moyenne vaut $0,16 + 0,64 = 0,80$.

L'estimateur biaisé est meilleur que l'estimateur sans biais (0,80 contre 1). On a accepté un petit biais en échange d'une forte baisse de variance. En général, l'estimateur $c \bar{y}$ a pour erreur $(c - 1)^2 \mu^2 + c^2 \sigma^2/n$, minimale pour

$$c^* = \frac{\mu^2}{\mu^2 + \sigma^2/n} = \frac{4}{4 + 1} = 0,8.$$

Plus les données sont bruitées (σ^2/n grand), plus il faut rétrécir. C'est exactement le principe de la **régularisation** (volume II, section 1.5) : on « tire » les coefficients vers zéro pour réduire la variance.

Une simulation de 400 000 tirages confirme : l'erreur quadratique moyenne vaut 1,00 pour la moyenne et 0,80 pour l'estimateur rétréci.

1.3.3 La décomposition biais-variance

Passons au cas général. On observe $y = f(x) + \varepsilon$, où f est la vraie fonction, et ε un bruit d'espérance nulle et de variance σ^2 , indépendant du jeu d'entraînement D . Fixons un point x et considérons la prédiction $\hat{f}(x) = \hat{f}_D(x)$, qui est aléatoire parce que D l'est. L'erreur quadratique d'une nouvelle observation y en ce point, **moyennée sur le bruit et sur le choix de D** , se décompose ainsi.

Théorème (décomposition biais-variance).

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\sigma^2}_{\text{bruit irréductible}} + \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{biais}^2} + \underbrace{\mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]}_{\text{variance}}.$$

Démonstration. Écrivons $y - \hat{f} = \varepsilon + (f - \hat{f})$ et développons le carré :

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[\varepsilon^2] + \mathbb{E}[(f - \hat{f})^2] + 2 \mathbb{E}[\varepsilon (f - \hat{f})].$$

Le premier terme vaut σ^2 . Le dernier est nul : ε est indépendant de $\hat{f}$ (qui ne dépend que de D) et d'espérance nulle, donc $\mathbb{E}[\varepsilon (f - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}[f - \hat{f}] = 0$. Pour le terme central, ajoutons et retranchons $\mathbb{E}[\hat{f}]$:

$$f - \hat{f} = (f - \mathbb{E}[\hat{f}]) + (\mathbb{E}[\hat{f}] - \hat{f}).$$

En développant, le double produit contient le facteur $\mathbb{E}[\mathbb{E}[\hat{f}] - \hat{f}] = 0$ et disparaît, d'où

$$\mathbb{E}[(f - \hat{f})^2] = (f - \mathbb{E}[\hat{f}])^2 + \mathbb{E}[(\hat{f} - \mathbb{E}[\hat{f}])^2] = \text{biais}^2 + \text{variance}. \quad \blacksquare$$

Trois lectures de cette formule :

1. Le terme σ^2 est un **plancher** : aucun modèle ne peut faire mieux que le bruit des données (on l'appelle l'erreur de Bayes dans le cas de la classification). Si une équipe annonce une erreur *inférieure* au bruit connu, elle s'est trompée ou a triché (fuite d'information, section 1.1.6).
2. Le biais mesure l'**écart systématique** entre ce que l'algorithme sait représenter et la réalité ; il baisse quand le modèle devient **plus souple**.
3. La variance mesure la **sensibilité** à l'échantillon ; elle augmente quand le modèle devient **plus souple** (plus de paramètres que de données pour les contraindre).

La conséquence est le **compromis biais-variance** : on ne peut pas minimiser les deux en même temps, et le meilleur modèle est un équilibre.

Deux précisions. La décomposition ci-dessus est exacte pour l'**erreur quadratique**. Pour la classification (perte 0-1) il existe des décompositions analogues, mais plus délicates ; l'intuition reste valable. Et dans les modèles modernes très surdimensionnés (grands réseaux de neurones), on observe parfois que l'erreur de test **rebaisse** quand la complexité continue de croître (phénomène de « double descente ») : le compromis classique est un cadre précieux, pas une loi universelle.

1.3.4 Voir le compromis : un polynôme de degré croissant

Rendons cela visible. La vraie fonction est $f(x) = \sin(1,5\pi x)$ sur $[0; 1]$ et le bruit a un écart-type $\sigma = 0,3$ (donc $\sigma^2 = 0,09$). Un « jeu d'entraînement » contient 30 points tirés au hasard. Pour chaque degré de polynôme de 1 à 7, on **simule 500 jeux d'entraînement**, on ajuste un polynôme à chacun, et on mesure, en 200 points de $[0; 1]$, le biais carré moyen et la variance moyenne de la prédiction.

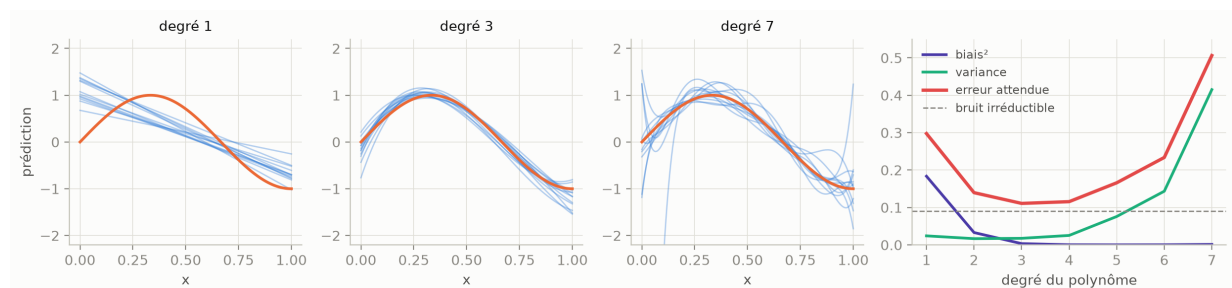


Figure 6: Biais et variance d'un polynôme de degré croissant. À gauche : 12 ajustements (bleu) de degré 1, 3 et 7 sur des jeux d'entraînement différents, et la vraie fonction (orange). À droite : biais carré, variance et erreur attendue en fonction du degré.

Les trois panneaux de gauche montrent des **paquets de flèches** : le degré 1 donne toujours à peu près la même droite, mais elle est loin de la courbe (biais fort, variance faible) ; le degré 7 donne des courbes très différentes d'un jeu à l'autre (variance forte) ; le degré 3 épouse bien la courbe, et d'un jeu à l'autre varie peu. À droite, le compromis : le biais s'effondre dès le degré 3, la variance croît, et l'erreur attendue est minimale au **degré 3** (0,111, à comparer au plancher de 0,09). Plus loin, la situation se dégrade vite : la variance dépasse 5 dès le degré 8 et 1 000 dès le degré 9, parce que, avec 30 points, un polynôme de degré 9 peut osciller sauvagement dans les zones sans donnée.

1.3.5 Sous-apprentissage, surapprentissage

Le vocabulaire courant désigne les deux extrémités :

- Le **sous-apprentissage** (*underfitting*) : le modèle est **trop rigide** pour représenter la structure des données ; le biais domine. Les erreurs d'entraînement et de validation sont élevées.
- Le **surapprentissage** (*overfitting*) : le modèle est **trop souple** ; il a appris le bruit de l'échantillon d'entraînement ; la variance domine. L'erreur d'entraînement est faible, l'erreur de validation beaucoup plus élevée.

Reproduisons cela sur les données de la boutique, avec un arbre de décision dont on fait croître la profondeur (un arbre plus profond pose plus de questions successives : c'est un modèle plus souple ; le chapitre 2 le détaille). Pour chaque profondeur de 1 à 20, on mesure l'AUC sur les données d'entraînement et par validation croisée.

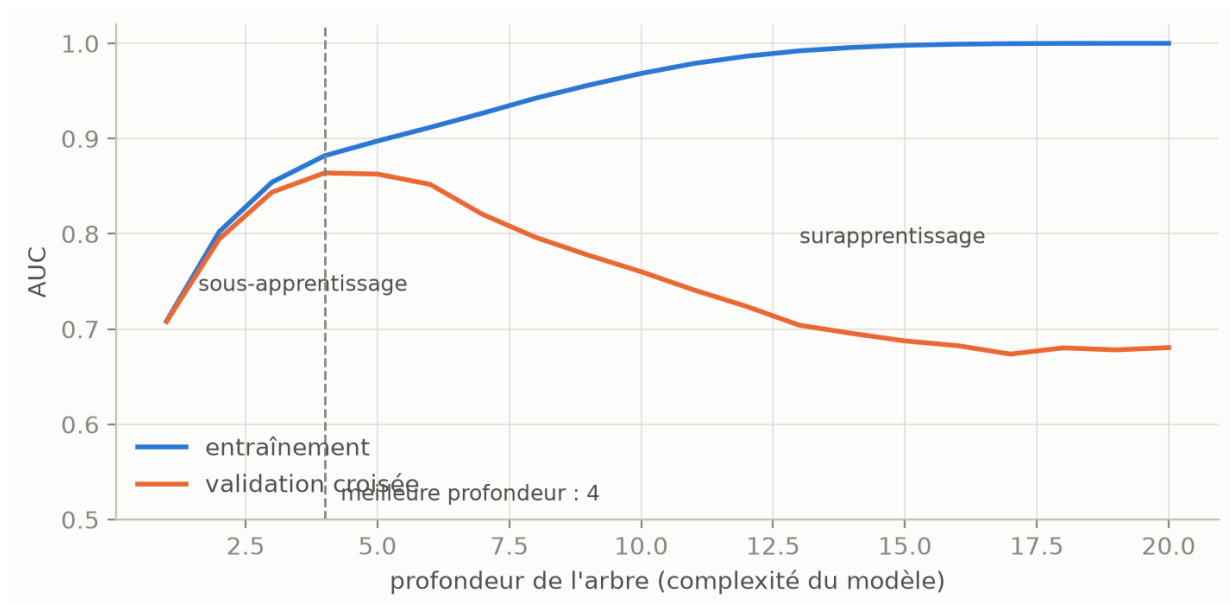


Figure 7: AUC d'un arbre de décision sur les données d'entraînement (bleu) et en validation croisée (orange) selon sa profondeur : sous-apprentissage à gauche, surapprentissage à droite.

C'est la signature classique : l'AUC d'entraînement ne fait que **monter** avec la complexité (elle atteint 1,00 : l'arbre profond a mémorisé tous les clients) ; l'AUC en validation monte, atteint un maximum à la profondeur 4 (0,864), puis **s'effondre** (0,68 à la profondeur 20). Un arbre de profondeur 20 est « parfait » sur ce qu'il a vu et à peine meilleur que le hasard sur ce qu'il n'a pas vu. Le choix de la complexité se lit **sur la courbe de validation**, jamais sur celle d'entraînement.

1.3.6 Les courbes d'apprentissage : diagnostiquer son modèle

La courbe précédente fait varier la complexité. Une **courbe d'apprentissage** fait varier la **quantité de données** d'entraînement, et répond à une question de gestionnaire : « si on collectait deux fois plus de données, cela servirait-il ? » On entraîne le modèle sur 200, 500, 1 000, 2 000, 4 000 puis 6 000 clients, et on trace l'AUC d'entraînement et de validation.

```
from sklearn.model_selection import learning_curve

tailles, train, val = learning_curve(modele_logit(), X_tr, y_tr, train_sizes=[200, 1000, 6000], cv=3,
    ← scoring="roc_auc")[:3]
print(tailles, train.mean(1).round(3), val.mean(1).round(3))
```

[200 1000 6000] [0.935 0.887 0.868] [0.831 0.851 0.86]

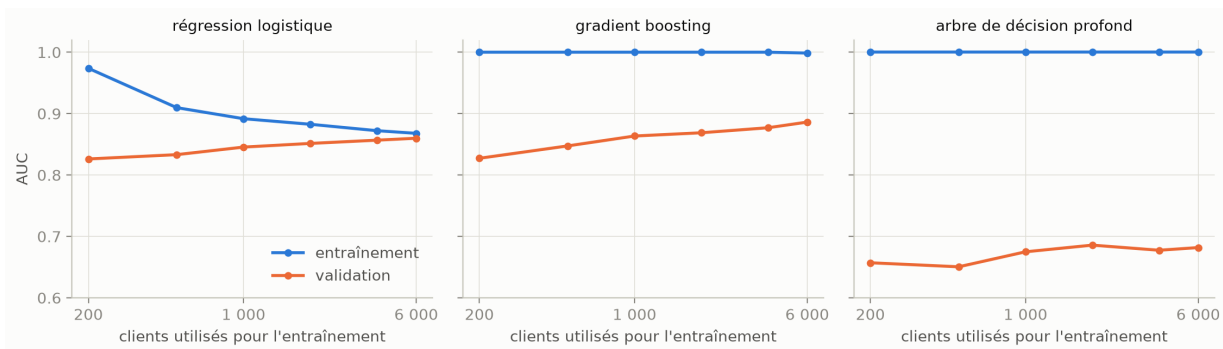


Figure 8: Courbes d'apprentissage de trois modèles sur les données de la boutique : AUC d'entraînement (bleu) et de validation (orange) selon le nombre de clients utilisés pour l'entraînement.

Chaque forme se lit comme un diagnostic :

- **Régression logistique** : les deux courbes **se rejoignent** à un niveau modeste (0,87 et 0,86 avec 6 000 clients). Écart faible, performance plafonnée : le modèle est **trop rigide** (biais). Ajouter des clients n'aidera presque pas ; il faut un modèle plus expressif ou de meilleures variables.
- **Arbre profond** : l'AUC d'entraînement vaut 1,00 dès le départ, celle de validation reste autour de 0,65 à 0,69, même avec beaucoup de données : l'écart est immense (variance). Il faut **contraindre** le modèle (limiter la profondeur, section 1.3.5) ou le moyenner (forêts, chapitre 2).
- **Gradient boosting** : l'AUC d'entraînement est parfaite (1,00) mais l'AUC de validation **continue de monter** avec les données (de 0,83 à 0,89), signe que **plus de données aideraient**. L'écart entre les deux courbes est grand, mais ce n'est pas un défaut ici : ce qui compte est le niveau de la courbe de validation, que le boosting domine nettement.

Le piège de l'écart. Un grand écart entre entraînement et validation n'est pas, à lui seul, la preuve d'un mauvais modèle : un modèle puissant peut mémoriser l'entraînement et très bien généraliser (c'est le cas du boosting ci-dessus). Ce qu'on regarde, c'est **la validation**, et si elle **progress**e quand on ajoute des données.

1.3.7 Que faire quand le modèle ne généralise pas ?

Diagnostic	Symptôme	Remèdes
Biais élevé (sous-apprentissage)	entraînement et validation tous deux mauvais, courbes qui se rejoignent	modèle plus souple, meilleures variables (chapitre 4), moins de régularisation
Variance élevée (surapprentissage)	entraînement excellent, validation nettement inférieure	plus de données , modèle plus simple, régularisation , arrêt précoce (section 1.5), moyenne de modèles (bagging, chapitre 2)
Bruit irréductible	tous les modèles plafonnent au même niveau	accepter la limite ; chercher de nouvelles informations plutôt qu'un meilleur algorithme

À retenir.

- $\mathbb{E}[(y - \hat{f})^2] = \sigma^2 + \text{biais}^2 + \text{variance}$: un plancher de bruit, une erreur systématique, une sensibilité à l'échantillon.
- Rendre un modèle plus souple **réduit le biais et augmente la variance** ; le meilleur modèle est un **compromis**. Un estimateur légèrement biaisé peut battre un estimateur sans biais (exemple : $0,8\bar{y}$).
- **Sous-apprentissage** : tout est mauvais. **Surapprentissage** : l'entraînement est excellent, la validation médiocre. On choisit la complexité sur la **courbe de validation**.
- Une **courbe d'apprentissage** dit si l'on manque de données (la validation monte encore) ou de souplesse (les courbes se rejoignent à un niveau bas).

Pour s'entraîner. Cahier, chapitre 1 : applications 1.5 et 1.6, exercices 1.7 et 1.8.

1.4 Modèles de référence et rigueur expérimentale

Un score n'a de sens que **comparé** à autre chose. « Notre modèle atteint 0,89 d'AUC » ne dit pas si c'est remarquable ou médiocre, et encore moins si l'écart avec le modèle précédent est réel ou imputable au hasard. Cette section donne trois outils de rigueur : les **modèles de référence** (ce qu'il faut battre), les **comparaisons avec incertitude** (de combien bat-on, et est-ce réel ?), et le **protocole de rapport** (comment présenter honnêtement un résultat).

1.4.1 Le modèle de référence : l'adversaire à battre

Avant d'entraîner quoi que ce soit de sophistiqué, on construit des **modèles de référence** (*base-lines*) très simples, qui fixent le niveau « gratuit ». Un modèle complexe qui ne les bat pas n'a aucune raison d'exister.

Référence	Principe	Ce qu'elle apprend
Naïve	toujours la classe majoritaire, ou toujours la probabilité moyenne (14 % de départs)	le niveau « sans information »
Règle métier	un score à partir d'une seule variable évidente (ici : plus la dernière commande est ancienne, plus le client est jugé risqué)	ce qu'un expert obtient sans modèle
Modèle simple	une régression logistique	ce qu'apporte un modèle linéaire standard, interprétable
Modèle actuel	ce qui est déjà en production, s'il y en a un	le seuil à franchir pour que le projet serve

Mesurons-les sur les données d'entraînement par validation croisée à 5 plis, avec quatre métriques : l'AUC, la précision moyenne (aire sous la courbe précision-rappel, qui vaut la prévalence de 14 % pour un modèle sans information), la perte logarithmique et l'exactitude.

```
from sklearn.dummy import DummyClassifier

naif = cross_val_score(DummyClassifier(strategy="most_frequent"), X_tr, y_tr, cv=5,
    ↪ scoring="accuracy")
print("exactitude du modèle naïf :", naif.mean().round(3))
```

exactitude du modèle naïf : 0.86

	AUC	précision moyenne	perte log.	exactitude
naïf (prévalence)	0.500	0.140	0.406	0.860
règle : récence seule	0.742	0.329	NaN	NaN
régression logistique	0.860	0.564	0.288	0.883
gradient boosting	0.888	0.617	0.270	0.892

Trois lectures :

1. Le modèle naïf obtient une exactitude de **0,860** et une AUC de **0,500** : il n'a rien appris et pourtant « se trompe » seulement dans 14 % des cas. Sa perte logarithmique (0,406) est celle d'une pièce biaisée à 14 % : $-[0,14 \ln 0,14 + 0,86 \ln 0,86] \approx 0,406$, ce que l'on peut vérifier à la main.
2. La **règle métier** (la récence seule) donne déjà une AUC de 0,74 : elle capte une partie réelle du signal. C'est le niveau « un analyste avec un tableur ».
3. La régression logistique grimpe à 0,860, le gradient boosting à 0,888. L'exactitude, elle, n'évolue presque pas (0,860 → 0,883 → 0,892) : on voit à nouveau pourquoi **elle est un mauvais juge** sur un problème déséquilibré.

Le bon réflexe. Présentez toujours un résultat sous la forme « *X, contre Y pour la référence* ». Un gain de 0,03 d'AUC sur une régression logistique peut valoir des milliers d'euros ou rien du tout selon le métier : c'est à la gérante, pas à l'algorithme, de le dire. Mais sans la référence, la question ne peut même pas être posée.

1.4.2 Comparer deux modèles : l'écart est-il réel ?

Le gradient boosting bat la régression logistique de 0,03 d'AUC. Mais nous avons vu (section 1.2.4) qu'une validation croisée est bruitée : cet écart est-il **réel**, ou aurait-il pu apparaître par hasard ? Quatre outils, qui répondent à des questions un peu différentes.

Principe commun : la comparaison appariée. On évalue les **deux modèles sur les mêmes découpages** et on étudie la **différence** des scores, découpage par découpage. Cela élimine la variabilité commune (un découpage « facile » est facile pour les deux) et ne garde que ce qui sépare réellement les modèles.

(1) **Le test t corrigé (Nadeau et Bengio, 2003).** On répète $J = 15$ découpages aléatoires 75 % / 25 % du jeu d'entraînement ($n_1 = 6750$ clients pour entraîner, $n_2 = 2250$ pour valider) et on note l'écart d'AUC d_j à chaque découpage. Le test t habituel traite les J écarts comme **indépendants** : $t = \bar{d} / \sqrt{s_d^2 / J}$. C'est faux, parce que les jeux d'entraînement des différents découpages se recouvrent beaucoup : les écarts sont corrélés, la variance de leur moyenne est **sous-estimée**, et les tests concluent trop souvent à une différence. Nadeau et Bengio corrigent la variance en la multipliant par un facteur lié au rapport des tailles :

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{J} + \frac{n_2}{n_1}\right) s_d^2}}, \quad \text{à comparer à une loi de Student à } J - 1 \text{ degrés de liberté.}$$

Ici $\frac{1}{J} + \frac{n_2}{n_1} = \frac{1}{15} + \frac{2250}{6750} = 0,067 + 0,333 = 0,4$: la correction multiplie la variance par 6 environ par rapport à la formule naïve (0,4 au lieu de $1/15 = 0,067$).

Sur nos données : écart moyen d'AUC $\bar{d} = 0,030$, écart-type des écarts $s_d = 0,0065$. Le test naïf donne $t = 17,7$ (absurdement significatif). Le test corrigé donne $t = 7,2$, encore très significatif,

avec un **intervalle de confiance à 95 % de 0,021 à 0,038** pour l'écart d'AUC. Conclusion honnête : le boosting est meilleur que la régression logistique, de **deux à quatre points d'AUC** sur ces données ; ce n'est pas un hasard.

(2) **Le test de McNemar.** Il compare deux **classifieurs** (des décisions, pas des scores) sur le **même** jeu de données. On ne regarde que les clients où les deux modèles **divergent** : b clients où le modèle A est juste et B faux, c clients où A est faux et B juste. Sous l'hypothèse « les deux modèles ont la même précision », ces désaccords se répartissent à pile ou face, et

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad \text{suit approximativement une loi du } \chi^2 \text{ à 1 degré de liberté.}$$

Sur un jeu de validation de 2 700 clients, avec la règle « contacter le client si le risque estimé dépasse 25 % » :

Les deux modèles divergent sur $b + c = 331$ clients : la régression logistique a raison et le boosting tort pour 120 d'entre eux ; c'est l'inverse pour 211. Si les deux modèles étaient équivalents, on s'attendrait à environ 165 de chaque côté. Le calcul donne $\chi^2 = \frac{(|120 - 211| - 1)^2}{331} = \frac{90^2}{331} = 24,5$, soit une probabilité critique de l'ordre de 10^{-6} : la différence est réelle. (L'exactitude passe de 83,8 % à 87,2 %.)

(3) **Le bootstrap du jeu de validation.** On tire 1 000 jeux de validation « de même taille » **avec remise** parmi les 2 700 clients, et pour chacun on recalcule l'écart d'AUC entre les deux modèles, **sur le même tirage** (appariement). La dispersion de ces 1 000 écarts donne un intervalle de confiance. Ici, l'écart observé est de 0,0235, et l'intervalle à 95 % va de **0,011 à 0,036** : il exclut zéro.

Quel outil pour quelle question ?

Outil	Question	Ce qu'il capture	Limite
Test t corrigé	la méthode A est-elle meilleure que B sur ce type de données ?	variabilité due à l'entraînement et à l'évaluation	suppose beaucoup de découpages ; approximatif
McNemar	deux décisions différent-elles ?	variabilité de l'évaluation, à seuil fixé	un seul seuil, ne mesure pas la qualité des scores
Bootstrap du jeu d'évaluation	de combien différent les scores (AUC, perte...) ?	variabilité due à la taille du jeu d'évaluation	ignore la variabilité due à l'entraînement

Aucun n'est « le bon » : on choisit selon la question, et mieux vaut en présenter deux qui concordent. Retenez surtout le geste : **toujours accompagner un écart de performance d'un intervalle**, et ne conclure que si celui-ci exclut une valeur négligeable.

1.4.3 Graines, reproductibilité et variabilité

Beaucoup d'algorithmes utilisent le hasard : initialisation, sous-échantillonnage de lignes ou de variables, découpage interne pour l'arrêt précoce. Leur résultat dépend d'une **graine** aléatoire. Mesurons ce que cela change : on entraîne 10 fois le même gradient boosting (avec arrêt précoce, qui tire au sort une partie de validation interne) sur le même jeu d'entraînement, avec 10 graines différentes, et on évalue chaque fois sur le même jeu de validation.

L'AUC varie de 0,881 à 0,890 selon la graine (écart-type 0,0025) : **neuf millièmes d'écart pour un même modèle**. Deux conséquences pratiques :

- Un écart de performance **inférieur à environ 0,005 d'AUC** entre deux modèles aléatoires est dans le bruit des graines : il ne faut pas en tirer de conclusion.
- Pour un résultat publié, on **fixe la graine** (reproductibilité : tout le monde retrouve le même nombre) **et** on rapporte la moyenne et l'écart-type sur plusieurs graines (honnêteté : le nombre n'est pas une constante de la nature).

La reproductibilité demande aussi de **tout consigner** : versions des bibliothèques, découpages (enregistrés, pas retirés au vol), paramètres, et un **pipeline** qui enchaîne le prétraitement et le modèle pour qu'aucune étape ne soit oubliée ou appliquée dans le mauvais ordre.

1.4.4 Le jeu de test, une seule fois

Après tout ce travail sur le jeu d'entraînement (comparaisons, validation croisée, graines), il reste à mettre le modèle à l'épreuve du **jeu de test** de 3 000 clients mis de côté à la section 1.1, **une fois**. Le protocole est strict :

1. toutes les décisions (modèle, variables, réglages, seuil) sont prises **avant**, sur l'entraînement et la validation ;
2. on entraîne le modèle final sur **tout le jeu d'entraînement** ;
3. on l'évalue **une seule fois** sur le jeu de test, et on rapporte le résultat **avec son intervalle** (bootstrap du jeu de test) ;
4. on ne modifie plus rien en fonction de ce résultat.

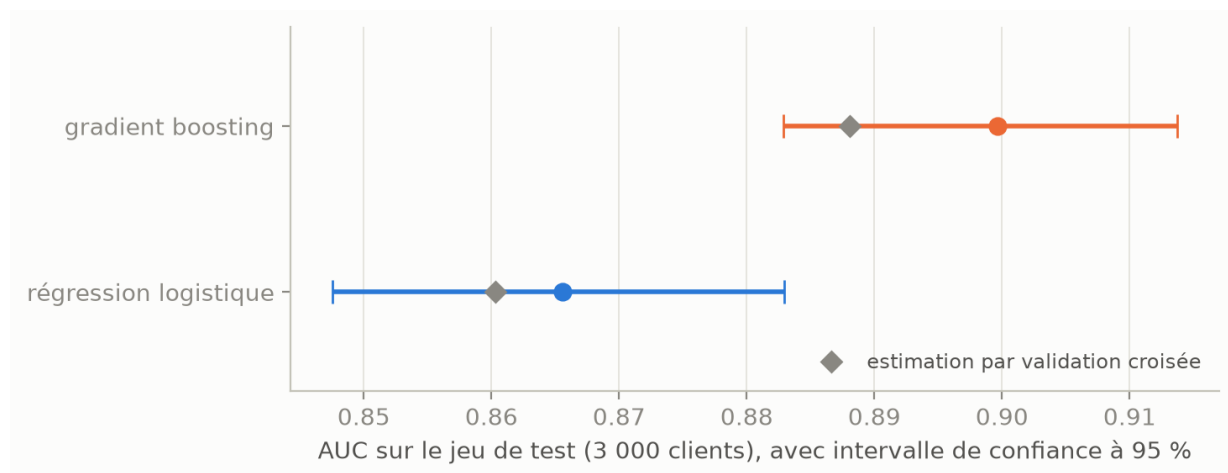


Figure 9: AUC finale des deux modèles sur le jeu de test (points colorés, avec intervalle de confiance à 95 % par bootstrap), et estimation par validation croisée (losanges gris).

Résultat final, sur des clients que personne n'avait regardés : **régression logistique 0,866** (intervalle à 95 % : 0,848 à 0,883) ; **gradient boosting 0,900** (0,883 à 0,914) ; écart de **0,034** (0,022 à 0,046). Les estimations par validation croisée (0,860 et 0,888, losanges gris) tombent bien dans ces intervalles : la démarche a tenu sa promesse. Remarquez la **largeur** des intervalles : malgré 3 000 clients de test, l'AUC n'est connue qu'à $\pm 0,016$ près (environ 420 clients partis seulement). Un modèle dont on annonce « 0,8996 » est annoncé avec deux chiffres de trop : « $0,90 \pm 0,015$ » est la bonne présentation.

Si le résultat sur le test vous déçoit. La tentation est de « retoucher » le modèle et de re-tester. Mais dès qu'un résultat sur le test guide une décision, le test est consommé : on n'a plus d'estimation honnête. Dans une vraie étude, on documente le résultat décevant tel quel, ou bien on met de côté un **nouveau** jeu de test frais.

1.4.5 Rapporter honnêtement : une liste de contrôle

Résumons la rigueur du chapitre en une liste, à parcourir avant de présenter un résultat à un collègue ou à la direction.

Point	Question à se poser	Exemple dans ce chapitre
Problème	quelle est la ligne ? la date de prédiction ? la cible, et quand est-elle connue ?	client au 31/12, départ à 90 jours
Fuite	chaque variable est-elle connue à la date de prédiction ? le prétraitement est-il dans le pipeline ?	commandes_apres_cible exclue
Découpage	imite-t-il l'usage réel (temps, groupes, strates) ?	75 % / 25 % stratifié
Référence	contre quoi se compare-t-on ?	naïf, règle de récurrence, régression logistique
Métrique	adaptée au métier ? complétée au-delà de l'exactitude ?	AUC, précision moyenne, perte logarithmique
Incertitude	intervalle ? écarts testés ? graines multiples ?	bootstrap, test t corrigé, 10 graines
Test	ouvert une seule fois ?	oui, section 1.4.4
Limites	ce que le modèle ne sait pas faire ?	données simulées ; 420 départs dans le test

À retenir.

- Un score se juge contre une **référence** : naïve, règle métier, modèle simple. L'exactitude d'un modèle naïf (0,860) rend ses 0,89 d'AUC éclairants.
- Comparer deux modèles, c'est comparer **des différences appariées** et leur associer un **intervalle** : test t **corrigé** (le t naïf est trop optimiste : 17,7 au lieu de 7,2), McNemar pour des décisions, bootstrap du jeu d'évaluation pour des scores.
- Un modèle aléatoire varie avec sa **graine** (écart-type 0,0025 d'AUC ici) : fixez-la, mais rapportez aussi la variabilité.
- Le **jeu de test** s'ouvre **une fois**, avec son intervalle ; l'AUC d'un jeu de 3 000 clients est connue à $\pm 0,016$ près.

Pour s'entraîner. Cahier, chapitre 1 : application 1.7, exercices 1.9 et 1.10.

1.5 ⊕ Pour aller plus loin : le réglage des hyperparamètres

Section optionnelle. Elle approfondit un geste que les chapitres suivants pratiquent sans cesse : choisir les réglages d'un modèle. On peut la sauter à la première lecture ; le piège de la section 1.5.5 mérite cependant d'être lu.

Un modèle d'apprentissage automatique a deux sortes de « boutons ». Les premiers sont **appris** à partir des données ; les seconds sont **fixés par vous avant l'apprentissage**. Bien régler les seconds change souvent beaucoup la performance, mais cela se fait avec une méthode, et un piège.

1.5.1 Paramètres et hyperparamètres

- Les **paramètres** sont les nombres que l'algorithme **apprend** : les coefficients d'une régression logistique, les questions posées dans les nœuds d'un arbre.
- Les **hyperparamètres** sont les réglages que **vous choisissez** avant l'entraînement et qui gouvernent la souplesse du modèle : la force de la régularisation, la profondeur maximale d'un arbre, la vitesse d'apprentissage d'un boosting. Ils contrôlent le compromis biais-variance de la section 1.3.

Modèle	Exemples d'hyperparamètres	Effet
Régression logistique	force de régularisation C	C petit : modèle rigide (biais), C grand : souple (variance)
Arbre de décision	profondeur maximale, taille minimale des feuilles	profond : souple
Gradient boosting	nombre d'itérations, vitesse d'apprentissage, nombre de feuilles par arbre, régularisation ℓ_2	plus d'itérations ou de feuilles : plus souple

Régler, c'est chercher la combinaison qui **maximise le score de validation** : une fonction coûteuse (chaque évaluation est une validation croisée), sans formule, qu'on ne sait évaluer qu'en l'essayant. C'est de l'**optimisation par boîte noire**.

1.5.2 La grille et la recherche aléatoire

La première idée est la **recherche sur grille** (*grid search*) : on liste quelques valeurs pour chaque hyperparamètre et on essaie **toutes les combinaisons**. Trois valeurs pour quatre hyperparamètres, c'est déjà $3^4 = 81$ combinaisons, multipliées par le nombre de plis : le coût croît **exponentiellement** avec le nombre d'hyperparamètres.

La **recherche aléatoire** (*random search*) tire les combinaisons au hasard dans des intervalles, pour un budget fixé. Bergstra et Bengio (2012) ont montré pourquoi elle est souvent plus efficace : **en pratique, peu d'hyperparamètres comptent vraiment** pour un problème donné, et on ne sait pas lesquels à l'avance. Avec une grille de $3 \times 3 = 9$ essais, chaque hyperparamètre n'est testé qu'à **trois valeurs** distinctes ; avec 9 tirages aléatoires, il est testé à **neuf valeurs** distinctes. Si seul l'un des deux compte, la recherche aléatoire a exploré trois fois plus de valeurs de celui-ci.

Un calcul à la main. Supposons que 5 % de l'espace des réglages soient « excellents ». Chaque tirage aléatoire tombe dans cette zone avec probabilité 0,05, donc n tirages indépendants **manquent** la zone avec probabilité $0,95^n$. La probabilité d'en toucher au moins un est $1 - 0,95^n$: 0,37 pour $n = 9$, et il faut $n = 59$ tirages pour atteindre 95 % (car $0,95^{59} \approx 0,05$). **Le budget nécessaire ne dépend pas du nombre d'hyperparamètres** : c'est l'avantage décisif sur la grille.

Essayons sur la prédiction de départ, avec un gradient boosting, sur un sous-échantillon de 5 000 clients (pour rester rapide) et une validation croisée à 3 plis. La grille : trois vitesses d'apprentissage $\times$ trois tailles d'arbre = 9 essais.

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV

cv = StratifiedKFold(3, shuffle=True, random_state=0)
grille = {"learning_rate": [0.03, 0.1, 0.3], "max_leaf_nodes": [8, 31, 63]}
gs = GridSearchCV(modele_hgb(), grille, cv=cv, scoring="roc_auc").fit(X5, y5)
print(gs.best_params_, round(gs.best_score_, 4))
```

```
{'learning_rate': 0.03, 'max_leaf_nodes': 8} 0.8975
```

Sur ces données, la grille trouve un réglage à **0,8975** d'AUC (vitesse 0,03, arbres de 8 feuilles) ; les 9 combinaisons s'échelonnent de 0,874 à 0,8975, ce qui montre que **le réglage compte** (jusqu'à 2,3 points). Cinq recherches aléatoires de 9 essais, avec des graines différentes, trouvent entre 0,892 et 0,897 (moyenne 0,895), **sans faire mieux** que la grille. Ce n'est pas un échec de la méthode : avec 9 essais et un espace réduit, la grille s'est trouvée bien placée. L'avantage de l'aléatoire apparaît quand l'espace compte **beaucoup** d'hyperparamètres ou quand on ne sait pas où chercher.

Où se trouve le bon réglage ? Ici, le meilleur modèle est **simple** (8 feuilles par arbre, apprentissage lent). Rien d'étonnant : le signal se résume à quelques seuils (section 1.3), et un modèle trop souple ajusterait le bruit. Cette observation rejoint la leçon du compromis biais-variance : le bon réglage est celui que la **validation** désigne, pas le plus puissant en apparence.

1.5.3 L'optimisation bayésienne : apprendre des essais précédents

La grille et le hasard **n'apprennent rien** de leurs essais : le vingtième tirage ignore que les dix-neuf premiers étaient mauvais dans telle région. L'**optimisation bayésienne** utilise l'historique pour décider où essayer ensuite : on construit un **modèle de substitution** du score de validation en fonction des hyperparamètres (un processus gaussien, ou, dans Optuna, des estimateurs de densité), et on essaie ensuite le point qui semble **le plus prometteur**, en équilibrant *exploitation* (près des bons points connus) et *exploration* (là où l'on ne sait rien).

Le plus répandu dans les bibliothèques actuelles est le **TPE** (*Tree-structured Parzen Estimator*). Son idée tient en trois phrases : on sépare les essais déjà faits en deux groupes, les **bons** (les 25 % meilleurs scores) et les **autres** ; on estime la densité $\ell(x)$ des réglages parmi les bons et la densité $g(x)$ parmi les autres ; on essaie ensuite le réglage x qui **maximise le rapport** $\ell(x)/g(x)$, c'est-à-dire un réglage « typique des bons essais et rare parmi les mauvais ».

```
import optuna

def objectif(essai):
    p = {"learning_rate": essai.suggest_float("learning_rate", 0.01, 0.5, log=True),
         "max_leaf_nodes": essai.suggest_int("max_leaf_nodes", 4, 128),
         "l2_regularization": essai.suggest_float("l2_regularization", 1e-3, 10, log=True),
         "min_samples_leaf": essai.suggest_int("min_samples_leaf", 5, 100)}
    return cross_val_score(modele_hgb(**p), X5, y5, cv=cv, scoring="roc_auc").mean()

etude = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(seed=0))
etude.optimize(objectif, n_trials=30)
print(round(etude.best_value, 4))
```

```
0.8951
```

(Le TPE démarre par 10 essais **aléatoires**, identiques ici à ceux de la recherche aléatoire puisque la graine est la même : c'est pourquoi les dix premiers points de la figure sont confondus.) Les deux méthodes finissent au même niveau : **0,895** pour le TPE, **0,898** pour la recherche aléatoire, un écart de 0,003, dans le bruit de la mesure (section 1.4.3). Dans cette expérience, **le TPE n'a pas fait mieux que le hasard** ; les dix premiers essais des deux méthodes valent la même moyenne (0,886), mais les **dix derniers** du TPE ont une moyenne de 0,893, contre 0,888 pour le hasard : il a bien appris à se concentrer sur les bonnes régions, sans que cela ait suffi, ici, à battre le meilleur tirage du hasard. Ce résultat est un bon exemple d'**honnêteté expérimentale** : on ne peut pas conclure qu'une méthode « plus intelligente » gagne toujours, surtout avec un budget de 30 essais sur 4 hyperparamètres. L'optimisation bayésienne tient surtout ses promesses quand

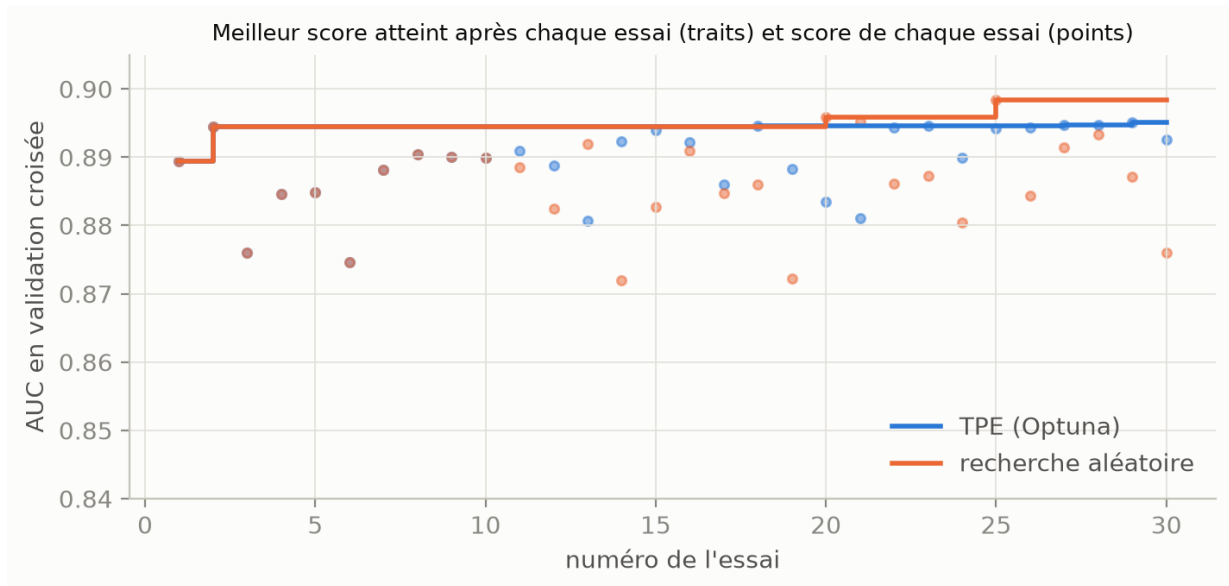


Figure 10: Optimisation de 4 hyperparamètres d'un gradient boosting en 30 essais : meilleur score atteint après chaque essai (traits) et score de chaque essai (points), pour le TPE d'Optuna (bleu) et pour la recherche aléatoire (orange).

chaque essai coûte très cher (réseaux de neurones, grands jeux de données) et que le budget est de l'ordre de quelques dizaines d'essais.

1.5.4 L'arrêt précoce

Pour les modèles construits **itération après itération** (boosting, réseaux de neurones), le **nombre d'itérations** est lui-même un hyperparamètre, et un régulateur très efficace : trop peu, le modèle sous-apprend ; trop, il surapprend. L'**arrêt précoce** (*early stopping*) évite d'avoir à le deviner : on met de côté une petite partie des données (15 % ici), on suit la perte sur cette partie à chaque itération, et on **s'arrête quand elle cesse de baisser** pendant un nombre fixé d'itérations (la « patience », ici 10).

La perte d'entraînement baisse sans cesse, celle de la validation interne atteint son minimum à l'itération 45 puis cesse de baisser ; l'algorithme s'arrête à l'itération 55 (45 plus la patience de 10), bien avant le maximum de 500. **On a évité de choisir le nombre d'arbres à la main.** Attention cependant : la partie de validation interne est tirée **au hasard**, ce qui rend le nombre d'itérations dépendant de la graine (c'est le phénomène de la section 1.4.3).

1.5.5 Le piège du réglage et la validation imbriquée

Voici le piège central de cette section. Chaque fois que l'on essaie une combinaison et qu'on lit son score de validation, on **apprend quelque chose du jeu de validation**. Après des centaines d'essais, le meilleur score est celui d'une combinaison **sélectionnée pour avoir bien marché sur cette validation**, en partie par chance : c'est le biais d'optimisme de la démonstration 1.1.2, appliqué à la sélection d'un réglage.

Mesurons-le dans un cas où l'on connaît la vérité : **il n'y a rien à apprendre**. On prend 150 clients dont on remplace les étiquettes par un tirage à pile ou face, et on règle un arbre de décision (profondeur, taille des feuilles, nombre de variables considérées) par 300 essais aléatoires évalués par validation croisée à 5 plis. On évalue ensuite le réglage retenu sur 1 000 clients « neufs », eux aussi à étiquettes aléatoires.

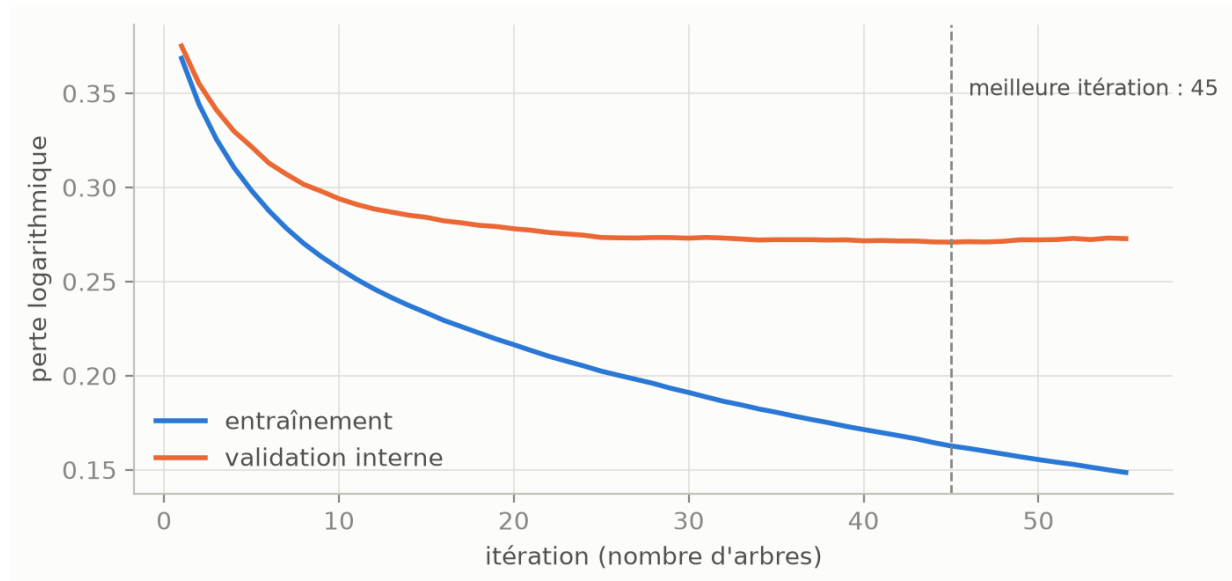


Figure 11: Perte logarithmique d'un gradient boosting sur les données d'entraînement (bleu) et sur la partie de validation interne (orange) selon le nombre d'itérations ; l'arrêt précoce interrompt l'entraînement peu après le minimum de la validation.

Le réglage sélectionné annonce une AUC de **0,66** en validation croisée, alors qu'il n'y a **aucun signal** ; sur des données neuves, il tombe à **0,53**, le hasard. Les 0,13 de différence mesurent l'auto-persuasion de la recherche : avec 300 essais, l'un d'eux a forcément eu de la chance. La parade est la **validation croisée imbriquée** (section 1.2.5) : la recherche d'hyperparamètres est refaite à **l'intérieur** de chaque pli d'une validation externe, et le score est mesuré sur le pli externe, que la recherche n'a jamais vu.

Le même calcul, **imbriqué**, pour 30 essais par recherche interne (X_a et y_a désignent les 150 clients aux étiquettes aléatoires) :

```
recherche = RandomizedSearchCV(DecisionTreeClassifier(random_state=0), dist_arbre, n_iter=30, cv=3,
    ↳ scoring="roc_auc", random_state=0)
imbrique = cross_val_score(recherche, Xa, ya_, cv=StratifiedKFold(5, shuffle=True, random_state=0),
    ↳ scoring="roc_auc")
print("validation imbriquée :", imbrique.mean().round(3))
```

```
validation imbriquée : 0.483
```

La validation imbriquée donne **0,48** : elle voit juste (pas de signal, donc le hasard). Elle estime la performance de la **procédure complète** « régler, puis entraîner », et c'est cette procédure qu'il faut évaluer.

Règles pour un réglage honnête.

- Le **jeu de test** (section 1.1) ne sert **jamais** au réglage.
- Limitez la **taille de la recherche** : plus on essaie de combinaisons sur peu de données, plus l'optimisme grandit. Ici, 300 essais sur 150 clients ; sur 9 000 clients et 30 essais, l'effet est bien plus faible.
- Quand l'estimation honnête de la performance compte (publication, décision d'investissement), utilisez la validation **imbriquée**.
- Après le réglage, **réentraînez** le modèle final avec les meilleurs hyperparamètres sur toutes les données d'entraînement.

À retenir.

- Les **hyperparamètres** sont fixés avant l'apprentissage et pilotent le compromis biais-variance ; on les règle par le score de **validation**.
- La **recherche aléatoire** coûte un budget fixe, quel que soit le nombre d'hyperparamètres ($1 - 0,95^n$ chances de toucher une zone excellente de 5 %) ; la grille croît exponentiellement.
- L'**optimisation bayésienne** (TPE, Optuna) exploite l'historique des essais : utile quand chaque essai coûte cher, **sans garantie** de gagner (ici : 0,895 contre 0,898 pour le hasard).
- L'**arrêt précoce** choisit le nombre d'itérations d'un modèle itératif en surveillant une validation interne.
- Le meilleur score d'une recherche est **optimiste** : sur du bruit pur, 0,66 annoncé pour 0,53 réel. Seule la validation **imbriquée** estime honnêtement la procédure.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.8 et 1.9, exercices 1.11 et 1.12.

Bilan du chapitre 1

Vous savez maintenant :

- **formuler** un problème d'apprentissage : la ligne du tableau, la **date de prédiction**, la cible et le moment où elle est connue ; distinguer la **perte** que l'algorithme minimise de la **métrique** qui guide la décision ;
- **démontrer** pourquoi l'erreur d'entraînement est optimiste, et **séparer les données** en trois rôles (entraînement, validation, test) selon un schéma adapté (aléatoire, stratifié, groupé, temporel) ;
- **repérer et éviter la fuite d'information** : variable connue après la date de prédiction (le mirage de 0,89 à 0,92 d'AUC), prétraitement ou sélection de variables hors du pipeline (0,88 d'AUC sur du pur bruit) ;
- **estimer** une performance par **validation croisée** : choisir k (5 ou 10), la stratifier, la grouper ou la rendre temporelle, et savoir que **l'écart-type entre plis n'est pas une barre d'erreur** ;
- **décomposer** l'erreur en bruit, biais carré et variance, **diagnostiquer** sous-apprentissage et surapprentissage sur une courbe de complexité et une **courbe d'apprentissage**, et en tirer les remèdes ;
- **se comparer à une référence** (naïve, règle métier, modèle simple), **comparer deux modèles avec un intervalle** (test t corrigé, McNemar, bootstrap), tenir compte de la variabilité des graines, et n'ouvrir le jeu de test qu'**une fois** ;
- (en option) **régler des hyperparamètres** (grille, recherche aléatoire, TPE avec Optuna, arrêt précoce) **sans s'auto-persuader**, grâce à la validation croisée imbriquée.

Le fil conducteur du chapitre tient en une phrase : **un modèle vaut ce que vaut la façon dont on l'a évalué**. Sur un même jeu de données, on peut annoncer 0,92 ou 0,89 d'AUC (fuite ou pas), 3,5 ou 7 d'erreur (validation aléatoire ou temporelle), 0,66 ou 0,53 (réglage optimiste ou honnête) : la différence n'est pas dans le modèle, mais dans la **discipline**.

Le chapitre 2 apporte les modèles : de la régression logistique revue comme un problème d'optimisation aux **arbres**, aux **forêts aléatoires** et au **gradient boosting**, qui s'est imposé comme la référence sur les tableaux de données. Chacun sera évalué avec les règles de ce chapitre.

Pour s'entraîner. Cahier, chapitre 1 : applications 1.1 à 1.9 (fuite d'information, loterie du découpage, choix de k , série temporelle, biais-variance, courbes d'apprentissage, comparaison rigoureuse, recherche d'hyperparamètres, validation imbriquée) et exercices 1.1 à 1.12.

Chapitre 2 : Apprentissage supervisé

« Tous les modèles de ce chapitre font la même chose : ils apprennent une règle à partir d'exemples dont on connaît la réponse. Ils ne diffèrent que par la **forme** de la règle qu'ils savent écrire. »

Le chapitre 1 a installé la démarche : poser le problème, réserver des données pour juger, valider, se méfier du surapprentissage. Il est temps de rencontrer les **modèles** eux-mêmes. Nous allons les parcourir comme une échelle : chaque barreau corrige un défaut du précédent.

Le chemin de ce chapitre

- **2.1 Modèles linéaires et logistiques** : le modèle le plus simple, revu avec les yeux de l'apprentissage automatique : une **fonction de perte**, un algorithme (la descente de gradient) et une **pénalité**.
- **2.2 Arbres de décision** : des règles « si... alors... » apprises automatiquement ; ils capturent les **seuils** et les **interactions** qu'un modèle linéaire ignore, mais ils sont instables.
- **2.3 Forêts aléatoires** : on moyenne beaucoup d'arbres différents ; la variance s'effondre.
- **2.4 Gradient boosting** : on construit les arbres l'un après l'autre, chacun corrigeant les erreurs des précédents. C'est, aujourd'hui, la famille la plus performante sur les données tabulaires. XGBoost et LightGBM en sont les deux implémentations vedettes.
- ⊕ **2.5 SVM, k plus proches voisins, Bayes naïf** : trois idées classiques, utiles à connaître.
- ⊕ **2.6 CatBoost, stacking et blending** : aller plus loin, et combiner des modèles sans tricher.

L'idée directrice : un seul problème, plusieurs règles. Dans tout le chapitre, nous posons la même question aux mêmes données, avec le même découpage et la même mesure de réussite. Seul le modèle change. Cela permet de comparer honnêtement, et de voir ce que chaque famille apporte (ou n'apporte pas).

Le problème qui nous accompagne

La gérante veut savoir, parmi ses clients, **lesquels vont cesser de commander** dans les 90 jours qui viennent, afin de leur proposer une attention particulière (un message, une offre). C'est le problème de **résiliation** (*churn*) : prédire une variable à deux issues, $\text{churn}_{90j} \in \{0, 1\}$, à partir de ce que l'on sait du client aujourd'hui.

Les données (décrites dans l'introduction du volume) sont un tableau `clients_ml.csv` de 12 000 clients, avec 19 variables d'entrée : âge, ville (20 modalités), canal d'acquisition, ancienneté,

nombre de commandes et montant des 12 derniers mois, récence (jours depuis la dernière commande), retours, satisfaction, tickets au support, programme de fidélité, promotions reçues, ouverture des courriels, délai de livraison... Certaines valeurs sont **manquantes** (13 % des satisfactions, par exemple). Environ 14 % des clients partent : le problème est **déséquilibré** sans l'être extrêmement (le chapitre 4 traitera les cas plus durs).

Conformément au chapitre 1, nous avons mis de côté un **jeu de test** (3 000 clients, un quart des données, tiré au hasard en conservant la proportion de départs) que nous n'utiliserons qu'à la fin, pour juger. Le **jeu d'entraînement** compte 9 000 clients ; quand il faut choisir un réglage, nous le faisons par validation croisée **à l'intérieur** de ce jeu (volume III, section 1.2).

Une colonne à ne pas toucher. Le fichier contient aussi `commandes_apres_cible`, le nombre de commandes des trois mois **suivants**. Elle contient la réponse en filigrane : un modèle qui l'utilise paraît parfait et ne servira à rien le jour où l'on prédit réellement l'avenir. C'est la **fuite d'information** (volume III, section 1.1). Nous l'excluons, avec les autres cibles, de toutes les variables d'entrée.

Ce qu'« apprendre » veut dire

Un modèle supervisé est, au fond, **une fonction** f qui associe à un client x (le vecteur de ses caractéristiques) une prédiction $f(x)$. Pour choisir f , on dispose de n exemples (x_i, y_i) et de trois ingrédients :

1. une **famille de fonctions** $\mathcal{F}$ (les droites, les arbres, les sommes d'arbres...) : c'est ce que le modèle *sait écrire* ;
2. une **perte** $\ell(y, f(x))$, qui mesure le coût d'une erreur ;
3. un **algorithme** qui cherche, dans $\mathcal{F}$, la fonction de perte moyenne minimale sur les exemples (la minimisation du **risque empirique**) : $\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{n} \sum_i \ell(y_i, f(x_i))$, souvent additionnée d'une pénalité qui limite la complexité.

Les sections suivantes varient ces trois ingrédients. Retenons dès maintenant la leçon du chapitre 1 : plus la famille $\mathcal{F}$ est riche, mieux elle ajuste les données d'entraînement, et plus elle risque d'apprendre le bruit. **Le bon modèle n'est pas le plus riche : c'est celui dont la richesse est adaptée à la quantité de données et à la structure du phénomène.**

Pour s'entraîner. Cahier, chapitre 2 : les applications 2.1 à 2.10 et les exercices 2.1 à 2.12 accompagnent les sections de ce chapitre ; chaque section renvoie aux siens.

2.1 Modèles linéaires et logistiques vus sous l'angle du ML

Intuition. Le modèle linéaire est le **point de départ obligé** : simple, rapide, lisible, souvent étonnamment bon. Mais l'apprentissage automatique ne le regarde pas comme la statistique : il y voit une **famille de fonctions**, une **perte** à minimiser et un **algorithme** (la descente de gradient) qui fait descendre cette perte pas à pas. Ces trois mots reviendront dans tout le chapitre.

Le volume II a déjà présenté la régression logistique (volume II, section 2.2) et la régularisation (volume II, section 1.5) avec les yeux du statisticien : un modèle probabiliste, des coefficients, des

tests. Nous ne le refaisons pas. Nous posons une autre question : **que se passe-t-il quand le but est de prédire, et pas d'expliquer ?**

2.1.1 Une perte pour chaque philosophie

Un modèle linéaire calcule un **score** $f(x) = w^\top x + b$: une somme pondérée des caractéristiques du client. Pour un problème à deux classes, on note $y \in \{-1, +1\}$ et on regarde la **marge**

$$m = yf(x).$$

Une marge positive signifie « bien classé » (le score et la réalité ont le même signe) ; plus elle est grande, plus le modèle est sûr de lui à raison ; une marge négative est une erreur, d'autant plus grave qu'elle est grande en valeur absolue.

Une **fonction de perte** attribue un coût à chaque marge. Le coût qui nous intéresse vraiment est le **0-1** : 1 si l'on se trompe, 0 sinon. Hélas, il est plat presque partout (sa pente est nulle) et il saute en $m = 0$: on ne sait pas le minimiser avec une descente de gradient. On le remplace donc par un **substitut convexe** qui le majore :

Perte	Formule en fonction de la marge m	Usage typique
0-1	$1[m \leq 0]$	ce que l'on voudrait minimiser
Charnière (<i>hinge</i>)	$\max(0, 1 - m)$	SVM (section 2.5)
Logistique	$\ln(1 + e^{-m})$	régression logistique, boosting
Quadratique	$(1 - m)^2$	moindres carrés appliqués à la classification

marge	0-1	charnière	logistique	quadratique
-2.0	1	3.0	2.127	9.0
-1.0	1	2.0	1.313	4.0
0.0	1	1.0	0.693	1.0
1.0	0	0.0	0.313	0.0
2.0	0	0.0	0.127	1.0

Lisez les lignes du tableau : pour une marge de -2 (une erreur franche), la perte logistique vaut environ 2,13 et la charnière 3 ; pour une marge de $+2$ (bien classé avec assurance), la charnière tombe à 0 (elle ne se soucie plus de cet exemple) tandis que la logistique reste faiblement positive (0,127) : elle continue à *pousser* un peu pour augmenter la confiance. La perte quadratique, elle, punit aussi les marges **très positives** ($(1 - 3)^2 = 4$) : elle déteste les clients « trop bien classés », ce qui n'a aucun sens pour la classification.

Le choix de la perte est un choix de modèle. Charnière + pénalité ℓ_2 = SVM. Logistique = régression logistique = maximum de vraisemblance d'un modèle de Bernoulli (volume II, section 2.2). Exponentielle e^{-m} = AdaBoost. Même famille de fonctions (le score linéaire), algorithmes et comportements différents.

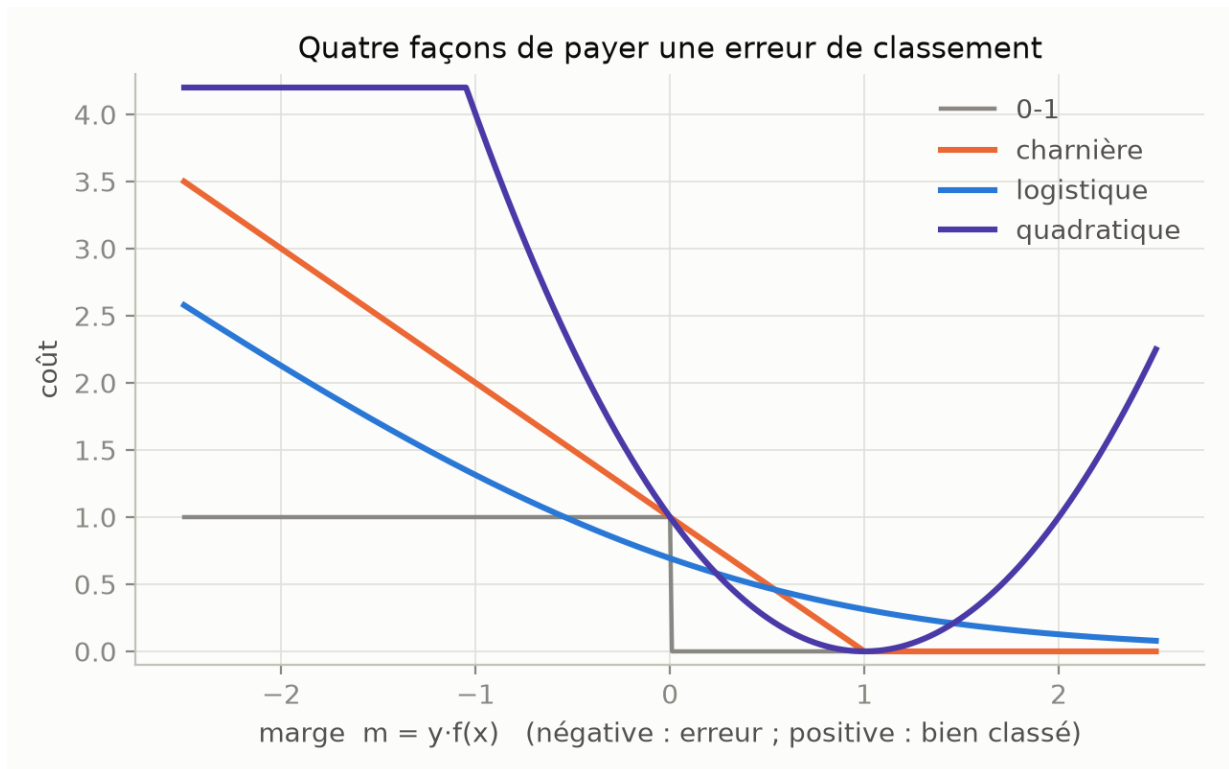


Figure 12: Quatre fonctions de perte en fonction de la marge. La perte 0-1 est un escalier ; les trois autres la majorent et sont convexes, donc minimisables par descente de gradient.

2.1.2 La régression logistique, une perte et un gradient

Pour la régression logistique, on code plutôt $y \in \{0, 1\}$ et l'on note $p_i = \sigma(f(x_i)) = 1/(1 + e^{-f(x_i)})$ la probabilité prédite. La perte moyenne est l'opposé de la log-vraisemblance :

$$L(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

Le gradient, en deux lignes. Posons $\ell(f) = -y \ln \sigma(f) - (1 - y) \ln(1 - \sigma(f))$. Comme $\sigma'(f) = \sigma(f)(1 - \sigma(f))$, on obtient

$$\frac{\partial \ell}{\partial f} = -\frac{y}{\sigma} \sigma(1 - \sigma) + \frac{1 - y}{1 - \sigma} \sigma(1 - \sigma) = -y(1 - \sigma) + (1 - y)\sigma = \sigma(f) - y = p - y.$$

Par la règle de la chaîne, $f = w^\top x + b$ donne

$$\nabla_w L = \frac{1}{n} \sum_i (p_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_i (p_i - y_i).$$

L'erreur de prédiction $p_i - y_i$ pondère chaque client : un client que l'on prédit à 0,9 alors qu'il n'est pas parti ($y = 0$) tire fort le modèle vers le bas ; un client bien prédit ($p_i \approx y_i$) ne le tire presque pas. Le Hessien, $\frac{1}{n} X^\top \text{diag}(p_i(1 - p_i)) X$, est semi-défini positif : la perte est **convexe**, elle n'a **qu'un seul** minimum, et la descente de gradient le trouve. (On retrouve ici les équations du maximum de vraisemblance du volume II ; la différence est que l'on cherche maintenant à les résoudre par un algorithme, sans formule fermée.)

2.1.3 La descente de gradient : un pas à la main

La descente de gradient répète : $w \leftarrow w - \eta \nabla_w L$, où $\eta > 0$ est le **pas** d'apprentissage (volume I, section 1.3.3). Voyons un pas, sur un jeu de données minuscule : quatre clients, une seule variable x (une « note de risque ») et la réponse y (1 = parti).

Client	x	y
1	-2	0
2	-1	0
3	1	1
4	3	1

On part de $w = 0, b = 0$. Alors $f = 0$ partout et $p_i = \sigma(0) = 0,5$ pour les quatre clients. Les erreurs de prédiction valent $p_i - y_i = (0,5, 0,5, -0,5, -0,5)$. Donc

$$\nabla_w L = \frac{1}{4}[0,5(-2) + 0,5(-1) - 0,5(1) - 0,5(3)] = \frac{-3,5}{4} = -0,875, \quad \frac{\partial L}{\partial b} = \frac{1}{4}(0,5 + 0,5 - 0,5 - 0,5) = 0.$$

Avec un pas $\eta = 0,5$, le nouveau coefficient est $w = 0 - 0,5 \times (-0,875) = 0,4375$ (et b ne bouge pas). La perte initiale vaut $\ln 2 \approx 0,693$; après le pas, elle baisse.

La perte passe de 0,693 à 0,396 : un seul pas a déjà nettement amélioré le modèle. Répétés des centaines de fois, ces pas convergent vers le minimum. Dans la **descente de gradient stochastique** (SGD), on ne calcule pas le gradient sur tous les clients mais sur un petit paquet tiré au hasard (un *mini-lot*) : chaque pas est bruité, mais il est $n/\text{taille du lot}$ fois moins cher, ce qui rend l'apprentissage possible sur des millions de lignes. C'est la méthode d'entraînement de presque tous les modèles à grande échelle, des modèles linéaires aux réseaux de neurones.

2.1.4 Pourquoi il faut mettre les variables à l'échelle

Sur nos données, la **récence** s'exprime en jours (de 0 à 365), le **nombre de commandes** est de l'ordre de la dizaine, le taux d'ouverture des courriels est entre 0 et 1. Une descente de gradient sur de telles variables brutes est un cauchemar : le gradient est dominé par la variable aux grandes valeurs, et le pas η qui évite l'explosion pour celle-là est ridiculement petit pour les autres.

Formellement, la vitesse de convergence dépend du **conditionnement** κ , le rapport entre la plus grande et la plus petite valeur propre du Hessien de la perte au voisinage du minimum : pour une perte quadratique, l'erreur est multipliée par $\frac{\kappa-1}{\kappa+1}$ à chaque pas au mieux. Plus κ est grand, plus la descente zigzague.

Sur un modèle à deux variables seulement (la récence et le nombre de commandes), le conditionnement du Hessien est d'environ 377 486 avec les variables brutes, et de 13,6 une fois les variables **standardisées** (moyenne 0, écart-type 1). Le graphique montre la conséquence : avec le plus grand pas de la grille pour lequel la perte décroît sans osciller (0,000), la perte vaut encore 0,50 après 400 itérations (le minimum est 0,342), tandis que la descente sur variables standardisées atteint 0,342, soit le minimum à quelques millièmes près.

Standardiser n'est pas facultatif... pour certains modèles. Les modèles entraînés par descente de gradient (linéaire, logistique, réseaux de neurones), à pénalité (Ridge, Lasso) ou fondés sur des **distances** (k plus proches voisins, SVM) exigent des variables à des échelles comparables. Les **arbres** et leurs dérivés (forêts, boosting) n'en ont pas besoin : ils ne com-

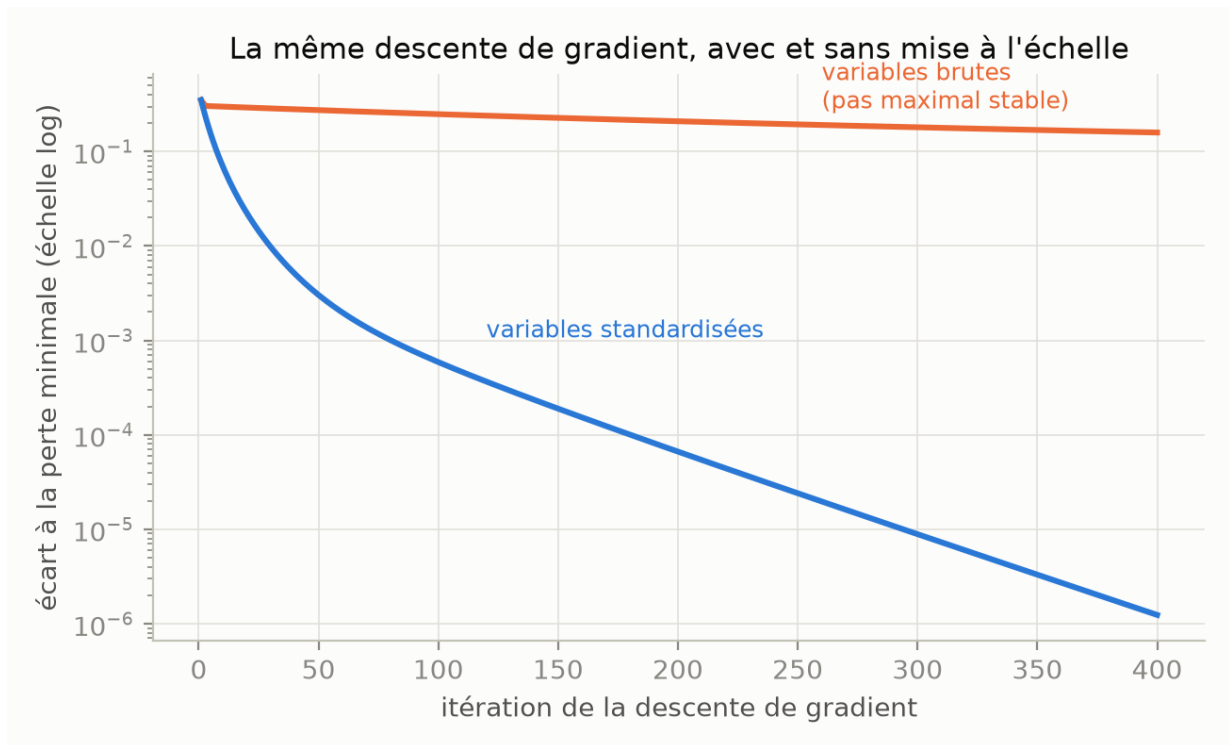


Figure 13: Écart à la perte minimale au fil des itérations, pour la même descente de gradient. Orange : variables brutes (le pas doit rester minuscule). Bleu : variables standardisées.

parent une variable qu'à des seuils, et une transformation croissante (changer l'unité, passer au logarithme) ne change ni l'ordre ni donc les découpages. Le chapitre 4 reviendra sur ces règles en détail (section 4.1).

2.1.5 La pénalité : une contrainte déguisée

Quand les variables sont nombreuses ou corrélées, les coefficients de la régression logistique peuvent devenir grands, instables, et le modèle sur-apprend. On ajoute à la perte une **pénalité** :

$$\min_{w,b} L(w,b) + \lambda \Omega(w), \quad \Omega(w) = \|w\|_2^2 \text{ (Ridge)} \quad \text{ou} \quad \|w\|_1 = \sum_j |w_j| \text{ (Lasso)}.$$

Pénalité et contrainte, deux visages du même problème. Par la théorie des multiplicateurs de Lagrange (volume I, section 1.3.5), minimiser $L + \lambda \Omega$ revient à minimiser L sous la **contrainte** $\Omega(w) \leq t$, avec t qui décroît quand λ croît. Choisir un λ plus grand, c'est se limiter à un plus **petit domaine** de coefficients possibles : le modèle est moins riche, donc moins sujet au surapprentissage. Le domaine $\|w\|_2 \leq t$ est un **disque** ; le domaine $\|w\|_1 \leq t$ est un **losange** dont les sommets sont sur les axes. La solution est le point où les courbes de niveau de la perte touchent le domaine. Sur un disque, ce point est « n'importe où » ; sur un losange, il est très souvent à **un sommet**, c'est-à-dire avec **un coefficient exactement nul**. D'où la propriété du Lasso : il **sélectionne** des variables.

Un cas simple permet de voir le mécanisme sans algorithme : si les variables sont orthonormées et la perte quadratique, le Lasso **seuille** les coefficients estimés z_j : $\hat{w}_j = \text{signe}(z_j) (|z_j| - \lambda)_+$, alors que Ridge les **rétrécit** : $\hat{w}_j = z_j / (1 + \lambda)$. Avec $z = (3; 0,8; -0,3)$ et $\lambda = 0,5$, le Lasso donne $(2,5; 0,3; 0)$

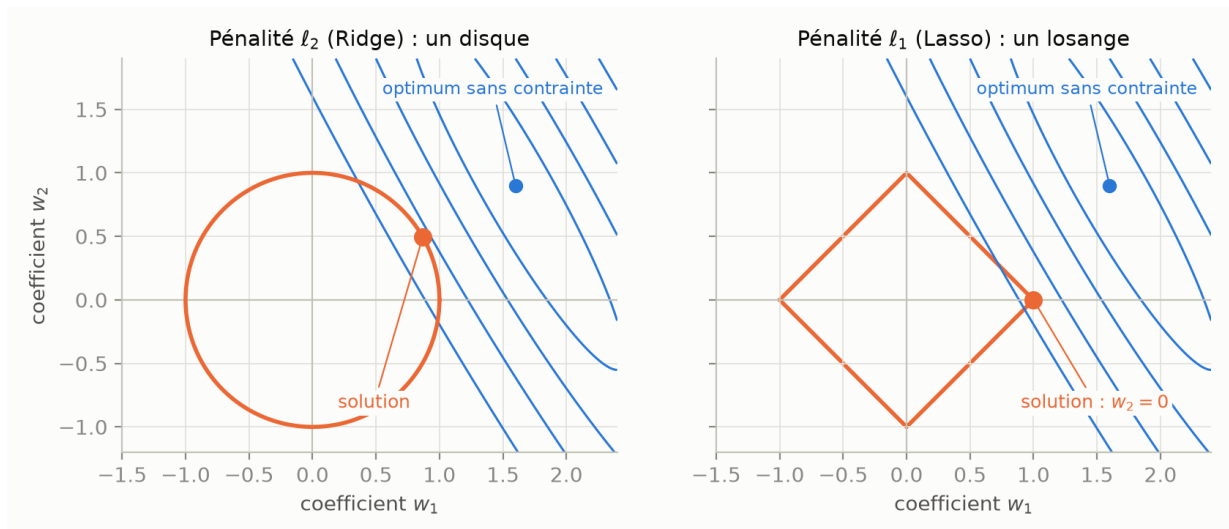


Figure 14: Courbes de niveau d'une perte (bleu) et domaine autorisé (orange) pour une pénalité ℓ_2 (disque) et ℓ_1 (losange). La solution du Lasso tombe sur un sommet du losange, où un coefficient est nul.

(la petite variable disparaît) et Ridge donne (2; 0,533; -0,2) (toutes survivent, rétrécies).

Sur la résiliation, voyons ce que fait le Lasso quand on fait varier la force de la pénalité. En `scikit-learn`, le réglage se nomme `C` et vaut l'**inverse** de la force : un petit `C` donne une forte pénalité.

C	AUC l1	AUC l2	nb coef. l1	nb coef. l2
0.001000	0.5000	0.8465	1.0	49.0
0.003162	0.8244	0.8512	5.0	49.0
0.010000	0.8487	0.8557	9.0	49.0
0.031623	0.8561	0.8587	12.0	49.0
0.100000	0.8587	0.8602	25.0	49.0
0.316228	0.8604	0.8605	40.0	49.0
1.000000	0.8604	0.8603	42.0	49.0
3.162278	0.8602	0.8602	43.0	49.0
10.000000	0.8602	0.8602	44.0	49.0

Lisez le tableau : avec une pénalité ℓ_1 très forte ($C = 0,001$), **un seul** coefficient sur 49 reste non nul et l'AUC en validation croisée tombe au hasard (0,5) ; en relâchant la pénalité, le nombre de coefficients non nuls remonte (44 sur 49 pour le `C` le plus grand testé), et l'AUC en validation croisée grimpe jusqu'à un plateau. Le meilleur réglage ici est une pénalité **l2** avec $C = 0,316$ (AUC de validation croisée 0,861). L'écart avec le modèle presque non pénalisé est minuscule : avec 9 000 clients et un nombre raisonnable de variables, la régularisation ne change pas grand-chose à la **précision** ; elle sert surtout à **stabiliser** les coefficients et, pour ℓ_1 , à **réduire** le modèle.

Voici le modèle de référence que nous garderons pour tout le chapitre : une régression logistique avec imputation médiane (et indicateurs de valeur manquante), standardisation et encodage des catégories, évaluée sur le jeu de test.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

logistique = make_pipeline(pre_lin, LogisticRegression(C=1.0, max_iter=1000))
logistique.fit(Xtr, ytr)
print(round(roc_auc_score(yte, logistique.predict_proba(Xte[:, 1]), 4))
```

0.8656

(Ici, `pre_lin` est le prétraitement décrit ci-dessus : imputation, standardisation, encodage.)

Avec une **AUC** de 0,866 sur le jeu de test (l'aire sous la courbe ROC : la probabilité qu'un client parti ait un score plus élevé qu'un client resté ; 0,5 = hasard, 1 = parfait ; chapitre 5, section 5.1), ce modèle de référence est déjà solide. Voyons maintenant ce qu'il **ne sait pas** faire.

2.1.6 Ce qu'un score linéaire ne sait pas écrire

Un score linéaire est une **somme** : chaque variable pousse le résultat dans un sens, indépendamment des autres, et proportionnellement à sa valeur. Il ne peut donc exprimer ni une **interaction** (« le risque explose quand *à la fois* la récence est grande *et* la satisfaction est basse ») ni un **seuil** (« rien ne se passe avant 150 jours, puis tout change »).

L'exemple d'école est le « ou exclusif » (XOR) : la classe dépend du **signe du produit** de deux variables.

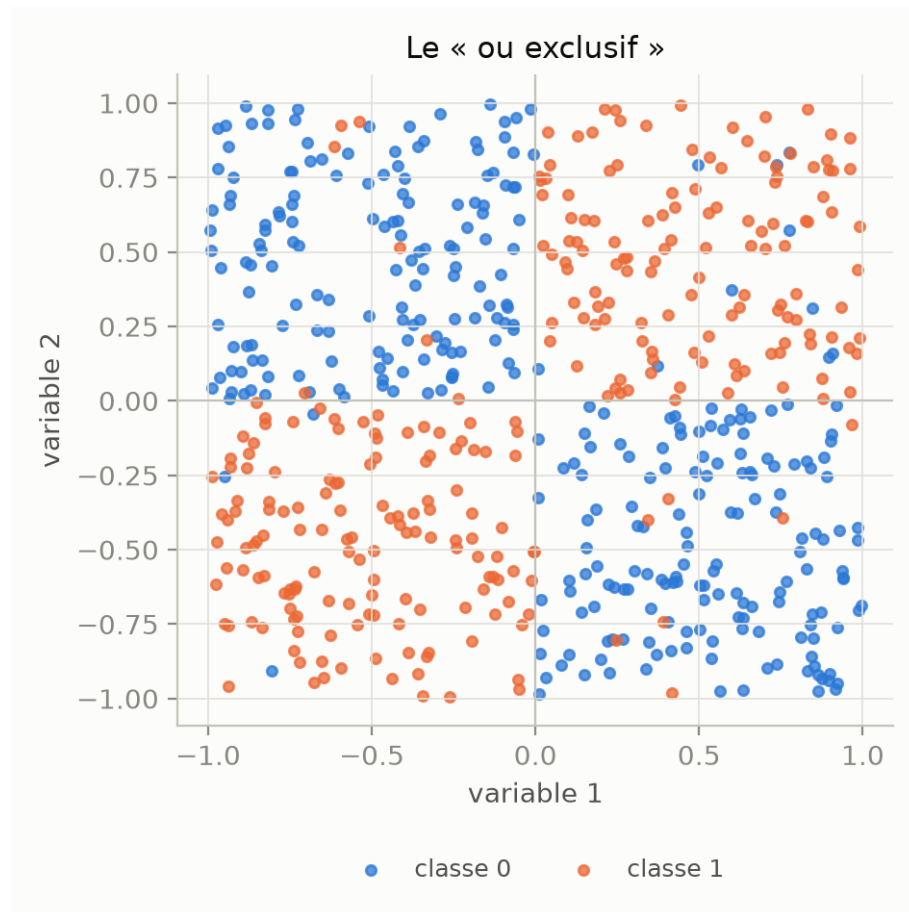


Figure 15: Deux classes disposées en damier : la classe dépend du signe du produit des deux variables. Aucune droite ne les sépare.

Une régression logistique sur ces deux variables obtient une AUC de 0,503 : **le hasard**. Aucune droite ne sépare un damier. Pourtant, si l'on **ajoute à la main** la variable « produit » x_1x_2 , l'AUC passe à 0,936. L'information était là ; c'est la **forme** de la règle qui manquait. Il existe deux réponses : fabriquer à la main les bonnes variables (les interactions, les seuils : c'est l'*ingénierie des variables* du chapitre 4), ou utiliser un modèle capable d'écrire des règles non linéaires **tout seul**. Les arbres sont le premier de ces modèles.

À retenir.

- Un modèle supervisé = **famille de fonctions + perte + algorithme**. Le modèle linéaire calcule un score $w^\top x + b$; la perte logistique (ou charnière) remplace le coût 0-1, non minimisable.
- Le gradient de la perte logistique est $\frac{1}{n} \sum (p_i - y_i)x_i$; la **descente de gradient** (et sa version stochastique, la SGD) le suit pas à pas. Elle exige des variables **à l'échelle** : le conditionnement du problème en dépend.
- **Pénalité = contrainte** : Ridge (disque) rétrécit, Lasso (losange) met des coefficients à zéro. C est l'inverse de la force de pénalisation.
- Un score linéaire **ne peut exprimer ni interaction ni seuil** ; sur la résiliation, il fournit pourtant une référence honorable (AUC 0,866).

Pour s'entraîner. Cahier, chapitre 2 : application 2.1 (descente de gradient écrite à la main) et application 2.2 (chemins de régularisation), exercices 2.1 à 2.3.

2.2 Arbres de décision

Intuition. Un arbre de décision, c'est le **questionnaire** que ferait un conseiller expérimenté : « Le client a-t-il commandé il y a plus de 140 jours ? Si oui, est-il satisfait ? Si non, rien à craindre. » Chaque question coupe l'ensemble des clients en deux groupes plus homogènes, et l'on recommence dans chaque groupe. L'arbre **apprend lui-même** les questions à poser, leur ordre et les seuils.

Les arbres ont trois qualités qui expliquent leur succès : ils sont **lisibles** (on peut tracer l'arbre et le raconter), ils n'exigent **ni mise à l'échelle ni imputation** pour fonctionner, et surtout ils écrivent des **règles non linéaires** : seuils, interactions, effets qui changent de sens selon la zone. Leur défaut, nous le verrons, est l'**instabilité**. C'est ce défaut que corrigeront les forêts (section 2.3) et le boosting (section 2.4).

2.2.1 Un arbre minuscule, entièrement à la main

Voici dix clients, décrits par la récence (jours depuis la dernière commande), la satisfaction (de 1 à 5) et le fait d'être partis dans les 90 jours.

Client	Récence	Satisfaction	Parti
1	15	4,5	0
2	30	3,0	0
3	45	4,0	0
4	70	4,1	0
5	95	3,1	0
6	120	4,4	0
7	160	3,0	1
8	200	4,3	0
9	260	2,9	1
10	320	3,8	1

Trois clients sur dix sont partis. Nous cherchons **la meilleure première question**, de la forme «

la variable j est-elle inférieure ou égale au seuil s ? ». Pour la comparer à d'autres, il nous faut une mesure de l'**homogénéité** d'un groupe.

2.2.2 Mesurer l'impureté

Dans un groupe où une proportion p de clients sont partis, on veut une mesure qui vaut 0 quand le groupe est **pur** ($p = 0$ ou $p = 1$) et qui est maximale quand il est le plus mélangé ($p = \frac{1}{2}$). Les trois mesures usuelles, pour deux classes, sont :

Mesure	Formule	Valeur maximale (en $p = \frac{1}{2}$)
Impureté de Gini	$G = 1 - p^2 - (1 - p)^2 = 2p(1 - p)$	0,5
Entropie (en bits)	$H = -p \log_2 p - (1 - p) \log_2 (1 - p)$	1
Taux d'erreur	$E = \min(p, 1 - p)$	0,5

L'impureté de Gini est celle qu'utilise `scikit-learn` par défaut. Elle s'interprète ainsi : c'est la probabilité de se tromper si l'on attribue au hasard à un client une étiquette tirée dans le groupe. L'entropie mesure l'information (en bits) qui manque pour connaître l'issue. En pratique, Gini et entropie donnent presque toujours des arbres très voisins ; le taux d'erreur, lui, est trop « plat » pour guider la construction.

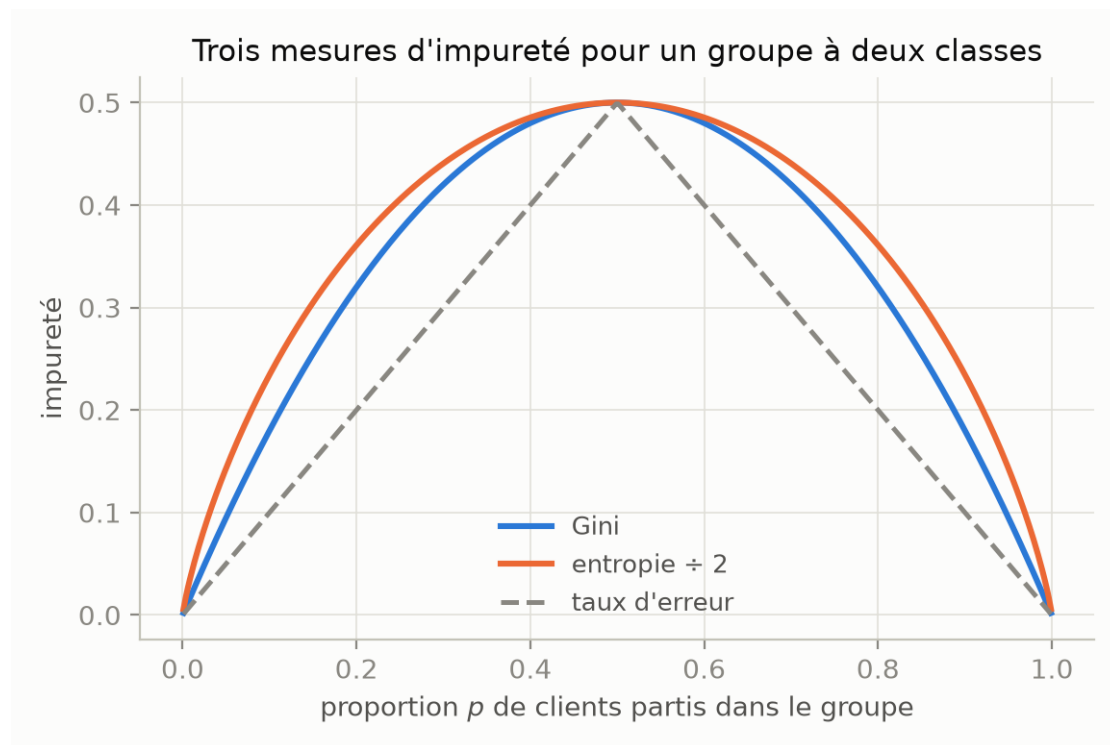


Figure 16: Impureté d'un groupe en fonction de la proportion de clients partis. Gini et entropie (divisée par deux pour tenir à l'échelle) ont la même allure en cloche ; le taux d'erreur est une tente à pointe.

Pour notre groupe de dix clients ($p = 0,3$), l'impureté de Gini vaut $1 - 0,3^2 - 0,7^2 = 1 - 0,09 - 0,49 = 0,42$ et l'entropie $-0,3 \log_2 0,3 - 0,7 \log_2 0,7 \approx 0,881$ bit.

2.2.3 Le meilleur découpage : un par un

Une question « $\text{récence} \leq s$? » coupe les clients en un groupe de gauche (de taille n_G) et un groupe de droite (n_D). Sa qualité est la **diminution d'impureté** :

$$\text{gain}(s) = G(\text{parent}) - \frac{n_G}{n} G(\text{gauche}) - \frac{n_D}{n} G(\text{droite}).$$

Il suffit d'essayer, comme seuils candidats, les **milieux** entre deux valeurs consécutives de la variable triée (au-dessus et au-dessous d'un tel seuil, les groupes sont les mêmes, quel que soit le seuil exact choisi dans l'intervalle). Faisons-le à la main pour le seuil $s = 140$ (entre 120 et 160) :

- groupe de gauche : les clients 1 à 6, tous restés ($p = 0$) : $G = 0$;
- groupe de droite : les clients 7 à 10, dont trois partis ($p = \frac{3}{4}$) : $G = 1 - \frac{9}{16} - \frac{1}{16} = 0,375$;
- impureté après découpage : $\frac{6}{10} \times 0 + \frac{4}{10} \times 0,375 = 0,15$; gain = $0,42 - 0,15 = 0,27$.

seuil	gauche (n, partis)	droite (n, partis)	impureté après	gain
22.5	1, 0	9, 3	0.4000	0.0200
37.5	2, 0	8, 3	0.3750	0.0450
57.5	3, 0	7, 3	0.3429	0.0771
82.5	4, 0	6, 3	0.3000	0.1200
107.5	5, 0	5, 3	0.2400	0.1800
140.0	6, 0	4, 3	0.1500	0.2700
180.0	7, 1	3, 2	0.3048	0.1152
230.0	8, 1	2, 2	0.1750	0.2450
290.0	9, 2	1, 1	0.3111	0.1089

Le tableau reprend ce calcul pour tous les seuils de récence. Le meilleur est bien $s = 140$, avec un gain de 0,27. Le meilleur seuil de satisfaction ne donne que 0,180 : la première question est donc « **récence ≤ 140 jours ?** ». À gauche, le groupe est pur : on s'arrête, la prédiction est « reste ». À droite, il reste quatre clients (7, 8, 9, 10) dont un est resté (le 8, très satisfait : 4,3) ; on recommence avec la satisfaction : le seuil $s = 4,05$ sépare les clients 7, 9 et 10 (satisfaction $\leq 4,05$, tous partis) du client 8 (resté). Les deux feuilles sont pures : l'arbre est terminé.

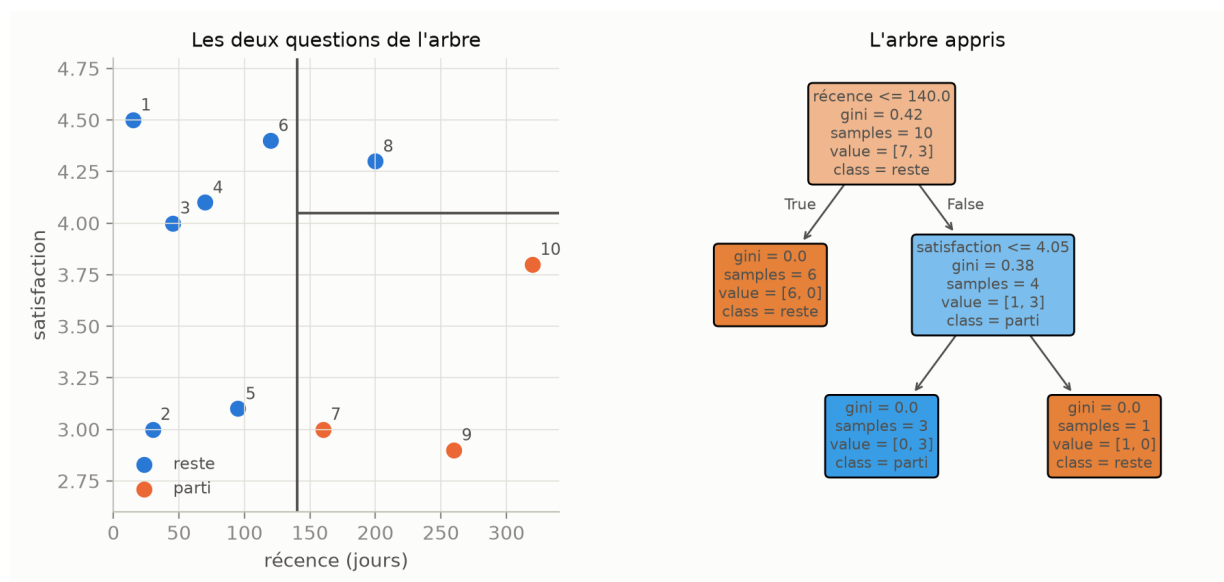


Figure 17: À gauche : les dix clients dans le plan (récence, satisfaction) et les deux coupes de l'arbre. À droite : l'arbre appris par scikit-learn, qui retrouve exactement les seuils du calcul à la main.

scikit-learn retrouve exactement ce calcul : première coupe à 140 jours, seconde à 4,05, 3 feuilles

pures. L'arbre peut se lire comme deux règles : « **si la récence dépasse 140 jours et la satisfaction est inférieure à 4,05, alors le client part** ; sinon il reste ». Remarquez que cette règle est une **interaction** : ni la récence seule ni la satisfaction seule ne suffit à prédire. Un score linéaire n'aurait pas pu l'écrire.

L'algorithme, en toutes lettres (CART). Pour construire un arbre à partir d'un jeu d'exemples : (1) pour **chaque variable** et **chaque seuil candidat**, calculer le gain d'impureté ; (2) retenir la coupe de **gain maximal** ; (3) recommencer **séparément** dans chacun des deux groupes ; (4) s'arrêter quand un critère d'arrêt est atteint (groupe pur, profondeur maximale, trop peu de clients). C'est un algorithme **glouton** : il choisit la meilleure coupe *maintenant*, sans se demander si une coupe moins bonne aujourd'hui permettrait de meilleures coupes demain. Il ne trouve donc pas forcément **le** meilleur arbre, mais il en trouve un bon très vite : en triant une fois chaque variable, évaluer tous les seuils d'une variable coûte de l'ordre de $n \log n$.

La **prédiction** d'une feuille est la classe majoritaire (ou la proportion de clients partis, utilisée comme probabilité) des exemples d'entraînement qui y tombent. Les variables catégorielles se traitent par des questions « la modalité est-elle dans cet ensemble ? » ou, dans `scikit-learn`, après encodage en indicatrices ; les valeurs manquantes sont gérées nativement par les versions récentes.

2.2.4 Les arbres de régression

Pour prédire une **quantité** (la dépense des six prochains mois, par exemple), la prédiction d'une feuille est la **moyenne** des valeurs de ses exemples, et le critère de coupe est la **réduction de la somme des carrés des écarts** (les écarts à la moyenne du groupe). Un exemple minuscule : six clients, x = nombre de commandes et y = dépense (en €).

x	1	2	3	4	5	6
y	10	14	12	40	44	42

La moyenne générale vaut 27, et la somme des carrés des écarts $(10 - 27)^2 + (14 - 27)^2 + \dots = 1366$. La coupe « $x \leq 3$ » donne deux feuilles : à gauche la moyenne est 12 (écarts : $-2, +2, 0$: somme des carrés 8), à droite la moyenne est 42 (écarts $-2, +2, 0$: somme des carrés 8). Après la coupe, la somme des carrés tombe de 1366 à 16 : le modèle prédit 12 pour les petits acheteurs et 42 pour les gros. C'est une fonction **en escalier**.

2.2.5 Jusqu'où laisser pousser l'arbre ?

Un arbre qu'on laisse pousser sans limite finit par isoler **chaque client** dans sa propre feuille : l'erreur d'entraînement tombe à zéro, mais l'arbre a appris le bruit. C'est l'exemple le plus pur de **surapprentissage** (volume III, section 1.3). On contrôle la complexité par des **hyperparamètres** : la profondeur maximale, le nombre minimal de clients par feuille, ou le nombre maximal de feuilles.

Voyons-le sur la résiliation : pour des profondeurs croissantes, on mesure l'AUC sur l'entraînement et, par validation croisée à cinq plis à **l'intérieur** de l'entraînement, sur des clients que l'arbre n'a pas vus.

La courbe d'entraînement (orange) monte sans cesse, jusqu'à 0,981 pour une profondeur de 14. La courbe de validation (bleue) monte jusqu'à une profondeur de 5 (AUC 0,863), puis **redescend** : au-delà, l'arbre mémorise. C'est la signature classique de l'arbitrage biais-variance. Un arbre trop

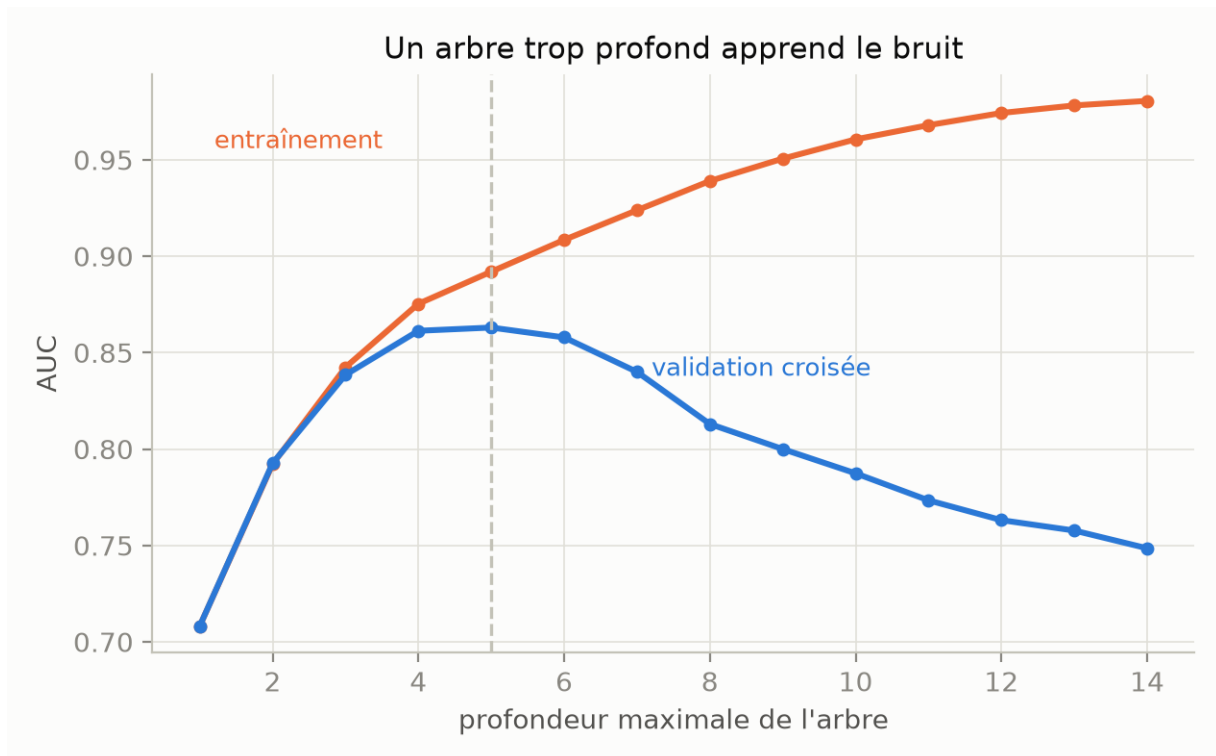


Figure 18: AUC d'un arbre de décision selon sa profondeur maximale : sur l'entraînement elle monte sans cesse, en validation croisée elle atteint un sommet puis redescend.

court (profondeur 2) est trop simple (AUC 0,793 en validation) ; un arbre trop profond (14) a trop de liberté (0,749). Sur le jeu de test, l'arbre de profondeur 5 obtient une AUC de **0,885**, contre 0,866 pour la régression logistique.

```
from sklearn.tree import DecisionTreeClassifier

arbre = DecisionTreeClassifier(max_depth=d_opt, min_samples_leaf=5, random_state=0)
arbre.fit(Xtr_o, ytr)
print(round(roc_auc_score(yte, arbre.predict_proba(Xte_o[:, 1]), 4))
```

```
0.8847
```

(Ici, `Xtr_o` est le tableau d'entraînement dont les catégories ont été transformées en indicatrices, et `d_opt` la profondeur retenue par la validation croisée.)

Une autre façon de limiter : élaguer. Plutôt que d'interdire à l'arbre de grandir, on le laisse pousser puis on **coupe les branches** qui apportent trop peu. L'élagage par **coût-complexité** minimise

$$R_{\alpha}(T) = R(T) + \alpha |T|,$$

où $R(T)$ est l'erreur de l'arbre T sur l'entraînement, $|T|$ son nombre de feuilles et $\alpha \geq 0$ le prix d'une feuille supplémentaire. Pour $\alpha = 0$, on garde tout ; quand α augmente, on supprime d'abord la branche dont la suppression coûte le moins par feuille retirée, et ainsi de suite jusqu'à la racine. On obtient une **suite d'arbres emboîtés**, et la validation croisée choisit α .

Ici, l'élagage retient $\alpha \approx 0,0006$: un arbre de 20 feuilles (au lieu de 458 pour l'arbre complet), d'AUC de validation 0,861 et d'AUC de test 0,873. Les deux méthodes (limiter la profondeur, élaguer) conduisent à des arbres comparables : retenons que **la complexité d'un arbre est un réglage à choisir par validation**, jamais à laisser au maximum.

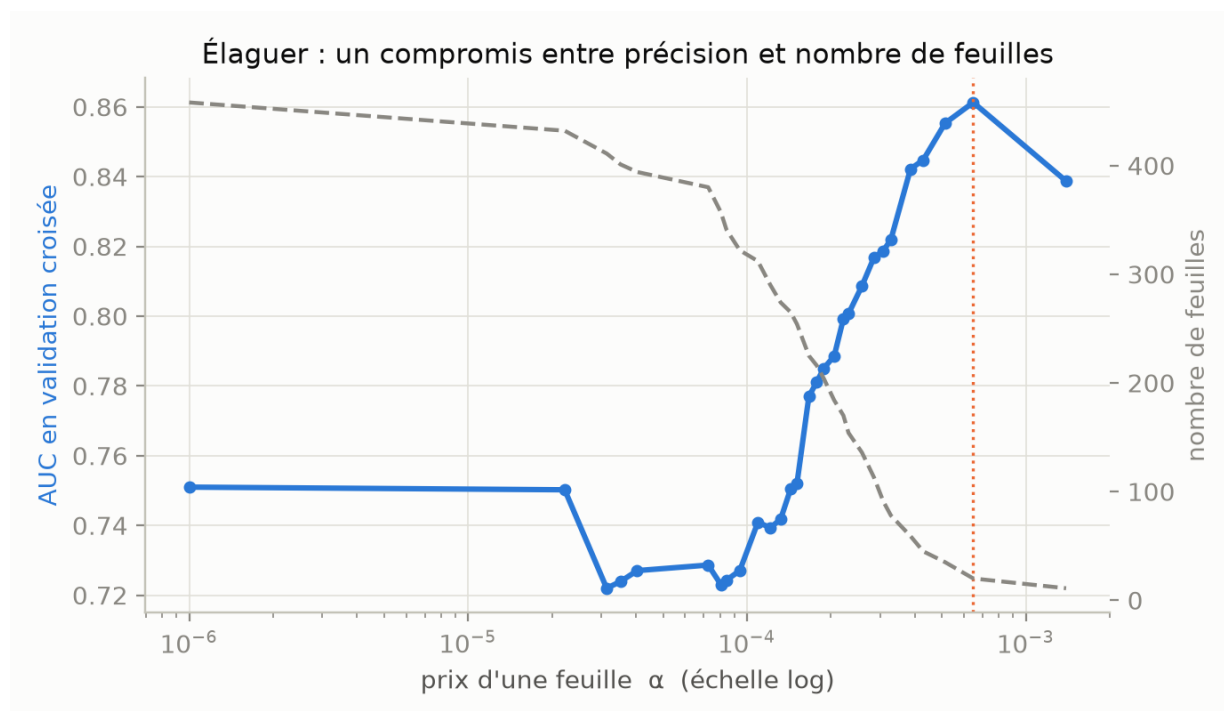


Figure 19: Élagage par coût-complexité : l'AUC en validation croisée (courbe bleue, axe de gauche) selon le prix α d'une feuille, et le nombre de feuilles de l'arbre (tirets gris, axe de droite). Le maximum (pointillé orange) correspond à un petit arbre d'une vingtaine de feuilles ; un arbre complet de 458 feuilles généralise moins bien.

2.2.6 Ce que les arbres savent écrire : effets non linéaires et interactions

Sur les données de résiliation, la régression logistique atteint 0,866 et l'arbre de profondeur 5 0,885 : un arbre **seul** dépasse déjà un peu le modèle linéaire. D'où vient l'écart ? Il ne vient pas d'une grande découverte, mais de **plusieurs petites formes** que le score linéaire ne sait pas écrire. Voyons les deux plus parlantes.

Un effet qui sature : le nombre de commandes. Un score linéaire attribue à une variable un effet qui va **toujours dans le même sens et au même rythme** : chaque commande supplémentaire retranche la même quantité de log-cote, que l'on passe de 0 à 1 commande ou de 11 à 12. Or le risque de départ ne se comporte pas ainsi : il est très élevé pour les clients qui n'ont **rien commandé** en douze mois, il chute dès les premières commandes, puis **se stabilise** : au-delà d'une demi-douzaine de commandes, une de plus ne change plus rien.

Le taux de départ observé est de 42 % pour les clients sans commande, de 15 % pour ceux qui en ont une ou deux, de 2 % à partir de six. Avec cette seule variable, la régression logistique atteint une AUC de test de 0,766 et l'arbre de profondeur 3 de 0,762 : la différence est faible, mais l'arbre dessine ce que montrent les données (un saut, puis un plateau), alors que la courbe logistique continue à descendre. Avec **deux** variables (le nombre de commandes et l'âge), les cartes de droite montrent la différence de **forme** : la régression logistique ne sait tracer que des bandes obliques, l'arbre découpe des rectangles, et isole notamment la bande horizontale des clients sans commande. L'AUC passe à 0,814 pour la première et à 0,831 pour le second.

Une interaction : deux conditions à la fois. La récence et la satisfaction illustrent l'autre limite. Regardons le taux de départ observé selon que la récence dépasse 150 jours et que la satisfaction est inférieure à 3,2 (en laissant de côté les clients dont la satisfaction manque), puis comparons-le à ce que prédit un modèle **additif** (logistique) sur ces deux variables.

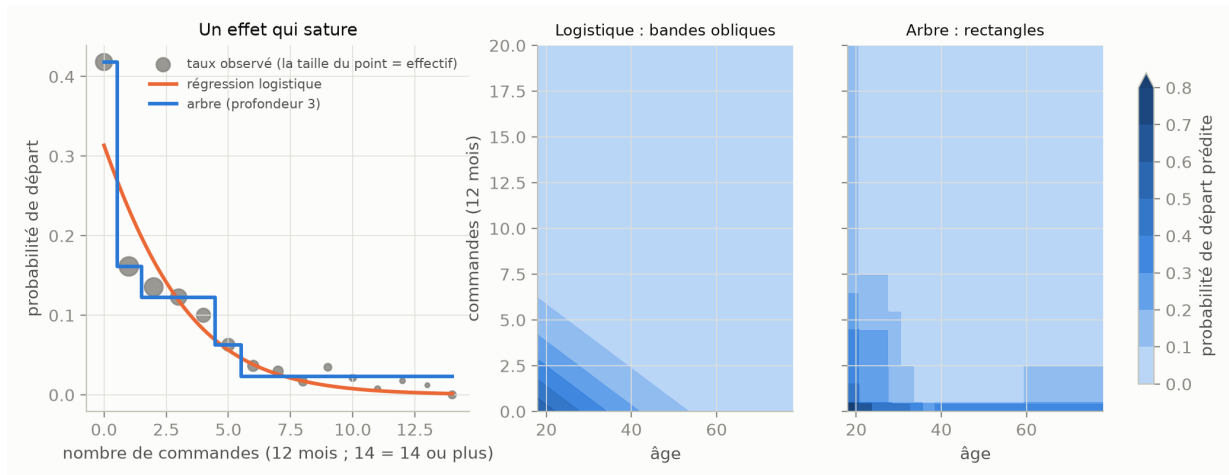


Figure 20: À gauche : le taux de départ observé selon le nombre de commandes (points gris), la prédiction d'une régression logistique (une courbe lisse qui ne sature pas) et celle d'un arbre (un escalier qui épouse le saut à 0 commande et le plateau). Au centre et à droite : probabilité de départ prédite en fonction de l'âge et du nombre de commandes, par une régression logistique (bandes obliques) et par un arbre (rectangles).

	récence	satisfaction	clients	observé	additif
> 150 jours	< 3,2		562	0.717	0.507
> 150 jours	≥ 3,2		1350	0.217	0.291
≤ 150 jours	< 3,2		1063	0.121	0.159
≤ 150 jours	≥ 3,2		4888	0.064	0.060

Dans le coin « récence > 150 jours *et* satisfaction < 3,2 » (562 clients d'entraînement), le taux de départ observé est de 72 %. Pour les clients qui ne remplissent aucune des deux conditions, il est de 6 % ; chaque condition **seule** le fait monter à 12 % (satisfaction basse) ou 22 % (récence élevée) ; les deux **ensemble** le multiplient par 11. Or un modèle additif ne peut qu'**ajouter** les deux effets (sur l'échelle de la log-cote) : il prédit 51 % pour le coin, soit vingt points de moins que la réalité, et surestime les deux cases voisines (29 % et 16 % prédits, contre 22 % et 12 % observés). Un arbre écrit cela avec deux questions.

La vérité programmée. Ces données sont simulées : nous pouvons donc révéler ce qui les produit. Le risque de départ contient bien une forte hausse quand la récence dépasse **150 jours** *et* la satisfaction est inférieure à **3,2** (un terme d'interaction), un effet du nombre de commandes qui **sature** à six, un surcroît de risque pour les clients anciens sans aucune commande, et d'autres interactions (trois tickets au support et beaucoup de retours ; beaucoup de promotions sans programme de fidélité). Le modèle linéaire en capte la part régulière ; les arbres captent aussi ces formes irrégulières, et la somme de ces petits gains explique l'écart global entre 0,866 et 0,885.

2.2.7 Le point faible : l'instabilité

Si les arbres sont aussi pratiques, pourquoi ne s'arrête-t-on pas là ? Parce qu'un arbre est **instable** : une petite modification des données peut changer complètement la première question, donc tout ce qui suit. L'algorithme étant glouton, une coupe presque aussi bonne que la meilleure, mais portant sur une autre variable, change toute la suite de l'arbre.

Mesurons-le sur un jeu modeste : on tire 30 sous-échantillons de **2 000 clients** dans le jeu d'entraînement, on ajuste sur chacun un arbre de profondeur 4 et l'on regarde **quelle variable**

ouvre l'arbre. (Avec les 9 000 clients complets, la première variable serait presque toujours la même ; c'est le **seuil** et la **suite** qui bougent, comme on le voit plus bas.)

Sur 30 sous-échantillons de 2 000 clients, 4 variables différentes ouvrent l'arbre : `recence_jours` dans 19 cas, `age` dans 5 cas, et les autres variables dans les cas restants. Et même quand la variable est la même, le seuil change : avec les 9 000 clients, 1 seule variable ouvre les 30 arbres bootstrap, mais le seuil de la première coupe va de 156 à 312 jours. Les prédictions de deux arbres de ce type ne sont corrélées qu'à 0,78 en moyenne sur le jeu de test : **chaque arbre raconte une histoire différente**. En termes du chapitre 1, un arbre profond a un **biais** faible et une **variance** élevée. Or il existe un remède universel contre la variance : **moyenner**. C'est le sujet de la section suivante.

À retenir.

- Un arbre pose une suite de questions « variable $\leq$ seuil ? » ; il est construit **gloutonnement**, en maximisant à chaque étape la **diminution d'impureté** (Gini ou entropie en classification, somme des carrés en régression).
- Il capture **seuils et interactions** et n'exige ni standardisation ni modèle de la loi des données ; ses règles sont lisibles.
- Sa complexité (profondeur, taille minimale des feuilles, élagage par coût-complexité $R(T) + \alpha|T|$) est un **hyperparamètre à régler par validation croisée**.
- Il est **instable** : forte variance, d'où les forêts (2.3) et le boosting (2.4).

Pour s'entraîner. Cahier, chapitre 2 : application 2.3 (construire un arbre à la main puis avec `scikit-learn`, élaguer), exercices 2.4 à 2.6.

2.3 Forêts aléatoires

Intuition. Demandez à un seul expert d'estimer le poids d'un bœuf : il se trompera, dans un sens ou dans l'autre. Demandez à cent experts **indépendants** et faites la moyenne : les erreurs de signes opposés se compensent, et l'on tombe très près de la vérité. Une forêt aléatoire est cette foule : des centaines d'arbres, chacun entraîné sur une version un peu différente des données, dont on moyenne les prédictions. Chaque arbre est instable (section 2.2.7) ; **la moyenne ne l'est presque plus**.

2.3.1 Le bagging : moyenner des arbres entraînés sur des échantillons bootstrap

L'idée porte le nom de **bagging** (*bootstrap aggregating*, Breiman, 1996). Pour construire B arbres, on répète B fois : (1) tirer un **échantillon bootstrap** : n clients tirés **avec remise** dans le jeu d'entraînement (certains clients apparaissent plusieurs fois, d'autres jamais) ; (2) ajuster un arbre profond sur cet échantillon. Pour prédire, on moyenne les B probabilités prédites (ou on prend le vote majoritaire).

Combien de clients distincts un échantillon bootstrap contient-il ? Un client donné n'est **pas** tiré à un tirage donné avec la probabilité $1 - \frac{1}{n}$, donc il n'est tiré **jamais** sur les n tirages avec la probabilité $(1 - \frac{1}{n})^n \rightarrow e^{-1} \approx 0,368$. Un échantillon bootstrap contient donc, en moyenne, **63,2 %** des clients distincts, et environ **36,8 %** des clients sont laissés de côté. Ces clients « hors sac » nous serviront en 2.3.4.

Vérification sur notre jeu d'entraînement : un échantillon bootstrap contient 63,4 % de clients distincts, pour une valeur théorique de 63,2 %.

2.3.2 Pourquoi moyenner réduit la variance : la formule

Soient B prédictions $T_1, \dots, T_B$ (celles de B arbres, en un point x fixé), de même variance σ^2 et de **corrélation** deux à deux ρ . Leur moyenne a pour variance

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B T_b\right) = \frac{1}{B^2} \left[\underbrace{B\sigma^2}_{\text{variances}} + \underbrace{B(B-1)\rho\sigma^2}_{\text{covariances}} \right] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

Lecture de la formule. Le second terme, $\frac{1-\rho}{B}\sigma^2$, **disparaît quand B augmente** : c'est la part de variance que la moyenne élimine. Le premier, $\rho\sigma^2$, **reste** : si les arbres sont corrélés, moyenner à l'infini n'enlève pas leur erreur commune. Deux cas extrêmes : si les arbres étaient **indépendants** ($\rho = 0$), la variance serait divisée par B ; s'ils étaient **identiques** ($\rho = 1$), la moyenne ne servirait à rien, la variance resterait σ^2 . Un autre point mérite d'être noté : la moyenne ne change pas le **biais** (c'est la moyenne des biais), c'est pourquoi on moyenne des arbres **profonds**, au biais faible et à la variance forte.

Un exemple chiffré avec $\sigma^2 = 1$ et $\rho = 0,3$: pour $B = 1$ la variance vaut 1 ; pour $B = 10$, $0,3 + 0,07 = 0,37$; pour $B = 100$, $0,3 + 0,007 = 0,307$; pour $B \rightarrow \infty$, $0,3$. On a éliminé presque toute la variance « évitable » avec une centaine d'arbres, mais jamais les 30 % dus à la corrélation.

Voyons ce que cela donne avec de vrais arbres. On tire 12 sous-échantillons de 4 500 clients dans le jeu d'entraînement (12 « jeux de données » différents) ; sur chacun on ajuste 30 arbres bootstrap. Pour 800 clients du jeu de test, on mesure la variance de la prédiction d'un seul arbre, puis de la moyenne de 5, 15, 30 arbres, **d'un jeu de données à l'autre**.

arbres moyennés	variance mesurée	formule $\rho\sigma^2 + (1-\rho)\sigma^2/B$
1	0.03060	0.03271
5	0.00709	0.00774
15	0.00340	0.00358
30	0.00253	0.00254

La variance d'un seul arbre est 0,0327 ; la corrélation estimée entre deux arbres du même jeu de données est 0,05, ce qui fixe un plancher $\rho\sigma^2 \approx 0,0015$. La corrélation est modeste en valeur absolue parce qu'une grande part de la variance d'un arbre profond vient de l'aléa du bootstrap lui-même, que la moyenne élimine ; mais c'est la part restante qui compte quand B est grand. En moyennant 30 arbres, la variance mesurée tombe à 0,0025, soit une division par environ 12,9 ; la colonne de droite montre que la formule, alimentée par ces deux estimations, retrouve les variances mesurées (à l'échantillonnage près : seulement 12 jeux de données).

2.3.3 La forêt aléatoire : décorréler les arbres

Le bagging a un défaut : si une variable est très prédictive (la récence, ici), **tous** les arbres l'utilisent en première coupe, donc se ressemblent : ρ reste élevé. Les **forêts aléatoires** (*random forests*, Breiman, 2001) ajoutent une idée simple : à **chaque coupe**, l'arbre ne considère qu'un **sous-ensemble tiré au hasard** de m variables parmi p (par défaut $m = \sqrt{p}$ en classification). Une variable dominante n'est plus disponible à toutes les coupes ; les arbres sont forcés d'explorer d'autres pistes, donc de se **décorréliser**. Chaque arbre individuel est un peu moins bon (il a moins de choix), mais la moyenne l'est davantage, parce que ρ a baissé.

Sur la même expérience, mais avec $m = \sqrt{p}$ variables tirées à chaque coupe, la corrélation entre deux arbres passe de 0,046 à 0,039 (plancher : de 0,0015 à 0,0011), et la variance de la moyenne de 30 arbres tombe de 0,0025 à 0,0011, soit une division par plus de deux, alors que chaque arbre

pris seul a une variance comparable (0,0273 contre 0,0327). Décorrélér fait gagner plus que ce que fait perdre l'appauvrissement de chaque arbre.

2.3.4 L'erreur « hors sac » : une validation gratuite

Chaque arbre n'a vu que 63,2 % des clients. Pour chaque client i , on peut donc faire voter **uniquement les arbres qui ne l'ont jamais vu**, soit environ 37 % de la forêt : on obtient une prédiction **honnête** pour ce client, comme en validation croisée, mais sans entraîner un seul modèle supplémentaire. C'est l'estimation **hors sac** (*out-of-bag*, OOB). On s'en sert pour régler les hyperparamètres d'une forêt à coût nul.

```
from sklearn.ensemble import RandomForestClassifier

foret = RandomForestClassifier(n_estimators=300, min_samples_leaf=5, max_features="sqrt",
                              oob_score=True, n_jobs=1, random_state=0)

foret.fit(Xtr_o, ytr)
print(round(roc_auc_score(yte, foret.predict_proba(Xte_o)[: , 1]), 4))

0.8964
```

Une forêt de 300 arbres obtient une AUC de **0,896** sur le jeu de test, contre 0,885 pour l'arbre unique de profondeur 5 et 0,866 pour la régression logistique. L'estimation hors sac, calculée **sans utiliser le jeu de test**, donne 0,888 : elle est proche de la valeur de test, avec un léger pessimisme (chaque client n'est évalué que par environ 37 % des arbres, donc par une sous-forêt plus petite).

2.3.5 Combien d'arbres ? Quels réglages ?

Une particularité des forêts : **ajouter des arbres ne fait pas surapprendre**. La performance monte puis se stabilise, parce que l'erreur de la moyenne converge vers une limite quand $B \rightarrow \infty$ (c'est la loi des grands nombres appliquée aux arbres ; le terme $\rho\sigma^2$ de la formule est un plancher, pas un risque). Il suffit donc de prendre « assez » d'arbres : plus, c'est seulement plus lent.

Avec un seul arbre de la forêt, l'AUC est de 0,752 ; avec 25 arbres, de 0,891 ; avec 300, de 0,896. Au-delà de quelques dizaines d'arbres, le gain devient marginal.

Les réglages qui comptent vraiment sont ailleurs : la **profondeur** ou la **taille minimale des feuilles** (le compromis biais-variance de chaque arbre), et surtout **max_features**, la fraction de variables examinées à chaque coupe. En voici l'effet, mesuré par l'erreur hors sac (donc sans toucher au jeu de test) :

max_features (fraction)	AUC hors sac	AUC test
0.05	0.8701	0.8814
0.10	0.8821	0.8925
0.20	0.8848	0.8967
0.40	0.8852	0.8948
0.70	0.8854	0.8945
1.00	0.8826	0.8942

L'AUC hors sac est maximale pour une fraction de 0,70 des variables (0,885), contre 0,883 quand on les examine toutes (c'est du bagging pur) et 0,870 quand on n'en examine presque aucune. Trop peu de variables : chaque arbre est trop pauvre ; trop : les arbres se ressemblent. Le sommet est entre les deux, et il est peu marqué : les forêts sont, parmi tous les modèles, **les plus tolérantes aux mauvais réglages**.

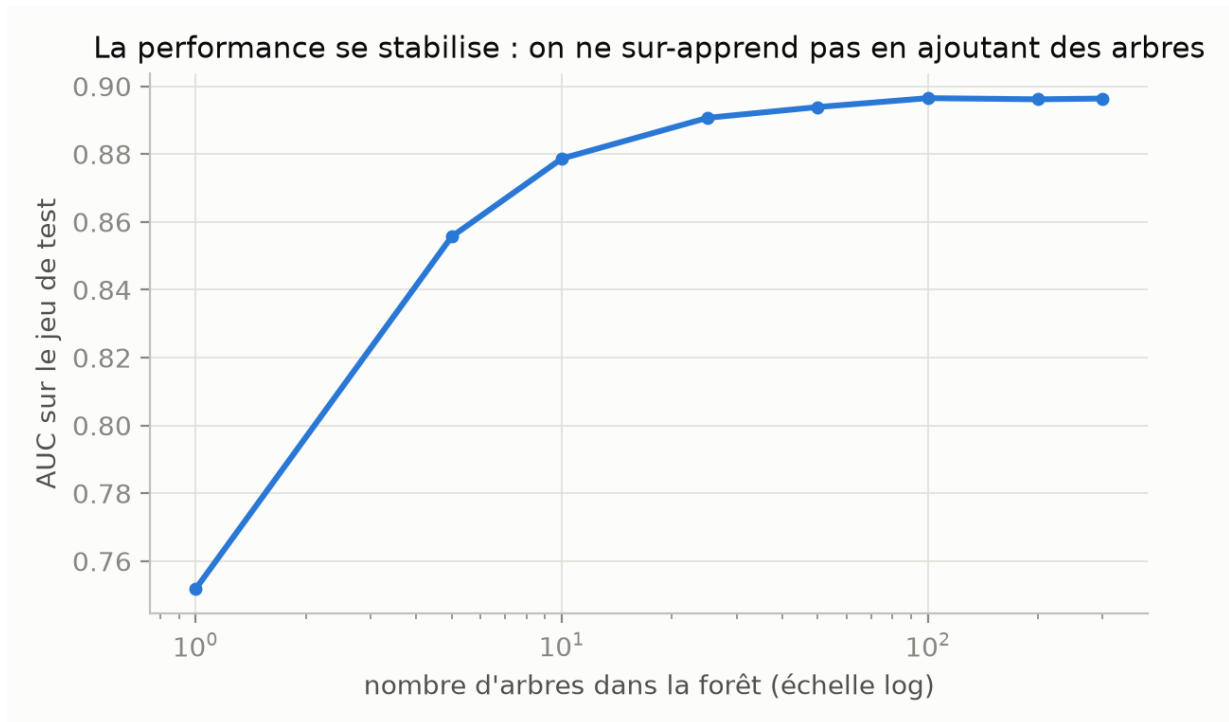


Figure 21: AUC d'une forêt sur le jeu de test selon le nombre d'arbres : elle monte vite puis forme un plateau.

2.3.6 Quelles variables comptent ? Importance et ses pièges

Une forêt ne se lit pas comme un arbre, mais on peut lui demander **quelles variables elle utilise le plus**. Deux mesures s'opposent.

- L'**importance par impureté** (*mean decrease in impurity*, MDI) additionne, pour chaque variable, les diminutions d'impureté de toutes les coupes où elle intervient. Elle est gratuite (calculée pendant l'entraînement), mais elle est **biaisée** : elle favorise les variables à **beaucoup de valeurs distinctes** (une variable continue offre beaucoup plus de seuils possibles, donc beaucoup plus d'occasions de trouver une coupe qui améliore *par hasard* l'impureté de l'entraînement) et elle est calculée **sur l'entraînement**, donc elle récompense aussi ce que la forêt a mémorisé.
- L'**importance par permutation** mélange au hasard les valeurs d'une variable sur des données **non vues** (le jeu de test) et mesure la **baisse de performance** : si le modèle s'effondre, la variable compte ; si rien ne change, elle ne compte pas.

Pour voir le biais, on ajoute au jeu deux variables **de pur bruit** : l'une continue (tirée dans une loi normale), l'autre un « identifiant » à 2 000 valeurs entières. Elles ne contiennent aucune information sur le départ.

Parmi les 47 colonnes, l'importance par impureté classe la variable de bruit continue au rang **11** et l'identifiant aléatoire au rang **12**, c'est-à-dire parmi les premières colonnes, devant des variables qui portent une vraie information : l'identifiant, sans aucune information, reçoit une importance de 0,0358. L'importance par permutation, elle, donne $-0,0005$ et $0,0001$ aux deux variables de bruit : **zéro, ou à peu près**, ce qui est la bonne réponse.

Deux précautions. (1) Ne vous fiez jamais à l'importance par impureté pour comparer des variables de types différents (continues contre catégories à peu de modalités) : préférez

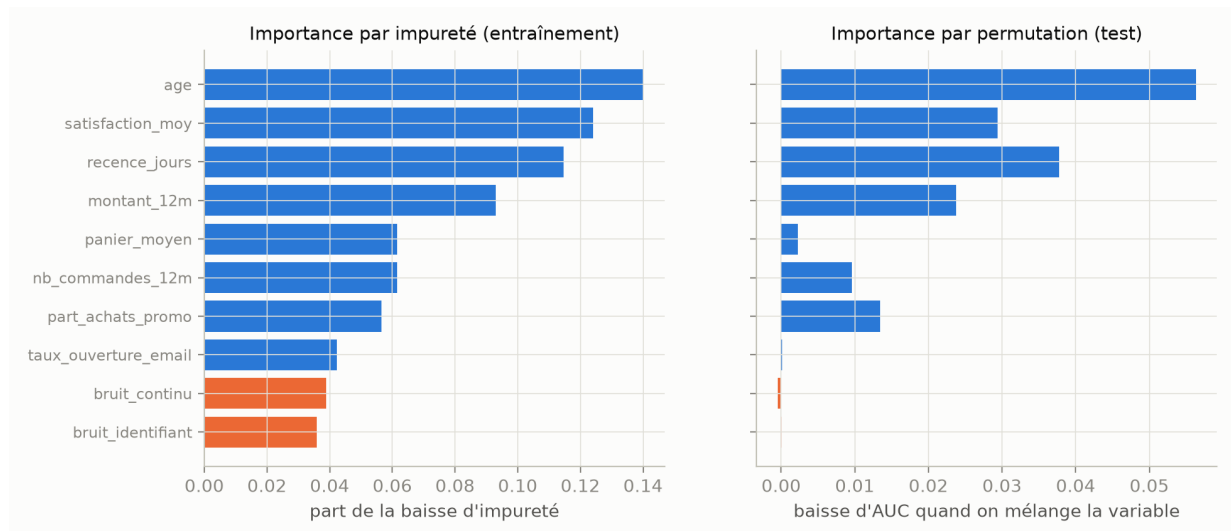


Figure 22: Les huit variables les plus importantes selon l'impureté et les deux variables de pur bruit (en orange) ajoutées au tableau, avec deux mesures d'importance. L'importance par impureté (à gauche) accorde de l'importance au bruit ; l'importance par permutation sur le jeu de test (à droite) la ramène à zéro.

l'importance par permutation, calculée sur des données de test. (2) Quand deux variables sont **fortement corrélées**, la forêt répartit l'importance entre elles (et la permutation en sous-estime chacune, car l'autre « compense ») : l'importance dit *ce que le modèle utilise, pas ce qui cause le phénomène*. La section 5.3 reprendra l'interprétation de façon plus complète (valeurs de Shapley, PDP).

À retenir.

- Le **bagging** moyenne des arbres profonds entraînés sur des échantillons bootstrap ; la variance de la moyenne vaut $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$: elle baisse avec B mais plafonne à $\rho\sigma^2$.
- La **forêt aléatoire** diminue ρ en ne laissant, à chaque coupe, qu'un sous-ensemble de variables : les arbres se ressemblent moins, la moyenne est meilleure.
- L'**erreur hors sac** (63,2 % / 36,8 % des clients) est une validation gratuite. **Ajouter des arbres ne fait pas surapprendre.**
- Les réglages qui comptent : profondeur ou taille des feuilles, `max_features`. Une forêt est robuste et exige peu de réglages.
- L'importance par **impureté** est biaisée (variables à nombreuses valeurs) ; préférez celle par **permutation**, mesurée sur des données non vues.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.4 (bagging écrit à la main) et 2.5 (importances et variables de bruit), exercices 2.7 à 2.8.

2.4 Gradient boosting : XGBoost et LightGBM

Intuition. Une forêt réunit des arbres **indépendants** et les moyenne. Le **boosting** fait l'inverse : il construit les arbres **un par un**, chacun étant chargé de **corriger les erreurs** de l'ensemble construit jusque-là. On commence par une prédiction grossière (la moyenne), on regarde où l'on se trompe, on entraîne un petit arbre à prédire *ces erreurs*, on l'ajoute (en le pondérant prudemment), et l'on recommence. Les forêts réduisent la **variance** ; le boosting réduit surtout le **biais**. Sur les données tabulaires, c'est aujourd'hui la famille de modèles la plus efficace.

2.4.1 Un exemple à la main : deux tours de boosting

Six clients, une variable x (le nombre de commandes) et une dépense y (en €).

x	1	2	3	4	5	6
y	3	5	4	12	20	13

Tour 0. Le modèle le plus simple prédit la **moyenne** : $F_0(x) = \frac{3+5+4+12+20+13}{6} = 9,5$. Les **résidus** (réalité moins prédiction) sont $r^{(1)} = y - F_0 = (-6,5; -4,5; -5,5; 2,5; 10,5; 3,5)$.

Tour 1. On ajuste un tout petit arbre (une seule coupe, une « souche ») pour **prédire les résidus**. La meilleure coupe est « $x \leq 3$ » : la moyenne des résidus à gauche vaut $-5,5$, à droite $+5,5$. Au lieu d'ajouter toute la correction, on n'ajoute que la moitié (le **pas d'apprentissage** vaut $\nu = 0,5$) : $F_1 = F_0 + 0,5 \times \text{souche}_1$. À gauche, $F_1 = 9,5 - 2,75 = 6,75$; à droite, $F_1 = 9,5 + 2,75 = 12,25$.

Tour 2. On recalcule les résidus $r^{(2)} = y - F_1$ et l'on ajuste une nouvelle souche. Cette fois, la meilleure coupe est « $x \leq 4$ » : elle isole les grosses dépenses des clients 5 et 6. Après ces deux tours, la prédiction vaut 5,6875 pour les clients 1 à 3, 11,1875 pour le client 4 et 14,375 pour les clients 5 et 6. Et ainsi de suite.

x	y	F_0	résidu 1	F_1	résidu 2	F_2
1	3.0	9.5	-6.5	6.75	-3.75	5.688
2	5.0	9.5	-4.5	6.75	-1.75	5.688
3	4.0	9.5	-5.5	6.75	-2.75	5.688
4	12.0	9.5	2.5	12.25	-0.25	11.188
5	20.0	9.5	10.5	12.25	7.75	14.375
6	13.0	9.5	3.5	12.25	0.75	14.375

Le tableau et la figure montrent le mécanisme : la somme des carrés des erreurs passe de 221,5 (tour 0) à 85,4 (tour 1), puis à 44,7 (tour 2). Chaque tour réduit ce qui reste d'erreur, et le « pas » ν empêche de corriger d'un coup, ce qui évite de s'ajuster au bruit. Après 100 tours, le modèle reproduit les données de façon quasi exacte : le boosting a un biais qui tend vers zéro, et c'est précisément pourquoi il faut **l'arrêter à temps**.

2.4.2 Le gradient boosting : une descente de gradient dans l'espace des fonctions

L'exemple semble un truc de bricoleur : « ajuster les résidus ». Friedman (2001) a montré qu'il s'agit en réalité d'une **descente de gradient**, non sur des paramètres, mais **sur la fonction elle-même**. Cela permet de traiter n'importe quelle perte, pas seulement le carré.

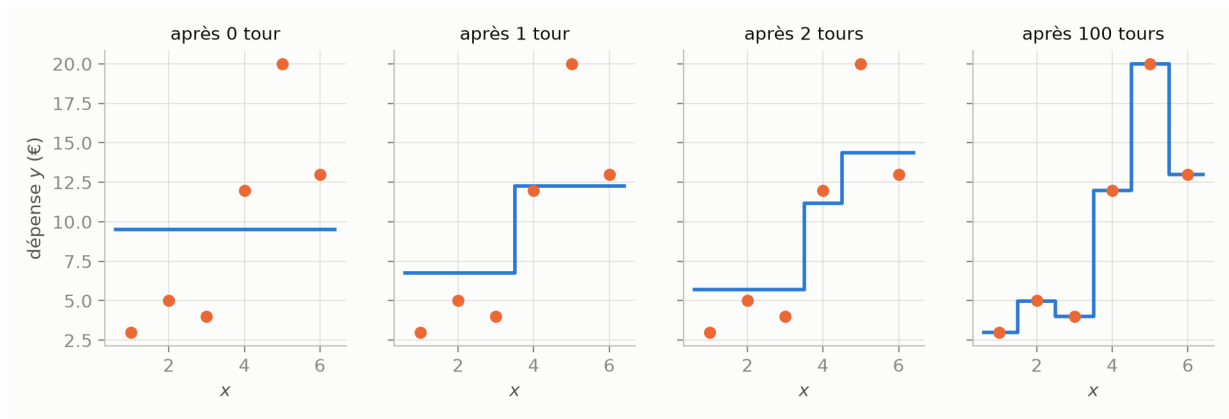


Figure 23: Les six clients (points orange) et la prédiction du modèle de boosting (courbe bleue) après 0, 1, 2 et 100 tours. À chaque tour, la fonction en escalier se rapproche des données.

La dérivation. On cherche une fonction F qui minimise la perte totale $L(F) = \sum_{i=1}^n \ell(y_i, F(x_i))$. Considérons les valeurs $F(x_1), \dots, F(x_n)$ comme n paramètres. Une descente de gradient consisterait à les déplacer dans la direction opposée au gradient :

$$F(x_i) \leftarrow F(x_i) - \nu \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)}.$$

Le problème : on ne veut pas modifier seulement les n valeurs observées, mais obtenir une fonction qui **généralise**. On ajuste donc un petit arbre h pour **approcher** les n valeurs $r_i = -\partial \ell / \partial F(x_i)$ (les « pseudo-résidus »), puis on ajoute νh à F . L'algorithme est donc :

1. $F_0 = \arg \min_c \sum_i \ell(y_i, c)$ (une constante) ;
2. pour $m = 1, \dots, M$: calculer les pseudo-résidus $r_i^{(m)} = -\left[\frac{\partial \ell(y_i, F)}{\partial F} \right]_{F=F_{m-1}(x_i)}$; ajuster un arbre h_m aux $r_i^{(m)}$; poser $F_m = F_{m-1} + \nu h_m$.

Perte quadratique $\ell = \frac{1}{2}(y - F)^2$: $r_i = y_i - F(x_i)$: le pseudo-résidu est le **résidu ordinaire**. C'est le cas de l'exemple. **Perte logistique** (classification), avec F la **log-cote** et $p = \sigma(F)$: on a montré (section 2.1.2) que $\partial \ell / \partial F = p - y$, donc $r_i = y_i - p_i$: chaque arbre apprend l'écart entre la réalité et la probabilité actuellement prédite. Changer la perte (valeur absolue, perte de quantile, perte de Poisson...) ne change **rien** à l'algorithme.

Quelques réglages découlent de cette vision. Le pas ν (*learning rate*) joue le rôle du pas de la descente de gradient : petit, il est prudent mais exige beaucoup d'arbres. La **profondeur** des arbres h_m fixe l'**ordre des interactions** que le modèle peut représenter : avec des souches (profondeur 1), le modèle final est une somme de fonctions d'**une seule variable** ; avec une profondeur 3, il peut combiner jusqu'à trois variables dans un même terme. Enfin, comme pour les forêts, on peut n'utiliser qu'une **fraction aléatoire** des clients (*subsample*) et des variables (*colsample*) à chaque tour : le **boosting stochastique** réduit la variance et accélère le calcul.

2.4.3 XGBoost : un objectif régularisé et une formule fermée

XGBoost (Chen et Guestrin, 2016) reprend ce schéma en lui apportant trois améliorations de fond. D'abord un objectif **régularisé** : la perte plus une pénalité sur la complexité de l'arbre, $\Omega(h) = \gamma T + \frac{1}{2}\lambda \sum_j w_j^2$ (T feuilles, w_j valeur de la feuille j). Ensuite un développement **au second ordre** de la perte : avec $g_i = \partial \ell / \partial F$ et $h_i = \partial^2 \ell / \partial F^2$ calculés au tour précédent, le coût d'un arbre se réécrit

$$\tilde{\mathcal{L}} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T, \quad G_j = \sum_{i \in \text{feuille } j} g_i, \quad H_j = \sum_{i \in \text{feuille } j} h_i.$$

Les formules à retenir. C'est un trinôme en w_j , minimal en

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \text{de valeur} \quad -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}.$$

La **qualité d'une coupe** (qui sépare un nœud en gauche et droite) est donc la diminution de coût obtenue :

$$\text{gain} = \frac{1}{2} \left[\frac{G_G^2}{H_G + \lambda} + \frac{G_D^2}{H_D + \lambda} - \frac{(G_G + G_D)^2}{H_G + H_D + \lambda} \right] - \gamma.$$

Une coupe n'est faite que si son gain est **positif** : γ est un seuil d'élagage intégré. Pour la perte logistique, $g_i = p_i - y_i$ et $h_i = p_i(1 - p_i)$.

Un exemple chiffré. Au tour 1 d'un modèle logistique, on part de $F_0 = 0$, donc $p_i = 0,5$ pour tous. Quatre clients tombent dans un nœud, avec $y = (1, 0, 1, 1)$: $g_i = (-0,5; +0,5; -0,5; -0,5)$ et $h_i = 0,25$ partout. Si le nœud reste une feuille : $G = -1, H = 1$ et, avec $\lambda = 1$, $w^* = \frac{1}{1+1} = 0,5$ (on relève la log-cote de 0,5 ; sans régularisation, ce serait 1). Si une coupe sépare les clients $\{1, 2\}$ de $\{3, 4\}$: $G_G = 0, H_G = 0,5$; $G_D = -1, H_D = 0,5$, donc

$$\text{gain} = \frac{1}{2} \left[\frac{0}{1,5} + \frac{1}{1,5} - \frac{1}{2} \right] = \frac{1}{2} (0,667 - 0,5) \approx 0,083 \quad (\gamma = 0).$$

Le calcul est confirmé : la valeur de feuille est 0,50 (1,00 sans régularisation) et le gain de la coupe 0,083. Le régulariseur λ **rétrécit** les valeurs de feuilles, surtout pour celles qui reposent sur peu de clients (petit H_j) : c'est une protection contre le surapprentissage, intégrée à l'algorithme.

Troisième apport : XGBoost est **rapide**. Il regroupe les valeurs des variables en **histogrammes** (au plus 256 intervalles) : au lieu d'essayer chaque seuil, on n'en essaie que 255, ce qui rend le calcul presque indépendant de n .

2.4.4 LightGBM et le choix d'une stratégie de croissance

LightGBM (Ke et coll., 2017) va plus loin dans la vitesse. Il utilise lui aussi des histogrammes, et ajoute deux idées : la **croissance par feuille** (*leaf-wise*) et l'économie de calcul sur les données. XGBoost (par défaut) fait grandir l'arbre **niveau par niveau** : toutes les feuilles du niveau courant sont coupées, puis celles du niveau suivant. LightGBM coupe à chaque étape **la feuille dont le gain est le plus grand**, où qu'elle soit : l'arbre devient asymétrique, avec parfois une longue branche profonde, et réduit plus vite l'erreur pour un même nombre de feuilles. Le réglage principal n'est donc plus la profondeur mais le **nombre de feuilles** (`num_leaves`), avec un garde-fou sur la taille minimale des feuilles (`min_child_samples`). La version de `scikit-learn`, `HistGradientBoostingClassifier`, reprend cette philosophie et n'exige aucune installation supplémentaire.

Trois précautions communes : (1) le boosting **surapprend si on le laisse tourner**, d'où l'**arrêt précoce** (section suivante) ; (2) il est **sensible au bruit des étiquettes** (il insiste sur les clients mal prédits, même quand c'est du hasard) ; (3) il **n'extrapole pas** : comme tout modèle à base d'arbres, il prédit une valeur constante en dehors du domaine des données d'entraînement.

2.4.5 Régler le boosting : pas, nombre d'arbres et arrêt précoce

Le pas ν et le nombre d'arbres M sont liés : diviser ν par deux oblige à doubler M pour atteindre un niveau de performance comparable. Plutôt que de choisir M à la main, on prend un ν petit et l'on **arrête quand la perte de validation cesse de baisser** : c'est l'**arrêt précoce** (*early stopping*). On réserve pour cela une partie de l'entraînement (ici 20 %) comme **jeu de validation**, distinct du jeu de test.

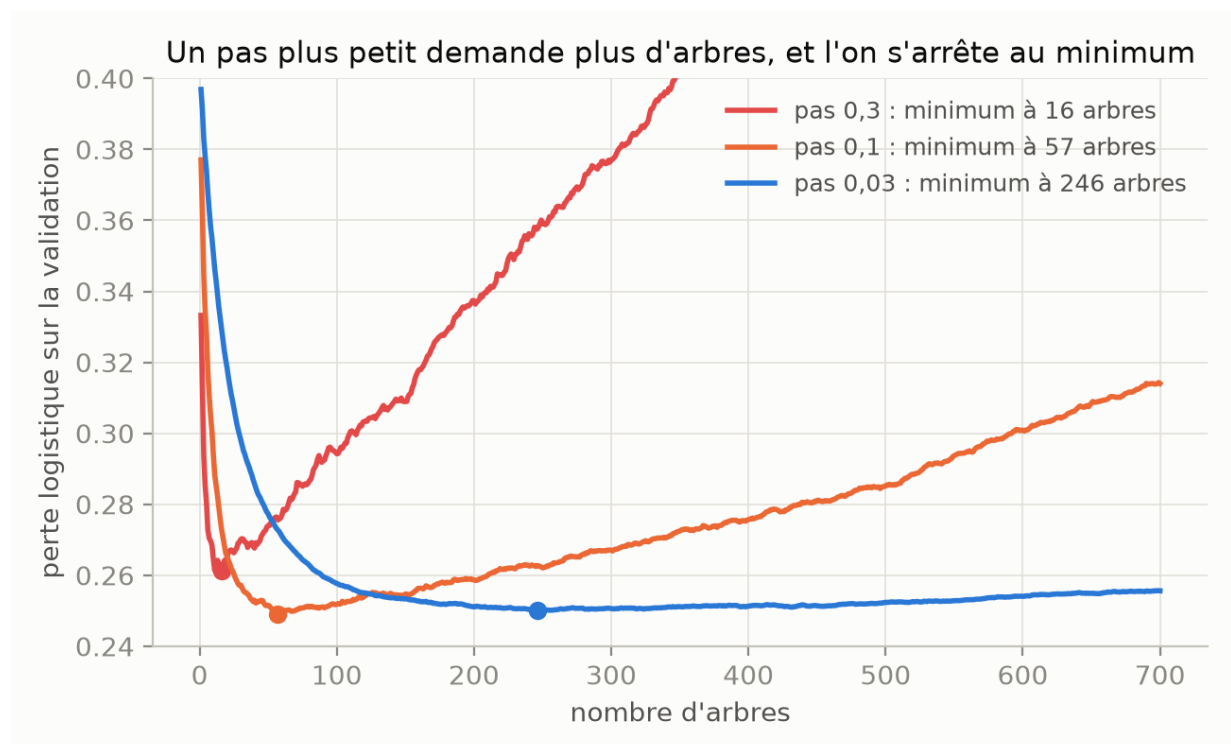


Figure 24: Perte logistique sur le jeu de validation selon le nombre d'arbres, pour trois pas d'apprentissage. Chaque courbe passe par un minimum puis remonte : le modèle se met à surapprendre. Plus le pas est petit, plus le minimum est atteint tard.

Les trois courbes ont la même forme : la perte de validation baisse, passe par un **minimum**, puis **remonte** (le modèle commence à s'ajuster au bruit de l'entraînement). Avec un pas de 0,3, le minimum est atteint après 16 arbres (perte 0,2612) ; avec 0,1, après 57 (0,2490) ; avec 0,03, après 246 (0,2500). Les pas de 0,1 et de 0,03 atteignent des minima équivalents, nettement meilleurs que celui du pas de 0,3 qui, trop gros, dépasse le fond : un petit pas atteint un minimum au moins aussi bon, au prix de plus d'arbres. La règle d'usage est de prendre un pas de 0,02 à 0,1 et de laisser l'arrêt précoce décider du nombre d'arbres.

Voici l'usage de XGBoost et de LightGBM, avec arrêt précoce (les catégories sont passées sous forme de type category de pandas) :

```
import xgboost as xgb

xg = xgb.XGBClassifier(n_estimators=1000, learning_rate=0.05, max_depth=4, subsample=0.8,
    ↪ colsample_bytree=0.8,
```

```

enable_categorical=True, early_stopping_rounds=30, eval_metric="logloss",
↳ n_jobs=1, random_state=0)
xg.fit(Xa, ya, eval_set=[(Xv, yv)], verbose=False)
print(xg.best_iteration, round(roc_auc_score(yte, xg.predict_proba(Xte_c)[: , 1]), 4))

```

```
227 0.8982
```

```

import lightgbm as lgb

lg = lgb.LGBMClassifier(n_estimators=1000, learning_rate=0.05, num_leaves=15, subsample=0.8,
↳ subsample_freq=1,
    colsample_bytree=0.8, n_jobs=1, random_state=0, verbose=-1)
lg.fit(Xa, ya, eval_set=[(Xv, yv)], callbacks=[lgb.early_stopping(30, verbose=False)])
print(lg.best_iteration_, round(roc_auc_score(yte, lg.predict_proba(Xte_c)[: , 1]), 4))

```

```
198 0.9026
```

(Ici, Xa/ya désignent 80 % de l'entraînement et Xv/yv les 20 % réservés à la validation ; Xte_c est le jeu de test, catégories en type category.)

Les deux bibliothèques s'arrêtent après 227 (XGBoost) et 198 (LightGBM) arbres et obtiennent respectivement des AUC de test de 0,898 et 0,903 : des valeurs voisines (la différence de 0,004 vient des détails d'implémentation : croissance par niveau ou par feuille, histogrammes, traitement des catégories), car l'algorithme de fond est le même. Les autres réglages usuels, avec leurs ordres de grandeur : profondeur 3 à 8 (XGBoost) ou 15 à 63 feuilles (LightGBM) ; taille minimale des feuilles ; subsample et colsample_bytree de 0,5 à 1 ; régularisation λ de 0 à 10. Le chapitre 1 (section 1.5, ☉) présente les méthodes pour les chercher de façon systématique.

2.4.6 Valeurs manquantes et catégories : sans artifice

Les modèles à base d'arbres modernes traitent **nativement** deux difficultés qui occupent un modèle linéaire : les valeurs manquantes et les catégories à nombreuses modalités.

- **Valeurs manquantes.** À chaque coupe, XGBoost, LightGBM et HistGradientBoostingClassifier **apprennent** vers quel côté envoyer les clients dont la valeur manque, en choisissant la direction qui réduit le plus la perte. Pas d'imputation : et le **fait d'être manquant**, souvent informatif (ici la satisfaction manque plus souvent quand elle est basse), est exploité directement.
- **Catégories.** Une variable comme ville (20 modalités) donnerait 20 colonnes en encodage par indicatrices. LightGBM et HistGradientBoostingClassifier cherchent directement, pour une variable catégorielle, le **meilleur partage des modalités en deux ensembles**.

	traitement	AUC test
	natif (NaN gardés, catégories natives)	0.9025
	NaN gardés, catégories en indicatrices	0.9018
	NaN gardés, catégories en codes entiers	0.9013
	médiane à la place des NaN, catégories en indicatrices	0.9019

Sur la résiliation, le traitement natif (valeurs manquantes conservées, catégories natives) donne une AUC de 0,903 ; les mêmes catégories passées en indicatrices 0,902 ; en codes entiers arbitraires (un « 3 » pour la ville D, ce qui n'a aucun sens) 0,901 ; et, si l'on **impute** d'abord les valeurs manquantes par la médiane (et que l'on perd donc l'information « manquant »), 0,902. Les écarts sont minuscules (au plus 0,001 d'AUC), de l'ordre du bruit d'échantillonnage : sur ces données, les variables importantes sont numériques et presque toutes renseignées, et la seule catégorie à nombreuses modalités (la ville) a un effet modeste. Ne tirez donc pas de ces chiffres qu'un traitement « bat » un autre ; retenez que le traitement natif **ne coûte rien** et simplifie le code. Sur des données plus riches en catégories ou en valeurs manquantes informatives, l'écart se creuse.

2.4.7 Le match : régression logistique, arbre, forêt, boosting

Comparons maintenant les quatre familles sur **le même découpage** et **la même mesure**. Nous mesurons l'AUC de deux façons : par **validation croisée à cinq plis** à l'intérieur de l'entraînement (qui donne aussi une dispersion), et sur le **jeu de test**, que nous n'ouvrons qu'une fois.

modèle	AUC validation croisée (moyenne)	écart-type entre plis	AUC test	perte
↪ logistique test				
régression logistique	0.8605	0.0098	0.8659	
↪ 0.2816				
arbre (profondeur choisie)	0.8630	0.0066	0.8847	
↪ 0.2584				
forêt aléatoire	0.8856	0.0085	0.8963	
↪ 0.2608				
gradient boosting	0.8900	0.0077	0.9025	
↪ 0.2437				

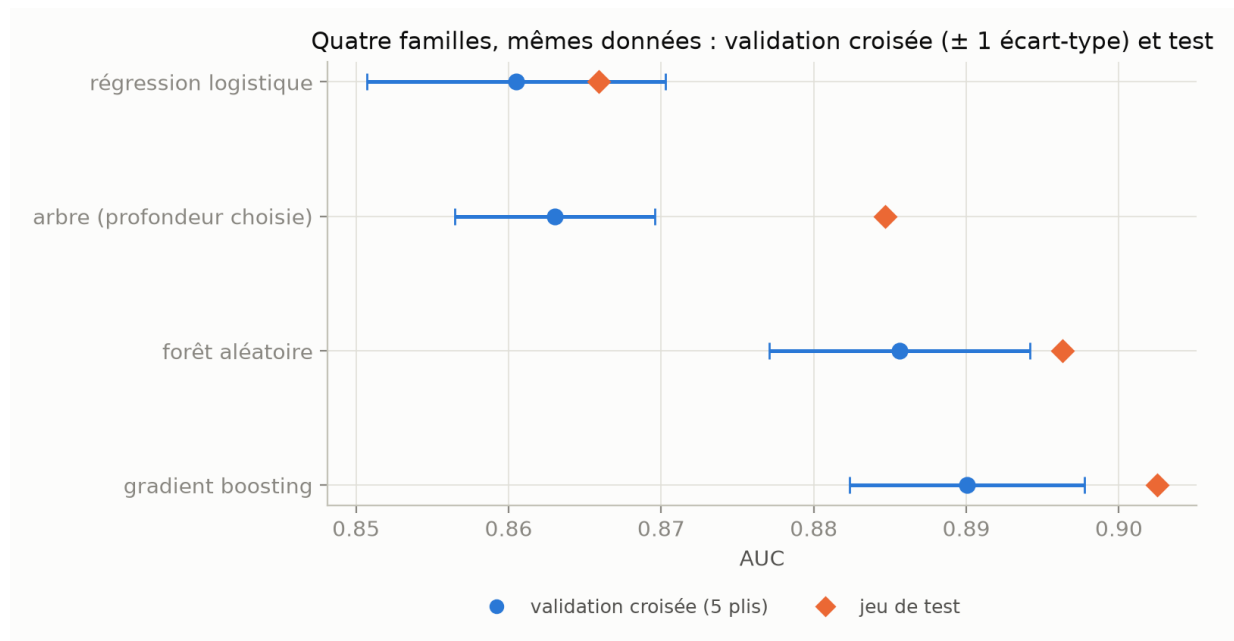


Figure 25: AUC de quatre familles de modèles sur la résiliation : moyenne et écart-type sur cinq plis de validation croisée (cercles bleus) et valeur sur le jeu de test (losanges orange).

Lecture, en trois temps.

1. **La hiérarchie.** Sur le jeu de test, la régression logistique obtient 0,866, l'arbre 0,885, la forêt 0,896 et le boosting 0,903. En validation croisée, l'ordre est le même pour la forêt et le boosting (0,886 et 0,890, contre 0,860 pour la régression logistique, avec des écarts-types entre plis de l'ordre de 0,008) : leur avance dépasse nettement cette dispersion. L'arbre, lui, est **indiscernable** de la régression logistique en validation croisée (0,863, écart-type 0,007) alors qu'il la dépasse sur le jeu de test : ne tirez pas de conclusion d'un seul chiffre. Remarquez aussi que, pour tous les modèles, l'AUC de test est un peu supérieure à celle de validation croisée : en validation croisée, chaque modèle n'apprend que sur 80 % de l'entraînement (7 200 clients) ; le modèle final voit les 9 000, et les modèles flexibles profitent davantage de données supplémentaires (courbes d'apprentissage, section 1.3).
2. **La comparaison appariée.** Pour savoir si l'avantage du boosting sur la régression logistique est **réel**, on ne compare pas deux intervalles de confiance isolés : on compare les deux modèles **sur les mêmes clients**. Pli par pli, le boosting gagne en moyenne 0,030 d'AUC (de 0,020 à 0,035 selon le pli). Sur le jeu de test, un **bootstrap apparié** (on rééchantillonne 400 fois les mêmes 3 000 clients et l'on recalcule la différence) donne un écart d'AUC de 0,037

avec un intervalle à 95 % de $[0,025 ; 0,049]$. Entre boosting et forêt, l'écart moyen est de 0,006 avec un intervalle $[-0,001 ; 0,014]$. Entre forêt et régression logistique, 0,030 $[0,021 ; 0,042]$. Un intervalle qui contient zéro signale une différence qu'on ne peut pas distinguer du hasard d'échantillonnage.

3. **La perte logistique** (dernière colonne) juge les **probabilités** et non plus seulement le classement. Elle va ici dans le même sens que l'AUC : 0,244 pour le boosting, 0,258 pour l'arbre, 0,261 pour la forêt et 0,282 pour la régression logistique (plus c'est bas, mieux c'est). Un modèle peut cependant très bien **ordonner** les clients et mal **estimer** leur probabilité ; la perte seule ne dit pas si les probabilités annoncées sont fiables. Vérifier cela, et le corriger, est le sujet de la section 5.2.

Pourquoi les arbres l'emportent ici. L'écart d'AUC n'est pas une loi de la nature : il dépend des données. Dans la simulation, le risque de départ dépend de **seuils** et d'**interactions** (récence supérieure à 150 jours *et* satisfaction inférieure à 3,2 ; trois tickets au support *et* beaucoup de retours ; beaucoup de promotions *sans* programme de fidélité...) que la régression logistique ne sait pas écrire. Sur des données où les effets sont réguliers et sans interactions marquées, la régression logistique, bien réglée, aurait **égalé** le boosting. C'est pourquoi on la garde toujours comme **modèle de référence** (chapitre 1, section 1.4) : si le boosting ne la bat pas nettement, il ne vaut pas sa complexité.

Le jeu de test est ouvert une fois. Nous avons regardé le jeu de test pour chaque famille afin de les comparer ici : c'est une démonstration. En situation réelle, on choisirait le modèle **par validation croisée sur l'entraînement**, et l'on n'ouvrirait le jeu de test qu'**une seule fois**, pour le modèle retenu.

À retenir.

- Le **gradient boosting** construit des arbres **séquentiellement** : chaque arbre ajuste le **pseudo-résidu** $-\partial\ell/\partial F$ du modèle courant (le résidu pour la perte quadratique, $y - p$ pour la perte logistique), pondéré par un petit pas ν . C'est une descente de gradient **dans l'espace des fonctions**.
- **XGBoost** ajoute un objectif régularisé et un développement au second ordre : valeur de feuille $-G/(H + \lambda)$, gain de coupe explicite. **LightGBM** et **HistGradientBoosting-Classif** ajoutent histogrammes et croissance par feuille.
- On règle surtout : **pas** ν (petit), **nombre d'arbres par arrêt précoce**, profondeur/feuilles, sous-échantillonnage, régularisation.
- Les modèles à arbres gèrent **valeurs manquantes** et **catégories** sans artifice ; ils n'ont pas besoin de standardisation ; ils n'extrapolent pas.
- Comparez **sur les mêmes données et les mêmes plis**, avec un écart **apparié** et son incertitude ; gardez un **modèle de référence** simple.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.6 (boosting écrit à la main), 2.7 (XGBoost et LightGBM : réglages, valeurs manquantes, catégories) et 2.8 (le match des modèles, avec comparaison appariée), exercices 2.9 et 2.10.

2.5 ⊕ Pour aller plus loin : SVM et noyaux, k plus proches voisins, Bayes naïf

Section optionnelle. Trois familles classiques, que l'on rencontre encore souvent et qui illustrent chacune une idée utile : la **marge** et l'astuce du **noyau** (SVM), la **ressemblance** (k plus proches voisins), l'**hypothèse d'indépendance** (Bayes naïf). Sur des données tabulaires de taille moyenne, le boosting les devance généralement ; elles gardent leur intérêt pédagogique et, pour certains problèmes (peu de données, texte, images, données très structurées), leur intérêt pratique.

2.5.1 Les machines à vecteurs de support (SVM)

L'idée de la marge. Parmi toutes les droites qui séparent deux classes, laquelle choisir ? Celle qui laisse le **plus de place** de part et d'autre : la droite qui maximise la **marge**, la distance entre elle et les clients les plus proches. Une droite collée à des clients est fragile ; une droite au milieu d'une zone vide l'est moins. Les clients qui touchent la marge sont les **vecteurs de support** : ce sont les **seuls** qui déterminent la solution (déplacer un client loin de la marge ne change rien).

Quand les classes se chevauchent (c'est le cas de presque toutes les vraies données), on autorise des violations de la marge, pénalisées. Le problème, pour un score $f(x) = w^\top x + b$, s'écrit exactement dans le cadre de la section 2.1 :

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i f(x_i)),$$

soit la **perte charnière** (section 2.1.1) plus une pénalité ℓ_2 . Le paramètre C règle le compromis : grand C , on punit fort les violations (marge étroite, risque de surapprentissage) ; petit C , on tolère (marge large, modèle plus simple).

L'astuce du noyau. Un score linéaire ne sépare pas toujours les classes. L'astuce est de **transformer les variables** pour les rendre séparables, puis d'y chercher une frontière linéaire. Exemple minuscule, en dimension 1 : sept clients repérés par une variable x , la classe 1 étant celle des clients proches de zéro.

x	-3	-2	-1	0	1	2	3
classe	0	0	1	1	1	0	0

Sur la droite des réels, aucun seuil ne sépare les 1 des 0 (les 1 sont au milieu). Mais si l'on associe à chaque x le point $\varphi(x) = (x, x^2)$, les clients de la classe 1 ont $x^2 \leq 1$ et ceux de la classe 0 ont $x^2 \geq 4$: la droite horizontale $x^2 = 2,5$ les sépare. Une **frontière linéaire dans l'espace transformé** correspond à une frontière **non linéaire** dans l'espace d'origine.

Le miracle est que l'on n'a jamais besoin de calculer φ : l'algorithme n'utilise les clients que par leurs **produits scalaires** $\varphi(x)^\top \varphi(x')$, et un **noyau** $k(x, x')$ les calcule directement. Par exemple, le noyau polynomial de degré 2, $k(x, x') = (xx' + 1)^2$, correspond à $\varphi(x) = (x^2, \sqrt{2}x, 1)$: pour $x = 2$ et $x' = 3$, $k = (6 + 1)^2 = 49$ et $\varphi(x)^\top \varphi(x') = 36 + 12 + 1 = 49$. Le noyau le plus utilisé, le noyau **gaussien** (RBF), $k(x, x') = \exp(-\gamma \|x - x'\|^2)$, correspond à un espace de dimension *infinie* ; il mesure la **ressemblance** de deux clients, proche de 1 s'ils sont voisins et de 0 s'ils sont éloignés.

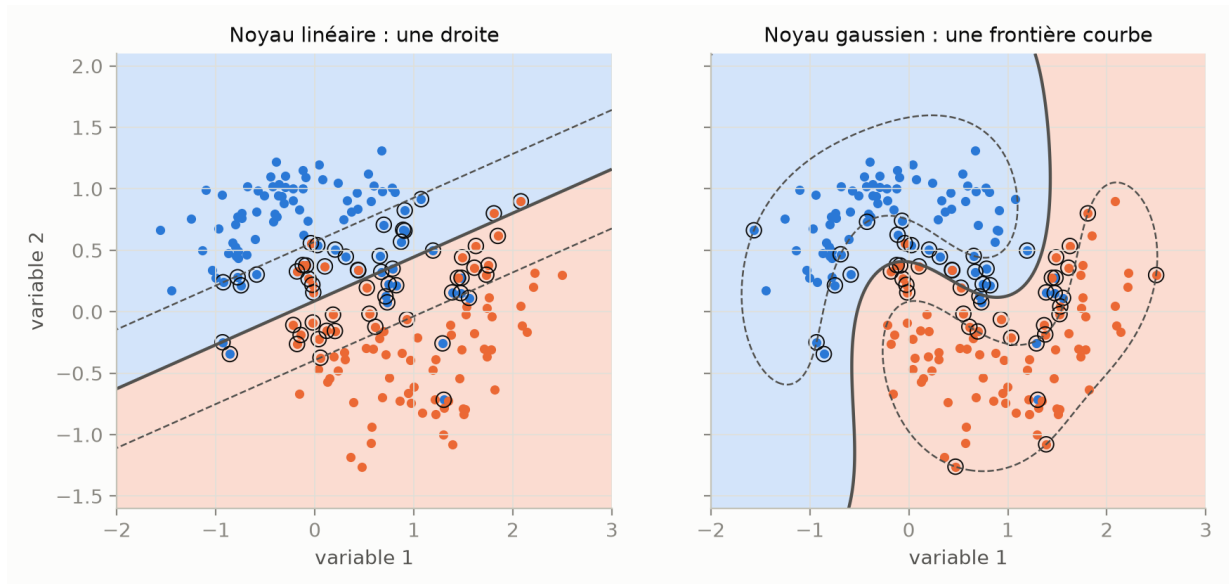


Figure 26: Deux SVM sur des données en forme de deux croissants. À gauche, une droite et ses marges (pointillés) : elle sépare mal. À droite, avec un noyau gaussien, la frontière épouse la forme des données. Les points cerclés sont les vecteurs de support.

Le paramètre γ règle la portée : grand, chaque client n'influence que son voisinage immédiat (frontière très tourmentée) ; petit, la frontière est lisse.

Sur ces données en deux croissants entremêlés, le SVM linéaire classe correctement 76 % des clients de test, le SVM à noyau gaussien 90 %. Le premier s'appuie sur 61 vecteurs de support, le second sur 54 (sur 180 points d'entraînement).

Sur la résiliation, le SVM exige une chose que le boosting ignore : des variables à l'échelle (le noyau gaussien repose sur des distances). Il ne fournit pas directement de probabilités (seulement un score : la distance signée à la frontière, que l'on peut calibrer, section 5.2), et son coût d'entraînement croît environ comme le **carré** du nombre de clients : au-delà de quelques dizaines de milliers de lignes, il devient lent.

```
from sklearn.svm import SVC

svm = make_pipeline(pre_lin, SVC(kernel="rbf", C=1.0, gamma="scale"))
svm.fit(Xtr, ytr)
print(round(roc_auc_score(yte, svm.decision_function(Xte)), 4))
```

```
0.8492
```

Sur nos clients, le SVM à noyau gaussien atteint une AUC de test de 0,849 et le SVM linéaire 0,862 (à comparer à 0,866 pour la régression logistique : même famille de fonctions, perte différente, résultats voisins). Sans mise à l'échelle des variables, le même SVM à noyau gaussien tombe à 0,757 : la récence en jours écrase tout le reste dans le calcul des distances.

2.5.2 Les k plus proches voisins (k-NN)

Le modèle le plus intuitif qui soit : pour prédire un nouveau client, on cherche, dans les données d'entraînement, les k **clients qui lui ressemblent le plus** et l'on prend le vote de leurs classes. Il n'y a pas d'« entraînement » : le modèle **est** les données. Tout repose sur deux choix : la **distance** (en général euclidienne, sur des variables standardisées) et k , qui règle le compromis biais-variance du chapitre 1 : $k = 1$ colle aux données (variance maximale, frontière en confettis), k très grand lisse tout (biais fort).

Le fléau de la dimension. Le k-NN suppose qu'**être proche** a un sens. Or, quand le nombre de variables d grandit, les distances **se concentrent** : tous les points deviennent à peu près aussi éloignés les uns des autres. Pour des points tirés uniformément dans un cube, le rapport entre la distance au voisin le plus proche et la distance au voisin le plus lointain tend vers 1 quand d grandit : la notion de « plus proche voisin » perd son sens.

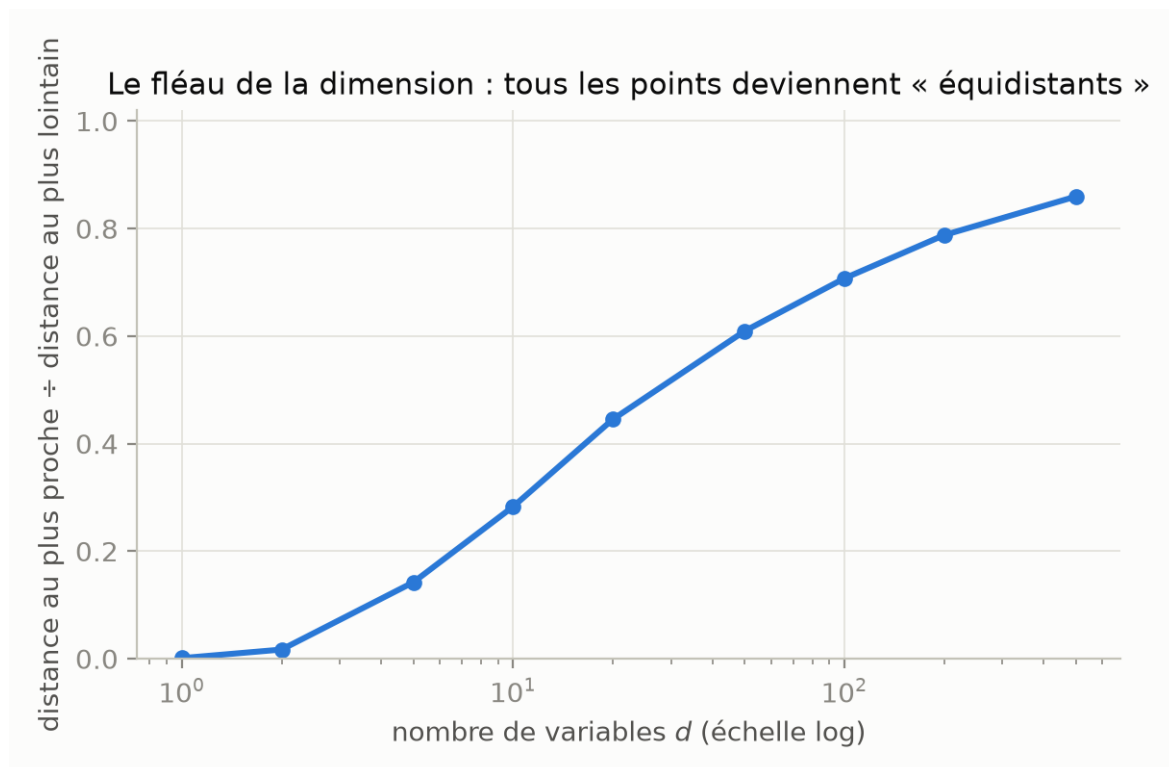


Figure 27: Rapport entre la distance au plus proche voisin et la distance au plus lointain pour des points tirés au hasard, en fonction du nombre de variables : il tend vers 1.

Le rapport moyen est de 0,02 en dimension 2, de 0,28 en dimension 10 et de 0,86 en dimension 500 : dans ce dernier cas, le voisin « le plus proche » est presque aussi loin que le plus lointain. Pour un k-NN, ajouter des variables **inutiles** noie les variables utiles.

Sur la résiliation (45 colonnes après encodage), la validation croisée retient $k = 150$ (AUC 0,849 ; avec $k = 1$, seulement 0,643 : la variance). Sur le jeu de test, l'AUC est de 0,858, et de 0,839 sans mise à l'échelle. Le k-NN fait moins bien que les modèles précédents ici : trop de variables peu informatives, et une prédiction lente (il faut comparer le client à *tous* les autres).

2.5.3 Bayes naïf

Le classifieur de **Bayes naïf** applique la règle de Bayes (volume I, section 2.1) en faisant une hypothèse brutale : **les variables sont indépendantes entre elles, une fois la classe connue**. Pour une classe k (parti, resté) et des variables $x_1, \dots, x_p$:

$$P(k | x) \propto P(k) \prod_{j=1}^p P(x_j | k).$$

On n'a donc besoin d'estimer que des lois à **une variable**, ce qui est facile, même avec peu de données. Un exemple chiffré : douze clients, dont 4 sont partis. Parmi les 4 partis, 1 avait ouvert le dernier courriel et 3 avaient ouvert un ticket au support ; parmi les 8 restés, 6 avaient ouvert le courriel et 2 un ticket. Un nouveau client **n'a pas ouvert** le courriel et **a ouvert** un ticket :

- score « parti » : $\frac{4}{12} \times \frac{3}{4} \times \frac{3}{4} = 0,1875$;
- score « resté » : $\frac{8}{12} \times \frac{2}{8} \times \frac{2}{8} = 0,0417$;
- probabilité de départ : $\frac{0,1875}{0,1875 + 0,0417} \approx 0,818$.

L'hypothèse d'indépendance est presque toujours **fausse** (la récence et le nombre de commandes sont liés) mais, pour **classer**, elle est souvent tolérable : le classement reste raisonnable même quand les probabilités sont fausses. Sur nos données, un Bayes naïf gaussien (sur les variables numériques seules) obtient une AUC de 0,840. En revanche, ses **probabilités** sont mauvaises : en comptant plusieurs fois la même information, il devient **sûr de lui à tort**. Quand il annonce plus de 90 % de risque de départ (570 clients du jeu de test), il prédit en moyenne 97 % alors que 43 % seulement de ces clients partent. À titre de comparaison, la régression logistique annonce plus de 90 % pour 3 clients, avec 91 % prédits et 100 % observés. La perte logistique du Bayes naïf vaut 0,74 (contre 0,28 pour la régression logistique). C'est un exemple parfait de modèle **bien classant mais mal calibré** (section 5.2).

2.5.4 Comment choisir entre elles ?

modèle	AUC test
régression logistique	0.8659
SVM linéaire	0.8617
SVM gaussien	0.8492
k-NN	0.8579
Bayes naïf gaussien	0.8395
gradient boosting	0.9025

Modèle	Hypothèse implicite	Atouts	Faiblesses
SVM à noyau	la ressemblance (le noyau) est bien choisie	peu de données, grande dimension, bonne frontière	exige des variables à l'échelle ; lent au-delà de quelques dizaines de milliers de lignes ; pas de probabilités directes
k-NN	les voisins se ressemblent	aucun entraînement, très simple, naturellement non linéaire	fléau de la dimension ; prédiction lente ; sensible à l'échelle
Bayes naïf	variables indépendantes entre elles à classe donnée	rapide, peu de données, texte	probabilités mal calibrées ; ignore les interactions

Le tableau du dessus récapitule les AUC de test : sur ces données, le boosting reste devant. Mais ces trois modèles ont une vertu : ils obligent à penser en termes de **distance**, de **marge** et d'**indépendance**, trois idées qui reviennent partout en apprentissage automatique.

À retenir.

- Le **SVM** cherche la droite de **marge maximale** ; il minimise la perte charnière plus une pénalité ℓ_2 . Avec un **noyau** (gaussien, polynomial), il fait des frontières non linéaires sans calculer les variables transformées. Mise à l'échelle obligatoire.
- Le **k-NN** prédit par vote des k plus proches voisins : k règle le compromis biais-variance ; le **fléau de la dimension** le rend fragile quand les variables sont nombreuses.
- Le **Bayes naïf** suppose l'indépendance des variables à classe donnée : rapide et robuste pour classer, **mal calibré** pour estimer des probabilités.

Pour s'entraîner. Cahier, chapitre 2 : application 2.9 (SVM, k-NN, Bayes naïf face au boosting, et le fléau de la dimension), exercice 2.11 (Bayes naïf à la main).

2.6 ⊕ Pour aller plus loin : CatBoost, stacking et blending

Section optionnelle. Deux idées pour aller au-delà d'un modèle unique : un boosting conçu pour les **variables catégorielles** (CatBoost), et la **combinaison** de plusieurs modèles différents (blending, stacking). Dans les deux cas, le danger principal est le même, et il est discret : la **fuite d'information** de la cible vers les variables.

2.6.1 CatBoost et le piège de l'encodage par la cible

Comment donner une variable à beaucoup de modalités (la ville, le code d'un produit) à un modèle ? L'encodage par indicatrices crée autant de colonnes que de modalités. Une idée tentante : remplacer la modalité par la **moyenne de la cible** dans cette modalité (l'**encodage par la cible**, *target encoding*). La ville D devient « 0,17 », parce que 17 % des clients de la ville D sont partis.

L'idée est excellente, **mais elle fuit** si on la calcule naïvement. La moyenne de la ville D contient la réponse du client lui-même : pour un client dont la ville ne compte que quelques clients, la valeur encodée *est* sa propre étiquette, à peine diluée. Le modèle apprend « une valeur élevée signifie parti » sur l'entraînement, où c'est vrai par construction, puis échoue sur de nouvelles données.

Pour le voir, fabriquons une variable **sans aucune information** : un identifiant de zone à 1 500 modalités, tiré au hasard, indépendant de la résiliation. Encodons-la naïvement par la moyenne de la cible, et ajustons un modèle logistique sur cette seule colonne.

Avec environ 6 clients par zone, le modèle ajusté sur l'encodage naïf atteint une AUC de **0,801** sur l'entraînement, sur une variable qui ne contient *rien*. Sur le jeu de test, l'AUC retombe à 0,505 : le hasard. C'est de la fuite pure : l'étiquette du client est passée dans la variable.

La solution de CatBoost : l'encodage ordonné. On met les clients dans un **ordre aléatoire** et l'on encode chaque client par la moyenne de la cible des **seuls clients qui le précèdent** dans cet ordre, avec un petit a priori pour lisser :

$$\text{enc}_i = \frac{\sum_{j < i, x_j = x_i} y_j + a p}{\#\{j < i : x_j = x_i\} + a'}$$

où p est la moyenne générale de la cible et a un poids d'a priori. L'étiquette d'un client n'intervient donc **jamais** dans sa propre valeur. Un exemple à la main : six clients de deux villes, $a = 1$ et $p = 0,5$.

ville	y	encodage_ordonne	encodage_naif
A	1	0.500	0.667
A	0	0.750	0.667
A	1	0.500	0.667
B	0	0.500	0.333
B	0	0.250	0.333
B	1	0.167	0.333

Pour la ville A, le premier client n'a pas d'historique : $\frac{0+0,5}{0+1} = 0,5$; le deuxième voit un seul prédécesseur, parti ($y = 1$) : $\frac{1+0,5}{1+1} = 0,75$; le troisième voit deux prédécesseurs, un parti et un resté : $\frac{1+0,5}{2+1} = 0,5$. L'encodage naïf aurait donné 0,667 à tous les clients de la ville A, y compris ceux qui ont contribué à ce 0,667. Sur notre variable de bruit, l'encodage ordonné ramène l'AUC d'entraînement à 0,528 : le modèle n'a plus rien à mémoriser, ce qui est la bonne réponse.

CatBoost (Prokhorenkova et coll., 2018) fait de cette idée un principe : encodage ordonné des catégories, mais aussi **boosting ordonné** (les pseudo-résidus d'un client sont calculés avec un modèle qui ne l'a pas vu), et des arbres **symétriques** (*oblivious trees*, la même question à tous les nœuds d'un niveau), plus rapides et moins sujets au surapprentissage. En pratique, on lui passe les colonnes catégorielles telles quelles :

```
from catboost import CatBoostClassifier

Xtr_cb, Xte_cb = Xtr.copy(), Xte.copy()
Xtr_cb[cat], Xte_cb[cat] = Xtr_cb[cat].fillna("manquant"), Xte_cb[cat].fillna("manquant")
catb = CatBoostClassifier(iterations=300, learning_rate=0.08, depth=6, cat_features=cat,
    ↪ random_seed=0, verbose=False, thread_count=1)
catb.fit(Xtr_cb, ytr)
print(round(roc_auc_score(yte, catb.predict_proba(Xte_cb)[: , 1]), 4))
```

0.9046

CatBoost obtient une AUC de test de 0,905, à comparer à 0,903 pour le boosting de la section 2.4 : sur ces données, avec peu de catégories, les deux sont équivalents. CatBoost brille surtout quand les variables catégorielles sont nombreuses ou à très grand nombre de modalités, et sa configuration par défaut est réputée robuste.

2.6.2 Combiner des modèles : le blending

Les modèles de la section 2.4 ne se trompent pas tous sur les mêmes clients. **Combiner** leurs prédictions peut donc faire mieux que le meilleur. Le plus simple est le **blending** (mélange) : une **moyenne** des probabilités prédites, éventuellement pondérée.

Pourquoi cela marche-t-il ? Par la même formule qu'en 2.3.2 : la variance de la moyenne de prédictions $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ ne diminue que si les prédictions sont **peu corrélées**. Moyenner trois modèles quasi identiques ne sert à rien ; moyenner trois modèles de **familles différentes** (un linéaire, une forêt, un boosting) apporte quelque chose.

Une règle essentielle : les prédictions utilisées pour **choisir** les poids ne doivent pas être des prédictions **faites sur les données d'entraînement** du modèle, qui sont trop belles. On utilise des

prédictions **hors pli** (*out-of-fold*, OOF) : pour chaque client, la prédiction d'un modèle entraîné sans lui, par validation croisée.

Les corrélations entre les prédictions hors pli sont de 0,88 (régression logistique et forêt), 0,83 (régression logistique et boosting) et 0,92 (forêt et boosting). La moyenne simple des trois modèles obtient une AUC de test de **0,900**, contre 0,903 pour le boosting seul : **le mélange est un peu moins bon que son meilleur membre**, parce que le modèle linéaire, nettement plus faible, dilue les autres. La moyenne forêt + boosting seulement obtient 0,904, un peu au-dessus. Moyenne des modèles de **niveau comparable**, ou pondérez-les (c'est ce que fait le stacking). Ces corrélations, toutes élevées, expliquent aussi la modestie du gain : ces modèles se trompent largement sur les mêmes clients.

2.6.3 Le stacking : laisser un modèle apprendre à combiner

Le **stacking** (empilement) pousse l'idée plus loin : au lieu de fixer les poids du mélange, on les **apprend** avec un **méta-modèle** (souvent une régression logistique), dont les variables d'entrée sont les prédictions des modèles de base. On passe de « moyenne égale » à « donne trois fois plus de poids au boosting qu'à la forêt, et un petit poids au modèle linéaire ». La condition de validité est celle du blending, et le piège est sournois : **le méta-modèle doit être entraîné sur des prédictions hors pli**.

```
# oof : une colonne par modèle de base, prédictions HORS PLI (calculées plus haut avec
↳ cross_val_predict)
meta = LogisticRegression().fit(logit_(np.clip(oof, 1e-4, 1 - 1e-4)), ytr)
print(meta.coef_.round(2))
```

```
[[0.09 0.47 0.57]]
```

(oof a été calculé plus haut : pour chaque client, la prédiction de chaque modèle de base entraîné **sans lui**, par validation croisée à cinq plis. Les prédictions de test s'obtiennent en réentraînant chaque modèle sur tout l'entraînement.)

Les coefficients appris sur des prédictions hors pli sont 0,09 (régression logistique), 0,47 (forêt) et 0,57 (boosting) ; l'AUC de test de l'empilement est **0,904**. Que se passe-t-il si l'on commet la faute et que l'on entraîne le méta-modèle sur les prédictions que chaque modèle fait **sur ses propres données d'entraînement** ? La forêt, qui a vu ces clients, y est presque parfaite (AUC 0,986 sur l'entraînement) ; le méta-modèle conclut que la forêt est un oracle et lui donne un poids énorme (coefficients $-1,67, 5,64, -0,12$). Résultat sur le jeu de test : 0,883 au lieu de 0,904. Le coût d'un méta-modèle fautif n'est pas toujours spectaculaire, mais il va toujours dans le mauvais sens, et il **fausse l'évaluation** si on l'évalue lui aussi sur des données déjà vues.

2.6.4 Quand cela vaut-il la peine ?

Le gain de l'empilement sur le meilleur modèle seul est ici de 0,002 d'AUC, avec un intervalle à 95 % de $[-0,001 ; 0,005]$ (bootstrap apparié sur le jeu de test). Aucune de ces combinaisons n'apporte plus qu'un gain marginal, et c'est typique : sur des données tabulaires de cette taille, **le boosting bien réglé capte l'essentiel**. Dans les compétitions de prédiction, où l'on se bat pour le troisième chiffre après la virgule, on empile systématiquement. Dans une entreprise, chaque modèle supplémentaire est un **coût** : plus de code, plus de dépendances, plus de pannes, plus de difficulté à expliquer. La question honnête n'est pas « peut-on gagner 0,002 ? » mais « ce gain justifie-t-il la complexité, au regard de la décision que l'on prend avec la prédiction ? »

À retenir.

- L'**encodage par la cible naïf** **fuit** : l'étiquette du client entre dans sa propre variable (sur du bruit pur, il « apprend » l'étiquette). **CatBoost** l'évite par l'**encodage ordonné** (on n'utilise que les clients qui précèdent).
- Le **blending** moyenne des modèles ; il ne marche que si leurs erreurs sont **peu corrélées**.
- Le **stacking** apprend la combinaison avec un méta-modèle, qui doit être entraîné sur des prédictions **hors pli**, jamais sur des prédictions faites sur les données d'entraînement.
- Les gains sont souvent minces ; comparez-les à leur **coût de complexité**.

Pour s'entraîner. Cahier, chapitre 2 : application 2.10 (encodage ordonné, CatBoost et empiement sans fuite), exercice 2.12 (blending et corrélation des erreurs).

Bilan du chapitre 2

Vous savez maintenant :

- **décrire** un modèle supervisé par ses trois ingrédients (famille de fonctions, perte, algorithme), comprendre pourquoi on remplace le coût 0-1 par un substitut convexe (logistique, charnière), et **calculer à la main** un pas de descente de gradient ;
- expliquer pourquoi la descente de gradient et les pénalités exigent des variables **à l'échelle**, et voir la pénalité ℓ_2 ou ℓ_1 comme une **contrainte** (disque ou losange) ;
- **construire un arbre** à la main (impureté de Gini ou entropie, meilleur seuil, gain), le régler (profondeur, taille des feuilles, élagage par coût-complexité) et dire **ce qu'il sait écrire** (seuils, interactions) et **ce qui le fragilise** (l'instabilité) ;
- démontrer que la **variance d'une moyenne** de prédictions corrélées vaut $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, en déduire le **bagging** et les **forêts aléatoires**, utiliser l'erreur **hors sac**, et **se méfier de l'importance par impureté** ;
- voir le **gradient boosting** comme une descente de gradient dans l'espace des fonctions (pseudo-résidu $-\partial\ell/\partial F$), calculer à la main une valeur de feuille et un gain de coupe **XG-Boost**, régler le pas et le nombre d'arbres par **arrêt précoce**, et profiter des valeurs manquantes et des catégories natives de **LightGBM** ;
- **comparer** plusieurs familles sur les mêmes données avec une **différence appariée** et son incertitude, en gardant un modèle de référence simple ;
- (en option) comprendre la **marge** et l'astuce du **noyau** des SVM, le **fléau de la dimension** des k-NN, l'indépendance du **Bayes naïf**, l'**encodage ordonné** de CatBoost et le **stacking sans fuite**.

Sur la résiliation, la hiérarchie obtenue est la suivante, en AUC sur le jeu de test :

Modèle	AUC
Régression logistique (référence)	0,866
Arbre de décision réglé	0,885
Forêt aléatoire	0,896
Gradient boosting	0,903

Trois leçons dépassent ce chapitre. **Un modèle plus riche n'est pas toujours meilleur** : l'avantage

des arbres ici vient des seuils et des interactions du problème, et ne serait pas apparu sur des données lisses. **L'évaluation prime sur l'algorithme** : jeu de test intact, validation croisée dans l'entraînement, comparaisons appariées, modèle de référence. Enfin, **chaque fois que l'on réutilise des étiquettes** (encodage par la cible, empilement), la fuite d'information guette.

Pour s'entraîner. Cahier, chapitre 2 : applications 2.1 à 2.10 et exercices 2.1 à 2.12.

Le chapitre 3 quitte le monde des étiquettes : que peut-on apprendre des clients quand on ne sait pas ce que l'on cherche ? Le chapitre 4 reviendra sur la préparation des variables (encodage, échelle, déséquilibre), et le chapitre 5 sur l'évaluation fine des modèles (calibration, interprétabilité) que nous avons ici seulement effleurée.

Chapitre 3 : Apprentissage non supervisé et réduction de dimension

« Un algorithme de classification rend toujours des groupes. La vraie question n'est pas *combien*, c'est *est-ce que ça existe ?* »

Jusqu'ici, dans ce volume, chaque observation portait une **étiquette** : le client est parti ou non, la commande est frauduleuse ou non. On pouvait donc **vérifier** un modèle en comparant ses réponses à la bonne réponse. Ce chapitre quitte ce confort. On dispose seulement de **variables** décrivant les clients, et on demande à la machine de **découvrir de la structure** : des groupes qui se ressemblent, des directions qui résument les données, des représentations plus courtes.

La difficulté change de nature. Quand il n'y a pas de bonne réponse à vérifier, comment savoir qu'un résultat est **bon** ? C'est la question qui traverse tout le chapitre, et la réponse tient en une idée : on ne vérifie pas un résultat non supervisé contre la vérité, on le **soumet à plusieurs épreuves** (cohérence interne, comparaison avec un hasard sans structure, stabilité quand on perturbe les données, utilité pour une décision) et on regarde si elles se rejoignent.

Ce que le volume II a déjà couvert. L'analyse en composantes principales (volume II, section 3.1), l'analyse factorielle (section 3.2), les k-means et la classification hiérarchique (section 3.3) y sont présentées avec leurs bases : distance, algorithme de Lloyd, choix du nombre de groupes par le coude et la silhouette, avertissement « une méthode rend toujours des groupes ». Nous **ne les refaisons pas** : nous allons plus loin, vers la **validation** rigoureuse des groupes, vers des méthodes de réduction qui ne sont pas linéaires, et vers des méthodes de classification qui ne cherchent pas des boules.

Le chemin de ce chapitre

- **3.1 Classification non supervisée et validation** : critères internes (inertie, silhouette, Calinski–Harabasz, Davies–Bouldin, statistique de l'écart), stabilité par rééchantillonnage, validation externe contre une vérité cachée, rôle de l'échelle et du choix des variables, lecture des groupes pour une décision.
- **3.2 Réduction de dimension** : pourquoi réduire (la malédiction de la dimension), rappel de l'ACP et reconstruction, ACP à noyau, SVD tronquée, factorisation non négative, projections aléatoires et le lemme de Johnson–Lindenstrauss, combien de dimensions garder.
- **⊕ 3.3 DBSCAN, classification hiérarchique, mélanges gaussiens** : des groupes qui ne sont pas des boules, des groupes « flous », l'algorithme EM.
- **⊕ 3.4 t-SNE et UMAP** : dessiner des données en haute dimension, et ce que ces dessins n'ont pas le droit de dire.

Pour s'entraîner. Le cahier de ce chapitre propose huit applications guidées (segmentation de bout en bout, statistique de l'écart, réduction des chiffres manuscrits, EM écrit à la main, t-SNE et UMAP...) et douze exercices corrigés.

Les données de ce chapitre

Deux jeux de données, de natures opposées, servent de fil conducteur.

- **Les clients de la boutique** (`donnees/clients_ml.csv`, 12 000 clients, **simulés**). Nous retenons sept variables de comportement : `age`, `nb_commandes_12m`, `montant_12m` (en logarithme, car très asymétrique), `recence_jours`, `part_achats_promo`, `taux_ouverture_email` et `nb_promos_recues_12m`. Le fichier contient aussi une variable `segment_vrai` : la **classe latente** qui a servi à fabriquer les comportements (quatre types : occasionnels, fidèles, chasseurs de promotions, grands paniers). Nous la gardons **de côté**, comme un examinateur garde le corrigé : elle servira à juger les méthodes en 3.1.6, mais aucune méthode ne la voit. Dans la vie réelle, cette vérité n'existe pas ; ici, elle nous permet de mesurer ce que valent nos critères.
- **Les chiffres manuscrits** (`load_digits` de `scikit-learn`, **réel**, intégré à la bibliothèque : 1 797 images de 8×8 pixels, donc 64 variables, représentant les chiffres de 0 à 9 écrits à la main). C'est un jeu classique pour les méthodes de réduction, parce qu'on peut **voir** les observations et savoir à quel chiffre elles correspondent.

3.1 Classification non supervisée et validation

Intuition. Un cartographe dessine des frontières entre des régions sans qu'aucun panneau n'indique où elles passent. Il peut dessiner des frontières **utiles** ou des frontières **arbitraires**, et rien, dans le dessin lui-même, ne dit laquelle des deux il a faites. La classification non supervisée est ce travail de cartographe appliqué à des clients : on veut des groupes, et on doit apprendre à distinguer un groupe qui **existe** d'un groupe que l'algorithme a **fabriqué**.

Cette section répond à quatre questions, dans l'ordre : *comment mesurer la qualité d'un découpage quand on n'a pas de bonne réponse* (3.1.2), *ce découpage fait-il mieux que le hasard* (3.1.3), *est-il solide si l'on change l'échantillon* (3.1.4), et *que vaut-il contre une vérité connue, dans le cas où on la connaît* (3.1.5). Les deux dernières sous-sections traitent de ce qui décide du résultat avant même de lancer l'algorithme (l'échelle et le choix des variables, 3.1.6) et de la façon d'utiliser les groupes dans une décision (3.1.7).

3.1.1 Le problème : des clients, aucune étiquette

La gérante de la boutique voudrait **segmenter** sa clientèle pour adapter ses envois : ne pas proposer la même chose à une cliente qui commande chaque mois et à une cliente qui n'est pas revenue depuis un an. Elle ne dispose d'aucune étiquette « type de client » ; elle a seulement, pour chaque client, les sept variables de comportement présentées en introduction, centrées et réduites.

Le volume II (section 3.3) a présenté l'outil standard, les **k-means** : on fixe un nombre de groupes k et on cherche la partition qui minimise la variance à l'intérieur des groupes (l'**inertie**). Cet outil pose immédiatement la question de ce chapitre : *quel k ?* Et plus fondamentalement : *les*

groupes trouvés sont-ils réels ? L'inertie ne peut pas répondre, puisqu'elle **décroît toujours** quand k augmente (au maximum, chaque client forme son propre groupe, et l'inertie vaut zéro). Il faut des critères qui pénalisent la complexité.

Trois familles de preuves. Pour juger un découpage non supervisé, on dispose de trois familles d'épreuves, qui se complètent :

1. les critères **internes** : le découpage est-il à la fois *compact* (les points d'un groupe sont proches) et *séparé* (les groupes sont éloignés) ? Ils n'utilisent que les données ;
2. la **stabilité** : si l'on perturbe un peu les données (on en retire 20 %, on en rééchantillonne), retrouve-t-on les mêmes groupes ? Un découpage qui change à chaque tirage décrit le hasard de l'échantillon, pas la clientèle ;
3. les critères **externes** : le découpage retrouve-t-il une structure connue par ailleurs ? Ils exigent une vérité, donc ne servent qu'en laboratoire ou, dans la vie réelle, avec une étiquette partielle ou une **utilité mesurée** (les groupes prédisent-ils le départ des clients ?).

3.1.2 Critères internes : compacité et séparation

Notons $C_1, \dots, C_k$ les groupes, μ_j leurs centres, $\bar{x}$ le centre global, n le nombre de points. Deux quantités décrivent tout découpage :

$$W = \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2 \quad (\text{dispersion intra-groupes}), \quad B = \sum_{j=1}^k n_j \|\mu_j - \bar{x}\|^2 \quad (\text{dispersion inter-groupes}).$$

W mesure la **compacité** ; B mesure la **séparation** des centres. On a toujours $W + B = T$, la dispersion totale, qui ne dépend pas du découpage. Trois critères en découlent.

Le coefficient de silhouette (Rousseeuw, 1987) se calcule point par point. Pour un point i du groupe C :

- $a(i)$ est la **distance moyenne** de i aux autres points de son groupe ;
- $b(i)$ est la distance moyenne de i aux points du **groupe le plus proche** (hors du sien) ;
- $s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$.

Une silhouette proche de 1 signifie « bien rangé » ($a \ll b$) ; proche de 0, « à la frontière » ; négative, « probablement dans le mauvais groupe ». On résume le découpage par la **silhouette moyenne**.

Un exemple entièrement à la main. Reprenons les six paniers moyens (en dizaines d'€) du volume II : 1, 2, 4, 9, 11, 12, découpés en $\{1, 2, 4\}$ et $\{9, 11, 12\}$.

- Point 4 : distances aux autres points de son groupe, 3 et 2, donc $a = 2,5$. Distances aux points de l'autre groupe, 5, 7 et 8, donc $b = 20/3 \approx 6,67$. Silhouette : $1 - 2,5/6,67 = 0,625$.
- Point 1 : $a = (1 + 3)/2 = 2$, $b = (8 + 10 + 11)/3 \approx 9,67$, $s \approx 0,793$. Point 2 : $a = 1,5$, $b = 26/3 \approx 8,67$, $s \approx 0,827$.
- Par symétrie, les points 12, 11 et 9 ont les mêmes silhouettes que 1, 2 et 4. La silhouette moyenne vaut $(0,793 + 0,827 + 0,625)/3 \approx 0,748$.

L'indice de Calinski-Harabasz compare séparation et compacité, en tenant compte du nombre de paramètres :

$$CH(k) = \frac{B/(k-1)}{W/(n-k)}.$$

Plus il est **grand**, meilleur est le découpage. Sur l'exemple : les moyennes sont $7/3$ et $32/3$, la moyenne globale est $6,5$, donc $W = 4,67 + 4,67 = 9,33$ et $B = 3 (7/3 - 6,5)^2 + 3 (32/3 - 6,5)^2 \approx 104,2$, d'où $CH = \frac{104,2/1}{9,33/4} \approx 44,6$.

L'**indice de Davies–Bouldin** mesure, pour chaque groupe, son pire « voisin » : avec s_j la distance moyenne des points du groupe j à son centre,

$$DB = \frac{1}{k} \sum_{j=1}^k \max_{l \neq j} \frac{s_j + s_l}{d(\mu_j, \mu_l)}.$$

Plus il est **petit**, mieux c'est. Sur l'exemple, $s_1 = s_2 \approx 1,11$ et la distance entre les centres vaut $8,33$: $DB = 2,22/8,33 \approx 0,267$.

Ces formules sont celles de `scikit-learn` (les trois résultats à la main coïncident avec la bibliothèque). Voyons ce qu'elles disent sur les 12 000 clients, pour k de 2 à 8. Pour mémoire, nous utilisons la version de la bibliothèque, qui estime la silhouette sur un échantillon de 4 000 clients (le calcul exact coûte n^2 distances).

k	inertie	silhouette	CH	DB
2	62835	0.304	4041	1.193
3	45645	0.331	5040	1.118
4	36775	0.284	5135	1.271
5	32812	0.261	4678	1.311
6	30347	0.244	4241	1.321
7	28830	0.231	3825	1.428
8	27500	0.226	3520	1.334

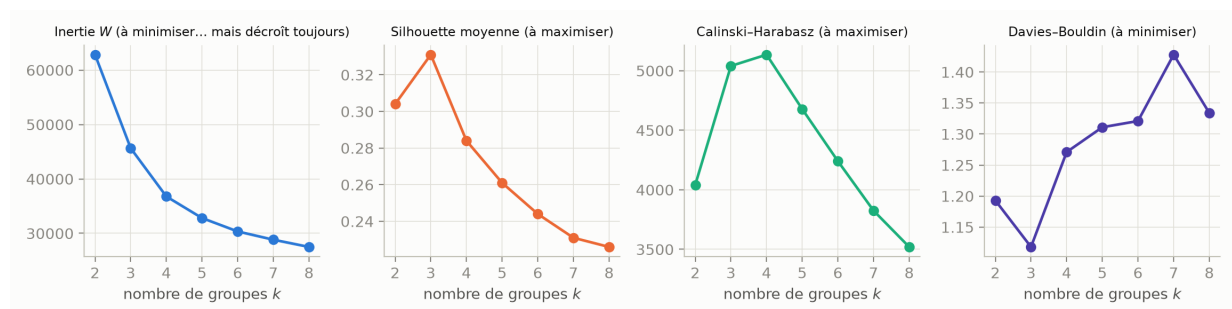


Figure 28: Quatre critères internes en fonction du nombre de groupes k pour les 12 000 clients. L'inertie décroît toujours ; les trois autres critères ne désignent pas le même k .

Les critères ne s'accordent pas, et c'est normal. L'inertie, comme on l'a dit, décroît partout sans coude net. La silhouette et l'indice de Davies–Bouldin préfèrent $k = 3$ (silhouette 0,331, DB 1,118), tandis que l'indice de Calinski–Harabasz est maximal à $k = 4$ (5 135, contre 5 040 à $k = 3$). Chaque critère encode une idée différente de la « bonne » séparation ; aucun ne détient la vérité. La pratique raisonnable consiste à retenir **une plage** de valeurs plausibles ($k = 3$ à 5 ici) et à départager avec les épreuves suivantes.

Piège : prendre le maximum d'un critère pour une réponse. Un critère qui culmine à $k = 3$ dit seulement que, *selon cette définition de la séparation*, trois groupes font mieux que deux ou quatre. Il ne dit pas que trois groupes existent. Les données peuvent former un continuum que tout découpage tranche arbitrairement : la silhouette aura quand même un maximum. C'est l'objet de la sous-section suivante.

3.1.3 Y a-t-il seulement des groupes ? La référence sans structure

Une silhouette de 0,33 est-elle « bonne » ? Cela dépend de ce qu'on obtiendrait **sans aucune structure de groupes**. L'idée est de fabriquer des données de même nature mais **sans groupes**, de les passer dans le même algorithme et de comparer. Deux fabrications sont classiques :

- **la référence par permutation** : on mélange indépendamment chaque colonne. Chaque variable garde sa distribution (asymétrie, valeurs extrêmes), mais les liens entre variables disparaissent ;
- **la référence uniforme** : on tire des points uniformément dans la boîte englobant les données.

k	données réelles	colonnes permutées	boîte uniforme
2	0.304	0.180	0.225
3	0.331	0.190	0.182
4	0.284	0.193	0.158
5	0.261	0.191	0.147

Sur des données **sans aucune structure de groupes**, k-means rend quand même des groupes, avec une silhouette qui n'est pas nulle : de l'ordre de 0,18 à 0,19 pour les colonnes permutées. Les clients réels font mieux (de 0,26 à 0,33 selon k), ce qui est un premier indice de structure. Mais la marge n'est pas écrasante : une silhouette « honnête » se lit **par rapport à sa référence**, jamais dans l'absolu.

La statistique de l'écart (*gap statistic*, Tibshirani, Walther et Hastie, 2001) systématise cette idée. Pour chaque k , on compare le logarithme de la dispersion intra-groupes observée à son espérance sous la référence :

$$\text{Gap}(k) = \mathbb{E}^*[\log W_k] - \log W_k,$$

où l'espérance $\mathbb{E}^*$ est estimée par un petit nombre B de jeux de référence. On retient le plus petit k tel que $\text{Gap}(k) \geq \text{Gap}(k+1) - s_{k+1}$, où s_{k+1} est l'écart-type des $\log W_{k+1}^*$ corrigé par $\sqrt{1 + 1/B}$. L'idée : on s'arrête quand ajouter un groupe n'apporte plus, au-delà du bruit, plus que ce que ferait le hasard.

k	gap (permutation)	gap (uniforme)
1	0.000	0.775
2	0.176	0.818
3	0.359	0.991
4	0.488	1.119
5	0.500	1.171
6	0.494	1.197
7	0.459	1.198
8	0.454	1.212

choix (règle de Tibshirani) : permutation = 5 | uniforme = 6

Avec la référence par permutation, l'écart culmine à $k = 5$ (0,500, juste devant 0,494 pour $k = 6$ et 0,488 pour $k = 4$) et la règle de Tibshirani retient $k = 5$: le gain est net jusqu'à $k = 4$ ou 5, puis s'aplatit. Avec la référence **uniforme**, l'écart est déjà de 0,775 pour un seul groupe et **ne cesse de croître** jusqu'à $k = 8$ (1,212) : il n'y a pas de maximum, et la règle ne s'arrête à $k = 6$ que parce que la courbe s'aplatit par endroits, sans rapport avec une structure. Presque n'importe quelle partition « bat » un nuage uniforme, parce que nos variables sont **asymétriques et corrélées**, ce que le nuage uniforme ne reproduit pas. Le choix de la référence est donc une **hypothèse** : une référence trop naïve rend n'importe quel découpage impressionnant.

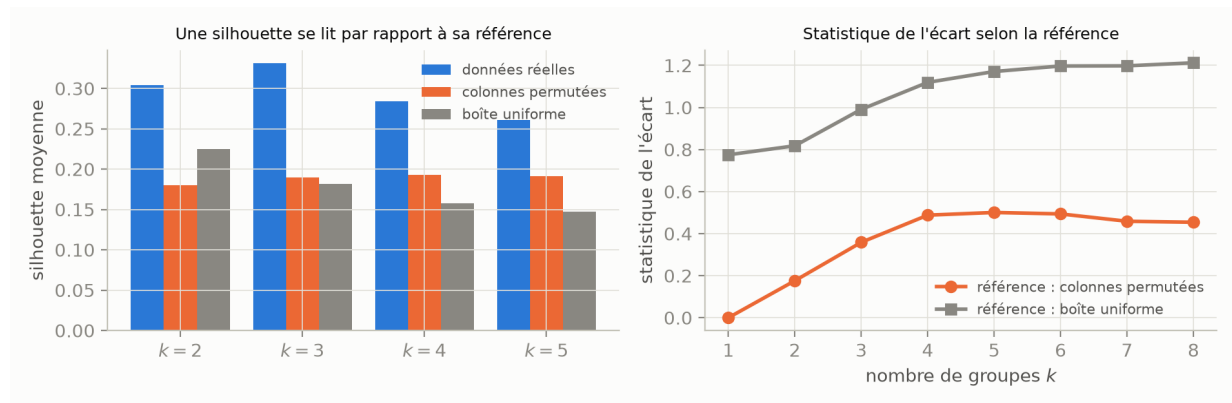


Figure 29: À gauche : silhouette moyenne des données réelles comparée à celle de deux références sans structure. À droite : statistique de l'écart selon la référence choisie ; avec la boîte uniforme, elle ne présente aucun maximum.

3.1.4 Stabilité : les mêmes groupes si l'on change l'échantillon ?

Un découpage solide doit survivre à une perturbation raisonnable des données. Le protocole est simple et s'applique à n'importe quelle méthode :

1. on ajuste le modèle de référence sur **tous** les clients ;
2. on tire B sous-échantillons de 80 % des clients (sans remise) ;
3. sur chacun, on ajuste de nouveau l'algorithme et on compare, sur les clients du sous-échantillon, le découpage obtenu à celui du modèle de référence, avec l'**indice de Rand ajusté** (ARI, défini en 3.1.5) ;
4. on moyenne : un ARI moyen proche de 1 signifie que le découpage se **reproduit**.

k	ARI moyen	écart-type
2	0.729	0.318
3	0.996	0.004
4	0.986	0.007
5	0.981	0.016
6	0.964	0.029
7	0.782	0.120

La lecture est instructive. De $k = 3$ à $k = 6$, les découpages sont remarquablement stables (ARI moyen de 0,996, 0,986, 0,981 puis 0,964) ; la stabilité s'effondre à $k = 7$ (0,782, avec un écart-type de 0,120), signe que k-means « choisit » entre plusieurs découpages équivalents. Le cas $k = 2$ est le plus curieux : un ARI moyen de 0,729 avec un très grand écart-type (0,318). Selon les sous-échantillons, l'algorithme tombe sur **deux découpages en deux groupes différents** ; aucun des deux ne s'impose. La stabilité recoupe donc les critères internes sur un point : elle écarte $k = 2$ (et $k \geq 7$), mais elle **ne suffit pas à choisir** entre 3, 4, 5 et 6.

Stable ne veut pas dire vrai. Un découpage peut être parfaitement reproductible et pourtant arbitraire (partager un nuage uniforme en deux moitiés, toujours au même endroit). La stabilité est une condition **nécessaire** de la validité, pas une condition suffisante : elle se combine avec la comparaison à une référence (3.1.3) et avec l'utilité (3.1.7).

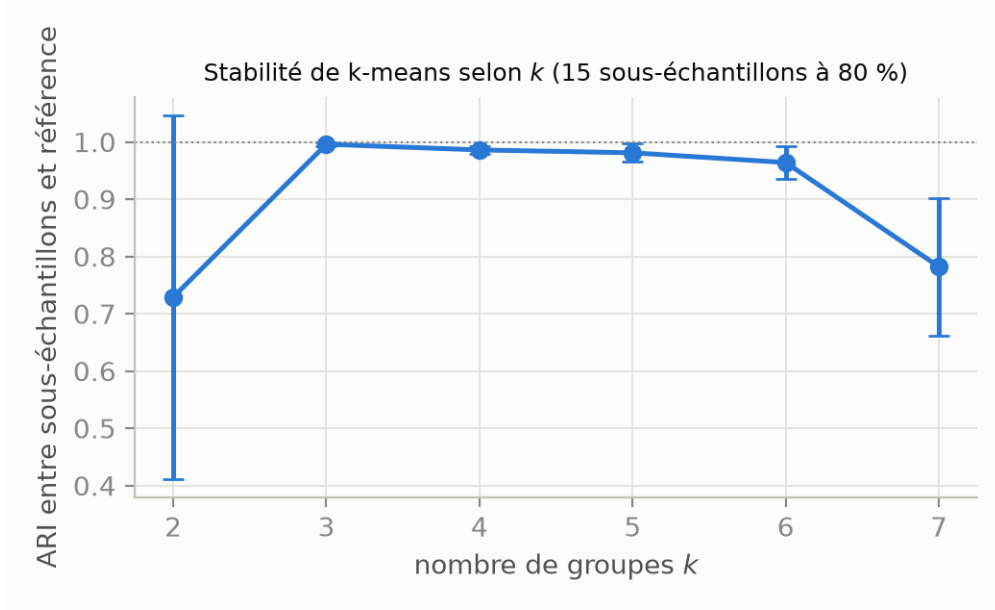


Figure 30: Stabilité de k-means : ARI moyen entre le découpage de référence et ceux de sous-échantillons de 80 %, avec son écart-type, pour k de 2 à 7.

3.1.5 Validation externe : confronter les groupes à une vérité connue

Ici, la simulation nous offre un luxe : nous connaissons la classe latente qui a engendré chaque client (`segment_vrai`). Nous pouvons donc mesurer à quel point chaque découpage la retrouve. Trois mesures, toutes calculées à partir du **tableau croisé** des groupes trouvés et des vrais groupes (n_{ij} est le nombre de clients dans le groupe trouvé i et le vrai groupe j , a_i et b_j les totaux des lignes et des colonnes) :

- la **pureté** : la part de clients appartenant, dans chaque groupe trouvé, à la classe majoritaire ; facile à lire, mais elle **augmente mécaniquement avec k** (à $k = n$, elle vaut 1) ;
- l'**information mutuelle normalisée** (NMI) : $I(U; V)$ divisée par la moyenne des entropies des deux découpages ; elle vaut 0 pour deux découpages indépendants et 1 pour deux découpages identiques ;
- l'**indice de Rand ajusté** (ARI) : il compte les **paires de clients** rangées de la même façon dans les deux découpages (ensemble ou séparés), corrigé de la valeur attendue **au hasard** :

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}.$$

L'ARI vaut 1 pour un accord parfait, **environ 0 pour un découpage aléatoire** (quel que soit k), et peut être négatif.

Un exemple à la main. Dix clients, deux vrais groupes de cinq ; l'algorithme en range quatre du premier et un du second dans son groupe 1, et inversement dans son groupe 2 : $n_{11} = 4, n_{12} = 1, n_{21} = 1, n_{22} = 4$. Alors $\sum_{ij} \binom{n_{ij}}{2} = 6 + 0 + 0 + 6 = 12$, $\sum_i \binom{a_i}{2} = \sum_j \binom{b_j}{2} = 10 + 10 = 20$, $\binom{10}{2} = 45$.

L'espérance au hasard vaut $20 \times 20 / 45 \approx 8,89$, le maximum 20, d'où $\text{ARI} = \frac{12 - 8,89}{20 - 8,89} \approx 0,28$: un accord qui paraît bon (8 clients sur 10 bien rangés) mais que la correction ramène à une valeur modeste, parce qu'avec deux groupes équilibrés le hasard seul rangerait déjà la moitié des clients correctement.

Appliquons ces mesures aux découpages de 3.1.2.

```
k   ARI   NMI
2  0.022 0.121
3  0.288 0.462
4  0.487 0.535
5  0.417 0.496
6  0.397 0.473
7  0.355 0.458
8  0.352 0.470
pureté à k = 4 : 0.799
```

C'est à $k = 4$, le nombre de vrais segments, que l'ARI est maximal (0,487) ; il retombe à 0,288 pour $k = 3$ et à 0,022 pour $k = 2$, découpage qui ne retrouve presque rien du vrai partage. Remarquez à quel point cette mesure externe est plus favorable à $k = 4$ que la silhouette, qui préférerait 3 : **la silhouette récompense la séparation géométrique, pas la fidélité à une structure de décision**. Mais même au meilleur k , l'accord n'est que moyen (ARI 0,487, pureté 0,799). Regardons pourquoi.

	vrai 0	vrai 1	vrai 2	vrai 3
groupe 0	5	22	2197	2
groupe 1	84	2879	23	567
groupe 2	1405	66	107	161
groupe 3	3112	590	20	760

Le tableau croisé montre trois choses. Les « chasseurs de promotions » (vrai segment 2) sont retrouvés presque tels quels : 2 197 sur 2 347 dans le groupe 0. Les « fidèles » (vrai 1) sont bien regroupés (2 879 sur 3 557 dans le groupe 1), mais 590 d'entre eux se retrouvent dans le groupe 3. Surtout, les « occasionnels » (vrai 0) et les « grands paniers » (vrai 3) ne sont pas séparés : le groupe 3 mélange à lui seul 3 112 occasionnels, 760 grands paniers et 590 fidèles, parce que, mesurés par ces sept variables, ces clients ne se distinguent pas : un client « grand panier » qui commande rarement ressemble à un occasionnel. Aucun algorithme, aucun k ne résoudra cela ; le problème est dans les **variables**, pas dans la méthode.

Piège : croire qu'un ARI de 0,5 est un échec de l'algorithme. Un ARI modéré mesure l'écart entre *deux* partitions : la vraie, et celle qu'on peut déduire des variables disponibles. Quand les classes se recouvrent dans l'espace des variables, même un algorithme parfait ne peut pas les séparer. En situation réelle, c'est précisément cet écart qu'il faut anticiper : les groupes que vous trouvez décrivent ce que les **données permettent de distinguer**, pas ce que le monde contient.

3.1.6 Les variables et l'échelle décident des groupes

Avant même de choisir k , deux décisions pèsent plus lourd que l'algorithme.

L'échelle. k-means repose sur la distance euclidienne : une variable en milliers d'€ écrase une variable comprise entre 0 et 1. Comparons trois préparations des mêmes clients.

	préparation	ARI avec la vérité
variables standardisées (montant en log)		0.487
montant en log, sans standardisation		0.081
montant brut, sans standardisation		0.110
standardisées + 5 variables de pur bruit		0.485

Sans standardisation, l'ARI s'effondre (de 0,487 à 0,081 avec le montant en logarithme, 0,110 avec le montant brut) : la variable `recence_jours`, dont l'échelle va de 0 à 365, impose sa géométrie à toutes les autres. Standardiser n'est pas un détail technique, c'est une **décision de modélisation**.

: on affirme que toutes les variables comptent à poids égal.

Le choix des variables. Ajouter cinq variables de pur bruit ne dégrade ici quasiment rien (ARI de 0,485) : avec sept variables informatives, le signal reste dominant. Ce n'est pas une règle générale. Quand le nombre de variables inutiles grandit, les distances se **brouillent** (nous verrons pourquoi en 3.2.1) et les groupes se diluent. Le conseil pratique est de choisir les variables **pour une raison métier** (ce qui distingue les comportements qu'on veut traiter différemment), plutôt que d'y verser « tout ce qu'on a ».

3.1.7 Lire les groupes : de la partition à la décision

Un découpage n'a de valeur que par ce qu'on peut **en faire**. L'étape finale consiste donc à **profiler** les groupes (moyennes des variables, taille) et, surtout, à les confronter à une variable d'**utilité** que l'algorithme n'a pas vue. Ici, le départ à 90 jours (churn_90j) joue ce rôle : il n'a servi ni à construire ni à choisir les groupes.

	part	age	commandes	montant	recence	part_promo	ouverture	churn	depense_6m
groupe									
0	0.19	29.59	4.28	124.03	62.15	0.75	0.57	0.20	52.37
1	0.30	43.58	7.30	372.64	39.02	0.19	0.56	0.01	168.24
2	0.14	35.34	0.11	3.49	363.85	0.19	0.29	0.41	22.16
3	0.37	36.29	2.28	121.05	91.66	0.16	0.28	0.11	57.80
churn global	: 0.14								

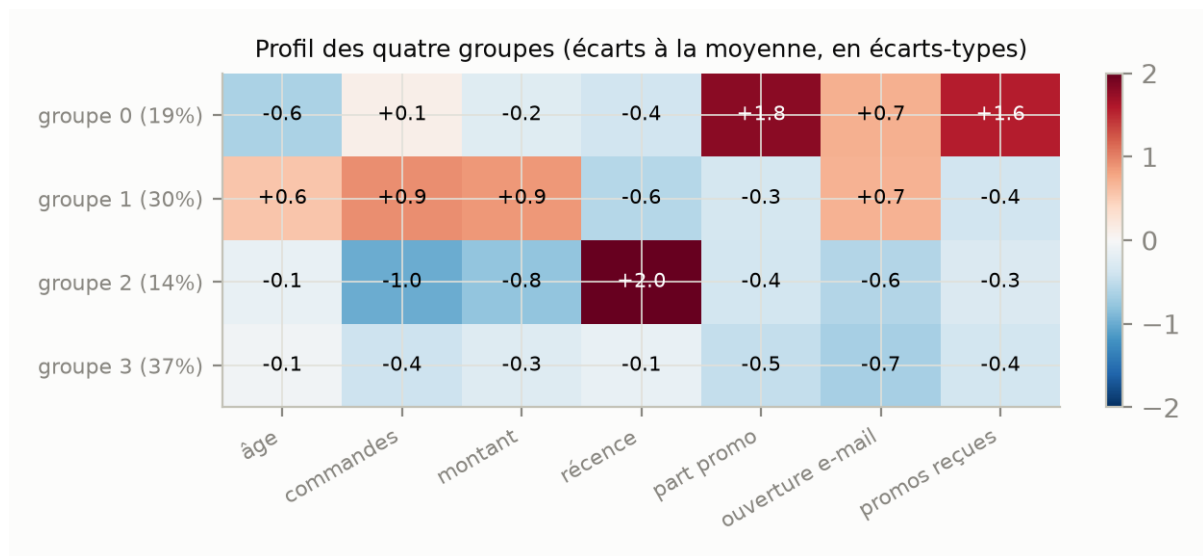


Figure 31: Profil des quatre groupes de k-means : écart de chaque moyenne à la moyenne générale, en écarts-types.

Les groupes se lisent sans effort. Le **groupe 1** (environ 30 % des clients) réunit les **fidèles** : 7,3 commandes par an, un panier annuel de 373 €, une récence de 39 jours, et un départ à 90 jours de seulement 1 %. Le **groupe 2** (environ 14 %) regroupe les **dormants** : à peine 0,11 commande par an, 364 jours depuis la dernière, et un départ à 41 %, trois fois la moyenne de 14 %. Le **groupe 0** (environ 19 %) est celui des **chasseurs de promotions** : 75 % des achats en promotion, neuf promotions reçues, et un départ à 20 %. Le **groupe 3** (environ 37 %) rassemble les **occasionnels réguliers**, au comportement moyen et à 11 % de départ.

Voilà ce que veut dire « un découpage utile » : le départ, que l'algorithme n'a jamais vu, **varie de 1 % à 41 % selon le groupe**. C'est une validation d'une troisième sorte, **par l'utilité**, qui compte plus que n'importe quelle silhouette : si la gérante envoie une offre de réactivation au seul groupe 2, elle

cible les clients dont le risque de départ est de 41 %. (Pour *prédire* le départ individu par individu, on utilisera plutôt un modèle supervisé, chapitre 2 ; la classification sert ici à **comprendre et à cibler**, pas à prédire.)

À retenir (validation d'une classification non supervisée).

- Un algorithme rend toujours des groupes : on ne juge pas un découpage dans l'absolu, mais **par rapport à une référence sans structure** (silhouette comparée, statistique de l'écart avec une référence choisie avec soin).
- Les **critères internes** (silhouette, Calinski–Harabasz, Davies–Bouldin) ne désignent pas toujours le même k : on retient une plage, pas un chiffre.
- La **stabilité** par sous-échantillonnage écarte les découpages qui décrivent le hasard de l'échantillon ; elle est nécessaire, non suffisante.
- Avec une vérité connue (laboratoire), l'**ARI** corrige les accords dus au hasard ; avec une vérité absente, c'est l'**utilité** (les groupes expliquent-ils un comportement non utilisé pour les construire ?) qui tranche.
- L'**échelle** et le **choix des variables** pèsent plus que l'algorithme.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.3, exercices 3.1 à 3.5.

3.2 Réduction de dimension

Intuition. Une photographie de 64 pixels n'a pas besoin de 64 nombres indépendants pour être reconnue : les pixels voisins se ressemblent, les traits se répètent. La réduction de dimension cherche la **description courte** que cachent les données longues : moins de variables, presque la même information. Elle sert à **comprendre** (peut-on dessiner les données ?), à **compresser**, à **débruiter** et à **accélérer** les modèles qui suivent.

Le volume II a présenté la plus célèbre de ces méthodes, l'ACP (section 3.1). Cette section commence par expliquer *pourquoi* réduire est souvent indispensable (la « malédiction de la dimension »), rappelle l'ACP en une page pour mesurer ce qu'elle garde, puis présente les méthodes qui prolongent l'ACP quand ses hypothèses ne tiennent pas : données non linéaires, données positives à interpréter par « parties », données si nombreuses qu'on ne peut plus les centrer, ou si larges qu'on préfère une projection au hasard.

3.2.1 Pourquoi réduire : la malédiction de la dimension

Quand on ajoute des variables, on s'attend à *mieux* décrire les clients. La géométrie dit le contraire dès qu'on raisonne en **distances**, base de nombreuses méthodes (k plus proches voisins, k -means, noyaux). Prenons des points tirés uniformément dans un cube de dimension d , et regardons les distances entre paires de points.

Pour deux points $\mathbf{x}$ et $\mathbf{y}$ dont les coordonnées sont indépendantes et de même loi, le carré de la distance $\|\mathbf{x} - \mathbf{y}\|^2 = \sum_{j=1}^d (x_j - y_j)^2$ est une somme de d termes indépendants : son espérance croît comme d et son écart-type comme $\sqrt{d}$. La **dispersion relative** des distances décroît donc comme $1/\sqrt{d}$: quand d grandit, **toutes les distances deviennent presque égales**, et la notion de « plus proche voisin » perd son sens.

dimension	rapport max/min	écart-type / moyenne
2	1429.48	0.475
5	32.82	0.283
10	7.32	0.193
50	2.11	0.086
200	1.42	0.042
1000	1.18	0.018

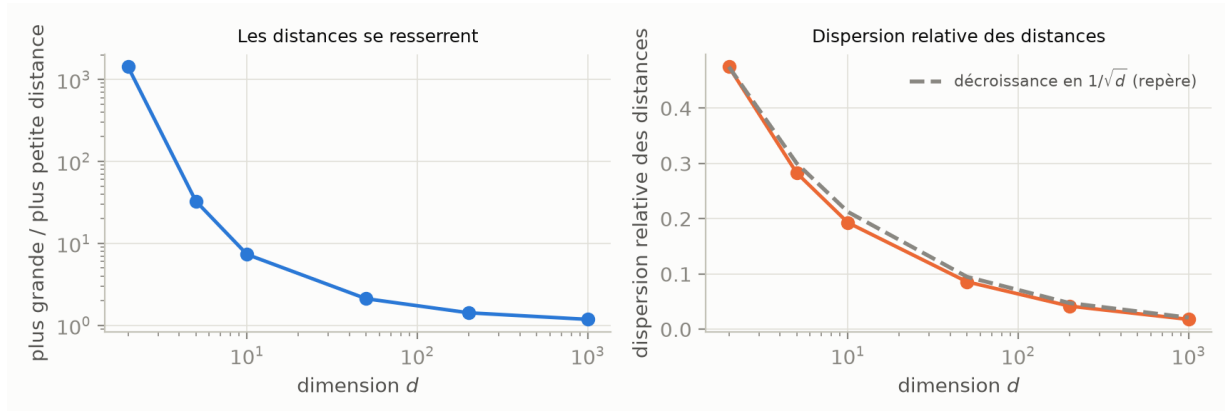


Figure 32: Distances entre 500 points tirés uniformément dans un cube de dimension d : le rapport entre la plus grande et la plus petite distance et la dispersion relative des distances décroissent avec d .

En dimension 2, la plus grande distance est environ 1 400 fois la plus petite ; en dimension 50, ce rapport tombe à 2,1 ; en dimension 1 000, à 1,18 : tous les points sont à peu près à la même distance de tous les autres. Un k-means ou un plus proche voisin qui s'appuie sur ces distances ne sait plus distinguer proche et lointain.

Nuance importante. La malédiction est la plus sévère quand les variables sont **indépendantes et toutes pertinentes**. Les données réelles ont presque toujours une structure bien plus pauvre que leur nombre de colonnes : les chiffres manuscrits ont 64 pixels mais occupent une portion très réduite de l'espace des images possibles. C'est ce qui rend la réduction de dimension **possible** : on ne comprime pas l'espace entier, seulement la petite région où vivent les données.

3.2.2 L'ACP en une page, et ce qu'elle garde

Rappelons l'essentiel (volume II, sections 3.1.3 et 3.1.4). Pour des données centrées $\mathbf{X}$ de n lignes et p colonnes, l'ACP cherche les directions orthogonales de variance maximale ; ce sont les vecteurs propres de la matrice de covariance, et la variance portée par la j -ième direction est la valeur propre λ_j . Garder les m premières composantes revient à projeter chaque observation sur le sous-espace de dimension m le plus proche des données.

Ce que l'on **perd** s'exprime exactement. D'après le théorème d'**Eckart-Young**, la meilleure approximation de rang m de $\mathbf{X}$ (au sens de la somme des carrés des erreurs) est obtenue par la SVD tronquée, et l'erreur quadratique de reconstruction vaut la somme des valeurs propres **écartées** :

$$\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 = (n-1) \sum_{j>m} \lambda_j, \quad \text{soit en proportion : } 1 - \frac{\lambda_1 + \dots + \lambda_m}{\lambda_1 + \dots + \lambda_p}.$$

L'erreur *relative* de reconstruction est donc exactement **un moins la part de variance expliquée**

: c'est ce qui rend le tableau de bord de l'ACP (le graphique cumulé des valeurs propres) aussi utile. Appliquons-le aux 1 797 chiffres manuscrits ($p = 64$).

composantes	variance gardée	erreur relative	précision 5-ppv
2	0.285	0.715	0.603
5	0.545	0.455	0.888
10	0.738	0.262	0.937
20	0.894	0.106	0.959
30	0.959	0.041	0.962
40	0.988	0.012	0.961

précision 5-ppv sur les 64 pixels bruts : 0.963

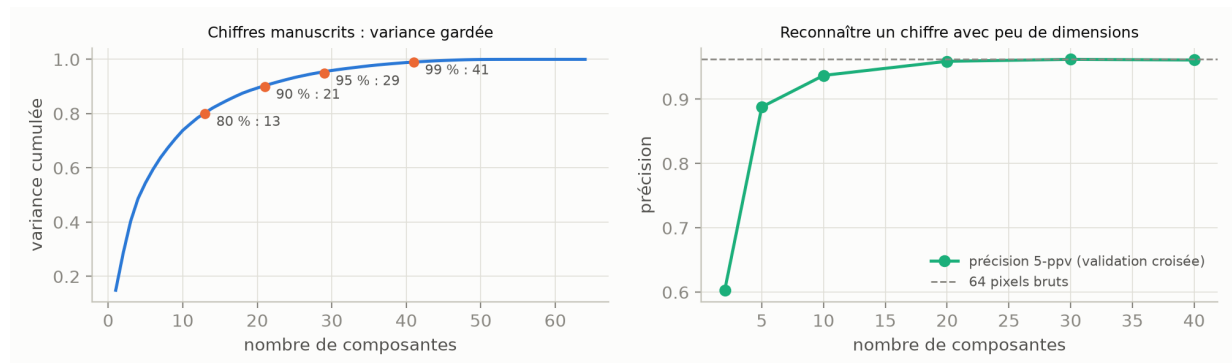


Figure 33: Gauche : variance cumulée des composantes principales des chiffres manuscrits et nombre de composantes nécessaires pour 80, 90, 95 et 99 %. Droite : précision d'un classifieur des 5 plus proches voisins selon le nombre de composantes gardées, comparée aux 64 pixels bruts.

Trois lectures. (i) Il faut 13 composantes pour garder 80 % de la variance, 21 pour 90 %, 29 pour 95 % et 41 pour 99 % : une compression par trois sans perte visible. (ii) L'erreur relative de reconstruction coïncide bien avec « un moins la variance gardée » (à 10 composantes : 0,738 de variance gardée et 0,262 d'erreur, ce que dit Eckart-Young). (iii) Surtout, **l'information utile pour reconnaître un chiffre survit à la compression** : avec 20 composantes, la précision d'un classifieur des 5 plus proches voisins est de 0,959 contre 0,963 sur les 64 pixels bruts, alors qu'avec 2 composantes elle tombe à 0,603. Le graphique des reconstructions le montre : à 2 composantes on devine à peine la forme, à 10 on lit le chiffre, à 30 on ne voit plus la différence.

3.2.3 Quand la structure n'est pas linéaire : l'ACP à noyau

L'ACP ne trouve que des directions **linéaires**. Sur deux cercles concentriques, aucune droite ne sépare le cercle intérieur du cercle extérieur : la première composante principale, qui est une direction, les mélange.

L'**ACP à noyau** (*kernel PCA*, Schölkopf et coll., 1998) contourne cette limite par une idée remarquable : au lieu de projeter les données elles-mêmes, on les envoie d'abord dans un espace de très grande dimension où elles deviennent séparables, puis on y fait une ACP, **sans jamais calculer explicitement** cet espace. Il suffit de connaître les **produits scalaires** entre points dans cet espace, que fournit une fonction appelée **noyau**, par exemple le noyau gaussien :

$$k(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|^2).$$

La méthode a quatre étapes : (1) calculer la matrice de Gram $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$; (2) la **centrer** dans l'espace transformé, $\tilde{K} = K - \mathbf{1}K - K\mathbf{1} + \mathbf{1}K\mathbf{1}$ où $\mathbf{1}$ est la matrice $n \times n$ dont toutes les entrées valent

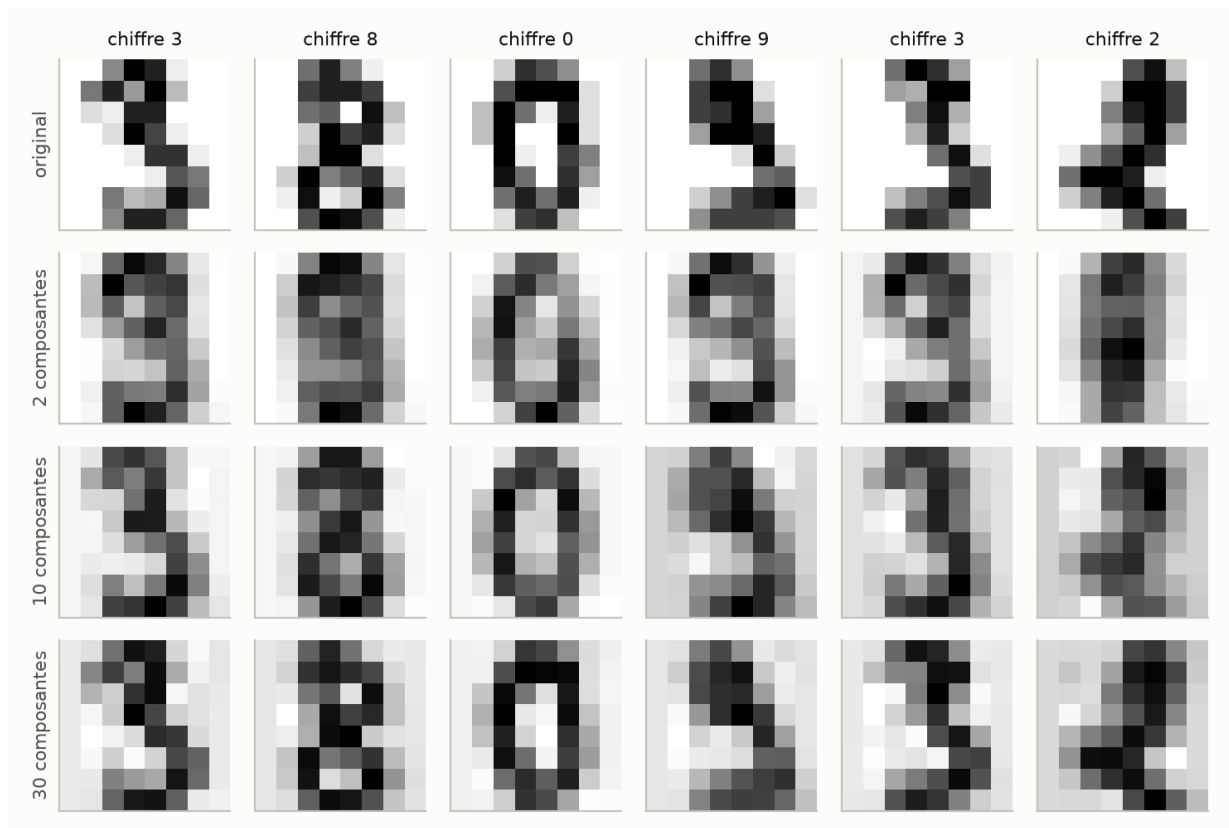


Figure 34: Quatre lignes : six chiffres originaux, puis leur reconstruction avec 2, 10 et 30 composantes principales.

$1/n$; (3) calculer ses valeurs propres λ_j et vecteurs propres $\mathbf{v}_j$; (4) les coordonnées du point i sur la j -ième composante sont $\sqrt{\lambda_j} v_{j,i}$.

précision d'une séparation par seuil sur la 1re composante : {'ACP': 0.498, 'ACP à noyau': 1.0}

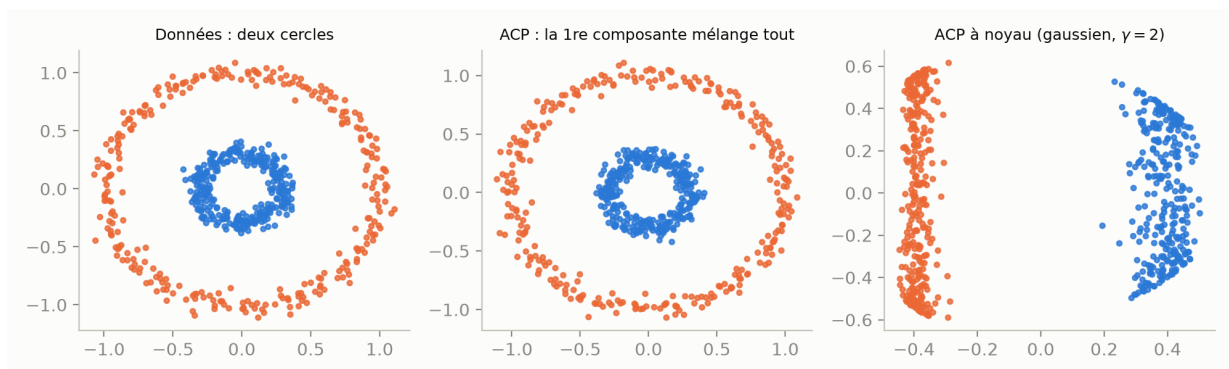


Figure 35: Deux cercles concentriques : les données, leur projection par ACP linéaire, puis par ACP à noyau gaussien, qui sépare les deux cercles le long de la première composante.

Une séparation par un simple seuil sur la première composante a une précision de 0,498 avec l'ACP (le hasard) et de 1,0 avec l'ACP à noyau. Deux mises en garde : le paramètre γ du noyau **décide** de ce que la méthode voit (trop grand, chaque point est isolé ; trop petit, le noyau devient presque linéaire) et il se règle par validation, comme n'importe quel hyperparamètre (section 1.5) ; et, contrairement à l'ACP, la **reconstruction** n'est pas directe (retrouver un point de l'espace d'origine à partir d'une position dans l'espace transformé est un problème d'« image réciproque

» approchée).

3.2.4 SVD tronquée : quand on ne peut pas centrer

L'ACP exige de **centrer** les données, ce qui détruit la **parcimonie** : une matrice de 100 000 documents et de 50 000 mots, presque toute faite de zéros, devient dense une fois centrée, donc impossible à stocker. La **SVD tronquée** (TruncatedSVD) applique la même décomposition en valeurs singulières **sans centrer** ; elle conserve la parcimonie et reste calculable. Quand les colonnes sont déjà centrées, elle coïncide avec l'ACP. Appliquée à des tableaux de comptages de mots, elle porte le nom d'**analyse sémantique latente** (LSA).

variance expliquée par 10 composantes : SVD tronquée 0.732 | ACP 0.738

Sur les chiffres, dont les pixels ne sont pas centrés, les deux méthodes donnent presque la même part de variance expliquée par 10 composantes (0,732 contre 0,738) : l'écart, minime, tient à l'absence de centrage.

3.2.5 La factorisation non négative : décrire par parties

Les composantes principales ont des coefficients positifs et négatifs : une image se décrit comme un mélange où certaines directions *retranchent* de l'information, ce qui rend les composantes difficiles à lire (« moitié d'un chiffre, moins un autre »). Quand les données sont **positives** (pixels, comptages, montants), on peut exiger une description **purement additive**.

La **factorisation non négative** (NMF, Lee et Seung, 1999) approche la matrice des données V ($n \times p$) par un produit de deux matrices à entrées positives ou nulles :

$$V \approx WH, \quad W \geq 0 \ (n \times m), \quad H \geq 0 \ (m \times p),$$

en minimisant $\|V - WH\|_F^2$. Chaque ligne de H est une « partie » (un motif de base) ; chaque observation est une **somme** de parties pondérées par sa ligne de W . Les mises à jour multiplicatives de Lee et Seung, $H \leftarrow H \circ \frac{W^T V}{W^T W H}$ et $W \leftarrow W \circ \frac{V H^T}{W H H^T}$ (produit et quotient *terme à terme*), préservent la positivité et diminuent l'erreur à chaque pas.

NMF, 10 composantes : précision 5-ppv 0.84 | ACP, 10 composantes : 0.937

Les dix parties sont des **motifs de traits** que l'on peut additionner pour composer un chiffre : c'est la lisibilité qu'on cherche. Elle a un prix : avec 10 composantes, un classifieur des 5 plus proches voisins atteint 0,840 sur la NMF contre 0,937 sur l'ACP. La NMF n'est donc **pas** une compression meilleure ; c'est une **représentation plus lisible**. Elle sert quand on veut interpréter (thèmes d'un corpus, profils d'achat, spectres).

3.2.6 Les projections aléatoires et le lemme de Johnson–Lindenstrauss

Dernière idée, qui surprend : pour réduire la dimension, **on peut projeter au hasard**. Choisissons une matrice R de k lignes et p colonnes dont les entrées sont des tirages indépendants d'une loi $\mathcal{N}(0, 1/k)$, et remplaçons chaque observation x par Rx . Cela paraît absurde (aucune information sur les données n'a servi à construire R), et pourtant :

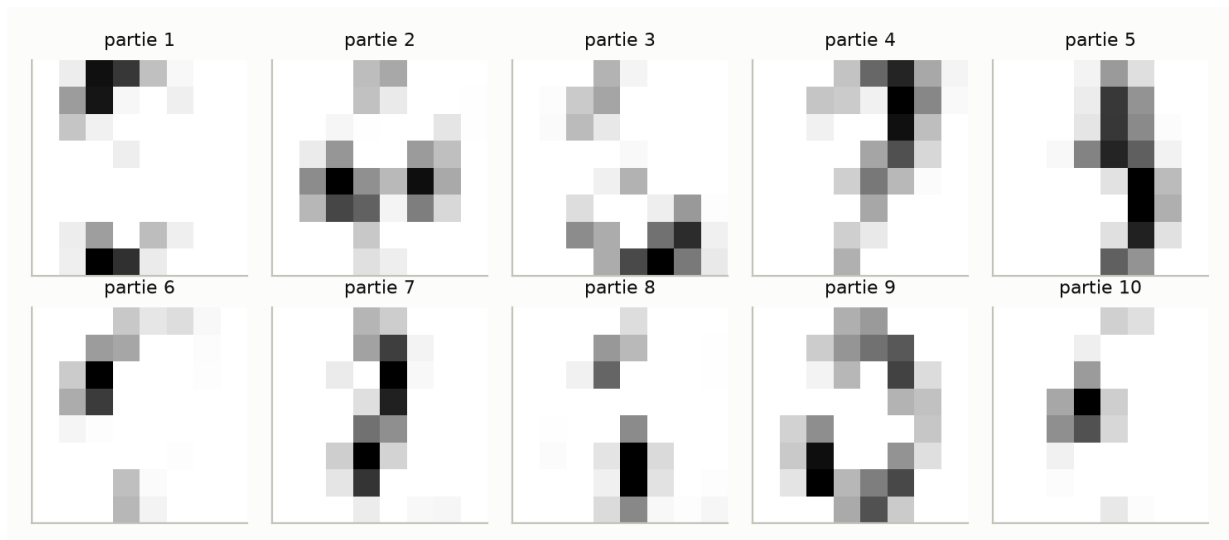


Figure 36: Les dix « parties » apprises par la factorisation non négative sur les chiffres manuscrits : chacune est un motif de traits que l'on additionne.

Lemme de Johnson–Lindenstrauss (1984). Pour tout $0 < \varepsilon < 1$ et tout ensemble de n points de $\mathbb{R}^p$, il existe une application linéaire vers $\mathbb{R}^k$ avec

$$k \geq \frac{4 \ln n}{\varepsilon^2/2 - \varepsilon^3/3}$$

qui préserve toutes les distances à un facteur près : $(1 - \varepsilon)\|\mathbf{x} - \mathbf{y}\|^2 \leq \|f(\mathbf{x}) - f(\mathbf{y})\|^2 \leq (1 + \varepsilon)\|\mathbf{x} - \mathbf{y}\|^2$ pour tous les couples. De plus, une projection gaussienne aléatoire convient avec une probabilité élevée.

Idée de la démonstration. Pour un vecteur fixe $\mathbf{u}$ et R gaussienne, $\|R\mathbf{u}\|^2/\|\mathbf{u}\|^2$ suit la loi χ_k^2/k , d'espérance 1 et de variance $2/k$: elle se **concentre** autour de 1 quand k grandit, avec une queue exponentielle. On applique ensuite une **borne de l'union** sur les $n(n-1)/2$ différences $\mathbf{x}_i - \mathbf{x}_j$: si la probabilité d'erreur de chacune est de l'ordre de n^{-2} , la probabilité qu'une seule échoue reste faible, ce qui impose k de l'ordre de $\ln n/\varepsilon^2$.

Deux remarques déroutantes. Le résultat est **indépendant de p** : seul le nombre de points n compte (de façon logarithmique). Et la borne est **pessimiste** : elle garantit *toutes* les paires. Mesurons ce qui se passe réellement sur nos 1 797 chiffres ($p = 64$).

k	ratio moyen	écart-type	5 % - 95 %	paires à ± 20 %
5	0.944	0.286	0.49 - 1.43	48.9%
10	0.978	0.202	0.65 - 1.32	66.7%
20	0.978	0.148	0.74 - 1.23	81.7%
40	0.963	0.096	0.81 - 1.12	95.0%
64	0.968	0.075	0.85 - 1.09	98.7%
100	0.970	0.062	0.87 - 1.07	99.7%
200	0.965	0.046	0.89 - 1.04	100.0%

bornes de la bibliothèque ($n = 1797$) : epsilon 0,5 -> 359 | epsilon 0,2 -> 1729

La borne théorique exige $k \geq 359$ pour $\varepsilon = 0,5$ et $k \geq 1729$ pour $\varepsilon = 0,2$, c'est-à-dire **plus que les 64 pixels d'origine** : elle n'est utile qu'en très grande dimension. Dans la pratique, la conservation est bien meilleure : avec $k = 40$, 95,0 % des distances sont conservées à ± 20 %, avec $k = 64$ 98,7 %, et avec $k = 200$ toutes. La moyenne des rapports est légèrement inférieure à 1 (entre 0,94 et 0,98 selon k) : c'est la distance, et non son carré, qui est mesurée, et l'inégalité de

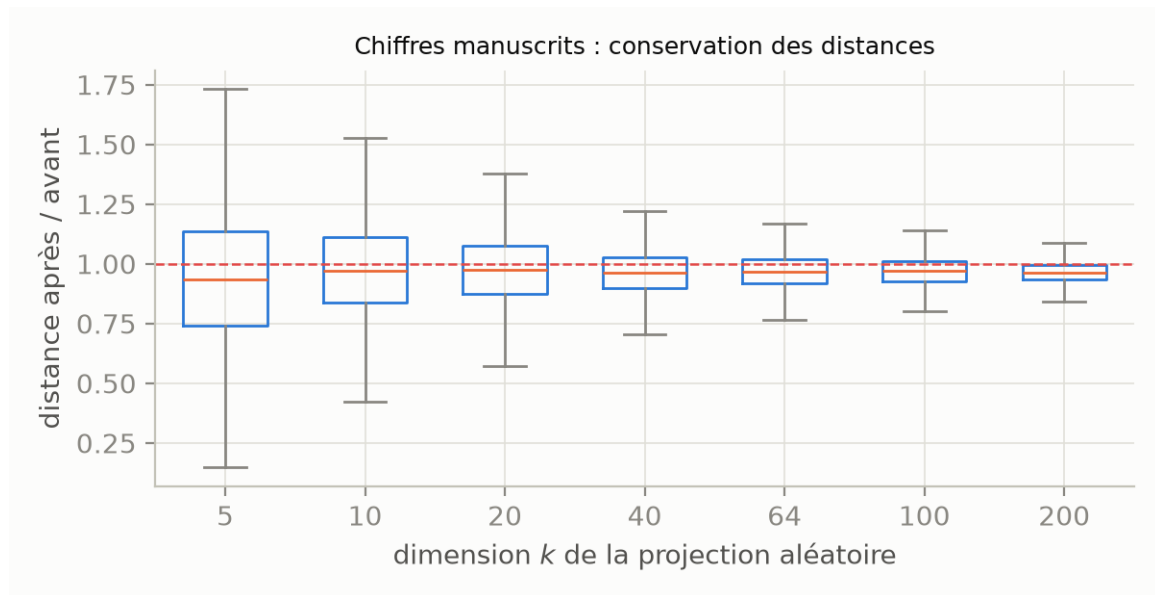


Figure 37: Rapport entre la distance après et avant projection aléatoire, pour 1 797 chiffres et des projections de dimension k de 5 à 200 : les boîtes se resserrent autour de 1 quand k augmente.

Jensen fait $\mathbb{E}\sqrt{X} \leq \sqrt{\mathbb{E}X}$.

Quand les projections aléatoires servent-elles ? Quand p est énorme (dizaines de milliers de colonnes), parce qu'elles sont **instantanées**, ne dépendent d'aucune donnée (on peut projeter un nouveau point sans ré-ajuster), et conservent les distances nécessaires aux méthodes de voisinage. Pour comprendre des données de dimension modérée, l'ACP reste préférable : elle choisit les directions **qui comptent**.

3.2.7 Combien de dimensions garder ? Et l'idée de variété

Il n'existe pas de règle universelle ; on combine trois indices.

1. **La variance gardée** : un seuil (80 %, 90 %, 95 %) ou un coude sur le graphique cumulé. Rapide, mais le seuil est arbitraire.
2. **La reconstruction** : l'erreur relative de reconstruction vaut un moins la variance gardée (Eckart–Young) ; on la compare à ce qu'on tolère.
3. **L'utilité en aval** : on mesure la performance de la tâche finale (la précision d'un classifieur, la qualité d'un regroupement) en fonction du nombre de dimensions, **par validation croisée** (section 1.2), et on s'arrête quand elle plafonne. Sur les chiffres, la précision plafonne vers 20 composantes : c'est le critère qui compte, parce qu'il mesure ce qu'on veut en faire.

Le volume II (section 3.1.6) mentionne aussi l'analyse parallèle, qui compare les valeurs propres à celles de données sans structure : c'est la même logique de « référence sans structure » que celle de 3.1.3.

L'hypothèse de variété. Pourquoi une compression est-elle possible ? Parce que les données réelles vivent souvent au voisinage d'une **variété** de faible dimension : une surface (courbe, repliée) plongée dans un espace de grande dimension. Le célèbre « rouleau suisse » en est l'illustration : des points sur une feuille enroulée, dans un espace à 3 dimensions, alors que la feuille est de dimension 2. L'ACP, qui ne sait projeter que sur un plan, **écrase les couches du rouleau les unes sur les autres** ; une méthode qui respecte les **distances le long de la surface** (Isomap, qui déroule la variété à partir du graphe des plus proches voisins) le déplie. Les

méthodes t-SNE et UMAP (section 3.4) reposent sur cette même hypothèse.

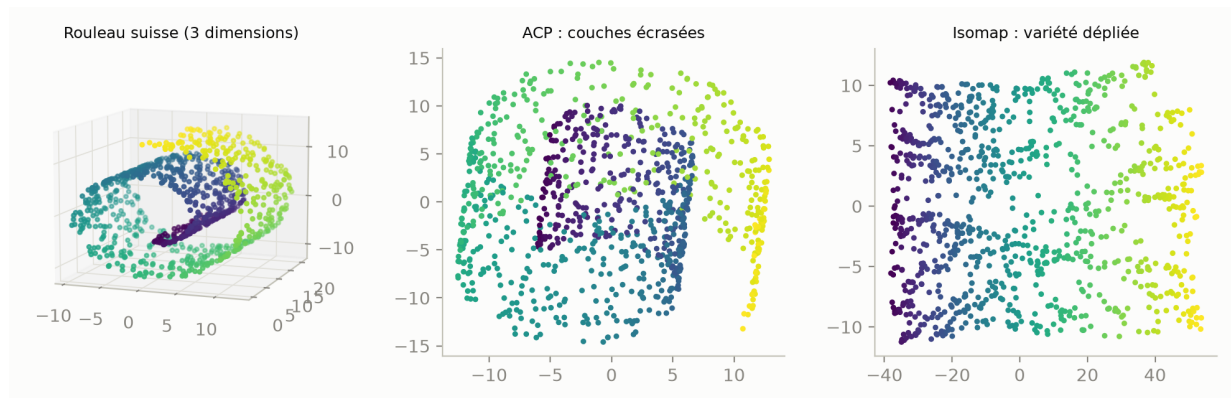


Figure 38: Le rouleau suisse en 3 dimensions, sa projection par ACP (les couches se recouvrent) et son dépliage par Isomap (la couleur, qui repère la position le long du rouleau, varie régulièrement).

3.2.8 Réduire avant de classer : sur les clients de la boutique

Revenons aux 12 000 clients. Une pratique répandue consiste à **réduire** les sept variables par ACP avant de lancer k-means. Est-ce neutre ?

composantes gardées	variance gardée		ARI avec la vérité (k=4)	
2	0.641		0.480	
3	0.762		0.489	
4	0.857		0.502	
5	0.932		0.479	
7	1.000		0.487	
	CP1	CP2	CP3	
âge	0.04	-0.43	0.80	
commandes	0.47	-0.20	-0.10	
montant (log)	0.52	-0.27	-0.22	
récence	-0.50	0.15	0.22	
part promo	0.25	0.58	0.13	
ouverture e-mail	0.39	0.16	0.46	
promos reçues	0.21	0.57	0.14	

La première composante (37 % de la variance) oppose la **récence** à l'**activité** (nombre de commandes, montant, ouverture des e-mails) : c'est un axe « client actif contre client endormi ». La deuxième (27 %) regroupe `part_achats_promo` et `nb_promos_recues_12m` : un axe « sensibilité aux promotions ». Garder 4 composantes (85,7 % de la variance) donne un ARI de 0,502 avec la vérité cachée, contre 0,487 avec les 7 variables : la réduction **ne détruit pas** la structure et la débruite même un peu. Avec seulement 2 composantes, on retombe à 0,480. La leçon : la réduction n'est pas neutre, et son effet se **mesure** avec les mêmes épreuves que la classification (3.1), au lieu de se supposer.

À retenir (réduction de dimension).

- En grande dimension, les distances se resserrent (dispersion relative en $1/\sqrt{d}$) : les méthodes de voisinage souffrent, d'où l'intérêt de réduire.
- L'ACP garde la variance, et l'erreur relative de reconstruction vaut **un moins la variance gardée** (Eckart–Young) ; le bon nombre de composantes se choisit par l'**utilité en aval**, validée par validation croisée.
- L'ACP à **noyau** capte des structures non linéaires ; la **SVD tronquée** évite de centrer

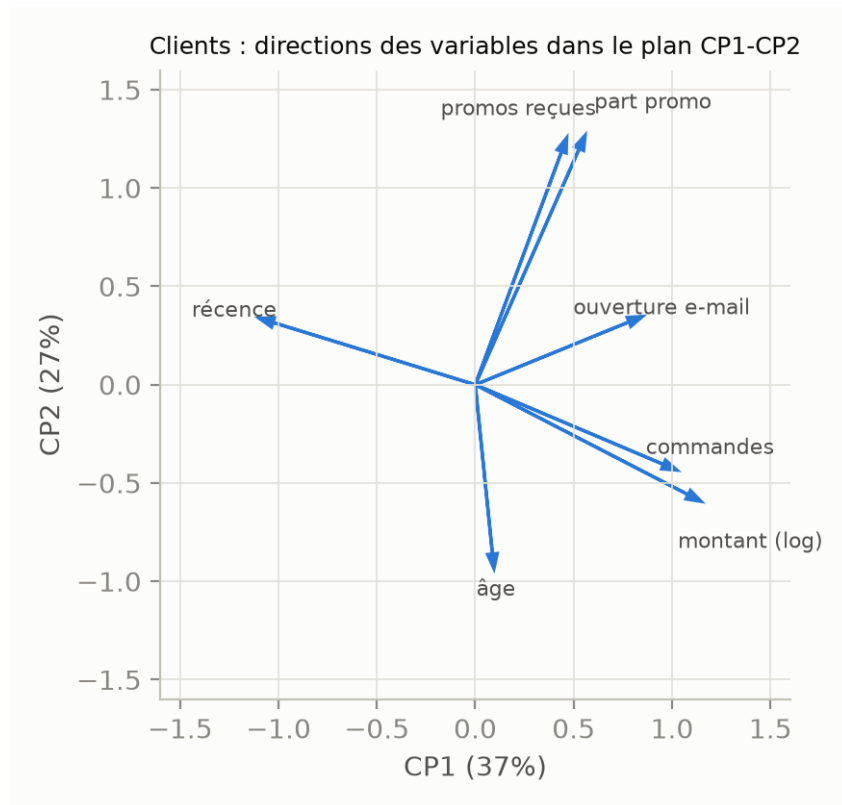


Figure 39: Cercle des corrélations de l'ACP sur les sept variables de comportement : la première composante oppose récence et activité, la deuxième regroupe les variables liées aux promotions.

(données parcimonieuses) ; la **NMF** donne des composantes **additives et lisibles** ; les **projections aléatoires** conservent les distances (Johnson-Lindenstrauss) à moindre coût quand p est énorme.

- Les données réelles vivent souvent près d'une **variété** de faible dimension : c'est ce qui rend la réduction possible, et ce que t-SNE et UMAP cherchent à déplier.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.4 et 3.8, exercices 3.6 à 3.8.

3.3 ⊕ Pour aller plus loin : DBSCAN, classification hiérarchique, mélanges gaussiens

Section optionnelle. Elle prolonge 3.1 avec trois méthodes qui lèvent chacune une hypothèse de k-means : que les groupes soient des **boules** (DBSCAN et les liens hiérarchiques), qu'ils soient **bien tranchés** (les mélanges gaussiens donnent des appartenances « floues »). On peut la sauter sans perdre le fil du chapitre.

k-means suppose, sans le dire, que chaque groupe est une région **convexe** et à peu près **sphérique** autour de son centre : il découpe l'espace en cellules de Voronoï. Quand les groupes ont une forme allongée, courbe ou imbriquée, cette hypothèse trahit les données. Le cas d'école est celui des **deux lunes** : deux croissants entrelacés. Un humain y voit tout de suite deux groupes ; k-means, qui doit couper l'espace par une droite, les mélange.

3.3.1 DBSCAN : les groupes comme régions denses

DBSCAN (Ester et coll., 1996) définit un groupe comme une **région dense** séparée d'autres régions denses par des zones vides. Il n'a pas besoin de connaître le nombre de groupes, accepte des formes quelconques, et surtout il a une réponse naturelle pour les points qui n'appartiennent à aucun groupe : ce sont du **bruit**.

Deux paramètres seulement : un rayon ε et un effectif minimal m (`min_samples`). Pour un point x , son **voisinage** est l'ensemble des points à distance au plus ε de lui (lui-même compris). Les points se classent alors en trois types :

- un point **cœur** a au moins m points dans son voisinage ;
- un point **de bordure** n'est pas un cœur, mais se trouve dans le voisinage d'un cœur ;
- un point **de bruit** n'est ni l'un ni l'autre.

Un groupe est l'ensemble des points obtenus en partant d'un cœur et en **suivant de proche en proche les cœurs voisins** (deux cœurs sont connectés s'ils sont à distance $\leq \varepsilon$), auxquels on ajoute leurs points de bordure.

Un exemple à la main, en dimension 1. Neuf valeurs : 1; 1,2; 1,4; 1,85; 2,5; 5; 5,1; 5,3; 9, avec $\varepsilon = 0,5$ et $m = 3$.

- Le voisinage de 1 est {1; 1,2; 1,4} : trois points, c'est un **cœur**. De même pour 1,2 (voisinage {1; 1,2; 1,4}) et pour 1,4 (voisinage {1; 1,2; 1,4; 1,85}).
- Le point 1,85 n'a que deux points dans son voisinage (1,4 et lui-même) : ce n'est pas un cœur ; mais il est à 0,45 du cœur 1,4, c'est un point **de bordure** : il rejoint le premier groupe.
- Le point 2,5 est à 0,65 du point le plus proche : ni cœur, ni voisin d'un cœur, c'est du **bruit**.
- 5, 5,1 et 5,3 forment un second groupe, chacun étant un cœur. Enfin 9 est du bruit.

valeur	type	groupe
1.00	cœur	0
1.20	cœur	0
1.40	cœur	0
1.85	bordure	0
2.50	bruit	-1
5.00	cœur	1
5.10	cœur	1
5.30	cœur	1
9.00	bruit	-1

Le tableau de la bibliothèque confirme le raisonnement : deux groupes (étiquettes 0 et 1), six cœurs, un point de bordure et deux points de bruit (étiquette -1).

Choisir ε et m . L'effectif minimal m se prend en général égal à $2p$ (p étant la dimension), ou plus quand les données sont bruitées. Le rayon ε se lit sur le **graphique des k -distances** : pour chaque point, on calcule la distance à son m -ième plus proche voisin, on trie ces distances, et on cherche le « coude » de la courbe, au-delà duquel les distances grimpent brusquement (les points isolés). Sur les deux lunes, avec $m = 5$, la distance au 5e voisin est de 0,057 pour la moitié des points et de 0,128 pour 99 % d'entre eux.

epsilon	groupes	points de bruit	ARI
0.05	35	266	0.021
0.10	2	4	0.987
0.15	2	0	1.000
0.20	2	0	1.000
0.30	1	0	0.000

La sensibilité à ε est nette. Trop petit (0,05), il **fragmente** les lunes en 35 groupes minuscules et déclare 266 points « bruit » (ARI 0,021). Dans l'intervalle 0,1 à 0,2, il retrouve les deux lunes (ARI 0,987 à 1,0). Trop grand (0,3), les deux lunes **fusionnent** en un seul groupe (ARI nul). C'est la

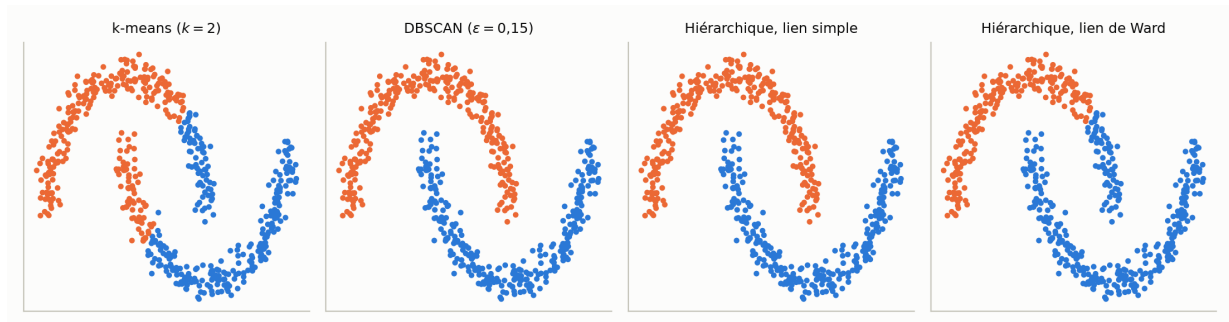


Figure 40: Deux lunes entrelacées : k-means, DBSCAN avec $\varepsilon = 0,15$, classification hiérarchique à lien simple et à lien de Ward. Les points gris seraient du bruit.

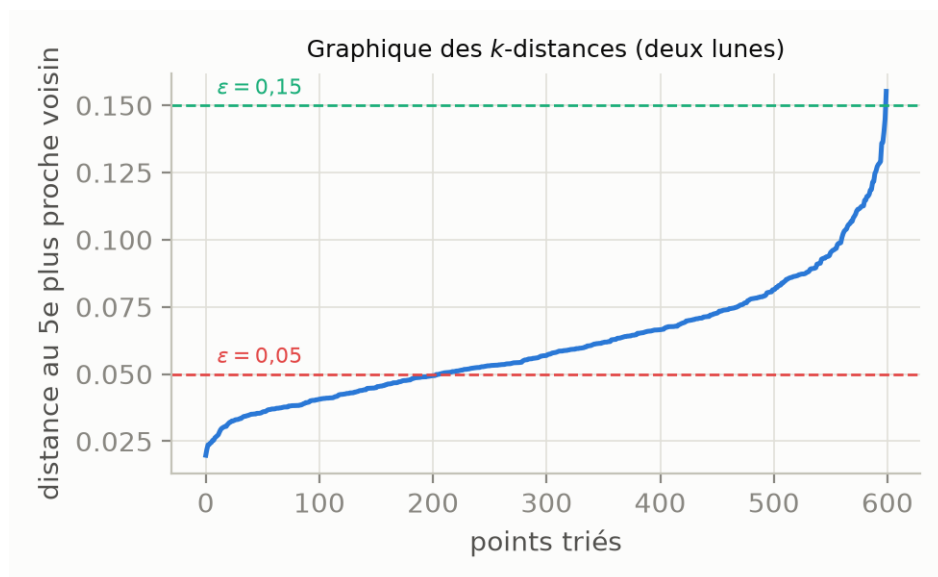


Figure 41: Graphique des k -distances des deux lunes (distance au 5e voisin, triée) avec deux valeurs de ε testées.

limite principale de DBSCAN : un seul rayon pour toute la population, donc une **densité unique**. Quand les groupes ont des densités très différentes, aucun ε ne convient.

DBSCAN ne marche pas partout. Sur les clients de la boutique, il échoue : les distributions sont asymétriques et le nuage forme un **continuum** de densités variables. Avec $m = 14$ ($= 2p$), $\varepsilon = 0,8$ donne 3 groupes mais **16,5 %** de bruit et un ARI de 0,214 avec la vérité (contre 0,487 pour k-means) ; $\varepsilon = 0,5$ déclare 86 % de bruit ; $\varepsilon = 1,2$ fusionne tout. La méthode excelle sur des formes géométriques bien séparées, pas sur des profils de comportement à frontières floues.

epsilon	groupes	part de bruit	ARI
0.5	7	0.860	-0.016
0.8	3	0.165	0.214
1.2	2	0.008	-0.002

3.3.2 La classification hiérarchique, revue par les liens

Le volume II (section 3.3.4) a présenté la classification ascendante hiérarchique : on part de n groupes d'un point et on fusionne à chaque étape les deux groupes les plus proches, ce qui produit un **dendrogramme** que l'on coupe à la hauteur voulue. Reste à définir la distance entre **deux groupes** : c'est le **lien** (*linkage*).

- **Lien simple** : distance minimale entre un point de chaque groupe. Il suit les « chaînes » de points proches, et retrouve donc les formes allongées, mais il est sensible au **chaînage** (un pont de quelques points fusionne deux groupes distincts).
- **Lien complet** : distance maximale. Il produit des groupes compacts de diamètre limité.
- **Lien moyen** : distance moyenne entre toutes les paires.
- **Lien de Ward** : on fusionne les deux groupes dont la fusion fait **le moins augmenter la variance intra W** ; c'est l'analogue hiérarchique de k-means, qui donne des groupes en boules.

Sur les deux lunes, l'effet du lien est spectaculaire (voir le tableau : le lien simple retrouve parfaitement les croissants, l'ARI du lien de Ward est modeste).

méthode	ARI
k-means (k=2)	0.252
DBSCAN (epsilon=0,15)	1.000
hiérarchique, lien simple	1.000
hiérarchique, lien complet	0.426
hiérarchique, lien moyen	0.557
hiérarchique, lien de Ward	0.557

Le lien simple atteint un ARI de 1,0 ; le lien moyen et celui de Ward, 0,557 ; le lien complet, 0,426 ; k-means, 0,252. **Aucun lien n'est « le bon »** : chacun encode une idée de la forme d'un groupe. Une limite pratique commune : la classification hiérarchique doit stocker les distances entre toutes les paires, soit un coût en mémoire de l'ordre de n^2 : acceptable pour quelques milliers de points, impossible pour nos 12 000 clients.

3.3.3 Les mélanges gaussiens : des appartenances « floues »

k-means affecte chaque client à **un** groupe, de façon tranchée. Or un client à mi-chemin entre fidèle et occasionnel n'appartient pas vraiment à l'un ou à l'autre. Les **mélanges gaussiens** (*Gaussian mixture models*, GMM) modélisent cela en supposant que les données sont tirées d'un **mélange de lois normales** :

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k), \quad \pi_k \geq 0, \quad \sum_k \pi_k = 1.$$

Chaque groupe k a son centre μ_k , sa **forme** (la matrice de covariance Σ_k : boule, ellipse, ellipse inclinée) et son **poids** π_k . La grande différence : pour un client $\mathbf{x}_i$, le modèle ne dit pas « groupe 2 », il donne les **probabilités d'appartenance** (ou *responsabilités*)

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_{l=1}^K \pi_l \mathcal{N}(\mathbf{x}_i | \mu_l, \Sigma_l)}.$$

Les paramètres se choisissent en maximisant la vraisemblance, mais la somme dans le logarithme rend le calcul direct impossible. On le contourne par l'algorithme **EM** (*Expectation–Maximization*, Dempster, Laird et Rubin, 1977), qui alterne deux étapes simples :

- **étape E** (espérance) : avec les paramètres actuels, calculer les responsabilités γ_{ik} ;
- **étape M** (maximisation) : remettre à jour les paramètres en traitant les γ_{ik} comme des poids :

$$N_k = \sum_i \gamma_{ik}, \quad \mu_k = \frac{1}{N_k} \sum_i \gamma_{ik} \mathbf{x}_i, \quad \Sigma_k = \frac{1}{N_k} \sum_i \gamma_{ik} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^\top, \quad \pi_k = \frac{N_k}{n}.$$

D'où viennent ces formules ? Dérivons la mise à jour de μ_k . La log-vraisemblance est $\ell = \sum_i \ln \sum_k \pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k)$. Comme $\partial_{\mu_k} \mathcal{N}(\mathbf{x} | \mu_k, \Sigma_k) = \mathcal{N} \Sigma_k^{-1} (\mathbf{x} - \mu_k)$, on obtient

$$\frac{\partial \ell}{\partial \mu_k} = \sum_i \underbrace{\frac{\pi_k \mathcal{N}(\mathbf{x}_i | \mu_k, \Sigma_k)}{\sum_l \pi_l \mathcal{N}(\mathbf{x}_i | \mu_l, \Sigma_l)}}_{\gamma_{ik}} \Sigma_k^{-1} (\mathbf{x}_i - \mu_k).$$

Annuler ce gradient donne $\sum_i \gamma_{ik} (\mathbf{x}_i - \mu_k) = 0$, soit $\mu_k = \sum_i \gamma_{ik} \mathbf{x}_i / N_k$: **une moyenne pondérée par les responsabilités**. Les autres formules se démontrent de même (avec un multiplicateur de Lagrange pour la contrainte $\sum_k \pi_k = 1$). On montre ensuite (par l'inégalité de Jensen, qui fournit une minoration de ℓ que l'étape E rend exacte au point courant) que **chaque tour EM ne peut pas faire baisser la vraisemblance** ; l'algorithme converge donc vers un maximum **local**, ce qui impose, comme pour k-means, de relancer plusieurs fois.

k-means est un cas limite d'EM. Si tous les groupes sont des boules de même variance σ^2 et de même poids, et que l'on fait tendre $\sigma \rightarrow 0$, les responsabilités deviennent des 0 ou des 1 (le groupe le plus proche l'emporte) : l'étape E devient l'affectation de Lloyd, l'étape M devient le recalcul des centres. Un mélange gaussien est un k-means « avec des nuances ».

Un tour d'EM à la main. Quatre valeurs, 0; 1; 4; 6, deux groupes. Pour que le calcul reste faisable à la main, on simplifie : les deux groupes ont la même variance $\sigma^2 = 1$ et le même poids 1/2, et **seules les moyennes** sont à estimer ; on part de $\mu_1 = 0$ et $\mu_2 = 3$. La responsabilité du groupe 1 pour une valeur x s'écrit $\gamma_1(x) = 1/(1 + e^{-[(x-\mu_2)^2 - (x-\mu_1)^2]/2})$.

- $x = 0$: $(9 - 0)/2 = 4,5$, donc $\gamma_1 = 1/(1 + e^{-4,5}) \approx 0,989$.
- $x = 1$: $(4 - 1)/2 = 1,5$, donc $\gamma_1 \approx 0,818$.
- $x = 4$: $(1 - 16)/2 = -7,5$, donc $\gamma_1 \approx 0,0006$; $x = 6$: $\gamma_1 \approx 0$.

Étape M : $\mu_1 = \frac{0 \times 0,989 + 1 \times 0,818 + 4 \times 0,0006}{0,989 + 0,818 + 0,0006} \approx 0,454$ et, avec $\gamma_2 = 1 - \gamma_1$, $\mu_2 = \frac{1 \times 0,182 + 4 \times 0,9994 + 6 \times 1}{0,011 + 0,182 + 0,9994 + 1} \approx 4,642$. Les deux centres se sont rapprochés de leurs vraies valeurs ; deux ou trois tours suffisent.

```
responsabilités du groupe 1 au tour 1 : [9.890e-01 8.176e-01 6.000e-04 0.000e+00]
tour 0 : mu = [0. 3.]
tour 1 : mu = [0.454 4.642]
tour 2 : mu = [0.504 4.998]
tour 3 : mu = [0.506 5.001]
```

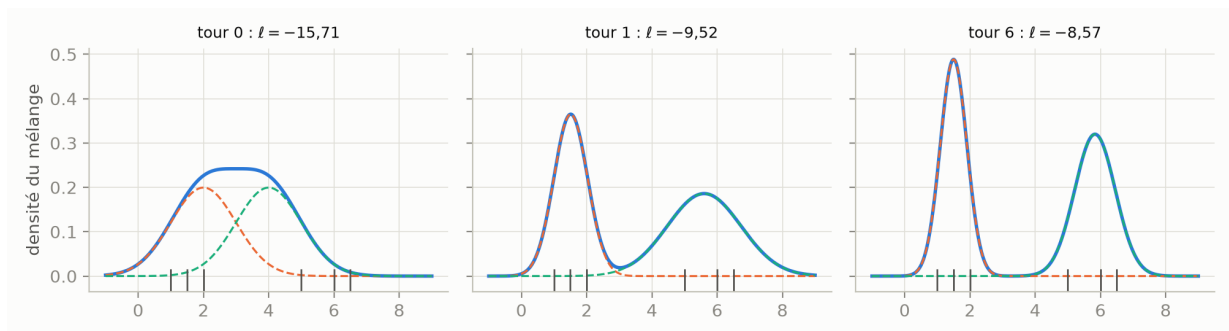


Figure 42: Trois étapes de l'algorithme EM sur six valeurs : le mélange (bleu) et ses deux composantes (pointillés) au départ, après un tour, puis à la convergence ; la log-vraisemblance ℓ augmente.

La bibliothèque confirme les nombres de la main : après un tour, $\mu = (0,454; 4,642)$. Le graphique montre un EM **complet** (moyennes, écarts-types et poids) sur six valeurs : la vraisemblance augmente à chaque tour, ce que garantit la théorie.

Choisir le nombre de composantes : le critère BIC. Contrairement à k-means, un GMM possède une **vraisemblance**, donc un critère de sélection de modèle : le **BIC** (*Bayesian Information Criterion*), $\text{BIC} = -2 \ln \hat{L} + q \ln n$, où q est le nombre de paramètres libres. Il récompense l'ajustement et pénalise la complexité (à minimiser). Sur les clients, avec $p = 7$ variables et une covariance complète, chaque composante coûte $7 + 28 = 35$ paramètres (plus un poids).

k	BIC	ARI avec la vérité
1	207243	0.000
2	180819	0.336
3	117562	0.276
4	106300	0.503
5	104420	0.469
6	102459	0.407
7	101505	0.358
8	101462	0.358

covariance	BIC	ARI avec la vérité
full	106300	0.503
diag	114379	0.477
spherical	197934	0.497
tied	181073	0.506

part de clients avec probabilité maximale $< 0,7$: 0.07

Le BIC chute violemment jusqu'à $k = 4$ (de 117 562 à 106 300 entre $k = 3$ et $k = 4$), puis ne gagne plus que de petits montants (104 420 à $k = 5$, 102 459 à $k = 6$, et à peine 43 de $k = 7$ à $k = 8$) : le

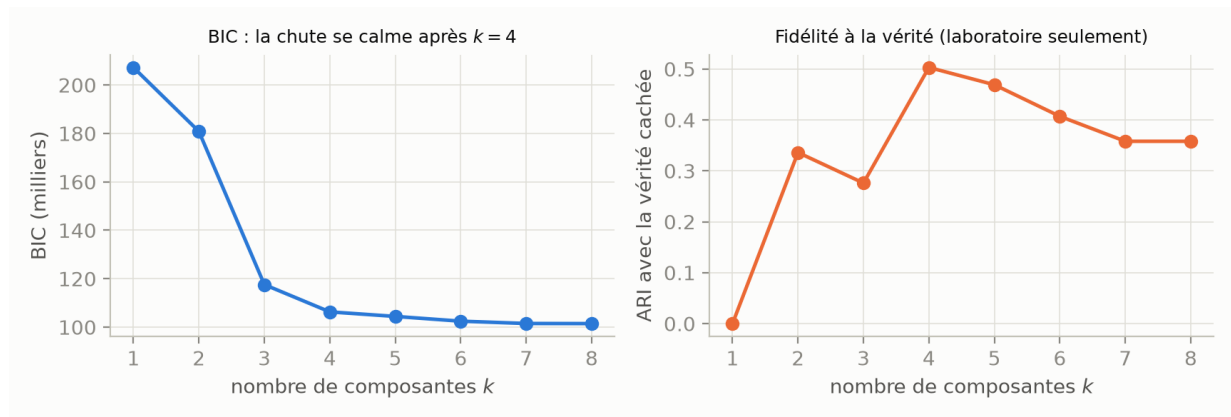


Figure 43: BIC d'un mélange gaussien à covariance complète selon le nombre de composantes (à gauche) et ARI avec la vérité cachée (à droite).

coude est à 4. Le BIC continue pourtant de décroître lentement, parce que les variables (comptages, montants) ne sont **pas exactement gaussiennes** : de petites composantes supplémentaires servent à épouser les asymétries. À $k = 4$, le mélange à covariance complète atteint un ARI de 0,503, un peu mieux que k-means (0,487).

Le tableau des types de covariance est une mise en garde. La covariance **sphérique** (des boules, comme k-means) a un BIC très mauvais (197 934) mais un ARI quasi identique (0,497) ; la covariance **complète** a le meilleur BIC (106 300). Le BIC juge **la qualité de l'ajustement de la densité**, l'ARI juge **la qualité de la partition** : ce sont deux objectifs différents, qui ne se classent pas toujours de la même façon.

Enfin, l'intérêt propre du mélange : 7,0 % des clients ont une probabilité maximale d'appartenance inférieure à 0,7 (et 16,8 % sous 0,9), c'est-à-dire qu'ils sont réellement « entre deux groupes ». Cette incertitude, qu'un k-means tranché cache, est une information utile : on peut réserver les messages très ciblés aux clients dont l'appartenance est sûre.

À retenir (méthodes au-delà de k-means).

- **DBSCAN** : groupes = régions denses, forme libre, bruit explicite ; mais un seul rayon ε (une seule densité), et inadapté à des données en continuum.
- **Classification hiérarchique** : le **lien** (simple, complet, moyen, Ward) définit la forme des groupes ; coût mémoire en n^2 .
- **Mélanges gaussiens** : appartenances **probabilistes**, formes elliptiques, estimation par EM (la vraisemblance ne baisse jamais, optimum local) ; le **BIC** guide le choix de K .
- Aucune méthode n'est supérieure en soi : on choisit selon la forme probable des groupes, puis on **valide** avec les épreuves de 3.1.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.5 et 3.6, exercices 3.9 à 3.11.

3.4 ⊕ Pour aller plus loin : t-SNE et UMAP

Section optionnelle. Elle présente les deux méthodes les plus utilisées pour **dessiner** des données de grande dimension. Surtout, elle explique ce que ces dessins permettent de conclure, et ce qu'ils ne permettent **pas** : c'est la partie la plus importante, et celle que la littérature rapide oublie.

L'ACP projette sur un plan en gardant les directions de grande variance : excellente pour comprendre la structure d'ensemble, mais elle écrase les groupes fins quand ils se distinguent par des différences *locales*. Les chiffres manuscrits en sont un exemple : sur un plan d'ACP, les dix chiffres se recouvrent largement. Les méthodes de **plongement non linéaire** visent un autre objectif : que **deux points voisins dans l'espace d'origine restent voisins sur le dessin**. Elles reposent sur l'hypothèse de variété vue en 3.2.7.

3.4.1 t-SNE : conserver les voisinages

t-SNE (van der Maaten et Hinton, 2008) fonctionne en deux temps.

(1) **Mesurer les voisinages dans l'espace d'origine.** Pour chaque point $\mathbf{x}_i$, on définit la probabilité que $\mathbf{x}_j$ soit « choisi comme voisin » de $\mathbf{x}_i$ par une courbe gaussienne centrée en $\mathbf{x}_i$:

$$p_{ji} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)}{\sum_{l \neq i} \exp(-\|\mathbf{x}_i - \mathbf{x}_l\|^2 / 2\sigma_i^2)}, \quad p_{ij} = \frac{p_{ji} + p_{ij}}{2n}.$$

La largeur σ_i est choisie **point par point** pour que la distribution $p_{\cdot|i}$ ait une **perplexité** donnée, $2^{H(p_{\cdot|i})}$, où H est l'entropie de Shannon : la perplexité se lit comme le **nombre effectif de voisins** de chaque point (5 à 50 en pratique). Un point dans une région dense reçoit un σ_i petit, un point isolé un σ_i grand : chacun a toujours « à peu près le même nombre de voisins ».

(2) **Retrouver ces voisinages sur un dessin.** On place chaque point à une position $\mathbf{y}_i$ du plan, avec des similarités

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}},$$

qui utilisent une loi de Student à un degré de liberté (une loi de Cauchy) au lieu d'une gaussienne : ses **queues lourdes** laissent de la place aux groupes, sur un dessin où l'espace manque (c'est le « problème d'entassement » que la gaussienne résout mal). On cherche les positions qui rendent q proche de p , en minimisant la **divergence de Kullback–Leibler**

$$\text{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \ln \frac{p_{ij}}{q_{ij}}$$

par descente de gradient, dont le gradient a une forme simple :

$$\frac{\partial \text{KL}}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij}) (\mathbf{y}_i - \mathbf{y}_j) (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}.$$

Chaque point est **attiré** par ses vrais voisins ($p_{ij} > q_{ij}$) et **repoussé** par les faux ($q_{ij} > p_{ij}$). Le problème n'est pas convexe : le résultat dépend de l'initialisation et du hasard.

3.4.2 UMAP : un graphe de voisins, puis un dessin

UMAP (McInnes, Healy et Melville, 2018) suit la même logique avec d'autres outils, issus de la topologie. On construit d'abord un **graphe pondéré des plus proches voisins** : chaque point est relié à ses n_{voisins} plus proches voisins avec un poids qui décroît avec la distance, **recalé sur la distance au voisin le plus proche** (ainsi chaque point est fortement relié à au moins un autre, quelle que soit la densité locale) ; on symétrise le graphe par une union « floue ». On cherche ensuite des positions dans le plan dont le graphe de similarités ressemble à ce graphe, en minimisant une **entropie croisée** par descente de gradient stochastique, avec un échantillonnage de paires éloignées pour la répulsion. Deux paramètres principaux : `n_neighbors` (taille du voisinage local ; petit = détails fins, grand = vue d'ensemble) et `min_dist` (à quel point les points d'un même groupe peuvent se serrer sur le dessin).

Par rapport à t-SNE, UMAP est **plus rapide** (surtout au-delà de quelques milliers de points) et sait **projeter un nouveau point** sur un dessin existant (méthode `transform`), ce que t-SNE ne sait pas faire.

3.4.3 Les chiffres manuscrits, vus par trois méthodes

Pour mesurer objectivement la qualité d'un plongement, on utilise la **fiabilité** (*trustworthiness*) : pour chaque point, on regarde ses k plus proches voisins **sur le dessin** et on pénalise ceux qui n'étaient pas parmi ses k plus proches voisins dans l'espace d'origine, d'autant plus que leur rang d'origine est lointain :

$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^n \sum_{j \in U_i(k)} (r(i, j) - k),$$

où $U_i(k)$ est l'ensemble des « faux voisins » du point i et $r(i, j)$ le rang de j parmi les voisins de i dans l'espace d'origine. Elle vaut 1 pour un plongement qui ne crée aucun faux voisin. Nous travaillons sur un échantillon de 800 chiffres (les méthodes sont lentes) et prenons $k = 10$.

méthode	fiabilité (k=10)	fidélité des distances entre chiffres
ACP	0.827	0.801
t-SNE, perplexité 5	0.987	0.710
t-SNE, perplexité 30	0.991	0.819
t-SNE, perplexité 100	0.984	0.836
UMAP, 5 voisins	0.985	0.475
UMAP, 15 voisins	0.987	0.674
UMAP, 50 voisins	0.984	0.701

aires de l'enveloppe convexe de chaque chiffre (t-SNE, perplexité 30) : de 85 à 915 | effectifs de 76 à 90

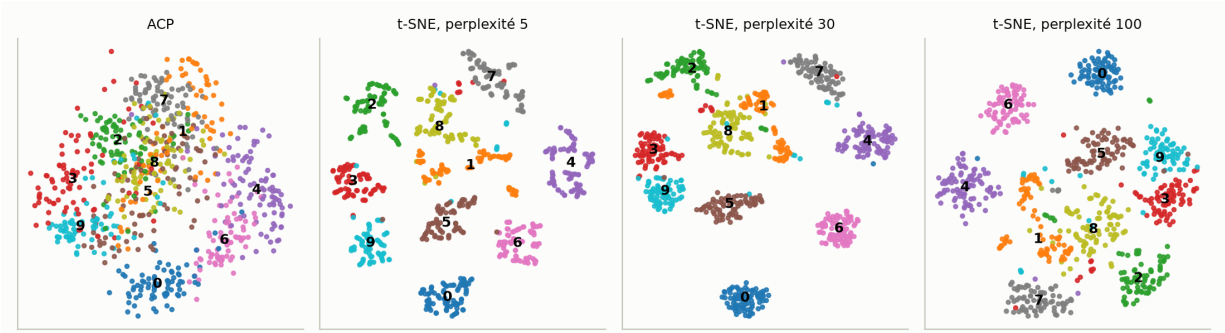


Figure 44: Les 800 chiffres manuscrits en deux dimensions par ACP et par t-SNE (perplexité 5, 30 et 100). Chaque couleur est un chiffre ; le numéro marque la médiane du chiffre.

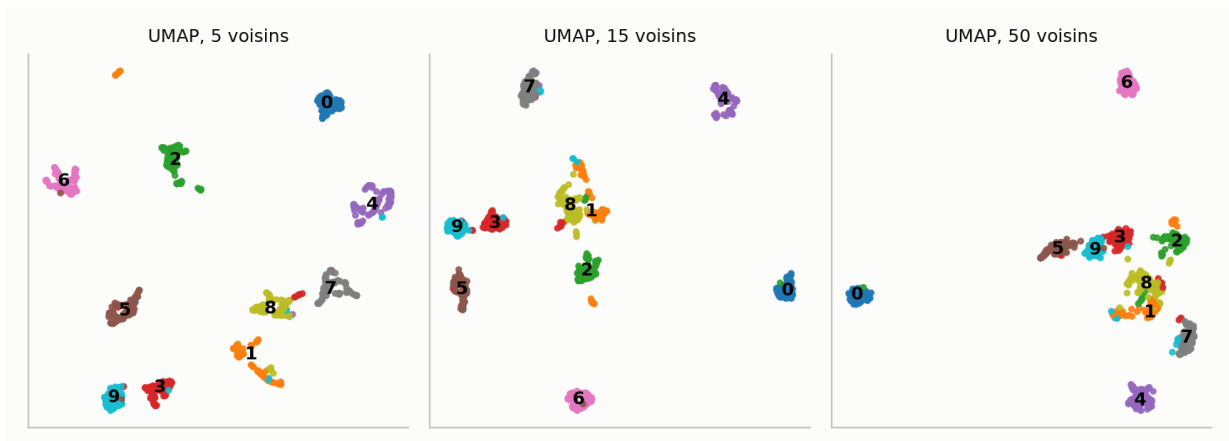


Figure 45: Les mêmes chiffres par UMAP, avec 5, 15 et 50 voisins.

La différence avec l'ACP saute aux yeux : sur le plan de l'ACP, les chiffres se recouvrent ; avec t-SNE et UMAP, **dix îlots** bien séparés apparaissent. La fiabilité le confirme : 0,827 pour l'ACP, de 0,984 à 0,991 pour t-SNE (le meilleur réglage est la perplexité 30, avec 0,991), de 0,984 à 0,987 pour UMAP. Ces méthodes **préservent remarquablement les voisinages locaux**. Mais regardez la colonne de droite du tableau.

3.4.4 Ce qu'on n'a pas le droit de lire sur un dessin

La dernière colonne du tableau mesure la **fidélité des distances entre chiffres** : on calcule le centre de chaque chiffre dans l'espace d'origine puis sur le dessin, et on mesure à quel point les deux classements de distances entre centres concordent (corrélation de rangs de Spearman). Surprise : l'ACP, qui a pourtant un moins bon voisinage local, obtient 0,801, **autant que** les meilleurs t-SNE (0,836 pour la perplexité 100, 0,819 pour 30) et nettement plus qu'UMAP à 15 voisins (0,674) ou 5 voisins (0,475). t-SNE à perplexité 5 (0,710) est moins fidèle que l'ACP. **Ces méthodes préservent le voisinage local, pas la géométrie globale**. D'où quatre interdicts.

1. **Ne pas lire les distances entre groupes.** Deux îlots proches sur le dessin ne sont pas forcément proches dans l'espace d'origine ; deux îlots lointains peuvent l'être aussi peu que d'autres.
2. **Ne pas lire la taille des groupes.** Les algorithmes dilatent les régions denses et contractent les régions clairsemées. Les aires de l'enveloppe convexe de chaque chiffre sur le dessin t-SNE (perplexité 30) varient de 85 à 915 unités (un rapport de plus de 10) pour des classes de tailles comparables (76 à 90 chiffres).
3. **Ne pas croire aux formes fines.** Un groupe allongé, un « pont » entre deux îlots dépendent des hyperparamètres : en changeant la perplexité de 5 à 100 (figure), la disposition d'ensemble et les îlots se réarrangent.
4. **Ne pas oublier que le résultat dépend du hasard.** L'optimisation n'est pas convexe. Avec trois initialisations aléatoires différentes, les dessins diffèrent, même si le fond reste stable.

graine	fiabilité	ARI des 10 groupes avec les vrais chiffres
0	0.991	0.811
1	0.990	0.807
2	0.990	0.793

ARI entre les groupes des graines 0 et 1 : 0.987 | 0 et 2 : 0.923

Les trois dessins sont différents (les îlots ne sont pas aux mêmes endroits), mais les **voisinages** restent les mêmes : fiabilité quasi identique (0,990 à 0,991) et, si on classe les points de chaque dessin en 10 groupes avec k-means, les groupes sont presque les mêmes d'une graine à l'autre

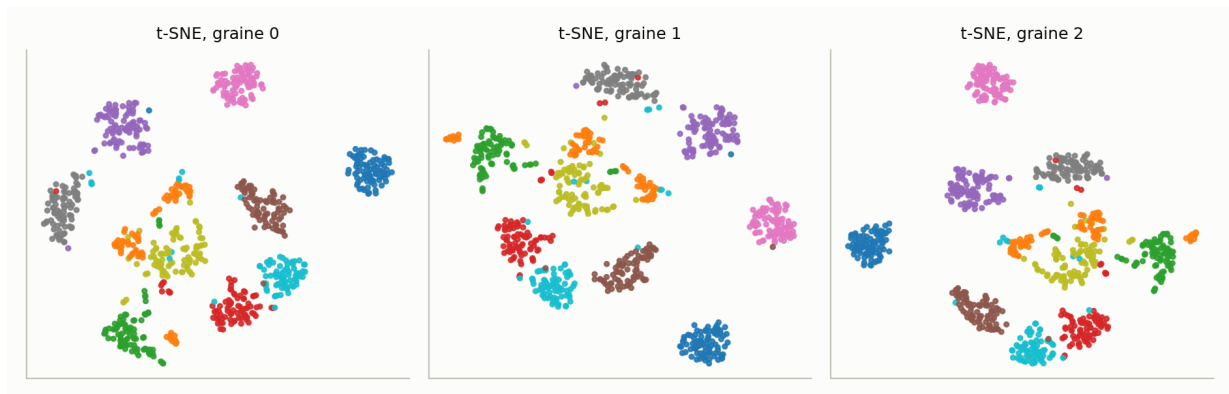


Figure 46: Trois exécutions de t-SNE avec des initialisations aléatoires différentes (perplexité 30) : les positions des îlots changent, les voisinages locaux restent.

(ARI de 0,987 entre les graines 0 et 1, 0,923 entre 0 et 2). La structure *locale* est stable ; la disposition *globale* ne l'est pas.

Règle d'usage. t-SNE et UMAP sont des outils d'**exploration visuelle** : ils suggèrent des hypothèses (ces clients forment-ils un groupe ? cette classe est-elle homogène ?) qu'il faut ensuite **vérifier avec des méthodes quantitatives** (3.1). Ne classez pas sur les coordonnées d'un plongement sans précaution, et ne publiez jamais un dessin sans préciser la méthode, ses hyperparamètres et la graine. Pour la reproductibilité : fixer `random_state`, préférer l'initialisation par ACP (`init="pca"`, plus stable pour la disposition globale) et noter le nombre de points.

3.4.5 Un dernier regard : les clients de la boutique

Que donne UMAP sur nos clients, dont on connaît la vérité cachée ?

îlot	clients	segment majoritaire	part du segment	commandes (moy.)	départ à 90 j
0	135	0	0.83	0.00	0.378
1	673	1	0.42	4.23	0.064
2	192	2	1.00	4.20	0.193

fiabilité de UMAP sur 1 000 clients : 0.967
 ARI avec la vérité, k-means sur les 7 variables : 0.491 | k-means sur le plan UMAP : 0.514

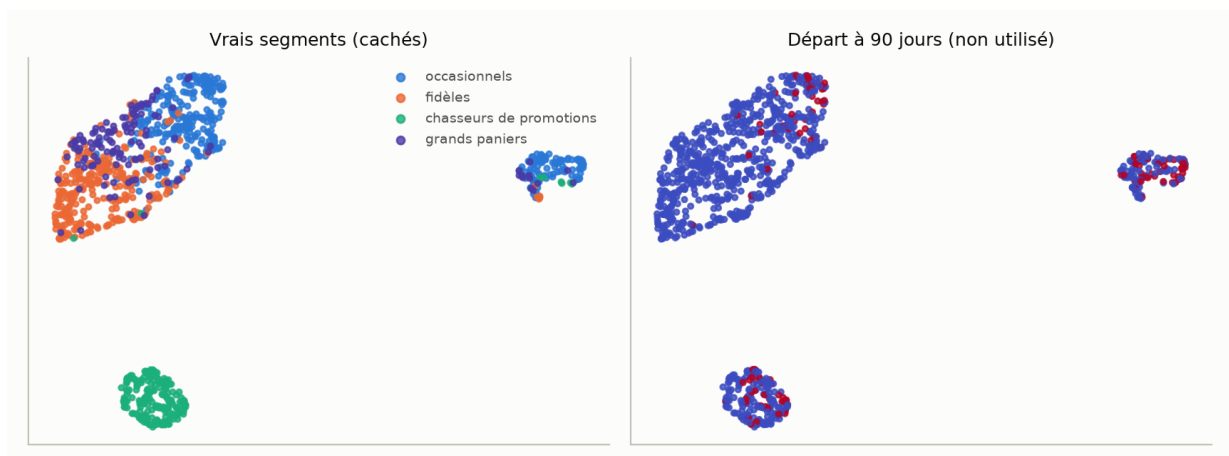


Figure 47: UMAP de 1 000 clients de la boutique, coloré par le vrai segment caché (à gauche) puis par le départ à 90 jours (à droite).

Contrairement aux chiffres, le dessin ne montre pas dix îlots nets : il en montre **deux très nets** et un grand nuage. Pour les isoler objectivement, on regroupe les points du plan par DBSCAN ($\varepsilon = 0,6$), ce qui donne le tableau ci-dessus.

- L'**îlot 2** (192 clients) est composé à 100 % de « chasseurs de promotions » (vrai segment 2) : leur comportement est si particulier qu'aucun autre client ne leur ressemble.
- L'**îlot 0** (135 clients, 83 % d'occasionnels) rassemble des clients qui n'ont **aucune commande** dans l'année (moyenne de 0,00) et qui partent à 37,8 % : ce sont les **dormants** repérés en 3.1.7.
- Le reste (673 clients) est un **nuage continu** où les fidèles ne forment que 42 % du total, mêlés aux grands paniers et aux occasionnels : c'est précisément la zone que k-means n'arrivait pas à séparer, et le plongement ne la sépare pas non plus.

Le plongement est fidèle aux voisinages (fiabilité de 0,967), mais k-means sur le plan UMAP ne retrouve qu'à peine mieux la vérité que k-means sur les variables d'origine (ARI de 0,514 contre 0,491). Le dessin a donc **confirmé** ce que les épreuves de 3.1 laissaient voir (des segments tranchés pour les comportements extrêmes, un continuum pour les autres) sans rien révéler de nouveau. Un dessin séduisant n'est pas une preuve de groupes, mais il aide à comprendre **où** la structure existe et où elle n'existe pas.

À retenir (t-SNE et UMAP).

- Objectif : conserver les **voisinages locaux** (t-SNE : divergence de Kullback–Leibler entre voisinages gaussiens et de Student ; UMAP : graphe flou de voisins et entropie croisée).
- La **fiabilité** (trustworthiness) mesure la qualité locale ; ces méthodes la poussent très haut, bien au-delà de l'ACP.
- Mais la **géométrie globale n'est pas conservée** : on ne lit ni les distances entre groupes, ni leur taille, ni les formes fines ; le résultat dépend de la perplexité, du nombre de voisins et de la graine.
- Ce sont des outils d'**exploration**, pas de démonstration : toute hypothèse lue sur un dessin se vérifie avec les épreuves de 3.1.

Pour s'entraîner. Cahier, chapitre 3 : application 3.7, exercice 3.12.

Bilan du chapitre 3

Vous savez maintenant :

- **juger un découpage sans bonne réponse** : critères internes (inertie, silhouette, Calinski–Harabasz, Davies–Bouldin), comparaison à une **référence sans structure** (silhouette comparée, statistique de l'écart, avec l'importance du choix de la référence), **stabilité** par sous-échantillonnage, **indice de Rand ajusté** et information mutuelle normalisée quand une vérité existe, et surtout **utilité** des groupes pour une décision ;
- repérer ce qui décide du résultat avant l'algorithme : l'**échelle** des variables (un ARI de 0,49 tombe à 0,08 sans standardisation) et le **choix des variables** ;
- expliquer la **malédiction de la dimension** (les distances se resserrent en $1/\sqrt{d}$) et choisir une méthode de réduction : **ACP** (erreur de reconstruction = un moins la variance gardée, théorème d'Eckart–Young), **ACP à noyau** pour le non-linéaire, **SVD tronquée** pour le parcimonieux, **NMF** pour des parties lisibles, **projections aléatoires** et lemme de Johnson–Lindenstrauss ;

- fixer le nombre de dimensions par l'**utilité en aval**, validée par validation croisée ;
- (en option) utiliser **DBSCAN** (groupes denses, bruit explicite, rayon ε unique), les **liens** de la classification hiérarchique (le lien simple retrouve des formes allongées), et les **mélanges gaussiens** estimés par **EM** (appartenances probabilistes, BIC) ;
- (en option) dessiner des données en haute dimension avec **t-SNE** et **UMAP**, mesurer leur **fiabilité**, et **ne pas lire** distances entre groupes, tailles et formes fines.

Trois messages à garder en mémoire. **Un résultat non supervisé se valide par plusieurs épreuves qui se rejoignent**, jamais par un seul chiffre. **Une méthode rend toujours un résultat** : la référence sans structure est votre meilleur garde-fou. Et **réduire n'est pas neutre** : son effet se mesure, il ne se suppose pas.

Le chapitre 4 revient aux **modèles supervisés** par l'angle le plus concret : les variables. Avant d'entraîner un modèle, il faut les encoder, les mettre à l'échelle, les fabriquer, les sélectionner, et composer avec des classes déséquilibrées. Beaucoup de gains en apprentissage automatique se jouent là.

Pour s'entraîner. Cahier, chapitre 3 : applications 3.1 à 3.8, exercices 3.1 à 3.12.

Chapitre 4 : Ingénierie des variables et données déséquilibrées

« Les algorithmes ne voient pas vos données : ils voient les nombres que vous leur donnez. »

Les chapitres précédents ont traité du **modèle** : comment l'évaluer (chapitre 1), quelles familles choisir (chapitre 2). Ce chapitre s'occupe de ce qui se passe **avant** le modèle : la façon dont on **présente** les données à l'algorithme. Un même modèle, sur les mêmes clients, peut passer d'un résultat médiocre à un très bon résultat selon la manière dont une ville est codée, dont une valeur manquante est remplacée, ou dont une règle de bon sens est transformée en variable. Et il peut se tromper de façon spectaculaire, tout en affichant 99 % d'exactitude, quand ce qu'il doit détecter est rare.

Le chemin de ce chapitre

- **4.1 Encodage, mise à l'échelle, transformations** : transformer des catégories, des unités et des valeurs manquantes en nombres que les modèles savent lire, **sans fuite d'information** ; assembler le tout dans un Pipeline.
- **4.2 Création et sélection de variables** : fabriquer des variables qui expriment ce que l'on sait du métier, découvrir des seuils à partir des données, repérer redondances et fuites.
- **4.3 Classes déséquilibrées** : pourquoi l'exactitude trompe, ce que font réellement les poids de classes, le rééchantillonnage, et le rôle décisif du **seuil de décision** et des **coûts**.
- **⊕ 4.4 Méthodes de sélection de variables** : filtre, enveloppe, méthodes intégrées, et le piège du biais de sélection.
- **⊕ 4.5 SMOTE et variantes** : fabriquer des exemples synthétiques, et surtout savoir quand ne pas le faire.
- **Bilan du chapitre.**

Fil rouge du chapitre : l'ordre des opérations. Presque toutes les erreurs graves de ce chapitre ont la même forme : une opération qui *apprend quelque chose des données* (une moyenne, une catégorie fréquente, un seuil, un échantillon synthétique) est appliquée **avant** la séparation entre entraînement et validation. Le jeu de validation « sait » alors des choses sur lui-même, et la performance affichée est trop belle. Retenez la règle ; nous la rencontrerons cinq fois : **tout ce qui est appris est appris sur le jeu d'entraînement, et seulement lui** (chapitre 1, section 1.1).

Les données du chapitre

Deux fichiers de la boutique servent d'exemples (tous deux **simulés** ; voir l'introduction du volume).

Fichier	Contenu	Cible	Sert pour
clients_ml.csv	12 000 clients observés au 31 décembre 2025 : âge, ville (20 modalités), canal d'acquisition, activité des 12 derniers mois, satisfaction, assistance, promotions...	churn_90j : le client ne commande plus dans les 90 jours suivants (14 % des clients)	encodage, échelles, valeurs manquantes, création de variables, sélection
transactions.csv	60 000 commandes en ligne : montant, heure, appareil, distance entre adresses, ancienneté du compte...	f fraude : la commande est frauduleuse (0,8 % des commandes)	classes déséquilibrées, SMOTE

Le découpage entraînement/test (75 % / 25 %, stratifié sur la cible, graine fixée) est le même dans tout le chapitre ; il est fait **une fois pour toutes, avant** tout apprentissage. Sur les variables numériques brutes, une régression logistique atteint une AUC de 0,858 sur le jeu de test et un gradient boosting 0,892 : ce sont nos **points de départ**. Les sections qui suivent montrent ce que valent, ou ne valent pas, les différentes façons de préparer les variables.

Pour s'entraîner. Cahier, chapitre 4 : le chapitre du cahier commence par une section « Préparation » qui recharge ces données et refait ce découpage.

4.1 Encodage, mise à l'échelle, transformations

Un modèle ne manipule que des **nombres**. Or une table de clients contient des villes, des canaux, des valeurs manquantes, des montants qui vont de 5 € à 3 000 €, des âges, des indicateurs 0/1. Passer de la table au tableau de nombres que le modèle consommera s'appelle le **prétraitement** (*preprocessing*). Cette section en détaille les quatre grandes opérations : encoder les catégories, mettre les variables à une échelle comparable, corriger les distributions asymétriques, et traiter les valeurs manquantes. La dernière sous-section montre comment tout assembler **sans fuite**.

4.1.1 Deux familles de modèles, deux besoins

Avant d'apprendre à transformer, demandons-nous **quand c'est nécessaire**. Les modèles se répartissent en deux familles très différentes face au prétraitement.

- Les modèles à **base de distances ou de pénalités** (k plus proches voisins, SVM, régression logistique régularisée, réseaux de neurones) comparent des variables entre elles par des sommes de carrés. Si une variable s'exprime en euros (de 0 à 3 000) et une autre en nombre de tickets (de 0 à 8), la première écrase l'autre : l'**échelle** décide, sans qu'on l'ait voulu.
- Les modèles à **base d'arbres** (arbres, forêts, boosting) ne font que **comparer une variable à un seuil**. Multiplier une variable par 1 000, ou lui appliquer une transformation croissante comme le logarithme, ne change aucune comparaison « inférieur à ? » : les partitions

possibles sont les mêmes. Ils sont donc **insensibles à l'échelle et aux transformations monotones**. En revanche, ils ne « voient » pas facilement qu'un ratio de deux variables compte, et la plupart des implémentations ne lisent pas une catégorie de texte (certaines, comme HistGradientBoosting, LightGBM ou CatBoost, acceptent des variables déclarées comme qualitatives).

Cette observation guide tout le reste : on choisit le prétraitement **en fonction du modèle**, et non pas une fois pour toutes.

4.1.2 Encoder une variable qualitative

Une **variable qualitative** (la ville, le canal d'acquisition, la catégorie de produit préférée) doit être traduite en nombres. Quatre méthodes couvrent presque tous les cas, auxquelles s'ajoute une cinquième pour les très grands nombres de modalités.

L'encodage disjonctif (*one-hot*). On crée une colonne 0/1 par modalité : pour le canal, canal_Boutique, canal_Site, canal_Réseaux. Un client de la boutique vaut (1,0,0). C'est la méthode par défaut pour les modèles linéaires : chaque modalité reçoit son propre coefficient, sans ordre imposé. Ses défauts apparaissent quand il y a beaucoup de modalités : 20 villes donnent 20 colonnes, 2 000 codes postaux en donneraient 2 000, presque toutes vides, et les arbres perdent en efficacité à fragmenter l'information.

L'encodage ordinal. On remplace chaque modalité par un entier (1, 2, 3...). Il n'est correct que si les modalités ont un **ordre naturel** (« jamais / parfois / souvent »). Appliqué à des villes, il invente un ordre qui n'existe pas : un modèle linéaire croirait que la ville 3 est « entre » les villes 2 et 4.

L'encodage par fréquence. On remplace chaque modalité par sa **fréquence** dans le jeu d'entraînement (la part des clients qui y habitent). Une seule colonne, aucune fuite de la cible, mais l'information « grande ou petite ville » est mélangée à toute autre.

L'encodage par la cible (*target encoding*). On remplace chaque modalité par la **moyenne de la cible** observée dans cette modalité : pour la ville, le taux de départ des clients qui y habitent. C'est puissant (une seule colonne, qui contient directement l'information utile), mais c'est aussi **la source de fuite d'information la plus classique** du prétraitement. Voyons pourquoi.

L'encodage par hachage (*hashing trick*). Quand une variable a des milliers, voire des millions de modalités (des identifiants de produits, des mots), on applique une **fonction de hachage** au nom de la modalité, on prend le résultat modulo m , et on place la modalité dans l'une des m colonnes d'un *one-hot* de largeur fixe (par exemple $m = 256$). Il n'y a pas de dictionnaire à garder et de nouvelles modalités sont acceptées sans erreur. Le prix : des **collisions**, c'est-à-dire des modalités différentes qui partagent une colonne, et des colonnes qu'on ne sait plus interpréter. C'est ce que fait FeatureHasher dans scikit-learn.

Un exemple minuscule

Huit clients, trois villes, et la cible « le client est parti » :

client	1	2	3	4	5	6	7	8
ville	A	A	A	B	B	B	C	A
parti	1	0	0	1	1	0	1	0

La moyenne de la cible par ville vaut : pour A (clients 1, 2, 3, 8), $\frac{1+0+0+0}{4} = 0,25$; pour B (clients 4, 5, 6), $\frac{1+1+0}{3} \approx 0,67$; pour C (client 7 seul), $\frac{1}{1} = 1$. La moyenne générale vaut $\frac{4}{8} = 0,5$.

Regardons le client 7. Il est **seul** dans sa ville, et son encodage vaut **1, c'est-à-dire exactement sa propre étiquette**. Le modèle apprend « les clients de la ville C partent toujours », alors qu'il s'est simplement relu lui-même. C'est la **fuite par la cible** : l'encodage d'une ligne a été calculé **en utilisant la cible de cette même ligne**. Plus une modalité est rare, plus l'encodage de ses lignes ressemble à leur étiquette, et plus l'effet est trompeur.

Deux remèdes

Le lissage. On « tire » la moyenne d'une modalité vers la moyenne générale μ , d'autant plus fort que la modalité est petite :

$$\text{encodage}(m) = \frac{n_m \bar{y}_m + k \mu}{n_m + k},$$

où n_m est l'effectif de la modalité, $\bar{y}_m$ sa moyenne et k un paramètre de lissage (un « nombre d'observations virtuelles » à la moyenne μ). Avec $k = 2$, la ville C passe de 1 à $\frac{1+2 \times 0,5}{1+2} \approx 0,67$: on ne fait plus confiance à un client isolé. Cette formule est une **moyenne pondérée** entre l'estimation locale et l'estimation globale, la même idée que le rétrécissement des modèles mixtes du volume II (section 1.7).

Le calcul hors pli (out-of-fold). On découpe le jeu d'entraînement en K plis. L'encodage des lignes du pli j est calculé **uniquement avec les lignes des autres plis**. Ainsi aucune ligne n'est jamais encodée avec sa propre étiquette. Les valeurs obtenues sont bruitées, et c'est tant mieux : ce bruit est celui que le modèle rencontrera réellement sur des données nouvelles. Pour encoder le jeu de test, on utilise les moyennes calculées sur **tout** le jeu d'entraînement.

Le piège du « leave-one-out ». Une variante populaire retire seulement la ligne courante du calcul de la moyenne de sa modalité. Elle paraît raisonnable, mais elle fuit **à l'envers** : dans une même ville, toutes les lignes dont la cible vaut 1 reçoivent une valeur *plus basse* que celles dont la cible vaut 0. Un arbre sépare alors parfaitement les deux groupes en lisant l'encodage. Préférez le calcul par plis.

Ce que cela change, en vrai

Pour mesurer l'effet de la fuite, il faut une variable à **forte cardinalité** : beaucoup de modalités, peu de lignes par modalité. Nos 20 villes comptent entre 141 et 1 788 clients, ce qui est trop peu « fin » pour que la fuite se voie. Nous ajoutons donc une variable fictive, un **code postal à 400 modalités tiré au hasard**, qui n'a *par construction* aucun lien avec le départ (22 clients en moyenne par code dans le jeu d'entraînement).

	encodage	AUC entraînement	AUC test
moyenne par code, calculée sur tout l'entraînement		0.678	0.51
moyenne par code, calculée hors pli		0.509 (non applicable)	

Le résultat est sans ambiguïté. Avec l'encodage naïf, la variable semble prédire le départ sur le jeu d'entraînement (AUC 0,678) alors qu'elle est **du pur bruit** : sur le jeu de test, l'AUC tombe à 0,510, c'est-à-dire au niveau du hasard. Le calcul hors pli, lui, donne honnêtement 0,509 dès l'entraînement : il ne se laisse pas tromper. Un modèle qui apprendrait sur la version naïve accorderait de l'importance à une variable qui n'en a pas.

Et pour les **vraies** villes de la boutique ? Le tableau suivant compare quatre encodages sur le jeu de test. Chaque nombre est une AUC, pour une régression logistique et pour un boosting.

encodage de la ville	AUC logistique	AUC boosting
sans la ville	0.858	0.892

disjonctif (one-hot)	0.863	0.895
fréquence	0.859	0.897
cible, hors pli	0.864	0.896
cible (TargetEncoder, lissé)	0.864	0.900

Les cinq lignes se ressemblent : ajouter l'information de la ville améliore l'AUC de quelques millièmes (de 0,858 à 0,864 pour la logistique, de 0,892 à 0,895–0,900 pour le boosting), mais **les écarts entre les encodages sont plus petits que l'incertitude** d'une AUC mesurée sur 3 000 clients de test (une erreur-type d'environ 0,011, formule de Hanley et McNeil ; section 1.4). N'en tirez pas de classement. La leçon n'est pas « tel encodage est le meilleur » ; elle est : **avec peu de modalités bien peuplées, tous conviennent ; avec beaucoup de modalités rares, seul le calcul hors pli reste honnête.**

À retenir (encodage). *One-hot* pour les modèles linéaires et peu de modalités ; ordinal seulement si l'ordre existe ; fréquence quand on veut une seule colonne sans la cible ; **cible** pour beaucoup de modalités, mais **toujours hors pli et lissée**. Et jamais avec les lignes du jeu de test.

4.1.3 Mettre les variables à la même échelle

La **mise à l'échelle** (*scaling*) remplace chaque variable x par une version dont l'unité a disparu. Quatre transformations dominent.

Transformation	Formule	Propriété
Standardisation	$\frac{x - \bar{x}}{s}$	moyenne 0, écart-type 1 ; sensible aux valeurs extrêmes
Min-max	$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$	tout dans $[0, 1]$; une valeur extrême écrase les autres
Robuste	$\frac{x - \text{médiane}}{Q_3 - Q_1}$	utilise médiane et écart interquartile, insensible aux extrêmes
Quantile	rang de x dans le jeu d'entraînement, ramené à $[0, 1]$	distribution uniforme ; défait toute asymétrie

Un exemple à la main montre leurs différences. Cinq montants d'achats : 20, 25, 30, 35 et 400 € (le dernier est un gros achat professionnel). La moyenne vaut 102 et l'écart-type $\approx 149,1$; la médiane vaut 30 et l'écart interquartile $35 - 25 = 10$.

montant	standardisé	min-max	robuste	quantile (rang)
20.0	-0.550	0.000	-1.0	0.00
25.0	-0.516	0.013	-0.5	0.25
30.0	-0.483	0.026	0.0	0.50
35.0	-0.449	0.039	0.5	0.75
400.0	1.999	1.000	37.0	1.00

La valeur extrême a deux effets opposés. Avec la **standardisation** et le **min-max**, les quatre petits montants se retrouvent **tassés** dans une toute petite plage (de $-0,55$ à $-0,45$; de 0 à 0,04) : l'information qui les distingue s'écrase. Avec la transformation **robuste**, ils gardent leur écart (-1 à $0,5$) et c'est la valeur extrême qui part très loin (37) : c'est exactement ce qu'on souhaite. La transformation **quantile** étale tout régulièrement et ne se soucie plus des écarts réels : elle est utile quand seule l'**ordre** compte.

Quel modèle a besoin de quoi ?

Mesurons-le sur les clients : 12 variables numériques, quatre modèles, avec et sans standardisation.

	modèle	sans mise à l'échelle	standardisation	quantile
	régression logistique	0.856	0.856	0.850
	k plus proches voisins (k = 25)	0.828	0.853	0.838
	SVM à noyau gaussien	0.746	0.788	0.781
	arbre de décision (profondeur 5)	0.859	0.859	0.859
L1, C = 0,01 -> conservées seulement SANS échelle : ['nb_promos_recues_12m', 'revenu_zone']				
L1, C = 0,01 -> conservées seulement AVEC standardisation : ['part_achats_promo',				
↪ 'programme_fidelite']				

Le tableau confirme la théorie. Pour le **k plus proches voisins**, la standardisation fait passer l'AUC de 0,828 à 0,853 : sans elle, le montant (en centaines d'euros) écrase tout dans le calcul des distances. Pour le **SVM**, le gain est de 0,746 à 0,788. L'**arbre** ne bouge pas (0,859 dans tous les cas) : les seuils changent de valeur, pas de sens. Quant à la **régression logistique**, son AUC ne change pas ici (0,856), parce que la solution d'un modèle sans pénalité ne dépend pas de l'unité des variables : si l'on multiplie une variable par 1 000, son coefficient est divisé par 1 000 et les prédictions restent les mêmes. Il en va autrement dès qu'on **pénalise** les coefficients (régularisation, volume II, section 1.5 ; nous y revenons au chapitre 2) : une pénalité compte les coefficients, et un coefficient dépend de l'unité de sa variable. La dernière ligne de la sortie montre l'effet sur une pénalité ℓ_1 forte (qui met à zéro les variables peu utiles) : sans mise à l'échelle, elle **écarte** programme_fidelite et part_achats_promo (des variables de petite échelle, 0/1 et 0 à 1, qui ont besoin de gros coefficients pour peser) et garde revenu_zone et nb_promos_recues_12m ; avec la standardisation, la sélection s'inverse. L'AUC bouge peu, mais **le modèle n'est plus le même** : sans mise à l'échelle, l'unité décide de ce qui est « important ».

Règle pratique. Standardisez pour tout ce qui calcule des distances ou pénalise des coefficients (kNN, SVM, régression régularisée, réseaux, ACP). Ne vous en souciez pas pour les arbres, forêts et boosting. En cas de valeurs extrêmes marquées, préférez la version robuste.

4.1.4 Corriger l'asymétrie : logarithme, Box-Cox, Yeo-Johnson

Les montants, les durées et les effectifs sont presque toujours **asymétriques à droite** : beaucoup de petites valeurs, quelques très grandes (c'est la situation des paniers, vue au volume I, section 3.1.3). Une transformation **concave** (le logarithme, la racine) resserre la queue de droite et rend la distribution plus symétrique. La famille de **Box-Cox** généralise cette idée avec un paramètre λ choisi sur les données :

$$x \mapsto \begin{cases} \frac{x^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \ln x & \lambda = 0 \end{cases} \quad (x > 0).$$

Pour $\lambda = 1$, on ne change rien (à une translation près) ; pour $\lambda = 0$, c'est le logarithme ; pour $\lambda = \frac{1}{2}$, une racine. Box-Cox exige des valeurs **strictement positives**. La version de **Yeo-Johnson** accepte aussi zéro et les valeurs négatives, en traitant chaque signe à part :

$$x \mapsto \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda} & x \geq 0, \lambda \neq 0 \\ \ln(x+1) & x \geq 0, \lambda = 0 \\ -\frac{(1-x)^{2-\lambda} - 1}{2-\lambda} & x < 0, \lambda \neq 2 \end{cases}$$

(le cas $x < 0, \lambda = 2$ est $-\ln(1-x)$). Le paramètre λ est choisi par maximum de vraisemblance, **sur le jeu d'entraînement**.

	brute	après $\log(1+x)$	après Yeo-Johnson
montant_12m	2.88	0.06	0.00
nb_commandes_12m	1.71	0.35	0.06
recence_jours	1.89	-0.48	-0.03
panier_moyen	2.54	0.75	-0.00

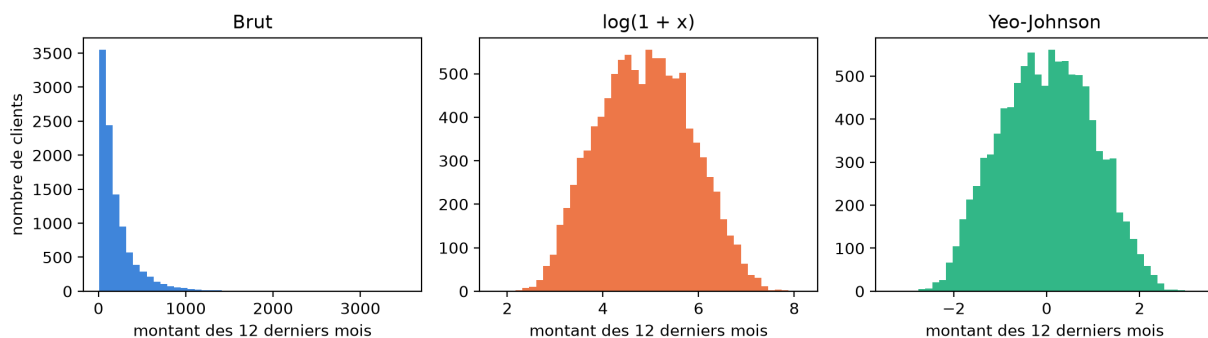


Figure 48: Distribution du montant dépensé sur 12 mois (clients ayant commandé) : brut, après le logarithme, après la transformation de Yeo-Johnson.

Le coefficient d'asymétrie (volume I, section 3.1) du montant passe de 2,88 à 0,06 avec le logarithme, et à 0,00 avec Yeo-Johnson : la distribution devient presque symétrique. Mais une transformation qui rend les histogrammes plus jolis **améliore-t-elle le modèle** ? Ici, non : la régression logistique sur les variables brutes obtient une AUC de 0,858, et de 0,852 après passage au logarithme. Le départ d'un client dépend de variables comme la récence *en jours* avec des seuils qui ont un sens dans cette unité ; les écraser ne l'aide pas. **Une transformation est une hypothèse sur la forme de la relation avec la cible**, pas un nettoyage neutre.

Normaliser n'est pas toujours utile. On transforme pour un modèle donné et pour une raison précise (une régression linéaire sensible aux extrêmes, un SVM, un graphique lisible), pas par réflexe. Les modèles à base d'arbres n'en tirent rien. Et il faut **toujours** mesurer sur un jeu de validation.

Découper en classes (*binning*) est une autre façon de se libérer de la forme : on remplace une variable continue par des classes (« moins de 30 jours », « 30 à 60 jours »...), ensuite encodées en *one-hot*. Cela permet à un modèle linéaire de représenter une relation non monotone ou à seuils, mais au prix de pertes d'information à l'intérieur des classes, et de frontières à choisir. Nous verrons mieux en 4.2 comment trouver des frontières utiles à partir des données.

4.1.5 Les valeurs manquantes

Une valeur manquante n'est pas un détail technique : elle **raconte quelque chose** sur la façon dont les données ont été produites. On distingue trois mécanismes (volume II, introduction : les hypothèses se vérifient).

- **MCAR** (*missing completely at random*) : l'absence ne dépend de rien. Un capteur tombe en panne au hasard. Supprimer les lignes ou imputer ne biaise pas.
- **MAR** (*missing at random*) : l'absence dépend d'autres variables **observées**. Les clients qui n'ont pas d'appareil enregistré sont plus souvent ceux venus en boutique. On peut corriger en s'appuyant sur ces variables.
- **MNAR** (*missing not at random*) : l'absence dépend de la valeur **manquante elle-même**. Les clients mécontents répondent moins à l'enquête de satisfaction. On ne peut pas la corriger complètement avec les seules données observées.

Nos clients contiennent quatre variables incomplètes, de natures différentes.

variable	part manquante	départs si manquante	départs si renseignée
panier_moyen	0.136	0.420	0.096
satisfaction_moy	0.128	0.115	0.144
appareil	0.152	0.154	0.138
delai_livraison_moy	0.120	0.135	0.141

Les chiffres montrent que **le fait d'être manquant est parfois très informatif**. Un panier manquant signifie « aucune commande en 12 mois », et ces clients partent dans **42 %** des cas contre 9,6 % pour les autres. L'absence de satisfaction, elle, est liée à un taux de départ plus faible (11,5 % contre 14,4 %). Les deux autres variables (appareil, livraison) n'ont aucun lien avec le départ, comme attendu d'une absence aléatoire.

Que faire ?

- **Supprimer les lignes incomplètes** : simple, mais dans ce jeu seules 66 % des lignes sont complètes ; on jette un tiers des clients et on biaise l'échantillon dès que l'absence est informative.
- **Imputer** par une valeur : la moyenne, la médiane, une constante, la valeur des plus proches voisins, ou une prédiction par un autre modèle (imputation itérative).
- **Ajouter un indicateur d'absence** : une colonne 0/1 « cette valeur était manquante », à côté de la valeur imputée. Le modèle peut ainsi utiliser l'absence comme information.
- **Laisser le modèle s'en occuper** : certains boostings (HistGradientBoosting, XGBoost, LightGBM) gèrent nativement les valeurs manquantes en apprenant, à chaque seuil, de quel côté les envoyer.

Comparons-les sur nos données.

	stratégie	AUC logistique	AUC boosting
	moyenne	0.859	0.894
	médiane	0.859	0.896
	médiane + indicateurs d'absence	0.858	0.896
	constante 0 + indicateurs d'absence	0.858	0.894
	plus proches voisins (k = 5)	0.859	0.894
	imputation itérative	0.859	0.894
	boosting : gestion native (aucune imputation)	-	0.892

Les sept lignes donnent **pratiquement le même résultat** : de 0,858 à 0,859 pour la logistique, de 0,892 à 0,896 pour le boosting. Dans ces données, le choix de l'imputation n'a quasiment aucune importance, d'abord parce que les variables qui comptent vraiment (récence, nombre de commandes) sont complètes, ensuite parce que l'information « pas de commande » est déjà portée par une variable observée. C'est rassurant, mais **pas universel** : le petit exemple suivant montre un cas où l'indicateur d'absence change tout.

Quand l'absence est l'information

Simulons 6 000 clients avec une variable x liée à la cible ($y = 1$ si le client part), et rendons x manquante **plus souvent quand le client part** : un mécanisme MNAR, comme un client qui a demandé à résilier et que l'on n'a plus mesuré.

	traitement	AUC
	imputation par la moyenne seule	0.610
	moyenne + indicateur d'absence	0.819

Dans cette simulation, la valeur est manquante pour 53,5 % des départs contre 9,4 % des autres. Quand l'absence est informative, l'imputation seule jette cette information : l'AUC reste modeste (0,610). Avec l'indicateur, le modèle apprend que « absent » veut dire « probablement parti », et l'AUC grimpe à 0,819. D'où une règle simple : **ajoutez systématiquement un indicateur d'absence pour les variables dont on soupçonne que l'absence est informative**, et laissez le modèle juger.

Imputer avant de séparer, c'est une fuite. La moyenne, la médiane ou les voisins utilisés pour remplir les trous doivent être calculés sur le jeu d'entraînement et **appliqués ensuite** au test. Imputer tout le tableau avant la séparation fait entrer de l'information du test dans l'entraînement. La fuite est faible pour une moyenne sur 12 000 lignes ; elle devient sérieuse avec de petits échantillons ou des imputations par modèle.

À retenir (valeurs manquantes). Comprenez d'abord *pourquoi* c'est manquant (structurel, aléatoire, informatif). Supprimer des lignes est le dernier recours. Imputez par la médiane, **ajoutez un indicateur** quand l'absence peut signifier quelque chose, ou laissez un boosting gérer le manque. Et imputez **à l'intérieur** de la validation croisée.

4.1.6 Tout assembler, sans fuite : Pipeline et ColumnTransformer

Nous avons rencontré quatre opérations qui **apprennent quelque chose des données** : l'encodage par la cible (des moyennes), la standardisation (une moyenne et un écart-type), l'imputation (une médiane), la transformation de puissance (un λ). Pour chacune, la règle est la même : l'apprendre sur le jeu d'entraînement (ou, dans une validation croisée, sur les plis d'entraînement uniquement), puis l'appliquer au reste.

Faire cela à la main, pli après pli, est source d'oublis. Les bibliothèques fournissent des objets qui **imposent** :

- un **Pipeline** enchaîne des étapes (prétraitement, puis modèle) et se comporte comme un seul modèle : `fit` apprend toutes les étapes sur les données qu'on lui donne, `predict` les applique ;
- un **ColumnTransformer** applique des traitements différents à des groupes de colonnes (numériques d'un côté, qualitatives de l'autre).

Passé à `cross_val_score`, un pipeline refait à **chaque pli** l'apprentissage complet du prétraitement sur les plis d'entraînement : il n'y a plus de fuite possible, même par maladresse.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

numeriques = make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler())
categories = make_pipeline(SimpleImputer(strategy="most_frequent"),
    ↳ OneHotEncoder(handle_unknown="ignore"))
prep = ColumnTransformer([("num", numeriques, NUM),
    ↳ ("cat", categories, ["ville", "canal_acquisition", "appareil",
    ↳ "categorie_preferee"])]])
modele = make_pipeline(pre, LogisticRegression(max_iter=3000))
scores = cross_val_score(modele, clients.loc[tr], ytr, cv=5, scoring="roc_auc")
print(f"AUC en validation croisée : {scores.mean():.3f} (± {scores.std():.3f})")
```

```
AUC en validation croisée : 0.860 (± 0.013)
```

Cet assemblage est le **modèle complet** : les quatorze variables numériques sont imputées (avec indicateurs) puis standardisées, les quatre variables qualitatives sont imputées puis encodées en *one-hot* (`handle_unknown="ignore"` évite une erreur si une modalité n'a pas été vue à l'entraînement), puis la régression logistique est ajustée. La validation croisée porte sur le jeu d'entraînement (le jeu de test reste intouchable, chapitre 1, section 1.2) et donne une AUC de 0,860, avec un écart-type de 0,013 d'un pli à l'autre. Une fois ajusté sur tout l'entraînement, le pipeline obtient 0,866 sur le jeu de test, cohérent avec la validation croisée et un peu meilleur que la régression logistique sur les seules variables numériques (0,858), grâce aux quatre variables qualitatives.

Un seul objet à sauvegarder. Une fois ajusté sur tout l'entraînement, le pipeline contient **tout** : prétraitement et modèle. Pour prédire sur de nouveaux clients, on lui donne la table brute. Impossible d'oublier une étape, ni d'appliquer à la production un prétraitement différent de celui de l'entraînement.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 (un pipeline complet pas à pas), 4.2 (l'encodage par la cible hors pli, écrit à la main) et 4.3 (comparer des stratégies pour les valeurs manquantes) ; exercices 4.1 à 4.5.

4.2 Création et sélection de variables

La section précédente apprenait à **présenter** les variables existantes. Celle-ci apprend à en **fabriquer** de nouvelles et à décider lesquelles garder. C'est souvent là que se joue l'essentiel de la performance : un modèle sait combiner des variables, mais il ne devine pas toujours les **notions** que vous, vous connaissez du métier (« un client qui ne revient plus », « une promotion qui attire des chasseurs d'affaires »).

4.2.1 Penser métier : des variables qui ont du sens

Une nouvelle variable est un **calcul sur les colonnes existantes** qui exprime une idée. Les familles les plus utiles sont les suivantes.

Les ratios et les taux. Un nombre brut est rarement comparable d'un client à l'autre : 3 retours, c'est beaucoup pour 4 commandes, très peu pour 40. Le **taux de retour** (retours divisés par commandes) et les **tickets d'assistance par commande** se comparent, eux. Une **fréquence mensuelle** (commandes divisées par l'ancienneté, plafonnée à 12 mois) corrige l'effet « client récent ».

Les profils RFM. En commerce, trois grandeurs résument un client : la **récence** (depuis combien de temps a-t-il commandé ?), la **fréquence** (combien de fois ?) et le **montant** (combien a-t-il dépensé ?). Nos données les contiennent déjà ; on les combine pour faire apparaître des profils (« gros mais endormi », « petit mais assidu »).

Les indicateurs de situation. Un 0/1 qui code un fait métier : *jamaï commandé, inactif depuis plus de cinq mois, a contacté l'assistance au moins trois fois*. Ils rendent explicites des **seuils** que les modèles linéaires ne peuvent pas apprendre seuls.

Les interactions. Le produit ou la combinaison de deux variables : un client peu satisfait **et** inactif est bien plus à risque que la somme des deux risques. Un modèle linéaire ne voit que des effets *additifs* ; l'interaction doit lui être donnée.

Les composantes temporelles. Une date se décompose en jour de la semaine, mois, heure, délai écoulé depuis un événement. Une heure se pose un problème particulier, traité plus loin (4.2.4).

Où trouver des idées ? (1) Interrogez les gens du métier : *qu'est-ce qui fait partir un client ?* (2) Regardez les erreurs du modèle de base : quels clients se trompe-t-il, qu'ont-ils en commun ? (3) Regardez les arbres : leurs premières questions suggèrent des seuils utiles (4.2.2). Une variable créée est une **hypothèse** ; elle se teste sur la validation.

4.2.2 Découvrir les seuils à partir des données

Dans quelle zone de récence le risque de départ s'envole-t-il ? Plutôt que de deviner, regardons le taux de départ observé **sur le jeu d'entraînement** (jamais sur le test : choisir un seuil en regardant le test serait une fuite), en croisant la récence et la satisfaction.

Deux faits ressortent. Le risque de départ est **faible et presque plat** jusqu'à 120 jours de récence (de 5,9 % à 9,3 %), puis il **monte** : 12,9 % entre 121 et 160 jours, 28,4 % entre 161 et 200, 36,5 % au-delà. Et la montée est **beaucoup plus forte quand la satisfaction est basse** : entre 161 et 200 jours, le taux de départ atteint 67 % chez les clients peu satisfaits (moins de 3,15) contre 15 % chez les autres. Le même nombre de jours sans commander n'a pas la même signification pour un client content et pour un client mécontent. C'est une **interaction avec seuil**, exactement ce qu'un modèle linéaire ne peut pas représenter avec les variables brutes.

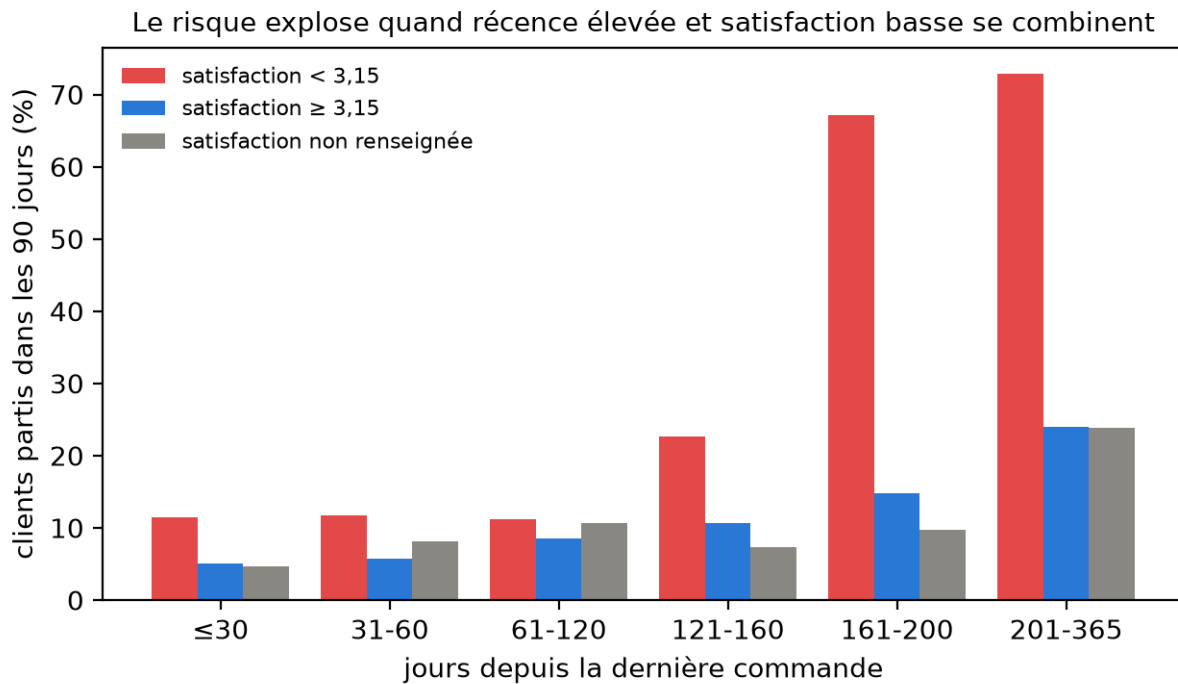


Figure 49: Taux de départ (en %) selon la récence, séparé selon la satisfaction : le risque est modéré jusqu'à environ 160 jours, puis explose, surtout chez les clients peu satisfaits.

Pour trouver les seuils avec précision, on peut laisser un **arbre peu profond** les proposer. Ses premières questions sont, par construction, celles qui séparent le mieux les départs des autres.

```
seuils proposés par l'arbre de profondeur 3 (entraînement uniquement) :
recence_jours <= 160.5
age <= 20.5
satisfaction_moy <= 3.15
age <= 28.5
```

L'arbre place en tête la **récence autour de 160 jours**, puis, dans la branche des clients inactifs, la **satisfaction autour de 3,15** ; il isole aussi un seuil de **part d'achats en promotion proche de 0,60** et un effet d'âge aux deux extrémités. Ces valeurs sont des **candidates** : on les transforme en variables et on mesure si elles aident.

4.2.3 Fabriquer, puis mesurer

Un indicateur de règle s'écrit en une ligne :

```
inactif_et_mecontent = ((clients["recence_jours"] > 160) & (clients["satisfaction_moy"] <
↳ 3.15)).astype(int)
print("part des clients concernés :", round(inactif_et_mecontent.mean(), 3))
```

```
part des clients concernés : 0.062
```

Construisons un jeu de variables enrichi, en trois étages : des variables **génériques** (ratios, fréquences, log), puis des **indicateurs de règles** construits avec les seuils découverts à l'étape précédente.

	mean		size	
retours	False	True	False	True
tickets3				
False	0.139	0.098	7591	1099

True	0.303	0.615	284	26
	variables	nombre de colonnes	AUC logistique	AUC boosting
	14 variables brutes	14	0.858	0.892
	+ 6 variables génériques	20	0.871	0.893
	+ 3 indicateurs de règles	23	0.900	0.900

La troisième règle vient d'un tableau croisé calculé sur l'entraînement : parmi les 26 clients qui cumulent au moins trois tickets d'assistance et plus de 20 % de retours, **61,5 %** partent, contre 13,9 % des clients qui n'ont ni l'un ni l'autre (et 30,3 % de ceux qui n'ont que les tickets). Le groupe est petit, mais l'écart est net.

Le tableau est riche d'enseignements.

- Les **variables génériques** font gagner à la régression logistique 0,013 point d'AUC (de 0,858 à 0,871) et ne changent presque rien au boosting (0,892 puis 0,893).
- Les **indicateurs de règles**, eux, font passer la régression logistique à **0,900** : un gain de 0,042 point par rapport aux variables brutes, soit **autant que le boosting** (0,900), alors que la régression logistique reste un modèle simple, rapide et interprétable. Le boosting, qui découvrait déjà seul ces seuils et ces interactions, gagne peu (0,008), moins que l'incertitude d'une AUC sur 3 000 clients (environ 0,011, section 4.1).

Ingénierie des variables et choix du modèle sont deux manières d'obtenir la même chose.

Un modèle flexible (arbres, boosting) apprend lui-même seuils et interactions ; un modèle simple a besoin qu'on les lui **donne**. Quand les deux atteignent le même niveau, le modèle simple l'emporte souvent par sa lisibilité. À l'inverse, ne vous attendez pas à ce que les mêmes variables aident un boosting.

Des seuils découverts sur l'entraînement, évalués sur le test. Les seuils (160 jours, 3,15, 0,60, ainsi que « au moins 3 tickets et plus de 20 % de retours ») ont été lus sur le jeu d'entraînement. Si nous les avons choisis en regardant le jeu de test, le gain affiché aurait été gonflé : c'est la forme discrète de la fuite d'information. Les règles du métier connues d'avance ne posent pas ce problème ; celles trouvées dans les données, si.

4.2.4 Les variables cycliques : l'exemple de l'heure

Une commande passée à 23 h et une autre à 1 h sont **proches** dans la journée, mais leurs valeurs (23 et 1) sont **éloignées** sur l'axe des nombres. Pour un modèle linéaire ou une distance, l'heure comme entier est trompeuse : le « bout » de la journée est collé à son « début ». La solution est de placer l'heure sur un **cercle**, en la remplaçant par deux coordonnées :

$$\text{heure} \mapsto \left(\sin \frac{2\pi h}{24}, \cos \frac{2\pi h}{24} \right).$$

Minuit et 23 h deviennent deux points voisins du cercle, tandis que minuit et midi en sont deux points opposés. La même idée s'applique au jour de la semaine (période 7) et au mois (période 12).

Voyons son effet sur les fraudes de transactions.csv, qui se concentrent la nuit.

	codage de l'heure	AUC	précision moyenne (PR-AUC)
	heure comme entier (0 à 23)	0.920	0.254
	heure en sinus et cosinus	0.928	0.262
	indicateur « nuit » (23 h à 4 h)	0.935	0.284

Sur le jeu d'entraînement, 4,1 % des commandes passées entre 23 h et 4 h sont frauduleuses, contre 0,6 % le reste du temps : la concentration nocturne se lit directement dans les taux par heure. Le codage de l'heure compte, mais **modestement** : l'AUC passe de 0,920 (heure comme entier) à 0,928 (sinus et cosinus) puis 0,935 (indicateur « nuit »), et la précision moyenne de 0,254 à 0,262 puis 0,284. Les écarts sont cohérents avec la théorie, mais sur 146 fraudes de test ils restent fragiles (4.3.6). L'indicateur est le plus économe et le plus parlant, **à condition de connaître la zone utile** (ici, lue sur l'entraînement) ; le couple sinus/cosinus est le choix général quand on ne sait pas d'avance où elle se trouve.

4.2.5 Agréger : passer de plusieurs lignes à une ligne par client

Beaucoup de jeux de données contiennent **plusieurs lignes par entité** : toutes les commandes d'un client, tous les passages d'une machine. Le modèle attend **une ligne par client**. On **agrège** alors, avec des fonctions qui résument chaque entité : le nombre de commandes, le montant total, moyen, maximal, l'écart-type, la date de la dernière commande, la part de commandes en promotion... Les variables nb_commandes_12m, montant_12m, panier_moyen et recence_jours de notre fichier de clients sont précisément des agrégats d'un historique de commandes.

Un exemple à la main : un client a passé quatre commandes de 30, 45, 45 et 120 €, la dernière il y a 12 jours. Ses variables agrégées sont : nombre = 4, total = 240, moyenne = 60, maximum = 120, médiane = 45, récence = 12 jours. Chaque agrégat est une **hypothèse sur ce qui compte** : la moyenne cache un gros achat isolé, le maximum le révèle.

Agréger dans le temps, c'est choisir une date. Les agrégats d'un client ne doivent utiliser que les commandes **antérieures à la date de prédiction**. Inclure une commande postérieure à la date de référence est une fuite d'information, la plus fréquente en pratique (voir plus bas).

4.2.6 Redondance, variance nulle et fuite

Avant de nourrir le modèle, trois contrôles de bon sens.

Les variables quasi constantes n'apportent rien (une colonne dont 99,9 % des valeurs sont égales). On les écarte sans regret.

Les variables redondantes portent la même information : montant_12m vaut approximativement le nombre de commandes multiplié par le panier moyen, et la part d'achats en promotion est liée au nombre de promotions reçues. Elles n'abîment pas un arbre, mais rendent les coefficients d'un modèle linéaire instables (volume II, section 1.3, multicollinéarité).

nb_commandes_12m	montant_12m	0.74
nb_promos_recues_12m	part_achats_promo	0.67
montant_12m	panier_moyen	0.50
nb_commandes_12m	nb_retours_12m	0.49

La fuite d'information est le contrôle le plus important. Notre fichier contient volontairement une variable piège, commandes_apres_cible, qui compte les commandes des **trois mois suivants**. Elle appartient au futur : on ne la connaît pas au moment de prédire. Comment la repérer ?

variable	AUC seule
montant_12m	0.780
commandes_apres_cible	0.760
nb_commandes_12m	0.752
recence_jours	0.740
age	0.717

La première idée est de regarder la **qualité de chaque variable prise seule** (AUC univariée). Elle échoue ici : la variable de fuite atteint 0,760, **moins** que le montant des 12 derniers mois (0,780), et elle n'est même pas la plus prédictive des variables « normales ». Cette fuite est **insidieuse** parce qu'elle est bruitée : un nombre de commandes futures est aléatoire, et seuls les clients qui ne reviendront plus ont systématiquement zéro. En revanche, elle **ajoute** de l'information que les autres variables n'ont pas : avec elle, le boosting passe de 0,892 à 0,928, un gain de 0,036 point que l'on ne sait expliquer par aucune idée du métier.

Aucun test automatique ne remplace donc la **question de bon sens** à poser pour chaque variable : *à quelle date cette valeur est-elle connue, par rapport à la date où je veux prédire ?* Pour commandes_apres_cible, la réponse est « trois mois **après** », la variable est exclue. Les autres signaux d'alerte sont un gain « trop beau » après l'ajout d'une seule variable, une importance démesurée d'une variable dont le nom évoque un **résultat** (« après », « résiliation », « solde final »), et une performance qui **s'effondre** en production (chapitre 1, section 1.1). Une variable qui rend le modèle bien meilleur que ce que le métier permet d'espérer est suspecte avant d'être précieuse.

À retenir (création et sélection). Fabriquez des variables qui expriment des idées métier (ratios, taux, indicateurs de situation, interactions) ; trouvez les seuils sur l'entraînement ; donnez à un modèle simple ce qu'un modèle flexible découvre seul. Codez les variables cycliques sur un cercle. Écartez variables constantes et redondantes, et **traquez la fuite** : une variable trop belle est suspecte.

Pour s'entraîner. Cahier, chapitre 4 : application 4.4 (créer des variables RFM et mesurer leur apport), exercices 4.6 à 4.8.

4.3 Classes déséquilibrées : pondération et rééchantillonnage

Une commande frauduleuse sur 120, un client qui part sur sept, une panne de machine sur mille : les événements qui nous intéressent le plus sont presque toujours **rare**s. Les modèles, eux, sont construits pour minimiser une erreur **moyenne** sur les données : quand 99 % des lignes appartiennent à une classe, ils apprennent surtout à bien traiter celle-là. Cette section explique pourquoi la métrique habituelle (l'exactitude) devient trompeuse, ce que font réellement les remèdes (poids, rééchantillonnage), et pourquoi le **seuil de décision** est souvent le levier le plus puissant et le plus négligé.

4.3.1 Pourquoi l'exactitude trompe

Notre fichier transactions.csv contient 60 000 commandes en ligne, dont 0,8 % de fraudes. Après le découpage habituel (70 % pour l'entraînement, 30 % pour le test, stratifié), le jeu de test compte 18 000 commandes dont 146 frauduleuses.

Considérons le **modèle le plus paresseux possible** : il répond « pas de fraude » pour toutes les commandes. Il se trompe sur les 146 fraudes et n'a raison que sur les autres, soit une **exactitude** (part de réponses justes) de $\frac{17\,854}{18\,000} \approx 99,19\%$. Un score qui paraît excellent, pour un modèle qui ne détecte **aucune** fraude.

La table de confusion rend l'illusion visible. Elle croise la réalité (en lignes) et la décision du modèle (en colonnes) :

	décision « pas de fraude »	décision « fraude »
réellement pas de fraude	vrais négatifs (VN) : 17 854	faux positifs (FP) : 0
réellement fraude	faux négatifs (FN) : 146	vrais positifs (VP) : 0

Deux mesures décrivent mieux ce qui compte quand la classe rare est celle qui nous intéresse (elles sont détaillées en 5.1).

$$\text{précision} = \frac{VP}{VP + FP} \quad (\text{parmi les alertes, quelle part est juste ?}), \quad \text{rappel} = \frac{VP}{VP + FN} \quad (\text{parmi les fraudes,}$$

Le modèle paresseux a un rappel de 0 % ; l'exactitude de 99,19 % n'en dit rien. D'où une règle absolue : **avec des classes déséquilibrées, ne jamais juger sur l'exactitude.**

Même l'AUC peut rassurer à tort

Entraînons une régression logistique sur ces données et mesurons deux scores de classement. L'**AUC-ROC** (volume II, section 2.2) compare les taux de vrais et de faux positifs à tous les seuils. La **précision moyenne** (PR-AUC, aire sous la courbe précision-rappel) mesure plutôt la qualité du haut du classement, là où se trouvent les alertes.

L'AUC-ROC vaut 0,921 : le modèle classe très bien. Mais la PR-AUC n'est que de 0,260 (un classement au hasard donnerait 0,008), et à un seuil de 0,5 la régression logistique ne détecte que **10 % des fraudes** (rappel 0,103) : sur 146 fraudes, elle en trouve 15 et en laisse passer 131. L'AUC-ROC est **optimiste** quand la classe positive est rare : les 17 854 négatifs comptent tant dans les taux de faux positifs qu'une petite erreur sur eux paraît négligeable. La PR-AUC, qui ignore les vrais négatifs, est plus sévère et plus informative.

À retenir. Exactitude : à proscrire. AUC-ROC : informative mais indulgente. **PR-AUC, précision et rappel** : les bons outils quand la classe rare est l'objet. (Détail et autres métriques : section 5.1.)

4.3.2 La pondération des classes

La première idée est de **dire au modèle que la classe rare compte davantage**. La perte d'une régression logistique est la perte logarithmique moyenne (volume II, section 2.2) :

$$L(\beta) = -\frac{1}{n} \sum_{i=1}^n [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

On la **pondère** : une erreur sur un exemple de la classe 1 coûte w fois plus qu'une erreur sur un exemple de la classe 0.

$$L_w(\beta) = -\frac{1}{n} \sum_{i=1}^n [w y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

C'est équivalent à **recopier w fois** chaque exemple positif. L'option `class_weight="balanced"` choisit w pour que les deux classes pèsent autant au total, c'est-à-dire $w = \frac{n_0}{n_1}$ (ici $w \approx 122,5$). Dans scikit-learn, c'est un simple paramètre du modèle :

```
from sklearn.linear_model import LogisticRegression

modele_pondere = LogisticRegression(class_weight="balanced", max_iter=3000)    # w = n0 / n1 pour la
↳ classe rare
# on peut aussi donner les poids à la main : class_weight={0: 1, 1: 20}
```

Ce que cela change exactement

Considérons le cas le plus simple, sans aucune variable explicative : une probabilité constante p pour tous. Si la proportion de positifs est π , la perte pondérée moyenne vaut $-[w\pi \ln p + (1 - \pi) \ln(1 - p)]$. En la dérivant par rapport à p et en annulant la dérivée,

$$\frac{w\pi}{p} - \frac{1 - \pi}{1 - p} = 0 \quad \Rightarrow \quad \frac{p^*}{1 - p^*} = w \frac{\pi}{1 - \pi}.$$

La probabilité estimée a donc une **cote multipliée par w** : la pondération n'a pas rendu le modèle « plus malin », elle a **déplacé ses probabilités vers le haut**. Pour une régression logistique à plusieurs variables bien spécifiée, le même raisonnement dit que seul le terme constant change, de $\ln w$, et que les autres coefficients (donc le **classement** des clients) ne bougent pas. Vérifions.

```
poids w = n0/n1 = 122.5 | ln w = 4.808 | décalage observé de la constante : 4.675
probabilité moyenne prédite : sans poids 0.008 | avec poids 0.2365 | fréquence réelle 0.0081
à 0,5 avec poids : précision 0.049, rappel 0.863 (FP 2432, FN 20)
probabilité moyenne après correction : 0.0098
```

Les constats confirment la théorie. Le poids vaut $w \approx 122$, soit $\ln w \approx 4,81$; la constante du modèle a bougé de 4,68, très près de la valeur prévue. Les probabilités moyennes passent de 0,8 % (la vraie fréquence) à **23,7 %** : elles sont **fausses**, gonflées, comme le prédit le calcul. Si on les corrige en divisant la cote par w ($p = \frac{p_w}{p_w + (1 - p_w)w}$), la moyenne retombe à 1,0 %, bien plus près de la fréquence réelle (0,8 %) ; l'écart restant vient du fait que le modèle n'est pas parfaitement spécifié. L'AUC-ROC reste pratiquement celle du modèle sans poids (0,921 contre 0,923) et la corrélation de rang entre les deux scores vaut 0,985 : **le classement n'a presque pas changé**.

Alors, à quoi sert la pondération ? À **déplacer le seuil implicite**. Avec un seuil de 0,5, le modèle pondéré déclare « fraude » dès que la probabilité *gonflée* dépasse 0,5, ce qui correspond à une probabilité réelle bien plus basse. Le rappel passe de 0,103 à **0,863** : sur 146 fraudes, il en trouve 126. Mais la précision s'effondre à 0,049 : il déclenche 2 432 fausses alertes, contre 9 auparavant. On a gagné 111 détections supplémentaires (de 15 à 126), au prix d'environ 2 400 vérifications inutiles de plus. Est-ce un bon marché ? **Cela dépend uniquement des coûts** (4.3.5).

Avec des poids, les probabilités ne sont plus des probabilités. Si vous avez besoin de probabilités fiables (chiffrer un risque, calculer une espérance de gain), **corrigez-les** comme ci-dessus ou recalibrez-les (section 5.2). Si vous ne vous servez que du classement, la pondération est sans conséquence.

Pour les modèles à base d'arbres, le mécanisme est un peu différent : les poids modifient l'importance de chaque exemple dans le calcul des impuretés et des gradients (chapitre 2), de sorte que les **modèles eux-mêmes** diffèrent, pas seulement leurs probabilités. Sur nos données, le boosting sans correction obtient une PR-AUC de 0,625 et le boosting pondéré de 0,643 : une différence trop faible, avec 146 fraudes de test, pour être distinguée du hasard (4.3.6).

4.3.3 Le rééchantillonnage aléatoire

La seconde famille de remèdes agit sur les **données** : on rééquilibre l'échantillon d'entraînement avant d'apprendre.

- Le **sous-échantillonnage** (*undersampling*) retire au hasard des exemples de la classe majoritaire. On garde par exemple 5 non-fraudes pour 1 fraude. Rapide, mais on **jette des données** : ici, plus de 95 % des commandes normales.
- Le **sur-échantillonnage** (*oversampling*) recopie au hasard des exemples de la classe minoritaire jusqu'à égalité. On ne perd rien, mais on duplique : le modèle voit la même fraude des dizaines de fois et peut la **mémoriser**.

Un exemple à la main : 1 000 commandes dont 10 fraudes. Sous-échantillonner à 5 contre 1 garde les 10 fraudes et 50 commandes normales tirées au hasard, soit 60 lignes ; sur-échantillonner à l'égalité garde les 990 commandes normales et recopie chacune des 10 fraudes 99 fois, soit 1 980 lignes.

traitement	AUC-ROC	PR-AUC	précision à 0,5	rappel à 0,5
aucun	0.921	0.260	0.625	0.103
poids équilibrés	0.923	0.187	0.049	0.863
sous-échantillonnage 5:1	0.921	0.220	0.119	0.534
sur-échantillonnage aléatoire	0.923	0.187	0.049	0.863

Le sur-échantillonnage aléatoire donne **exactement** les mêmes résultats que les poids équilibrés (même AUC, même précision, même rappel) : recopier chaque fraude w fois et la pondérer par w sont, pour une régression logistique, une seule et même opération. Le sous-échantillonnage place le compromis ailleurs (précision 0,119, rappel 0,534) : on a rééquilibré à 5 contre 1, et non à 1 contre 1, donc le décalage des probabilités est moindre. Aucune des trois stratégies n'améliore l'AUC-ROC (0,921 à 0,923) : elles **déplacent le point de fonctionnement**, elles ne rendent pas le modèle meilleur.

Rééchantillonner avant de séparer est une fuite d'information. Si l'on sur-échantillonne la table complète puis qu'on la sépare en entraînement et validation (ou qu'on lance une validation croisée), les copies d'une même fraude se retrouvent des deux côtés : le modèle est « testé » sur des exemples qu'il a déjà vus. Le score devient absurde. Nous le mesurons en 4.5. La règle : le rééchantillonnage ne s'applique qu'aux **plis d'entraînement**, dans le pipeline.

4.3.4 Le seuil de décision : le levier oublié

Un modèle de classification produit une **probabilité** ; la **décision** (« fraude » ou non) vient d'un **seuil** : on déclare « fraude » si la probabilité dépasse s . Le seuil de 0,5 n'a aucune raison d'être le bon : il n'est optimal que si les deux erreurs coûtent autant et si les probabilités sont fidèles. Faire varier s déplace le modèle le long de sa **courbe précision-rappel** : un seuil bas détecte plus de fraudes (rappel haut) mais déclenche plus de fausses alertes (précision basse), un seuil haut fait l'inverse.

La figure dit l'essentiel. Le boosting domine la régression logistique sur **toute** la plage (par exemple, à rappel de 0,5, il garde une précision bien supérieure). Et pondérer ou non le boosting ne change presque pas la courbe : les deux courbes se superposent à peu près. **Une courbe précision-rappel est la carte de tous les compromis possibles ; choisir un seuil, c'est choisir un point sur cette carte.**

Où se placer ? Pas en regardant le jeu de test (le choisir sur le test, c'est de nouveau une fuite) : on choisit le seuil sur des **prédictions hors pli** du jeu d'entraînement, obtenues par validation croisée, puis on l'applique au test.

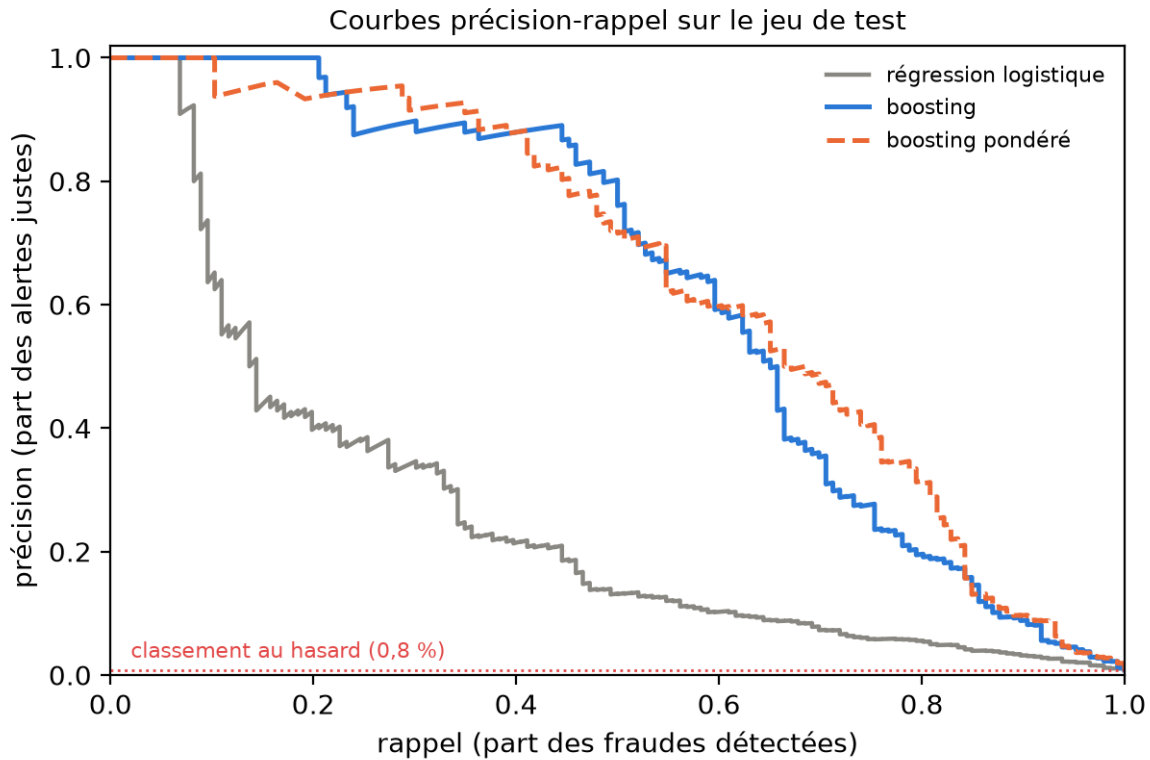


Figure 50: Courbes précision-rappel de trois modèles sur les 18 000 commandes de test : le boosting domine nettement la régression logistique, et la pondération déplace peu la courbe elle-même.

Avec le boosting, le seuil qui maximise le F1 sur les prédictions hors pli est à $s = 0,42$; sur le test, il donne un F1 de 0,598 (précision 0,678, rappel 0,534), contre 0,601 au seuil de 0,5 (précision 0,710, rappel 0,521). **Aucune différence** : le F1 donne le même poids aux deux erreurs et les probabilités du boosting sont bien calibrées, le seuil de 0,5 était déjà presque optimal pour ce critère. Optimiser un seuil n'est utile que **pour un critère qui correspond à l'usage réel** ; le F1 est rarement ce critère. La section suivante part des coûts.

4.3.5 La vue par les coûts

Le meilleur seuil n'est pas celui qui maximise une métrique abstraite, c'est celui qui **minimise ce que coûtent les erreurs**. Supposons qu'une fraude non détectée coûte en moyenne $c_{FN} = 100$ € (marchandise perdue), et qu'une fausse alerte coûte $c_{FP} = 5$ € (vérification manuelle). Pour une commande dont la **vraie** probabilité de fraude est p :

- si on la laisse passer, le coût espéré est $p \times c_{FN}$;
- si on la bloque pour vérification, le coût espéré est $(1 - p) \times c_{FP}$.

On bloque quand le second est plus petit que le premier :

$$p c_{FN} > (1 - p) c_{FP} \iff p > \frac{c_{FP}}{c_{FP} + c_{FN}}.$$

Le **seuil optimal** ne dépend que du rapport des coûts : $s^* = \frac{5}{105} \approx 0,048$ ici. Une fraude coûte vingt fois plus qu'une fausse alerte, il faut donc bloquer dès que la probabilité dépasse 4,8 %. Cette formule suppose que les probabilités sont **calibrées** : celles du boosting non pondéré le sont à peu près, pas celles d'un modèle pondéré (4.3.2).

règle de décision	coût total sur le test (€)
seuil 0,5 (défaut)	7155
seuil théorique 0.048	5575
seuil minimisant le coût hors pli (0.025)	5420
aucune alerte (modèle paresseux)	14600
boosting pondéré, seuil 0,5 : coût 4595 € ; au seuil équivalent corrigé 0.86 : 5365 €	
coût du boosting pondéré à 0,5 moins coût du boosting au seuil théorique : moyenne -1031 €, intervalle à 95 % [-2198 ; -37]	

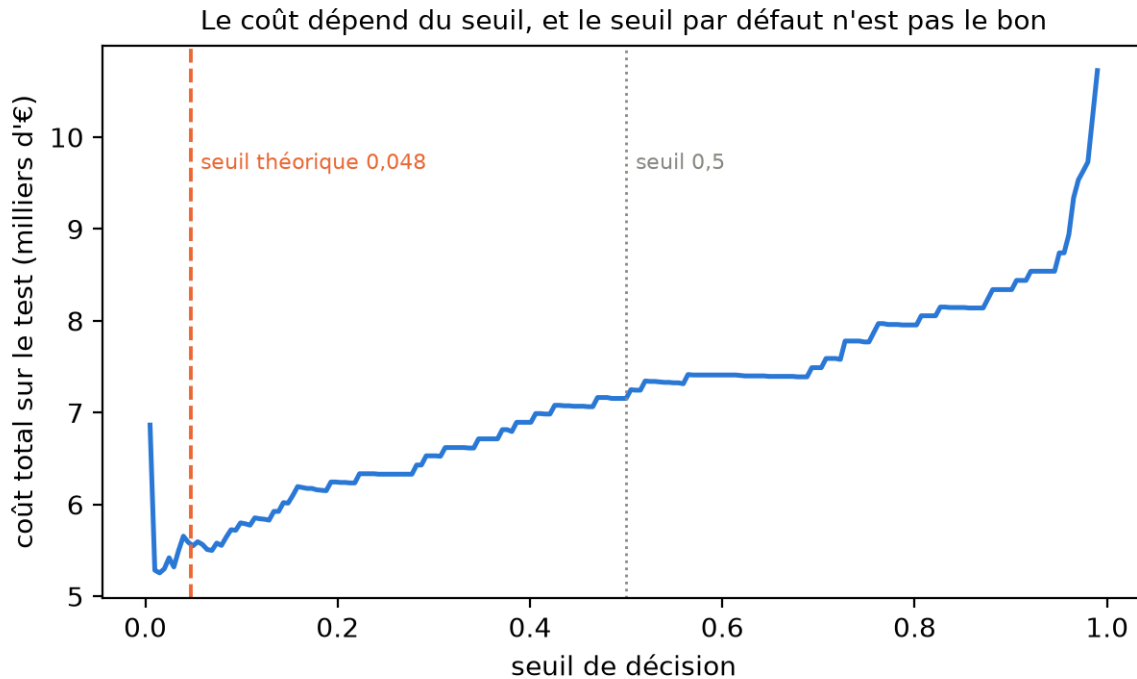


Figure 51: Coût total des erreurs de décision sur les 18 000 commandes de test en fonction du seuil, pour le boosting : le coût est minimal pour des seuils bas, bien en dessous de 0,5.

En code, appliquer un seuil est une simple comparaison de la probabilité prédite :

```
proba = gb.predict_proba(Xb)[: , 1]          # probabilité de fraude de chaque commande de test
alerte = proba >= 0.048                      # on bloque dès que la probabilité dépasse le seuil
print(alerte.sum(), "commandes bloquées sur", len(alerte))
```

231 commandes bloquées sur 18000

Le tableau et la figure disent trois choses. (1) Le seuil par défaut de 0,5 est **le plus coûteux** des seuils raisonnables : 7 155 €. (2) Le seuil théorique de 0,048 réduit le coût de 22 % (5 575 €), et le seuil qui minimise le coût sur les prédictions hors pli (0,025) donne un résultat voisin (5 420 €) : sur 146 fraudes, la formule suffit, sans chercher le seuil exact. (3) Ne rien bloquer coûterait 14 600 €.

Le boosting **pondéré**, à son seuil par défaut de 0,5, coûte 4 595 € : moins que toutes les lignes précédentes. Il ne faut pas y voir un miracle de la pondération. Les poids ont modifié les **arbres eux-mêmes**, pas seulement les probabilités (4.3.2). Et l'avantage, de 1 031 € en moyenne, est mesuré avec une grande marge (intervalle à 95 % par *bootstrap* du test : de -2 198 à -37 €) : il exclut de peu zéro, mais le *bootstrap* ne rééchantillonne que le test et **ignore la variabilité due à l'entraînement**. Avec 146 fraudes, retenez surtout le constat solide : **le seuil par défaut est mauvais** dès que les erreurs n'ont pas le même coût.

Le rôle des poids, revisité. Pondérer les classes, rééchantillonner et déplacer le seuil sont **trois façons d'obtenir le même effet** : changer le compromis entre faux négatifs et faux positifs. Si vous connaissez vos coûts, le plus propre est de **laisser le modèle apprendre des probabilités fidèles, puis de choisir le seuil par les coûts**. Poids et rééchantillonnage sont des commodités quand l'algorithme apprend mal avec de rares positifs, pas des objectifs en soi.

4.3.6 Comparer honnêtement : et le déséquilibre modéré ?

Avec 146 fraudes de test, toute comparaison de deux stratégies est entachée d'une **forte incertitude**. Mesurons-la par un *bootstrap* du jeu de test (volume I, section 3.3.5) : on rééchantillonne 500 fois les 18 000 commandes de test, et on observe la dispersion de la PR-AUC.

boosting	moyenne 0.623	intervalle à 95 % [0.543 ; 0.701]
boosting pondéré	moyenne 0.642	intervalle à 95 % [0.561 ; 0.716]
écart	moyenne 0.020	intervalle à 95 % [-0.028 ; 0.072]

Les intervalles de la PR-AUC du boosting et du boosting pondéré se **recouvrent largement** : l'intervalle de leur écart contient zéro. Nous n'avons **aucune preuve** que la pondération améliore le classement du boosting ; seule l'écart de seuil de décision, lui, est réel.

Reste le cas du **déséquilibre modéré**, celui du départ des clients (14 %). Les mêmes stratégies y changent-elles quelque chose ?

modèle	AUC-ROC	PR-AUC	précision à 0,5	rappel à 0,5
boosting	0.899	0.655	0.689	0.458
boosting pondéré	0.899	0.663	0.504	0.717

À 14 % de positifs, les deux modèles ont presque le même classement (AUC-ROC et PR-AUC proches) ; la pondération déplace surtout le compromis précision-rappel à 0,5, comme avant. **Un déséquilibre de cet ordre n'appelle pas de traitement spécial** : un bon modèle, des probabilités fiables, et un seuil choisi selon l'usage (par exemple, contacter les 10 % de clients les plus à risque, que la capacité du service client permet). Les précautions de cette section prennent toute leur valeur quand la classe rare passe sous quelques pour cent.

À retenir (classes déséquilibrées). (1) Jugez avec précision, rappel et PR-AUC, jamais avec l'exactitude. (2) Les poids de classes et le rééchantillonnage **déplacent les probabilités et le seuil implicite** ; ils n'améliorent pas, en général, le classement. (3) Le **seuil de décision** se choisit par les coûts, sur des prédictions hors pli, jamais sur le test. (4) Rééchantillonner uniquement à l'intérieur des plis d'entraînement. (5) Avec peu de positifs de test, **chiffrez l'incertitude** de vos comparaisons.

Pour s'entraîner. Cahier, chapitre 4 : application 4.5 (comparer toutes les stratégies sur les fraudes) et application 4.6 (choisir un seuil par les coûts) ; exercices 4.9 à 4.11.

4.4 ⊕ Pour aller plus loin : les méthodes de sélection de variables

Section optionnelle. Elle approfondit la sélection de variables vue en 4.2 : les trois grandes familles de méthodes, et surtout le piège qui rend la plupart des sélections trop optimistes. On peut la sauter à la première lecture.

Quand un jeu de données compte des dizaines ou des milliers de variables, on veut en garder **un sous-ensemble utile** : pour des modèles plus rapides, plus lisibles, moins sujets au surapprentissage. Les méthodes se classent selon **le moment** où elles interviennent par rapport au modèle.

4.4.1 Trois familles de méthodes

Les méthodes de filtre jugent chaque variable **avant** le modèle, par une statistique indépendante de lui. Elles sont rapides et simples, mais regardent les variables **une par une**, donc ignorent les interactions.

Les méthodes d'enveloppe (*wrapper*) **essaient** des sous-ensembles en entraînant le modèle et en mesurant sa performance. Elles tiennent compte des interactions et du modèle choisi, mais coûtent cher : chaque essai est un apprentissage complet.

Les méthodes intégrées (*embedded*) sélectionnent **pendant** l'apprentissage : une pénalité ℓ_1 qui met des coefficients à zéro, ou les importances qu'un arbre calcule en se construisant. Elles sont le meilleur compromis coût-qualité, avec leurs propres biais.

4.4.2 Les filtres : information mutuelle et χ^2

Le filtre le plus général est l'**information mutuelle**. Pour une variable X et la cible Y , elle mesure combien connaître X réduit l'incertitude sur Y :

$$I(X; Y) = \sum_{x,y} p(x, y) \ln \frac{p(x, y)}{p(x) p(y)}.$$

Elle vaut zéro si et seulement si X et Y sont **indépendantes**, et elle capte toute dépendance, pas seulement linéaire. Contrairement à une corrélation, elle voit une relation en U ou à seuil. Pour deux variables **qualitatives**, le test du χ^2 d'indépendance (volume I, section 3.4.6) joue un rôle analogue : on garde les variables dont le lien avec la cible est le plus significatif.

Un exemple à la main, avec deux variables binaires et 100 clients dont 20 partent. Si 50 clients ont reçu une promotion et que 10 d'entre eux partent (autant que dans l'autre moitié), la promotion n'apporte aucune information sur le départ : $I = 0$. Si au contraire 18 des 20 partants viennent du groupe « sans promotion », alors savoir qu'un client a eu la promotion change beaucoup la probabilité qu'il parte : $I > 0$.

4.4.3 Les enveloppes : RFE et sélection progressive

La **suppression récursive de variables** (*recursive feature elimination*, RFE) entraîne le modèle avec toutes les variables, élimine la moins importante (par exemple celle au plus petit coefficient en valeur absolue), et recommence jusqu'à ce qu'il reste le nombre voulu. La **sélection progressive** (*forward selection*) fait l'inverse : on part de zéro variable et on ajoute à chaque étape celle qui améliore le plus le score en validation croisée.

Pour p variables, la sélection progressive jusqu'à k variables nécessite environ $p + (p - 1) + \dots$ essais, c'est-à-dire de l'ordre de kp validations croisées : raisonnable pour 14 variables, prohibitif pour 5 000.

4.4.4 Les méthodes intégrées : ℓ_1 , importances d'arbre, permutation

La **pénalité ℓ_1** (Lasso, volume II, section 1.5) met à zéro les coefficients des variables peu utiles : la sélection est un effet secondaire de l'apprentissage. Elle suppose des variables **standardisées** (4.1.3).

L'**importance par impureté** d'une forêt ou d'un arbre additionne, pour chaque variable, la réduction d'impureté obtenue à tous les nœuds où elle est utilisée. Elle est gratuite, mais **biaisée** : une variable continue ou à beaucoup de modalités offre plus de seuils possibles, donc plus d'occasions de réduire l'impureté **par hasard**, même si elle est du pur bruit.

L'**importance par permutation** corrige ce défaut. On entraîne le modèle, puis on mesure la baisse de performance sur un jeu de **validation** quand on **mélange au hasard** les valeurs d'une seule variable (ce qui détruit son lien avec la cible en préservant sa distribution). Plus la baisse est grande, plus la variable compte ; une variable de bruit entraîne une baisse nulle.

Que choisissent-elles sur nos clients ?

Nous ajoutons aux 14 variables numériques **cinq variables de bruit pur** (tirées au hasard, sans lien avec la cible : trois continues et deux discrètes), et nous comparons les méthodes. Toutes les sélections sont calculées sur le jeu d'entraînement ; l'importance par permutation utilise une partie de l'entraînement mise de côté comme validation.

	méthode	variables retenues	dont bruit	AUC test (logistique)
toutes les variables (14 vraies + 5 de bruit)		19	5	0.857
	information mutuelle	8	0	0.855
	RFE	8	0	0.857
	sélection progressive	8	0	0.858
	pénalité ℓ_1	14	3	0.858
	permutation (forêt)	8	1	0.858

importance par impureté (forêt), variables de bruit : {'bruit_continu_1': 0.038, 'bruit_continu_2': 0.0382, 'bruit_continu_3': 0.0359, 'bruit_discret_1': 0.021, 'bruit_discret_2': 0.0241}

importance par impureté, médiane des 14 vraies variables : 0.0435

importance par permutation, variables de bruit : {'bruit_continu_1': 0.0003, 'bruit_continu_2': -0.0008, 'bruit_continu_3': 0.001, 'bruit_discret_1': -0.0002, 'bruit_discret_2': -0.0}

Trois méthodes (information mutuelle, RFE, sélection progressive) retiennent chacune 8 variables parmi 19 et **aucune des cinq variables de bruit**. La pénalité ℓ_1 (avec $C = 0,05$) en garde 14, dont **trois variables de bruit** : à ce niveau de pénalité, elle laisse de petits coefficients non nuls ; son résultat dépend du paramètre C , à régler par validation croisée (chapitre 1, 1.5). La sélection par permutation d'une forêt laisse passer une variable de bruit (`bruit_continu_3`) dans ses huit premières.

Côté performance, **aucune sélection n'améliore l'AUC** : de 0,855 à 0,858 pour tous les sous-ensembles, contre 0,857 avec les 19 variables. Ici la sélection ne sert donc pas la performance

mais la **simplicité** : un modèle à huit variables est aussi bon qu'un modèle à dix-neuf, et plus facile à expliquer et à maintenir.

Le contraste est plus net sur les **importances**. Dans la forêt, les trois variables de bruit **continues** obtiennent une importance par impureté de 0,036 à 0,038, presque celle de la variable réelle médiane (0,0435) ; l'importance par permutation les ramène à $0,000 \pm 0,001$, alors qu'elle donne 0,046 à la récence.

L'importance par impureté peut récompenser du bruit. Les trois variables de bruit **continues** obtiennent une importance par impureté de 0,036 à 0,038, parce qu'elles offrent de nombreux seuils à tester (et les deux variables de bruit discrètes, à douze modalités, de 0,021 à 0,024) ; l'importance par permutation, mesurée sur un jeu de validation, les ramène près de zéro. Pour décider quelles variables garder, préférez la permutation (section 5.3).

4.4.5 Le piège : sélectionner avant de valider

Voici l'erreur la plus répandue de toute la sélection de variables. On dispose de beaucoup de variables et de peu de clients ; on commence par **choisir les variables les plus liées à la cible sur tout le jeu de données**, puis on **évalue** le modèle sur ces variables par validation croisée. Le résultat semble excellent. Il est faux.

Pour le voir, prenons le cas extrême : 150 clients, 500 variables **entièrement aléatoires**, et une cible **tirée à pile ou face**, donc sans aucun lien avec les variables. Aucun modèle ne peut faire mieux que le hasard (AUC de 0,5). On répète l'expérience de deux manières, sur 40 jeux de données différents.

- **Sélection avant la validation croisée (fautive)** : on garde les 20 variables les mieux classées sur les 150 clients, puis on valide par validation croisée sur ces 20 colonnes.
- **Sélection dans la validation croisée (correcte)** : la sélection fait partie du Pipeline, donc elle est refaite à chaque pli sur les plis d'entraînement seulement.

sélection AVANT la validation croisée : AUC moyenne 0.831 (min 0.762, max 0.909)
sélection DANS la validation croisée : AUC moyenne 0.510 (min 0.351, max 0.663)

Le contraste est total. En sélectionnant d'abord, **tous** les jeux simulés donnent une AUC bien supérieure à 0,5 (de 0,76 à 0,91, moyenne 0,83) pour des données sans aucune information ; en sélectionnant à l'intérieur de la validation, on retrouve honnêtement le hasard (moyenne 0,51, de 0,35 à 0,66 selon les jeux, ce qui rappelle aussi qu'une seule AUC sur 150 clients est très bruitée). Pourquoi ? Parmi 500 variables aléatoires, on en trouve toujours 20 qui, **par chance**, ressemblent à la cible sur ces 150 clients ; la sélection les a choisies *en regardant les étiquettes de tous les clients, y compris ceux des plis de validation*. Le pli de validation n'est plus vierge : l'information sur ses étiquettes a servi à choisir les colonnes.

Règle absolue. Toute opération qui **regarde la cible** (sélection de variables, choix de seuils, encodage par la cible, réglage) doit être **dans** le pipeline validé, refaite pli par pli. Et une fois le modèle final choisi, le jeu de test ne doit servir qu'**une** fois, à la fin.

À retenir (sélection). Filtre : rapide, aveugle aux interactions. Enveloppe : fidèle au modèle, mais coûteuse. Intégrée : bon compromis. Préférez l'**importance par permutation** à l'importance par impureté. Et mettez la sélection **dans le pipeline** : sinon la validation croisée est faussée.

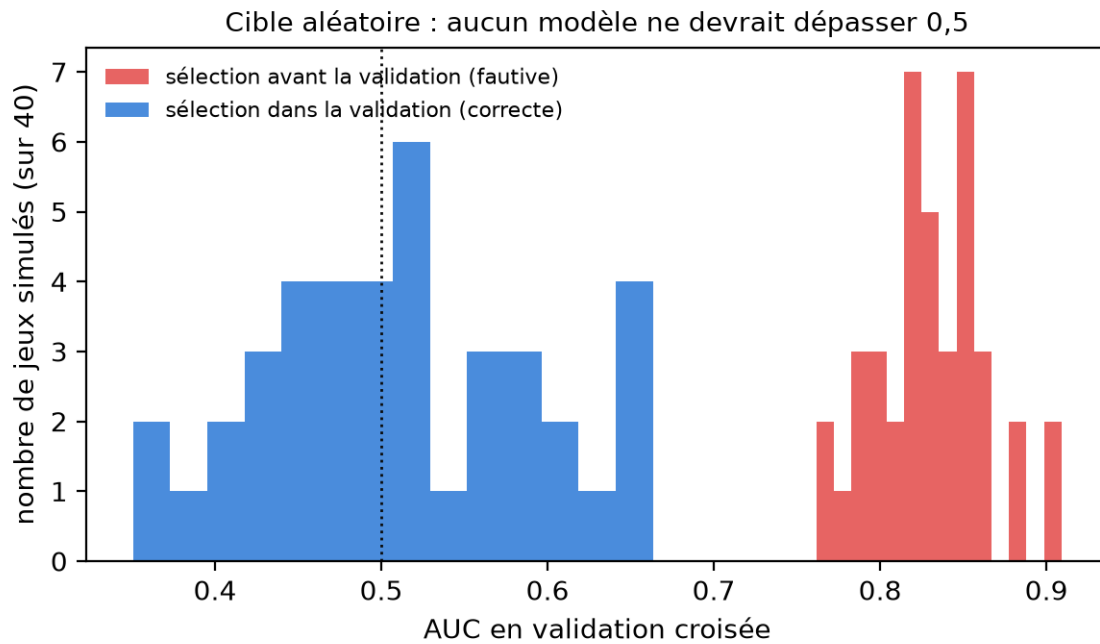


Figure 52: Distribution de l'AUC en validation croisée sur 40 jeux simulés où la cible est du pur hasard : la sélection de variables faite avant la validation donne une AUC trompeusement élevée, celle faite dans la validation reste autour de 0,5.

Pour s'entraîner. Cahier, chapitre 4 : application 4.7 (comparer filtre, enveloppe et méthodes intégrées, et refaire le piège de la sélection) ; exercices 4.12.

4.5 ⊕ Pour aller plus loin : SMOTE et ses variantes

Section optionnelle. Elle présente la méthode de rééchantillonnage la plus connue pour les classes rares, et, surtout, montre sur nos données **quand elle ne sert à rien**. On peut la sauter à la première lecture.

Le sur-échantillonnage aléatoire (4.3.3) recopie les exemples rares. Son défaut est évident : il fabrique des **doublons exacts**, que le modèle peut apprendre par cœur. L'idée de **SMOTE** (*Synthetic Minority Over-sampling TEchnique*, Chawla et al., 2002) est de créer des exemples **nouveaux mais plausibles**, en **interpolant** entre exemples rares voisins.

4.5.1 L'idée : interpoler entre voisins

Pour fabriquer un exemple synthétique, SMOTE procède ainsi :

1. choisir au hasard un exemple de la classe rare, x ;
2. trouver ses k plus proches voisins **dans la classe rare** (par défaut $k = 5$) et en choisir un au hasard, x' ;
3. tirer un nombre λ au hasard entre 0 et 1, et créer le point $x_{\text{nouveau}} = x + \lambda (x' - x)$, sur le segment qui relie x à x' .

On répète jusqu'à obtenir l'équilibre voulu. Un exemple à la main avec une fraude réelle de notre

jeu d'entraînement et son **plus proche voisin** parmi les fraudes, décrites par deux variables (le montant et l'ancienneté du compte en jours) :

```
types des deux fraudes (1 = compte neuf, 2 = prise de contrôle) : [1, 1]
fraude 1 : [72.4  2. ] | fraude 2 : [77.4  1. ]
point synthétique pour lambda = 0,4 : [74.4  1.6]
```

La première fraude est une commande de 72,4 € sur un compte vieux de 2 jours ; son plus proche voisin parmi les fraudes, une commande de 77,4 € sur un compte de 1 jour (les deux sont des fraudes du même type, le « compte neuf »). Pour $\lambda = 0,4$, le point synthétique est $x_1 + 0,4(x_2 - x_1)$, soit une commande d'environ 74,4 € sur un compte de 1,6 jour : un « cousin » plausible des deux fraudes, situé à 40 % du chemin de la première vers la seconde.

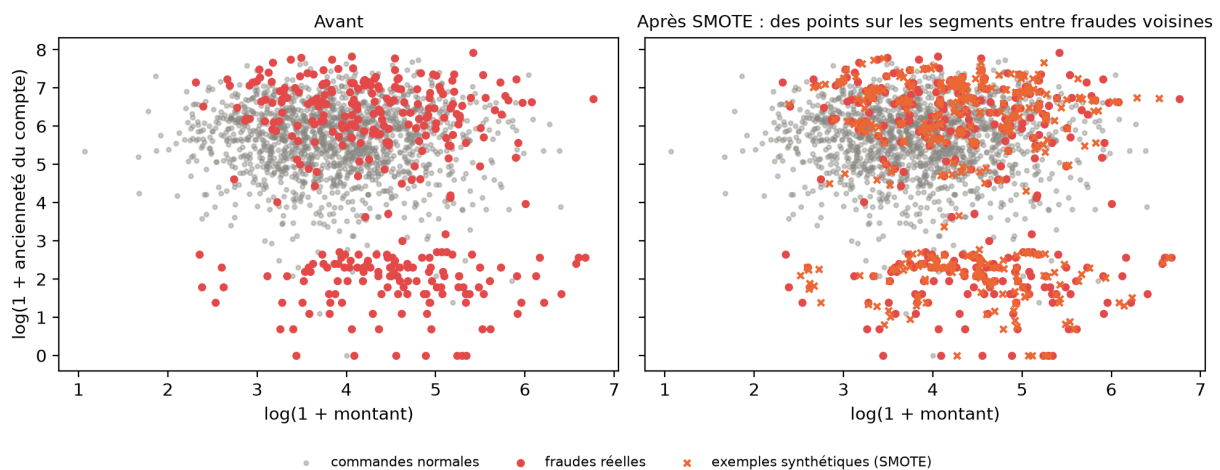


Figure 53: À gauche : fraudes (rouge) et commandes normales (gris) dans le plan (montant, ancienneté du compte) en échelle logarithmique. À droite : les exemples synthétiques créés par SMOTE (croix orange) s'alignent sur des segments entre fraudes voisines et restent dans la zone des fraudes.

La figure montre le mécanisme et sa limite. Les croix orange sont **entre** des fraudes voisines : SMOTE remplit les trous de la région des fraudes. Les fraudes forment ici **deux groupes nets** (les comptes très récents, en bas, et les comptes anciens mêlés aux commandes normales, en haut), et comme les $k = 5$ voisins d'une fraude sont presque toujours du même groupe, les points synthétiques restent dans chacun : l'espace vide entre les deux groupes n'est pas comblé. Mais l'algorithme **suppose que la région des fraudes est compacte** : que n'importe quel point d'un segment entre deux voisines est une fraude plausible. Si un groupe est petit, si k est grand ou si un segment traverse une zone de commandes normales, les points synthétiques deviennent faux. Remarquez enfin que le groupe du haut est **entièrement mêlé aux commandes normales** : y ajouter des fraudes synthétiques ne facilite pas leur séparation.

4.5.2 Les variantes

Plusieurs variantes corrigent des défauts de l'algorithme de base. Elles sont toutes disponibles dans la bibliothèque `imbalanced-learn` (`imblearn`).

Variante	Idée	Quand l'envisager
Borderline-SMOTE	ne crée des points que près de la frontière : à partir des exemples rares dont les voisins sont en majorité de la classe normale	on veut renforcer la zone d'incertitude plutôt que le cœur de la classe

Variante	Idée	Quand l'envisager
ADASYN	crée plus de points pour les exemples rares difficiles (entourés de voisins normaux), moins pour les faciles	les classes se mélangent fortement
SMOTE-NC	gère les variables qualitatives : la catégorie du point synthétique est la plus fréquente parmi les k voisins	des variables non numériques sont mêlées aux numériques
SMOTEN	variante pour des variables toutes qualitatives	données entièrement catégorielles

Toutes partagent la même limite : elles **inventent des exemples** à partir d'exemples existants, donc ne créent pas d'information nouvelle sur la classe rare. Elles ne font que **remplir** la région déjà observée.

4.5.3 Dans un pipeline, et seulement dans le pli d'entraînement

Comme tout ce qui apprend sur les données, SMOTE ne doit s'appliquer qu'au jeu **d'entraînement**. La bibliothèque `imbalanced-learn` fournit un Pipeline qui le garantit : l'étape de rééchantillonnage n'est exécutée **que pendant fit**, jamais au moment de prédire ni d'évaluer sur le pli de validation.

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import cross_val_score

pipe = Pipeline([("echelle", StandardScaler()),
                 ("smote", SMOTE(k_neighbors=5, random_state=0)),
                 ("modele", LogisticRegression(max_iter=3000))])
scores = cross_val_score(pipe, Xa, ya, cv=5, scoring="average_precision")
print(f"PR-AUC en validation croisée : {scores.mean():.3f} (± {scores.std():.3f})")
```

```
PR-AUC en validation croisée : 0.212 (± 0.021)
```

4.5.4 Faut-il vraiment rééchantillonner ? Une comparaison honnête

Comparons, en validation croisée à 5 plis sur les 42 000 commandes d'entraînement, les stratégies de 4.3 et de 4.5 pour deux modèles. Chaque nombre est la moyenne sur 5 plis, avec son écart-type d'un pli à l'autre.

	modèle et traitement	AUC-ROC	PR-AUC
logistique + sur-échantillonnage aléatoire	logistique, aucun traitement	0.907 (± 0.019)	0.280 (± 0.059)
	logistique, poids équilibrés	0.910 (± 0.018)	0.219 (± 0.041)
	logistique + SMOTE	0.908 (± 0.020)	0.214 (± 0.039)
	logistique + Borderline-SMOTE	0.907 (± 0.018)	0.195 (± 0.028)
	logistique + ADASYN	0.908 (± 0.020)	0.212 (± 0.038)
	boosting, aucun traitement	0.950 (± 0.010)	0.575 (± 0.040)
	boosting, poids équilibrés	0.966 (± 0.013)	0.544 (± 0.072)
	boosting + SMOTE	0.945 (± 0.011)	0.500 (± 0.033)

Trois enseignements se dégagent.

1. **Le boosting domine la régression logistique**, quel que soit le traitement : PR-AUC de 0,50 à 0,58 contre 0,19 à 0,28. Le choix du modèle pèse bien plus que celui du traitement.

2. **Rééchantillonner n'améliore pas la PR-AUC ; elle baisse.** Pour la logistique, de 0,280 (sans traitement) à 0,195–0,219 avec poids, sur-échantillonnage, SMOTE ou ses variantes ; pour le boosting, de 0,575 à 0,544 avec poids et à 0,500 avec SMOTE. Les écarts entre variantes de SMOTE (0,195 à 0,214) sont du même ordre que l'écart-type d'un pli à l'autre (0,03 à 0,04) : **aucune ne se distingue.**
3. L'AUC-ROC, elle, ne dit pas la même chose : elle reste stable (0,907 à 0,911 pour la logistique) ou monte un peu avec les poids (0,950 à 0,966 pour le boosting). Deux métriques de classement, deux verdicts : c'est un rappel que la **PR-AUC**, centrée sur le haut du classement, est la mesure pertinente quand la classe est rare.

L'erreur à ne jamais commettre

Que se passe-t-il si l'on rééquilibre **toute la table d'abord**, puis qu'on lance la validation croisée sur le résultat ? C'est une erreur de débutant très fréquente, parce qu'elle semble innocente.

boosting, sur-échantillonnage aléatoire AVANT la validation croisée :	PR-AUC 1.000
boosting, SMOTE AVANT la validation croisée	: PR-AUC 0.999

Les scores s'envolent à **1,000** (sur-échantillonnage aléatoire) et **0,999** (SMOTE) : un modèle presque parfait sur un problème que le même boosting, évalué correctement, ne résout qu'à 0,575 de PR-AUC. Aucune magie : avec le sur-échantillonnage avant la séparation, les **copies d'une même fraude se retrouvent des deux côtés** de la validation ; avec SMOTE, les points synthétiques sont des interpolations entre fraudes voisines, dont une partie se trouve dans le pli de validation. Le modèle est évalué sur des exemples qu'il a, en pratique, déjà vus.

Rééchantillonner dans le pipeline, jamais avant. Le jeu de validation (et le jeu de test) ne doivent contenir **que des exemples réels, dans leur proportion réelle**. C'est la seule façon de savoir comment le modèle se comportera en production, où les fraudes ne seront ni dupliquées ni synthétiques.

4.5.5 Que retenir de SMOTE ?

Pour les modèles d'aujourd'hui (boosting, forêts), nos résultats vont dans le même sens que l'expérience de nombreux praticiens : **SMOTE est rarement meilleur que la simple pondération des classes ou que le réglage du seuil**, et il peut dégrader les probabilités et le classement (ici, la PR-AUC du boosting passe de 0,575 à 0,500). Son intérêt est plus net pour des modèles qui apprennent mal avec très peu de positifs (certains réseaux de neurones, des modèles basés sur les distances), ou quand on ne dispose d'**aucun** moyen de pondérer. Avant de l'utiliser :

1. **mesurez** la performance sans rééchantillonnage, avec poids, avec choix de seuil par les coûts (4.3) ;
2. si vous rééchantillonnez, faites-le **dans le pipeline** ;
3. **recalibrez** les probabilités ensuite (section 5.2), car elles sont faussées.

À retenir (SMOTE). SMOTE crée des exemples rares par interpolation entre voisins ; ses variantes (Borderline, ADASYN, SMOTE-NC) en changeant la zone ou le type de variables. Il n'ajoute pas d'information, il est sensible à la forme de la classe rare, et sur des modèles à base d'arbres il fait rarement mieux que les poids ou le seuil. **Dans le pipeline, ou pas du tout.**

Pour s'entraîner. Cahier, chapitre 4 : application 4.8 (SMOTE et variantes dans un pipeline, et la fuite par rééchantillonnage) ; exercices 4.13 et 4.14.

Bilan du chapitre 4

Vous savez maintenant :

- **préparer les variables selon le modèle** : encoder une variable qualitative (disjonctif, ordinal, fréquence, cible), mettre à l'échelle (standardisation, min-max, robuste, quantile) quand le modèle compare des distances ou pénalise des coefficients, corriger l'asymétrie (logarithme, Box-Cox, Yeo-Johnson) quand cela aide vraiment, et **savoir que les arbres n'en ont pas besoin** ;
- **encoder par la cible sans fuite** : calcul hors pli et lissage, et pourquoi le calcul naïf transforme du bruit en « signal » (AUC de 0,678 à l'entraînement, 0,510 au test) ;
- **traiter les valeurs manquantes** en comprenant leur mécanisme (MCAR, MAR, MNAR, structurel), ajouter un **indicateur d'absence** quand l'absence est informative, et ne jamais imputer avant la séparation ;
- **assembler un Pipeline** avec ColumnTransformer, pour que tout ce qui est appris le soit sur les plis d'entraînement seulement ;
- **fabriquer des variables métier** (ratios, taux, RFM, indicateurs de situation, interactions, variables cycliques), **découvrir des seuils sur l'entraînement**, et comprendre qu'un modèle simple muni des bonnes variables peut égaler un modèle flexible (0,900 d'AUC pour la logistique enrichie, comme pour le boosting) ;
- **repérer la fuite d'information** par la question « quand cette valeur est-elle connue ? » : aucune statistique ne la détecte à coup sûr ;
- **traiter des classes déséquilibrées** : ne jamais juger sur l'exactitude, lire précision, rappel et PR-AUC, comprendre que **poids et rééchantillonnage déplacent les probabilités et le seuil** sans améliorer le classement, et choisir le **seuil par les coûts** ($s^* = \frac{c_{FP}}{c_{FP} + c_{FN}}$) sur des prédictions hors pli ;
- (en option) **comparer les méthodes de sélection** (filtre, enveloppe, intégrée), préférer l'importance par **permutation**, et **mettre la sélection dans la validation croisée** ; **utiliser SMOTE et ses variantes** dans un pipeline, en sachant qu'il n'améliore pas toujours, et qu'appliqué avant la validation il produit des scores absurdes.

Le fil rouge du chapitre tient en une phrase : **ce qui apprend, apprend sur l'entraînement, et rien que lui**. Encodage par la cible, imputation, mise à l'échelle, choix de seuils, sélection de variables, rééchantillonnage : cinq façons de tricher par inadvertance, que le Pipeline et la validation croisée rendent impossibles quand on les utilise correctement.

Le chapitre 5 aborde la question qui conclut toute modélisation : **comment juger un modèle, lui faire confiance et l'expliquer ?** Métriques adaptées au problème, calibration des probabilités, interprétabilité (importance par permutation, SHAP, LIME), puis équité et incertitude.

Pour s'entraîner. Cahier, chapitre 4 : applications 4.1 à 4.8 et exercices 4.1 à 4.14.

Chapitre 5 : Évaluation, calibration et interprétabilité

« Un modèle qui prédit bien ne vaut rien tant qu'on ne sait pas dire à *quel point*, pour *quelle décision* et *pourquoi*. »

Les quatre chapitres précédents ont appris à **construire** des modèles : séparer les données, valider, comparer des familles d'algorithmes, préparer les variables. Il reste l'essentiel : savoir ce que valent ces modèles une fois qu'on les a. Trois questions, qui reviennent à chaque projet, structurent ce chapitre.

1. **Est-il bon ?** Mais bon *pour quoi faire* ? Une exactitude de 90 % peut cacher un modèle inutile ; une AUC de 0,90 peut cacher un modèle mal réglé. Choisir la bonne mesure, et le bon seuil de décision, est un acte de métier avant d'être un acte statistique.
2. **Peut-on se fier à ses probabilités ?** Quand le modèle annonce « 30 % de risque de départ », se passe-t-il vraiment quelque chose environ trois fois sur dix ? C'est la **calibration**, condition de toute décision fondée sur un coût attendu.
3. **Pourquoi dit-il cela ?** Un modèle qui prédit sans que personne ne puisse l'expliquer est un modèle difficile à corriger, à défendre, ou à débusquer quand il triche. C'est l'**interprétabilité**.

Le chemin de ce chapitre

- **5.1 Métriques** : de la matrice de confusion à la courbe ROC, à la courbe précision-rappel, aux coûts, au lift ; les métriques de régression.
- **5.2 Calibration** : le diagramme de fiabilité, pourquoi certains modèles mentent sur leurs probabilités, Platt, la régression isotonique, et l'effet des poids de classes.
- **5.3 Interprétabilité** : importance par permutation, effets partiels, LIME, et les valeurs de Shapley avec SHAP.
- ⊕ **5.4 Équité, biais et éthique** : mesurer les écarts entre groupes sur un jeu **réel** de crédit, et ce que l'on ne peut pas avoir en même temps.
- ⊕ **5.5 Prédiction conforme** : transformer n'importe quel modèle en un modèle qui annonce des *ensembles* ou des *intervalles* avec une garantie de couverture.

Comment lire ce chapitre. Les sections 5.1 à 5.3 forment le socle. Les sections 5.4 et 5.5 sont facultatives. Le livre démontre et explique ; le code qui produit chaque nombre cité est exécuté en coulisses (voir l'introduction du volume), et les exercices et applications sont dans le cahier.

Le fil rouge : un même problème, trois modèles

Tout le chapitre s'appuie sur le même problème, celui du volume : **prédire, pour chacun des 12 000 clients de la boutique, s'il aura cessé de commander dans les 90 jours** (la variable `churn_90j`, qui vaut 1 pour 14 % des clients). Les données sont **simulées** (`clients_ml.csv`), ce qui permet de savoir ce qui a été programmé.

Conformément à la règle du chapitre 1 (section 1.1), les clients sont répartis une fois pour toutes en trois jeux, **stratifiés** sur la cible (chacun garde 14 % de partants) :

Jeu	Clients	Rôle
Entraînement	7 200	ajuster les modèles
Calibration (ou validation)	2 400	régler les seuils, calibrer les probabilités, calibrer la prédiction conforme
Test	2 400	évaluer une seule fois , à la fin, sans jamais s'en servir pour décider

Trois modèles sont ajustés sur le jeu d'entraînement, avec leurs réglages par défaut (le réglage fin est l'objet de la section 1.5) : une **régression logistique** sur variables centrées-réduites, une **forêt aléatoire** (150 arbres) et un **gradient boosting**. Voici leurs scores sur le jeu de test, mesurés avec les outils de la section suivante :

Modèle	AUC	Précision moyenne (AP)	Log-loss	Brier
Régression logistique	0,859	0,560	0,289	0,0872
Forêt aléatoire	0,888	0,624	0,269	0,0808
Gradient boosting	0,897	0,650	0,255	0,0758

Dans ce tableau, le boosting domine sur toutes les colonnes. Dans la pratique, on voudrait savoir **de combien** il domine, si la différence est due au hasard, ce qu'elle change pour la décision et si ses probabilités sont crédibles : c'est précisément ce que ce chapitre apprend à faire.

Deux jeux de données interviendront en plus : le **jeu réel** `credit_default.csv` (30 000 clients d'une banque, 22 % de défauts ; source : Yeh et Lien, 2009, licence CC0) pour l'équité en 5.4, et les mêmes clients de la boutique pour la prédiction conforme en 5.5.

Pour s'entraîner. Cahier, chapitre 5 : le chapitre tout entier est accompagné d'applications et d'exercices ; chaque section ci-dessous indique les siens.

5.1 Métriques

Une métrique répond à une question précise. Choisir celle qui répond à la **bonne** question, avant de comparer des modèles, évite des semaines de travail dans la mauvaise direction. Cette section suit un fil : on part d'un modèle qui sort un **score** par client, on en fait une **décision** (contacter ou non) en choisissant un seuil, puis on mesure la **valeur** de cette décision.

5.1.1 Ce que l'on mesure : un score, une décision, une valeur

Trois objets se confondent souvent, et chacun a ses mesures.

- Le **score** (ou la probabilité) que le modèle attribue à chaque client. On le juge avec des mesures indépendantes de tout seuil : l'**AUC**, la **précision moyenne**, la **log-loss**, le **score de Brier**.
- La **décision** obtenue en comparant le score à un **seuil** : « contacter si le risque dépasse t ». On la juge avec des mesures qui dépendent du seuil : exactitude, précision, rappel, F_1 ...
- La **valeur** de cette décision, en euros : ce que rapporte la campagne, une fois déduits ses coûts. C'est la seule mesure qui compte pour la gérante, et celle qui permet de choisir le seuil.

Intuition. Un thermomètre (le score) n'est pas un diagnostic (la décision), et un diagnostic n'est pas un traitement réussi (la valeur). Un bon thermomètre est nécessaire ; il n'est jamais suffisant.

Dans toute la suite, le **cas positif est le départ** ($\text{churn}_{90j} = 1$). On appelle *positif* un client qui part et *alerte* un client que le modèle signale comme risqué.

5.1.2 La matrice de confusion et ses dérivés

Fixons un seuil. Chaque client tombe dans l'une des quatre cases du tableau croisant la réalité et la prédiction :

	Prédit : fidèle	Prédit : partant
Réel : fidèle	vrais négatifs (VN)	faux positifs (FP), les <i>fausses alertes</i>
Réel : partant	faux négatifs (FN), les <i>départs manqués</i>	vrais positifs (VP)

Un exemple entièrement à la main. Sur 1 000 clients, 140 vont partir. Le modèle émet 120 alertes, dont 84 sont justes. Il y a donc $VP = 84$, $FP = 120 - 84 = 36$, $FN = 140 - 84 = 56$ et $VN = 1\,000 - 84 - 36 - 56 = 824$. À partir de ces quatre nombres :

- l'**exactitude** (*accuracy*) est la part de bonnes réponses : $\frac{VP + VN}{n} = \frac{84 + 824}{1000} = 0,908$;
- la **précision** est la part d'alertes justes : $\frac{VP}{VP + FP} = \frac{84}{120} = 0,70$;
- le **rappel** (*recall*, ou *sensibilité*, ou taux de vrais positifs) est la part des départs détectés : $\frac{VP}{VP + FN} = \frac{84}{140} = 0,60$;
- la **spécificité** est la part des fidèles correctement laissés en paix : $\frac{VN}{VN + FP} = \frac{824}{860} \approx 0,958$; son complément, $1 - \text{spécificité}$, est le **taux de fausses alertes** (FPR) ;
- le F_1 est la moyenne **harmonique** de la précision et du rappel : $F_1 = \frac{2PR}{P + R} = \frac{2 \times 0,7 \times 0,6}{1,3} \approx 0,646$.

Le F_1 pondère précision et rappel également. Si manquer un départ coûte plus cher qu'une fausse alerte, on utilise sa généralisation, le F_β : $F_\beta = \frac{(1 + \beta^2) PR}{\beta^2 P + R}$. Avec $\beta = 2$, le rappel compte deux fois plus que la précision ($F_2 \approx 0,618$ ici) ; avec $\beta = 0,5$, c'est l'inverse ($F_{0,5} \approx 0,677$).

Enfin, le **coefficient de corrélation de Matthews** (MCC) résume les quatre cases d'un seul nombre entre -1 et 1 :
$$\text{MCC} = \frac{VP \cdot VN - FP \cdot FN}{\sqrt{(VP + FP)(VP + FN)(VN + FP)(VN + FN)}}$$
, soit
$$\frac{69\,216 - 2\,016}{\sqrt{120 \cdot 140 \cdot 860 \cdot 880}} \approx 0,596$$
. C'est la corrélation entre prédiction et réalité ; elle reste informative même quand les classes sont très déséquilibrées.

Le piège de l'exactitude. Ici, un modèle qui répondrait « personne ne part » aurait une exactitude de $860/1000 = 86\%$. Notre modèle à $90,8\%$ ne fait donc que quatre points de mieux, alors qu'il a détecté 60% des départs. Quand une classe est rare, l'exactitude récompense surtout la prudence. Comparez-la toujours à celle du modèle « toujours la classe majoritaire ».

Voyons maintenant ces mesures sur le **vrai** jeu de test (2 400 clients dont 337 partants), avec le boosting et le seuil habituel de $0,5$.

Voici les quatre cases :

	Prédit : fidèle	Prédit : partant
Réel : fidèle (2 063)	1 996	67
Réel : partant (337)	174	163

On en tire : exactitude $0,900$ (contre $0,860$ pour « personne ne part »), précision $0,709$, rappel $0,484$, spécificité $0,968$, $F_1 = 0,575$, $F_2 = 0,517$, $F_{0,5} = 0,648$ et $\text{MCC} = 0,533$. La lecture est limpide : quand le modèle alerte, il a raison sept fois sur dix, mais il laisse passer plus de la moitié des départs.

```
from sklearn.metrics import confusion_matrix, precision_score, recall_score, roc_auc_score

yhat = (p >= 0.5).astype(int) # p : probabilités prédites du boosting sur le jeu de
↪ test
print(confusion_matrix(yt, yhat)) # lignes : réalité ; colonnes : prédiction
print(round(precision_score(yt, yhat), 3), round(recall_score(yt, yhat), 3), round(roc_auc_score(yt,
↪ p), 3))
```

```
[[1996  67]
 [ 174 163]]
0.709 0.484 0.897
```

Ces mesures sont disponibles en une ligne dans `scikit-learn`. Le plus difficile est de les lire.

5.1.3 Le seuil change tout

Le seuil de $0,5$ n'a rien de sacré : il n'est « naturel » que si fausses alertes et départs manqués coûtent pareil. Voyons ce que devient la même sortie du modèle pour d'autres seuils :

Seuil	Alertes	Précision	Rappel	F_1
0,1	692	0,399	0,819	0,536
0,2	460	0,517	0,706	0,597
0,3	350	0,603	0,626	0,614
0,5	230	0,709	0,484	0,575

En abaissant le seuil, on signale plus de clients : le **rappel monte** (on détecte plus de départs) mais la **précision baisse** (plus de fausses alertes). C'est un **compromis**, pas un réglage à optimiser une fois pour toutes. Le F_1 est maximal autour de 0,3, mais ce « maximum » ne dit rien de ce qui compte vraiment pour la boutique ; nous y reviendrons en 5.1.7 en passant aux coûts.

Ce qu'il faut retenir du seuil. Le même modèle peut être un détecteur prudent (précision 71 %, rappel 48 %) ou un détecteur large (précision 40 %, rappel 82 %). Comparer deux modèles à un seuil fixé arbitrairement mélange la qualité du score et le choix du seuil. Pour juger le **score** seul, il faut des mesures qui balayent tous les seuils.

5.1.4 La courbe ROC et l'AUC

La **courbe ROC** (*Receiver Operating Characteristic*) balaie tous les seuils possibles. Pour chaque seuil t , elle place un point d'abscisse le taux de fausses alertes $FPR(t)$ et d'ordonnée le rappel $TPR(t)$. Un seuil très élevé ne signale personne : on est en $(0,0)$. Un seuil nul signale tout le monde : on est en $(1,1)$. Entre les deux, plus la courbe se rapproche du coin supérieur gauche $(0,1)$, meilleur est le score ; la diagonale correspond au hasard.

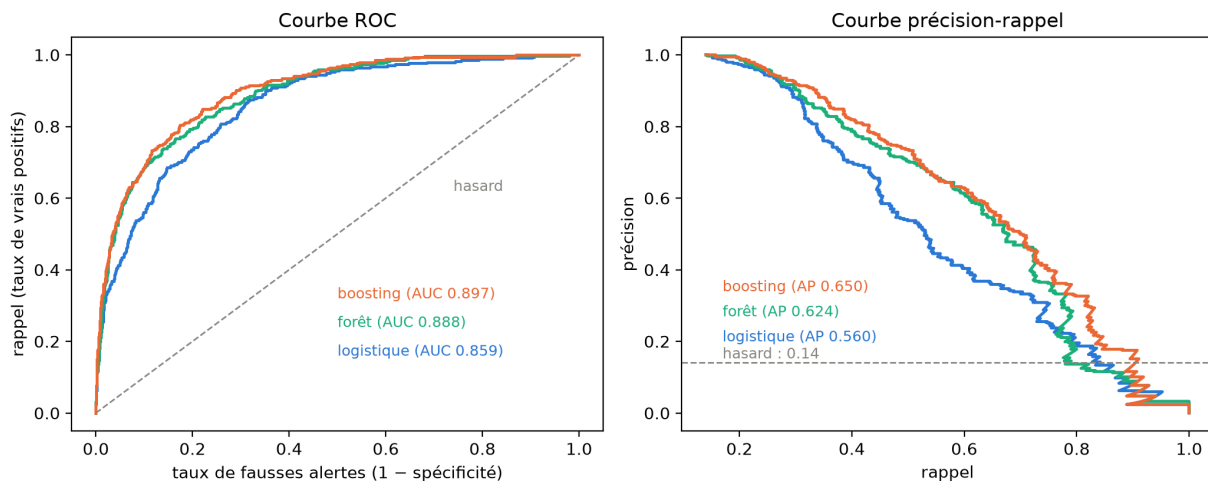


Figure 54: Courbes ROC (à gauche) et précision-rappel (à droite) des trois modèles sur le jeu de test. Le boosting domine, mais l'écart est bien plus lisible sur la courbe précision-rappel.

L'aire sous la courbe ROC (AUC, Area Under the Curve) résume la courbe en un nombre entre 0,5 (hasard) et 1 (classement parfait). Elle a une interprétation probabiliste remarquable.

Démonstration : l'AUC est une probabilité. Notons S^+ le score d'un client qui part tiré au hasard et S^- celui d'un client fidèle tiré au hasard, indépendamment. Pour un seuil t , $TPR(t) = P(S^+ > t)$ et $FPR(t) = P(S^- > t)$. Quand t parcourt les valeurs de la plus grande à la plus petite, FPR augmente de 0 à 1, et sa « vitesse » est la densité f^- de S^- : $dFPR = -f^-(t) dt$. Donc

$$AUC = \int TPR dFPR = \int_{-\infty}^{+\infty} P(S^+ > t) f^-(t) dt = P(S^+ > S^-),$$

la dernière égalité venant de la formule des probabilités totales en conditionnant par la valeur $S^- = t$. (En cas d'égalité de scores, on compte une demi-paire.) ■

L'AUC est donc la probabilité qu'un client qui part ait un score plus élevé qu'un client qui reste. Une AUC de 0,90 signifie : si l'on prend un partant et un fidèle au hasard, le modèle les

classe dans le bon ordre neuf fois sur dix. Sur notre jeu de test, il y a $337 \times 2\,063 = 695\,231$ paires (partant, fidèle) ; en les comparant une à une, on trouve exactement la même valeur que le calcul de l'AUC : **0,8971**.

Ce résultat est la même chose que la **statistique U du test de Mann-Whitney** (volume I, section 3.7.2) : $AUC = U/(n^+n^-)$. Tester si deux modèles ont la même AUC, c'est comparer des rangs.

L'AUC ne dit rien de la calibration ni du seuil. Deux scores qui rangent les clients dans le même ordre ont exactement la même AUC, même si l'un annonce 1 % et l'autre 90 % pour le même client. L'AUC mesure le **classement**, pas la justesse des probabilités (voir 5.2).

5.1.5 Précision-rappel : ce que la ROC ne voit pas

Sur la courbe de droite de la figure, on trace la **précision en fonction du rappel**. Son résumé est la **précision moyenne** (*average precision*, AP) : $AP = \sum_n (R_n - R_{n-1})P_n$, la moyenne des précisions obtenues à chaque nouveau départ détecté. Le « hasard » n'est pas une diagonale : c'est une droite horizontale à hauteur de la **prévalence** (14 % ici).

Pourquoi deux courbes ? Parce qu'elles ne réagissent pas de la même façon quand les positifs sont rares. La ROC compare des **taux** calculés séparément parmi les positifs et parmi les négatifs ; la précision, elle, mélange les deux populations : sa valeur dépend du **nombre de négatifs**. Une expérience le montre bien. Multiplions (par un poids) le nombre de clients fidèles par 10, sans changer les scores : la prévalence tombe de 14 % à 1,6 %.

	Avant	Après (négatifs $\times 10$)
Prévalence	0,140	0,016
AUC	0,8971	0,8971
Précision moyenne	0,6496	0,2343

L'AUC n'a pas bougé d'un millièmme ; la précision moyenne s'est effondrée. Elle a raison : avec 10 fois plus de fidèles, une alerte a beaucoup plus de chances d'être une fausse alerte. La ROC rend ce modèle « aussi bon qu'avant » parce qu'elle regarde le classement ; la courbe précision-rappel rend compte de ce que l'on vivra en réalité, à savoir une montagne de fausses alertes.

Règle pratique. Quand les positifs sont rares (fraude à 1 %, départ à 5 %...) et que ce sont eux qui intéressent, **regardez la courbe précision-rappel**. L'AUC reste pratique pour comparer des classements, mais elle peut rester élevée (0,95 !) pour un modèle dont 90 % des alertes sont fausses.

5.1.6 Juger les probabilités : log-loss et score de Brier

Les mesures précédentes ne regardent que l'**ordre** des scores. Si l'on veut utiliser la probabilité elle-même (pour calculer un coût attendu, par exemple), il faut une mesure qui juge sa **justesse**. Deux sont standard. Pour n clients de probabilités prédites $\hat{p}_i$ et de résultats $y_i \in \{0, 1\}$:

$$\text{log-loss} = -\frac{1}{n} \sum_i [y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)], \quad \text{Brier} = \frac{1}{n} \sum_i (\hat{p}_i - y_i)^2.$$

La log-loss est la **vraisemblance négative** : c'est ce que minimise la régression logistique (volume II, section 2.2). Elle punit très fort un modèle qui annonce « 1 % » pour un événement qui arrive. Le Brier est l'erreur quadratique moyenne des probabilités : plus doux, mais plus robuste.

Ces deux mesures sont des **règles de score propres** : en espérance, elles sont minimisées en annonçant la **vraie** probabilité. Pour le Brier, c'est immédiat. Si la vraie probabilité d'un départ, pour un type de client, est q , et que l'on annonce p , alors $E[(p - Y)^2] = (p - q)^2 + q(1 - q)$: le premier terme s'annule exactement pour $p = q$, et le second est la part d'incertitude irréductible. Impossible de « tricher » en exagérant ou en minimisant.

Il faut un point de repère. Le modèle constant, qui annonce 14 % à tout le monde, a un Brier de $0,14 \times 0,86 \approx 0,1207$ et une log-loss de 0,4057. Notre boosting obtient 0,0758 et 0,2548, soit un **gain de 37 % sur le Brier** (le « score de compétence » $1 - 0,0758/0,1207$). La régression logistique est à 0,0872 et 0,2886 : meilleure que le hasard, moins bonne que la forêt (0,0808 ; 0,2687) et que le boosting. L'ordre est le même qu'avec l'AUC, mais ce n'est pas une règle : on verra en 5.2 des modèles dont l'AUC est bonne et le Brier mauvais.

5.1.7 Choisir le seuil par les coûts

Voici enfin la manière de choisir un seuil qui ait un sens. La gérante veut proposer une offre de rétention aux clients à risque. Chaque contact coûte 5 €. Un client contacté qui allait partir est « sauvé » avec une probabilité de 40 %, et un client sauvé vaut 60 € de marge future. Pour un client dont la probabilité de départ est p , le **gain net attendu** d'un contact est

$$g(p) = p \times 0,4 \times 60 - 5 = 24p - 5.$$

Le seuil optimal. Il faut contacter si et seulement si $g(p) > 0$, c'est-à-dire si $p > \frac{5}{24} \approx 0,208$. Plus généralement, avec un coût de contact c , une probabilité de succès s et une valeur V , le seuil optimal est $t^* = \frac{c}{sV}$. Il ne dépend que de l'économie du problème, pas du modèle. (Sous la forme classique des « coûts de mauvaise classification », $t^* = \frac{C_{FP}}{C_{FP} + C_{FN}}$.)

Cette formule suppose que p est une **vraie** probabilité : il faut donc un modèle calibré (section 5.2). Vérifions-la sur le jeu de test, en comptant pour chaque seuil le gain net réalisé :

	Seuil	Clients contactés	Gain net
	0,10	692	3 164 €
	0,20	460	3 412 €
	0,208 (théorique)	446	3 386 €
	0,30	350	3 314 €
	0,50 (habituel)	230	2 762 €
	contacter tout le monde	2 400	-3 912 €

Le message est triple. D'abord, **le seuil de 0,5 laisse environ 700 € sur la table** (2 762 € contre 3 473 € au meilleur seuil de la grille, 0,18). Ensuite, le seuil théorique (0,208) donne 3 386 €, soit 87 € de moins que le maximum observé : la courbe est plate autour de l'optimum, et l'optimum exact sur un jeu de 2 400 clients est bruité. Enfin, **contacter tout le monde perd de l'argent** : c'est ce que fait, en pratique, une campagne sans modèle.

Le gain mesuré sur le jeu de test sert à illustrer, pas à choisir. Pour choisir le seuil dans un vrai projet, on utilise le jeu de **calibration** (ou la validation croisée), puis on mesure une dernière fois sur le jeu de test.

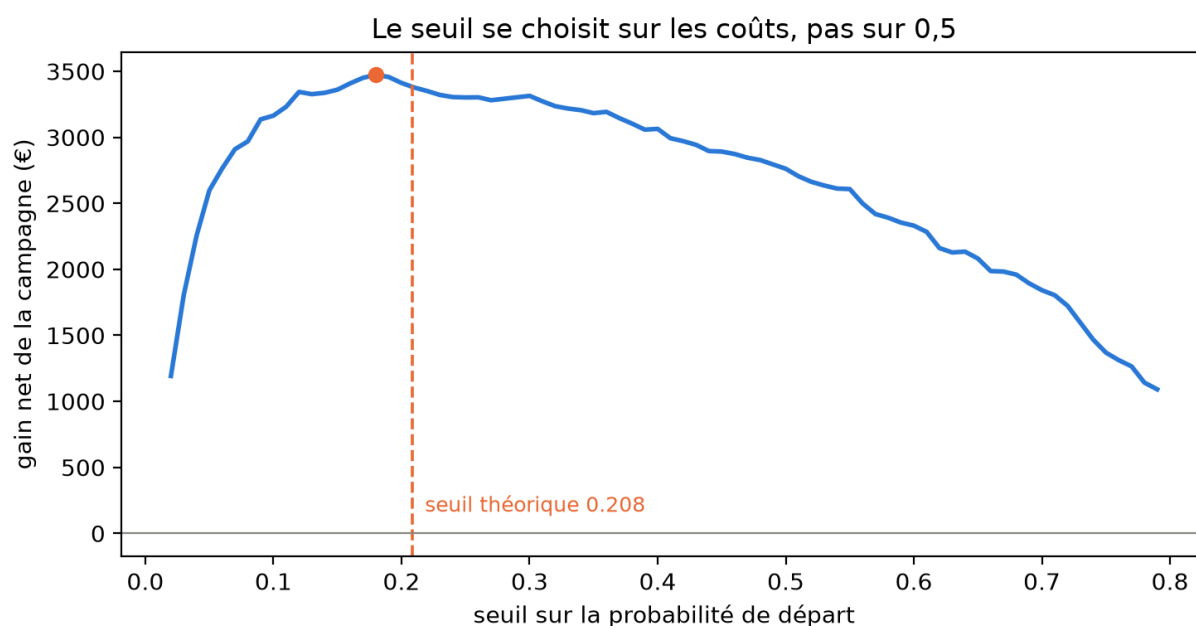


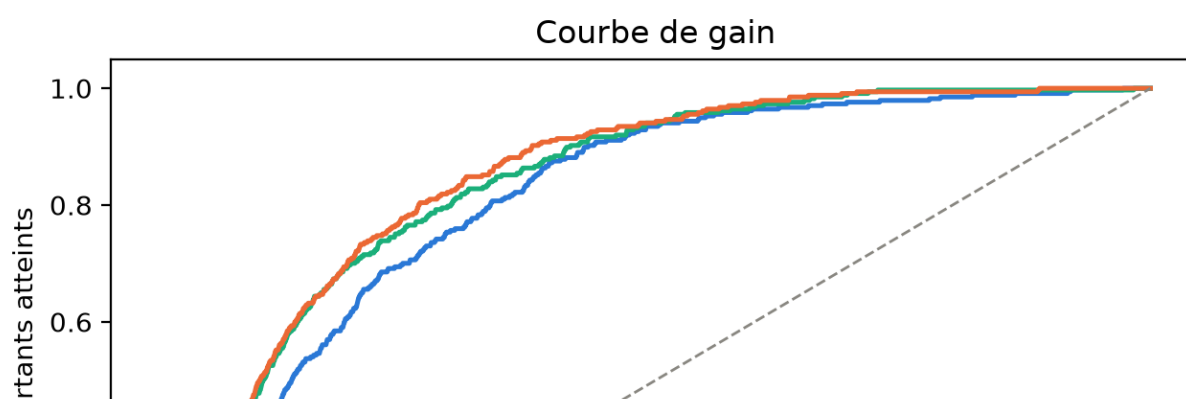
Figure 55: Gain net de la campagne selon le seuil. Le maximum observé est atteint vers 0,18 ; la courbe est plate entre 0,15 et 0,30.

5.1.8 Gain et lift : la lecture du marketing

Une campagne n'a souvent pas de seuil : elle a un **budget** (« nous pouvons contacter 20 % des clients »). On classe alors les clients par score décroissant, on les découpe en déciles, et on regarde où se trouvent les partants.

Décile	Clients	Partants	Taux de départ	Lift	Part cumulée des partants
1	240	168	70,0 %	4,99	49,9 %
2	240	76	31,7 %	2,26	72,4 %
3	240	36	15,0 %	1,07	83,1 %
4	240	27	11,2 %	0,80	91,1 %
5	240	10	4,2 %	0,30	94,1 %
6	240	11	4,6 %	0,33	97,3 %
7	240	6	2,5 %	0,18	99,1 %
8	240	1	0,4 %	0,03	99,4 %
9	240	2	0,8 %	0,06	100,0 %
10	240	0	0,0 %	0,00	100,0 %

Le **lift** d'un décile est son taux de départ divisé par le taux moyen (14 %) : le premier décile « vaut » 4,99 fois un tirage au hasard. La dernière colonne donne la **courbe de gain : en contactant 20 % des clients seulement, on atteint 72 % des partants**. Aucun partant dans le dernier décile : le modèle repère bien les clients qui ne partiront pas, ce qui permet de ne pas les déranger.



À quoi sert cette lecture ? Elle parle le langage du budget : « pour 20 % de l'effort, 72 % du résultat ». Elle ne demande aucun seuil, et elle se compare directement à un tirage au hasard, ce qui la rend lisible par des non-spécialistes.

5.1.9 Plus de deux classes

Quand la cible a plus de deux modalités, on garde la matrice de confusion (une ligne par classe réelle), et on **moyenne** les métriques par classe de trois façons :

- la moyenne **macro** calcule la métrique par classe, puis moyenne sans pondération : chaque classe pèse autant ;
- la moyenne **micro** additionne tous les VP, FP, FN avant de calculer : chaque *client* pèse autant (pour un problème à une seule étiquette par client, le F_1 micro est égal à l'exactitude) ;
- la moyenne **pondérée** moyenne par classe, avec des poids proportionnels aux effectifs.

Pour illustrer, tentons de reconnaître à quel **segment** appartient un client (4 segments latents simulés, dont les proportions sont 39, 29, 20 et 12 %) à partir de ses comportements. Ce n'est qu'une démonstration : ailleurs, ce segment ne sert jamais d'entrée.

Segment réel prédit	0	1	2	3	Rappel
0 (936 clients)	873	44	0	19	0,933
1 (703)	30	655	5	13	0,932
2 (479)	0	1	478	0	0,998
3 (282)	31	7	0	244	0,865

L'exactitude est de 0,9375, le F_1 micro lui est égal (0,9375), le F_1 pondéré vaut 0,9374 et le F_1 **macro** 0,9328. Le macro est le plus bas parce qu'il donne autant de poids à la plus petite classe (les « grands paniers », 12 % des clients, rappel 0,865) qu'à la plus grande. **Si une classe rare est précisément celle qui vous importe, c'est le macro, ou le rappel par classe, qu'il faut regarder.**

5.1.10 Régression : choisir sa perte

Pour une cible numérique, les métriques mesurent l'écart entre la valeur prédite $\hat{y}_i$ et la valeur réelle y_i . Prenons la **dépense des six mois suivants** (en euros). Elle est difficile à prédire : 36 % des clients du jeu de test ne dépensent rien, et le reste a une distribution très étalée (moyenne 83,2 €, écart-type 125,7 €).

Mesure	Formule	Remarque
MAE (erreur absolue moyenne)	$\frac{1}{n} \sum y_i - \hat{y}_i $	en euros ; robuste aux valeurs extrêmes ; cible la médiane
RMSE (racine de l'erreur quadratique)	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	en euros ; punit les gros écarts ; cible la moyenne
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	part de variance expliquée par rapport à la moyenne
MAPE	$\frac{1}{n} \sum \frac{ y_i - \hat{y}_i }{ y_i }$	erreur relative ; indéfinie si $y_i = 0$, asymétrique
Perte pinball	$\max(\tau(y - q), (\tau - 1)(y - q))$	juge un quantile q de niveau τ

Un gradient boosting entraîné pour minimiser l'erreur quadratique obtient sur le jeu de test **MAE = 59,2 €, RMSE = 98,6 € et $R^2 = 0,385$** . Pour savoir si c'est bien, il faut des références : prédire

la moyenne d'entraînement à tout le monde donne $MAE = 83,5$ € et $RMSE = 125,7$ € ; prédire la médiane (41,9 €) donne $MAE = 75,1$ €. Le modèle réduit donc l'erreur absolue de 29 % par rapport à la moyenne. On vérifie la cohérence du R^2 : $1 - (98,6/125,7)^2 \approx 0,385$.

MAE et RMSE ne mesurent pas la même chose. Le RMSE est toujours supérieur ou égal au MAE, et l'écart entre les deux dit si les erreurs sont régulières ou concentrées. Ici, les 1 % de plus grosses erreurs représentent 35 % de l'erreur quadratique mais seulement 10 % de l'erreur absolue. Si quelques gros clients mal prédits sont l'enjeu, le RMSE est le bon juge ; si l'on veut juger le client « typique », c'est le MAE.

Le piège du MAPE. Il est calculé en divisant par la valeur réelle : impossible pour les 860 clients du jeu de test dont la dépense est nulle. En se limitant aux dépenses positives, on trouve 60 %, un nombre qui dépend beaucoup de la façon dont on traite les petits montants. Quant au sMAPE (qui divise par la moyenne de $|y|$ et $|\hat{y}|$), il vaut ici 1,07, car chaque prédiction positive d'une dépense nulle donne une erreur de 200 %. **Sur une cible qui contient des zéros, évitez les erreurs relatives.**

Prédire un quantile. Parfois, on ne veut pas la valeur la plus probable mais une valeur « haute » : *quelle dépense sera dépassée dans seulement 10 % des cas ?* (pour dimensionner un stock, par exemple). On entraîne alors le modèle avec la **perte pinball** de niveau $\tau = 0,9$, qui pénalise davantage de sous-estimer que de surestimer. Le modèle quantile obtient une perte pinball de 17,2, contre 29,0 pour le modèle de la moyenne utilisé comme prédicteur de quantile. Mais il faut vérifier sa **couverture** : seuls 85,8 % des clients ont une dépense inférieure au quantile annoncé, et non 90 %. (Le modèle de la moyenne, lui, en couvre 61,2 %.) Un quantile estimé n'est pas garanti : en 5.5, on le réparera.

À retenir.

- L'exactitude se compare toujours au modèle « classe majoritaire » ; avec des classes rares, préférez rappel, précision, F_β , MCC.
- L'AUC est la probabilité qu'un positif ait un score supérieur à celui d'un négatif (statistique de Mann-Whitney) : elle mesure le classement, pas la justesse des probabilités, et elle ne voit pas la prévalence.
- Avec des positifs rares, la **courbe précision-rappel** et l'AP disent ce que l'AUC cache.
- La **log-loss** et le **Brier** jugent les probabilités ; ce sont des règles de score propres.
- Le seuil se déduit des **coûts** : $t^* = c/(sV)$ pour une campagne, à condition que les probabilités soient calibrées.
- Le **lift** et la courbe de gain parlent le langage du budget.
- En régression, **MAE** (médiane) et **RMSE** (moyenne) répondent à des questions différentes ; évitez le MAPE avec des zéros ; la perte **pinball** juge un quantile.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 à 5.4, exercices 5.1 à 5.6.

5.2 Calibration

Une probabilité n'est utile que si on peut la prendre au mot. Quand la gérante lit « 30 % » sur la fiche d'un client, elle en déduit un coût attendu, un seuil, une priorité. Si ce « 30 % » est en réalité « 55 % », tout ce qui en découle est faux, même si le classement des clients est parfait. Cette section apprend à **vérifier** que les probabilités sont justes, à comprendre pourquoi elles ne le sont pas toujours, et à les **réparer**.

5.2.1 Qu'est-ce qu'une probabilité « juste » ?

Un modèle est **calibré** si, parmi tous les clients à qui il attribue une probabilité p , une proportion p d'entre eux subit effectivement l'événement :

$$P(Y = 1 \mid \hat{p} = p) = p \quad \text{pour tout } p.$$

Il s'agit d'une propriété distincte de la **discrimination** (la capacité à bien classer, mesurée par l'AUC). Un score peut classer parfaitement et être très mal calibré : si l'on divise toutes les probabilités d'un modèle parfaitement calibré par deux, l'ordre reste identique (l'AUC ne bouge pas), mais plus aucune probabilité n'est juste.

Un exemple à la main. Vingt clients : dix reçoivent la probabilité 0,10 et deux d'entre eux partent (taux observé 0,20) ; les dix autres reçoivent 0,60 et cinq partent (taux observé 0,50). Le modèle est trop optimiste dans le premier groupe (il annonce 10 %, la réalité est de 20 %) et trop pessimiste dans le second (60 % annoncés, 50 % observés).

On le résume avec le **diagramme de fiabilité** (*reliability diagram*) : on regroupe les clients en classes de probabilités voisines (ici, par classes de même effectif), et l'on trace, pour chaque classe, le **taux observé** en fonction de la **probabilité moyenne prédite**. Un modèle calibré suit la diagonale. L'**erreur de calibration attendue** (ECE) est l'écart moyen à la diagonale, pondéré par l'effectif n_b de chaque classe b :

$$\text{ECE} = \sum_b \frac{n_b}{n} |\bar{p}_b - \bar{y}_b|.$$

Sur notre exemple, $\text{ECE} = \frac{1}{2} \times 0,10 + \frac{1}{2} \times 0,10 = 0,10$.

L'ECE est bruitée. Avec 10 classes de 240 clients, le taux observé d'une classe où le risque est de 14 % fluctue naturellement de $\sqrt{0,14 \times 0,86/240} \approx 0,022$. Une ECE de 0,01 ou 0,02 est donc **indiscernable d'une calibration parfaite** sur ce jeu. L'ECE dépend aussi du découpage en classes. Elle se lit avec le diagramme, jamais seule.

5.2.2 Qui est bien calibré, qui ne l'est pas ?

Comparons cinq modèles sur le jeu de test : les trois du chapitre, plus deux modèles « abîmés » à dessein : un **boosting surajusté** (300 itérations, taux d'apprentissage 0,3, arbres de 63 feuilles) et une **régression logistique avec poids de classes** (`class_weight="balanced"`, qui donne autant de poids aux 14 % de partants qu'aux 86 % de fidèles).

Modèle	AUC	Brier	Log-loss	ECE	Probabilité moyenne
Régression logistique	0,859	0,0872	0,2886	0,012	0,136
Forêt aléatoire	0,888	0,0808	0,2687	0,031	0,139
Boosting (réglages par défaut)	0,897	0,0758	0,2548	0,014	0,128
Boosting surajusté	0,876	0,1010	0,7143	0,098	0,094
Logistique pondérée	0,857	0,1542	0,4588	0,209	0,349

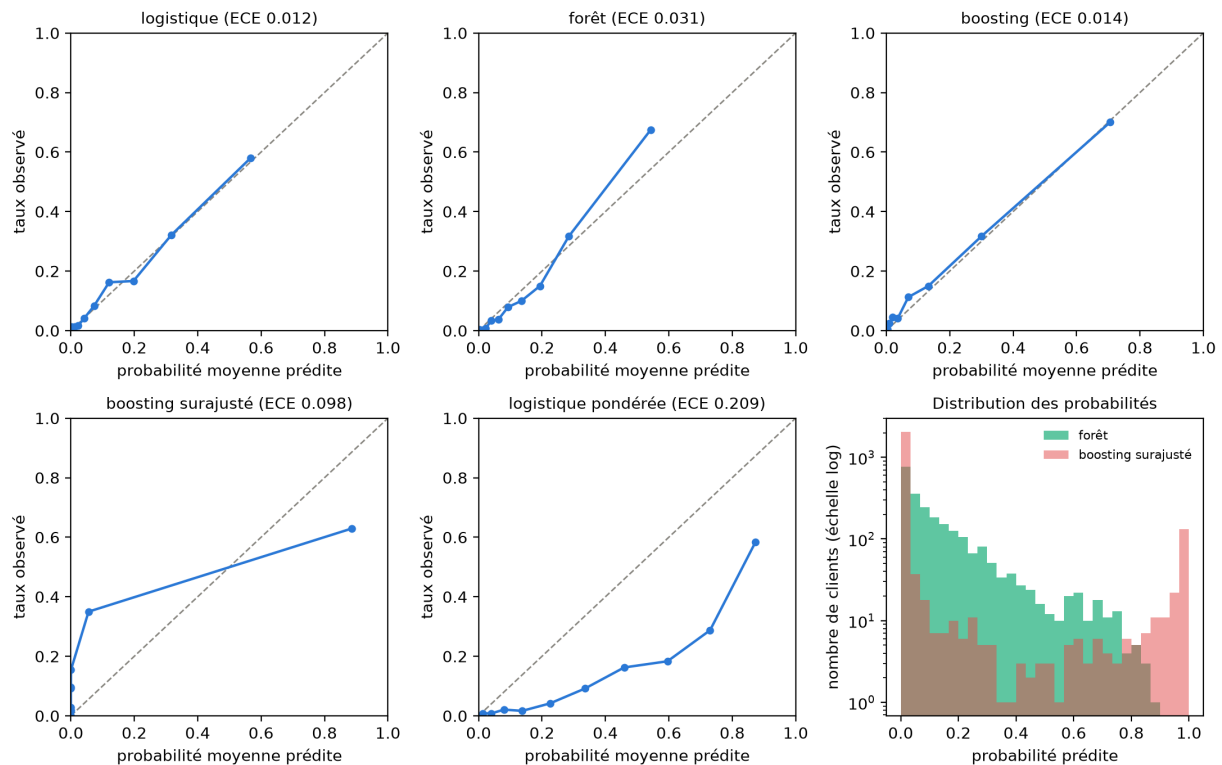


Figure 57: Diagrammes de fiabilité des cinq modèles sur le jeu de test (classes de même effectif), et distribution des probabilités de deux d'entre eux. Un modèle calibré suit la diagonale.

(La prévalence observée sur le jeu de test est de 0,140.)

Chaque modèle raconte une histoire différente.

La régression logistique est calibrée par construction. Elle maximise la vraisemblance, donc annule la dérivée de la log-vraisemblance par rapport à l'ordonnée à l'origine : $\sum_i (y_i - \hat{p}_i) = 0$. La somme des probabilités prédites sur le jeu d'entraînement est égale au nombre de partants : en moyenne, le modèle est juste (probabilité moyenne 0,136 pour une prévalence de 0,140). Quand le modèle est correctement spécifié, cette justesse vaut aussi à tous les niveaux. Et même s'il ne l'est pas, la calibration reste en général très honorable.

La forêt est trop prudente aux extrêmes. Elle moyenne les proportions observées dans les feuilles de ses arbres, avec au moins 5 clients par feuille : ses probabilités sont tirées vers le milieu. Dans la classe des clients les plus à risque, elle annonce 54 % pour un taux observé de 68 % ; dans la classe la moins à risque, elle annonce 0,3 % pour 0,4 %. C'est un modèle *sous-confiant* (ECE 0,031, la plus mauvaise des trois modèles « sains »).

Le boosting par défaut est bien calibré (ECE 0,014, log-loss 0,255) : il optimise la log-loss, avec un petit pas d'apprentissage, et s'arrête avant de surajuster. Mais le **boosting surajusté** est le contraire de la forêt : il est *sur-confiant*. Ses probabilités se collent à 0 ou à 1 (huit classes sur dix ont une probabilité moyenne prédite inférieure à 0,001), et la classe des plus à risque reçoit 89 % pour un taux observé de 63 %. Sa log-loss est de 0,714, près de trois fois celle du boosting réglé. Retenez qu'un boosting n'est bien calibré que si on le laisse peu s'entraîner.

La logistique pondérée classe aussi bien que la logistique ordinaire (AUC 0,857 contre 0,859) **mais ses probabilités sont fausses** : elle annonce en moyenne 35 % de départs alors qu'ils sont 14 %. Les poids de classes déplacent la prévalence apparente. On y revient en 5.2.6.

Ce que montre ce tableau. L'AUC et la calibration sont indépendantes. La logistique pondérée a une AUC quasi identique à celle de la logistique ordinaire et un Brier presque deux fois plus mauvais (0,154 contre 0,087). Ne choisissez pas un modèle sur la seule AUC si vous comptez utiliser ses probabilités.

Et les autres modèles ? Les SVM produisent des scores qui sont des distances à une frontière, pas des probabilités (section 2.5) ; les k plus proches voisins annoncent des proportions de voisins, tirées vers les extrêmes quand k est petit ; le Bayes naïf est souvent sur-confiant, car l'indépendance supposée entre variables compte plusieurs fois la même information. Dans tous les cas, **vérifiez**.

5.2.3 Réparer : la méthode de Platt

Si le défaut de calibration est une déformation régulière des probabilités, on peut la corriger après coup par une **recalibration** : on apprend une fonction g telle que $g(\hat{s})$ soit calibré, où $\hat{s}$ est le score du modèle. La première méthode est celle de **Platt** : on suppose que le log-odds de la probabilité vraie est une fonction affine du log-odds du score,

$$P(Y = 1 \mid \hat{s}) = \sigma(a \cdot \text{logit}(\hat{s}) + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Deux paramètres (a, b) , estimés par maximum de vraisemblance : c'est une **régression logistique à une variable** (le logit du score), ajustée sur le jeu de **calibration**, qui n'a pas servi à entraîner le modèle.

Les paramètres s'interprètent. Une **pente** $a > 1$ étire les probabilités vers les extrêmes (corrige un modèle sous-confiant) ; une pente $a < 1$ les écrase vers le centre (corrige un modèle sur-confiant) ; l'**ordonnée** b déplace le niveau global. Sur nos trois modèles abîmés :

Modèle	Pente a	Ordonnée b	Lecture
Forêt	1,42	0,55	étire : le modèle était sous-confiant
Boosting surajusté	0,28	-0,19	écrase fortement : il était sur-confiant
Logistique pondérée	1,06	-1,79	quasi pas de changement de forme, mais un grand décalage de niveau

5.2.4 Réparer : la régression isotonique

Platt suppose une forme précise (une sigmoïde). La **régression isotonique** n'en suppose aucune, sauf que la probabilité vraie est une fonction **croissante** du score. Elle ajuste, par moindres carrés, la fonction en escalier croissante la plus proche des résultats observés sur le jeu de calibration (par l'algorithme classique « pool adjacent violators » : on fusionne les marches voisines qui violent l'ordre). Elle est donc plus flexible, mais elle demande **plus de données** (compter au moins quelques milliers d'exemples), et elle produit des paliers : plusieurs clients reçoivent exactement la même probabilité.

Voici, pour les trois modèles abîmés, l'effet des deux méthodes, ajustées sur les 2 400 clients du jeu de calibration et évaluées sur le jeu de test :

Modèle	Méthode	Brier	Log-loss	ECE
Forêt	brut	0,0808	0,2687	0,031
	Platt	0,0785	0,2629	0,009
	isotonique	0,0780	0,2741	0,014
Boosting surajusté	brut	0,1010	0,7143	0,098
	Platt	0,0842	0,2911	0,042
	isotonique	0,0821	0,2734	0,012
Logistique pondérée	brut	0,1542	0,4588	0,209
	Platt	0,0881	0,2906	0,015
	isotonique	0,0887	0,3129	0,019

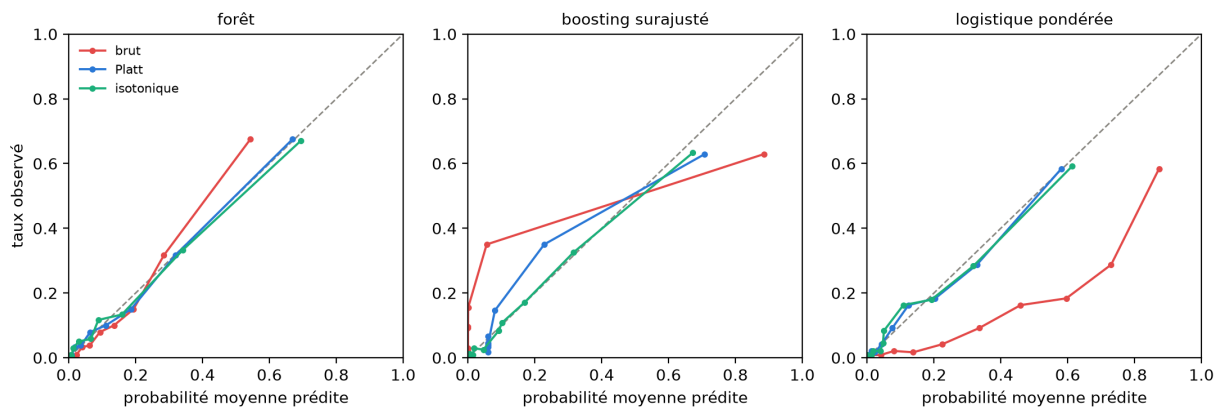


Figure 58: Diagrammes de fiabilité avant (rouge) et après recalibration par Platt (bleu) ou régression isotonique (vert), sur le jeu de test.

La recalibration est spectaculaire pour les deux modèles les plus abîmés : le Brier de la logistique pondérée passe de 0,154 à 0,088, soit quasiment celui de la logistique ordinaire (0,087), et celui du boosting surajusté de 0,101 à 0,082. Pour la forêt, qui était déjà presque calibrée, le gain est modeste mais réel.

Deux nuances. **Platt est plus stable avec peu de données** (deux paramètres seulement) mais ne peut corriger que des déformations en S ; ici, sur le boosting surajusté, il laisse une ECE de 0,042 que la méthode isotonique ramène à 0,012. **L'isotonique, plus souple, peut perdre en log-loss** : sur la forêt, elle passe de 0,2687 à 0,2741, car ses paliers produisent des probabilités proches de 0 pour des clients dont le risque n'est pas nul. Le Brier, plus tolérant, s'améliore dans les deux cas.

5.2.5 Calibrer sans tricher

La recalibration est un **modèle de plus**, qui peut lui aussi surajuster. Trois règles.

1. **Calibrez sur des données que ni le modèle ni vous-même n'avez utilisées pour l'entraîner.** Calibrer sur le jeu d'entraînement revient à corriger un modèle pour les données qu'il connaît déjà par cœur, et donc à ne rien corriger.
2. **Ne touchez pas au jeu de test avant la fin.** Il sert à *mesurer* la calibration, pas à l'obtenir.
3. **Si les données sont comptées, utilisez la validation croisée** : à chaque pli, on ajuste le modèle sur les autres et on calibre sur le pli. `CalibratedClassifierCV` fait cela automatiquement.

```
from sklearn.calibration import CalibratedClassifierCV
```

```

foret = RandomForestClassifier(150, min_samples_leaf=5, random_state=0)
foret_calibree = CalibratedClassifierCV(foret, method="isotonic", cv=3).fit(Xtr, ytr) # 3 plis :
↳ chaque pli sert à calibrer le modèle ajusté sur les deux autres
p_cal = foret_calibree.predict_proba(Xte)[: , 1]
print("Brier :", round(M.brier_score_loss(yte, p_cal), 4), "| ECE :", round(ece(yte.to_numpy(),
↳ p_cal), 4), "| AUC :", round(M.roc_auc_score(yte, p_cal), 4))

```

Brier : 0.0782 | ECE : 0.0089 | AUC : 0.8888

Sur la forêt, la calibration croisée donne un Brier de 0,0782 et une ECE de 0,009, contre 0,0808 et 0,031 avant. L'AUC est quasiment inchangée (0,8888 contre 0,8881) : la recalibration est une fonction croissante des scores, donc elle conserve l'ordre, sauf les égalités que peut créer une fonction en escalier.

Recalibrez après tout changement. Une calibration dépend de la population : si les clients de demain diffèrent de ceux d'hier (nouvelle campagne, nouveau canal), les probabilités dérivent. Un tableau de bord de production doit contenir un diagramme de fiabilité calculé sur les données récentes.

5.2.6 Poids de classes et rééchantillonnage : un décalage de prévalence

Le chapitre 4 (section 4.3) présente des remèdes au déséquilibre de classes : **pondérer** les exemples rares, ou **rééchantillonner** (suréchantillonner les rares, sous-échantillonner les fréquents). Ils améliorent souvent le rappel, mais ils déplacent la prévalence que « voit » le modèle, donc ses probabilités. Heureusement, le décalage est **exactement corrigible**.

Correction d'un décalage de prévalence. D'après la formule de Bayes (volume I, section 2.1.6), la cote *a posteriori* est la cote *a priori* multipliée par un rapport de vraisemblance qui ne dépend que des variables : $\frac{P(Y = 1 | x)}{P(Y = 0 | x)} = \frac{\pi}{1 - \pi} \times \text{LR}(x)$, où π est la prévalence. Un modèle entraîné comme si la prévalence valait π' (avec `class_weight="balanced"`, on a $\pi' = \frac{1}{2}$) apprend $\frac{\pi'}{1 - \pi'} \times \text{LR}(x)$: le même $\text{LR}(x)$, mais un autre *a priori*. On retrouve donc la vraie cote en la multipliant par $\frac{\pi/(1 - \pi)}{\pi'/(1 - \pi')}$.

Avec $\pi = 0,1404$ (prévalence d'entraînement) et $\pi' = \frac{1}{2}$, le facteur est $0,1404/0,8596 \approx 0,163$. Appliquée à la logistique pondérée, la correction ramène la probabilité moyenne de **0,349 à 0,135** (la prévalence du test est 0,140), le Brier de **0,1542 à 0,0881** et l'ECE de **0,209 à 0,015**, sans aucun jeu de calibration et sans changer l'AUC. Cela explique l'ordonnée trouvée par Platt en 5.2.3 : $b = -1,79$, très proche de $\ln(0,163) = -1,81$. **Platt retrouve le décalage de prévalence tout seul.**

Quand faut-il corriger ? Si vous n'utilisez que le **classement** (cibler les 20 % les plus à risque, ou calculer une AUC), le décalage est sans importance : l'ordre est conservé. Si vous utilisez la **probabilité** (calculer un seuil par les coûts comme en 5.1.7, estimer un nombre attendu de départs), il faut corriger ou recalibrer. Le même raisonnement vaut après un suréchantillonnage ou un sous-échantillonnage (section 4.3 et § 4.5) : la prévalence apparente est celle de l'échantillon rééquilibré.

À retenir.

- Un modèle est **calibré** si $P(Y = 1 | \hat{p} = p) = p$. La calibration est indépendante de la

discrimination (AUC).

- Le **diagramme de fiabilité** la révèle ; l'ECE la résume mais est bruitée ($\approx \pm 0,02$ avec 240 clients par classe).
- La régression logistique est calibrée par construction ; les forêts sont sous-confiantes ; un boosting trop entraîné est sur-confiant ; les poids de classes décalent la prévalence.
- **Platt** (deux paramètres, sigmoïde) et la **régression isotonique** (escalier croissant, plus de données) recalibrent sur un jeu **séparé**.
- Un décalage de prévalence se corrige exactement : $\text{cote} \times \frac{\pi/(1-\pi)}{\pi'/(1-\pi')}$.
- Une recalibration doit être **surveillée** : elle dépend de la population.

Pour s'entraîner. Cahier, chapitre 5 : application 5.5, exercices 5.7 et 5.8.

5.3 Interprétabilité

Un modèle de boosting de 150 arbres n'a pas de « formule » que l'on puisse lire. Pourtant, il faut pouvoir répondre à des questions simples : *quelles variables comptent ? Pourquoi ce client est-il signalé ? Le modèle a-t-il appris quelque chose de raisonnable, ou une erreur ?* Cette section présente les outils standard, du plus simple au plus fondé, et surtout leurs **limites**.

5.3.1 Expliquer quoi, et à qui ?

Une « explication » peut répondre à plusieurs questions, qu'il faut distinguer.

- **Globale ou locale.** Une explication *globale* décrit le comportement général du modèle (« la récence est la variable la plus déterminante »). Une explication *locale* décrit **une** prédiction (« ce client est signalé à 92 % surtout parce que sa dernière commande date de plus d'un an »).
- **Intrinsèque ou post hoc.** Certains modèles sont *interprétables par construction* : une régression logistique (chaque coefficient est un effet sur la cote, volume II, section 2.2), un arbre peu profond (section 2.2). Pour les autres, on applique après coup des méthodes qui interrogent le modèle comme une boîte noire.
- **Pour qui ?** La gérante veut une phrase ; le data scientist veut un diagnostic ; l'auditeur veut une traçabilité. L'explication utile dépend de l'usage.

Dans toute la section, le modèle expliqué est un **gradient boosting LightGBM** de 150 arbres (AUC de 0,902 et Brier de 0,0744 sur le jeu de test), entraîné sur les 7 200 clients d'entraînement.

Une explication explique le modèle, pas le monde. Si le modèle dit « les clients de moins de 28 ans partent plus », l'explication le montrera fidèlement. Elle ne dit pas que *l'âge cause le départ* : elle dit que *le modèle utilise l'âge*. La confusion entre les deux est l'erreur la plus fréquente (voir 5.3.7).

5.3.2 L'importance par permutation

L'idée est d'une simplicité désarmante : **si une variable est utile, détruire son information doit dégrader le modèle**. On mesure la performance sur le jeu de test (ici l'AUC), puis on **mélange au hasard** les valeurs d'une seule variable (on casse son lien avec la cible et avec les autres variables) et l'on mesure à nouveau. La **baisse de performance** est l'importance de cette variable. On répète le mélange plusieurs fois (ici 5) pour obtenir un écart-type.

Variable	Baisse d'AUC	Écart-type
age	0,0655	0,0049
recence_jours	0,0612	0,0073
satisfaction_moy	0,0409	0,0029
montant_12m	0,0403	0,0047
part_achats_promo	0,0200	0,0022
programme_fidelite	0,0088	0,0034
nb_commandes_12m	0,0072	0,0013

Sept variables dominent ; les 40 autres (dont les 20 villes) pèsent chacune moins de 0,003. C'est cohérent avec ce qui a été programmé : l'âge (par un effet en U), la récence, la satisfaction, le montant et la part d'achats en promotion jouent un rôle central dans le départ.

La méthode a deux avantages : elle marche avec **n'importe quel modèle**, et elle mesure l'effet sur la **performance** (ce qui compte), pas sur la structure interne. Elle a aussi deux limites sérieuses.

Les variables corrélées se partagent l'importance. Si deux variables portent la même information, mélanger l'une laisse l'autre compenser : aucune n'apparaît importante, alors que leur information l'est. Expérience : on ajoute au jeu une copie *bruitée* de *recence_jours* (corrélation de l'ordre de 0,99, écart-type du bruit de 15 jours). La récence valait seule 0,0612 d'AUC ; avec la copie, l'importance se répartit entre les deux variables (0,0189 pour l'originale et 0,0126 pour la copie, soit 0,0315 au total, la moitié de l'importance initiale), alors que l'AUC du modèle est pratiquement inchangée. Aucune des deux ne semble cruciale, et pourtant leur information l'est. **Ne concluez jamais « cette variable est inutile » d'une importance faible quand des variables voisines existent.**

L'importance n'est pas un effet. L'importance dit « le modèle perd 0,06 d'AUC sans la récence », pas « la récence augmente (ou diminue) le risque ». Pour la *direction* de l'effet, il faut les outils suivants.

5.3.3 Effets moyens et individuels : PDP et ICE

Le **graphique de dépendance partielle** (PDP, *partial dependence plot*) répond à la question « *que fait le modèle quand je fais varier cette variable ?* ». On fixe la variable x_j à une valeur v pour **tous** les clients, on fait prédire le modèle, et l'on prend la moyenne des probabilités ; on répète pour une grille de valeurs v :

$$\text{PDP}_j(v) = \frac{1}{n} \sum_{i=1}^n \hat{f}(x_{ij} := v, x_{i,-j}).$$

Si l'on ne moyenne pas, on obtient une courbe par client : ce sont les courbes **ICE** (*individual conditional expectation*). Le PDP est leur moyenne.

Le PDP de la récence est presque plat jusqu'à 100 jours (probabilité moyenne de 0,079 à 10 jours et de 0,087 à 100 jours), puis monte brutalement : 0,137 à 150 jours, 0,189 à 200 jours, 0,203 à 300

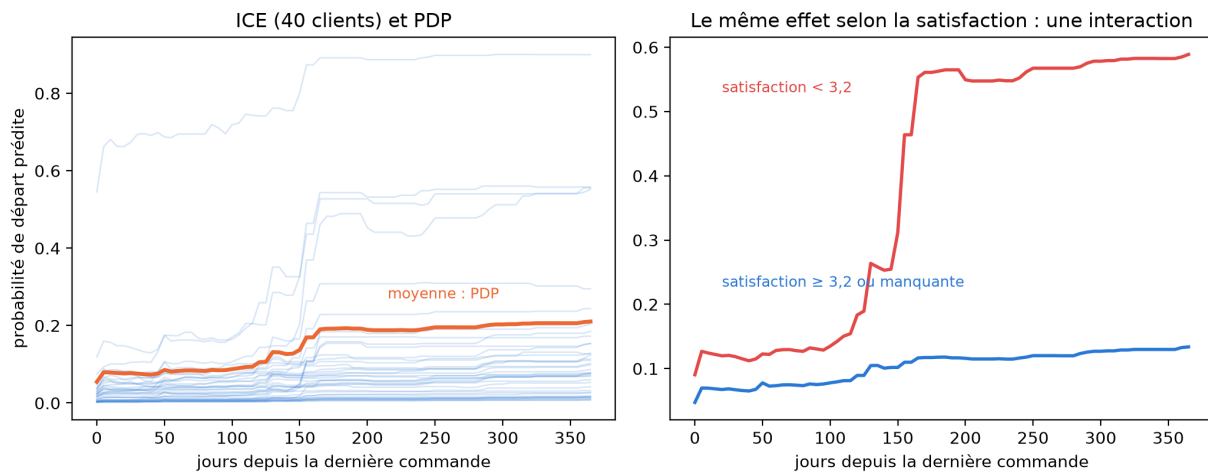


Figure 59: À gauche : courbes ICE (40 clients, en bleu) et leur moyenne, le PDP (en orange), pour la récence. À droite : le même PDP calculé séparément pour les clients peu satisfaits et les autres.

jours. Une lecture sans nuance conclurait que « le risque augmente avec la récence, surtout entre 100 et 200 jours ».

Les courbes ICE disent qu'il y a **quelque chose de plus**. Elles ne sont pas parallèles : le PDP moyenne des comportements très différents. Coupons les clients en deux groupes selon leur satisfaction. Pour les clients à satisfaction inférieure à 3,2 (17 % de l'échantillon), la probabilité moyenne passe de 0,135 à 100 jours à 0,549 à 200 jours ; pour les autres, de 0,078 à 0,116. C'est exactement ce qui avait été programmé : la récence ne devient dangereuse que combinée à une faible satisfaction. Un effet qui dépend d'une autre variable est une **interaction** ; le PDP seul l'aurait masquée, les ICE l'ont révélée.

Limite du PDP. Fixer la récence à 365 jours *pour tous* les clients crée des clients irréalistes (un client qui a passé dix commandes le mois dernier et dont la dernière commande date d'un an). Le PDP suppose les variables indépendantes ; quand elles ne le sont pas, il évalue le modèle là où il n'a pas de données.

5.3.4 LIME : un modèle simple autour d'un client

LIME (*Local Interpretable Model-agnostic Explanations*) explique une prédiction individuelle en la **remplaçant localement par un modèle simple**. Pour un client x_0 :

1. on fabrique des milliers de clients fictifs *autour* de x_0 (en perturbant ses variables) ;
2. on fait prédire le modèle complexe sur ces clients fictifs ;
3. on ajuste une **régression linéaire pondérée** (les clients proches de x_0 comptent plus) pour approcher ces prédictions ;
4. les coefficients de cette régression sont l'explication.

Autrement dit, LIME minimise $\sum_k w_k (\hat{f}(z_k) - g(z_k))^2$ sur des points z_k voisins de x_0 , avec g linéaire (et parcimonieuse).

Prenons un client du jeu de test dont la probabilité de départ est de 0,548 (il est finalement resté). Avec la graine 0, LIME répond que ce risque s'explique par : récence supérieure à 148 jours (+0,147), âge inférieur à 28 ans (+0,133), part d'achats en promotion supérieure à 0,34 (+0,051), montant annuel inférieur à 40 € (+0,048), absence de programme de fidélité (+0,035). C'est une explication lisible, en phrases.

Mais relançons LIME avec d'autres graines, sur **le même client et le même modèle**. Les poids changent (avec la graine 1, l'âge passe de +0,133 à +0,115) et les variables retenues parmi les cinq premières ne sont pas toujours les mêmes : le recouvrement moyen (indice de Jaccard) entre les ensembles de cinq variables de deux graines est de 0,80, et il descend à 0,67 pour certaines paires. **Une explication qui varie avec la graine aléatoire n'est pas une explication stable**. LIME est utile pour *se faire une idée* d'un cas, pas pour le *justifier* devant un tiers sans vérification.

5.3.5 Les valeurs de Shapley

On aimerait une méthode qui répartisse la prédiction entre les variables de façon **équitable et sans ambiguïté**. La théorie des jeux coopératifs en a une, inventée par Lloyd Shapley en 1953.

Le problème du partage. Trois joueurs coopèrent et gagnent ensemble une somme. Comment partager le gain équitablement, sachant que chaque joueur contribue différemment, et que certains ne servent qu'en présence d'autres ? Ici, les « joueurs » sont des *variables* (la récence R, la satisfaction S et le nombre de tickets de support T), et le « gain » est la *probabilité de départ* prédite pour un client. On note $v(S)$ le gain qu'obtient la coalition S de variables connues. Imaginons, pour un client donné, les valeurs suivantes :

Coalition	$\emptyset$	R	S	T	R, S	R, T	S, T	R, S, T
v	0,14	0,30	0,20	0,15	0,62	0,34	0,22	0,66

($v(\emptyset) = 0,14$ est la prévalence : le risque sans aucune information ; $v(R, S, T) = 0,66$ est la prédiction du modèle quand on connaît tout.) La récence et la satisfaction ont un effet conjoint fort : $v(R, S) = 0,62$, bien plus que $0,30 + 0,20 - 0,14 = 0,36$. Comment attribuer les $0,66 - 0,14 = 0,52$ de risque ?

L'idée de Shapley. Faisons arriver les variables **dans un ordre**, et comptons ce que chacune apporte en arrivant. Pour que l'ordre n'avantage personne, on prend la moyenne sur **tous les ordres possibles** (ici $3! = 6$). Voici les apports marginaux de chaque variable :

Ordre d'arrivée	Apport de R	Apport de S	Apport de T
R, S, T	$0,30 - 0,14 = 0,16$	$0,62 - 0,30 = 0,32$	$0,66 - 0,62 = 0,04$
R, T, S	0,16	$0,66 - 0,34 = 0,32$	$0,34 - 0,30 = 0,04$
S, R, T	$0,62 - 0,20 = 0,42$	$0,20 - 0,14 = 0,06$	0,04
S, T, R	$0,66 - 0,22 = 0,44$	0,06	$0,22 - 0,20 = 0,02$
T, R, S	$0,34 - 0,15 = 0,19$	$0,66 - 0,34 = 0,32$	$0,15 - 0,14 = 0,01$
T, S, R	$0,66 - 0,22 = 0,44$	$0,22 - 0,15 = 0,07$	0,01
Moyenne	$1,81/6 \approx 0,302$	$1,15/6 \approx 0,192$	$0,16/6 \approx 0,027$

La récence est responsable de 0,302 du risque, la satisfaction de 0,192 et les tickets de 0,027. Leur somme est $0,302 + 0,192 + 0,027 = 0,52 = v(R, S, T) - v(\emptyset)$: **le gain est exactement réparti, sans reste**.

Définition générale. Pour un jeu à p joueurs, la valeur de Shapley du joueur j est

$$\varphi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|! (p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)],$$

la moyenne de son apport marginal sur tous les ordres d'arrivée. C'est **la seule** attribution qui vérifie quatre propriétés raisonnables : (1) **efficacité** : $\sum_j \varphi_j = v(N) - v(\emptyset)$; (2) **symétrie**

: deux joueurs qui apportent la même chose à toutes les coalitions reçoivent la même valeur ; (3) **joueur nul** : un joueur qui n'apporte jamais rien reçoit zéro ; (4) **additivité** : si l'on additionne deux jeux, les valeurs s'additionnent.

Le calcul exact demande d'évaluer 2^p coalitions : impraticable pour 47 variables (plus de 10^{14} coalitions). C'est le rôle de SHAP.

5.3.6 SHAP : Shapley appliqué à un modèle

SHAP (*SHapley Additive exPlanations*) applique cette idée à un modèle : les « joueurs » sont les variables, et $v(S)$ est la **prédiction moyenne du modèle quand seules les variables de S sont connues** (les autres étant intégrées sur leur distribution). On obtient, pour **chaque client et chaque variable**, une valeur φ_{ij} , avec la propriété d'efficacité :

$$\hat{f}(x_i) = \varphi_0 + \sum_{j=1}^p \varphi_{ij},$$

où φ_0 est la **valeur de base** (la prédiction moyenne du modèle). Pour un classifieur à arbres, la prédiction est exprimée en *log-cote* ; une valeur positive augmente le risque, une valeur négative le diminue.

L'ingénieux **TreeSHAP** calcule ces valeurs **exactement** pour des arbres, en un temps polynomial, au lieu des 2^p coalitions. En pratique, deux lignes suffisent :

```
import shap

explicateur = shap.TreeExplainer(mg)                # mg : le modèle LightGBM entraîné plus haut
valeurs = explicateur.shap_values(Xte.iloc[:500])    # une valeur de Shapley par client et par variable
print(np.shape(valeurs), "| valeur de base (log-cote) :",
      ↪ round(float(np.ravel(explicateur.expected_value)[-1]), 3))
```

```
(500, 47) | valeur de base (log-cote) : -2.993
```

On obtient une matrice de 500 clients par 47 variables, et la valeur de base vaut $-2,993$ en log-cote, soit une probabilité de 0,048. (Elle est très inférieure à la prévalence de 14 % : la valeur de base est la *moyenne des log-cotes*, et non la log-cote de la moyenne. Quelques clients très risqués tirent la moyenne des probabilités vers le haut sans déplacer beaucoup celle des log-cotes.) Vérifions l'efficacité : pour chaque client, la valeur de base plus la somme de ses 47 valeurs SHAP doit redonner exactement la sortie brute du modèle. L'écart maximal sur les 500 clients est de 10^{-14} : c'est de l'arrondi informatique. Rien n'est « approximatif » : l'explication est une **décomposition exacte** de la prédiction.

Lecture globale. La moyenne des valeurs absolues de SHAP par variable donne une importance globale, cette fois dans l'unité du modèle (la log-cote). En tête : l'âge (0,683), le montant annuel (0,648), la récence (0,558), la part d'achats en promotion (0,278), la satisfaction (0,267), le nombre de commandes (0,218) et le programme de fidélité (0,204). Le classement est proche de celui de la permutation, mais pas identique :

Variable	Rang SHAP	Rang permutation
age	1	1
montant_12m	2	4
recence_jours	3	2
part_achats_promo	4	5

Variable	Rang SHAP	Rang permutation
satisfaction_moy	5	3
nb_commandes_12m	6	7
programme_fidelite	7	6
taux_ouverture_email	8	47
panier_moyen	10	45

L'écart le plus net concerne `taux_ouverture_email` (8^e en SHAP, 47^e en permutation) : cette variable a un effet **réel mais faible** sur chaque client, que SHAP restitue, mais qui ne se traduit que par une dégradation négligeable de l'AUC quand on la mélange. Ce sont deux questions différentes : SHAP répond à « *combien cette variable déplace-t-elle les prédictions ?* », la permutation à « *combien le modèle perd-il à l'ignorer ?* ».

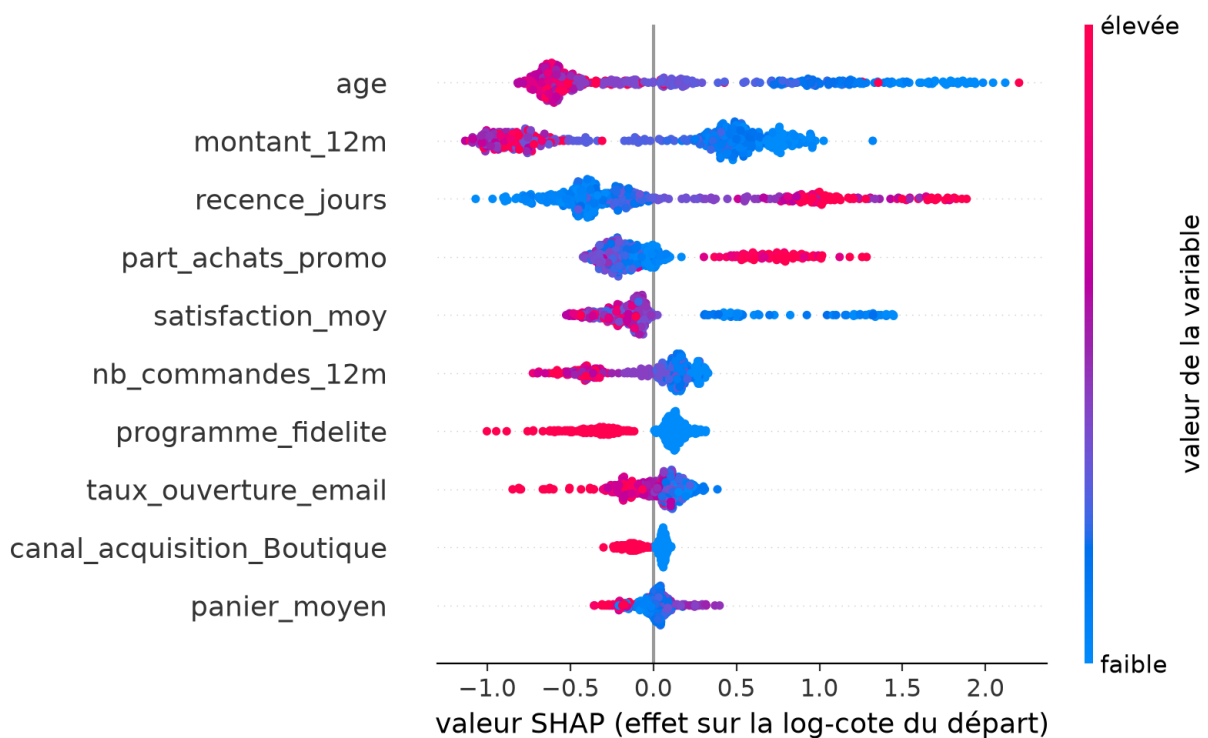


Figure 60: Résumé SHAP : un point par client (500 clients) et par variable ; position horizontale = effet sur la log-cote ; couleur = valeur de la variable (rouge : élevée).

Comment lire le résumé. Chaque ligne est une variable, chaque point un client ; la position horizontale est l'effet du client sur la log-cote du départ (à droite : le risque augmente) et la couleur est la valeur de la variable pour ce client (rouge : élevée). On y retrouve ce qui a été programmé. La **récence** : les points rouges (longue absence) sont à droite, jusqu'à +1,9 ; les points bleus (achat récent) à gauche. La **satisfaction** : les clients peu satisfaits (points bleus) poussent le risque de +0,3 à +1,5, les clients satisfaits le diminuent légèrement. Le **montant annuel** : un montant élevé protège (jusqu'à -1,1), un montant nul ou faible expose. Le **programme de fidélité** : les membres (rouge) sont du côté protecteur. Enfin l'**âge** montre l'effet **en U** : les clients les plus jeunes (points bleus) ont des contributions fortement positives, qui atteignent +2,2, la zone centrale est négative, et quelques clients plus âgés (points rouges, à droite) retrouvent des contributions positives. Une courbe moyenne, ou une seule importance, n'aurait pas montré cette forme.

Comment lire la dépendance. Pour la récence, la contribution passe d'environ -1 (achat très récent) à 0 vers 100 jours, puis grimpe nettement après 150 jours. Au-delà de 150 jours, les points

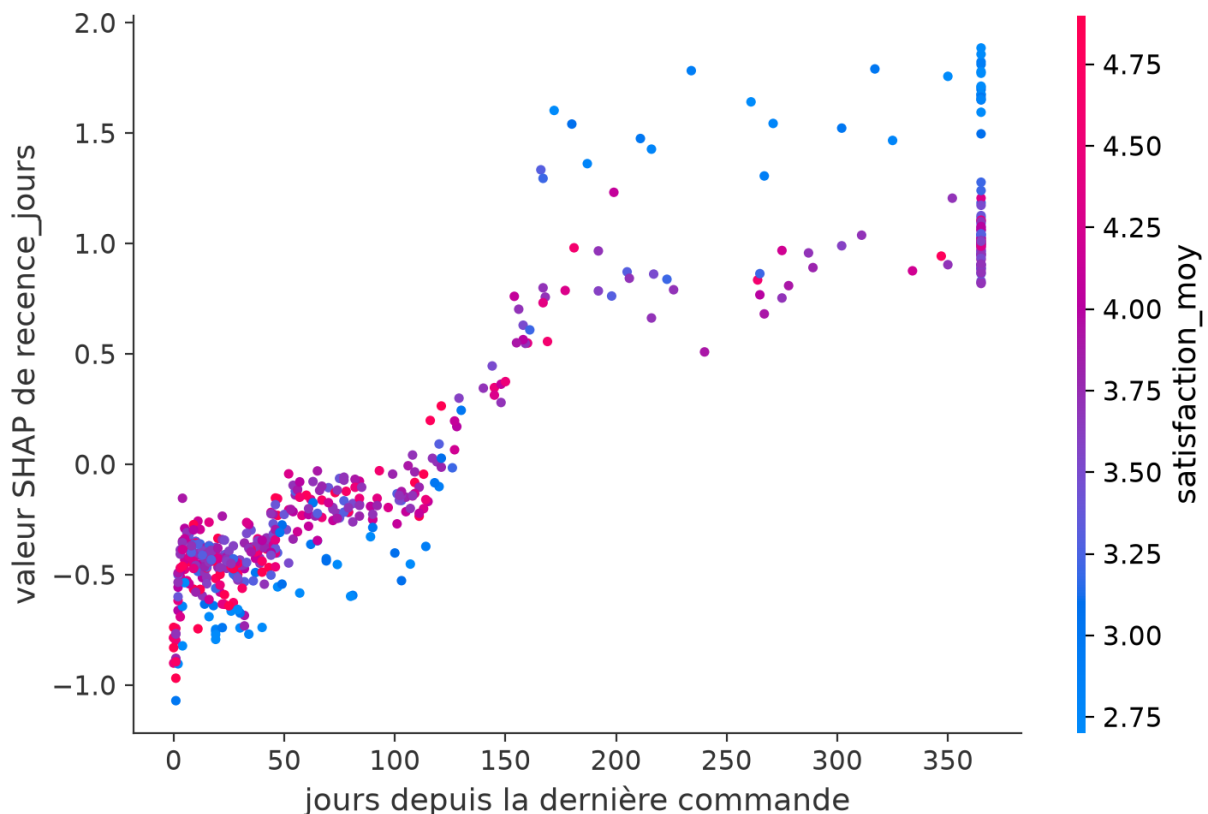


Figure 61: Dépendance de la valeur SHAP de la récence à la récence elle-même, colorée par la satisfaction.

se séparent verticalement selon leur couleur : les clients **peu satisfaits** (bleus) atteignent +1,3 à +1,9, les clients **satisfaits** (rouges ou violets) restent entre +0,8 et +1,2. C'est l'interaction repérée en 5.3.3, cette fois mesurée sur chaque client. L'empilement vertical à 365 jours correspond aux clients qui n'ont passé aucune commande dans l'année.

Lecture locale. Prenons, parmi les 500, le client le plus à risque. Sa sortie brute est de 2,398 en log-cote (probabilité 0,917), contre -2,993 pour la valeur de base (probabilité 0,048). La différence, 5,39, est répartie entre les variables : **récence** (365 jours, c'est-à-dire aucune commande depuis un an) +1,78, **satisfaction** (2,4) +1,21, **âge** (24 ans) +0,77, **montant annuel** (0 €) +0,76, **nombre de commandes** (0) +0,28, **ancienneté** (38 mois) +0,22, et d'autres plus petites. Chaque contribution est lisible par un non-spécialiste : « ce client est signalé parce qu'il n'a rien acheté depuis un an, qu'il est peu satisfait et qu'il est jeune ».

5.3.7 Les pièges de l'interprétation

SHAP n'est pas de la causalité. Les valeurs SHAP décrivent comment le *modèle* utilise les variables. Elles ne disent pas ce qui arriverait si l'on *intervenant* sur la variable. Que la satisfaction ait une forte contribution ne dit pas qu'augmenter la satisfaction fera baisser le départ ; cela relève de l'inférence causale (volume II, chapitre 7, facultatif).

Les variables corrélées brouillent la lecture. Comme en 5.3.2, quand deux variables portent la même information, SHAP la partage entre elles de façon dépendante du modèle. Lire « la variable A pèse deux fois plus que B » n'a de sens que si A et B sont à peu près indépendantes.

Une explication peut être fausse sans que le modèle le soit. Les explications dépendent d'hypothèses (la distribution de fond utilisée pour « supprimer » les variables, le nombre de

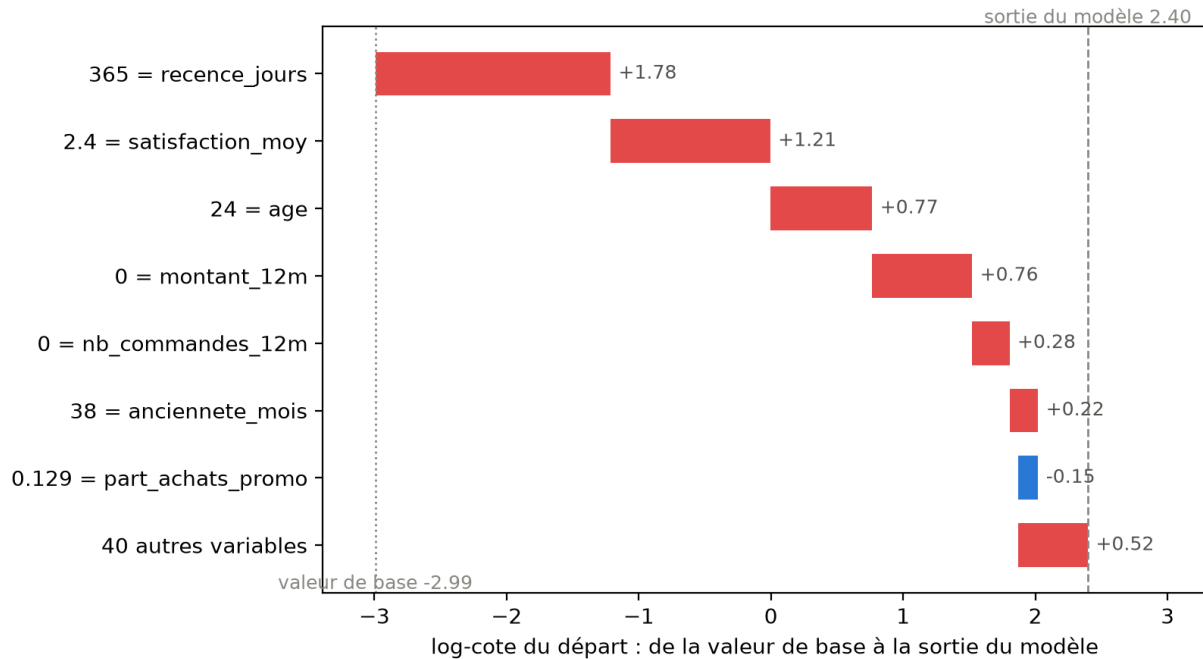


Figure 62: Décomposition de la prédiction du client le plus à risque parmi les 500 : de la valeur de base aux 8 plus grandes contributions.

perturbations de LIME...). Deux outils peuvent donner deux récits différents pour la même prédiction ; il faut le savoir.

Mais l'interprétabilité sert aussi à débusquer les erreurs. Rappelez-vous le piège du chapitre 1 : la colonne `commandes_apres_cible`, qui contient les commandes des trois mois *suivants*, donc l'avenir. Entraînons le même modèle en la laissant parmi les variables. L'AUC sur le jeu de test monte de 0,902 à **0,933**, ce qui est tentant. Mais SHAP trahit immédiatement le modèle : la première variable est `commandes_apres_cible` (importance moyenne 1,394), bien devant l'âge (0,531) et le montant annuel (0,452). Un modèle de prévision dont la variable la plus importante est une information du futur est un modèle qui triche. **Regardez toujours les variables que votre modèle juge les plus importantes : si l'une est suspecte, vous avez probablement une fuite d'information.**

À retenir.

- Distinguez l'explication **globale** et **locale**, et ce que l'on explique : le **modèle**, pas le monde.
- L'**importance par permutation** mesure la perte de performance sans une variable ; elle est trompée par les variables corrélées.
- **PDP** et **ICE** montrent *comment* une variable agit ; les **ICE** révèlent les interactions que le PDP moyenne.
- **LIME** ajuste un modèle linéaire local : lisible, mais **instable** (varie avec la graine).
- Les **valeurs de Shapley** répartissent la prédiction de façon unique (efficacité, symétrie, joueur nul, additivité) ; **TreeSHAP** les calcule exactement pour les arbres. Leur somme redonne la prédiction.
- Utilisez l'interprétabilité comme **outil de diagnostic** : elle débusque les fuites d'information.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.6 et 5.7, exercices 5.9 et 5.10.

5.4 ⊕ Pour aller plus loin : équité, biais et éthique

Section optionnelle. Elle suppose acquises les sections 5.1 (métriques) et 5.2 (calibration).

Un modèle peut être excellent « en moyenne » et traiter très différemment deux groupes de personnes. Quand ses décisions touchent des personnes (accorder un crédit, repérer des clients à relancer, trier des candidatures), cette différence n'est plus seulement un défaut technique : c'est une question de justice, et parfois de droit. Cette section ne prétend pas la résoudre. Elle apprend à la **mesurer**, à comprendre pourquoi **aucune mesure ne suffit seule**, et à discuter des remèdes avec honnêteté.

5.4.1 Un jeu de données réel, et un avertissement

Pour une fois, nous quittons la boutique : le sujet exige des données **réelles**, avec des attributs sensibles. Nous utilisons le jeu public « Default of Credit Card Clients » (Yeh et Lien, 2009 ; licence CC0) : **30 000 clients d'une banque de Taïwan en 2005**, dont 22,1 % ont fait défaut le mois suivant. Pour chaque client : le montant du crédit, le **sexe**, le niveau d'études, la situation matrimoniale, l'**âge**, six mois d'historique de remboursement (retards, factures, paiements) et la cible (le défaut). On entraîne un gradient boosting sur 70 % des clients, et l'on évalue sur les 9 000 autres.

Ce que cette étude est, et n'est pas. C'est une illustration méthodologique sur un échantillon ancien et particulier. Les résultats décrivent *ce modèle sur ces données*. Ils ne disent rien sur les pratiques d'un établissement réel, ni sur ce qu'il faudrait faire ailleurs. Dans plusieurs pays, utiliser le sexe ou l'âge dans une décision de crédit est restreint ou interdit : le cadre juridique varie selon les lieux et les domaines, et ceci n'est pas un conseil juridique.

Le modèle atteint une AUC de **0,777** sur le jeu de test. Pour passer de scores à des décisions, on fixe un seuil qui **signale 25 % des dossiers** (les plus risqués), soit un score supérieur à 0,271 : par exemple, des dossiers à examiner de plus près.

5.4.2 Mesurer : les critères d'équité

Découpons le jeu de test par groupes. Voici ce que donne le modèle selon le **sexe** (1 = hommes, 2 = femmes dans les données d'origine) :

Groupe	Effectif	Taux de défaut	Taux d'alerte	Rappel	Fausse alertes	Précision	Probabilité moyenne prédite	AUC
Femmes	5 353	0,206	0,234	0,571	0,147	0,502	0,213	0,780
Hommes	3 647	0,244	0,273	0,574	0,176	0,513	0,230	0,771

```

alerte = s_avec >= t_alerte                                # les 25 % de dossiers jugés les plus risqués
audit = pd.DataFrame({"sexe": g_sexe, "défaut": y5, "alerte": alerte})
print(audit.groupby("sexe").mean().round(3))                # taux de défaut et taux d'alerte, par
↳ groupe

```

	défaut	alerte
sexe		
femmes	0.206	0.234
hommes	0.244	0.273

Et selon l'âge (trois classes) :

Groupe	Effectif	Taux de défaut	Taux d'alerte	Rappel	Fausse alertes	Précision	AUC
Moins de 30 ans	2 885	0,216	0,258	0,608	0,162	0,509	0,785
30 à 44 ans	4 497	0,212	0,234	0,533	0,154	0,483	0,770
45 ans et plus	1 618	0,255	0,279	0,610	0,165	0,559	0,781

Plusieurs définitions de l'équité circulent. Chacune compare une grandeur entre les groupes :

- **Parité démographique** : le **taux d'alerte** est le même dans tous les groupes. Elle ne regarde pas la réalité : elle compare seulement les décisions.
- **Égalité des chances** : le **rappel** (taux de vrais positifs) est le même. Parmi les clients qui feront défaut, chaque groupe est détecté dans la même proportion.
- **Cotes égalisées** (*equalized odds*) : le **rappel** et le **taux de fausses alertes** sont les mêmes. Parmi ceux qui ne feront pas défaut, chaque groupe est signalé à tort dans la même proportion.
- **Parité prédictive** : la **précision** est la même. Une alerte a la même probabilité d'être justifiée dans tous les groupes.
- **Calibration par groupe** : $P(Y = 1 \mid \hat{p} = p, \text{groupe}) = p$ pour chaque groupe.

Mesurons les écarts maximaux entre groupes. Selon le **sexe**, le **rappel** est quasiment identique (0,571 contre 0,574 : écart de **0,003**), l'égalité des chances est donc presque réalisée ; mais le **taux d'alerte** diffère de **0,039** (23,4 % contre 27,3 %), et le **taux de fausses alertes** de **0,030** (14,7 % contre 17,6 %). Selon l'**âge**, le tableau est différent : l'écart de **rappel** est de **0,077** (les 30 à 44 ans sont moins bien détectés : 53,3 % contre 61 % pour les autres), et l'écart de **précision** de **0,076**.

Le même modèle paraît équitable ou non selon le critère. Pour le sexe, il passe presque l'égalité des chances et échoue à la parité démographique ; pour l'âge, il échoue à l'égalité des chances. Aucun critère ne dit à lui seul si « le modèle est équitable ». Choisir le critère est un choix **de valeurs**, pas de statistique.

Une partie de l'écart de taux d'alerte entre hommes et femmes s'explique simplement : **les taux de défaut diffèrent dans les données** (24,4 % contre 20,6 %). Un modèle bien calibré signale davantage le groupe où le défaut est plus fréquent ; l'écart de taux d'alerte (0,039) est du même ordre que l'écart de taux de défaut (0,038). La calibration par groupe est d'ailleurs bonne : l'ECE vaut 0,0205 pour les hommes et 0,0144 pour les femmes ; elle est un peu moins bonne pour les 45 ans et plus (0,039, sur seulement 1 618 clients), alors qu'elle est de 0,016 et 0,020 pour les deux autres classes d'âge.

5.4.3 Supprimer la variable sensible ne suffit pas

La réaction la plus courante est : « *retirons le sexe des variables, le modèle ne pourra plus discriminer* ». C'est l'**équité par ignorance** (*fairness through unawareness*). Essayons.

L'AUC passe de 0,777 à 0,774 : le sexe apporte presque rien au pouvoir prédictif. Les écarts entre groupes diminuent un peu, sans disparaître : l'écart de taux d'alerte selon le sexe passe de 0,039 à **0,030**, celui de fausses alertes de 0,030 à 0,019, mais l'écart de **précision** *augmente*, de 0,011 à 0,025. Et surtout, l'information sexe **n'a pas disparu des autres variables** : un modèle entraîné à retrouver le sexe à partir des variables restantes (le niveau d'études, la situation matrimoniale, le

montant du crédit, les habitudes de paiement...) atteint une AUC de **0,644**, nettement supérieure à 0,5. Les variables qui restent sont des **proxys** (substituts) du sexe.

Retirer une variable sensible n'efface ni le biais ni l'information. Cela empêche seulement de la mesurer : sans la colonne « sexe », on ne peut plus auditer les écarts entre groupes. Il faut en général **garder** l'attribut pour l'audit, même si le modèle ne l'utilise pas pour décider.

5.4.4 Pourquoi on ne peut pas tout avoir

Peut-on exiger **à la fois** l'égalité des chances (même rappel), la parité prédictive (même précision) et l'égalité des fausses alertes ? En général, **non**, dès que les taux de défaut diffèrent entre les groupes. C'est un résultat mathématique, pas un défaut de modèle.

L'identité qui interdit de tout égaliser. Pour un groupe de taux de défaut p , de précision PPV, de rappel $1 - \text{FNR}$ et de taux de fausses alertes FPR, on a, puisque ces quatre grandeurs sont des fonctions des mêmes quatre effectifs (VP, FP, FN, VN) :

$$\text{FPR} = \frac{p}{1-p} \cdot \frac{1 - \text{PPV}}{\text{PPV}} \cdot (1 - \text{FNR}).$$

En effet, $\text{PPV} = \frac{VP}{VP + FP}$ donne $FP = VP \cdot \frac{1 - \text{PPV}}{\text{PPV}}$; avec $VP = (1 - \text{FNR}) \cdot pn$ et $\text{FPR} = FP / ((1 - p)n)$, on retrouve la formule. **Si deux groupes ont la même précision et le même rappel mais des taux de défaut p différents, ils ne peuvent pas avoir le même taux de fausses alertes.**

Vérifions sur nos données : pour les femmes ($p = 0,2057$, précision 0,502, taux de faux négatifs 0,4287), la formule donne un taux de fausses alertes de 0,1468, exactement la valeur observée ; pour les hommes ($p = 0,244$, précision 0,5125, taux de faux négatifs 0,4258), 0,1763, là aussi exactement la valeur observée. Imaginons maintenant les deux groupes avec la **même précision** (0,507) et le **même rappel** (0,573), moyennes des valeurs observées : la formule impose des fausses alertes de 0,144 pour les femmes et de 0,180 pour les hommes. L'écart de 0,036 est exigé par l'écart entre les taux de défaut.

Ce que cela signifie. Il n'existe pas de modèle imparfait qui soit « équitable » pour tous les critères quand les taux de base diffèrent (c'est le résultat connu sous le nom de *théorème d'impossibilité* d'Alexandra Chouldechova et de Jon Kleinberg et ses coauteurs, 2016-2017). Il faut **choisir** quel type d'erreur doit être égalisé, et assumer ce choix.

5.4.5 Corriger : des seuils par groupe

L'une des corrections les plus simples est un **post-traitement** : utiliser un seuil **différent par groupe**. Comparons trois politiques pour le sexe, puis pour l'âge, dans un cadre de coûts où manquer un défaut coûte 5 unités et signaler à tort 1 unité :

Politique	Variable	Seuils par groupe	Alertes	Coût pour 1 000 dossiers	Écart d'alerte	Écart de rappel	Écart de précision	Écart de fausses alertes
Seuil unique	sexe	0,271 / 0,271	25,0 %	596,1	0,039	0,003	0,011	0,030

Politique	Variable	Seuils par groupe	Alertes	Coût pour 1 000 dossiers	Écart d'alerte	Écart de rappel	Écart de précision	Écart de fausses alertes
Égalité des chances	sexe	0,267 (F) / 0,272 (H)	25,1 %	597,0	0,036	0,001	0,016	0,025
Parité démographique	sexe	0,256 (F) / 0,291 (H)	25,0 %	594,9	0,000	0,040	0,052	0,009
Seuil unique	âge	0,271 pour tous	25,0 %	596,1	0,044	0,077	0,076	0,011
Égalité des chances	âge	0,235 / 0,294 / 0,307	25,5 %	601,1	0,036	0,002	0,143	0,054
Parité démographique	âge	0,255 / 0,277 / 0,302	25,0 %	598,3	0,000	0,051	0,121	0,031

(Pour l'âge, les seuils sont dans l'ordre 30-44 ans, moins de 30 ans, 45 ans et plus.)

On lit trois choses. **Chaque politique égalise ce qu'elle vise** : la parité démographique annule l'écart de taux d'alerte, l'égalité des chances ramène l'écart de rappel de 0,077 à 0,002 pour l'âge. **Chaque politique dégrade autre chose** : en égalisant le rappel entre classes d'âge, l'écart de *précision* grimpe de 0,076 à 0,143 et l'écart de fausses alertes de 0,011 à 0,054 : c'est l'identité de 5.4.4 en action. Enfin, **le coût moyen varie peu** (entre 595 et 601 unités pour 1 000 dossiers) : ici, corriger les écarts ne coûte presque rien en performance globale. Ce n'est pas une règle générale.

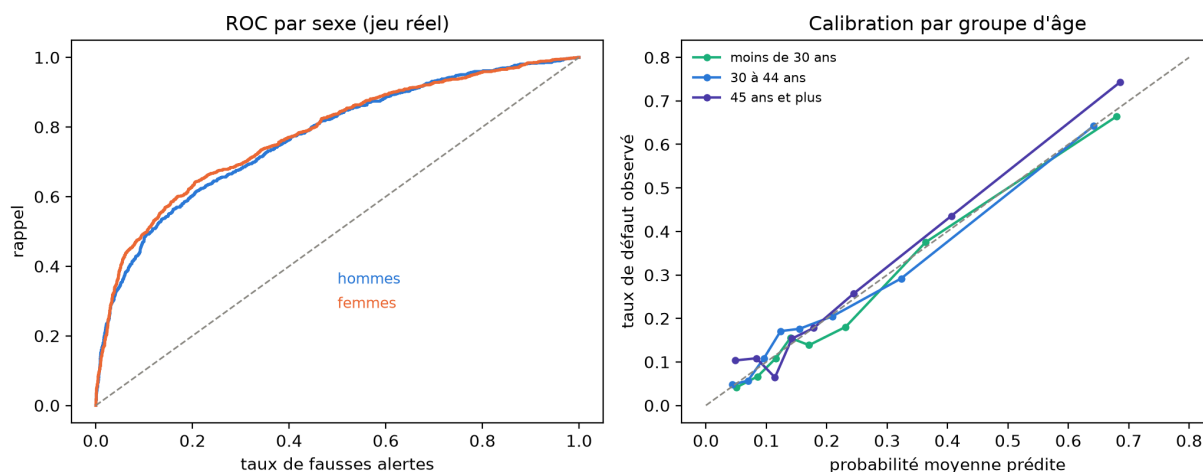


Figure 63: À gauche : courbes ROC par sexe sur le jeu réel ; à droite : calibration par classe d'âge. Les écarts sont faibles mais pas nuls.

5.4.6 L'éthique commence où le calcul s'arrête

Les chiffres ci-dessus ne répondent pas aux questions qui comptent le plus.

- **La cible est-elle neutre ?** Ici, la cible est « a fait défaut ». Mais le défaut dépend des décisions de crédit passées (qui a obtenu un crédit, à quel montant), qui ont elles-mêmes pu être biaisées. Un modèle entraîné sur l'historique reproduit parfois ce qu'il a hérité, avec une apparence d'objectivité.
- **Qui est dans les données ?** Les clients refusés par le passé n'y figurent pas : on ne sait pas s'ils auraient remboursé. Le modèle est évalué sur les seuls dossiers qu'il a vus.
- **Quel critère, et qui le choisit ?** Les critères de 5.4.2 correspondent à des idées différentes de la justice, incompatibles dès que les taux de base diffèrent. Les arbitrer n'est pas une décision que le data scientist doit prendre seul.
- **Que fait-on des personnes concernées ?** Explication de la décision, droit de contestation, intervention humaine, suivi dans le temps : ce sont des éléments de la **gouvernance** du modèle, pas de ses statistiques.
- **Les boucles de rétroaction.** Un modèle qui refuse des crédits modifie les données de demain. Surveillez les écarts entre groupes **après** le déploiement, pas seulement avant.

À retenir.

- Auditez le modèle **par groupe** : taux d'alerte, rappel, fausses alertes, précision, calibration. Gardez l'attribut sensible pour l'audit.
- Les critères (parité démographique, égalité des chances, cotes égalisées, parité prédictive) mesurent des choses **différentes** ; un même modèle peut en satisfaire un et pas l'autre.
- Quand les taux de base diffèrent, on **ne peut pas** tout égaliser : $FPR = \frac{p}{1-p} \cdot \frac{1-PPV}{PPV} \cdot (1 - FNR)$.
- **Retirer** la variable sensible ne suffit pas : des proxys subsistent.
- Des **seuils par groupe** égalisent une grandeur au prix d'une autre. Le choix du critère est un choix de valeurs, à discuter avec toutes les parties concernées.

Pour s'entraîner. Cahier, chapitre 5 : applications 5.8 et 5.9, exercices 5.11 et 5.12.

5.5 ⊕ Pour aller plus loin : quantifier l'incertitude, la prédiction conforme

Section optionnelle. Elle prolonge 5.2 (calibration) et 5.1.10 (prédire un quantile).

Un modèle de classification annonce « 62 % de risque de départ ». Un modèle de régression annonce « 83 € de dépense ». Dans les deux cas, **le modèle ne dit pas à quel point il peut se tromper**. La calibration (5.2) répare les probabilités, mais seulement approximativement, et sous l'hypothèse que la recalibration est bonne. La **prédiction conforme** (*conformal prediction*) propose autre chose : transformer *n'importe quel* modèle déjà ajusté en un modèle qui annonce non pas une valeur, mais un **ensemble** (en classification) ou un **intervalle** (en régression), avec une **garantie** de couverture valable à distance finie, sans hypothèse sur la loi des données.

5.5.1 L'objectif : une garantie de couverture

On se donne un niveau d'erreur α (par exemple 0,10). On veut, pour un nouveau client, un ensemble $C(x)$ de réponses plausibles tel que

$$P(Y \in C(X)) \geq 1 - \alpha.$$

En classification, $C(x)$ est un sous-ensemble des classes ($\{\text{fidèle}\}$, $\{\text{partant}\}$, ou les deux). En régression, c'est un intervalle $[\ell(x), u(x)]$. Cette probabilité porte sur le tirage du client *et* du jeu de calibration : on parle de **couverture marginale**.

5.5.2 La recette : la prédiction conforme « séparée »

La version la plus simple, dite **split conformal**, utilise trois jeux (ceux du fil rouge) : l'entraînement (pour ajuster le modèle), la **calibration** (pour mesurer ses erreurs), et le test (pour vérifier).

1. On ajuste le modèle sur le jeu d'entraînement.
2. On définit un **score de non-conformité** $s(x, y)$, qui est grand quand le couple (x, y) est « surprenant » pour le modèle. En classification, on prend $s(x, y) = 1 - \hat{p}_y(x)$: un moins la probabilité que le modèle attribuait à la **vraie** classe.
3. On calcule ce score pour chacun des n clients du jeu de calibration, et l'on prend son **quantile** d'ordre $\lceil (n+1)(1-\alpha) \rceil / n$: le k -ième plus petit score, avec $k = \lceil (n+1)(1-\alpha) \rceil$. Notons-le $\hat{q}$.
4. Pour un nouveau client, l'ensemble prédit est $C(x) = \{y : s(x, y) \leq \hat{q}\}$: toutes les classes dont le score ne dépasse pas $\hat{q}$.

Un exemple à la main. Neuf clients de calibration ($n = 9$), $\alpha = 0,20$. Leurs scores, triés : 0,05 ; 0,10 ; 0,14 ; 0,22 ; 0,31 ; 0,38 ; 0,47 ; 0,55 ; 0,71. On a $k = \lceil 10 \times 0,8 \rceil = 8$: $\hat{q}$ est le 8^e score, **0,55**. Un client dont le modèle annonce $\hat{p}_{\text{partant}} = 0,62$ donne le score 0,38 pour la classe « partant » et 0,62 pour « fidèle ». L'ensemble est $\{y : s \leq 0,55\} = \{\text{partant}\}$. Un autre client avec $\hat{p}_{\text{partant}} = 0,48$ donne 0,52 et 0,48 : les deux sont $\leq 0,55$, l'ensemble est $\{\text{fidèle}, \text{partant}\}$: le modèle **avoue son hésitation**.

Appliquons cela aux 2 400 clients de calibration et au gradient boosting du chapitre, avec $\alpha = 0,10$:

```
scores_cal = 1 - pcal[np.arange(len(y_cal)), y_cal]      # non-conformité : 1 - probabilité de la VRAIE
↳ classe, sur le jeu de calibration
k = int(np.ceil((len(scores_cal) + 1) * (1 - alpha)))    # rang du quantile : ⌈(n + 1)(1 - α)⌉
q = np.sort(scores_cal)[k - 1]                         # le k-ième plus petit score
ensembles = (1 - ptest) <= q                          # pour chaque client de test : les classes dont
↳ le score est <= q
couvert = ensembles[np.arange(len(y5c)), y5c]          # la vraie classe est-elle dans l'ensemble ?
print("k =", k, "sur", len(scores_cal), "| q =", round(q, 4), "| couverture :", round(couvert.mean(),
↳ 4))
```

```
k = 2161 sur 2400 | q = 0.5221 | couverture : 0.9025
```

C'est le 2 161^e plus petit des 2 400 scores ($k = \lceil 2401 \times 0,9 \rceil = 2 161$) : $\hat{q} = 0,5221$. Un client reçoit la classe k dans son ensemble si la probabilité de cette classe est au moins $1 - 0,5221 = 0,4779$. En classification binaire, cela revient à dire : **si la probabilité de départ est comprise entre 0,478 et 0,522, on répond « les deux » ; sinon, on répond une seule classe**. Sur le jeu de test, 99,4 % des clients reçoivent une réponse unique, et 0,6 % reçoivent les deux ; la **couverture globale vaut 0,9025**, conforme à l'objectif de 90 %.

5.5.3 Pourquoi cela marche

La garantie est étonnamment simple à démontrer. Elle ne suppose **ni** que le modèle est bon, **ni** que les probabilités sont calibrées, **ni** une loi particulière : seulement que les données de calibration et le nouveau client sont **échangeables** (par exemple, tirés indépendamment de la même population).

Théorème (couverture du « split conformal »). Soient $(X_i, Y_i), i = 1, \dots, n+1$, échangeables, et $s_i = s(X_i, Y_i)$ les scores calculés avec un modèle fixé indépendamment d'eux. Soit $\hat{q}$ le k -ième plus petit des n scores de calibration, $k = \lceil (n+1)(1-\alpha) \rceil$ (par convention $\hat{q} = +\infty$ si $k > n$). Alors

$$1-\alpha \leq P(s_{n+1} \leq \hat{q}) \leq 1-\alpha + \frac{1}{n+1} \quad (\text{borne supérieure : si les scores sont presque sûrement distincts}).$$

Démonstration. L'événement $\{s_{n+1} \leq \hat{q}\}$ signifie que s_{n+1} est au plus égal au k -ième plus petit des n scores de calibration, c'est-à-dire que le **rang** de s_{n+1} parmi les $n+1$ scores est au plus k . Par échangeabilité, les $n+1$ scores jouent des rôles symétriques : si les scores sont tous distincts, le rang de s_{n+1} est **uniforme** sur $\{1, \dots, n+1\}$. La probabilité vaut donc $\frac{k}{n+1} = \frac{\lceil (n+1)(1-\alpha) \rceil}{n+1} \in \left[1-\alpha, 1-\alpha + \frac{1}{n+1}\right)$. S'il y a des égalités, le rang n'est plus uniforme mais $P(\text{rang} \leq k) \geq \frac{k}{n+1}$, et la borne inférieure subsiste. ■

Remarquez ce que la démonstration n'utilise pas : le modèle, la calibration, la loi des données. Plus le modèle est bon, plus les ensembles sont **petits** ; mais la couverture, elle, est garantie quel que soit le modèle.

La garantie, vue par simulation. La couverture garantie est une moyenne sur le tirage du jeu de calibration : pour *un* jeu de calibration donné, la couverture réelle fluctue. On peut même dire comment : elle suit une **loi Bêta**($k, n+1-k$). Pour le voir, on mélange 1 000 fois les 4 800 clients des jeux de calibration et de test, on prend à chaque fois 300 clients pour calibrer et l'on mesure la couverture sur les 4 500 autres.

Ici $n = 300$ et $k = \lceil 301 \times 0,9 \rceil = 271$: la théorie donne une loi Bêta(271 ; 30), de moyenne 0,9003 et d'écart-type 0,0172. La simulation donne une moyenne de **0,9003** et un écart-type de **0,0177**, avec des couvertures allant de 0,841 à 0,945. La moyenne coïncide avec la théorie (0,9003, dans l'intervalle garanti [0,9 ; 0,9033]) et l'écart-type est proche de 0,0172. La théorie décrit fidèlement l'expérience.

Combien de clients de calibration ? Avec $n = 300$, la couverture réelle d'un jeu de calibration donné peut tomber à 85 % ou monter à 94 % ($\pm$ deux écarts-types : $\pm 3,5$ points). Avec $n = 2 400$, l'écart-type est de l'ordre de 0,006. La garantie est valide pour tout n , mais plus n est grand, plus la couverture *réelle* se rapproche de la couverture *annoncée*.

5.5.4 Garantie marginale, pas conditionnelle

Voici la nuance qui se perd le plus souvent. Calculons la couverture **séparément** pour chaque classe réelle :

	Couverture	Taille moyenne de l'ensemble
Globale	0,9025	1,006
Parmi les clients qui partent	0,4955	

	Couverture	Taille moyenne de l'ensemble
Parmi les clients qui restent	0,969	

La couverture globale est de 90 %, mais elle est de **49,6 % seulement pour les clients qui partent**, ceux qui nous intéressent. Comment est-ce possible ? La garantie est *marginale* : elle moyenne sur tous les clients. Or 86 % des clients restent, et le modèle les couvre à 97 % ; cela suffit à atteindre 90 % en moyenne, même en laissant à découvert un partant sur deux. C'est l'analogie, en prédiction d'ensembles, du piège de l'exactitude de 5.1.2.

Le remède : la prédiction conforme de Mondrian (conditionnelle à la classe). On calcule un quantile **par classe** (en n'utilisant que les clients de calibration de cette classe). La garantie devient valable *dans chaque classe* :

	LAC (global)	Mondrian
Couverture parmi les partants	0,4955	0,9021
Couverture parmi les fidèles	0,969	0,9157
Taille moyenne des ensembles	1,006	1,213

Le prix : des ensembles plus gros (en moyenne 1,21 classe au lieu de 1,01), c'est-à-dire plus d'hésitations affichées. C'est normal : on ne peut pas être à la fois sûr de couvrir les partants à 90 % et de rester précis, avec un modèle dont l'AUC est de 0,90.

Il existe d'autres scores de non-conformité. Le score **APS** (*adaptive prediction sets*) cumule les probabilités des classes par ordre décroissant ; avec une petite part d'aléa, il donne ici une couverture de 0,900 et une taille moyenne de 1,107, avec 4,5 % d'ensembles vides. On note que le choix du score est un compromis entre taille des ensembles et homogénéité de la couverture.

5.5.5 En régression : des intervalles avec garantie

Le même principe s'applique à une cible numérique, ici la dépense à six mois. Le score est l'**erreur absolue** $s(x, y) = |y - \hat{f}(x)|$, et l'intervalle est $[\hat{f}(x) - \hat{q}, \hat{f}(x) + \hat{q}]$ (borné par zéro, une dépense ne pouvant être négative).

Avec $\alpha = 0,10$, la demi-largeur est de $\hat{q} = 142,4$ €. La couverture sur le jeu de test vaut **0,910**, conforme à la garantie. Mais le défaut est immédiat : **la largeur est la même pour tous les clients**, qu'ils soient de petits ou de gros dépensiers. Séparons les clients en trois tiers selon la dépense prévue :

Tiers de la dépense prévue	Couverture (intervalle de largeur constante)	Couverture (CQR)	Largeur moyenne CQR
Prévision basse	0,994	0,922	74 €
Prévision moyenne	0,975	0,916	138 €
Prévision haute	0,760	0,892	344 €

Les intervalles de largeur constante sont **trop larges** pour les petits clients (couverture de 99 %, soit un gaspillage de précision) et **trop étroits** pour les gros (76 %, bien en dessous de l'objectif). On préfère des intervalles qui s'adaptent. La **régression quantile conformalisée** (CQR) en offre un moyen : on entraîne d'abord deux modèles de régression quantile (aux niveaux 5 % et 95 %, avec la perte pinball de 5.1.10), qui fournissent un intervalle $[\hat{\ell}(x), \hat{u}(x)]$ de largeur variable. Mais,

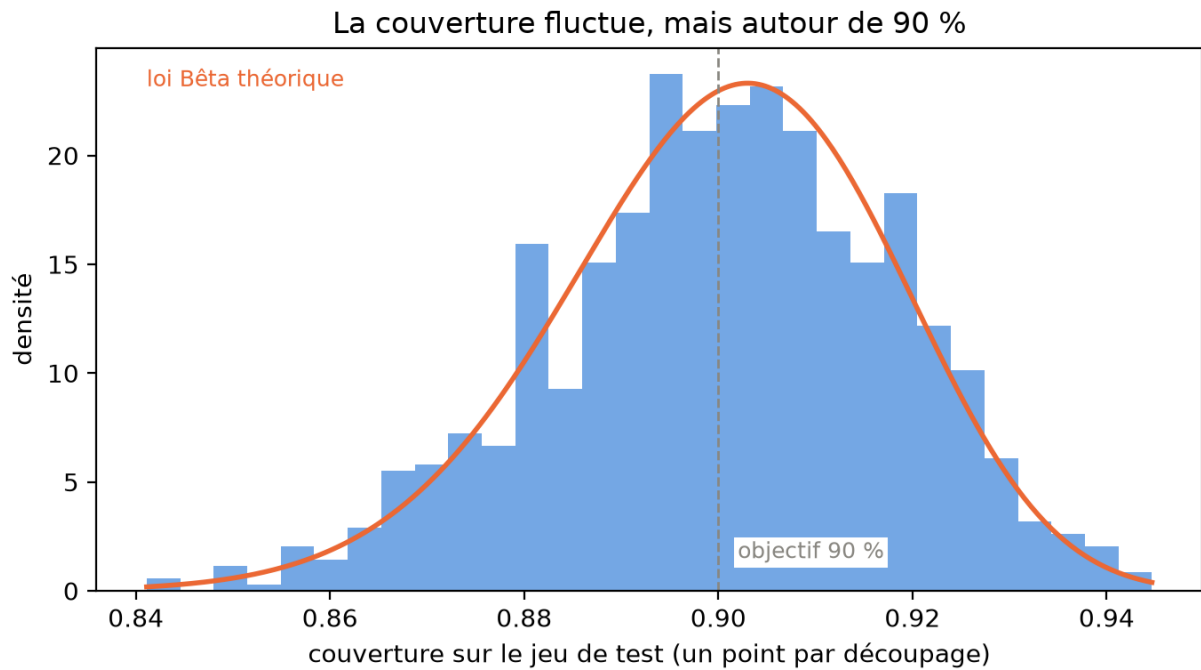


Figure 64: Couverture sur le jeu de test pour 1 000 découpages aléatoires (300 clients de calibration à chaque fois), et densité de la loi Bêta théorique. La couverture fluctue, mais autour de 90 %.

on l'a vu en 5.1.10, un quantile estimé n'a aucune garantie. On corrige donc par conformalisation : le score est $s(x, y) = \max(\hat{\ell}(x) - y, y - \hat{u}(x))$ (négatif quand y est dans l'intervalle), et l'intervalle final est $[\hat{\ell}(x) - \hat{q}, \hat{u}(x) + \hat{q}]$.

Ici, la correction est **exactement nulle** ($\hat{q} = 0$) : les deux modèles quantiles couvraient déjà 90 % des clients de calibration. (Avec 36 % de clients à dépense nulle, le modèle du quantile à 5 % prédit exactement 0 pour beaucoup d'entre eux, et leur score de non-conformité est exactement 0.) La couverture est de **0,910**, la largeur moyenne de **185 €** (contre 210 € pour l'intervalle constant : des intervalles *plus courts* à couverture égale), et les couvertures par tiers sont beaucoup plus homogènes (0,92 ; 0,92 ; 0,89). La largeur s'adapte : 74 € pour les clients peu dépensiers, 344 € pour les gros.

5.5.6 Limites et bonnes pratiques

- **La couverture reste marginale.** Même avec CQR, la couverture parmi les clients dont la dépense est strictement positive n'est que de 0,860 (les 36 % de clients qui ne dépensent rien sont couverts à 100 % et relèvent la moyenne). Une garantie *conditionnelle* à un sous-groupe demande de calibrer sur ce sous-groupe (comme Mondrian) ou des méthodes plus avancées.
- **L'échangeabilité est une vraie hypothèse.** Si les clients de demain ne ressemblent pas à ceux de la calibration (changement de saison, de population), la garantie tombe. Surveillez la couverture empirique en production, et recalibrez.
- **La garantie ne remplace pas un bon modèle.** Un mauvais modèle donne des ensembles énormes ou des intervalles très larges : ils sont valides, mais inutiles. La largeur moyenne est donc, en soi, une mesure de qualité.
- **Il faut des données de calibration** (quelques centaines au minimum) que l'on ne peut pas réutiliser pour l'entraînement. La variante *cross-conformal* ou *jackknife+* évite de sacrifier des données, au prix d'un calcul plus lourd.

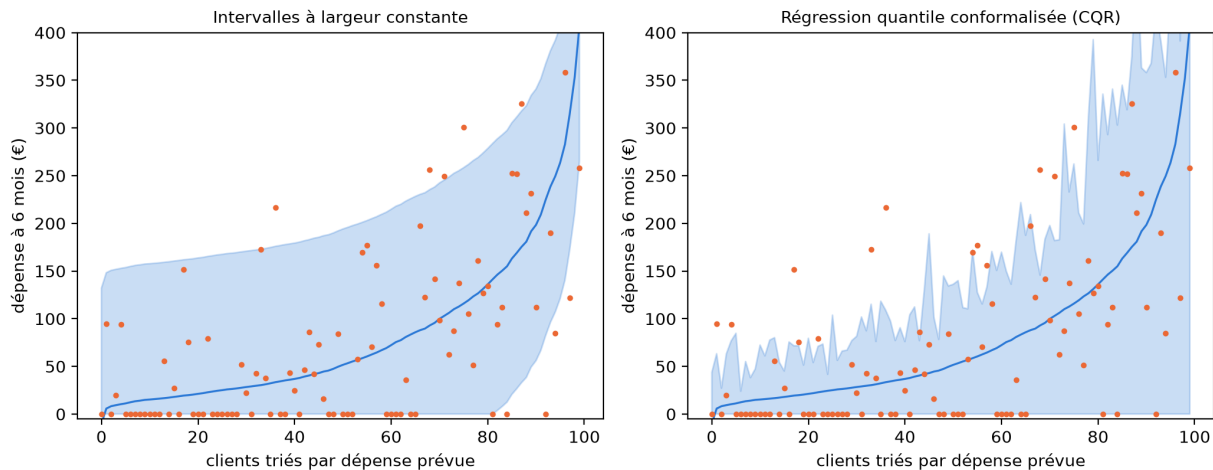


Figure 65: Intervalles à 90 % pour un client sur 24 trié par dépense prévue : largeur constante (à gauche) et CQR (à droite). Les points orange sont les dépenses réelles.

À retenir.

- La prédiction conforme transforme **n'importe quel modèle** en un modèle qui annonce un **ensemble** ou un **intervalle** avec une couverture garantie $\geq 1 - \alpha$.
- Recette : un score de non-conformité, son quantile d'ordre $\lceil (n + 1)(1 - \alpha) \rceil / n$ sur un jeu de calibration séparé, puis l'ensemble $\{y : s(x, y) \leq \hat{q}\}$.
- La preuve ne demande que l'**échangeabilité** : le rang d'un score parmi $n + 1$ est uniforme.
- La garantie est **marginale** : elle peut cacher une couverture très faible pour une classe rare (49,6 % pour les partants !) ; la version de **Mondrian** la rend valable par classe.
- En régression, des intervalles de largeur constante sont inadaptés ; **CQR** les rend adaptatifs.

Pour s'entraîner. Cahier, chapitre 5 : application 5.10, exercices 5.13 et 5.14.

Bilan du chapitre 5

Vous savez maintenant :

- **choisir la bonne mesure** : exactitude comparée au modèle « classe majoritaire », précision, rappel, F_β et MCC pour une décision à seuil ; **AUC** (probabilité qu'un positif soit mieux classé qu'un négatif) et **précision moyenne** pour juger un score, la seconde étant la seule à voir la prévalence ; **log-loss** et **Brier** pour juger des probabilités ; MAE, RMSE, R^2 et perte pinball en régression, en évitant le MAPE quand la cible contient des zéros ;
- **transformer un score en décision** par les coûts, avec le seuil $t^* = c/(sV)$, et lire le **gain** et le **lift** quand on raisonne en budget ;
- **vérifier la calibration** d'un modèle par un diagramme de fiabilité, comprendre pourquoi les forêts sont sous-confiantes, le boosting surajusté sur-confiant et les poids de classes décalent la prévalence, et **réparer** par la méthode de Platt, la régression isotonique ou la correction exacte d'un décalage de prévalence, sur un jeu séparé ;
- **expliquer un modèle** : importance par permutation (et son piège avec les variables corrélées), PDP et ICE, LIME (et son instabilité), **valeurs de Shapley** et SHAP, avec la propriété

d'efficacité qui décompose exactement chaque prédiction ; et utiliser l'interprétabilité pour **débusquer une fuite d'information** ;

- (en option) **auditer l'équité** d'un modèle par groupe, comprendre qu'on ne peut pas égaliser à la fois rappel, précision et fausses alertes quand les taux de base diffèrent, et ne pas confondre retirer une variable sensible et supprimer le biais ;
- (en option) **quantifier l'incertitude** par la prédiction conforme : une garantie de couverture valable à distance finie pour n'importe quel modèle, mais marginale, et non conditionnelle.

Un fil conducteur traverse le chapitre : **une métrique, un seuil ou une explication ne valent que par la question à laquelle on les rattache**. L'AUC qui satisfait le data scientist, l'exactitude qui rassure le client et le seuil de 0,5 que l'on a toujours utilisé répondent à trois questions différentes ; aucune n'est « la » question de la gérante, qui est en euros.

Les chapitres complémentaires qui suivent appliquent ces outils à des problèmes particuliers : la détection d'anomalies et de fraude (chapitre 6, où la classe positive est extrêmement rare, donc où la courbe précision-rappel est reine), les systèmes de recommandation (chapitre 7), l'apprentissage semi-supervisé et actif (chapitre 8) et l'apprentissage par renforcement (chapitre 9).

Pour s'entraîner. Cahier, chapitre 5 : applications 5.1 à 5.10 et exercices 5.1 à 5.14.

Chapitre 6 : ⊕ Détection d'anomalies et de fraude

« Chercher une aiguille dans une botte de foin, sans savoir à quoi ressemble l'aiguille.
»

Chapitre complémentaire. Ce chapitre est entièrement facultatif : les chapitres 1 à 5 se lisent et se comprennent sans lui. Il s'adresse à celles et ceux qui veulent voir comment l'apprentissage automatique s'applique à un problème particulier et très répandu : repérer ce qui sort de l'ordinaire.

Une boutique en ligne reçoit chaque jour des centaines de commandes. La très grande majorité sont normales. Quelques-unes, une sur cent environ, sont frauduleuses : une carte volée, un compte piraté, une adresse de livraison jetable. Chaque fraude coûte le montant de la commande (la boutique rembourse la victime et perd la marchandise) ; chaque vérification manuelle coûte du temps. La gérante ne peut pas regarder toutes les commandes ; elle peut en examiner quelques dizaines par jour. **Lesquelles ?**

C'est le problème de la **détection d'anomalies** : classer les observations de la plus suspecte à la moins suspecte, de façon qu'en regardant les premières, on trouve beaucoup plus de fraudes que par hasard. Il a trois particularités qui le distinguent des problèmes de classification du chapitre 2 :

- **les cas intéressants sont rarissimes** (moins d'une commande sur cent), ce qui change la façon de juger un modèle ;
- **on ne sait pas toujours à quoi ressemble une fraude** : les fraudeurs changent de méthode, et ce que l'on n'a jamais vu ne figure dans aucune étiquette ;
- **l'erreur coûte cher dans les deux sens** : rater une fraude coûte la commande, ratisser trop large coûte du temps de vérification et des clients honnêtes importunés.

Le chemin de ce chapitre

- **6.1 Le problème et son évaluation** : ce qu'est une anomalie, pourquoi l'apprentissage supervisé ne suffit pas toujours, et surtout comment **mesurer** un détecteur quand 99 % des cas sont normaux (l'exactitude ne veut plus rien dire).
- **6.2 Méthodes statistiques, distances et densités** : le z-score et sa version robuste, la distance de Mahalanobis, les plus proches voisins et le *Local Outlier Factor* (LOF). Les idées les plus anciennes, souvent les plus solides.
- **6.3 La forêt d'isolement** : isoler un point par des coupures aléatoires, et la raison pour laquelle les anomalies s'isolent vite.

- **6.4 Autoencodeurs** : apprendre à reconstruire les données normales, et mesurer l'erreur de reconstruction. Suivi d'une **comparaison honnête** de toutes les méthodes du chapitre.
- **Bilan du chapitre** et renvois vers le cahier d'exercices.

Pour s'entraîner. Chaque section de ce chapitre se termine par un renvoi vers le chapitre 6 du **cahier d'exercices et d'applications** du volume.

Les données et le protocole

Nous travaillons sur le fichier `donnees/transactions.csv` : **60 000 commandes en ligne** de la boutique, avec, pour chacune, le montant, l'heure, le canal, le mode de paiement, et quelques indices de comportement.

Variable	Signification
montant	montant de la commande, en €
heure, jour_semaine	moment de la commande
appareil_connu	1 si l'appareil a déjà servi à commander sur ce compte
distance_facturation_livraison_km	distance entre l'adresse de facturation et l'adresse de livraison
nb_commandes_24h	nombre de commandes du compte dans les 24 heures précédentes
age_compte_jours	ancienneté du compte, en jours
ip_pays_différent	1 si le pays de l'adresse IP diffère du pays de facturation
delai_depuis_derniere_cmd_h	heures écoulées depuis la dernière commande du compte
nb_articles, mode_paiement, canal	taille de la commande, moyen de paiement, canal
fraude, type_fraude	l'étiquette (0/1) et le type de fraude (0 : aucune ; 1 : compte neuf ; 2 : prise de contrôle)

Les données sont **simulées** (graine fixe) : la boutique et ses clients sont fictifs, et nous connaissons la façon dont les fraudes ont été fabriquées. Le fichier contient **486 fraudes** sur 60 000 commandes, soit **0,81 %**, de deux types :

- **type 1, « compte neuf »** : un compte tout juste créé commande un montant élevé, souvent la nuit, avec une adresse de livraison éloignée de l'adresse de facturation ;
- **type 2, « prise de contrôle »** : un compte ancien est piraté ; le fraudeur commande depuis un appareil inconnu, une adresse IP d'un autre pays, en rafale.

Les fraudes réelles se déguisent : aucun des indices ci-dessus n'est infaillible, et beaucoup de commandes **normales** y ressemblent (un client en voyage, un nouveau compte qui fait un cadeau). C'est volontaire.

Le protocole de tout le chapitre. Nous mettons de côté 30 % des commandes (18 000, dont 146 fraudes) comme **jeu de test**, qui ne sert qu'à **juger**. Les méthodes **non supervisées** sont ajustées sur les 70 % restants **sans jamais voir l'étiquette** ; elles n'ont donc aucun avantage sur la réalité, où l'on ne connaît pas la fraude à l'avance. Parmi ces 42 000 commandes d'apprentissage, 10 000 forment un **échantillon de référence** (qui sert à ajuster les méthodes

de voisinage et les autoencodeurs) et les 32 000 autres un **jeu de validation étiqueté** (246 fraudes), qui sert à **choisir les réglages** (le nombre de voisins, l'architecture d'un réseau...). Choisir un réglage sur le jeu de test le rendrait inutilisable pour juger : c'est la rigueur d'évaluation du chapitre 1 (sections 1.1 et 1.4). Quant aux données d'apprentissage, elles contiennent elles-mêmes des fraudes (0,8 %) : un détecteur non supervisé réel est lui aussi entraîné sur des données « contaminées ».

6.1 Le problème et son évaluation

Avant de chercher *comment* détecter les anomalies, il faut s'entendre sur ce que l'on cherche et sur la façon de savoir si l'on a réussi. Cette section est la plus importante du chapitre : elle explique pourquoi les réflexes des chapitres précédents (l'exactitude, une simple séparation apprentissage/test) ne suffisent plus quand 99 % des cas sont normaux.

6.1.1 Qu'est-ce qu'une anomalie ?

Une **anomalie** est une observation qui s'écarte tellement de ce qui est habituel qu'elle semble provenir d'un autre mécanisme. On en distingue trois sortes, selon ce qui est étrange :

Type	Ce qui est étrange	Exemple dans la boutique
Anomalie ponctuelle	l'observation, prise seule	une commande de 2 400 € alors que le panier habituel est de 60 €
Anomalie contextuelle	l'observation <i>dans son contexte</i>	une commande de 80 € à trois heures du matin, passée depuis un compte créé la veille ; la même commande à midi, depuis un compte ancien, est banale
Anomalie collective	un groupe d'observations, banales une à une	quinze commandes de 20 €, en dix minutes, sur le même compte

Dans nos données, chaque ligne est une commande, mais plusieurs colonnes résument son contexte (l'ancienneté du compte, le nombre de commandes des dernières 24 heures) : c'est ainsi que l'on transforme une anomalie contextuelle ou collective en une anomalie *ponctuelle dans un espace de bonnes variables*. Choisir ces variables est, ici comme ailleurs, la moitié du travail (chapitre 4).

Anomalie n'est pas fraude. Une anomalie est un fait **statistique** (« c'est rare ») ; une fraude est un fait **métier** (« c'est malhonnête »). Un client fidèle qui offre un cadeau de 900 € est une anomalie qui n'est pas une fraude (fausse alerte) ; un fraudeur prudent qui commande 40 € depuis un vieux compte piraté est une fraude qui n'est pas une anomalie (fraude manquée). Un détecteur d'anomalies ne produit jamais que des **pistes** : c'est la vérification qui tranche.

La figure ci-dessous montre pourquoi le problème est difficile. Les deux types de fraude se distinguent des commandes normales sur plusieurs variables, mais **les distributions se chevauchent largement** : aucune variable ne suffit.

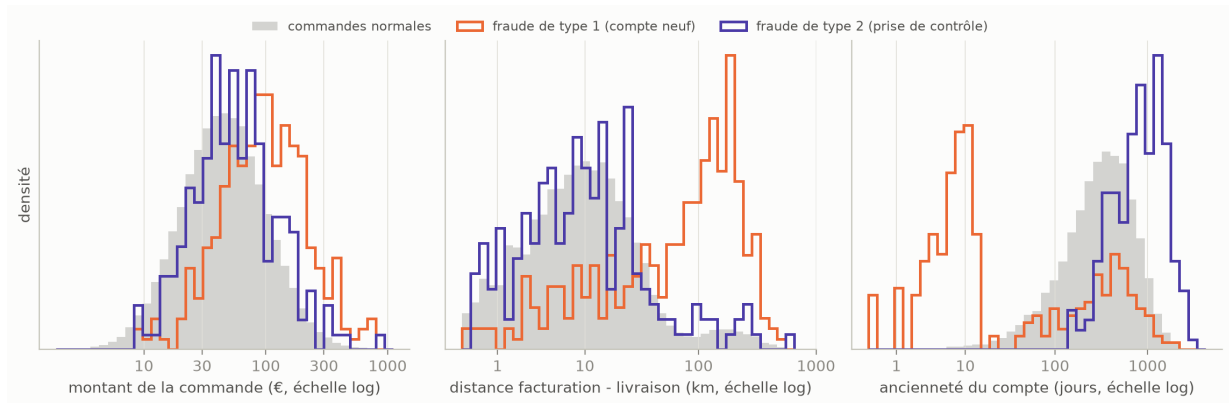


Figure 66: Distribution du montant, de la distance entre les adresses et de l'ancienneté du compte (échelles logarithmiques) pour les commandes normales (gris) et les deux types de fraude (orange et violet). Les fraudes sont décalées, mais leurs distributions chevauchent celle des commandes normales.

6.1.2 Pourquoi l'apprentissage supervisé ne suffit pas toujours

Si l'on dispose d'étiquettes (« fraude » ou « normale »), pourquoi ne pas simplement entraîner un classifieur, comme au chapitre 2 ? Cela marche, et nous le ferons en 6.1.4 : quand les étiquettes sont abondantes et représentatives, c'est souvent la **meilleure** solution. Mais plusieurs obstacles apparaissent, et ils expliquent l'existence de tout un champ de méthodes non supervisées :

1. **Le déséquilibre extrême.** Sur 60 000 commandes, 486 sont des fraudes : environ **une pour 123**. Le modèle voit très peu d'exemples positifs, et un classifieur qui répond « normale » partout se trompe à peine (6.1.3). Le chapitre 4 (section 4.3) présente les remèdes (pondération, rééchantillonnage).
2. **Le délai des étiquettes.** Une fraude n'est *confirmée* que lorsque la victime conteste le débit, souvent **un à trois mois plus tard**. Les étiquettes décrivent donc le monde d'il y a deux mois : le modèle apprend le passé.
3. **L'adversaire s'adapte.** Les fraudeurs changent de méthode dès qu'ils se font prendre. Un modèle entraîné sur les fraudes d'hier reconnaît les fraudes d'hier.
4. **Les fraudes jamais vues.** Un classifieur ne reconnaît que ce qui ressemble à des exemples étiquetés. Un détecteur d'anomalies, lui, signale *tout ce qui est inhabituel*, y compris un type de fraude inédit.
5. **Les étiquettes sont biaisées.** Seules les commandes **examinées** ou **contestées** reçoivent une étiquette ; les fraudes jamais découvertes figurent dans les données comme « normales » (un biais de sélection, au sens du volume II, section 7.1).

Aucune de ces difficultés n'interdit le supervisé ; mais elles justifient d'avoir **plusieurs outils**, de savoir les comparer, et de les combiner (6.4.5).

6.1.3 L'exactitude ne veut plus rien dire

Imaginons un détecteur paresseux qui répond « normale » à chaque commande. Sur notre jeu de test, son **exactitude** (la proportion de réponses justes) est élevée :

L'exactitude du détecteur paresseux est de **99,19 %** : il n'y a rien là d'extraordinaire, c'est simplement la part des commandes normales ($1 - 0,0081$). Il ne détecte pourtant **aucune** fraude. Une mesure qui donne 99,19 % à un détecteur inutile ne peut pas servir à choisir un détecteur.

Un second piège, plus subtil, concerne la **probabilité qu'une alerte soit juste**. Supposons un détecteur plutôt bon : il repère **80 %** des fraudes (rappel) et ne déclenche une fausse alerte que

sur 1 % des commandes normales (taux de fausses alertes). Sur 100 000 commandes avec une fraude pour 123 commandes (0,8 %), combien d'alertes sont de vraies fraudes ? Par la formule de Bayes (volume I, section 2.1.6), avec F = « la commande est une fraude » et A = « le détecteur déclenche une alerte » :

$$P(F | A) = \frac{P(A | F) P(F)}{P(A | F) P(F) + P(A | \bar{F}) P(\bar{F})} = \frac{0,80 \times 0,008}{0,80 \times 0,008 + 0,01 \times 0,992} = \frac{0,0064}{0,01632} \approx 0,39.$$

Moins de quatre alertes sur dix sont de vraies fraudes, alors que le détecteur est excellent sur le papier. La raison est la même que pour les tests médicaux du volume I : quand l'événement est rare, même un faible taux de fausses alertes sur l'immense majorité normale produit beaucoup plus de fausses alertes que de vraies. C'est pourquoi il faut raisonner en **précision** (« parmi les alertes, quelle part est juste ? »), et pas seulement en taux d'erreur.

À retenir pour tout problème rare. L'exactitude mesure surtout la classe majoritaire. Il faut deux questions complémentaires : « *combien de fraudes trouve-t-on ?* » (le **rappel**) et « *parmi les alertes, combien sont de vraies fraudes ?* » (la **précision**).

6.1.4 Un modèle supervisé de référence

Avant de regarder les méthodes non supervisées, fixons un point de comparaison : un **classifieur supervisé** entraîné avec les étiquettes du jeu d'apprentissage. C'est la méthode du chapitre 2 (gradient boosting, section 2.4), avec une **pondération des classes** pour compenser le déséquilibre (section 4.3). Les détails d'un tel modèle sont ceux des chapitres 2 et 4 ; il suffit ici de l'ajuster et de scorer le jeu de test :

```
from sklearn.ensemble import HistGradientBoostingClassifier

modele = HistGradientBoostingClassifier(class_weight="balanced", max_iter=150, random_state=0)
modele.fit(X_app, y_app)                                # les étiquettes sont utilisées ici
scores_gbm = modele.predict_proba(X_test)[: , 1]        # probabilité estimée de fraude
```

Ce modèle de référence obtient, sur le jeu de test, une aire sous la courbe ROC de **0,971** : un résultat qui semble presque parfait. Nous allons voir que cette mesure est trompeuse (6.1.5), car la réalité est beaucoup plus modeste : au budget de 180 alertes (1 % des commandes), 55 % des alertes sont de vraies fraudes et 68 % des fraudes du test sont retrouvées.

6.1.5 Les bons outils de mesure

Un détecteur produit un **score** (plus il est élevé, plus la commande est suspecte) ; on déclenche une alerte au-dessus d'un seuil. Pour chaque seuil, on compte quatre nombres : les vraies alertes (VP), les fausses alertes (FP), les fraudes manquées (FN) et les commandes normales laissées passer (VN). Sur un exemple à la main, 10 000 commandes dont 80 fraudes, et un détecteur qui déclenche 100 alertes dont 40 justes :

	Fraude	Normale	Total
Alerte	$VP = 40$	$FP = 60$	100
Pas d'alerte	$FN = 40$	$VN = 9\,860$	9 900
Total	80	9 920	10 000

- **Exactitude** : $(40 + 9\,860)/10\,000 = 99,0\%$ (moins bonne que celle du détecteur paresseux, 99,2 %, pourtant bien plus utile).
- **Précision** : $VP/(VP + FP) = 40/100 = 40\%$: sur dix alertes, quatre sont justes.
- **Rappel** : $VP/(VP + FN) = 40/80 = 50\%$: la moitié des fraudes est retrouvée.
- **Mesure F1** (la moyenne harmonique des deux) : $2 \times 0,4 \times 0,5 / (0,4 + 0,5) \approx 0,44$.

Précision et rappel varient en sens inverse quand on déplace le seuil : abaisser le seuil augmente le rappel (on trouve plus de fraudes) mais diminue la précision (on déclenche plus de fausses alertes). La **courbe précision-rappel** (courbe PR) trace cette tension pour tous les seuils, et son aire est la **précision moyenne** (*average precision*, AP). La courbe **ROC**, vue au chapitre 5 (section 5.1), trace le rappel en fonction du taux de fausses alertes, et son aire est l'AUC.

Sur des données déséquilibrées, l'AUC est trop optimiste. Le taux de fausses alertes se calcule par rapport aux 59 000 commandes normales : même 1 000 fausses alertes ne représentent qu'environ 2 % de ce total, et la courbe ROC reste collée au coin supérieur gauche. La précision, elle, est calculée par rapport aux **alertes** : 1 000 fausses alertes écrasent les quelques centaines de vraies. La courbe PR est donc beaucoup plus sévère, donc plus informative. Un détecteur **aléatoire** a une AUC de 0,5 mais une AP égale à la prévalence (ici 0,008) : l'AP se lit en comparaison de ce plancher.

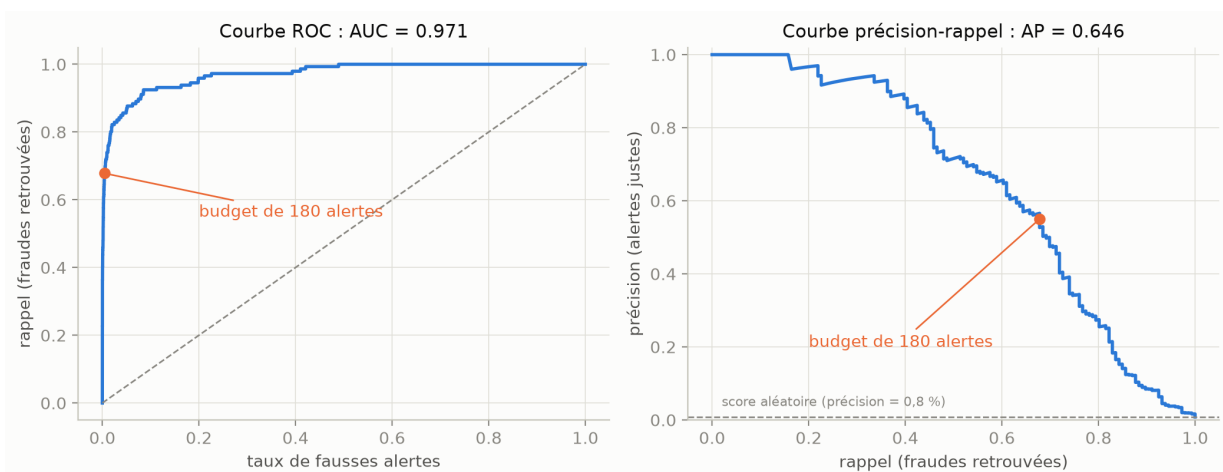


Figure 67: Le même détecteur supervisé, vu par la courbe ROC (à gauche) et par la courbe précision-rappel (à droite). L'AUC de 0,971 donne l'impression d'un détecteur presque parfait, alors que la précision chute dès que l'on veut retrouver plus de la moitié des fraudes. Le point orange est le fonctionnement au budget de 180 alertes.

Le même détecteur obtient une AUC de **0,971** et une AP de **0,646**. Au budget de 180 alertes, on n'a que 99 vraies alertes et 81 fausses : le taux de fausses alertes n'est que de **0,45 %** (ce qui paraît négligeable sur la courbe ROC), alors que **45 % des alertes sont fausses** (ce qui compte pour la gérante). C'est la courbe PR qui dit la vérité opérationnelle.

Trois mesures sont utilisées dans tout le chapitre, avec un détecteur évalué à **budget fixé** :

- l'**AP** (précision moyenne), qui résume toute la courbe PR ;
- la **précision à k** : la part de vraies fraudes parmi les k commandes les plus suspectes, où k est le nombre de fraudes du test ;
- le **rappel au budget** : la part des fraudes retrouvées parmi les 180 alertes (1 % des commandes), décomposé par type de fraude.

6.1.6 Du score à l'argent

Le budget d'alertes est une contrainte de la gérante, pas une loi de la nature. Le bon nombre d'alertes dépend des **coûts**. Posons des hypothèses simples :

- une fraude non détectée **coûte le montant de la commande** (remboursement et marchandise perdue) ;
- une alerte **coûte 4 €** de vérification (que la commande soit frauduleuse ou non, on doit la regarder) ;
- une fraude détectée est bloquée : elle ne coûte rien de plus que sa vérification.

Si l'on déclenche les n alertes les plus suspectes, le coût total est la somme des montants des fraudes qui passent entre les mailles, plus $4n$. On peut tracer ce coût en fonction de n :

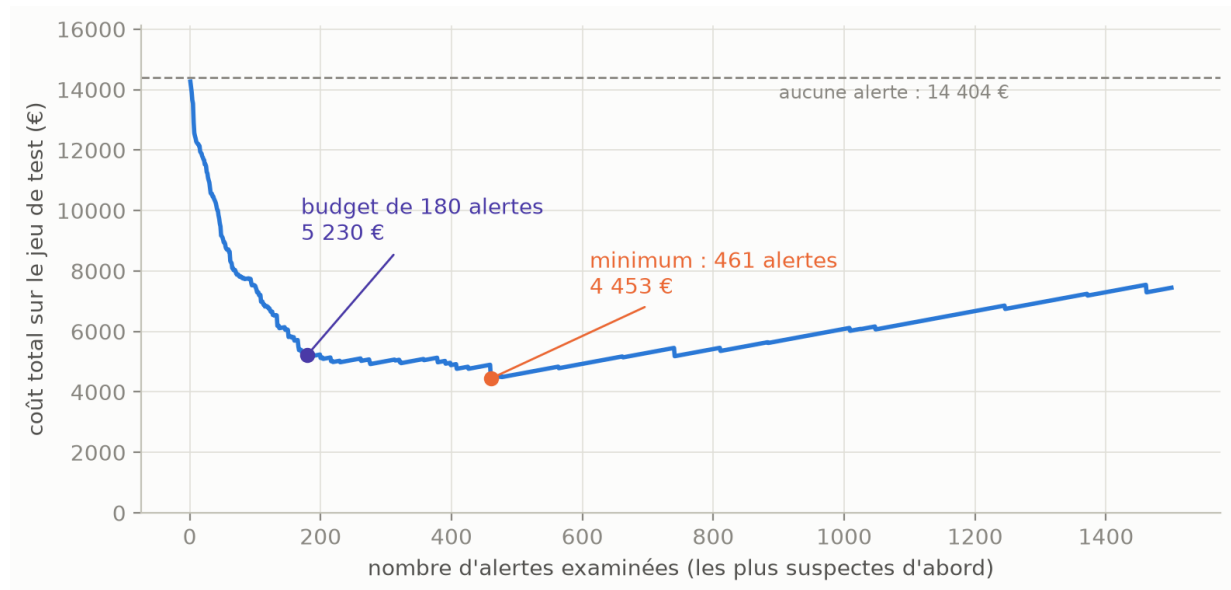


Figure 68: Coût total (fraudes manquées plus vérifications) en fonction du nombre d'alertes examinées, avec le détecteur supervisé. Sans aucune alerte, la boutique perd tout le montant des fraudes. Le coût passe par un minimum, puis remonte quand les vérifications coûtent plus que les fraudes qu'elles retrouvent.

Sans aucune alerte, les fraudes du jeu de test coûtent **14 404 €**. Avec les 180 alertes du budget, le coût tombe à **5 230 €** ; il est minimal pour **461 alertes** (environ 2,6 % des commandes) à **4 453 €**, soit **69 % de moins** que sans détecteur. Au-delà, chaque alerte supplémentaire coûte plus (4 €) que ce qu'elle rapporte (les fraudes restantes sont de moins en moins probables dans les alertes en queue de liste).

Deux remarques sur cette courbe. Elle dépend entièrement des **hypothèses** (le coût d'une vérification, le coût d'une fraude manquée) : la gérante les connaît mieux que le modélisateur, et il faut les lui demander. Et elle dit où s'arrêter : un seuil se choisit sur le **coût**, pas sur une mesure statistique abstraite (le chapitre 5, section 5.1, revient sur le choix d'un seuil pour un classifieur).

À retenir.

- Une anomalie est un fait statistique (« c'est rare ») ; une fraude est un fait métier. Le détecteur produit des **pistes** classées, que l'on vérifie.
- Le supervisé est souvent le meilleur choix quand les étiquettes sont abondantes ; il souffre du déséquilibre, du délai des étiquettes, de l'adaptation des fraudeurs et des fraudes inédites.

- **L'exactitude est inutile** (99,19 % pour un détecteur qui ne détecte rien). Il faut la **précision** et le **rappel**, résumés par la courbe PR et l'**AP**, comparée au plancher de la prévalence.
- La courbe ROC est trop optimiste quand les cas positifs sont rarissimes.
- Un seuil se choisit en comparant les **coûts** (fraude manquée contre vérification).

Pour s'entraîner. Cahier, chapitre 6 : application 6.1, exercices 6.1 à 6.4.

6.2 Méthodes statistiques, distances et densités

Les méthodes les plus anciennes de détection d'anomalies reposent sur une idée simple : **une observation est suspecte si elle est loin des autres**. Tout l'art consiste à définir « loin ». Cette section présente quatre définitions de plus en plus fines : la distance à la **médiane**, la distance qui **tient compte des corrélations** (Mahalanobis), la distance **aux voisins les plus proches**, et la distance **relative à la densité locale** (LOF). Ces méthodes ne font jamais appel à l'étiquette : elles apprennent ce qu'est une commande « normale » à partir des seules variables.

6.2.1 Le z-score, et pourquoi il faut parfois le rendre robuste

Le **z-score** d'une valeur x mesure son écart à la moyenne en nombre d'écarts-types : $z = (x - \bar{x})/s$. On juge anormale une valeur dont $|z|$ dépasse 3. Mais la moyenne et l'écart-type sont eux-mêmes **sensibles aux anomalies**.

Prenons sept montants de commandes d'un même compte : 12, 15, 14, 13, 16, 15 et **480** €. La moyenne vaut 80,7 € et l'écart-type 176,1 € : l'anomalie a gonflé les deux mesures qui devaient la repérer. Son z-score n'est que de $(480 - 80,7)/176,1 = 2,27$, **sous le seuil habituel de 3** : le z-score classique ne voit pas la commande de 480 €. C'est le phénomène de **masquage** : l'anomalie se cache derrière sa propre influence.

Le remède est de remplacer la moyenne par la **médiane** et l'écart-type par l'**écart absolu médian** (MAD, *median absolute deviation*) :

$$\text{MAD} = \text{médiane}(|x_i - \text{médiane}(x)|), \quad z^{\text{rob}} = \frac{x - \text{médiane}(x)}{1,4826 \text{ MAD}}.$$

Sur nos sept montants : la médiane vaut 15, les écarts absolus à 15 sont 3, 0, 1, 2, 1, 0 et 465, rangés 0, 0, 1, 1, 2, 3, 465, de médiane **1**. Le MAD vaut 1 et le z-score robuste de 480 est $465/(1,4826 \times 1) \approx 314$: l'anomalie saute aux yeux. La médiane et le MAD, eux, ne bougent presque pas quand on ajoute une valeur extrême : on dit qu'ils sont **robustes**.

D'où vient la constante 1,4826 ? Elle rend le MAD comparable à un écart-type. Pour une loi normale $\mathcal{N}(\mu, \sigma^2)$, la moitié des valeurs est à moins de $a\sigma$ de la médiane, où a vérifie $P(|Z| \leq a) = \frac{1}{2}$, c'est-à-dire $a = \Phi^{-1}(0,75) \approx 0,6745$. Donc $\text{MAD} \approx 0,6745 \sigma$ et $\sigma \approx \text{MAD}/0,6745 = 1,4826 \text{ MAD}$.

Plusieurs variables. Nos commandes ont dix variables. Pour une variable continue ou de comptage, on calcule le z-score de chaque variable, puis on additionne les valeurs absolues : une fraude qui est *modérément* étrange sur plusieurs variables obtient un score élevé, même si aucune variable ne dépasse seule le seuil. Les variables binaires (appareil inconnu, IP étrangère) n'ont pas

de z-score qui ait un sens ; nous n'utilisons ici que les six variables continues ou de comptage : montant, distance entre les adresses, nombre de commandes des dernières 24 heures, ancienneté du compte, délai depuis la dernière commande et nombre d'articles (en logarithme pour les quatre premières, car elles sont très asymétriques : chapitre 4, section 4.1).

Le piège du MAD nul. Si plus de la moitié des valeurs sont égales (donc à la médiane), alors le MAD vaut **zéro** et le z-score robuste est indéfini. C'est exactement le cas du nombre de commandes des dernières 24 heures : **74 %** des commandes sont les premières de la journée pour leur compte, la médiane vaut 0, le MAD vaut 0. Si l'on ignore cette variable (ou si on la laisse à zéro), le détecteur « robuste » devient aveugle à la **rafale de commandes**, qui est justement le signe des prises de contrôle de compte. Une solution classique est d'utiliser, quand le MAD est nul, l'**écart absolu moyen** à la médiane multiplié par 1,2533 (la constante qui le rend comparable à un écart-type pour une loi normale : $E|X - \mu| = \sigma\sqrt{2/\pi}$, donc $\sigma \approx 1,2533 E|X - \mu|$).

Les trois variantes donnent des résultats **très différents** (précision moyenne AP, et part des fraudes de chaque type retrouvées parmi les 180 alertes) :

z-score (somme des écarts)	AP	rappel type 1	rappel type 2
classique	0,433	0,46	0,48
robuste, MAD seul	0,296	0,57	0,17
robuste, avec repli	0,331	0,22	0,60

Il ne faut pas en conclure que « robuste » est un défaut, mais que **le choix de l'échelle revient à choisir le poids de chaque variable**. Le MAD seul ignore la rafale de commandes (variable à MAD nul) : il retrouve bien les fraudes de type 1 (compte neuf) mais presque aucune de type 2 (prise de contrôle). Le repli donne à la rafale une échelle étroite (0,38 contre 1 à 1,5 pour les autres variables) : le moindre excès de commandes pèse lourd, la prise de contrôle ressort, et les fraudes de type 1 passent au second plan. Le z-score classique, avec ses échelles larges, équilibre les deux. Ici la contamination (0,8 %) est trop faible pour fausser sensiblement moyenne et écart-type, ce qui explique que la version classique s'en sorte bien. La robustesse est une **assurance** : elle coûte un peu quand il n'y a pas de sinistre, et elle sauve quand il y en a un.

Reste un défaut fondamental : le z-score regarde **chaque variable séparément**. Une commande peut avoir un montant banal, une heure banale et une distance banale, et être pourtant inhabituelle par la **combinaison** (par exemple un petit montant, de nuit, depuis un compte vieux de deux jours). La section suivante prend en compte ces liaisons.

6.2.2 La distance de Mahalanobis

Considérons deux variables corrélées, par exemple le montant et le nombre d'articles : les grosses commandes contiennent en général plus d'articles. Un point (grosse commande, peu d'articles) est inhabituel *parce qu'il va à contre-courant de la corrélation*, sans être extrême sur aucune variable. La distance euclidienne ne le voit pas ; la **distance de Mahalanobis** le voit, car elle mesure l'écart en tenant compte de la forme du nuage :

$$d_M^2(x) = (x - \mu)^\top \Sigma^{-1} (x - \mu),$$

où μ est le vecteur des moyennes et Σ la matrice de covariance.

Un exemple à la main. Deux variables standardisées (moyenne 0, variance 1) de corrélation $\rho = 0,9$: $\Sigma = \begin{pmatrix} 1 & 0,9 \\ 0,9 & 1 \end{pmatrix}$, d'inverse $\Sigma^{-1} = \frac{1}{0,19} \begin{pmatrix} 1 & -0,9 \\ -0,9 & 1 \end{pmatrix}$ (le déterminant vaut $1 - 0,81 = 0,19$). Comparons deux points à la même distance euclidienne de l'origine :

- $x = (2, 2)$, qui suit la corrélation : $d_M^2 = \frac{1}{0,19}(4 - 2 \times 0,9 \times 4 + 4) = \frac{0,8}{0,19} \approx 4,21$;
- $x = (2, -2)$, qui va à contre-courant : $d_M^2 = \frac{1}{0,19}(4 + 2 \times 0,9 \times 4 + 4) = \frac{15,2}{0,19} = 80$.

Les deux points sont à la distance euclidienne $\sqrt{8}$ de l'origine, mais l'un est parfaitement banal et l'autre rarissime. Mahalanobis dit que le premier est dans le nuage et le second hors de portée.

Pourquoi cette formule, et quelle valeur seuil ? Écrivons la décomposition de Cholesky $\Sigma = LL^T$ et posons $y = L^{-1}(x - \mu)$. Si $x \sim \mathcal{N}(\mu, \Sigma)$, alors $y \sim \mathcal{N}(0, I_p)$: on a « blanchi » les variables, qui deviennent indépendantes et de variance 1. Or $y^T y = (x - \mu)^T L^{-T} L^{-1} (x - \mu) = (x - \mu)^T \Sigma^{-1} (x - \mu) = d_M^2$. C'est donc une somme de p carrés de lois normales réduites indépendantes, qui suit une **loi du χ^2 à p degrés de liberté**. Un point est « à rejeter » au niveau 99,9 % si d_M^2 dépasse le quantile 0,999 de cette loi : 13,8 pour $p = 2$, 29,6 pour $p = 10$ (nos dix variables).

Cette théorie suppose des données gaussiennes, ce qui n'est pas le cas de nos commandes (certaines variables sont binaires, d'autres très asymétriques). Sur le jeu de test, **2,3 %** des commandes normales dépassent le seuil de 29,6, alors que la théorie en prévoirait 0,1 %. Le seuil théorique n'est donc **pas calibré** ; nous ne l'utilisons pas pour décider, mais pour **classer** (on garde les 180 commandes de plus grande distance).

Le masquage, encore. La moyenne μ et la covariance Σ sont **estimées sur les données**, donc faussées par les anomalies elles-mêmes. Quand elles sont nombreuses ou groupées, elles gonflent Σ et leurs distances s'écraquent. L'estimateur à **déterminant de covariance minimal** (MCD, *minimum covariance determinant*) calcule μ et Σ sur le sous-ensemble de $h \approx 90$ % des points dont la covariance a le plus petit déterminant : on laisse de côté les points qui étirent le nuage. La figure montre l'effet sur un exemple simulé où 20 % des points forment un groupe lointain.

L'ellipse classique, étirée par les contaminants, **les englobe** : **aucun** d'entre eux (0 %) ne dépasse le seuil, contre 100 % avec l'estimateur robuste. Sur nos transactions, en revanche, les fraudes ne représentent que 0,8 % des données : il n'y a pas de masquage à craindre, et l'estimateur robuste n'apporte pas grand-chose (précision moyenne de 0,410 contre 0,382 pour la version classique, avec un temps de calcul bien plus long). **La robustesse a un coût, qu'il faut payer seulement si la contamination le justifie** : plus les anomalies sont nombreuses ou groupées dans les données d'apprentissage, plus elle devient nécessaire.

6.2.3 Les plus proches voisins

La distance de Mahalanobis suppose un seul nuage de forme elliptique. Les vraies données ont souvent plusieurs groupes, des formes courbes, des zones vides. L'idée des **plus proches voisins** s'affranchit de toute forme : *une commande est suspecte si ses voisins sont loin*. Pour chaque commande, on calcule la **distance moyenne à ses k plus proches voisins** dans un échantillon de référence de commandes (supposées en majorité normales) ; plus elle est grande, plus la commande est isolée.

Le nombre de voisins k est un **réglage**. Avec $k = 1$, un seul voisin proche suffit à « innocenter » une commande, et le score est bruité ; avec k très grand, on compare la commande à la masse entière

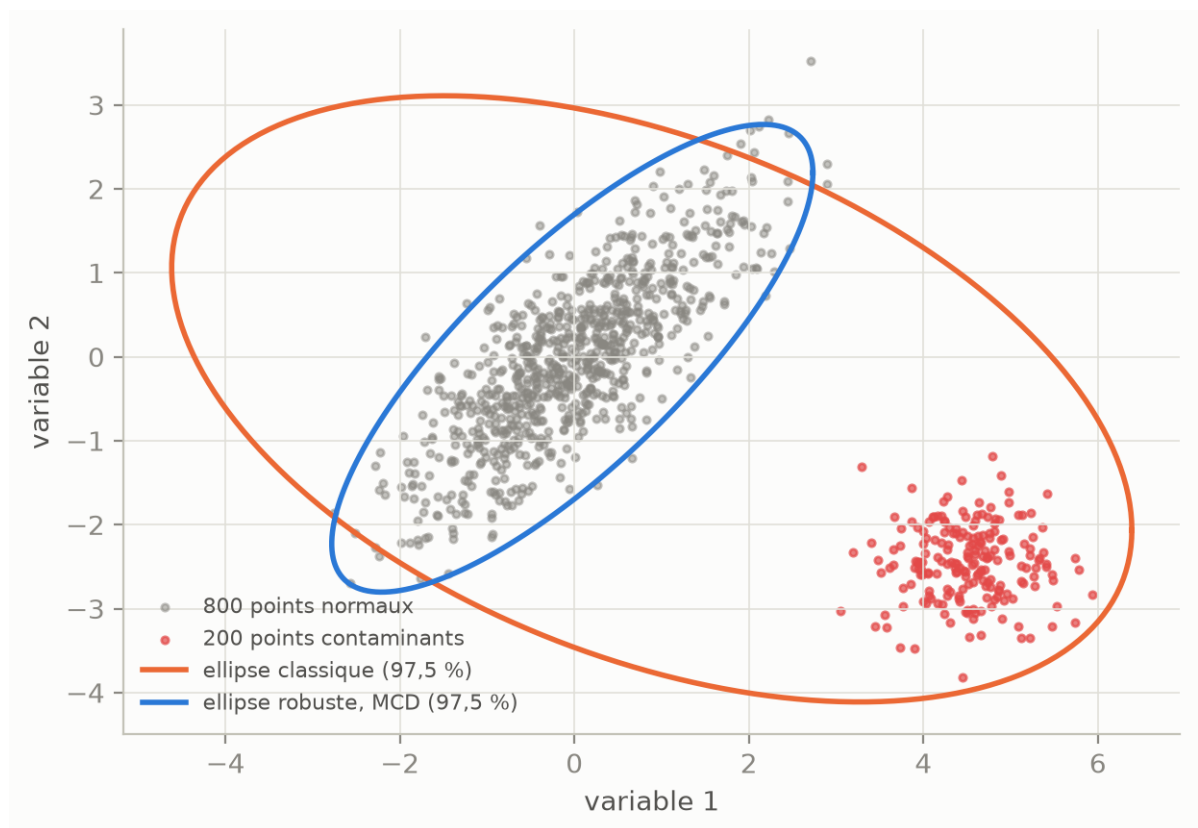


Figure 69: Un exemple simulé : 800 points normaux (gris) corrélés et 200 points contaminants (rouge). L'ellipse classique (orange), estimée sur tous les points, est étirée vers les contaminants et les englobe ; l'ellipse robuste MCD (bleue) épouse les points normaux et laisse les contaminants à l'extérieur.

des commandes, et l'on perd la finesse. Nous le choisissons **sur le jeu de validation** (jamais sur le test), parmi 1, 3, 5, 10, 30 et 100 :

k	AP validation	AP test
1	0.261	0.303
3	0.321	0.378
5	0.350	0.411
10	0.382	0.444
30	0.400	0.460
100	0.399	0.448

k choisi sur la validation : 30

La précision moyenne sur le jeu de validation passe de 0,261 pour $k = 1$ à **0,400 pour $k = 30$** , puis plafonne (0,399 pour $k = 100$) : nous retenons $k = 30$. Le jeu de test donne le même classement des réglages (0,303 pour $k = 1$, 0,460 pour $k = 30$), ce qui rassure sur le choix.

```
from sklearn.neighbors import NearestNeighbors

voisins = NearestNeighbors(n_neighbors=k_knn).fit(Z_app[reference])    # 10 000 commandes de
↳ référence, variables standardisées
distances, _ = voisins.kneighbors(Z_test)
score_knn = distances.mean(axis=1)                                   # distance moyenne aux k plus
↳ proches voisins
```

Quelques points de méthode :

- **Il faut des variables à des échelles comparables** (chapitre 4, section 4.1) : sans cela, la variable d'échelle la plus grande décide seule des distances. La standardisation est le choix par défaut, et nous l'appliquons à tous les détecteurs de ce chapitre pour les comparer sur le même pied. Elle n'est pourtant pas toujours le meilleur : ici nos variables ont déjà été passées au logarithme et ont des échelles voisines, alors que les deux variables binaires (IP étrangère, appareil inconnu) ont un écart-type faible (0,20 et 0,30, contre 0,56 à 1,22 pour les autres) : standardiser les **multiplie par 5 et 3,3** et leur donne un poids très supérieur dans la distance. Avec les variables brutes, la précision moyenne des mêmes voisins monte à **0,511** (contre 0,460) ; en ne standardisant que les variables continues et en laissant les deux variables binaires en 0/1, on obtient **0,513**, ce qui confirme l'explication. L'application 6.3 du cahier reproduit ces calculs.
- **Le coût de calcul.** Comparer chaque commande à toutes les autres est quadratique ; nous utilisons un **échantillon de référence** de 10 000 commandes, ce qui suffit à décrire le comportement normal (l'application 6.3 montre que la précision moyenne passe de 0,37 avec 500 commandes de référence à 0,46 avec 2 000, puis ne progresse plus que lentement, jusqu'à 0,48 avec 20 000). Des structures d'index (arbres, graphes de voisinage approchés) accélèrent le calcul quand la dimension est modeste.

Cette méthode toute simple obtient une précision moyenne de **0,460** sur le jeu de test, et retrouve **69 %** des fraudes de type 2 dans les 180 alertes (contre 26 % de celles de type 1). Elle repose sur très peu d'hypothèses, et c'est l'une des raisons pour lesquelles elle reste un excellent premier essai.

6.2.4 Le facteur local d'anomalie (LOF)

Une limite des distances aux voisins : elles supposent que la **densité est la même partout**. Imaginons deux groupes de clients, l'un très concentré (des habitudes d'achat très régulières) et l'autre très étalé. Un point à 1,3 unité du groupe concentré est une **vraie anomalie pour ce groupe**, mais sa distance aux voisins (1,3) est *inférieure* aux distances ordinaires dans le groupe étalé. Une distance globale le noie.

Le **LOF** (*Local Outlier Factor*) compare la densité autour d'un point à celle de ses voisins. Voici les définitions, pour un entier k :

1. la **k -distance** $d_k(p)$: la distance de p à son k -ième plus proche voisin ; $N_k(p)$ est l'ensemble de ses k plus proches voisins ;
2. la **distance d'atteignabilité** de p depuis o : $rd_k(p, o) = \max\{d_k(o), d(p, o)\}$ (on ne descend pas en dessous de la k -distance du voisin, ce qui lisse les fluctuations) ;
3. la **densité d'atteignabilité locale** : $lrd_k(p) = \left(\frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} rd_k(p, o) \right)^{-1}$, l'inverse de la distance d'atteignabilité moyenne ;
4. le **facteur d'anomalie** $LOF_k(p) = \frac{1}{|N_k(p)|} \sum_{o \in N_k(p)} \frac{lrd_k(o)}{lrd_k(p)}$: le rapport entre la densité des voisins et celle de p .

Un LOF voisin de 1 signifie que p est aussi dense que ses voisins ; un LOF **nettement supérieur à 1** signifie que p est plus isolé que ses voisins.

Un exemple à la main : six points sur une droite, aux abscisses 0; 1; 3; 4,4; 6,1; 10, avec $k = 2$. Le tableau donne, pour chaque point, ses deux voisins, sa 2-distance, sa densité et son LOF (calculés ligne à ligne avec les formules ci-dessus, puis vérifiés avec `scikit-learn`) :

abscisse	voisins (abscisses)	2-distance	densité lrd	LOF
0.0	[1.0, 3.0]	3.0	0.4000	1.176
1.0	[0.0, 3.0]	2.0	0.4000	1.176
3.0	[4.4, 1.0]	2.0	0.5405	0.733
4.4	[3.0, 6.1]	1.7	0.3922	1.220
6.1	[4.4, 3.0]	3.1	0.4167	1.119
10.0	[6.1, 4.4]	5.6	0.2105	1.921

(vérification : mêmes LOF que `scikit-learn`)

Lecture : le point d'abscisse **10**, isolé au bout de la droite, a un LOF de **1,92** ; les points au centre du groupe (3 ; 4,4 ; 6,1) ont des LOF proches de 1 (0,73 ; 1,22 ; 1,12) ; le point 3, entouré, est même plus dense que ses voisins (0,73).

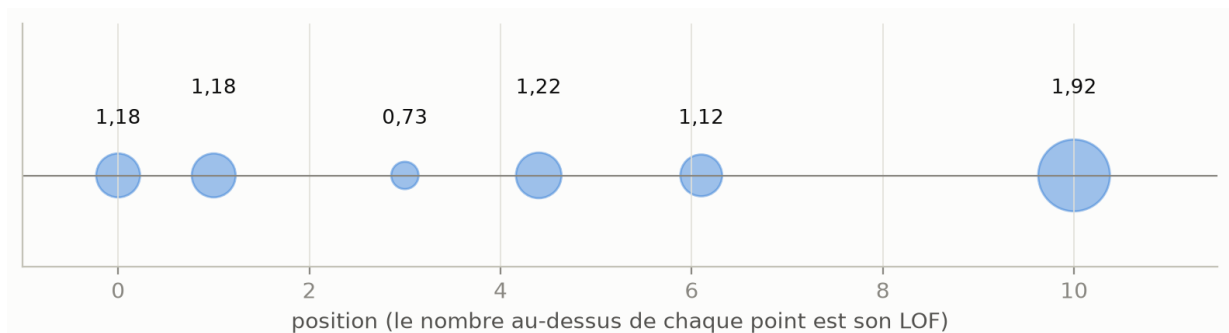


Figure 70: Six points sur une droite. La surface de chaque disque est proportionnelle au carré du facteur d'anomalie (LOF) : le point isolé à 10 a le LOF le plus élevé (1,92).

Voici maintenant l'avantage du LOF sur la distance aux voisins, sur un exemple simulé : un groupe concentré de 200 points (écart-type 0,25), un groupe étalé de 100 points (écart-type 1,5),

et un point ajouté à 1,3 unité du groupe concentré (cercle orange).

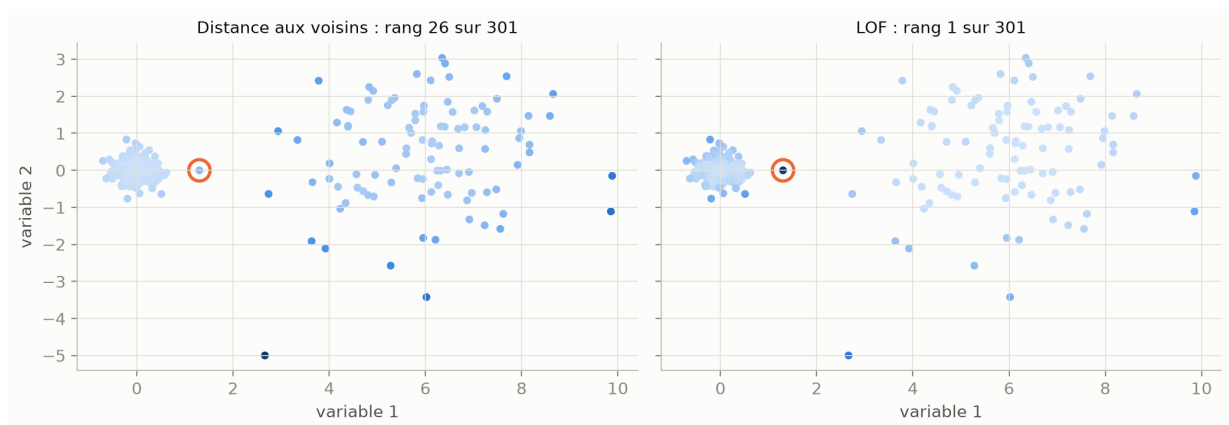


Figure 71: Deux groupes de densités différentes. Les points sont colorés par la distance moyenne aux voisins (à gauche) ou par le LOF (à droite) ; plus la couleur est foncée, plus le score est élevé. Le point ajouté près du groupe concentré (cercle orange) est noyé parmi les points du groupe étalé à gauche, et il est le plus anormal à droite.

Selon la distance aux voisins, le point ajouté n'arrive qu'au **26e rang** sur 301 : les points du groupe étalé, naturellement plus éloignés les uns des autres, le dominant (la plus grande distance du groupe étalé dépasse 4). Selon le LOF, il est **premier**, avec un LOF d'environ 5 : relativement à ses voisins, il est cinq fois moins dense.

Le LOF dépend lui aussi d'un réglage k , et beaucoup plus que les voisins simples. Nous le choisissons sur le jeu de validation, parmi 10, 30, 100 et 300 :

k	AP validation	AP test
10	0.114	0.113
30	0.243	0.306
100	0.408	0.471
300	0.433	0.492

k choisi sur la validation : 300

La précision moyenne sur la validation passe de **0,114** pour $k = 10$ à **0,433** pour $k = 300$: un LOF réglé à la légère est un très mauvais détecteur, un LOF bien réglé est l'un des meilleurs. La raison est que la densité « locale » estimée avec peu de voisins est très bruitée ; avec 300 voisins, elle devient stable. Le meilleur k est en bout de grille : une grille plus large aurait peut-être fait mieux, ce que nous n'avons pas exploré.

```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(n_neighbors=k_lof, novelty=True).fit(Z_app[reference])
score_lof = -lof.score_samples(Z_test) # novelty=True : on score de nouveaux points
```

Sur nos transactions, le LOF réglé à $k = 300$ obtient une précision moyenne de **0,492** sur le jeu de test, mieux que les voisins simples (0,460) ; il retrouve **52 %** des fraudes de type 1 et **49 %** de celles de type 2. Le LOF repère les commandes isolées *par rapport à leur voisinage*, ce qui convient à des fraudes qui s'éloignent localement des habitudes sans être extrêmes dans l'absolu.

6.2.5 Ce que voient ces méthodes, et ce qu'elles ratent

Toutes les méthodes de cette section sont ajustées sans étiquette, et toutes ont reçu le même jeu d'apprentissage. Le tableau résume leurs performances sur le jeu de test.

	AUC	AP	précision à k	rappel type 1	rappel type 2
z-score classique	0.939	0.433	0.452	0.457	0.477
z-score robuste (avec repli)	0.930	0.331	0.342	0.222	0.600
Mahalanobis	0.946	0.382	0.356	0.198	0.615
Mahalanobis robuste (MCD)	0.949	0.410	0.404	0.235	0.677
Plus proches voisins (k = 30)	0.959	0.460	0.432	0.259	0.692
LOF (k = 300)	0.944	0.492	0.493	0.519	0.492

Deux enseignements, que la section 6.4.5 reprendra avec les méthodes suivantes :

1. **Chaque méthode définit autrement ce qui est « normal »**, donc trouve d'autres fraudes. Mahalanobis, les voisins (avec 69 %) retrouvent surtout le type 2 et peu le type 1 (de 20 à 26 %) ; le z-score classique les équilibre (0,46 et 0,48) ; et le LOF réglé à $k = 300$ retrouve les deux types à parts à peu près égales (52 % et 49 %). Il n'existe pas de « meilleure méthode » dans l'absolu.
2. **Le réglage compte autant que la méthode.** Le LOF passe d'une précision moyenne de 0,11 (avec 10 voisins) à 0,49 (avec 300) ; les voisins simples varient de 0,30 à 0,46 selon k . Aucun réglage n'est universel, et sans étiquette on ne peut pas savoir lequel convient : c'est pourquoi on garde un **jeu de validation étiqueté**, même petit, pour régler, et un jeu de test pour juger.

À retenir.

- Le **z-score** juge chaque variable seule ; moyenne et écart-type sont faussés par les anomalies (**masquage**) : médiane et **MAD** (constante 1,4826) sont robustes. Attention au **MAD nul** (plus de la moitié de valeurs identiques).
- La **distance de Mahalanobis** $d_M^2 = (x - \mu)^\top \Sigma^{-1} (x - \mu)$ tient compte des corrélations ; sous hypothèse gaussienne elle suit un χ_p^2 . La version robuste (MCD) n'est utile que si la contamination est importante.
- La **distance aux k plus proches voisins** ne suppose aucune forme ; il faut des variables à des échelles comparables et un **k choisi sur un jeu de validation**.
- Le **LOF** compare la densité d'un point à celle de ses voisins : il détecte les anomalies **locales** quand les densités varient ; il est **très sensible à k** (précision moyenne de 0,11 à 0,49 selon le réglage).
- Aucune de ces méthodes n'est la meilleure partout ; chacune a ses fraudes de prédilection.

Pour s'entraîner. Cahier, chapitre 6 : applications 6.2 et 6.3, exercices 6.5 à 6.7.

6.3 La forêt d'isolement

Les méthodes de la section précédente décrivent d'abord ce qui est **normal** (un centre, une covariance, une densité), puis déclarent anormal ce qui s'en écarte. Elles font un travail inutile : on ne cherche pas à bien décrire les commandes normales, mais à repérer les rares qui ne le sont pas. La **forêt d'isolement** (*isolation forest*, Liu, Ting et Zhou, 2008) prend le problème à l'envers : au lieu de décrire le normal, elle mesure **la facilité avec laquelle on isole chaque point**.

6.3.1 L'idée : isoler plutôt que décrire

Voici huit valeurs : sept proches les unes des autres (2 ; 3 ; 3,5 ; 4 ; 4,5 ; 5 ; 5,5) et une très éloignée (30). On joue à un jeu : on tire une **coupure au hasard** entre le minimum et le maximum, ce qui sépare les valeurs en deux groupes ; on garde le groupe qui contient la valeur étudiée, on recommence, et on compte le nombre de coupures nécessaires pour la **laisser seule**.

- Pour la valeur 30, la première coupure l'isole dès qu'elle tombe entre 5,5 et 30 : c'est le cas avec la probabilité $(30 - 5,5)/(30 - 2) = 87,5\%$. Il suffit d'**une coupure ou deux**.
- Pour la valeur 4, au milieu du groupe, il faut en général **cinq coupures** : elle est entourée de voisines qu'il faut écarter une à une.

En répétant ce jeu avec 5 000 séquences de coupures aléatoires, on obtient le nombre moyen de coupures nécessaires pour isoler chaque valeur :

valeur	coupures moyennes pour l'isoler
2.0	2.95
3.0	4.19
3.5	4.70
4.0	4.76
4.5	4.70
5.0	4.40
5.5	3.55
30.0	1.14

probabilité que 30 soit isolée dès la 1re coupure (simulation) : 0.869 ; calcul exact : 0,875

La valeur 30 est isolée en **1,14 coupure** en moyenne, contre 4,4 à 4,8 pour les valeurs centrales. **Les anomalies sont peu nombreuses et différentes : elles s'isolent vite.** C'est tout le principe de la méthode : la longueur moyenne du « chemin » qui mène à une observation est une mesure d'anormalité, sans qu'on ait jamais eu à décrire ce qu'est la normalité.

Pourquoi cela marche. Les points normaux se trouvent dans des régions denses : il faut beaucoup de coupures pour les séparer de leurs voisins. Un point isolé a autour de lui du vide : presque n'importe quelle coupure le sépare du reste.

6.3.2 La longueur de chemin et le score d'anomalie

Cette idée s'applique à plusieurs variables : à chaque étape, on choisit **une variable au hasard**, puis une **coupure au hasard** entre le minimum et le maximum de cette variable dans le groupe courant. On répète jusqu'à ce que chaque point soit seul (ou qu'une hauteur maximale soit atteinte). Le résultat est un **arbre d'isolement** ; la **longueur de chemin** $h(x)$ d'un point x est le nombre de coupures qui l'isolent. Une **forêt** de nombreux arbres, construits chacun sur un sous-échantillon, donne une longueur moyenne $E[h(x)]$.

Pour transformer cette longueur en score comparable d'un jeu de données à l'autre, on la normalise. Un arbre d'isolement de n points a la même structure qu'un **arbre binaire de recherche** : la longueur moyenne d'un chemin y est connue. C'est celle d'une recherche infructueuse, soit

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(i) \approx \ln(i) + 0,5772 \text{ (constante d'Euler),}$$

où $H(i)$ est le i -ième nombre harmonique. On définit alors le **score d'anomalie**

$$s(x, n) = 2^{-E[h(x)]/c(n)}$$

Le score est toujours entre 0 et 1, et se lit ainsi :

- si $E[h(x)]$ est **très petit** (isolé presque immédiatement), $s \rightarrow 1$: forte anomalie ;
- si $E[h(x)] = c(n)$ (un point moyen), $s = 2^{-1} = 0,5$: rien de particulier ;
- si $E[h(x)]$ est **grand** (profondément enfoui), $s \rightarrow 0$: point très normal.

n	c(n)
2	1.000
8	3.296
16	4.696
64	7.472
256	10.245
1000	12.970

score pour $n = 256$ ($c = 10.24$) : $\{ 'E[h] = 2' : 0.873, 'E[h] = 4' : 0.763, 'E[h] = 10.2' : 0.502, 'E[h] = 20' : 0.258 \}$

Pour $n = 256$ points, $c(256) \approx 10,2$: un point isolé en 2 coupures a un score de 0,87, un point isolé en 4 coupures de 0,76, un point moyen de 0,50, et un point qu'il faut 20 coupures pour isoler de 0,26.

Une approximation, pas une identité. La normalisation $c(n)$ est une *analogie* avec les arbres de recherche (c'est l'argument de l'article d'origine). Nos coupures sont tirées uniformément entre le minimum et le maximum, et non selon les rangs, ce qui change un peu la structure des arbres. En simulant directement des coupures uniformes sur des échantillons de n valeurs normales, on trouve une profondeur moyenne légèrement supérieure à $c(n)$ (de 7 à 10 % de plus, pour $n = 8, 64$ et 256). Cela ne change pas l'**ordre** des scores, qui est ce dont on se sert pour classer les alertes ; seul le point d'équilibre à 0,5 est un peu décalé.

6.3.3 De l'arbre à la forêt

Dans la pratique, la forêt d'isolement construit t **arbres** (200 dans nos essais). Chacun est bâti sur un **petit sous-échantillon** de ψ points tirés au hasard (256 par défaut), avec une hauteur maximale de $\lceil \log_2 \psi \rceil = 8$: inutile de pousser l'arbre plus profond, puisque les anomalies s'isolent bien avant. Ces choix donnent trois propriétés précieuses :

- **Elle ne calcule aucune distance** et n'estime ni moyenne ni covariance : son coût de construction ne dépend presque pas de la taille n du jeu de données (chaque arbre n'utilise que ψ points), et le score d'un point coûte $t \log \psi$ opérations. Elle passe à l'échelle de millions de lignes.
- **Le sous-échantillonnage la protège du masquage et de l'« engorgement »** (*swamping*) : avec peu de points par arbre, les anomalies ne sont plus noyées dans des régions denses de points normaux voisins, et les points normaux ne sont plus pris pour des anomalies parce qu'ils voisinent avec elles.
- **Elle est invariante aux changements d'échelle linéaires** de chaque variable (la coupure est tirée dans l'étendue de la variable), mais pas aux transformations non linéaires comme le logarithme : c'est pourquoi nos variables très asymétriques (montant, distance, ancienneté, délai) sont transformées en logarithme avant tout (chapitre 4, section 4.1).

Les choix de t et de ψ comptent. Voici la précision moyenne (AP) obtenue sur le jeu de test pour différentes valeurs, en moyenne sur trois graines :

nombre d'arbres ($\psi = 256$) : $\{10: 0.216, 50: 0.318, 200: 0.366\}$
 taille du sous-échantillon ψ (100 arbres) : $\{32: 0.287, 64: 0.3, 256: 0.355, 1024: 0.373, 4096: 0.398\}$

Avec trop peu d'arbres (10), le score est bruité ; au-delà de 100 arbres, on gagne peu. Quant au sous-échantillon, ici, **plus il est grand, mieux c'est** (de 0,29 pour $\psi = 32$ à 0,40 pour $\psi = 4096$)

: la valeur de 256 recommandée par l'article d'origine est une valeur par défaut raisonnable, pas un optimum. Comme toujours, ces réglages se décident mieux avec quelques étiquettes pour évaluer.

6.3.4 Le paramètre de contamination et le seuil

La forêt produit un **score**, pas une décision. Pour décider, scikit-learn propose un paramètre **contamination** : la proportion supposée d'anomalies dans les données. Il fixe le **seuil** au quantile correspondant des scores d'apprentissage. Sa valeur par défaut, "auto", utilise un seuil fixe (score de 0,5) tiré de l'article d'origine.

Sur nos données, le seuil par défaut déclenche une alerte sur **20,5 %** des commandes (3 699 alertes) : on retrouve **92,5 %** des fraudes, mais la précision tombe à **3,6 %**, et la gérante croulerait sous les vérifications. En fixant `contamination = 0,0081` (la vraie proportion de fraudes), on obtient environ 150 alertes, de précision **34 %**, et un rappel de **35 %**.

Cet exemple dit quelque chose d'important : **la contamination n'est pas un paramètre statistique que les données révèlent, c'est un choix opérationnel**. En pratique on ne connaît pas la proportion de fraudes ; ce que l'on connaît, c'est le **budget d'alertes** que l'on peut traiter. C'est pourquoi nous classons toujours les commandes par score et gardons les 180 plus suspectes (1 % du test), plutôt que de nous fier à un seuil théorique.

6.3.5 Sur nos transactions

```
from sklearn.ensemble import IsolationForest

foret = IsolationForest(n_estimators=200, random_state=0).fit(Z_app)
score_isolement = -foret.score_samples(Z_test)
↳ anormal                                     # aucune étiquette
                                                # score s : plus grand = plus
```

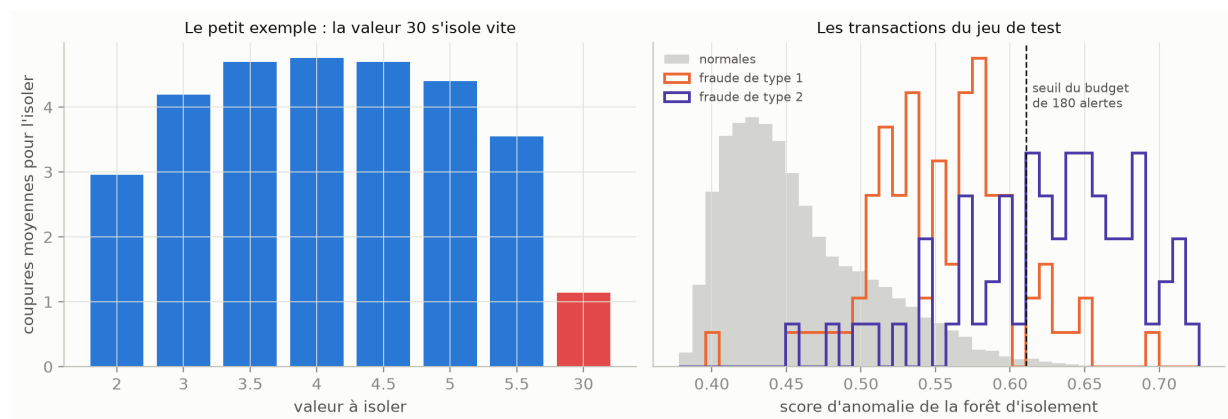


Figure 72: À gauche, le petit exemple de huit valeurs : nombre moyen de coupures nécessaires pour isoler chacune (la valeur 30, en rouge, s'isole en une coupure environ). À droite, la distribution des scores de la forêt d'isolement sur le jeu de test pour les commandes normales (gris) et les deux types de fraude ; la ligne pointillée marque le seuil qui garde 180 alertes.

La forêt d'isolement obtient une précision moyenne de **0,332** : un peu en dessous des voisins (0,444), au niveau de Mahalanobis (0,382) ou du LOF (0,306). Sa force est qu'elle est **rapide et sans réglage délicat**. Mais comme le montre la figure de droite, ses scores séparent bien les fraudes de **type 2** (dont le score moyen de 0,626 est bien au-dessus de celui des commandes normales, 0,459) et beaucoup moins bien les fraudes de **type 1** (0,555) : 68 % des fraudes de type 2 sont dans les 180 alertes, contre 12 % seulement de celles de type 1.

Pourquoi cette différence ? Ce n'est pas faute de valeurs extrêmes : la vérification ci-dessus montre que 78 % des fraudes de type 1 ont au moins une variable à plus de 3 écarts-types (contre 89 % pour le type 2 et 10 % pour les commandes normales), et qu'elles s'écartent en moyenne de plus de 1,5 écart-type sur 3 variables (3,6 pour le type 2). Les deux types sont donc bien loin de la normale, le type 2 un peu plus. Nous n'avons **pas démontré** pourquoi cela suffit à placer les fraudes de type 2 beaucoup plus haut dans le classement de la forêt. Une hypothèse, que nous n'avons pas testée, est que les variables caractéristiques du type 2 (appareil inconnu, adresse IP étrangère, rafale de commandes) sont rares et à valeurs discrètes, donc qu'une seule coupure suffit à les isoler, alors que celles du type 1 (montant, distance) ont des queues lourdes qui rendent de nombreuses commandes **normales** presque aussi faciles à isoler. Le seul fait établi est empirique : sur ces données, la forêt classe bien le type 2 et mal le type 1.

À retenir.

- La forêt d'isolement mesure la **facilité à isoler** un point par des coupures aléatoires : les anomalies, rares et différentes, s'isolent vite.
- Le score $s = 2^{-E[h(x)]/c(n)}$ vaut 0,5 pour un point moyen et tend vers 1 pour une anomalie ; $c(n) = 2H(n-1) - 2(n-1)/n$ est une normalisation approchée.
- Elle est **rapide**, sans distance ni hypothèse de loi, et passe à l'échelle grâce au sous-échantillonnage.
- La **contamination** est un choix opérationnel (le budget d'alertes), pas une vérité statistique : le seuil par défaut peut donner des milliers de fausses alertes.
- Sa qualité dépend de la nature des anomalies : ici elle retrouve bien les fraudes de type 2 et mal celles de type 1.

Pour s'entraîner. Cahier, chapitre 6 : application 6.4, exercices 6.8 et 6.9.

6.4 Autoencodeurs, et comparaison des méthodes

La forêt d'isolement mesure la facilité à *isoler* un point. Un **autoencodeur** suit une logique opposée : il apprend à *reproduire* ses entrées après les avoir comprimées, et l'on juge anormal ce qu'il ne parvient pas à reproduire. Cette section présente l'idée, son lien exact avec l'ACP, un exemple avec `scikit-learn`, et ses pièges. Elle se termine par la **comparaison honnête** de toutes les méthodes du chapitre.

6.4.1 L'idée : compresser, puis reconstruire

Un **autoencodeur** est un réseau de neurones formé de deux parties :

- un **encodeur** f qui transforme une commande $x \in \mathbb{R}^p$ en un petit résumé $z = f(x) \in \mathbb{R}^q$ avec $q < p$ (le **goulot d'étranglement**) ;
- un **décodeur** g qui reconstruit $\hat{x} = g(z)$, de même taille que x .

On l'entraîne à minimiser l'**erreur de reconstruction** $\sum_i \|x_i - g(f(x_i))\|^2$ sur des données, **sans aucune étiquette**. Comme le goulot est étroit, le réseau ne peut pas recopier ses entrées : il doit retenir ce qui est **typique** (les régularités de la plupart des commandes). Une commande normale se reconstruit bien, avec une petite erreur. Une commande qui ne suit pas ces régularités se reconstruit mal : son **erreur de reconstruction** est grande, et c'est notre **score d'anomalie**.

L'image du portraitiste. Un dessinateur à qui on montre un visage pendant trois secondes retient les traits habituels (deux yeux, un nez, une bouche) et dessine un visage moyen ressemblant. Si le visage comporte trois yeux, le dessin ne le reproduira pas : l'écart entre le visage et le dessin dénonce l'anomalie. Cela ne marche que si la mémoire du dessinateur est **limitée** : avec une mémoire parfaite, il recopierait aussi le troisième œil.

6.4.2 L'ACP est un autoencodeur linéaire

Si l'encodeur et le décodeur sont **linéaires**, trouver les meilleures matrices revient à chercher l'approximation de rang q du tableau de données la plus proche au sens des moindres carrés. Le **théorème d'Eckart-Young** (volume I, section 1.1.4 sur la décomposition en valeurs singulières) dit que cette meilleure approximation est la projection sur les q premières directions principales : c'est exactement l'ACP (volume II, section 3.1, et en particulier 3.1.7 sur la compression et la reconstruction). Un autoencodeur linéaire n'est donc rien d'autre qu'une ACP ; les autoencodeurs non linéaires la généralisent à des surfaces courbes.

Un exemple à la main. Reprenons les deux variables standardisées de corrélation $\rho = 0,9$ de la section 6.2.2. Les directions principales sont $(1, 1)/\sqrt{2}$ (variance $\lambda_1 = 1,9$) et $(1, -1)/\sqrt{2}$ (variance $\lambda_2 = 0,1$). Avec une seule composante ($q = 1$), on reconstruit chaque point par sa projection sur $(1, 1)/\sqrt{2}$.

- Le point $x = (2, 2)$ est **sur** cette direction : sa reconstruction est exacte, **erreur 0**.
- Le point $x = (2, -2)$ est **perpendiculaire** à elle : la projection est nulle, la reconstruction est l'origine, et l'erreur au carré vaut $\|x\|^2 = 8$. Sa coordonnée sur la seconde direction est $y_2 = (2 + 2)/\sqrt{2} = 2,83$, d'où $y_2^2 = 8$.

C'est le même verdict que celui de Mahalanobis (section 6.2.2, où les deux points avaient pour distances 4,21 et 80), et le lien est exact : $d_M^2 = \sum_j y_j^2 / \lambda_j$, soit pour le second point $8/0,1 = 80$. L'erreur de reconstruction de l'ACP, elle, ne garde que les composantes **écartées, sans les diviser par leur variance** (8 au lieu de 80). La distance de Mahalanobis les pondère par l'inverse de leur variance : elle est plus sensible aux petites directions.

Appliquons cela à nos transactions, en reconstruisant avec $k = 1$ à 9 composantes. Le nombre de composantes joue le rôle de la taille du goulot ; nous le **choisissons sur le jeu de validation**, et nous regardons ensuite ce que donne le jeu de test :

Le résultat est contre-intuitif : **la meilleure ACP est celle qui garde une seule composante**, et la qualité s'effondre ensuite (sur le jeu de test, la précision moyenne passe de 0,39 pour une composante à 0,14 pour deux, puis à 0,02-0,06 pour trois ou plus, contre un plancher de 0,008). La validation choisit la même valeur, une composante, qui n'explique pourtant que **12,8 %** de la variance : nos dix variables sont peu corrélées entre elles, donc il y a peu de structure linéaire à compresser, et l'erreur de reconstruction avec une composante est presque la somme des carrés des dix variables standardisées, c'est-à-dire une distance au centre (proche du z-score classique de la section 6.2.1). Avec peu de composantes, l'erreur de reconstruction agrège beaucoup de directions : elle est grande dès que la commande s'écarte du schéma principal. Avec davantage de composantes, le modèle devient capable de reconstruire aussi les **directions dans lesquelles se trouvent les fraudes** : l'anomalie se « cache » dans le sous-espace appris. C'est le **piège central de la reconstruction** : *un modèle trop riche reconstruit aussi les anomalies*, et sa capacité doit donc être **contrainte**.

6.4.3 Un autoencodeur non linéaire avec scikit-learn

scikit-learn n'a pas de bibliothèque d'autoencodeurs, mais un réseau de neurones de régression (MLPRegressor) entraîné à prédire **ses propres entrées** en est un : couches cachées de tailles décroissantes puis croissantes, avec un goulot au milieu. Le réseau ci-dessous a trois couches cachées de 6, 3 et 6 neurones (un goulot de 3 pour 10 variables) :

```
from sklearn.neural_network import MLPRegressor

ae = MLPRegressor(hidden_layer_sizes=(6, 3, 6), activation="tanh", max_iter=100, random_state=0)
ae.fit(Z_app[reference], Z_app[reference])           # apprendre à reconstruire ses
↳ propres entrées
erreur = ((ae.predict(Z_test) - Z_test) ** 2).sum(axis=1) # erreur de reconstruction = score
↳ d'anomalie
```

Trois choix de méthode méritent attention :

- **Les données d'apprentissage sont contaminées.** Le réseau est entraîné sur des commandes qui contiennent environ 0,8 % de fraudes. Tant qu'elles sont rares, il les traite comme du bruit et ne les apprend pas ; si elles étaient nombreuses, il apprendrait à les reconstruire.
- **Les variables doivent être standardisées,** sinon les variables d'échelle la plus grande dominent l'erreur de reconstruction.
- **L'architecture est un réglage qu'on ne peut pas tester sur le jeu de test.** La tentation est grande d'essayer plusieurs tailles, de regarder laquelle donne la meilleure précision moyenne sur le jeu de test, et de la présenter comme résultat. C'est exactement la faute de méthode que condamne le chapitre 1 (section 1.4) : le jeu de test aurait servi à choisir, donc il ne mesurerait plus rien. Nous utilisons donc le **jeu de validation** défini plus haut.

Nous comparons cinq architectures, chacune entraînée avec **quatre graines aléatoires** (le résultat d'un réseau dépend de son initialisation), sur l'échantillon de référence. Pour chaque graine, l'erreur de reconstruction est mise à l'échelle avec la moyenne et l'écart-type de l'erreur **sur la validation** (aucune information de test), puis on moyenne les quatre scores : c'est un petit **comité** de réseaux.

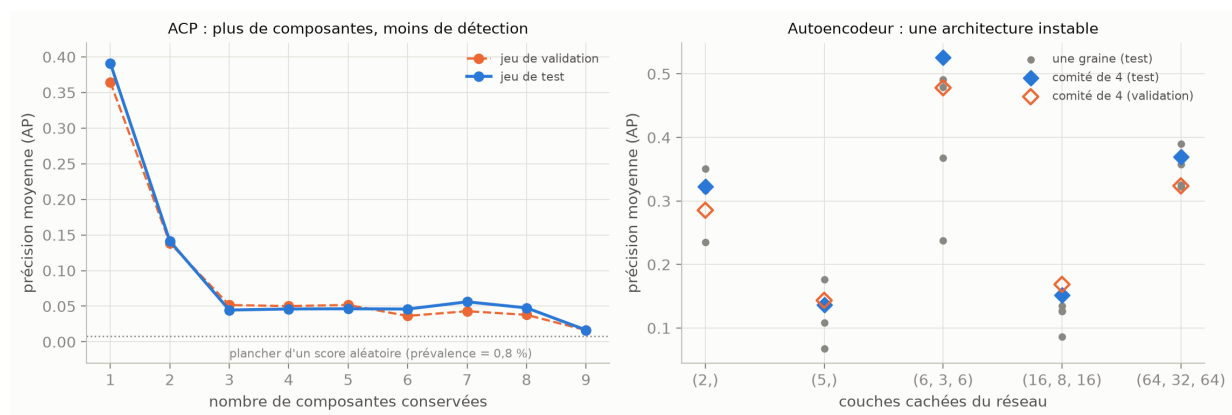


Figure 73: À gauche : précision moyenne de l'ACP selon le nombre de composantes conservées, sur le jeu de validation (orange, pointillé) et le jeu de test (bleu). À droite : précision moyenne de cinq architectures d'autoencodeur : les petits points gris sont les quatre graines individuelles (jeu de test), le losange bleu le comité de quatre réseaux (test), le losange orange vide le même comité sur le jeu de validation.

Ce que montre la figure de droite est instructif et **ne se résume pas en une règle simple** :

- **Les graines comptent.** Pour l'architecture (6, 3, 6), la précision moyenne des quatre réseaux va de 0,24 à 0,49 : le même modèle, entraîné quatre fois, donne des détecteurs de qualité

très différente. Un autoencodeur isolé est une loterie.

- **Le comité stabilise.** Moyenner les quatre scores donne mieux que **chacun** des quatre réseaux isolés : **0,53 sur le jeu de test** pour (6, 3, 6).
- **La capacité n'est pas monotone.** L'architecture (6, 3, 6) (comité à 0,53) fait mieux que (5,) et (16, 8, 16) (comités à 0,14 et 0,15), mais plus grand n'est pas mieux (le réseau (64, 32, 64) donne 0,37), et plus petit non plus ((2,) donne 0,32). Nous n'avons **pas d'explication simple** de ce non-monotonisme. Une hypothèse naturelle, un entraînement trop court (100 itérations), est **infirmée** : en autorisant 400 itérations aux deux architectures les moins bonnes, la précision moyenne ne change pas (cahier, application 6.5). Les réseaux convergent vers des solutions différentes selon leur initialisation et leur taille, et un réseau n'est pas bon ou mauvais *par principe*.
- **Le jeu de validation désigne la bonne architecture.** Celle qu'il choisit, (6, 3, 6), est celle qui est la meilleure sur le test : on peut donc rapporter son résultat comme une estimation honnête (aucune information du test n'a servi à choisir).

Ce dernier point demande tout de même une réserve : **un jeu de validation étiqueté** (ici 246 fraudes) est nécessaire pour choisir. Un détecteur non supervisé n'a pas besoin d'étiquettes pour *fonctionner*, mais il en a besoin pour *être réglé*. Quelques centaines d'exemples confirmés suffisent, et la gérante en dispose après quelques semaines.

Les pièges de l'autoencodeur.

- *Un goulot trop large* : le réseau recopie tout, y compris les anomalies (erreur faible partout).
- *Des données d'apprentissage trop contaminées* : le réseau apprend à reconstruire les fraudes.
- *Une instabilité d'une graine à l'autre* : toujours **moyenner plusieurs réseaux**.
- *Un score qui additionne des erreurs de variables d'échelles ou de natures différentes* (continues, binaires) : les variables binaires rares pèsent lourd quand elles prennent leur valeur rare. Standardiser ne règle pas tout.

6.4.4 La version PyTorch

Dans la pratique, on écrit plutôt les autoencodeurs avec une bibliothèque d'apprentissage profond, qui permet des architectures plus riches (convolutions pour les images, couches récurrentes pour les séquences), un entraînement par lots sur carte graphique, et un contrôle fin de l'optimisation (arrêt précoce, régularisation). Voici l'équivalent PyTorch du réseau précédent. **Ce code n'est pas exécuté dans ce livre** : PyTorch n'est pas installé dans l'environnement qui a produit les sorties.

```
import torch
from torch import nn

autoencodeur = nn.Sequential(
    nn.Linear(10, 6), nn.Tanh(), nn.Linear(6, 3), nn.Tanh(),      # encodeur : 10 -> 3
    nn.Linear(3, 6), nn.Tanh(), nn.Linear(6, 10))               # décodeur : 3 -> 10
optimiseur = torch.optim.Adam(autoencodeur.parameters(), lr=1e-3)
X_app_t = torch.tensor(Z_app[reference], dtype=torch.float32)
for epoque in range(100):
    optimiseur.zero_grad()
    perte = ((autoencodeur(X_app_t) - X_app_t) ** 2).sum(dim=1).mean() # erreur de reconstruction
    perte.backward(); optimiseur.step()
```

Non exécuté. Les résultats chiffrés de cette section proviennent exclusivement de ML-PRegressor ; le code PyTorch ci-dessus est donné à titre d'illustration.

6.4.5 Comparer honnêtement les méthodes

Nous avons maintenant toutes les méthodes. Pour ne pas se tromper soi-même, rappelons le protocole : toutes les méthodes non supervisées ont été ajustées **sans étiquette** sur le jeu d'apprentissage ; leurs réglages ont été **choisis sur le jeu de validation** (nombre de voisins, nombre de voisins du LOF, nombre de composantes de l'ACP, architecture de l'autoencodeur) ou **fixés à l'avance** sans réglage possible (z-scores, Mahalanobis, 200 arbres pour la forêt d'isolement) ; le jeu de test n'a servi qu'à les juger. La dernière ligne est une **combinaison** fixée à l'avance, sans aucun réglage : on classe les commandes par chacun des trois détecteurs de familles différentes (voisins, forêt d'isolement, autoencodeur) et on prend la **moyenne des rangs**.

	AUC	AP	précision à k	rappel type 1	rappel type 2
Gradient boosting (supervisé)	0.971	0.646	0.616	0.716	0.631
z-score classique	0.939	0.433	0.452	0.457	0.477
Mahalanobis	0.946	0.382	0.356	0.198	0.615
Mahalanobis robuste (MCD)	0.949	0.410	0.404	0.235	0.677
Plus proches voisins (k = 30)	0.959	0.460	0.432	0.259	0.692
LOF (k = 300)	0.944	0.492	0.493	0.519	0.492
Forêt d'isolement	0.937	0.332	0.342	0.123	0.677
ACP (erreur de reconstruction)	0.946	0.391	0.363	0.198	0.615
Autoencodeur (comité de 4)	0.962	0.526	0.514	0.519	0.677
Moyenne des rangs (3 familles)	0.958	0.438	0.404	0.259	0.662

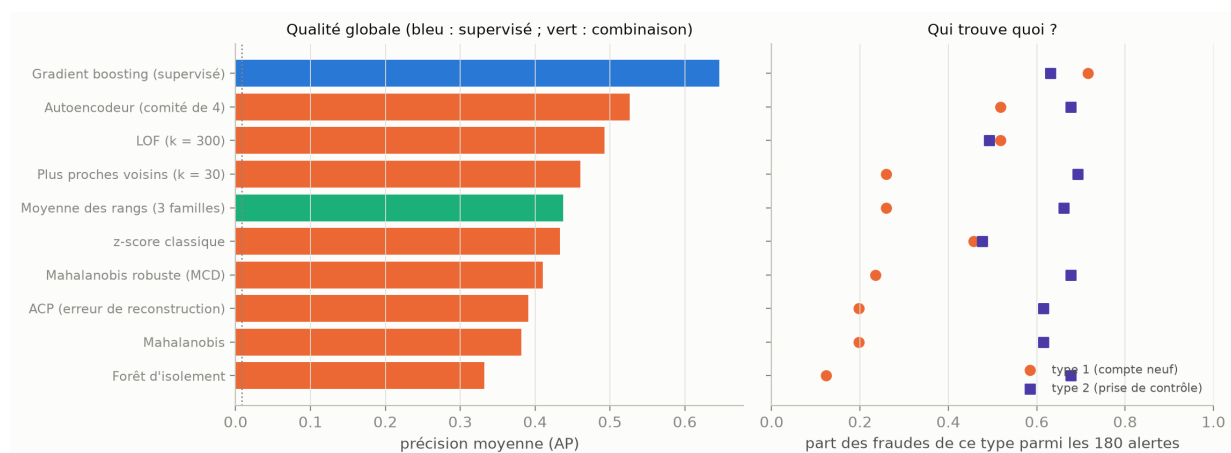


Figure 74: À gauche : précision moyenne de chaque méthode (la ligne pointillée est le plancher d'un score aléatoire). À droite : pour chaque méthode, la part des fraudes de type 1 (cercles orange) et de type 2 (carrés violets) retrouvées parmi les 180 alertes du budget.

Que lire dans ce tableau ?

1. **Le supervisé gagne quand on a des étiquettes** : précision moyenne de 0,65, contre 0,53 pour le meilleur détecteur non supervisé (le comité d'autoencodeurs) et 0,49 pour le LOF réglé. L'écart est le prix de ne pas connaître la fraude : il est modeste, et le détecteur supervisé aura du mal avec une fraude inédite.
2. **Les méthodes non supervisées surpassent largement le hasard** (plancher à 0,008) : une précision moyenne de 0,33 à 0,53, c'est de 41 à 65 fois mieux.
3. **Elles ne trouvent pas les mêmes fraudes**. Mahalanobis, les voisins, la forêt d'isolement et l'ACP retrouvent surtout le type 2 (de 0,62 à 0,69, contre 0,12 à 0,26 pour le type 1) ; le z-score classique les équilibre (0,46 et 0,48) ; les **deux meilleurs détecteurs non supervisés, le comité d'autoencodeurs (0,52 et 0,68) et le LOF réglé (0,52 et 0,49), retrouvent bien les deux types**.
4. **Les meilleurs détecteurs sont aussi les plus délicats à régler**. Le LOF vaut 0,11 avec 10

voisins et 0,49 avec 300 ; l'architecture (5,) donne 0,14 et (6, 3, 6) 0,53. Les méthodes simples (le z-score classique à 0,43, les voisins à 0,46) sont beaucoup moins sensibles à leurs réglages. La sophistication peut être payante, mais **à condition de disposer d'un jeu de validation pour régler**, et il vaut mieux **essayer d'abord les méthodes simples**.

5. **Une combinaison n'est pas magique.** La moyenne des rangs de trois familles (voisins, forêt d'isolement, autoencodeur) obtient une précision moyenne de 0,44 : **moins** que son meilleur membre (le comité d'autoencodeurs, 0,53). Les deux autres membres, qui retrouvent mal le type 1, tirent le classement moyen vers le bas. Une combinaison aide quand ses membres sont de qualité comparable et vraiment complémentaires ; elle ne remplace pas la mesure.

Les limites de cette comparaison. (i) Les données sont **simulées** : les fraudes y ont été fabriquées d'une certaine façon, et les classements pourraient changer sur des fraudes réelles. (ii) Un seul jeu de test, de 146 fraudes : les différences de quelques centièmes ne sont pas significatives (une autre graine pour la séparation changerait certains rangs). (iii) Les détecteurs ne sont comparés qu'au **même budget d'alertes** et à une répartition de coût simple. (iv) Aucun n'est évalué sur une dérive dans le temps, qui est le vrai défi de la fraude réelle.

En pratique, on retient de ce chapitre une démarche plus qu'une méthode :

- **commencer simple** (distance aux voisins ou Mahalanobis, un modèle supervisé de référence si l'on a des étiquettes) ;
- **évaluer sous déséquilibre** avec l'AP, le rappel au budget et le coût, jamais avec l'exactitude ;
- **ne pas régler sur le jeu de test** : garder un jeu de validation étiqueté, même petit ;
- **tester des combinaisons** de familles de détecteurs, sans présumer qu'elles aident (voir ci-dessus), et utiliser leurs scores comme variables d'entrée d'un modèle supervisé quand des étiquettes existent ;
- **surveiller dans le temps** : la fraude évolue, la qualité du détecteur aussi (un contrôle régulier sur les alertes confirmées en témoignage) ;
- se rappeler qu'un détecteur ne produit que des **pistes** : le dernier mot revient à une vérification humaine, dont le temps est le véritable budget.

À retenir.

- Un **autoencodeur** apprend à reconstruire les données normales à travers un goulot étroit ; l'**erreur de reconstruction** est le score d'anomalie. Linéaire, il est **équivalent à l'ACP**.
- Un modèle trop riche reconstruit aussi les anomalies : la **capacité doit être contrainte** (peu de composantes, goulot étroit), et le bon réglage se **choisit sur un jeu de validation**, jamais sur le test.
- Un réseau isolé est instable d'une graine à l'autre : on **moyenne un comité**.
- Chaque méthode trouve d'autres fraudes ; le supervisé est le meilleur quand les étiquettes existent ; une **combinaison** naïve ne bat pas forcément son meilleur membre.
- Les comparaisons sur un jeu simulé et un seul jeu de test sont des **indications**, pas des lois.

Pour s'entraîner. Cahier, chapitre 6 : applications 6.5 et 6.6, exercices 6.10 à 6.12.

Bilan du chapitre 6

Vous savez maintenant :

- **distinguer** anomalie ponctuelle, contextuelle et collective, et **ne pas confondre** anomalie (fait statistique) et fraude (fait métier) ;
- **expliquer** pourquoi l'apprentissage supervisé suffit rarement (déséquilibre extrême, délai des étiquettes, adversaire qui s'adapte, fraudes inédites, étiquettes biaisées) et quand il reste le meilleur choix ;
- **évaluer un détecteur sous déséquilibre extrême** : l'exactitude est trompeuse (99,19 % pour un détecteur inutile), la formule de Bayes explique pourquoi la précision est faible, la **courbe précision-rappel** et l'**AP** remplacent la ROC, et le **budget d'alertes** et le **coût** fixent le seuil ;
- **utiliser le z-score** et sa version **robuste** (médiane, MAD de constante 1,4826, piège du MAD nul), la **distance de Mahalanobis** (loi du χ^2 , masquage, MCD), la **distance aux plus proches voisins** et le **LOF** (densité locale) ;
- **comprendre la forêt d'isolement** : longueur de chemin, score $s = 2^{-E[h]/c(n)}$, sous-échantillonnage, et pourquoi la contamination est un choix opérationnel ;
- **construire un autoencodeur** avec `scikit-learn`, savoir qu'il est équivalent à l'ACP quand il est linéaire, que sa capacité doit être contrainte, qu'un réseau isolé est instable et qu'un comité de réseaux stabilise ;
- **comparer honnêtement** plusieurs détecteurs : un jeu de validation étiqueté pour régler, le jeu de test seulement pour juger, et se méfier des combinaisons naïves, qui ne battent pas forcément leur meilleur membre.

Le fil rouge du chapitre est le même que celui du volume : **la rigueur d'évaluation compte plus que la sophistication du modèle**. Les distances et les densités de la section 6.2 sont des idées anciennes qui rivalisent avec des méthodes récentes ; l'autoencodeur, le plus moderne, n'a été meilleur que parce que nous l'avons réglé sur un jeu de validation et moyenné sur plusieurs graines. Sur un problème où les erreurs coûtent de l'argent, la bonne question n'est jamais « quel est le meilleur algorithme ? », mais « *combien d'argent économise-t-on pour un budget de vérification donné, et comment le sait-on sans se tromper soi-même ?* ».

Pour s'entraîner. Cahier, chapitre 6 : applications 6.1 à 6.6 et exercices 6.1 à 6.12.

Le chapitre 7 (complémentaire, lui aussi) change de problème : au lieu de signaler ce qui est étrange, il s'agit de **recommander** à chaque client les produits qu'il aimera, à partir des achats de tous les clients.

Chapitre 7 : ⊕ Systèmes de recommandation

« Montrer à chacun les quelques produits, parmi des milliers, qu'il a une vraie chance d'aimer : voilà un problème de prédiction où la bonne réponse n'existe que dans l'avenir. »

Chapitre complémentaire. Ce chapitre est entièrement facultatif : les chapitres 1 à 5 se lisent et se comprennent sans lui. Il s'adresse à celles et ceux qui veulent voir comment les idées du volume (validation honnête, modèles de référence, factorisation, évaluation) se déclinent dans un problème qui n'est ni une classification ni une régression : **classer des produits pour chaque client**.

Quand la gérante ouvre la page d'accueil de sa boutique en ligne, elle aimerait que chaque visiteur voie en premier les produits qu'il est le plus susceptible d'acheter. Elle n'a pas de variable « client à retenir » à prédire, comme au chapitre 2 : elle a un **historique d'achats** et une question plus délicate, « parmi les 150 produits du catalogue, lesquels montrer à ce client-là, dans quel ordre ? ».

C'est le problème de la **recommandation**. Il est plus ouvert qu'il n'y paraît : on ne dispose que d'achats (jamais de « non, ce produit ne m'intéresse pas »), la plupart des cases du tableau clients × produits sont vides, les produits populaires écrasent les autres, et les recommandations changent elles-mêmes les achats futurs. Le volume a donné les outils pour s'y prendre : une démarche d'évaluation rigoureuse (chapitre 1), des modèles de référence à battre (section 1.4), de la régularisation (section 2.1), une factorisation de matrice (la décomposition en valeurs singulières du volume I, section 1.1.4, et l'ACP du volume II, section 3.1).

Le chemin de ce chapitre

- **7.1 Données d'interaction et références simples** : ce que l'on observe vraiment, la matrice d'interactions, et deux premières méthodes qui ne demandent aucun apprentissage sophistiqué : la **popularité** et le **filtrage par contenu**.
- **7.2 Filtrage collaboratif** : « ceux qui ont acheté comme vous ont aussi acheté... ». Les méthodes de **voisinage**, entre clients et entre produits.
- **7.3 Factorisation matricielle** : résumer chaque client et chaque produit par quelques **facteurs latents**, appris en minimisant une erreur régularisée ; le cas des achats **implicites**.
- **7.4 Évaluation et démarrage à froid** : comment mesurer un classement, pourquoi un découpage aléatoire peut tromper, ce que valent les recommandations pour un **nouveau client** ou un **nouveau produit**, et pourquoi les recommandations modifient les données qui serviront à les améliorer.

Le fil rouge du chapitre. À chaque étape, la même question : *par rapport à quoi ?* Une recommandation « personnalisée » n'a d'intérêt que si elle fait mieux que montrer à tout le monde les produits les plus vendus. Nous verrons que, sur les données de la boutique, cette référence très simple est étonnamment difficile à battre.

Les données et le protocole

Nous utilisons deux fichiers de la boutique (simulés, comme tous ceux du volume) : `donnees/interactions.csv` (qui a acheté quoi, et combien de fois) et `donnees/produits_ml.csv` (catégorie, prix et caractère « nouveau » de chaque produit).

Le tableau contient **3 000 clients** et **150 produits** ; les clients ont passé en tout **27 687** achats distincts (une paire client-produit compte une fois, quel que soit le nombre d'exemplaires achetés). Sur les 450 000 cases possibles du tableau, seules **6,2 %** sont remplies : c'est ce que l'on appelle une matrice **creuse**.

Pour évaluer honnêtement les méthodes, nous appliquons dès maintenant la règle du chapitre 1 : trois jeux de données, séparés **avant** de regarder quoi que ce soit.

- On ne retient pour l'évaluation que les clients ayant au moins **5 achats** : ils sont **2 365**.
- Pour chacun, on met de côté au hasard environ **25 %** de ses achats : ce sont les achats du **jeu de test**, que personne ne regardera avant la fin (**6 523** achats au total).
- Parmi les achats restants, on met de côté environ **20 %** : c'est le **jeu de validation** (**3 987** achats), qui servira à choisir les hyperparamètres.
- Tout le reste, **17 177** achats, forme le **jeu d'entraînement**. Les clients qui ont moins de 5 achats restent dans l'entraînement, mais ne sont pas évalués.

Une méthode reçoit donc les achats d'entraînement, produit pour chaque client évalué un **classement** des produits qu'il n'a pas encore achetés, et on regarde dans quelle mesure les achats retirés apparaissent en tête de liste. Les métriques précises sont définies en 7.4.1 ; en attendant, reprenez l'idée : plus les achats cachés remontent haut dans le classement, meilleur est le modèle.

Une limite à connaître dès le départ. Ces données ne contiennent **pas de dates**. Nous ne pouvons donc pas découper « le passé » et « l'avenir », comme on le ferait dans un vrai projet, et nous retirons des achats au hasard, client par client. Nous verrons en 7.4.3, sur une simulation où le temps existe, à quel point ce choix peut flatter les résultats.

Pour s'entraîner. Cahier, chapitre 7 : l'application 7.1 reconstruit la matrice d'interactions, la popularité et le filtrage par contenu pas à pas, et la préparation du cahier présente le protocole d'évaluation.

7.1 Données d'interaction et références simples

Avant de chercher à faire du « sur mesure », il faut comprendre ce que contiennent les données d'une boutique, et se donner des **références** : des méthodes si simples que tout modèle sophistiqué devra les battre pour justifier son existence. Cette section pose le vocabulaire, la matrice sur laquelle tout le chapitre repose, et deux premières méthodes : la popularité et le filtrage par contenu.

7.1.1 Ce que l'on recommande, et à qui

On appelle **utilisateurs** les clients à qui l'on recommande, et **articles** (*items*) ce que l'on recommande : ici, des produits. Un système de recommandation peut poursuivre deux tâches différentes, que l'on confond souvent.

- **Prédire une note** : « quelle note ce client donnerait-il à ce produit ? ». C'est un problème de régression sur une grandeur observée de temps en temps.
- **Classer** : « quels sont les dix produits à afficher en premier pour ce client ? ». Ce qui compte n'est pas la valeur précise d'un score, mais l'**ordre** : un produit placé en dixième position et un autre en première ne rapportent pas la même chose.

La seconde tâche est celle de la gérante. Une note de 3,9 plutôt que 4,1 n'a aucune importance si les deux produits sont bien classés l'un par rapport à l'autre, alors qu'une liste qui oublie le produit que le client aurait acheté est un échec. Le chapitre est donc organisé autour du **classement** ; la prédiction de notes n'apparaît qu'au cahier.

Le but commercial n'est jamais un score : c'est une conversion plus élevée, un panier plus grand, ou la **découverte** de produits que le client n'aurait pas trouvés seul. Nous verrons en 7.4 que ces objectifs ne se mesurent pas tous avec la même métrique.

7.1.2 Retours explicites et implicites

Les signaux dont on dispose sont de deux natures.

- Un retour **explicite** est une opinion exprimée : une note de 1 à 5, un « j'aime ». Il est précieux, mais rare : dans les données de la boutique, seulement un achat sur quatre est accompagné d'une note.
- Un retour **implicite** est un comportement : un achat, un clic, une page consultée. Il est abondant, mais ambigu : acheter un produit ne prouve pas qu'on l'a aimé (c'était peut-être un cadeau), et ne pas l'acheter ne prouve pas qu'on ne l'aimerait pas.

L'absence n'est pas un « non ». Dans la matrice d'achats, une case vide signifie « le client n'a pas acheté ce produit », ce qui mélange trois situations : il ne l'a jamais vu, il l'a vu et n'en veut pas, ou il l'achètera demain. Traiter les cases vides comme des refus est l'erreur la plus fréquente. Elle a deux conséquences que nous retrouverons : en 7.3, on donne aux cases vides un **poids faible** plutôt qu'un poids égal à celui des achats ; en 7.4, on se rappelle qu'un produit recommandé que le client n'a pas acheté **n'est pas forcément une erreur** : l'évaluation sur achats cachés sous-estime la valeur réelle d'un bon classement.

Dans les données de la boutique, la colonne `nb_achats` compte les exemplaires achetés : **un achat sur trois environ** (33 %) concerne plus d'un exemplaire. Nous l'ignorerons dans la suite (un client « a acheté » ou « n'a pas acheté » le produit), ce qui revient à utiliser un signal binaire ; la fin de l'application 7.3 du cahier propose d'exploiter la quantité comme mesure de confiance.

7.1.3 La matrice d'interactions

Notons R la matrice dont la ligne u correspond au client u , la colonne i au produit i , et dont l'entrée vaut $R_{ui} = 1$ si le client a acheté le produit, 0 sinon. Voici un exemple minuscule, que nous garderons pour tous les calculs à la main de la section : 5 clients et 6 produits.

Client	P1	P2	P3	P4	P5	P6
U1	1	1	0	1	0	0

Client	P1	P2	P3	P4	P5	P6
U2	1	0	1	1	0	0
U3	0	1	1	1	1	0
U4	1	1	0	0	0	1
U5	1	0	1	1	1	0

Lisez la matrice par lignes (ce que chaque client a acheté) ou par colonnes (qui a acheté chaque produit). Le client U1 a acheté P1, P2 et P4. Le produit P4 a été acheté par U1, U2, U3 et U5. Ici plus de la moitié des cases sont remplies (17 sur 30) ; dans la réalité, ce n'est presque jamais le cas.

Sur les données de la boutique, la même matrice a 3 000 lignes et 150 colonnes. On ne la stocke jamais comme un grand tableau plein de zéros, mais en **format creux** : on ne garde que les positions des cases remplies. Le fichier d'achats est justement un tableau « une ligne par achat », que l'on convertit ainsi :

```
from scipy.sparse import csr_matrix

R = csr_matrix((np.ones(len(inter)), (inter.id_client - 1, inter.id_produit - 1)), shape=(3000, 150))
print(R.shape, R.nnz)  # nombre de cases remplies
```

```
(3000, 150) 27687
```

Deux régularités frappent immédiatement quand on regarde les marges de la matrice.

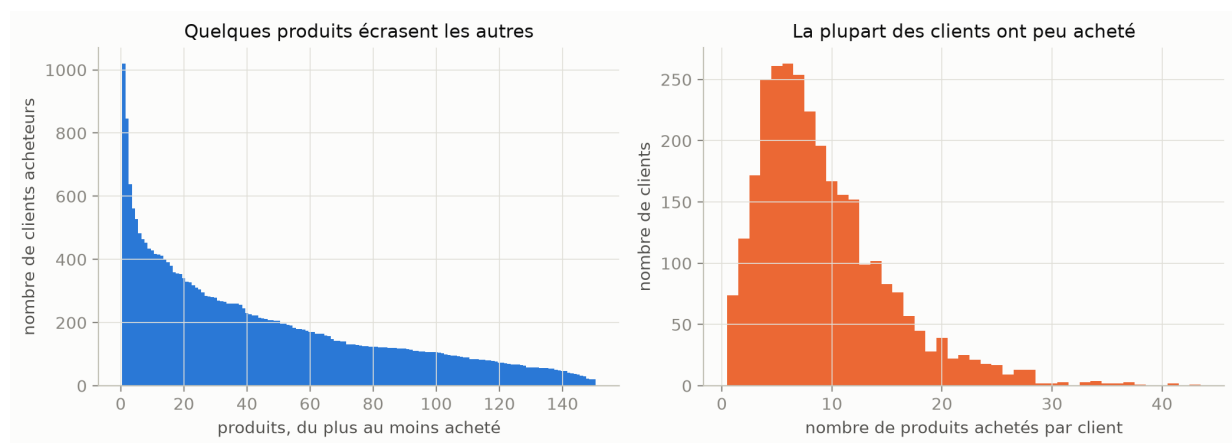


Figure 75: À gauche : nombre de clients ayant acheté chaque produit, du plus au moins acheté (le plus acheté l'a été par plus de mille clients, le moins acheté par une vingtaine). À droite : nombre de produits distincts achetés par client.

Le produit le plus acheté l'a été par **1 020** clients, le moins acheté par **20** : un rapport de plus de cinquante entre les deux. Les dix premiers produits concentrent à eux seuls **21 %** des achats. Du côté des clients, la médiane est de **8** produits achetés, avec une queue longue (au plus **43**). Cette asymétrie est typique : on parle de **longue traîne**. Elle a deux conséquences : un client a peu d'historique pour apprendre ses goûts, et un produit peu acheté a peu de données pour qu'on le recommande à bon escient.

7.1.4 La similarité cosinus

Presque toutes les méthodes de cette section et de la suivante reposent sur une même question : **à quel point deux vecteurs se ressemblent-ils ?** Deux clients sont proches s'ils ont acheté les mêmes produits ; deux produits sont proches s'ils ont été achetés par les mêmes clients. Il faut une mesure.

La **similarité cosinus** entre deux vecteurs a et b est le cosinus de l'angle qu'ils forment :

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \|b\|} = \frac{\sum_k a_k b_k}{\sqrt{\sum_k a_k^2} \sqrt{\sum_k b_k^2}}.$$

Elle vaut 1 si les deux vecteurs pointent dans la même direction, 0 s'ils sont orthogonaux. Pour des vecteurs d'achats (des 0 et des 1), le calcul se simplifie : si n_i et n_j sont les nombres de clients ayant acheté les produits i et j , et n_{ij} le nombre de ceux qui ont acheté **les deux**,

$$\cos(i, j) = \frac{n_{ij}}{\sqrt{n_i n_j}}.$$

Pourquoi cette formule. Le produit scalaire de deux vecteurs binaires compte les positions où les deux valent 1, soit n_{ij} . La norme au carré d'un vecteur binaire est son nombre de 1, soit n_i . Le numérateur mesure les achats communs ; le dénominateur, $\sqrt{n_i n_j}$, est la moyenne géométrique des tailles : il **normalise** pour qu'un produit très vendu ne soit pas automatiquement « proche de tout ».

À la main. Reprenons le petit tableau. Les produits P3 et P5 ont été achetés respectivement par $n_3 = 3$ clients (U2, U3, U5) et $n_5 = 2$ clients (U3, U5), dont $n_{35} = 2$ en commun. Donc

$$\cos(P3, P5) = \frac{2}{\sqrt{3 \times 2}} = \frac{2}{\sqrt{6}} \approx 0,816.$$

À l'inverse, P5 et P6 n'ont aucun acheteur commun : $n_{56} = 0$ et la similarité vaut 0.

Cosinus, Jaccard, corrélation. On rencontre aussi l'indice de **Jaccard** $n_{ij}/(n_i + n_j - n_{ij})$, qui compte la part d'acheteurs communs parmi tous les acheteurs, et la **corrélation** de Pearson, qui centre les vecteurs. Pour des données binaires et creuses, le cosinus est le choix courant : il ignore les zéros communs (deux produits qu'aucun des deux clients n'a achetés ne se « ressemblent » pas), ce que la corrélation ne fait pas.

7.1.5 La référence : la popularité

La méthode la plus simple consiste à recommander à tout le monde **les produits les plus achetés**, en retirant à chaque client ceux qu'il a déjà. Elle n'est pas personnalisée (deux clients ayant acheté la même chose reçoivent la même liste) et ne demande aucun apprentissage : un simple comptage.

À la main. Dans le petit tableau, les nombres d'acheteurs sont (4, 3, 3, 4, 2, 1) pour P1 à P6. Le client U1 a acheté P1, P2 et P4 ; il reste P3 (3 acheteurs), P5 (2) et P6 (1). La popularité recommande donc P3, puis P5, puis P6, dans cet ordre.

Pourquoi parler de référence ? Parce que, dans presque tous les catalogues, les produits les plus vendus le sont **pour une raison** : ils plaisent à beaucoup de monde. Un modèle qui n'arrive pas à faire mieux que de montrer ces produits n'a rien appris de personnel sur le client.

Mesurons-la sur les données de la boutique, avec la méthode d'évaluation décrite plus haut : on recommande **10 produits** à chaque client évalué, et on regarde quelle part de ses achats cachés (jeu de validation) figure parmi ces 10. Cette part s'appelle le **rappel@10**. Comme point de repère, une liste **tirée au hasard** donne les résultats suivants.

	précision@10	rappel@10	MAP@10	NDCG@10	au moins un achat retrouvé
au hasard	0.012	0.071	0.024	0.040	0.119
popularité	0.039	0.241	0.094	0.143	0.352
contenu	0.020	0.112	0.033	0.059	0.184

Une liste tirée au hasard retrouve **7,1 %** des achats cachés dans ses dix produits ; la popularité en retrouve **24,1 %**, soit **plus de trois fois plus**. Autrement dit, sur ce catalogue, une grande partie de l'information utile tient dans un simple comptage. Ce que la personnalisation pourra apporter s'ajoute à ce socle.

7.1.6 Le filtrage par contenu

Le **filtrage par contenu** recommande à un client des produits qui **ressemblent à ceux qu'il a déjà achetés**, en s'appuyant sur les caractéristiques des produits (catégorie, prix, description...) et non sur le comportement des autres clients.

La construction se fait en trois temps. On décrit chaque produit par un vecteur de caractéristiques ; on résume chaque client par un **profil**, la moyenne des vecteurs de ses produits ; on classe les produits par ressemblance (cosinus) avec ce profil.

À la main, avec une mise en garde. Décrivons trois produits par un vecteur (catégorie A, catégorie B, prix), le prix étant exprimé en dizaines d'euros : $P1 = (1, 0, 2)$, $P2 = (1, 0, 3)$ et deux candidats, $P3 = (0, 1, 2,5)$ et $P4 = (1, 0, 2,5)$. Un client a acheté $P1$ et $P2$; son profil est la moyenne, $(1, 0, 2,5)$. Le candidat $P4$ (même catégorie, même prix moyen) a un cosinus de 1, ce qui est rassurant. Mais le candidat $P3$, d'une **autre catégorie**, obtient

$$\cos = \frac{0 \times 1 + 1 \times 0 + 2,5 \times 2,5}{\sqrt{1 + 6,25} \sqrt{1 + 6,25}} = \frac{6,25}{7,25} \approx 0,862.$$

Le résultat est presque aussi élevé que pour le bon candidat : le prix, qui n'est pas à la même échelle que les indicatrices de catégorie, **domine le calcul**.

Les échelles comptent. Le cosinus (comme toute distance) est sensible à l'échelle des variables. Avant de comparer des produits, il faut **mettre les caractéristiques à la même échelle** : ici, centrer et réduire le logarithme du prix. C'est la même idée que la standardisation du chapitre 4 (section 4.1) et de l'ACP (volume II, section 3.1).

Sur les données de la boutique, les caractéristiques disponibles sont pauvres : une catégorie parmi quatre, et un prix. Le résultat, **11,2 %** de rappel@10, se situe entre le hasard (7,1 %) et la popularité (24,1 %) : le contenu apporte un signal, mais bien moins que le comportement collectif. Le filtrage par contenu n'est pourtant pas inutile. Il fonctionne **sans historique d'autres clients**, ce qui en fait la méthode de choix pour un **produit tout neuf** (section 7.4.5), et ses recommandations sont faciles à expliquer (« parce que vous avez acheté un produit de la même catégorie »).

À retenir (7.1).

- Recommander, c'est **classer** des produits pour chaque client ; le résultat se juge sur l'ordre, pas sur un score.
- Les achats sont un retour **implicite** : une case vide n'est pas un refus. La matrice clients × produits est **creuse** et sa « longue traîne » limite l'information disponible.

- La **similarité cosinus** mesure la ressemblance de deux vecteurs ; pour des achats binaires, $\cos(i, j) = n_{ij} / \sqrt{n_i n_j}$. Les caractéristiques doivent être à la même échelle.
- Deux références à battre : la **popularité** (très solide) et le **contenu** (utile surtout au démarrage à froid).

Pour s'entraîner. Cahier, chapitre 7 : application 7.1 (matrice d'interactions, popularité et contenu à la main), exercices 7.1 à 7.3.

7.2 Filtrage collaboratif : les méthodes de voisinage

Le filtrage par contenu regarde les **produits**. Le **filtrage collaboratif** regarde le **comportement collectif** : il recommande à un client ce qu'ont acheté des clients qui lui ressemblent, ou des produits qui ressemblent à ce qu'il a déjà acheté, sans jamais avoir besoin de savoir ce qu'est un produit. Cette section présente les méthodes les plus directes, dites de **voisinage**, qui n'ont pas de paramètres à ajuster au sens du chapitre 2 : tout le travail se joue dans la définition de la ressemblance et dans le nombre de voisins retenus.

7.2.1 L'intelligence collective

L'idée tient en une phrase : *les gens qui ont aimé les mêmes choses dans le passé aimeront probablement les mêmes choses dans l'avenir*. Elle se décline de deux façons symétriques.

- Les **voisins de clients** (*user-user*) : pour recommander au client u , on cherche les k clients dont l'historique ressemble le plus au sien, et on lui propose ce qu'ils ont acheté et qu'il n'a pas encore acheté.
- Les **voisins de produits** (*item-item*) : pour chaque produit candidat, on mesure sa ressemblance avec les produits que u a déjà achetés. Deux produits se ressemblent s'ils sont **achetés par les mêmes clients**.

Dans les deux cas, la ressemblance est la similarité cosinus de la section 7.1.4, appliquée soit aux lignes de la matrice R (les clients), soit à ses colonnes (les produits).

7.2.2 Les voisins de clients

Retours implicites. Pour des achats binaires, on retient pour chaque client u l'ensemble $N_k(u)$ de ses k voisins les plus proches (au sens du cosinus), puis on attribue à chaque produit i le score

$$\hat{s}_{ui} = \sum_{v \in N_k(u)} \text{sim}(u, v) R_{vi}.$$

Un produit est bien classé s'il a été acheté par beaucoup de voisins, d'autant plus proches de u que leur similarité est élevée. On ne classe que les produits que u n'a pas déjà achetés.

Retours explicites. Avec des notes, une difficulté apparaît : certains clients notent toujours haut, d'autres toujours bas. On corrige ce biais en **centrant** les notes sur la moyenne de chaque client, $\bar{r}_u$, et on prédit

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} \text{sim}(u, v) (r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |\text{sim}(u, v)|}.$$

La note prédite est la moyenne du client plus la moyenne **pondérée** des écarts que ses voisins ont à leur propre moyenne.

À la main. Quatre clients ont noté quatre produits (A à D) ; nous voulons prédire la note de u_1 pour le produit D.

Client	A	B	C	D	Moyenne $\bar{r}$
u_1 (cible)	5	3	4	?	4,00
u_2	4	2	5	4	3,75
u_3	2	5	1	2	2,50
u_4	5	4	4	5	4,50

On mesure la ressemblance sur les produits notés par les deux clients (A, B, C), après centrage. Le vecteur centré de u_1 est $(1, -1, 0)$, de norme $\sqrt{2} \approx 1,414$. Pour u_2 : $(0,25, -1,75, 1,25)$, de norme $\approx 2,165$, et le produit scalaire avec u_1 vaut $0,25 + 1,75 = 2$, d'où $\text{sim}(u_1, u_2) = \frac{2}{1,414 \times 2,165} \approx 0,653$. De même $\text{sim}(u_1, u_3) \approx -0,717$ (goûts opposés) et $\text{sim}(u_1, u_4) \approx 0,816$.

Chacun a noté le produit D ; l'écart de leur note à leur moyenne vaut $+0,25$ pour u_2 , $-0,5$ pour u_3 et $+0,5$ pour u_4 . La prédiction est donc

$$\hat{r}_{1D} = 4 + \frac{0,653 \times 0,25 + (-0,717) \times (-0,5) + 0,816 \times 0,5}{0,653 + 0,717 + 0,816} = 4 + \frac{0,930}{2,187} \approx 4,43.$$

Remarquez le rôle de u_3 : ses goûts sont opposés à ceux de u_1 (similarité négative) et il a mal noté D ; le signe négatif de la similarité transforme cette mauvaise note en un argument **en faveur** de D. En pratique on écarte souvent les voisins de similarité négative, car cette inférence (« il n'aime pas ce que j'aime, donc ce qu'il déteste me plaira ») est fragile.

7.2.3 Les voisins de produits

L'approche symétrique raisonne sur les **colonnes**. Notons I_u l'ensemble des produits achetés par u . Le score d'un produit candidat j est la somme de ses ressemblances avec les produits de I_u :

$$\hat{s}_{uj} = \sum_{i \in I_u} \text{sim}(i, j).$$

On peut ne retenir, pour chaque produit j , que ses k produits les plus proches (ses **voisins**) ; les autres similarités sont mises à zéro. Matriciellement, en notant S la matrice des similarités entre produits (diagonale nulle), tout le calcul est un produit de matrices : $\hat{S} = R S$.

À la main. Reprenons le petit tableau de 7.1.3 (5 clients, 6 produits) et calculons les similarités nécessaires avec $\cos(i, j) = n_{ij} / \sqrt{n_i n_j}$. Les nombres d'acheteurs sont $(4, 3, 3, 4, 2, 1)$. Le client $U1$, qui a acheté P1, P2 et P4, a trois produits candidats :

- **P3** : $\cos(P3, P1) = \frac{2}{\sqrt{12}} = 0,577$, $\cos(P3, P2) = \frac{1}{\sqrt{9}} = 0,333$, $\cos(P3, P4) = \frac{3}{\sqrt{12}} = 0,866$.
Score : 1,777.
- **P5** : $\frac{1}{\sqrt{8}} = 0,354$, $\frac{1}{\sqrt{6}} = 0,408$, $\frac{2}{\sqrt{8}} = 0,707$. Score : 1,469.
- **P6** : $\frac{1}{\sqrt{4}} = 0,5$, $\frac{1}{\sqrt{3}} = 0,577$, 0. Score : 1,077.

Le classement est P3, P5, P6. Ici, il coïncide avec celui de la popularité ; la personnalisation ne devient visible que sur des historiques plus variés, comme ceux de la boutique.

Pourquoi on préfère souvent les voisins de produits. Dans une boutique, le catalogue (150 produits) est bien plus petit et plus **stable** que la clientèle (3 000 clients, qui changent d'une semaine à l'autre) : la matrice de similarités entre produits est petite et évolue lentement, on peut donc la calculer à l'avance. Un client n'a de plus que quelques achats, ce qui rend la mesure de sa ressemblance avec les autres clients bruitée, alors que la ressemblance de deux produits s'appuie sur tous les clients qui les ont achetés. Enfin, les recommandations sont faciles à expliquer : « parce que vous avez acheté P1 ».

Voici ce calcul sur les données de la boutique. Une seule fonction de bibliothèque suffit pour la similarité, un produit de matrices pour les scores :

```
from sklearn.metrics.pairwise import cosine_similarity

S = cosine_similarity(R_train.T)      # similarité entre produits (colonnes de R)
np.fill_diagonal(S, 0)
scores = R_train @ S                 # score du produit j pour le client u : somme des similarités
```

7.2.4 Rétrécissement et choix du voisinage

Une similarité calculée sur peu de données est peu fiable. Imaginez deux paires de produits : la paire *a* a un cosinus de 0,8 obtenu sur seulement $n_{ij} = 2$ clients communs ; la paire *b* a un cosinus de 0,6 obtenu sur 40 clients communs. Laquelle croire ? Plutôt la seconde, malgré sa valeur plus faible, parce que 0,8 sur deux clients peut être une coïncidence.

Le **rétrécissement** (*shrinkage*) traduit cette intuition : on multiplie la similarité par un facteur qui tend vers 0 quand le nombre de co-achats est faible,

$$\text{sim}'(i, j) = \text{sim}(i, j) \times \frac{n_{ij}}{n_{ij} + \lambda},$$

où $\lambda > 0$ est un paramètre de prudence. Avec $\lambda = 10$, la paire *a* devient $0,8 \times \frac{2}{12} \approx 0,133$ et la paire *b* devient $0,6 \times \frac{40}{50} = 0,48$: leur ordre s'**inverse**. C'est le même principe que la régularisation du chapitre 2 (section 2.1) : on tire vers zéro les estimations les moins fiables.

Il reste deux réglages : le **nombre de voisins** *k* (trop petit, on gaspille de l'information ; trop grand, on dilue le signal avec des voisins peu ressemblants) et le coefficient λ . Ce sont des hyperparamètres : on les choisit sur le **jeu de validation**, jamais sur le jeu de test (chapitre 1, section 1.1 et section 1.5).

7.2.5 Premiers résultats et limites

La figure et la grille donnent plusieurs enseignements.

- **La popularité est battue, mais de peu.** Elle obtient un NDCG@10 de 0,143 (rappel@10 de 24,1 %). Les voisins de produits, bien réglés, atteignent 0,158 (rappel de 25,8 %) et les voisins de clients 0,165 (rappel de 27,2 %).
- **Un voisinage trop étroit fait perdre.** Avec 5 produits voisins seulement, les voisins de produits tombent à 0,141 : **en dessous de la popularité**. Avec 10 voisins de clients, on tombe à 0,099. Le meilleur réglage garde tous les produits (149 voisins) ou 300 clients voisins : avec un catalogue de 150 produits, restreindre le voisinage jette presque toute l'information.
- **Le rétrécissement aide quand le voisinage est étroit, pas quand il est large.** Avec $k = 5$ voisins de produits, passer de $\lambda = 0$ à $\lambda = 10$ fait passer le NDCG de 0,141 à 0,148 ; avec tous les voisins, il n'apporte plus rien (0,158 dans les deux cas). Les paires de produits ont en effet **peu de co-achats** sur l'entraînement (médiane de 3, et 84,5 % des paires en ont moins

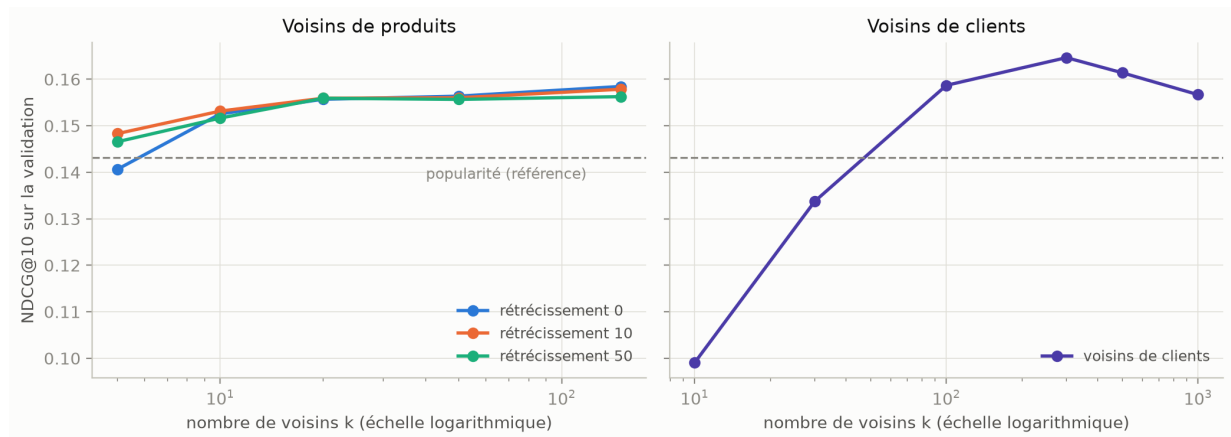


Figure 76: NDCG@10 sur le jeu de validation selon le nombre de voisins retenus. À gauche, voisins de produits pour trois valeurs du rétrécissement ; à droite, voisins de clients. La ligne en tirets est la popularité.

de 10) : chaque similarité est bruitée, mais le bruit se moyenne quand on additionne les similarités avec tous les produits du client.

- **Au-delà de 300 voisins de clients, on dilue** : le NDCG redescend à 0,157 pour 1 000 voisins.
- **L'écart entre les deux méthodes est petit** (0,006 de NDCG) : le jeu de validation compte 2 365 clients, ce qui ne permet pas de conclure laquelle est meilleure. Nous attendrons l'intervalle de confiance de 7.4.2.

Choisir un hyperparamètre sur la validation, puis le juger sur le test. Les chiffres ci-dessus sont ceux de la **validation**, utilisée pour choisir k et λ : ils sont légèrement optimistes (on a gardé le meilleur de plusieurs essais). La comparaison honnête de toutes les méthodes se fera en 7.4.2, sur le jeu de test, avec des modèles réentraînés sur l'entraînement **et** la validation.

Le coût de calcul. Les voisins de produits demandent la matrice des similarités entre produits : pour m produits, $m(m-1)/2$ paires, soit 11 175 ici, un calcul instantané. Les voisins de clients demandent les similarités entre clients : pour n clients, $n(n-1)/2$ paires, soit environ 4,5 millions ici (3 000 clients). Pour un site de plusieurs millions de clients, cette matrice ne tient plus en mémoire : on passe à des méthodes de **voisins approchés** ou, plus souvent, à la factorisation de la section 7.3. Le nombre d'opérations pour une matrice creuse dépend de la somme des carrés des tailles d'historique, $\sum_u d_u^2$ pour les voisins de produits, où d_u est le nombre d'achats du client u : un client très actif pèse plus lourd que tous les autres.

Les limites. Les méthodes de voisinage sont simples, interprétables et difficiles à battre, mais elles ne **généralisent** pas : deux produits ne sont liés que s'ils ont des clients communs, de sorte qu'un produit peu acheté, ou un client sans historique, restent sans recommandation. Elles suivent aussi la popularité (les produits très achetés ont des voisins nombreux) : nous mesurerons ce biais en 7.4.4. La factorisation matricielle répond à une partie de ces limites en résumant chaque client et chaque produit par quelques facteurs latents.

À retenir (7.2).

- Le filtrage collaboratif recommande à partir du **comportement collectif** : voisins de clients (lignes de R) ou voisins de produits (colonnes de R), avec la similarité cosinus.
- Pour les achats binaires, $\hat{s}_{uj} = \sum_{i \in I_u} \text{sim}(i, j)$; avec des notes, on **centre** par la moyenne du client.
- Le **rétrécissement** $n_{ij}/(n_{ij} + \lambda)$ protège des similarités calculées sur peu de co-achats ;

k et λ se choisissent sur la **validation**.

- Les voisins de produits sont plus stables et plus faciles à expliquer ; les deux approches ne recommandent rien sans co-achats.

Pour s'entraîner. Cahier, chapitre 7 : application 7.2 (voisins écrits à la main, rétrécissement), exercices 7.4 à 7.6.

7.3 Factorisation matricielle

Les méthodes de voisinage ne relient deux produits que s'ils ont des clients **en commun**. La factorisation matricielle fait mieux : elle résume chaque client et chaque produit par un petit nombre de **facteurs latents**, des grandeurs cachées que l'on n'observe pas mais que l'on apprend à partir des achats. Deux produits qui n'ont aucun acheteur commun peuvent alors avoir des facteurs proches, et c'est cela qui permet de généraliser. Ce principe est celui qui a dominé les systèmes de recommandation pendant une décennie ; il s'appuie sur les idées de la décomposition en valeurs singulières (volume I, section 1.1.4) et de la régularisation (section 2.1).

7.3.1 L'idée : des facteurs latents

Associons à chaque client u un vecteur $p_u \in \mathbb{R}^k$ et à chaque produit i un vecteur $q_i \in \mathbb{R}^k$, avec k petit (quelques unités à quelques dizaines). L'**affinité** prédite entre le client et le produit est leur produit scalaire :

$$\hat{r}_{ui} = p_u^\top q_i = \sum_{f=1}^k p_{uf} q_{if}.$$

On peut imaginer que chaque coordonnée f est un « goût » : le produit i possède plus ou moins ce trait (valeur q_{if}), le client y est plus ou moins sensible (valeur p_{yf}), et l'affinité additionne les rencontres entre les deux. Rien n'impose de nommer ces goûts : on les laisse émerger.

En notation matricielle, la matrice R (n clients $\times m$ produits) est approchée par un produit de deux matrices plus petites, $R \approx P Q^\top$, où P est $n \times k$ et Q est $m \times k$.

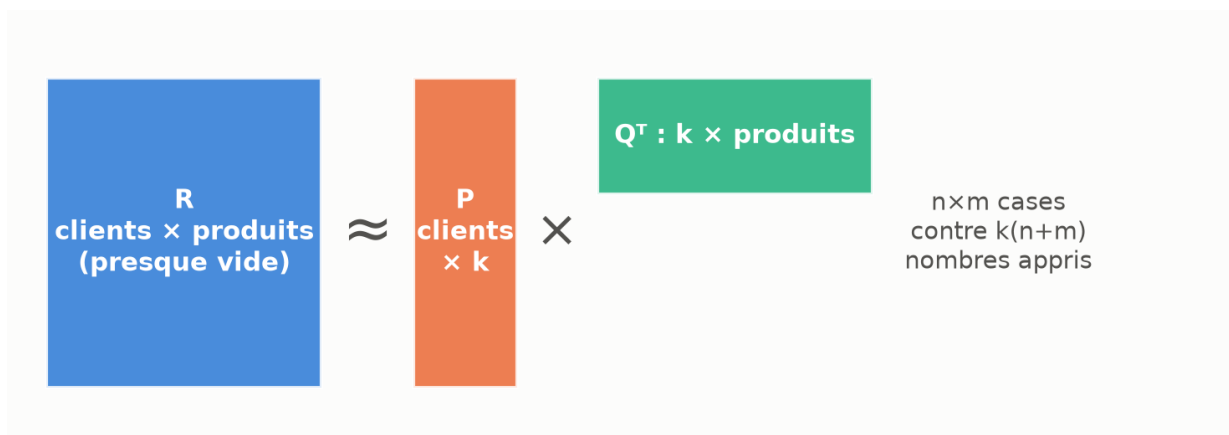


Figure 77: Factorisation matricielle : la grande matrice d'achats est approchée par le produit d'une matrice de facteurs de clients (une ligne par client) et d'une matrice de facteurs de produits (une colonne par produit).

Le gain d'économie est immédiat : au lieu des 450 000 cases de la matrice, on apprend $k(n + m)$ nombres, soit **18 900** pour $k = 6$. Surtout, ces nombres sont **partagés** : un produit, quel que soit le client, a le même vecteur q_i , appris à partir de tous ses acheteurs.

7.3.2 L'objectif : des moindres carrés régularisés

Pour apprendre P et Q à partir de retours **explicites** (des notes), on minimise l'erreur de reconstruction sur les cases **observées** uniquement, notées Ω , avec une pénalité qui retient les vecteurs de devenir démesurés :

$$\min_{P, Q} \sum_{(u,i) \in \Omega} (r_{ui} - p_u^\top q_i)^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right).$$

Les deux termes jouent les rôles que l'on connaît. Le premier mesure l'erreur sur ce que l'on a vu. Le second, la **régularisation de Ridge** du volume II (section 1.5), empêche le surapprentissage : sans lui, avec assez de facteurs, on reproduirait exactement les notes connues et on prédirait n'importe quoi ailleurs (nous en verrons un exemple en 7.3.4).

Pourquoi on n'utilise que les cases observées. Dans une matrice de notes, une case vide n'est pas un zéro : c'est une note inconnue. Pénaliser l'écart à zéro sur ces cases forcerait le modèle à prédire des notes nulles. L'objectif ne somme donc que sur Ω . Pour des achats (retours implicites), les cases vides ont un statut différent, que traite la section 7.3.5.

7.3.3 Deux algorithmes

Le problème n'est pas convexe en (P, Q) simultanément (c'est un produit de deux inconnues), mais il l'est **en chacun des deux séparément**. Deux stratégies en découlent.

La descente de gradient stochastique (SGD). On parcourt les notes connues une à une. Pour la note r_{ui} d'erreur $e_{ui} = r_{ui} - p_u^\top q_i$, on déplace les deux vecteurs un peu dans le sens qui réduit l'erreur, avec un pas γ :

$$p_u \leftarrow p_u + \gamma (e_{ui} q_i - \lambda p_u), \quad q_i \leftarrow q_i + \gamma (e_{ui} p_u - \lambda q_i).$$

C'est la descente de gradient du volume I (section 1.3.3), appliquée à un exemple à la fois.

Les moindres carrés alternés (ALS). On fixe Q : l'objectif devient, pour chaque client u , une régression de Ridge de ses notes sur les vecteurs des produits qu'il a notés. Notons Q_u la matrice dont les lignes sont les q_i des produits notés par u , et r_u le vecteur de ses notes ; l'annulation du gradient donne la solution **explicite**

$$p_u = (Q_u^\top Q_u + \lambda I)^{-1} Q_u^\top r_u.$$

Puis on fixe P et on résout de même pour chaque produit. On **alterne** jusqu'à stabilisation. Chaque étape ne peut que faire baisser l'objectif, donc la suite converge, vers un minimum local (qui dépend de l'initialisation).

Pourquoi la formule. Pour un client donné, l'objectif en p_u est $\|r_u - Q_u p_u\|^2 + \lambda \|p_u\|^2$: exactement une régression de Ridge, dont les équations normales sont $(Q_u^\top Q_u + \lambda I) p_u = Q_u^\top r_u$ (volume II, section 1.5.1). Les n problèmes (un par client) sont **indépendants** : on peut les résoudre en parallèle, ce qui fait la force de l'ALS pour les grands catalogues.

7.3.4 Une itération à la main

Prenons quatre clients, quatre produits, onze notes connues et cinq cases vides (symbole « ? »).

	P1	P2	P3	P4
Client 1	5	3	?	1
Client 2	4	?	?	1
Client 3	1	1	?	5
Client 4	?	1	5	4

Avec un seul facteur ($k = 1$), chaque client et chaque produit est un simple nombre, et la formule d'ALS devient une division. Prenons $\lambda = 1$ et initialisons tous les facteurs des produits à $q = (1, 1, 1, 1)$.

Étape 1 : les clients. Pour le client 1, qui a noté P1, P2 et P4 (notes 5, 3, 1) :

$$p_1 = \frac{5 \times 1 + 3 \times 1 + 1 \times 1}{(1^2 + 1^2 + 1^2) + 1} = \frac{9}{4} = 2,25.$$

De même $p_2 = \frac{4 + 1}{2 + 1} = 1,667$, $p_3 = \frac{1 + 1 + 5}{3 + 1} = 1,75$ et $p_4 = \frac{1 + 5 + 4}{3 + 1} = 2,5$. Le terme +1 au dénominateur est la régularisation : il tire chaque facteur vers zéro.

Étape 2 : les produits. On fixe maintenant $p = (2,25; 1,667; 1,75; 2,5)$. Pour le produit P1, noté par les clients 1, 2 et 3 (notes 5, 4, 1) :

$$q_1 = \frac{5 \times 2,25 + 4 \times 1,667 + 1 \times 1,75}{2,25^2 + 1,667^2 + 1,75^2 + 1} = \frac{19,67}{11,90} \approx 1,652.$$

On trouve de même $q_2 \approx 0,715$, $q_3 \approx 1,724$ (une seule note, celle du client 4) et $q_4 \approx 1,249$.

L'objectif baisse. Après l'étape 1, la somme des erreurs au carré sur les notes connues, plus la pénalité, vaut **59,17** ; après l'étape 2, **47,93**. Une seconde passe sur les clients donne $p \approx (2,01; 1,49; 1,48; 2,37)$ et un objectif de **46,91**. Après une centaine d'alternances, il se stabilise à **45,66**.

Un seul facteur ne suffit pas à représenter ce tableau (deux groupes de clients aux goûts opposés : les clients 1 et 2 notent P1 haut et P4 bas, les clients 3 et 4 l'inverse) ; que se passe-t-il avec deux facteurs et presque pas de régularisation ($\lambda = 0,1$) ? L'erreur sur les notes connues tombe à **0,06**, contre 1,42 avec un seul facteur : le modèle a **recopié** les onze notes. Mais l'une de ses prédictions pour les cases vides est absurde : il prédit **5,87** pour le client 3 et le produit P3, une note supérieure au maximum possible de 5. C'est le surapprentissage en miniature : onze observations pour seize paramètres. Régulariser (et valider !) n'est pas facultatif.

7.3.5 Retours implicites : donner aux cases vides un poids faible

Pour des achats, la situation est différente : la matrice ne contient que des 1 (achats) et des cases vides. Si l'on ne somme que sur les achats observés, le modèle apprend à tout prédire à 1 et n'apprend rien. Si l'on traite toutes les cases vides comme des 0 avec le même poids que les achats, on enseigne au modèle que les produits non achetés sont rejetés, ce qui est faux (7.1.2). La solution de **Hu, Koren et Volinsky** (2008) est un compromis : on garde **toutes** les cases, mais on pondère par la **confiance** que l'on a dans chaque observation.

Pour chaque paire (u, i) , notons $x_{ui} = 1$ si le client a acheté le produit et 0 sinon, et donnons-lui la confiance

$$c_{ui} = 1 + \alpha R_{ui}.$$

Une case vide a la confiance minimale 1 ; un achat a la confiance $1 + \alpha$, avec α de l'ordre de quelques unités. L'objectif devient

$$\min_{p,Q} \sum_{u,i} c_{ui}(x_{ui} - p_u^\top q_i)^2 + \lambda \left(\sum_u \|p_u\|^2 + \sum_i \|q_i\|^2 \right),$$

avec, pour un client, la solution ALS $p_u = (Q^\top C_u Q + \lambda I)^{-1} Q^\top C_u x_u$, où C_u est la matrice diagonale des confiances de u . Le calcul paraît lourd (il porte sur tous les produits), mais une astuce le rend rapide : $Q^\top C_u Q = Q^\top Q + \alpha Q_u^\top Q_u$, où $Q^\top Q$ est calculé une fois pour tous les clients et Q_u ne contient que les produits **achetés** par u .

À la main. Un seul facteur ($k = 1$), trois produits dont les facteurs sont $q = (1; 0,5; 2)$, un client qui a acheté P1 et P3, $\alpha = 4$ (confiance 5 sur les achats) et $\lambda = 1$. Le numérateur $\sum c x q = 5 \times 1 + 5 \times 2 = 15$; le dénominateur $\sum c q^2 + \lambda = 5 \times 1 + 1 \times 0,25 + 5 \times 4 + 1 = 26,25$. Donc $p_u = 15/26,25 \approx 0,571$: les cases vides pèsent dans le dénominateur, mais faiblement.

7.3.6 La SVD tronquée comme référence

Il existe une version plus simple, et qui sert de **référence** : la décomposition en valeurs singulières tronquée (volume I, section 1.1.4) de la matrice d'achats. Le théorème d'**Eckart-Young** affirme que la meilleure approximation de rang k de R au sens des moindres carrés est obtenue en gardant les k plus grandes valeurs singulières. Elle s'obtient en une ligne :

```
from sklearn.decomposition import TruncatedSVD

svd = TruncatedSVD(n_components=4, random_state=0)
Z = svd.fit_transform(R_train)          # facteurs des clients
scores = Z @ svd.components_           # reconstruction : score de chaque produit pour chaque client
```

La différence avec l'ALS pondéré est instructive : la SVD traite **toutes les cases vides comme des zéros de même poids que les achats**, et ne régularise pas. Elle approche donc surtout la popularité et se met à surapprendre dès que k grandit.

7.3.7 Choisir k et λ , et ce que valent les facteurs

Il reste à régler le nombre de facteurs k , la régularisation λ et la confiance α (fixée ici à 8). Comme pour les voisins, le choix se fait sur le jeu de **validation**.

Le tableau et la figure se lisent en quatre points.

- **Le meilleur réglage est $k = 6$ facteurs et $\lambda = 100$:** NDCG@10 de **0,168** (rappel@10 de 27,6 %), contre 0,165 pour les meilleurs voisins de clients, 0,158 pour les voisins de produits et 0,143 pour la popularité. Le gain sur la popularité est réel, mais modeste : 2,5 points de NDCG.
- **Plus de facteurs demandent plus de régularisation.** Avec $\lambda = 20$, le NDCG atteint 0,155 pour $k = 4$, puis baisse jusqu'à 0,125 pour $k = 12$: c'est le surapprentissage. Avec $\lambda = 100$, il reste stable à 0,168 de $k = 6$ à $k = 12$: la pénalité maintient en pratique la complexité effective du modèle, quel que soit k .
- **Une régularisation trop forte écrase tout.** Avec $\lambda = 150$, le NDCG vaut 0,143 pour **toutes** les valeurs de k : exactement celui de la popularité. Les facteurs de personnalisation sont réduits à zéro et il ne reste que la direction « produit populaire ». Le meilleur réglage se situe donc **près d'un précipice**, ce qui rappelle qu'on doit explorer une grille assez large pour voir l'autre côté de l'optimum (l'échelle de λ dépend aussi de α et de la taille des données).

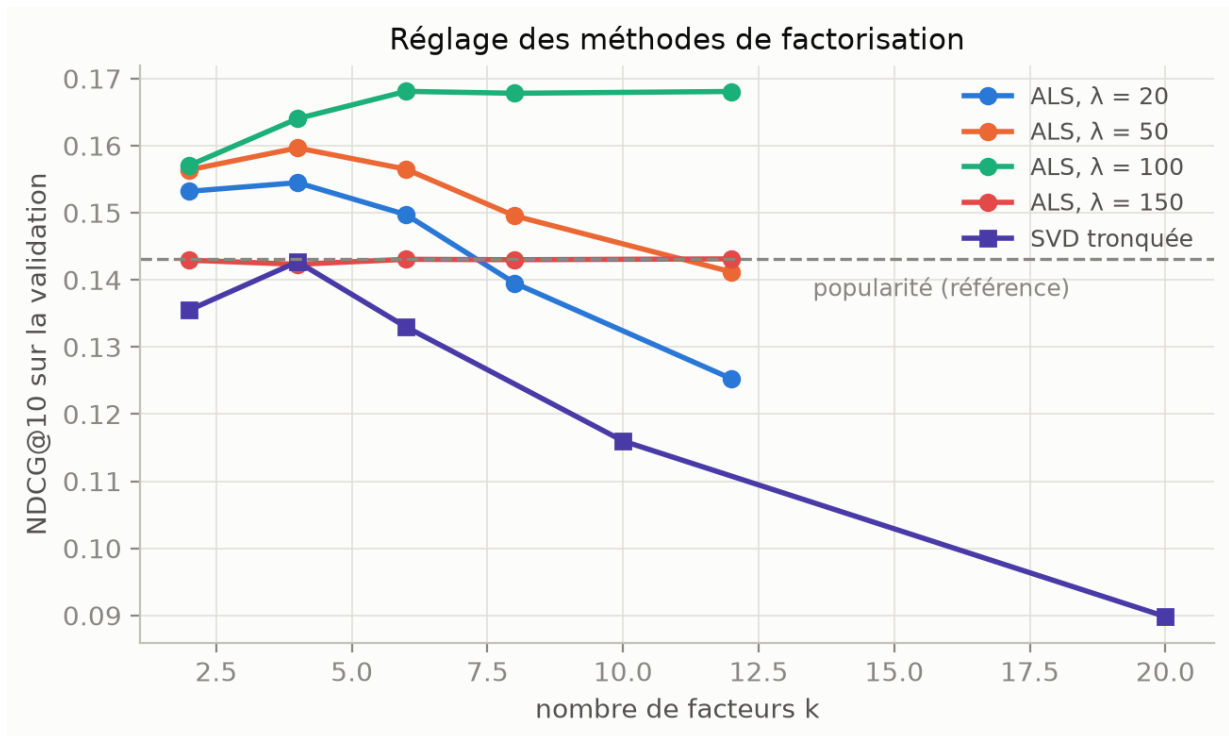


Figure 78: NDCG@10 sur le jeu de validation selon le nombre de facteurs, pour l'ALS implicite avec trois niveaux de régularisation et pour la SVD tronquée. La ligne en tirets est la popularité.

- **La SVD tronquée ne fait pas mieux que la popularité.** Son meilleur NDCG (0,143, avec $k = 4$) est celui de la popularité, puis il baisse jusqu'à 0,090 pour $k = 20$: sans pondération ni régularisation, elle apprend surtout le bruit.

Que valent les facteurs eux-mêmes ? Projets les vecteurs des produits appris par le meilleur modèle sur leurs deux directions principales (l'ACP du volume II, section 3.1).

La projection montre une structure : les produits de la catégorie C se regroupent en haut, ceux de la catégorie A en bas à gauche, ceux de la catégorie B à droite, la catégorie D étant plus diffuse. Les groupes se **recouvrent** pourtant : la dispersion moyenne à l'intérieur d'une catégorie (0,40) est proche de la distance moyenne entre les centres de deux catégories (0,45). Mais la projection en deux dimensions **écrase** l'information : dans l'espace complet des six facteurs, **86 %** des cinq plus proches voisins d'un produit (au sens du cosinus de leurs facteurs) appartiennent à sa catégorie, contre environ 26 % si les voisins étaient tirés au hasard. Les facteurs ne sont donc pas des catégories : ils capturent des goûts qui recoupent largement, mais pas exactement, le rangement du catalogue. (Les données ont été fabriquées avec six facteurs latents, dont les produits d'une même catégorie partagent une part ; le modèle retrouve cette trace sans connaître ni les catégories ni cette vérité.)

N'interprétez pas trop les axes. Les facteurs latents ne sont définis qu'à **une rotation près** : si l'on remplace P par PA et Q par QA^{-T} pour une matrice inversible A , le produit PQ^T est inchangé (et la pénalité l'est aussi pour une rotation orthogonale). Une direction de l'espace des facteurs n'a donc pas de sens propre ; seules comptent les **distances** et les **produits scalaires**. Nommer un axe « goût pour le traditionnel » est une histoire que l'on se raconte, pas un résultat.

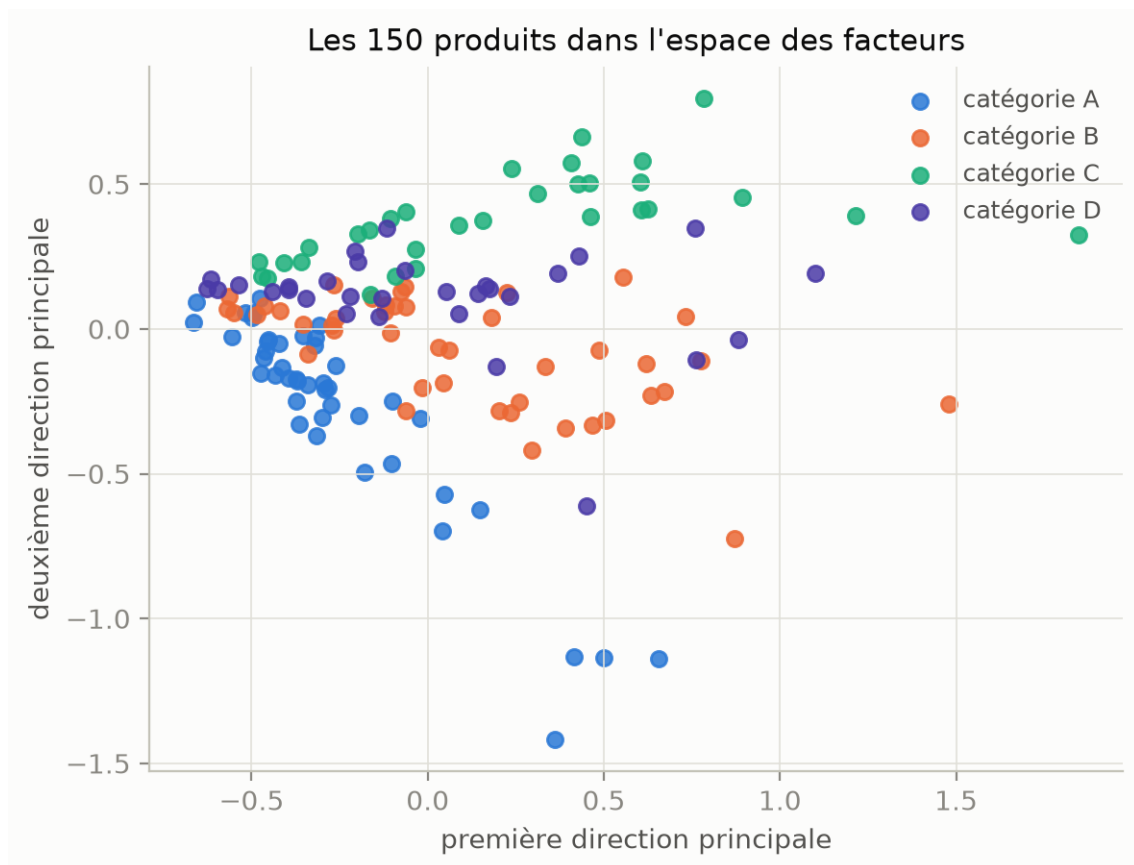


Figure 79: Les 150 produits projetés sur les deux premières directions de l'espace des facteurs appris par l'ALS, colorés par catégorie. Le modèle ne connaissait pas les catégories : elles se regroupent en partie.

À retenir (7.3).

- La factorisation approche $R \approx PQ^T$: chaque client et chaque produit reçoit un vecteur de k **facteurs latents**, l'affinité étant leur produit scalaire. Elle **généralise** à des produits sans acheteurs communs.
- On minimise une erreur de reconstruction **régularisée** (Ridge) par **SGD** ou par **moindres carrés alternés** (chaque étape est une régression de Ridge).
- Pour des achats, on pondère par la **confiance** : cases vides à poids faible, achats à poids $1 + \alpha$.
- La SVD tronquée est une référence simple, qui traite les cases vides comme des zéros.
- k et λ se règlent sur la validation ; trop de facteurs sans régularisation = surapprentissage.

Pour s'entraîner. Cahier, chapitre 7 : applications 7.3 (ALS implicite écrit à la main) et 7.4 (factorisation sur les notes explicites par SGD), exercices 7.7 à 7.9.

7.4 Évaluation et démarrage à froid

Les trois sections précédentes ont comparé des méthodes sur le jeu de validation, avec des métriques annoncées mais pas encore définies. Cette dernière section fait le travail de rigueur qui, depuis le chapitre 1, est le fil rouge du volume : définir précisément ce que l'on mesure, comparer **honnêtement** les méthodes sur le jeu de test, s'interroger sur ce que ces chiffres ne disent pas, et regarder ce qui arrive quand le client ou le produit est **nouveau**.

7.4.1 Évaluer un classement

Pour un client donné, un système produit une liste ordonnée de k produits (ici $k = 10$). On dispose de l'ensemble des produits **pertinents** pour ce client : ceux qu'il a effectivement achetés et que l'on avait mis de côté (jeu de test). Cinq métriques complémentaires comparent la liste à cet ensemble.

- **Précision@k** : part des k produits recommandés qui sont pertinents. Elle répond à : « combien de ma vitrine sert-elle ? ».
- **Rappel@k** : part des produits pertinents qui figurent dans la liste. Elle répond à : « quelle part de ce que le client voulait ai-je trouvée ? ».
- **Taux de succès@k** (*hit rate*) : part des clients pour lesquels **au moins un** produit pertinent figure dans la liste.
- **MAP@k** (*mean average precision*) : pour chaque client, on moyenne la précision aux rangs où apparaît un produit pertinent, puis on moyenne sur les clients. Elle récompense de **placer les bons produits haut**.
- **NDCG@k** (*normalised discounted cumulative gain*) : on attribue à chaque produit pertinent un gain $1 / \log_2(\text{rang} + 1)$, qui décroît avec le rang ; on somme (DCG), puis on divise par la valeur maximale possible (IDCG) pour obtenir un nombre entre 0 et 1.

À la main. Une liste de $k = 5$ produits, dont les deuxième et quatrième sont pertinents ; le client avait en tout 3 produits pertinents dans le jeu de test (le troisième n'a pas été recommandé).

Rang	1	2	3	4	5
Pertinent ?	non	oui	non	oui	non

- Précision@5 = $2/5 = 0,4$; rappel@5 = $2/3 \approx 0,667$; succès@5 = 1.
- Précision aux rangs des succès : $1/2$ au rang 2, $2/4$ au rang 4. La précision moyenne (AP) est la somme de ces précisions divisée par le nombre de produits pertinents (borné par k), soit $(0,5 + 0,5)/3 \approx 0,333$.
- $DCG = \frac{1}{\log_2 3} + \frac{1}{\log_2 5} = 0,631 + 0,431 = 1,062$. Le meilleur classement possible placerait les 3 produits pertinents aux rangs 1, 2 et 3 : $IDCG = 1 + 0,631 + 0,5 = 2,131$. Donc $NDCG@5 = 1,062/2,131 \approx 0,498$.

Quelle métrique choisir ? Cela dépend de l'usage. Si l'on affiche une longue liste que le client parcourt, le **rappel** compte. Si la vitrine ne montre que trois produits, la **précision** et la position (**NDCG**, **MAP**) comptent. Aucune ne remplace la mesure commerciale finale (panier, conversion) : elles ne sont que des indicateurs hors ligne. Dans la suite, nous retenons le **NDCG@10** comme métrique principale et le **rappel@10** comme métrique de lecture facile.

Le « faux négatif » de l'évaluation. Un produit recommandé que le client n'a pas acheté compte comme une erreur, alors qu'il est peut-être un excellent choix qu'il n'avait pas vu. L'évaluation sur achats cachés **sous-estime** donc la qualité réelle de tout système, et pénalise davantage ceux qui recommandent des produits peu connus (7.4.4).

7.4.2 Un protocole honnête et des comparaisons avec incertitude

Voici la comparaison finale. Elle suit la discipline du chapitre 1 :

1. les hyperparamètres ont été choisis sur la **validation** (sections 7.2 et 7.3) ;
2. chaque méthode est maintenant **réentraînée sur l'entraînement et la validation réunis** (tout sauf le jeu de test) ;
3. elle est évaluée **une seule fois** sur le jeu de test, qui n'a servi à aucun choix ;
4. chaque métrique est accompagnée d'un **intervalle de confiance** obtenu par bootstrap sur les clients (volume I, section 3.3.5).

méthode	rappel@10	IC rappel	NDCG@10	IC NDCG
Au hasard	0.070 [0.063 ; 0.077]		0.044 [0.039 ; 0.048]	
Popularité	0.244 [0.232 ; 0.256]		0.174 [0.165 ; 0.182]	
Contenu	0.122 [0.113 ; 0.130]		0.075 [0.069 ; 0.080]	
Voisins de produits	0.270 [0.258 ; 0.282]		0.199 [0.189 ; 0.208]	
Voisins de clients	0.288 [0.275 ; 0.300]		0.209 [0.200 ; 0.218]	
SVD tronquée	0.255 [0.243 ; 0.267]		0.179 [0.171 ; 0.188]	
ALS implicite	0.295 [0.282 ; 0.307]		0.216 [0.206 ; 0.225]	

	comparaison	écart	NDCG@10	IC 95 %
ALS implicite – Popularité		0.0420	[0.036 ; 0.048]	
Voisins de clients – Popularité		0.0352	[0.030 ; 0.041]	
Voisins de produits – Popularité		0.0250	[0.018 ; 0.031]	
ALS implicite – Voisins de clients		0.0068	[0.002 ; 0.011]	
ALS implicite – Voisins de produits		0.0169	[0.011 ; 0.022]	
SVD tronquée – Popularité		0.0053	[-0.005 ; 0.015]	
clients : ALS meilleur 0.391 égalité 0.446 dont les deux à zéro 0.362 ALS moins bon 0.163				

On y lit quatre choses.

- **Toutes les méthodes personnalisées battent le hasard et la popularité, sauf la SVD tronquée** (NDCG 0,179 contre 0,174 pour la popularité : écart non significatif, voir plus bas).
- **L'ALS implicite est la meilleure** : NDCG@10 de **0,216** (intervalle de confiance [0,206 ; 0,225]) et rappel@10 de **29,5 %**. En moyenne, 0,84 des dix produits recommandés correspond à un achat caché (précision@10 de 0,084), et près de trois achats cachés sur dix sont

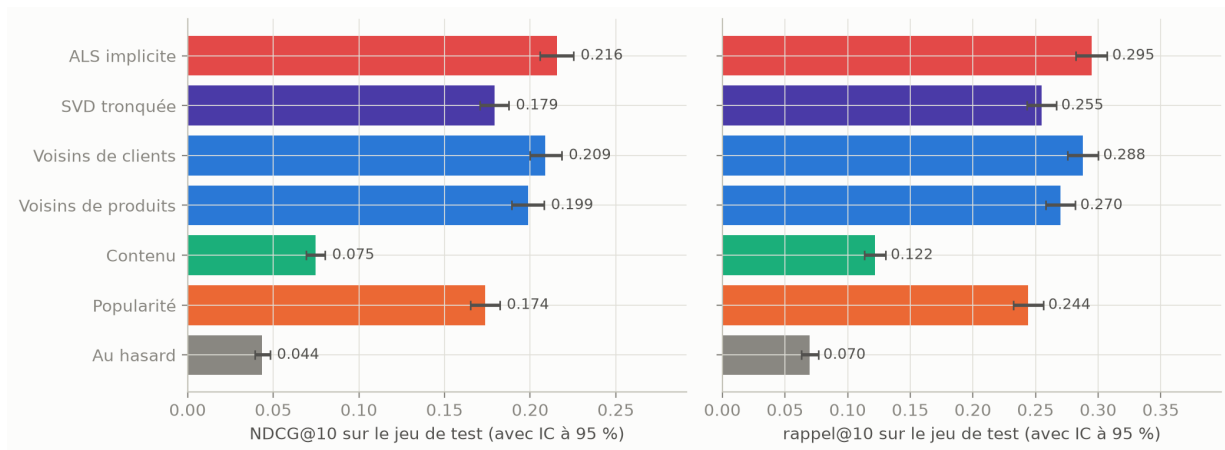


Figure 80: Performance des sept méthodes sur le jeu de test (2 365 clients) : NDCG@10 à gauche, rappel@10 à droite, avec l'intervalle de confiance à 95 % obtenu par bootstrap sur les clients.

retrouvés. Les voisins de clients (0,209) et les voisins de produits (0,199) suivent ; la popularité atteint 0,174 (rappel de 24,4 %). Le gain de la meilleure méthode sur la référence est donc de **4 points de NDCG** et de **5 points de rappel** : réel, mais ce n'est pas une révolution.

- **Le contenu seul est loin derrière** (0,075), bien au-dessus du hasard (0,044) mais en dessous de la popularité.
- **Les niveaux du test ne se comparent pas à ceux de la validation** (0,216 contre 0,168 pour l'ALS) : le test cache davantage d'achats par client (6 523 au total contre 3 987), ce qui facilite la tâche, et les modèles sont réentraînés avec 23 % d'achats de plus. Seules les comparaisons entre méthodes **sur un même jeu** ont un sens.

Les intervalles de confiance de deux méthodes peuvent se chevaucher alors que leur **différence** est significative : les deux méthodes sont évaluées sur les **mêmes clients**, et la comparaison est **appariée** (chapitre 1, section 1.4). On calcule donc, client par client, la différence de leur NDCG, puis un intervalle de confiance de la moyenne de cette différence.

Les écarts se lisent ainsi :

- **L'ALS bat nettement la popularité** : +0,042 de NDCG, avec un intervalle de [0,036 ; 0,048] qui est loin de zéro.
- **L'ALS bat aussi les voisins de clients**, de peu : +0,0068, intervalle [0,002 ; 0,011]. L'intervalle exclut zéro, donc sur ces données la factorisation est très probablement un peu meilleure ; il faut toutefois nuancer, car ce résultat porte sur **un seul jeu de données** et **une seule initialisation** de l'ALS. En validation, nous n'avions pas pu départager les deux méthodes ; le jeu de test, plus gros, le permet de justesse.
- **La SVD tronquée ne se distingue pas de la popularité** : +0,005, intervalle [−0,005 ; 0,015] qui contient zéro.
- **L'avantage moyen cache des situations très inégales**. L'ALS fait mieux que la popularité pour 39,1 % des clients, moins bien pour 16,3 %, et fait **jeu égal** pour 44,6 %, dont 36,2 points où ni l'une ni l'autre ne retrouve aucun achat. Le gain moyen est le fruit d'un avantage net sur une minorité de clients.

7.4.3 Pourquoi un découpage aléatoire peut tromper

Nous avons évalué en retirant des achats **au hasard**, client par client. Dans un vrai projet, les achats ont des dates, et ce découpage pose un problème : le jeu d'entraînement contient des achats **postérieurs** à certains achats du jeu de test. Le modèle « voit l'avenir » : il connaît, par exemple, la popularité qu'aura un produit au moment où on lui demande de la prédire. Le chapitre 1 (section 1.1) l'a dit pour les modèles supervisés ; c'est encore plus net en recommandation, où les goûts et les catalogues changent.

Nos données n'ont pas de dates : nous ne pouvons pas mesurer l'écart. Simulons donc un monde où le temps existe. 2 000 clients et 100 produits, deux périodes ; entre les deux, la popularité de **30 produits** change nettement (certains montent, d'autres chutent). Le but est de prédire les achats de la **seconde** période de produits que le client n'avait pas achetés dans la première. Deux protocoles :

- **découpage aléatoire** : on mélange les deux périodes, on retire 25 % des achats de chaque client au hasard, on entraîne sur le reste ;
- **découpage temporel** : on entraîne sur la première période et on teste sur la seconde.

méthode	découpage aléatoire	découpage temporel
Popularité	0.328	0.221
Voisins de produits	0.397	0.259
ALS implicite	0.398	0.260

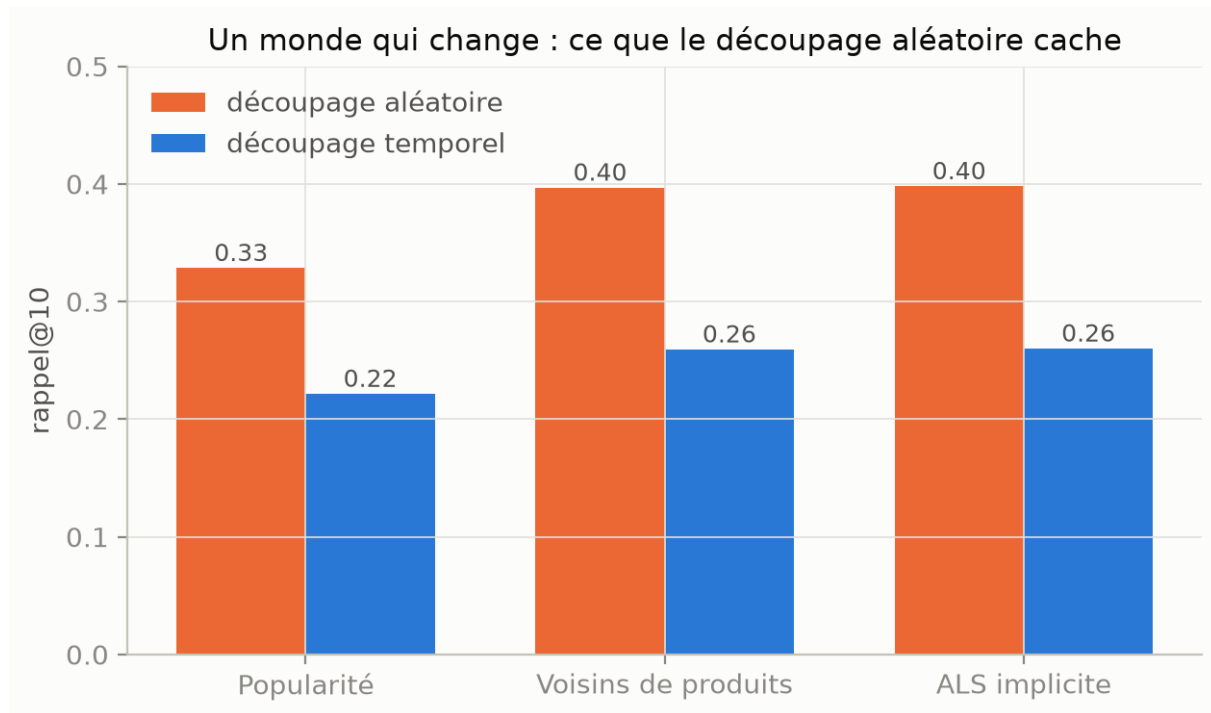


Figure 81: Rappel@10 de trois méthodes sur un monde simulé où la popularité de 30 produits change entre deux périodes, selon que l'on évalue avec un découpage aléatoire ou un découpage temporel.

Le découpage aléatoire **gonfle toutes les performances** : le rappel@10 de la popularité passe de 0,22 (temporel) à 0,33 (aléatoire), soit **48 % de plus** ; celui des voisins de produits de 0,26 à 0,40 (+53 %) et celui de l'ALS de 0,26 à 0,40 (+53 %). Le modèle entraîné sur un mélange des deux périodes connaît déjà les produits devenus populaires pendant la seconde, que le modèle « du passé » ne pouvait pas deviner. Et le découpage aléatoire **exagère aussi l'intérêt de la personna-**

lisation : l'ALS dépasse la popularité de 0,07 de rappel en évaluation aléatoire, de 0,04 seulement en évaluation temporelle. Le classement des méthodes reste le même (hyperparamètres fixés sans réglage dans cette simulation, une seule simulation).

Conséquence pratique. Avec des données datées, on découpe **toujours dans le temps** : on entraîne sur le passé, on valide sur la période suivante, on teste sur la plus récente. C'est le même principe que la validation des séries temporelles (volume II, section 4.3). Notre évaluation par retrait aléatoire est donc, sur ce jeu sans dates, la meilleure option disponible, mais ses chiffres sont **optimistes en valeur absolue** ; les comparaisons entre méthodes y sont plus fiables que les niveaux.

7.4.4 Au-delà de la précision : couverture, nouveauté, biais de popularité

Une méthode qui recommande toujours les mêmes dix produits peut avoir un bon rappel et pourtant décevoir la gérante : le catalogue entier ne bénéficie pas de la vitrine, et les clients ne découvrent rien. Trois indicateurs complètent la précision.

- La **couverture** : part des 150 produits qui apparaissent au moins une fois dans une liste de dix.
- La **nouveauté** : le « degré de surprise » moyen des produits recommandés, mesuré par $-\log_2$ de leur part de popularité ; plus elle est élevée, plus on recommande des produits peu connus.
- La **diversité** d'une liste : part des paires de produits d'une même liste qui appartiennent à des catégories différentes.

méthode	NDCG@10	couverture	nouveauté (bits)	part des 10 plus populaires	diversité
Au hasard	0.044	1.000	7.695	0.058	0.748
Popularité	0.174	0.160	5.711	0.827	0.801
Contenu	0.075	1.000	7.741	0.048	0.115
Voisins de produits	0.199	0.480	6.016	0.433	0.614
Voisins de clients	0.209	0.433	5.909	0.536	0.704
SVD tronquée	0.179	0.380	6.176	0.356	0.577
ALS implicite	0.216	0.340	5.903	0.517	0.686

Les chiffres appellent quelques remarques.

- La **popularité est la moins variée** : seuls **24 produits sur 150** (16 %) apparaissent dans les listes, **83 %** des emplacements sont occupés par les dix produits les plus vendus, et la nouveauté est la plus basse (5,7 bits).
- **Parmi les méthodes personnalisées, la couverture est de 48 %** pour les voisins de produits, 43 % pour les voisins de clients, 38 % pour la SVD et **34 % pour l'ALS**. Celle qui a la meilleure précision est donc la moins variée de ce groupe : 52 % de ses emplacements sont occupés par les dix produits les plus populaires, contre 43 % pour les voisins de produits.
- **Le contenu couvre tout le catalogue** (100 %) avec la nouveauté la plus élevée (7,7 bits, comparable au hasard), mais sa **diversité** est de 0,12 : ses listes sont presque entièrement de **la même catégorie**, le client est enfermé dans ce qu'il a déjà acheté. C'est la « bulle de filtre ».
- **Le hasard a une couverture parfaite et ne sert à rien** (NDCG de 0,044) : un indicateur de variété ne dit rien de la qualité s'il est lu seul.

Le compromis précision-découverte. Ces indicateurs vont rarement dans le même sens que la précision. Un système qui améliore la couverture et la nouveauté aide les produits de la longue traîne, mais il prend plus de risques sur chaque recommandation individuelle. Le

bon réglage dépend d'un choix commercial (vendre plus aujourd'hui ou élargir les achats de demain) que la donnée seule ne tranche pas.

7.4.5 Le démarrage à froid

Toutes les méthodes précédentes ont besoin d'**historique**. Que faire face à un client qui n'a encore rien acheté, ou à un produit qui n'a pas encore été acheté par personne ? C'est le problème du **démarrage à froid** (*cold start*).

Un nouveau client. Plaçons-nous du point de vue d'un client dont on ne connaît que m achats. On prend les clients ayant au moins 11 achats, on en met un quart à l'écart comme « nouveaux », on entraîne les modèles sur les autres, et on révèle aux nouveaux clients seulement m de leurs achats (0, 1, 2, 3, 5 ou 8) : le but est de retrouver tous les autres. Pour l'ALS, le facteur d'un nouveau client se calcule en **une seule étape** de la section 7.3 (le *fold-in*), sans réentraîner le modèle. À $m = 0$, aucune méthode personnalisée n'a rien à utiliser : elles retombent toutes sur la popularité.

	Popularité	Voisins de produits	ALS implicite
achats connus			
0	0.198	0.198	0.198
1	0.201	0.197	0.210
2	0.208	0.237	0.240
3	0.209	0.259	0.248
5	0.216	0.269	0.274
8	0.224	0.293	0.294

Un nouveau produit. Le problème est plus dur encore : un produit sans acheteur n'a **aucune** colonne utilisable par le filtrage collaboratif, ni par la popularité. Seul le **contenu** (catégorie, prix) peut parler. Pour le mesurer, on retire de l'entraînement les **26 produits récents** (nouveau = 1) ; pour chaque client qui en a acheté, on classe les 26 produits récents à partir de ses **autres achats** et on regarde si ceux qu'il a réellement achetés arrivent en tête. Une liste au hasard retrouverait en moyenne $5/26 \approx 19,2\%$ des achats parmi ses cinq premiers choix.

	rappel@5 parmi les 26 nouveaux	AUC par client
Au hasard	0.195	0.498
Catégorie seule	0.296	0.617
Catégorie et prix	0.290	0.592

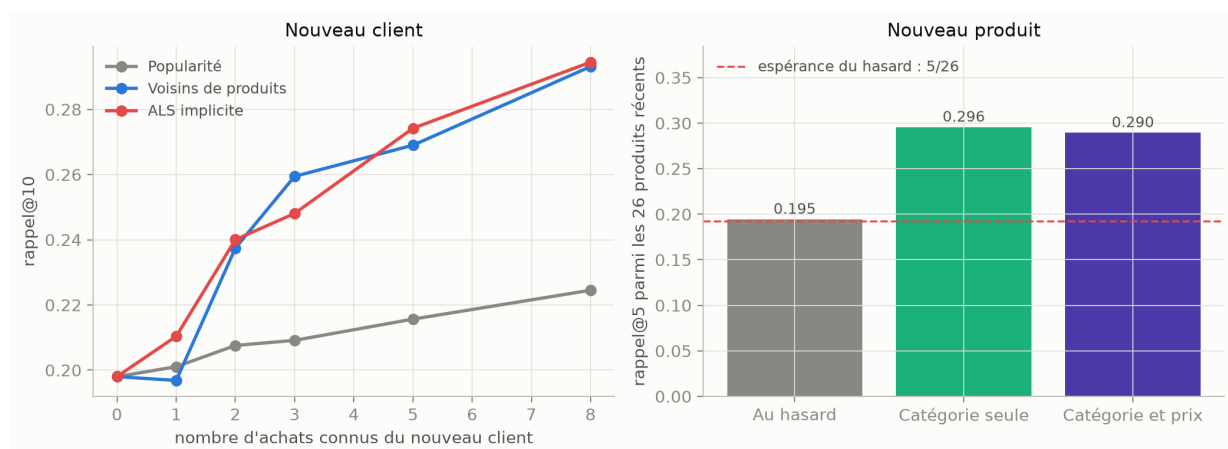


Figure 82: À gauche : rappel@10 en fonction du nombre d'achats connus d'un nouveau client, pour trois méthodes. À droite : rappel@5 parmi les 26 produits récents, avec la ligne du hasard.

Pour un nouveau client, la courbe de gauche se lit en trois temps.

- Sans aucun achat connu, **toutes les méthodes sont à égalité** (rappel@10 de 19,8 %) : ce sont, par construction, des recommandations de popularité.
- Avec **un seul achat**, l'ALS (21,0 %) dépasse à peine la popularité (20,1 %), et les voisins de produits (19,7 %) font un peu moins bien qu'elle ; sur 250 nouveaux clients, ces écarts ne sont pas mesurables.
- À partir de **deux achats**, l'écart devient net (24,0 % pour l'ALS et 23,7 % pour les voisins de produits, contre 20,8 %) et il atteint 7 points à huit achats (29,4 % et 29,3 % contre 22,4 %). La popularité progresse elle aussi un peu, car les produits déjà connus sont retirés de la liste, ce qui libère des places.

Pratiquement, deux ou trois achats suffisent pour que la personnalisation décolle : d'où l'intérêt d'un parcours d'accueil qui recueille rapidement quelques préférences.

Pour un nouveau produit, le contenu fait mieux que le hasard : en classant les 26 produits récents par ressemblance de catégorie avec les achats du client, on retrouve **29,6 %** de ses achats dans les cinq premiers choix (AUC de 0,62), contre 19,5 % pour une liste tirée au hasard (l'espérance est de 5/26, soit 19,2 %). Ajouter le prix **n'apporte rien** (29,0 % ; AUC de 0,59) : sur ces données simulées, le prix n'est en effet lié à aucun goût. Le gain est réel mais modeste : un produit sans historique reste difficile à recommander, et c'est une raison d'**organiser son lancement** (mise en avant volontaire, exploration) plutôt que d'attendre que les ventes le fassent remonter.

7.4.6 Les recommandations changent les données

Il reste la limite la plus profonde. Les achats que nous utilisons pour apprendre ne sont pas tombés du ciel : ils ont été **influencés par ce que le site affichait**. Un produit en vitrine est acheté plus souvent *parce qu'il est en vitrine*. Un modèle entraîné sur ces achats apprend, en partie, ce que le modèle précédent montrait. C'est une **boucle de rétroaction**.

Simulons-la. 1 500 clients, 60 produits dont l'attrait réel est connu (nous le programmons). À chaque tour, la vitrine montre 5 produits à chaque client ; il en achète un avec une probabilité qui dépend de son **goût réel** ; la vitrine du tour suivant est recalculée à partir des achats observés. Quatre politiques de vitrine :

1. **les produits les plus vendus** (le classement par popularité de 7.1.5) ;
2. les produits les plus vendus, avec **20 % d'exploration** (un emplacement sur cinq est tiré au hasard) ;
3. le classement par **taux d'achat** (achats divisés par expositions, lissé), avec 20 % d'exploration ;
4. le classement par taux d'achat, sans exploration.

	part des achats sur 5 produits bons produits en vitrine (sur 5)	produits achetés (sur 60)	corrélation
↪ avec l'attrait réel ventes seules		0.96	47.25
↪ 0.78	0.50		
ventes + 20 % de hasard		0.78	59.62
↪ 0.82	0.50		
taux d'achat + 20 % de hasard		0.83	59.62
↪ 0.94	4.62		
taux d'achat seul		0.80	45.88
↪ 0.99	4.88		

Le tableau et les courbes montrent un résultat net.

- Avec le classement par ventes seules, la boucle se referme : 96 % des achats se concentrent sur cinq produits, seuls 47 produits sur 60 sont achetés au moins une fois, et **la vitrine ne contient en moyenne que 0,5 des cinq meilleurs produits réels**. Les ventes mesurent ce qu'on a montré, pas ce que les clients aiment.

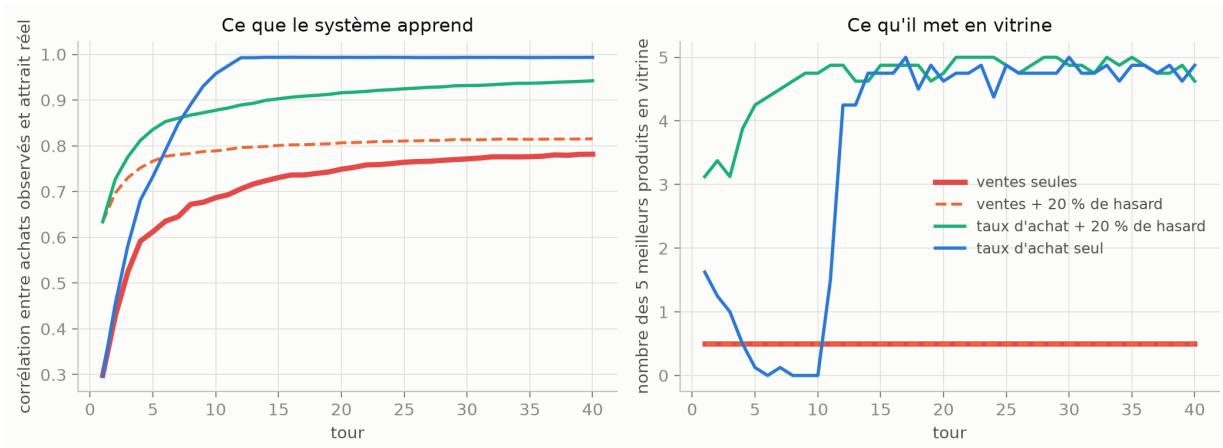


Figure 83: Boucle de rétroaction simulée sur 40 tours (moyenne de 8 simulations) pour quatre politiques de vitrine : corrélation entre les achats observés et l'attrait réel des produits (à gauche) et nombre des cinq meilleurs produits présents en vitrine (à droite).

- **Ajouter 20 % d'exploration ne suffit pas** : on achète alors presque tous les produits (59,6 sur 60), la corrélation avec l'attrait réel passe de 0,78 à 0,82, mais la vitrine ne contient toujours que 0,5 des cinq meilleurs. Le problème n'est pas seulement de montrer plus de produits, c'est de **lire correctement** les ventes.
- **Le classement par taux d'achat** (achats par exposition) corrige cette lecture : la corrélation monte à 0,94 avec exploration et 0,99 sans, et la vitrine contient **4,6 à 4,9 des cinq meilleurs produits**.
- **Sans exploration explicite, la méthode découvre quand même**, mais lentement : le lissage donne à un produit jamais montré un taux optimiste, de sorte que les produits peu convaincants quittent la vitrine. La courbe bleue montre une dizaine de tours pendant lesquels la vitrine ne contient presque aucun bon produit, avant qu'elle ne bascule ; pendant ce temps, seuls 46 produits sur 60 sont achetés.

Ce que cela implique en pratique.

- Un modèle de recommandation évalué **hors ligne** (comme ici) mesure sa capacité à prédire des achats *influencés par le système précédent*. Seul un **test en conditions réelles** (A/B test, volume II, chapitre 7) mesure l'effet *causal* des recommandations sur les achats.
- Il faut **réserver de l'exploration** : montrer de temps en temps des produits que le modèle n'aurait pas choisis, pour apprendre ce qu'on ignore.
- Un produit recommandé n'est pas toujours un produit **en plus** : il peut simplement avoir été acheté à la place d'un autre (cannibalisation). Un bon indicateur est le chiffre d'affaires total, pas la part des ventes qui passent par les recommandations.

À retenir (7.4).

- On évalue un **classement** : précision@k, rappel@k, succès@k, MAP@k, NDCG@k ; aucune ne remplace la mesure commerciale.
- Protocole : hyperparamètres choisis sur la validation, réentraînement sur tout sauf le test, **une seule** évaluation sur le test, **intervalles de confiance** et comparaisons **appariées**.
- Avec des données datées, **on découpe dans le temps** : un découpage aléatoire fait voir l'avenir au modèle et gonfle les performances.

- Couverture, nouveauté et diversité complètent la précision ; la popularité est un biais à surveiller.
- **Démarrage à froid** : sans historique, tout retombe sur la popularité ; le contenu est le seul recours pour un produit nouveau.
- Les recommandations modifient les achats futurs : prévoir de l'exploration et valider par des tests réels.

Pour s'entraîner. Cahier, chapitre 7 : applications 7.5 (métriques et intervalles de confiance), 7.6 (démarrage à froid), 7.7 (découpage aléatoire contre temporel), 7.8 (boucle de rétroaction) et 7.9 (mélange de méthodes et diversité), exercices 7.10 à 7.12.

Bilan du chapitre 7

Vous savez maintenant :

- **formuler** un problème de recommandation comme un problème de **classement** (et non de prédiction de notes), distinguer retours **explicites** et **implicites**, et ne pas confondre une case vide avec un refus ;
- manier la **matrice d'interactions creuse** et la **similarité cosinus** (pour des achats binaires, $\cos(i, j) = n_{ij} / \sqrt{n_i n_j}$) ;
- construire deux **références** à battre, la **popularité** et le **filtrage par contenu**, et expliquer pourquoi la première est si difficile à dépasser ;
- écrire un **filtrage collaboratif de voisinage** (clients ou produits), le **centrer** quand on dispose de notes, le protéger par **rétrécissement** et en régler le voisinage sur la validation ;
- expliquer la **factorisation matricielle** $R \approx PQ^T$, son objectif de moindres carrés **régularisés**, les algorithmes **SGD** et **ALS**, la **pondération par la confiance** des retours implicites et la place de la **SVD tronquée** comme référence ;
- **évaluer un classement** avec $\text{précision}@k$, $\text{rappel}@k$, $\text{succès}@k$, $\text{MAP}@k$ et $\text{NDCG}@k$, comparer des méthodes avec des **intervalles de confiance appariés**, et regarder au-delà de la précision (**couverture**, **nouveauté**, **diversité**) ;
- reconnaître les pièges propres à la recommandation : le **découpage aléatoire** qui gonfle les résultats quand les données sont datées, le **démarrage à froid** (nouveau client, nouveau produit) et la **boucle de rétroaction** entre recommandations et achats.

Un message à retenir : **sur ce catalogue, une méthode très simple (la popularité) fait déjà presque tout, et la personnalisation n'ajoute que quelques points**, mesurables seulement avec un protocole rigoureux. Le plus souvent, l'essentiel du travail d'un système de recommandation n'est pas l'algorithme, mais la qualité de l'évaluation, la gestion du démarrage à froid et l'exploration qui entretient les données dont il se nourrit.

Pour s'entraîner. Cahier, chapitre 7 : applications 7.1 à 7.9 et exercices 7.1 à 7.12.

Le chapitre suivant du volume (chapitre 8, également facultatif) retourne le problème : quand les **étiquettes** manquent ou coûtent cher à obtenir, comment apprendre avec peu d'exemples étiquetés, et lesquels demander en priorité ?

Chapitre 8 : ⊕ Apprentissage semi-supervisé et actif

Chapitre complémentaire. Ce chapitre est entièrement facultatif : les chapitres 1 à 5 se lisent et se comprennent sans lui. Il s'adresse à celles et ceux qui se heurtent, en pratique, à un problème très courant : **on a beaucoup de données, mais très peu d'étiquettes**.

« Les données coulent à flots ; les étiquettes, elles, se paient au compte-gouttes. »

Tout au long du volume, nous avons supposé que chaque ligne du tableau venait avec sa **réponse** : ce client a-t-il résilié ou non, cette image représente-t-elle un 3 ou un 8, cette commande est-elle une fraude. Dans la vie réelle, cette réponse a souvent un **prix** :

- pour savoir si un client a résilié, il faut **attendre** trois mois, ou l'appeler (quelques euros par appel) ;
- pour savoir si une image montre tel chiffre ou tel défaut sur un produit, il faut qu'un **humain** la regarde ;
- pour savoir si une commande est frauduleuse, il faut une **enquête**.

La boutique, elle, possède des dizaines de milliers de lignes de données *brutes* (clients, images, commandes) mais seulement quelques dizaines de lignes *étiquetées*. Deux familles de méthodes répondent à cette situation, avec deux questions différentes :

	Question posée	Le lecteur retiendra
Apprentissage semi-supervisé	« Les données non étiquetées que j'ai déjà peuvent-elles m'aider à mieux apprendre avec peu d'étiquettes ? »	exploiter ce qu'on a gratuitement
Apprentissage actif	« Si je ne peux payer que <i>b</i> étiquettes, lesquelles dois-je demander ? »	choisir ce qu'on achète

Intuition. Imaginez que vous apprenez à reconnaître des champignons. Un expert est disponible une heure. Vous pouvez (1) *regarder des milliers de photos sans légende* en vous disant « ces deux-là se ressemblent donc sont sans doute de la même espèce » : c'est le **semi-supervisé** ; ou (2) *choisir avec soin les dix champignons que vous montrerez à l'expert*, en lui apportant ceux qui vous font le plus hésiter : c'est l'**actif**. Les deux se combinent.

Le chemin de ce chapitre

- **8.1 Pourquoi et hypothèses** : ce que les données non étiquetées *peuvent* apporter, les quatre hypothèses qui le permettent, les cas où elles **nuisent**, et le seul outil d'évaluation honnête de ce chapitre, la **courbe d'apprentissage selon le budget d'étiquettes**.
- **8.2 Auto-apprentissage et propagation d'étiquettes** : deux méthodes concrètes, l'une qui étiquette elle-même les cas faciles, l'autre qui fait « couler » les étiquettes le long d'un graphe de similarité.
- **8.3 Apprentissage actif** : la boucle « entraîner, choisir, demander, recommencer », les critères de choix (incertitude, marge, entropie, comité, densité), le **piège du biais d'échantillonnage**, le **démarrage à froid** et un petit calcul de coût.

Pour s'entraîner. Le cahier de ce chapitre contient huit applications guidées (courbes d'apprentissage, auto-apprentissage, propagation, co-apprentissage, boucle active écrite à la main, comité, biais d'échantillonnage, calcul de coût) et douze exercices corrigés. Chaque section du livre indique celles qui la prolongent.

Les données du chapitre

Deux jeux de données serviront d'un bout à l'autre, **sans aucun téléchargement** :

Jeu	Nature	Taille	Rôle
Chiffres manuscrits (load_digits de scikit-learn)	réel : images 8 × 8 pixels de chiffres écrits à la main, 10 classes	1 797 images ; nous en gardons 1 347 comme réservoir (<i>pool</i>) à étiqueter et 450 comme jeu de test	des classes bien groupées : le cas où les méthodes de ce chapitre brillent
Clients de la boutique (donnees/clients_ml.csv, résiliation à 90 jours (14 volume III)	simulé : 12 000 clients, %)	9 000 en réservoir, 3 000 en test	des variables mélangées (nombres, catégories) et des classes qui se chevauchent : le cas où ces méthodes déçoivent

Les étiquettes de ces deux jeux sont en réalité toutes connues : c'est ce qui permet de **simuler** un budget d'étiquettes limité (on « cache » presque toutes les étiquettes, puis on mesure ce que les méthodes savent en faire) et de vérifier, à la fin, si elles ont deviné juste. Les clients, eux, sont plus difficiles : des variables de natures différentes et des classes qui se chevauchent, comme dans la vraie vie.

Une convention à connaître. Dans scikit-learn, une observation **sans étiquette** se marque par la valeur -1 dans le vecteur des étiquettes. Les méthodes semi-supervisées reçoivent *tout* le tableau de variables *X*, et un vecteur *y* où seules quelques entrées sont renseignées.

```
y_partiel = np.full(len(y), -1) # -1 = « pas d'étiquette »
y_partiel[indices_etiquetes] = y[indices_etiquetes]
```

8.1 Pourquoi et hypothèses

Avant de présenter des méthodes, il faut comprendre **pourquoi** des données sans étiquette pourraient aider, à quelles **conditions**, et comment **vérifier honnêtement** qu'elles aident. Cette section pose le cadre ; les deux suivantes l'utilisent.

8.1.1 Le problème de l'étiquette chère

Reprenons les notations du volume. Nous disposons de deux paquets de données :

- un petit ensemble **étiqueté** $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^{n_\ell}$, où l'on connaît la réponse y_i ;
- un grand ensemble **non étiqueté** $\mathcal{U} = \{x_j\}_{j=1}^{n_u}$, où l'on ne connaît que les variables x_j .

La situation typique est $n_u \gg n_\ell$: quelques dizaines d'exemples étiquetés, des milliers d'autres sans réponse. On distingue trois façons de s'en sortir :

Paradigme	Ce qu'on utilise	Ce qu'on produit
Supervisé	seulement $\mathcal{L}$	un modèle f qui prédit y à partir de x
Non supervisé	seulement les x	des groupes, des axes, une structure (chapitre 3)
Semi-supervisé	$\mathcal{L}$ et $\mathcal{U}$	un modèle f , appris avec l'aide de la structure de $\mathcal{U}$

Deux nuances de vocabulaire sont utiles pour lire la littérature :

- l'apprentissage est **transductif** quand on ne cherche qu'à étiqueter les points de $\mathcal{U}$ que l'on a *déjà* sous la main (typiquement : les 5 000 clients de la base à qualifier) ;
- il est **inductif** quand on veut un modèle f capable de prédire sur de **nouveaux** points (les clients de demain).

La seule mesure du succès. Une méthode semi-supervisée n'a de valeur que si elle fait mieux que le modèle **supervisé entraîné sur les mêmes étiquettes** $\mathcal{L}$. Comparer à un modèle qui disposerait de *plus* d'étiquettes n'aurait aucun sens : l'enjeu est précisément ce que l'on gagne à **budget d'étiquettes égal**.

8.1.2 Ce que les données non étiquetées apportent

Un point x_j sans étiquette ne nous dit rien, à lui seul, sur sa classe. Alors comment pourrait-il aider ? Parce que l'**ensemble** des x_j nous renseigne sur la loi des variables, $p(x)$: où se concentrent les données, où sont les creux, quelle est la forme des groupes. Si cette structure est **liée** à la classe, l'information est précieuse.

Un exemple minuscule, entièrement à la main. Une seule variable x , deux classes A et B. Deux clients seulement sont étiquetés :

- le client A vaut $x = -1,4$;
- le client B vaut $x = +0,6$.

Entraîné sur ces deux points, un classifieur « centre le plus proche » place sa frontière au milieu : $(-1,4 + 0,6)/2 = -0,4$. Mais six autres clients, **sans étiquette**, ont été observés :

-2,1, -1,8, -1,2, +1,3, +1,7, +2,0.

Ils forment **visiblement deux paquets**, un à gauche et un à droite d'un creux vide autour de 0. La frontière à -0,4 passe au bord du paquet de gauche : elle est suspecte. Corrigions-la en une étape :

1. **Étiqueter provisoirement** chaque point non étiqueté selon la frontière actuelle (-0,4) : les trois de gauche sont classés A, les trois de droite B.
2. **Recalculer les centres** avec tous les points : $\bar{x}_A = \frac{-1,4 - 2,1 - 1,8 - 1,2}{4} = -1,625$ et $\bar{x}_B = \frac{0,6 + 1,3 + 1,7 + 2,0}{4} = 1,4$.
3. **Recalculer la frontière** : $\frac{-1,625 + 1,4}{2} = -0,1125$.

Si les deux classes sont des cloches symétriques de même largeur, la vraie frontière est en 0. La frontière est passée de -0,4 à -0,11 : **elle s'est rapprochée du creux**, grâce aux six points sans étiquette.

Le principe derrière l'exemple : la vraisemblance mixte. Supposons un modèle génératif $p(x, y | \theta) = p(y) p(x | y, \theta)$ (par exemple, deux lois normales). Pour un point étiqueté, la contribution à la log-vraisemblance est $\log p(x_i, y_i | \theta)$. Pour un point **non** étiqueté, on ne voit pas y_j ; on **somme** sur toutes ses valeurs possibles :

$$\ell(\theta) = \sum_{i \in \mathcal{L}} \log p(x_i, y_i | \theta) + \sum_{j \in \mathcal{U}} \log \sum_c p(y_j = c) p(x_j | y_j = c, \theta).$$

Le second terme est exactement celui d'un **mélange de lois** (nous retrouverons les mélanges gaussiens en section 3.3 du présent volume). On le maximise par l'algorithme **EM** : l'étape **E** calcule, pour chaque point sans étiquette, la probabilité $r_{jc} = P(y_j = c | x_j, \theta)$ (« à quel point ce point appartient-il à la classe c ? ») ; l'étape **M** réestime θ en comptant chaque point à hauteur de ces probabilités, les points étiquetés comptant pour 1 dans leur classe. L'exemple ci-dessus est une version **dure** d'EM : les probabilités r_{jc} y valent 0 ou 1.

Ce qu'il faut en retenir : les points sans étiquette n'agissent que par le terme $\log p(x_j | \theta)$, c'est-à-dire par ce qu'ils disent de **la loi des x** . Ils ne peuvent aider **que si** cette loi contient de l'information sur la frontière entre classes.

8.1.3 Les quatre hypothèses qui permettent d'aider

La théorie et la pratique du semi-supervisé reposent sur quelques **hypothèses sur le monde**, jamais vérifiables à 100 % :

Hypothèse	Énoncé en une phrase	Ce que ça autorise
Lissage (<i>smoothness</i>)	deux points proches dans une région dense ont probablement la même étiquette	propager une étiquette aux voisins
Groupes (<i>cluster</i>)	les points d'un même groupe partagent la même classe	étiqueter un groupe entier à partir d'un seul de ses points
Basse densité	la frontière entre classes passe par une région peu peuplée	déplacer la frontière vers les creux (notre exemple à la main)

Hypothèse	Énoncé en une phrase	Ce que ça autorise
Variété (<i>manifold</i>)	les données vivent près d'une surface de faible dimension, et ce sont les distances le long de cette surface qui comptent	utiliser un graphe de voisinage plutôt que la distance « à vol d'oiseau »

Voyons-les au travail sur deux jeux de 300 points en deux dimensions. En haut, deux « lunes » entrelacées : les groupes sont nets, la frontière naturelle passe dans le creux. En bas, deux nuages qui **se chevauchent** : il n'y a pas de creux, la structure de $p(x)$ ne dit rien de plus que ce que disent les étiquettes. Dans les deux cas, on ne donne que **trois étiquettes par classe**.

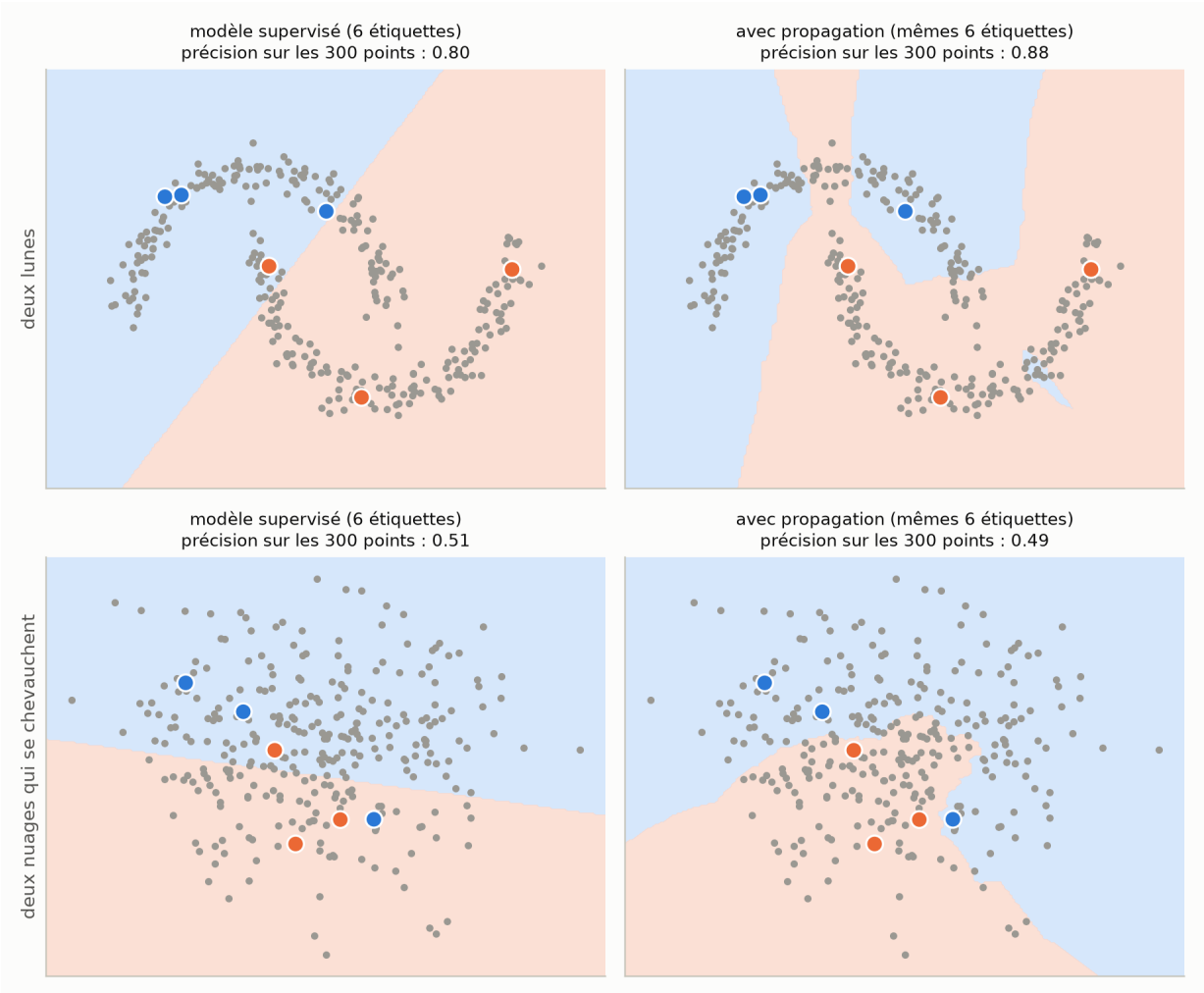


Figure 84: En haut, deux lunes entrelacées : avec trois étiquettes par classe, un modèle supervisé trace une frontière droite ; la propagation, qui s'appuie sur la forme des groupes, suit le creux entre les lunes. En bas, deux nuages qui se chevauchent : il n'y a pas de creux à exploiter, la propagation n'apporte rien. Les points colorés sont les six points étiquetés, les points gris sont les autres. Chaque ligne montre un tirage représentatif (dont le gain est le plus proche du gain moyen sur 20 tirages).

Sur 20 tirages différents des six étiquettes, la précision moyenne sur les lunes passe de **0,82** (supervisé) à **0,90** (propagation) : un gain net, qui vient de ce que la méthode a *vu* la forme des deux croissants. Sur les nuages qui se chevauchent, elle **ne bouge pas** (0,59 pour le supervisé, 0,57 pour la propagation, des différences très inférieures à l'écart-type d'un tirage à l'autre).

Le même algorithme, deux résultats opposés. Rien, dans la méthode, ne l'a prévenue de ce qui allait se passer. C'est la **structure des données** qui décide. Avant de recourir au semi-supervisé, posez-vous toujours la question : *est-il plausible que la forme de $p(x)$ renseigne sur la frontière entre classes ?*

8.1.4 Quand ça aide, quand ça nuit

Il n'existe pas de « repas gratuit » : les données sans étiquette ne sont pas toujours une bonne affaire. Voici les situations à surveiller.

Situation	Effet	Pourquoi
L'hypothèse est vraie (groupes nets, variété)	gain souvent important quand les étiquettes sont très rares	l'information de $p(x)$ est utile
L'hypothèse est fausse (classes qui se chevauchent, variables hétérogènes)	aucun gain , voire perte	la méthode « suit » une structure sans rapport avec les classes
Les étiquettes disponibles sont déséquilibrées ou peu représentatives	la méthode amplifie le défaut	elle étend ce qu'elle croit savoir à tous les voisins
Les données sans étiquette viennent d'ailleurs (autre période, autre population)	perte probable	$p(x)$ n'est plus celle du problème visé
Un modèle sûr de lui à tort (auto-apprentissage)	dérive	il se nourrit de ses propres erreurs (section 8.2.2)

Un exemple chiffré du troisième cas, sur nos chiffres manuscrits. On tire 50 étiquettes, mais de façon **déséquilibrée** : le chiffre 0 reçoit six fois plus de chances d'être tiré que chacun des autres. En moyenne, 39 % des étiquettes sont alors des « 0 », contre 10 % attendus.

Avec ces étiquettes mal réparties, le modèle supervisé atteint **0,70** de précision ; la propagation (qui s'appuie sur le graphe de voisinage) reste robuste, à **0,89** ; mais l'**auto-apprentissage descendant** à **0,65**, en dessous du supervisé : il prend ses propres préjugés pour des certitudes. Nous comprendrons le mécanisme en 8.2.2.

Règle pratique. Si vous hésitez, commencez par la méthode **non supervisée** de votre choix (chapitre 3) : regardez si les groupes existent et s'ils ressemblent à vos classes sur le petit échantillon étiqueté. Si oui, le semi-supervisé a de bonnes chances d'aider ; sinon, passez directement à l'apprentissage actif (section 8.3), qui ne fait pas ce pari.

8.1.5 Évaluer à budget d'étiquettes fixé : la courbe d'apprentissage

La bonne façon de mesurer l'intérêt d'une méthode de ce chapitre est de tracer une **courbe d'apprentissage selon le budget d'étiquettes** : on fait varier le nombre n_ℓ d'étiquettes disponibles, et pour chacun on mesure la qualité sur un **jeu de test** étiqueté, tenu à l'écart. Le protocole est strict :

1. **Un jeu de test fixe**, tiré au hasard, jamais utilisé pour apprendre ni pour choisir quoi que ce soit (section 1.1). Ici : 450 images, **toutes** étiquetées.
2. **Un réservoir** dont on « cache » presque toutes les étiquettes (ici 1 347 images).
3. Pour chaque budget n_ℓ : tirer **plusieurs** ensembles d'étiquettes différents (ici 10, avec des graines fixées), car un seul tirage est trompeur quand n_ℓ est petit.

4. **Les mêmes étiquettes pour toutes les méthodes** : ainsi la comparaison est **appariée**, et la différence est due à la méthode et non à la chance du tirage.
5. Reporter **la moyenne et l'écart-type** (ou un intervalle) sur les tirages, pas une valeur unique.
6. **Aucun réglage fin** sur le jeu de test. Et attention : avec 10 étiquettes, on ne peut pas faire de validation croisée sérieuse. Si l'on garde des étiquettes pour régler un hyperparamètre, **elles comptent dans le budget**.

Voici la courbe du modèle **supervisé seul** (une régression logistique) sur les chiffres. C'est la référence à battre.

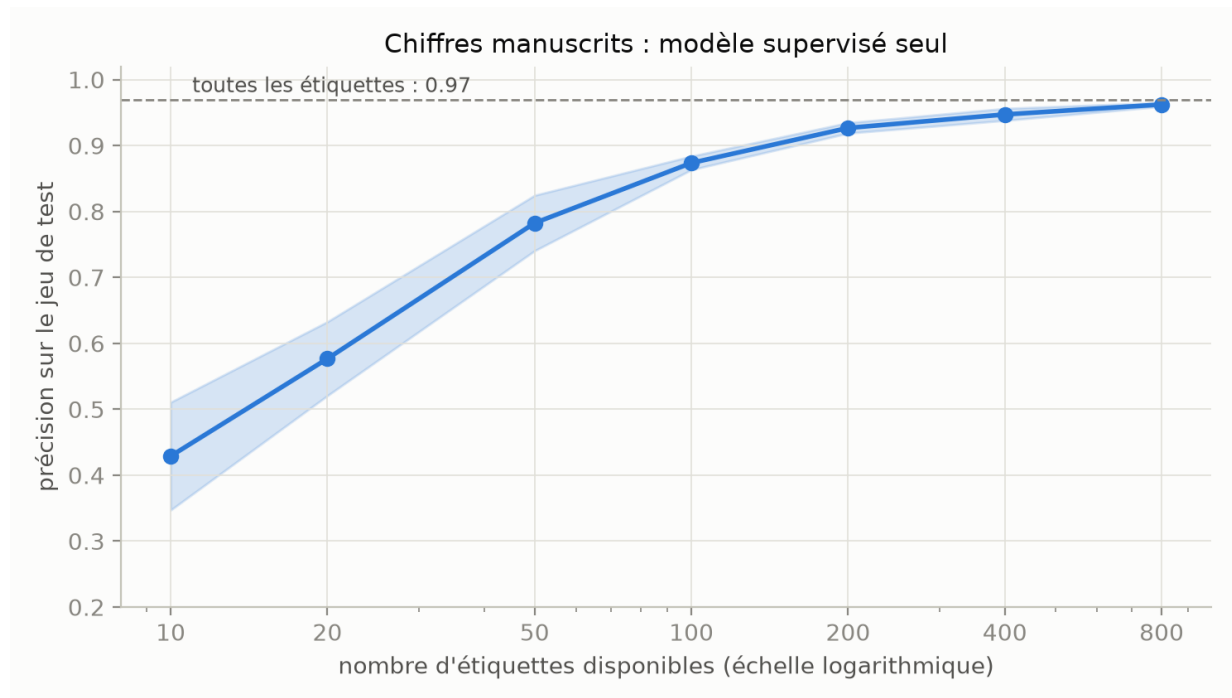


Figure 85: Précision d'une régression logistique sur le jeu de test des chiffres manuscrits, selon le nombre d'images étiquetées (moyenne et écart-type sur 10 tirages ; échelle logarithmique en abscisse). La courbe monte très vite avec les premières étiquettes puis s'aplatit vers la valeur obtenue avec toutes les étiquettes.

Trois lectures de cette courbe :

- **Elle est très raide au début.** Avec 10 étiquettes, la précision est de **0,43** ; avec 50, de **0,78** ; avec 100, de **0,87**. C'est dans ce régime que les méthodes de ce chapitre ont le plus à offrir.
- **Elle s'aplatit ensuite.** Avec 200 étiquettes, **0,93** ; avec les 1 347, **0,97**. Passé un certain budget, les étiquettes supplémentaires rapportent peu : l'apprentissage actif (8.3) cherche à *atteindre plus vite* ce plateau.
- **L'écart entre tirages est énorme à petit budget** : $\pm 0,08$ à 10 étiquettes, contre $\pm 0,01$ à 100. Sur nos dix tirages de 10 étiquettes, la précision va de **0,33** à **0,57** : un tirage isolé pourrait nous faire croire à n'importe quelle valeur dans cet intervalle. C'est pourquoi on **répète** les tirages.

À retenir.

- Le semi-supervisé utilise les points **sans étiquette** pour apprendre la forme de $p(x)$; l'actif choisit **quels points faire étiqueter**.
- Les données sans étiquette n'aident que si $p(x)$ renseigne sur les classes : hypothèses

de lissage, de groupes, de basse densité, de variété. Même méthode, même budget : gain sur des lunes entrelacées, rien sur des nuages qui se chevauchent.

- Une méthode ne vaut que par rapport au **modèle supervisé entraîné sur les mêmes étiquettes**. Des étiquettes mal réparties peuvent faire **perdre** à une méthode ce qu'elle gagnait.
- L'outil de mesure est la **courbe d'apprentissage selon le budget d'étiquettes** : jeu de test fixe, tirages répétés, **mêmes étiquettes** pour toutes les méthodes, moyenne et écart-type.

Pour s'entraîner. Cahier, chapitre 8 : application 8.1, exercices 8.1 à 8.3.

8.2 Auto-apprentissage et propagation d'étiquettes

Deux familles de méthodes concrètes exploitent les points sans étiquette. L'**auto-apprentissage** laisse le modèle étiqueter lui-même les cas dont il est sûr, puis s'entraîne dessus. La **propagation d'étiquettes** fait au contraire « couler » les étiquettes connues le long d'un graphe de similarité. Nous les construisons à la main, puis nous les mesurons sur nos deux jeux, avec les courbes d'apprentissage de la section 8.1.5.

8.2.1 L'auto-apprentissage

L'idée est celle du bon élève qui se corrige tout seul : on entraîne un premier modèle sur les rares étiquettes, on lui fait **prédire** les points sans étiquette, on garde les prédictions dont il est **très sûr**, on les ajoute au jeu d'entraînement comme si elles étaient vraies (on parle de **pseudo-étiquettes**), et on recommence.

Algorithme d'auto-apprentissage (*self-training*), avec un seuil de confiance $\tau \in]0, 1[$:

1. Entraîner le modèle f sur $\mathcal{L}$.
2. Calculer, pour chaque $x_j \in \mathcal{U}$, la classe prédite $\hat{y}_j$ et la confiance $p_j = \max_c P(c | x_j)$.
3. Pour chaque j tel que $p_j \geq \tau$: ajouter $(x_j, \hat{y}_j)$ à $\mathcal{L}$ et retirer x_j de $\mathcal{U}$.
4. Recommencer à l'étape 1 jusqu'à ce qu'aucun point ne dépasse le seuil (ou qu'on ait atteint un nombre maximal de tours).

Le modèle de base peut être **n'importe quel** classifieur capable de donner des probabilités. Avec scikit-learn, c'est une enveloppe autour de ce modèle, et il suffit de passer le tableau complet avec les étiquettes partielles (convention -1, introduction du chapitre) :

```
from sklearn.semi_supervised import SelfTrainingClassifier

base = LogisticRegression(max_iter=2000)
auto = SelfTrainingClassifier(base, threshold=0.9).fit(Xp, y_partiel)
seul = LogisticRegression(max_iter=2000).fit(Xp[indices_etiquetes], yp[indices_etiquetes])
print("auto-apprentissage :", round(auto.score(Xt, yt), 3), "| supervisé seul :",
      ↪ round(seul.score(Xt, yt), 3))
```

```
auto-apprentissage : 0.831 | supervisé seul : 0.831
```

Sur ce tirage de 50 étiquettes, l'auto-apprentissage fait **exactement aussi bien** que le modèle supervisé seul (0,831 des deux côtés) : il n'a rien apporté. Un seul tirage ne prouve rien, dans un sens comme dans l'autre. Mesurons sur 10 tirages, en faisant varier le seuil. Pour chaque seuil,

nous indiquons aussi **combien** de pseudo-étiquettes ont été ajoutées et **quelle part était juste** (on la connaît, car nous avons caché les vraies étiquettes sans les perdre).

Seuil τ	Précision sur le test	Pseudo-étiquettes ajoutées (moyenne)	Part des pseudo-étiquettes qui sont justes
0,60	0,759	1 216	0,783
0,80	0,593	888	0,734
0,90	0,706	176	0,912
0,95	0,781	2	1,000
0,99	0,782	0	(aucune)

Le modèle supervisé seul, avec les mêmes 50 étiquettes, atteint **0,782**. Aucun seuil ne fait **mieux**. Les deux extrêmes sont instructifs :

- à $\tau = 0,99$ et $\tau = 0,95$, le modèle n'est jamais assez sûr de lui : presque aucune pseudo-étiquette n'est ajoutée, et le résultat est celui du supervisé seul (**0,78**) ;
- à $\tau = 0,6$ ou $0,8$, le modèle ajoute **plusieurs centaines** de pseudo-étiquettes dont **environ un cinquième à un quart sont fausses** (0,78 et 0,73 justes) : il apprend sur ses propres erreurs.

Le seuil intermédiaire de 0,90 est le plus dangereux de façon trompeuse : 91 % des 176 pseudo-étiquettes sont justes, mais celles qui sont fausses sont **faussement confiantes**, et la précision finale baisse tout de même à 0,71.

8.2.2 Pourquoi l'auto-apprentissage peut s'auto-tromper

Ce qui précède n'est pas un accident de réglage : c'est le **biais de confirmation** de la méthode. Trois mécanismes s'additionnent.

1. **Une confiance qui n'est pas une probabilité.** Un modèle logistique entraîné sur 50 images en 64 dimensions est typiquement **trop sûr de lui** : il annonce 0,95 là où il se trompe un cas sur cinq. Un seuil de 0,9 sur une probabilité mal calibrée (section 5.2) n'est pas un seuil de 90 % de réussite.
2. **L'erreur devient une donnée.** Une fausse pseudo-étiquette entre dans le jeu d'entraînement avec le même poids qu'une vraie. Le tour suivant, le modèle s'y adapte, et il devient *plus* sûr de lui sur des points voisins : l'erreur se **renforce** au lieu de se corriger.
3. **La classe déjà majoritaire s'étend.** Reprenons le jeu d'étiquettes déséquilibré de 8.1.4 (39 % de « 0 » parmi les étiquettes, contre 10 % dans la population). Regardons de quelles classes sont les pseudo-étiquettes ajoutées.

En moyenne, **88 %** des pseudo-étiquettes ajoutées sont des « 0 » (et 85 % de ces points sont réellement des 0 : le modèle ne se trompe pas beaucoup sur eux), alors que les « 0 » ne forment que 10 % des images. Les points que le modèle juge sûrs sont, de très loin, ceux de la classe qu'il a le plus vue : il l'ajoute massivement à son jeu d'entraînement, le déséquilibre initial **s'aggrave** (c'est l'explication la plus plausible, que nous n'avons pas isolée par une expérience dédiée), et la précision tombe à 0,65 (section 8.1.4).

Quand l'auto-apprentissage peut fonctionner. Il a ses bons cas : quand le modèle initial est déjà **bon** (de l'ordre de 90 % de précision) et que les groupes sont bien séparés, ajouter ses cas faciles élargit un peu la base sans la polluer. Mais c'est exactement le cas où l'on a le *moins* besoin du semi-supervisé. Avec très peu d'étiquettes et un modèle médiocre, il est le

plus dangereux.

Quelques garde-fous usuels : un seuil **élevé** ; une probabilité **calibrée** (section 5.2) ; ajouter des pseudo-étiquettes **par classe** en proportion de leur fréquence attendue plutôt que par seuil global ; ne jamais s'évaluer sur les pseudo-étiquettes ; et surtout **comparer au supervisé seul sur les mêmes étiquettes**.

8.2.3 Le co-apprentissage

Le **co-apprentissage** (*co-training*) est une variante qui limite le biais de confirmation en faisant s'entraider **deux** modèles. On suppose que chaque observation possède **deux** « vues » $x = (x^{(1)}, x^{(2)})$, c'est-à-dire deux jeux de variables, et que :

1. **chaque vue suffit** à prédire la classe (chacune porte assez d'information pour un modèle correct) ;
2. les deux vues sont **indépendantes sachant la classe**.

On entraîne un modèle par vue ; chacun étiquette les points sans étiquette dont il est le plus sûr, et **ces pseudo-étiquettes servent à entraîner l'autre**. L'idée est qu'une erreur du premier modèle est, par indépendance, un cas « ordinaire » pour le second, qui peut la corriger.

L'exemple de référence est celui d'une page web, décrite par son **texte** et par les **liens qui pointent vers elle**. Pour la boutique, on pourrait imaginer décrire un client par son **comportement d'achat** d'un côté et par son **profil déclaré** (âge, ville) de l'autre, si chacun suffisait à prédire un attribut.

Les conditions sont **exigeantes** et rarement réunies. Sur nos chiffres manuscrits, on peut tenter de prendre comme vues la **moitié haute** et la **moitié basse** de l'image : aucune des deux moitiés ne suffit à elle seule, et elles sont fortement dépendantes (elles décrivent le même tracé). Le cahier propose cet essai (application 8.4) ; il se solde par un **échec instructif**, bien pire que le supervisé seul. C'est la leçon à retenir : une méthode dont les hypothèses ne sont pas vérifiées n'est pas « un peu moins efficace », elle peut être franchement nuisible.

8.2.4 Propager les étiquettes le long d'un graphe

Changeons de point de vue. Au lieu d'un modèle qui se prédit lui-même, construisons un **graphe de similarité** entre *tous* les points (étiquetés ou non) et laissons les étiquettes **se diffuser** de proche en proche, comme de l'encre dans un réseau de canaux.

Le graphe. Chaque observation est un **nœud**. On relie deux nœuds proches : par exemple chaque point à ses k plus proches voisins. On obtient une matrice de **poids** W ($W_{ij} > 0$ si i et j sont voisins, 0 sinon, W symétrique), les **degrés** $d_i = \sum_j W_{ij}$, et la matrice normalisée

$$S = D^{-1/2} W D^{-1/2}, \quad S_{ij} = \frac{W_{ij}}{\sqrt{d_i d_j}}.$$

La diffusion. Notons Y la matrice des étiquettes ($n \times c$) : la ligne i vaut le vecteur indicateur de la classe si i est étiqueté, et zéro sinon. On part de $F^{(0)} = Y$ et on répète

$$F^{(t+1)} = \alpha S F^{(t)} + (1 - \alpha) Y, \quad \alpha \in]0, 1[.$$

À chaque tour, chaque nœud reçoit une moyenne pondérée des scores de ses voisins (le terme $\alpha S F$), tout en gardant une part $(1 - \alpha)$ de son étiquette d'origine (ce qui empêche les étiquettes connues de se diluer). À la fin, chaque nœud reçoit la classe de **plus grand score** : $\hat{y}_i = \arg \max_c F_{ic}$. C'est l'algorithme de **propagation** (ou d'étalement, *label spreading*) de Zhou et coll.

Pourquoi cela converge, et vers quoi. Le point clé est que les valeurs propres de S sont dans $[-1, 1]$. En effet, $S = D^{-1/2}WD^{-1/2}$ est **semblable** à $P = D^{-1}W$ (puisque $S = D^{1/2}PD^{-1/2}$), et P est une matrice **stochastique** (ses lignes sont positives et somment à 1), dont les valeurs propres sont de module au plus 1. Le rayon spectral de αS est donc au plus $\alpha < 1$. En dépliant la récurrence :

$$F^{(t)} = (\alpha S)^t Y + (1 - \alpha) \sum_{s=0}^{t-1} (\alpha S)^s Y \xrightarrow{t \rightarrow \infty} F^* = (1 - \alpha) (I - \alpha S)^{-1} Y,$$

car $(\alpha S)^t \rightarrow 0$ et la série de Neumann $\sum_s (\alpha S)^s$ converge vers $(I - \alpha S)^{-1}$. **On n'a donc pas besoin d'itérer** : une résolution de système linéaire suffit.

Ce que l'on minimise. F^* est l'unique minimiseur de

$$J(F) = \frac{1}{2} \sum_{i,j} W_{ij} \left\| \frac{F_i}{\sqrt{d_i}} - \frac{F_j}{\sqrt{d_j}} \right\|^2 + \mu \|F - Y\|_F^2, \quad \mu = \frac{1-\alpha}{\alpha}.$$

Le premier terme est un terme de **lissage** : il est petit quand deux nœuds proches ont des scores proches (c'est l'hypothèse de lissage de 8.1.3). Le second est un terme de **fidélité** : il demande de ne pas trop s'éloigner des étiquettes connues. En développant, le premier terme vaut $\text{tr}(F^\top (I - S)F)$, et annuler le gradient donne $(I - S)F + \mu(F - Y) = 0$, soit $F = \frac{\mu}{1+\mu} (I - \frac{1}{1+\mu} S)^{-1} Y$, ce qui est exactement F^* avec $\alpha = 1/(1 + \mu)$.

Un exemple à la main : six nœuds. Deux triangles reliés par une seule arête : les nœuds 1, 2, 3 d'un côté, 4, 5, 6 de l'autre, et l'arête 3–4 au milieu. Le nœud 1 est étiqueté **A**, le nœud 6 est étiqueté **B**, les quatre autres ne le sont pas. Les degrés sont $d = (2, 2, 3, 3, 2, 2)$. Les poids normalisés dont nous avons besoin sont

$$S_{12} = \frac{1}{\sqrt{2 \cdot 2}} = 0,5, \quad S_{13} = S_{23} = \frac{1}{\sqrt{2 \cdot 3}} \approx 0,408, \quad S_{34} = \frac{1}{\sqrt{3 \cdot 3}} = \frac{1}{3},$$

et symétriquement de l'autre côté ($S_{56} = 0,5$, $S_{45} = S_{46} \approx 0,408$). Prenons $\alpha = 0,5$. Calculons le premier tour pour la colonne de la classe A, en partant de $F_A^{(0)} = (1, 0, 0, 0, 0, 0)$:

- nœud 1 : $0,5 \cdot (S_{12} \cdot 0 + S_{13} \cdot 0) + 0,5 \cdot 1 = 0,5$;
- nœud 2 : $0,5 \cdot S_{21} \cdot 1 + 0 = 0,5 \cdot 0,5 = 0,25$;
- nœud 3 : $0,5 \cdot S_{31} \cdot 1 = 0,5 \cdot 0,408 \approx 0,204$;
- nœuds 4, 5, 6 : aucun voisin n'a encore de score A : 0.

Le score A est passé du nœud 1 à ses deux voisins, **atténué** par la distance. Au deuxième tour, il atteint le nœud 4 (à travers l'arête 3–4), toujours très faiblement. Les deux colonnes se calculent ensemble ; en laissant converger (ou en résolvant directement F^*), on obtient :

Nœud	1	2	3	4	5	6
Score A	0,577	0,177	0,159	0,030	0,008	0,008
Score B	0,008	0,008	0,030	0,159	0,177	0,577
Classe	A	A	A	B	B	B

La classe attribuée à chaque nœud correspond à la **forme du graphe** : les trois nœuds du triangle de gauche reçoivent A, ceux de droite reçoivent B. Remarquez que le nœud 3, voisin direct du nœud 4 et donc exposé aux deux classes, penche malgré tout (0,159 contre 0,030) du côté de son triangle : l'étiquette s'est propagée **à l'intérieur** des groupes bien plus qu'**entre** eux, parce qu'il y

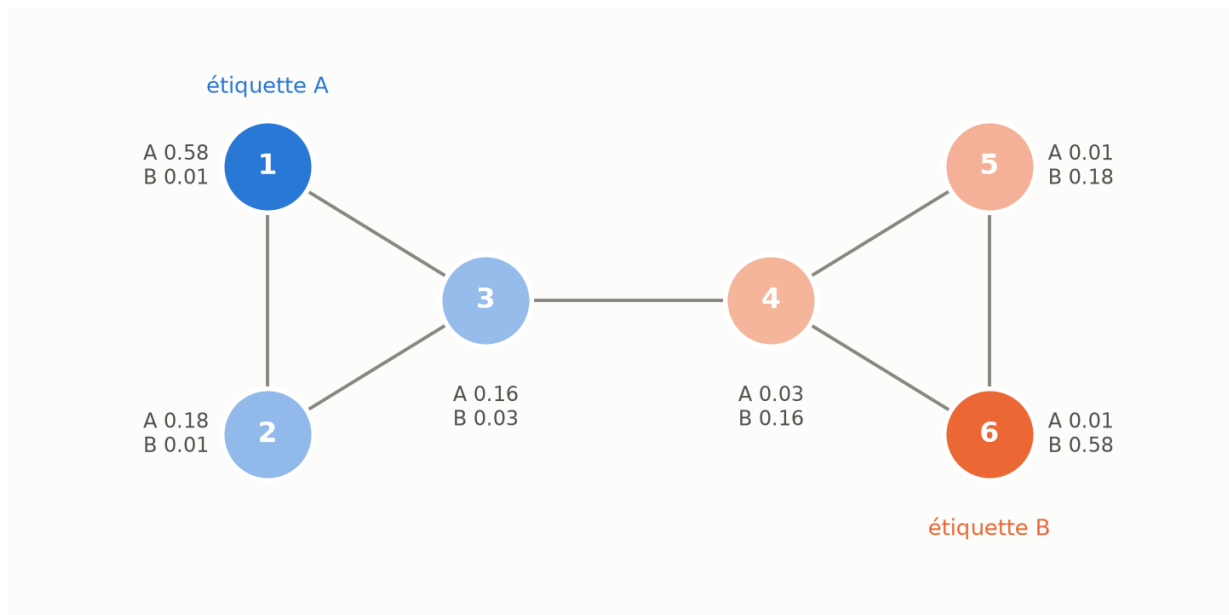


Figure 86: Propagation d'étiquettes sur six nœuds : deux triangles reliés par une arête. Seuls les nœuds 1 (classe A) et 6 (classe B) sont étiquetés. Les scores finaux (A et B) sont indiqués sous chaque nœud ; l'intensité de la couleur suit le score le plus élevé.

a beaucoup de liens dans chaque triangle et un seul au milieu. C'est exactement l'hypothèse des groupes de 8.1.3.

8.2.5 La propagation sur les chiffres manuscrits

Passons à la pratique. Pour les chiffres, le graphe relie chaque image à ses **7 plus proches voisins** (distance euclidienne entre les 64 pixels). Voici l'appel, comme pour l'auto-apprentissage :

```
from sklearn.semi_supervised import LabelSpreading

prop = LabelSpreading(kernel="knn", n_neighbors=7, alpha=0.2).fit(Xp, y_partiel)
print("propagation :", round(prop.score(Xt, yt), 3), "| auto-apprentissage :", round(auto.score(Xt, yt), 3), "| supervisé seul :", round(seul.score(Xt, yt), 3))
```

```
propagation : 0.949 | auto-apprentissage : 0.831 | supervisé seul : 0.831
```

Sur ce même tirage de 50 étiquettes, la propagation atteint **0,95** de précision, contre 0,83 pour le supervisé seul et pour l'auto-apprentissage. Un seul tirage ne prouve rien : voici la courbe d'apprentissage complète, avec les **mêmes étiquettes** pour les trois méthodes, 10 tirages par budget.

Étiquettes	10	20	50	100	200
Supervisé seul	0,429	0,577	0,782	0,874	0,927
Auto-apprentissage ($\tau = 0,9$)	0,429	0,577	0,706	0,840	0,923
Propagation	0,540	0,779	0,913	0,940	0,966

Trois constats :

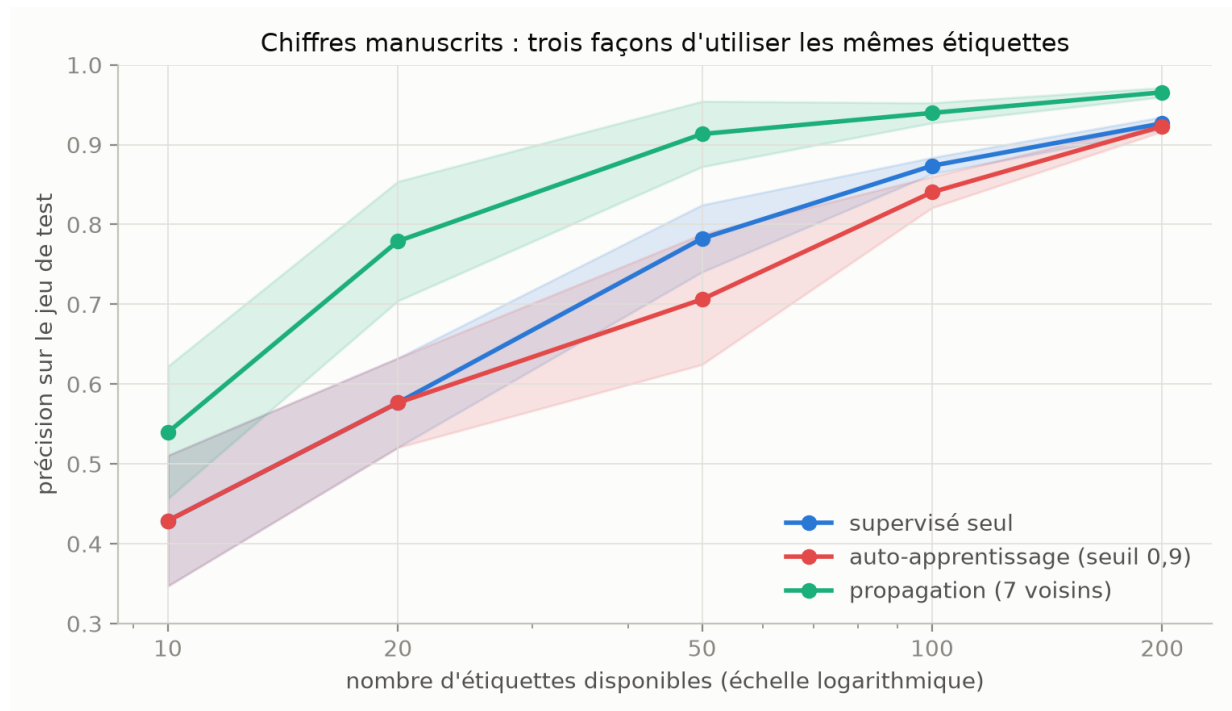


Figure 87: Précision sur le jeu de test des chiffres manuscrits selon le nombre d'étiquettes, pour le modèle supervisé seul, l'auto-apprentissage et la propagation d'étiquettes (moyenne et écart-type sur 10 tirages, mêmes étiquettes pour les trois méthodes). La propagation domine nettement à petit budget ; l'auto-apprentissage ne fait pas mieux que le supervisé.

- Avec **20 étiquettes**, la propagation gagne **20 points** de précision sur le modèle supervisé (0,78 contre 0,58) ; avec 50, **13 points**. C'est considérable, et le gain est **plus grand là où la courbe supervisée est la plus raide** (8.1.5).
- Le gain **diminue avec le budget** : à 200 étiquettes, 4 points. Quand on a beaucoup d'étiquettes, la structure du graphe n'apprend plus grand-chose de neuf.
- La propagation fait mieux que le supervisé dans **les 10 tirages, à chacun des cinq budgets**.
- L'auto-apprentissage ne fait **jamais mieux** que le supervisé : il est à égalité aux petits budgets (le modèle n'est presque jamais sûr de lui) puis moins bon.

Transductif ou inductif ? Avec 50 étiquettes, la précision de la propagation sur les points sans étiquette *du réservoir* (c'est l'usage **transductif**) et sur le jeu de test (l'usage **inductif**, via `predict`) sont très voisines : respectivement 0,90 et 0,91 en moyenne. Pour un modèle destiné à prédire de nouveaux clients demain, il faut prévoir **de re-propager** (ou de remplacer la propagation par un classifieur entraîné sur ses étiquettes) : le graphe n'existe que sur les points qu'il contient.

Les réglages comptent, surtout le nombre de voisins. Avec 20 étiquettes, voici la précision selon k (nombre de voisins) et α :

	$\alpha = 0,2$	$\alpha = 0,9$
voisins k		
3	0.290	0.309
7	0.779	0.780
15	0.785	0.780
30	0.775	0.754
60	0.739	0.685

- $k = 3$ est **catastrophique** (0,29). Le graphe de scikit-learn est **orienté** : chaque image ne regarde que ses k voisins, et une étiquette ne peut atteindre une image que si celle-ci compte un point déjà atteint parmi ses k plus proches voisins. Avec $k = 3$ et 20 étiquettes, **86 %** des

images non étiquetées ne reçoivent **aucun score** (13 % avec $k = 7$, 0,5 % avec $k = 15$) : la méthode ne sait plus que dire. (Un graphe **symétrisé**, où l'on relie deux images dès que l'une est voisine de l'autre, ne souffre pas de ce défaut : voir l'application 8.3 du cahier.)

- De $k = 7$ à $k = 30$, le résultat est stable et bon (entre 0,75 et 0,79). C'est la zone où le graphe est connexe sans mélanger les classes.
- À $k = 60$, les voisins deviennent trop lointains : des liens traversent les frontières entre chiffres et la précision recule (0,74 pour $\alpha = 0,2$, 0,69 pour $\alpha = 0,9$).
- Le paramètre α compte peu tant que k est raisonnable.

Le coût. Un graphe à noyau gaussien sur n points est une matrice $n \times n$ dense : pour $n = 100\,000$, c'est 10^{10} nombres, hors de portée. Le noyau à k **plus proches voisins** (utilisé ici) est **creux** et passe à l'échelle. Quant au choix de k , on ne peut pas le régler par validation sur 20 étiquettes : on le choisit par principe (graphe connexe, k de l'ordre de 7 à 15) et on le contrôle avec le diagnostic de la section suivante.

8.2.6 Quand le graphe ne dit rien : les clients de la boutique

La propagation brille sur les chiffres. Qu'en est-il sur les **clients de la boutique**, où l'on veut prédire la résiliation à 90 jours avec très peu d'étiquettes ? Même protocole : étiquettes tirées avec leurs proportions (14 % de résiliations ; au moins 2 positifs), mesure par l'**aire sous la courbe ROC** (AUC, section 5.1) puisque les classes sont déséquilibrées.

Étiquettes	Supervisé (AUC, test)	Auto- apprentissage (AUC, test)	Supervisé (AUC, points atteints)	Propagation (AUC, points atteints)	Part du réservoir atteinte
30	0,719	0,718	0,705	0,510	74 %
100	0,734	0,731	0,727	0,527	96 %
300	0,791	0,792	0,787	0,577	100 %

(La propagation est évaluée sur les clients non étiquetés que les étiquettes ont atteints, avec le supervisé évalué sur ces mêmes clients pour la comparaison : il y est à peu près identique à son résultat sur le test.)

Le verdict est net : l'auto-apprentissage **ne change rien**, et la propagation est à **peine meilleure que le hasard** (AUC de 0,51 à 0,58, pour 0,5 d'un tirage au sort) là où le supervisé atteint 0,71 à 0,79. Pourquoi ? Parce que l'hypothèse de 8.1.3 est fausse ici. Un diagnostic simple le montre : à quel point les voisins d'un point partagent-ils son étiquette ?

	Voisins de même étiquette	Part attendue au hasard
Chiffres (7 voisins)	96,9 %	10,0 %
Clients (7 voisins)	80,5 %	75,9 %

Sur les chiffres, **97 %** des 7 plus proches voisins d'une image portent le même chiffre : le graphe est presque parfaitement « propre ». Sur les clients, **80,5 %** seulement, à peine plus que les **75,9 %** qu'on obtiendrait en choisissant des voisins au hasard (puisque 86 % des clients ne résilient pas). Parmi les clients qui résilient, 27 % seulement de leurs voisins résilient aussi (pour un taux de base de 14 %) : le lien existe, mais il est faible. La distance entre deux clients, calculée sur 49 variables (dont 34 colonnes indicatrices de villes, de canaux, d'appareils et de catégories) de natures très différentes, **ne reflète pas ce qui fait résilier** : un seuil sur la récence, une interaction

entre tickets de support et retours, c'est-à-dire des seuils et des interactions que les arbres du chapitre 2 savent capturer, mais pas une distance. Le graphe relie des clients qui se ressemblent *en apparence*, pas des clients qui se ressemblent *par leur risque*.

Un test à faire avant de propager. Même avec peu d'étiquettes, on peut estimer ce taux d'**homophilie** : parmi les points *étiquetés* qui sont voisins l'un de l'autre, quelle part partage la même étiquette ? Comparez-le à la part attendue au hasard. S'il est proche, ne comptez pas sur la propagation. Une autre piste est d'apprendre **d'abord** une représentation adaptée (par exemple un modèle supervisé sur les étiquettes disponibles, puis un graphe construit sur ses sorties), mais cela sort du cadre de ce chapitre.

À retenir.

- L'**auto-apprentissage** ajoute les pseudo-étiquettes dont le modèle est sûr. Avec peu d'étiquettes et un modèle médiocre, il se confirme dans ses erreurs et **ne bat pas** le supervisé seul (chiffres, 50 étiquettes : jamais mieux que 0,78).
- Les erreurs **se renforcent** (biais de confirmation) ; une classe déjà majoritaire s'étend. Garde-fous : seuil élevé, probabilités calibrées, comparaison systématique au supervisé sur les mêmes étiquettes.
- Le **co-apprentissage** demande deux vues suffisantes et indépendantes : condition rarement réunie.
- La **propagation d'étiquettes** diffuse les étiquettes sur un graphe de similarité ; elle se calcule en fermé, $F^* = (1 - \alpha)(I - \alpha S)^{-1}Y$, et minimise lissage + fidélité. Sur les chiffres, **+20 points** à 20 étiquettes ; le réglage de k est critique ($k = 3$ morcelle le graphe).
- Elle suppose que **les voisins se ressemblent par la classe** : sur les clients (80,5 % de voisins de même étiquette contre 75,9 % au hasard), elle échoue. **Mesurez l'homophilie avant de propager.**

Pour s'entraîner. Cahier, chapitre 8 : applications 8.2 à 8.5, exercices 8.4 à 8.6.

8.3 Apprentissage actif

L'apprentissage semi-supervisé exploite ce qu'on possède déjà. L'**apprentissage actif** pose l'autre question : *puisqu'il faut payer pour chaque étiquette, lesquelles acheter ?* Au lieu de faire étiqueter des exemples tirés au hasard, on laisse le **modèle choisir** ceux dont il a le plus à apprendre.

8.3.1 L'idée et la boucle

Imaginez un élève qui prépare un examen avec un professeur disponible une heure. Un élève passif écoute ce que le professeur choisit de dire. Un élève **actif** pose les questions sur ce qu'il ne comprend pas encore. Pour le même temps, il apprend plus vite. L'apprentissage actif applique cette idée à un modèle : il demande l'étiquette des points qui, croit-il, lui apprendront le plus.

On distingue trois cadres ; nous travaillerons dans le premier, le plus courant :

Cadre	Principe	Exemple
Par réservoir (<i>pool-based</i>)	on dispose d'un grand ensemble non étiqueté ; à chaque tour, on choisit dans cet ensemble ce qu'on fait étiqueter	les 9 000 clients dont on ignore le comportement futur
En flux (<i>stream-based</i>)	les points arrivent un à un ; on décide à la volée de demander ou non leur étiquette	des transactions qui défilent
Par requêtes synthétiques	le modèle <i>invente</i> le point dont il veut l'étiquette	rarement possible : un humain doit savoir étiqueter un point qui n'existe pas

La boucle d'apprentissage actif (par réservoir, par lots de b points) :

1. Choisir un petit **jeu initial** et le faire étiqueter ; entraîner un modèle f .
2. Calculer, pour chaque point x non étiqueté, un **score d'utilité** $u(x)$ (nous en voyons plusieurs en 8.3.2 à 8.3.4).
3. Faire étiqueter les b points de **plus grand score** ; les ajouter au jeu étiqueté.
4. Réentraîner f . Si le budget n'est pas épuisé, retourner à l'étape 2.

Tout l'art est dans le choix du score u . Pour la stratégie de la marge (8.3.2), l'étape 2 se résume à quelques lignes :

```
P = modele.predict_proba(X_libres)           # probabilités du modèle sur les points non
↳ étiquetés
tri = np.sort(P, axis=1)
marge = tri[:, -1] - tri[:, -2]              # écart entre les deux classes les plus probables
a_demander = libres[np.argsort(marge)[:10]]  # les 10 points sur lesquels le modèle hésite le
↳ plus
```

Deux précautions dès le départ. Un modèle appris sur des points *choisis* n'est plus appris sur un échantillon représentatif : cela a des conséquences sur la mesure de sa qualité et sur ses probabilités (8.3.6). Et au début, un modèle presque ignorant ne sait pas bien ce qui l'informerait : c'est le problème du démarrage à froid (8.3.7).

8.3.2 Mesurer l'incertitude

Le critère le plus naturel : demander les points sur lesquels le modèle **hésite**. Pour un point x , le modèle donne des probabilités $P(c | x)$ pour chaque classe c . Trois façons de mesurer l'hésitation :

Critère	Formule (grand = plus incertain)	Idée
Confiance minimale (<i>least confidence</i>)	$1 - \max_c P(c x)$	la meilleure classe est-elle peu probable ?
Marge	$-(P(c_1 x) - P(c_2 x))$, où c_1, c_2 sont les deux classes les plus probables	le modèle hésite-t-il entre deux classes ?
Entropie	$-\sum_c P(c x) \ln P(c x)$	les probabilités sont-elles étalées sur toutes les classes ?

Pour deux classes, les trois critères sont **équivalents** : ils classent les points dans le même ordre

(le plus incertain est celui dont la probabilité est la plus proche de $1/2$). Avec trois classes ou plus, ils peuvent **diverger**. Voici trois candidats à étiqueter, pour un problème à trois classes :

Candidat	Probabilités	$1 - \max$	Écart entre les deux meilleures	Entropie
a	(0,50; 0,45; 0,05)	0,50	0,05	0,856
b	(0,40; 0,30; 0,30)	0,60	0,10	1,089
c	(0,60; 0,20; 0,20)	0,40	0,40	0,950

(Par exemple, pour le candidat a : $-(0,5 \ln 0,5 + 0,45 \ln 0,45 + 0,05 \ln 0,05) = 0,347 + 0,359 + 0,150 = 0,856$.) Les trois critères donnent **trois classements différents** :

- la **confiance minimale** met b en premier (sa meilleure classe n'a que 0,40), puis a, puis c ;
- la **marge** met a en premier (le modèle hésite quasiment à pile ou face entre deux classes : 0,50 contre 0,45), puis b, puis c ;
- l'**entropie** met b en premier (probabilités très étalées), puis c, puis a : elle juge le candidat a *moins* incertain que c, parce que la troisième classe est quasi exclue.

Lequel choisir ? La **marge** vise précisément la **frontière de décision** : un point dont deux classes sont à égalité est exactement un point qui déplacerait la frontière si on connaissait sa vraie classe. Quand il y a beaucoup de classes (10 chiffres), l'entropie peut au contraire être élevée pour un point que le modèle sait assigner à deux ou trois candidats parmi dix : une partie de l'étiquette « achetée » est gaspillée sur des classes sans rapport. Nous le vérifierons en 8.3.5.

8.3.3 Le comité de modèles

Une autre façon de repérer un point utile : **demandeur l'avis de plusieurs modèles** et regarder s'ils se disputent. C'est la stratégie du **comité** (*query by committee*).

On construit un comité de M modèles (ici $M = 5$), tous entraînés sur le **jeu étiqueté actuel**, mais rendus différents : chacun est appris sur un **rééchantillonnage avec remise** du jeu étiqueté (bootstrap, volume I, section 3.3.5). Chaque modèle **vote** pour une classe. On mesure le **désaccord** par l'**entropie du vote** : si v_c est la part des modèles qui votent pour la classe c , le score est $-\sum_c v_c \ln v_c$.

Pour un comité de cinq modèles et deux classes :

Répartition des votes	Part des votes	Entropie du vote
5 contre 0	(1; 0)	0
4 contre 1	(0,8; 0,2)	0,500
3 contre 2	(0,6; 0,4)	0,673

L'unanimité donne un score nul ; la division la plus équilibrée, le plus grand. On fait donc étiqueter les cas qui **départagent** les modèles encore plausibles : en réduisant le nombre de modèles compatibles avec les données, chaque étiquette est utile.

Le comité a un **avantage** sur la marge : il mesure l'incertitude *du modèle* (ce qu'il ne sait pas, parce qu'il manque de données) et non seulement l'incertitude *des données* (un point intrinsèquement ambigu, dont aucune étiquette ne réduira le doute). Son **coût** : M entraînements par tour au lieu d'un.

8.3.4 Densité et représentativité

Les critères d'incertitude ont un défaut connu : ils peuvent désigner des **valeurs aberrantes**. Un point très atypique, loin de toutes les autres données, rend le modèle perplexe ; mais l'étiqueter n'apprend presque rien sur les cas courants, qui forment l'essentiel de ce que l'on veut prédire.

Pour limiter ce risque, on pondère l'incertitude par la **densité** : la valeur d'un point est son incertitude *multipliée* par sa ressemblance moyenne avec le reste du réservoir. Un point à la fois **incertain** et **typique** est le meilleur candidat :

$$u(x) = \underbrace{H(x)}_{\text{incertitude}} \times \underbrace{\left(\frac{1}{n} \sum_{x'} \text{sim}(x, x') \right)}_{\text{densité}}^\beta.$$

Ici, la similarité est le cosinus de l'angle entre deux vecteurs, et $\beta = 1$. Une autre famille de critères, que nous ne calculerons pas, vise à **estimer directement la réduction de l'erreur** que procurerait chaque étiquette (*expected error reduction*) : très coûteuse, car elle impose de réentraîner le modèle pour chaque point candidat et chaque étiquette possible.

Représentatif ou informatif ? L'**incertitude** cherche les points *informatifs* (qui déplaceraient la frontière) ; la **densité** cherche les points *représentatifs* (qui ressemblent à la masse). Aucun des deux n'est suffisant seul. En pratique on les combine, ou l'on alterne : un tour sur deux, quelques points tirés au hasard pour garder un contact avec la distribution réelle.

8.3.5 Comparer les stratégies

Comparons six stratégies sur les **chiffres manuscrits**, avec le protocole de 8.1.5 : le même jeu initial de 10 images tirées au hasard pour toutes les stratégies, 10 tirages différents, lots de 10 images, jusqu'à 150 étiquettes, précision mesurée sur le jeu de test de 450 images. Le modèle est la régression logistique.

Stratégie	30 étiquettes	60	100	150	Premier budget où la précision moyenne atteint 0,90
Aléatoire	0,673	0,815	0,890	0,919	130
Incertain	0,607	0,808	0,901	0,938	100
Marge	0,744	0,886	0,928	0,950	80
Entropie	0,576	0,755	0,864	0,917	140
Comité	0,655	0,833	0,898	0,931	110
Entropie × densité	0,527	0,718	0,842	0,901	150

Que lit-on ?

- **La marge est la grande gagnante.** À 60 étiquettes, elle dépasse le tirage au hasard de 7 points (0,886 contre 0,815), et elle le dépasse dans **les 10 tirages**. Elle atteint 0,90 de précision avec **80 étiquettes**, contre **130** pour le tirage au hasard : **38 % d'étiquettes en moins**. Elle est aussi plus **régulière** : l'écart-type d'un tirage à l'autre à 60 étiquettes est de 0,014, contre 0,048 pour le tirage au hasard.
- **Plusieurs stratégies sont pires que le hasard au début.** Avec 30 étiquettes, la confiance minimale (0,607), l'entropie (0,576) et l'entropie pondérée par la densité (0,527) sont **en dessous** du tirage aléatoire (0,673). La confiance minimale passe devant le hasard aux points

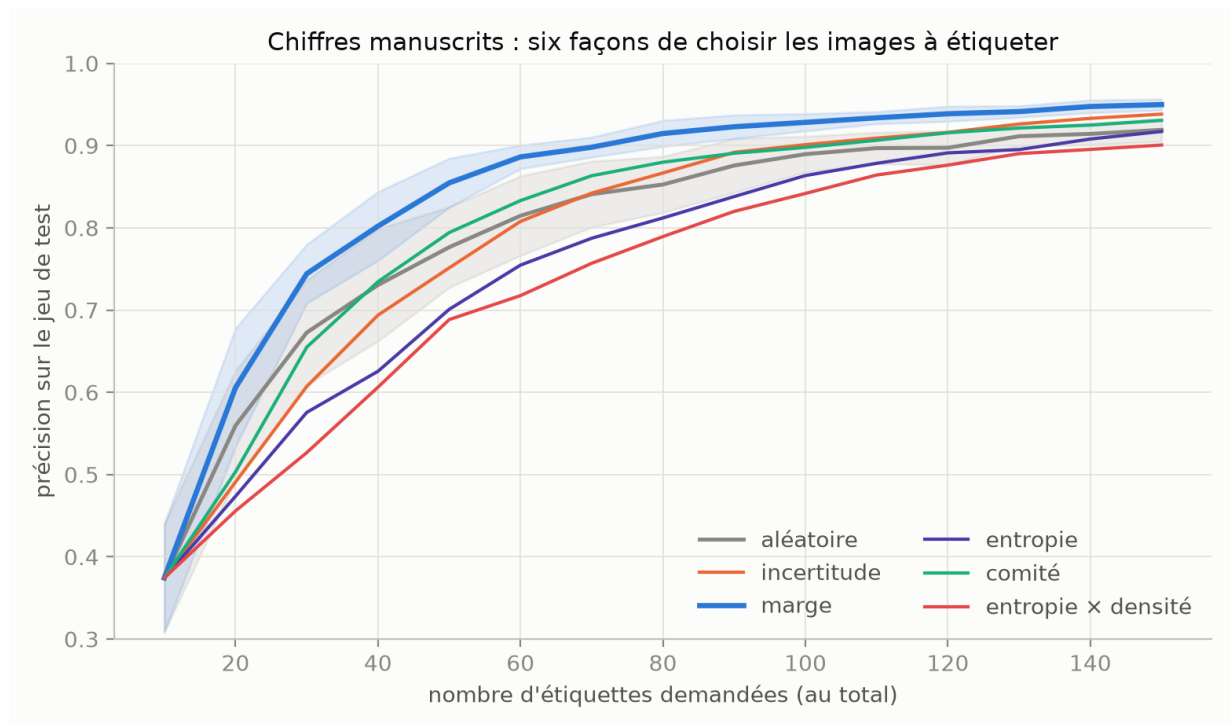


Figure 88: Précision sur le jeu de test des chiffres manuscrits selon le nombre d'étiquettes demandées, pour six stratégies de choix (moyenne sur 10 tirages ; bandes : écart-type pour le tirage au hasard et pour la marge). La marge domine nettement ; l'entropie et l'entropie pondérée par la densité font moins bien que le tirage au hasard.

de contrôle de 100 et 150 étiquettes (0,901 contre 0,890, puis 0,938 contre 0,919) ; l'entropie ne le dépasse jamais dans cette fenêtre (0,917 contre 0,919 à 150).

- **Le comité se place entre les deux** : un peu meilleur que le hasard à 60 étiquettes (0,833 contre 0,815), mais loin de la marge, au prix de cinq entraînements par tour.
- **La densité n'aide pas ici**. L'entropie pondérée par la densité est la **moins bonne** de toutes. Les chiffres manuscrits n'ont presque pas de valeurs aberrantes : le garde-fou coûte plus qu'il ne rapporte. Sur des données bruitées, le verdict pourrait s'inverser.

Pourquoi un critère raisonnable peut-il perdre contre le hasard ? Au début, le modèle n'a vu que quelques images de quelques chiffres. Ses « incertitudes » sont celles d'un modèle ignorant : l'entropie sur dix classes, par exemple, est élevée pour *presque* toute image qui n'est pas proche d'un exemple connu, c'est-à-dire pour beaucoup d'images très différentes, sans que cela dise lesquelles seraient les plus instructives. C'est une explication plausible, que ces expériences ne démontrent pas ; ce qui est établi, c'est le classement, mesuré sur les mêmes tirages. Retenez surtout qu'il faut **mesurer** : aucune stratégie n'est sûre de battre le hasard.

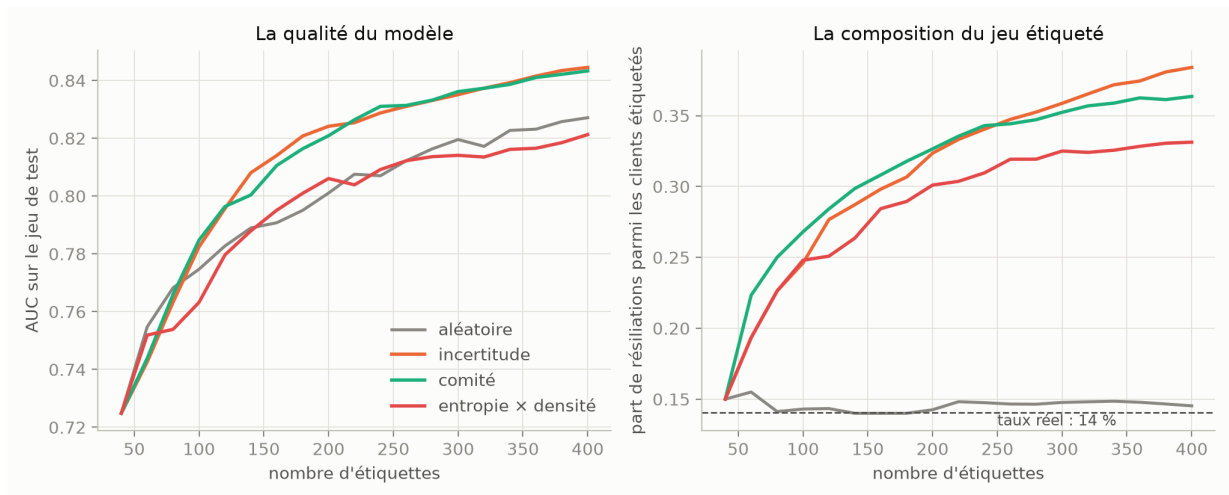


Figure 89: À gauche : AUC sur le jeu de test selon le nombre d'étiquettes, pour quatre stratégies sur les clients de la boutique (moyenne de 10 tirages). À droite : part de résiliations parmi les clients étiquetés ; le tirage au hasard reste au taux réel de 14 %, les stratégies actives s'en éloignent beaucoup.

8.3.6 Le piège du biais d'échantillonnage

Passons aux **clients de la boutique**, où l'on cherche à prédire la résiliation à 90 jours. On démarre avec 40 étiquettes (tirées en respectant la proportion de résiliations, au moins 2), par lots de 20 jusqu'à 400. Le modèle est une régression logistique ; la qualité est mesurée par l'AUC sur un jeu de test de 3 000 clients tirés au hasard.

Deux enseignements.

1. L'apprentissage actif aide, modestement. À 400 étiquettes, l'AUC est de **0,845** pour la confiance minimale et **0,843** pour le comité, contre **0,827** au hasard. Pour atteindre 0,80 d'AUC, il faut **200** étiquettes au hasard mais **140** avec l'une ou l'autre stratégie active (30 % de moins). La densité, ici encore, n'aide pas (180 étiquettes).

2. Mais le jeu étiqueté n'est plus représentatif, et ses probabilités non plus. Le graphique de droite le montre : parmi les 400 clients étiquetés par la stratégie d'incertitude, **38 %** résilient, alors que le taux réel est de **14 %**. Normal : en demandant les clients « frontière », on est tombé sur beaucoup de cas intermédiaires. Le tirage au hasard, lui, reste fidèle (14,5 %). Conséquence concrète sur les **probabilités prédites** : la probabilité moyenne de résiliation prédite sur le jeu de test est de **0,078** pour le modèle actif, alors que la vérité est **0,14** : le modèle **sous-estime** le risque de 44 %. Celui entraîné au hasard donne 0,142. L'ordre des clients (l'AUC) est meilleur ; les **niveaux** de probabilité, eux, sont faux.

Trois règles d'hygiène.

1. **Le jeu de test doit être tiré au hasard** et n'avoir *jamais* été utilisé pour choisir des points. Évaluer un modèle actif sur des points actifs serait doublement trompeur.
2. **On ne peut pas estimer un taux (de résiliation, de fraude...) sur le jeu étiqueté.** Celui-ci a été choisi pour être atypique.
3. **Si l'on a besoin de probabilités fiables**, il faut les **recalibrer** (section 5.2) sur un petit échantillon étiqueté **tiré au hasard**.

Essayons la règle 3 : on fait étiqueter **300 clients de plus, au hasard**, et l'on recalcule les probabilités du modèle par une régression logistique à une variable (« mise à l'échelle de Platt », section 5.2). Après recalibrage, la probabilité moyenne du modèle actif passe de **0,078 à 0,141** (le taux réel est

0,14), et son **score de Brier** (l'erreur quadratique moyenne des probabilités, section 5.1) de **0,1040 à 0,0920**. Le modèle tiré au hasard passe de 0,1014 à 0,0959 : le modèle actif, une fois recalibré, est le meilleur des deux. Mais ces 300 étiquettes ont un **coût** : nous y revenons en 8.3.8.

8.3.7 Le démarrage à froid

Reste la question du **premier lot**. Un modèle sans données ne sait pas ce qu'il ignore. Deux phénomènes se combinent.

Il manque des classes. Sur les chiffres, 10 images tirées au hasard ne couvrent en moyenne que **6,5 chiffres sur 10** ; le modèle ne peut tout simplement pas prédire les classes qu'il n'a jamais vues, et sa précision initiale n'est que de **0,37**. Sur les clients, la probabilité qu'un tirage de 10 clients ne contienne **aucun résilisateur** est de $0,86^{10} \approx 0,22$: l'entraînement échouerait une fois sur cinq.

Une parade simple : des médoïdes. On regroupe d'abord les données non étiquetées avec les k-means (volume II, section 3.3), puis on fait étiqueter, pour chaque groupe, l'image **la plus proche de son centre** (le *médoïde*). Avec 10 groupes sur les chiffres, les 10 médoïdes couvrent en moyenne **9,2 chiffres** et donnent une précision de départ de **0,71**, contre 0,37 pour 10 étiquettes au hasard.

Mais la suite est plus nuancée. Si l'on poursuit ensuite avec la stratégie de la marge, l'avantage des médoïdes **fond vite** :

Étiquettes	10	30	60	100
Marge, départ aléatoire	0,374	0,744	0,886	0,928
Marge, départ par médoïdes	0,713	0,781	0,887	0,926

À 60 étiquettes, les deux démarrages sont **à égalité** : la stratégie active corrige elle-même un mauvais départ en quelques tours. Le démarrage à froid pèse donc surtout lorsque le budget est **très petit**, ou lorsqu'il manque une **classe rare** (comme les résiliateurs) que la stratégie ne risque pas de découvrir seule : on garantit alors sa présence en incluant, par exemple, quelques clients déjà connus pour avoir résilié.

8.3.8 Combien ça rapporte ?

L'apprentissage actif n'est pas gratuit : il faut un système capable de **réentraîner** et de **servir** des questions à des annotateurs qui attendent, et chaque tour est un aller-retour. Pour savoir s'il en vaut la peine, on raisonne en **euros**.

Hypothèses de calcul (inventées pour l'illustration, à remplacer par vos coûts réels) : faire étiqueter une image de chiffre coûte **0,40 €** ; faire étiqueter un client (par un appel) coûte **4 €**.

	Étiquettes pour atteindre le but	Coût d'étiquetage
Chiffres , précision $\geq 0,90$: tirage au hasard	130	52 €
Chiffres , précision $\geq 0,90$: marge	80	32 €
Clients , AUC $\geq 0,80$: tirage au hasard	200	800 €
Clients , AUC $\geq 0,80$: incertitude	140	560 €

Les économies : **20 €** (38 %) sur les chiffres, **240 €** (30 %) sur les clients. Mais regardons le piège de 8.3.6 : si l'on a besoin de **probabilités calibrées**, le modèle actif exige 300 étiquettes aléatoires supplémentaires, soit **1 200 €**, bien plus que les 240 € économisés. Le modèle tiré au hasard, lui, est déjà calibré (0,142 de probabilité moyenne). La conclusion dépend donc de l'**usage** :

- si l'on veut seulement **classer** les clients (appeler les 10 % les plus à risque), l'AUC suffit, et l'apprentissage actif **fait économiser** ;
- si l'on veut **chiffrer** le risque (calculer une perte attendue en euros), le coût de recalibrage peut **annuler** le gain.

Quand s'arrêter ? Le gain de précision par étiquette décroît : plus on avance, moins une étiquette supplémentaire rapporte. Sur les chiffres, voici le gain par tranche de 10 étiquettes :

Tranche	60 → 70	100 → 110	140 → 150
Tirage au hasard	+ 2,6 points	+ 0,7 point	+ 0,5 point
Marge	+ 1,2 point	+ 0,6 point	+ 0,2 point

(Le gain de la marge est plus faible en valeur absolue parce qu'elle part déjà plus haut.) Une règle simple : **continuer tant que le gain attendu vaut plus que le coût**. Supposons qu'un point de précision supplémentaire vaille 5 € (hypothèse) : dix étiquettes coûtent 4 €, il faut donc un gain d'**au moins 0,8 point** pour 10 étiquettes. Avec la marge, à 60 étiquettes le gain (1,2 point, soit 5,9 €) dépasse le coût (4 €) : on continue ; à 100 étiquettes (0,6 point, soit 2,8 €), non : on s'arrête, autour de **80 à 100 étiquettes**.

À retenir.

- L'**apprentissage actif** fait étiqueter les points les plus utiles : boucle *entraîner, scorer, demander, recommencer*. Il vise le même niveau avec **moins d'étiquettes**.
- **Critères** : confiance minimale, **marge** (hésitation entre deux classes), entropie, **comité** (désaccord de modèles), pondération par la **densité** (éviter les points aberrants). Les critères diffèrent dès trois classes ; sur les chiffres, la **marge** gagne nettement (80 étiquettes pour 0,90 au lieu de 130), l'entropie et la densité font **moins bien que le hasard**.
- Le jeu étiqueté n'est pas un échantillon représentatif : jeu de test **aléatoire**, pas d'estimation de taux sur le jeu étiqueté, **recalibrage** sur un petit échantillon aléatoire (probabilité moyenne 0,078 avant, 0,141 après).
- **Démarrage à froid** : 10 étiquettes aléatoires couvrent 6,5 classes sur 10 ; des **médoïdes** (9,2 classes) aident surtout à très petit budget.
- Raisonner en **euros** : l'économie d'étiquettes peut être annulée par le coût de recalibrage ; s'arrêter quand le gain attendu ne couvre plus le coût.

Pour s'entraîner. Cahier, chapitre 8 : applications 8.6 à 8.8, exercices 8.7 à 8.12.

Bilan du chapitre 8

Vous savez maintenant :

- **situer** les deux familles de méthodes qui répondent au problème de l'**étiquette chère** : le **semi-supervisé**, qui exploite les données déjà disponibles mais sans étiquette, et l'**apprentissage actif**, qui choisit quelles étiquettes acheter ;
- **énoncer** les hypothèses qui permettent aux points sans étiquette d'aider (lissage, groupes, basse densité, variété), voir sur un exemple à la main comment un déplacement de frontière en résulte, et **reconnaître les cas où elles nuisent** (classes qui se chevauchent, étiquettes déséquilibrées, variables hétérogènes) ;

- **évaluer honnêtement** une méthode par une **courbe d'apprentissage selon le budget d'étiquettes** : jeu de test aléatoire et fixe, tirages répétés, **mêmes étiquettes** pour toutes les méthodes, comparaison au supervisé seul ;
- **expliquer** l'**auto-apprentissage** et son biais de confirmation, le **co-apprentissage** et ses conditions, et la **propagation d'étiquettes** (diffusion sur un graphe, solution fermée $F^* = (1 - \alpha)(I - \alpha S)^{-1}Y$, minimisation d'un critère de lissage et de fidélité), la calculer **à la main** sur six nœuds ;
- **diagnostiquer** par l'**homophilie** (part de voisins de même étiquette) si un graphe a des chances d'aider : oui sur les chiffres (97 %), non sur les clients de la boutique (80,5 % contre 75,9 % au hasard) ;
- **écrire** la boucle d'apprentissage actif, **calculer** les critères d'incertitude (confiance minimale, **marge**, entropie), le **comité** et la pondération par la **densité**, et mesurer lequel est le meilleur *sur votre problème* (chiffres : la marge, 80 étiquettes pour 0,90 au lieu de 130) ;
- **éviter** les pièges de l'apprentissage actif : jeu étiqueté non représentatif et probabilités faussées (0,078 prédit pour 0,14 réel), jeu de test à tirer au hasard, **démarrage à froid** ;
- **raisonner en coût** : économie d'étiquettes contre coût de recalibrage, et règle d'arrêt « continuer tant que le gain attendu vaut plus que le coût ».

Deux messages à garder. **Le semi-supervisé et l'actif ne sont pas des baguettes magiques** : chacun repose sur une hypothèse sur les données, et la seule façon de savoir si elle tient est de **mesurer** à budget d'étiquettes égal. Et **les données ne sont pas des étiquettes** : tout le travail de ce chapitre vient de ce qu'on sait *faire* de ce qu'on possède (la structure) et de ce qu'on *décide d'acheter* (les étiquettes).

Pour s'entraîner. Cahier, chapitre 8 : applications 8.1 à 8.8 et exercices 8.1 à 8.12.

Le chapitre 9, également facultatif, change de décor : au lieu d'apprendre à partir d'exemples étiquetés, un agent apprend à **décider** en interagissant avec un environnement, c'est l'apprentissage par renforcement. Les bandits manchots y retrouvent une idée de ce chapitre : choisir *quoi essayer* pour apprendre le plus vite.

Chapitre 9 : ⊕ Bases de l'apprentissage par renforcement

« Personne n'apprend à faire du vélo en lisant un manuel : on tombe, on ajuste, on recommence. »

Chapitre complémentaire. Ce chapitre est entièrement facultatif : on peut lire le volume sans lui. Il suppose le chapitre 1 (démarche d'évaluation) et s'appuie sur trois résultats des volumes précédents : les chaînes de Markov (volume I, section 2.6.2), le test A/B (volume I, section 3.4.5) et la mise à jour bayésienne d'une probabilité (volume II, section 6.1). Il est indépendant des chapitres 2 à 8.

Jusqu'ici, dans ce volume, un modèle recevait un **tableau de données déjà constitué** et apprenait à prédire une colonne. Personne ne se demandait *comment* ces données avaient été produites. Dans la vie d'une boutique, pourtant, beaucoup de questions sont d'une autre nature :

- **Quelle bannière afficher** sur la page d'accueil, sachant que l'on ne connaît pas à l'avance celle qui convertit le mieux, et que chaque affichage « gaspillé » sur une mauvaise bannière coûte des ventes ?
- **Quand et combien commander** pour ne pas tomber en rupture, sans immobiliser de la marchandise qui ne se vendra pas ?
- **Quelle offre envoyer** à quel client, et à quel moment du cycle de vie, pour qu'il devienne fidèle ?

Dans chaque cas, on **agit**, le monde **répond** (une vente, une rupture, un abandon), et la réponse dépend de ce que l'on a fait. Les données que l'on recueillera demain dépendent des décisions d'aujourd'hui. C'est le terrain de l'**apprentissage par renforcement** (*reinforcement learning*, RL) : apprendre, par essais et erreurs, **une façon d'agir** qui maximise une récompense cumulée.

Le chemin de ce chapitre

- **9.1 Processus de décision markoviens** : le vocabulaire (agent, état, action, récompense), la notion de **retour actualisé**, les **équations de Bellman** démontrées, et un premier algorithme, l'**itération de la valeur**, calculé à la main sur un exemple à trois états.
- **9.2 Bandits manchots** : le cas le plus simple, sans état, où le seul enjeu est le dilemme **explorer ou exploiter**. On compare le test A/B, ϵ -glouton, UCB et l'échantillonnage de Thompson sur quatre bannières.
- **9.3 Q-learning** : apprendre sans connaître les règles du jeu. On démontre la règle de mise à jour, on oppose **Q-learning et SARSA**, et on apprend à gérer un petit stock.

- **9.4 Du Q-learning à l'apprentissage profond** : pourquoi la table ne suffit plus, ce que changent les réseaux de neurones, et les précautions (récompenses mal posées, sécurité, éthique). Section de lecture, sans exécution.

Ce qui change par rapport à l'apprentissage supervisé

	Apprentissage supervisé (chapitres 1 à 5)	Apprentissage par renforcement
Ce qu'on reçoit	des exemples étiquetés : l'entrée et la bonne réponse	une récompense après l'action, souvent différée et partielle
D'où viennent les données	elles préexistent (le tableau est donné)	elles dépendent des décisions prises : l'agent influence ce qu'il observe
Ce qu'on apprend	une fonction qui prédit	une politique : quelle action prendre dans quel état
Difficulté propre	le surapprentissage	l'attribution du crédit (quelle action passée explique ce gain ?) et l'exploration
Évaluation	un jeu de test mis de côté	la récompense cumulée obtenue en agissant

Le point de départ est donc le même (des données, un objectif), mais deux difficultés nouvelles apparaissent : **le feedback est retardé** (la commande passée aujourd'hui ne montre son effet que dans trois jours) et **les données sont biaisées par nos propres choix** (on n'observe pas ce qui serait arrivé avec l'autre bannière).

Trois petits mondes pour tout le chapitre

Les chapitres précédents travaillaient sur des tableaux de clients. Ici, l'apprentissage par renforcement a besoin d'un **environnement** avec lequel interagir. Aucune bibliothèque dédiée n'est nécessaire : nous écrivons à la main trois environnements simples. Ils sont **simulés** (graines fixes, déclarées à chaque fois), et nous connaissons donc les vrais paramètres, ce qui permet de vérifier ce que l'algorithme apprend.

Environnement	Section	Question	Graine(s)
Quatre bannières	9.2	laquelle afficher, sachant que les taux de conversion (4,0 ; 5,2 ; 5,8 et 7,0 %) sont inconnus ?	900 (200 répétitions), 4, 1 et 77
Le cycle de vie d'un client (3 états) et le stock (5 états)	9.1, 9.3	quelle action dans quel état ?	1 (évaluation), 5 et 100 à 109 (apprentissage)
Un couloir d'entrepôt (grille 4 × 10)	9.1, 9.3	comment traverser sans entrer dans la zone dangereuse ?	0 à 9 et 3

Pas de données à charger. Ce chapitre n'utilise aucun fichier : tout est simulé dans le texte et dans les blocs de calcul. Les mêmes environnements, avec leur code complet, sont reconstitués pas à pas dans le cahier.

Un chapitre d'introduction, pas un manuel d'ingénierie. L'apprentissage par renforcement « pour de vrai » (robotique, jeux, recommandation en ligne à grande échelle) demande des simulateurs, beaucoup de calcul et une grande prudence. Nous voulons ici comprendre **les idées** et leurs limites, sur des problèmes assez petits pour être résolus **exactement** et ainsi comparés à ce que l'apprentissage trouve.

9.1 Processus de décision markoviens

Cette section pose le langage de tout le chapitre. Nous partons d'une boucle très simple (l'agent agit, le monde répond), nous définissons ce que l'agent cherche à maximiser, puis nous démontrons les deux équations qui permettent de le calculer : les équations de **Bellman**. Elles servent de fil conducteur jusqu'au Q-learning.

9.1.1 Apprendre en agissant : la boucle agent-environnement

Un problème d'apprentissage par renforcement se décrit par deux acteurs et un échange répété :

- l'**agent** (celui qui décide : la gérante, ou le programme qui l'assiste) ;
- l'**environnement** (tout le reste : les clients, les fournisseurs, le hasard).

À chaque instant $t = 0, 1, 2, \dots$:

1. l'agent observe l'**état** S_t de la situation (le niveau de stock, le type de client) ;
2. il choisit une **action** A_t (commander 3 unités, envoyer une offre) ;
3. l'environnement répond par une **récompense** R_{t+1} (le gain de la journée, en €) et un **nouvel état** S_{t+1} .

On obtient ainsi une trajectoire $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$. Le but de l'agent n'est pas de maximiser la prochaine récompense : c'est de maximiser **le cumul des récompenses futures**. Commander beaucoup aujourd'hui coûte cher tout de suite et ne rapporte qu'au fil des jours suivants ; une bonne politique accepte ce compromis.

Intuition. Un joueur d'échecs ne regarde pas seulement la prise de pion immédiate : il sacrifie parfois une pièce pour une position gagnante dix coups plus loin. L'apprentissage par renforcement formalise exactement cette idée : **une récompense immédiate faible peut valoir mieux qu'un gain immédiat qui conduit à une impasse.**

9.1.2 Récompense, retour et actualisation

Pour comparer des séquences de décisions, on résume l'avenir par un seul nombre, le **retour** (*return*) à partir de l'instant t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}, \quad 0 \leq \gamma < 1.$$

Le **facteur d'actualisation** γ pondère l'avenir : un euro reçu dans k jours ne compte que γ^k euro aujourd'hui. Il a deux rôles :

- un rôle **économique** : un euro tout de suite vaut mieux qu'un euro dans un an (on l'a rencontré, sous forme de taux d'actualisation, dans le projet du volume II) ;
- un rôle **mathématique** : il garantit que la somme infinie est finie, tant que les récompenses restent bornées.

Si la récompense vaut toujours r , la série est géométrique, et

$$G_t = r(1 + \gamma + \gamma^2 + \dots) = \frac{r}{1 - \gamma}.$$

Cette formule donne l'**horizon effectif** : avec $\gamma = 0,9$ on « voit » environ $1/(1 - \gamma) = 10$ pas devant soi ; avec $\gamma = 0,95$, environ 20. Au bout de cet horizon, le poids d'une récompense a été divisé par près de trois (le calcul exact est donné ci-dessous : $0,9^{10} \approx 0,35$ et $0,95^{20} \approx 0,36$).

Choisir γ n'est pas neutre. Un γ proche de 0 produit un agent **myope** (il ne regarde que le jour même et ne commandera jamais pour demain) ; un γ proche de 1 produit un agent **patient**, mais dont l'apprentissage est plus lent et plus instable. En pratique, γ traduit la valeur que l'entreprise accorde au futur : ce n'est pas un réglage technique.

9.1.3 États, actions, transitions : le processus de décision markovien

Pour calculer, il faut un cadre précis. Un **processus de décision markovien** (MDP, *Markov decision process*) est la donnée de :

- un ensemble d'**états** $\mathcal{S}$ et d'**actions** $\mathcal{A}$;
- des **probabilités de transition** $P(s' | s, a)$: la probabilité d'arriver en s' quand on agit avec a depuis s ;
- une **récompense** $r(s, a)$ (ou son espérance) ;
- un facteur d'actualisation γ .

Le mot « markovien » porte l'hypothèse essentielle : **le futur ne dépend du passé qu'à travers l'état présent**. C'est la propriété des chaînes de Markov du volume I (section 2.6.2), à laquelle on a simplement ajouté des décisions : pour chaque action, une chaîne de Markov différente. Cette hypothèse est un choix de modélisation : si l'état « stock = 2 » ne contient pas l'information « un jour férié approche », la propriété est violée, et il faut enrichir l'état.

Notre premier MDP est volontairement minuscule, pour que tous les calculs se fassent à la main. Il représente le **cycle de vie d'un client** :

- trois **états** : client *occasionnel*, *régulier*, *fidèle* ;
- deux **actions** à chaque période : *attendre*, ou *envoyer une offre* ;
- les transitions sont **déterministes** : une offre fait passer d'occasionnel à régulier (en coûtant 1 € de remise), puis de régulier à fidèle ; attendre laisse l'état inchangé ; un client fidèle le reste ;
- les récompenses (marge nette par période) sont : 0 € (occasionnel qui attend), +1 € (régulier qui attend), +4 € (fidèle qui attend) ; l'offre coûte 1 € à un occasionnel (récompense -1 €), rapporte 0 € à un régulier, et fait perdre 1 € de marge à un fidèle (+3 € au lieu de +4 €) ;
- le facteur d'actualisation vaut $\gamma = 0,9$.

La figure résume ce MDP ; elle montre aussi, à droite, le résultat de l'algorithme que nous allons dérouler.

9.1.4 Politiques et fonctions de valeur

Une **politique** π est une règle de décision : $\pi(a | s)$ est la probabilité de choisir l'action a dans l'état s (une politique **déterministe** associe une action à chaque état). Pour juger une politique, on définit deux fonctions.

La **valeur d'un état** sous la politique π est le retour moyen quand on part de s et que l'on suit π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s].$$

La **valeur d'une action** (fonction Q) est le retour moyen quand on part de s , qu'on **commence** par l'action a puis qu'on suit π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a].$$

$V^\pi(s)$ répond à la question « combien vaut la situation s si je continue comme ça ? » ; $Q^\pi(s, a)$ à « combien vaudrait la situation si je faisais d'abord a ? ». L'objectif est de trouver la **meilleure** politique : celle dont la valeur est maximale en tout état.

9.1.5 Les équations de Bellman : démonstration

Le retour obéit à une relation **réursive** évidente : le retour d'aujourd'hui est la prochaine récompense, plus le retour de demain actualisé,

$$G_t = R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \dots) = R_{t+1} + \gamma G_{t+1}.$$

Équation de Bellman d'espérance. Pour toute politique π et tout état s ,

$$V^\pi(s) = \sum_a \pi(a | s) \left[r(s, a) + \gamma \sum_{s'} P(s' | s, a) V^\pi(s') \right].$$

Démonstration. On part de la définition et on applique la relation réursive :

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s] = \mathbb{E}_\pi[R_{t+1} | S_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} | S_t = s].$$

Le premier terme vaut $\sum_a \pi(a | s) r(s, a)$. Pour le second, on conditionne sur l'état suivant (loi de l'espérance totale) :

$$\mathbb{E}_\pi[G_{t+1} | S_t = s] = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) \mathbb{E}_\pi[G_{t+1} | S_{t+1} = s'].$$

Or, par la **propriété de Markov**, l'espérance de G_{t+1} sachant $S_{t+1} = s'$ ne dépend pas de ce qui s'est passé avant : elle vaut exactement $V^\pi(s')$. En regroupant, on obtient l'équation. ■

En mots : la valeur d'un état est la récompense moyenne immédiate, plus la valeur actualisée de l'état où l'on atterrit. Si l'on connaît la valeur de demain, on connaît celle d'aujourd'hui. Pour une politique fixée, c'est un **système linéaire** (une équation par état) : on peut le résoudre directement ou par itérations.

9.1.6 L'équation d'optimalité

Parmi toutes les politiques, il existe (pour un MDP fini avec $\gamma < 1$) une politique **optimale** π^* dont la valeur $V^*(s) = \max_{\pi} V^{\pi}(s)$ est au moins aussi bonne que toute autre, **dans tous les états à la fois**. Sa valeur obéit à la version « avec un max » de l'équation de Bellman :

$$V^*(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' | s,a) V^*(s') \right], \quad Q^*(s,a) = r(s,a) + \gamma \sum_{s'} P(s' | s,a) \max_{a'} Q^*(s',a').$$

L'argument est le **principe d'optimalité** de Bellman : une politique optimale, à partir d'un état donné, doit choisir l'action qui maximise « récompense immédiate + valeur optimale de la suite » ; elle ne peut pas faire mieux en gâchant la suite. Et dès que l'on connaît Q^* , la politique optimale est immédiate : **dans chaque état, choisir l'action de Q^* la plus élevée**,

$$\pi^*(s) = \arg \max_a Q^*(s,a).$$

C'est ce qui rend la fonction Q si commode : on y lit directement la décision à prendre, sans avoir besoin de connaître les probabilités de transition.

9.1.7 L'itération de la valeur : un exemple calculé à la main

L'équation d'optimalité est une équation de **point fixe** : V^* est la fonction qui ne change plus quand on lui applique le membre de droite. L'idée la plus simple pour la trouver : partir d'une valeur quelconque (par exemple $V_0 = 0$) et appliquer l'équation **encore et encore**,

$$V_{k+1}(s) = \max_a \left[r(s,a) + \gamma \sum_{s'} P(s' | s,a) V_k(s') \right].$$

C'est l'**itération de la valeur** (*value iteration*). Faisons-la à la main sur le cycle de vie du client. Dans ce MDP déterministe, la somme sur s' se réduit à un seul terme, donc $V_{k+1}(s) = \max_a [r(s,a) + 0,9 V_k(\text{état suivant})]$.

Itération 1 (à partir de $V_0 = (0, 0, 0)$ pour occasionnel, régulier, fidèle).

- Fidèle : attendre donne $4 + 0,9 \times 0 = 4$; l'offre donne 3. Le max vaut **4**.
- Régulier : attendre $1 + 0 = 1$; offre $0 + 0 = 0$. Le max vaut **1**.
- Occasionnel : attendre 0 ; offre -1 . Le max vaut **0**.

Itération 2 (à partir de $V_1 = (0, 1, 4)$).

- Fidèle : attendre $4 + 0,9 \times 4 = 7,6$. Régulier : attendre $1 + 0,9 \times 1 = 1,9$; **offre** $0 + 0,9 \times 4 = 3,6$: le max vaut **3,6**. Occasionnel : attendre $0 + 0,9 \times 0 = 0$; offre $-1 + 0,9 \times 1 = -0,1$: le max vaut **0**.

Itération 3 (à partir de $V_2 = (0; 3,6; 7,6)$).

- Fidèle : $4 + 0,9 \times 7,6 = 10,84$. Régulier : offre $0 + 0,9 \times 7,6 = 6,84$ (contre $1 + 0,9 \times 3,6 = 4,24$ en attendant). Occasionnel : **offre** $-1 + 0,9 \times 3,6 = 2,24$, qui l'emporte enfin sur attendre (0).

Voici les premières itérations, avec la politique **gloutonne vis-à-vis de V_k** : celle qui, dans chaque état, choisit l'action au meilleur rendement d'après V_k (c'est justement elle qui sert à calculer V_{k+1}) :

itération k	$V_k(\text{occasionnel})$	$V_k(\text{régulier})$	$V_k(\text{fidèle})$	politique gloutonne (occasionnel, régulier, fidèle)
0	0	0	0	attendre, attendre, attendre
1	0	1	4	attendre, offre, attendre
2	0	3,6	7,6	offre, offre, attendre
3	2,24	6,84	10,84	offre, offre, attendre
4	5,156	9,756	13,756	offre, offre, attendre

Deux enseignements, que la figure confirme. **D'abord**, la politique gloutonne est correcte dès V_2 : l'ordre « offre, offre, attendre » ne change plus ensuite, alors que les valeurs sont encore très loin de leur limite. **Ensuite**, les valeurs montent régulièrement vers un plafond. On peut le calculer exactement : un client fidèle qui attend rapporte 4 € par période pour toujours, donc $V^*(\text{fidèle}) = 4/(1 - 0,9) = 40$; un régulier qui envoie l'offre gagne 0 € puis devient fidèle : $V^*(\text{régulier}) = 0 + 0,9 \times 40 = 36$ (c'est mieux que d'attendre pour toujours, qui rapporterait $1/(1 - 0,9) = 10$) ; un occasionnel qui envoie l'offre : $V^*(\text{occasionnel}) = -1 + 0,9 \times 36 = 31,4$. La valeur optimale vaut donc (31,4 ; 36 ; 40), ce que l'algorithme retrouve.

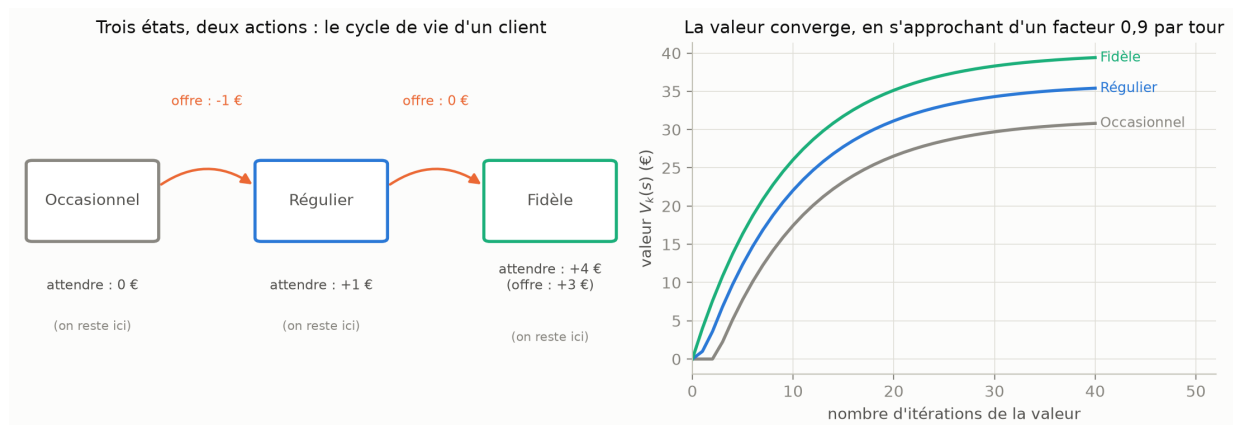


Figure 90: À gauche : le MDP du cycle de vie d'un client (trois états, deux actions). À droite : l'itération de la valeur, qui monte vers les valeurs optimales 31,4 ; 36 et 40.

La valeur d'un état, c'est son avenir. Un client « fidèle » vaut 40 €, un « occasionnel » 31,4 €, bien qu'il ne rapporte rien tout de suite : sa valeur vient de ce qu'il peut devenir. L'offre, qui coûte 1 € aujourd'hui, est un **investissement** que l'algorithme sait justifier. C'est le calcul de la valeur d'un client du projet du volume II, vu comme un problème de décision.

9.1.8 Pourquoi l'itération converge : un argument de contraction

L'itération de la valeur ne converge pas par chance. Notons T l'opérateur qui transforme une fonction de valeur V en la fonction TV donnée par le membre de droite de l'équation d'optimalité. On mesure la distance entre deux fonctions de valeur par leur écart maximal $\|V - W\|_\infty = \max_s |V(s) - W(s)|$.

Théorème. Pour tout V, W : $\|TV - TW\|_\infty \leq \gamma \|V - W\|_\infty$. On dit que T est une **contraction** de rapport γ .

Démonstration. On utilise l'inégalité $|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|$ (si le maximum de x est atteint en a_0 , alors $\max y \geq y_{a_0} \geq x_{a_0} - \max |x - y|$, et symétriquement). Pour chaque état s ,

$$|(TV)(s) - (TW)(s)| \leq \max_a \left| \gamma \sum_{s'} P(s' | s, a) (V(s') - W(s')) \right| \leq \gamma \|V - W\|_\infty,$$

car une moyenne pondérée de nombres de valeur absolue au plus $\|V - W\|_\infty$ ne dépasse pas ce maximum. ■

Le **théorème du point fixe de Banach** en découle : T a un unique point fixe, c'est V^* , et l'itération converge vers lui **quelle que soit la valeur de départ**, avec la garantie

$$\|V_k - V^*\|_\infty \leq \gamma^k \|V_0 - V^*\|_\infty.$$

Dans notre exemple, l'erreur est dominée par l'état fidèle, dont la valeur de départ est 0 pour une limite de 40, soit $40 \times 0,9^k$: 36 après 1 itération, 32,4 après 2, 29,16 après 3, environ 23,6 après 5, 13,9 après 10 et 4,9 après 20. La convergence est **géométrique**, mais **lente quand γ est proche de 1** : c'est la rançon de la patience.

Pour s'entraîner. Cahier, chapitre 9 : application 9.1, exercices 9.1 à 9.4.

9.1.9 Le problème de stock

Passons à un problème plus réaliste, que nous retrouverons en 9.3. Chaque matin, la gérante observe le **stock** d'un article, entre 0 et 4 unités (l'étagère en contient 4 au plus), et décide **combien en commander** (de 0 à ce qui reste de place) ; la livraison est immédiate. Dans la journée, la **demande** est aléatoire : 0, 1, 2 ou 3 clients souhaitent l'article, avec les probabilités 0,2 ; 0,4 ; 0,3 ; 0,1 (soit 1,3 client par jour en moyenne). La récompense de la journée est :

- +3 € par article vendu ;
- -1 € par article commandé (son prix d'achat) ;
- -4 € de **frais de livraison fixes**, dès que l'on passe commande, quelle que soit la quantité ;
- -0,5 € par article invendu en fin de journée (coût de stockage) ;
- -1 € par client servi en rupture (un client déçu coûte de la fidélité).

L'état est le stock du matin (5 valeurs), l'action la quantité commandée, et nous prenons $\gamma = 0,95$. Les transitions sont aléatoires (la demande), mais **connues** ici, ce qui permet de résoudre le problème exactement par itération de la valeur. Pour comparer, nous évaluons aussi trois règles simples, en les simulant sur 100 000 jours (graine 1).

Les valeurs optimales sont $V^* = (2,05 ; 4,14 ; 6,73 ; 8,62 ; 10,05)$ € pour un stock de 0, 1, 2, 3 ou 4 en début de journée ; 372 itérations suffisent pour que deux itérations successives diffèrent de moins d'un milliardième d'euro. La politique optimale est :

Attendre que l'étagère soit vide, puis la remplir entièrement (commander 4 si le stock vaut 0, rien sinon).

Cette règle peut surprendre : elle accepte des ruptures. Elle s'explique par les **frais de livraison fixes** : en commandant rarement mais en grande quantité, on les partage entre beaucoup d'articles. Voyons ce que les quatre politiques rapportent en moyenne par jour :

Politique	Gain moyen par jour
Ne jamais commander	-1,30 €
Remplir l'étagère chaque jour	-1,95 €
Si le stock vaut 0 ou 1, remonter à 3	-0,33 €
Politique optimale (attendre la rupture, remplir à 4)	+0,27 €

Le premier chiffre se vérifie à la main : sans jamais commander, on ne vend rien et on subit la pénalité de rupture pour chaque client, soit $1,3 \times 1 = 1,3$ € de perte par jour. Le second montre que « remplir tous les jours » est ruineux : on paie chaque jour les frais fixes. La règle de bon sens de la ligne trois (« dès que ça baisse, on remonte à 3 ») perd encore de l'argent : le calcul exact trouve mieux.

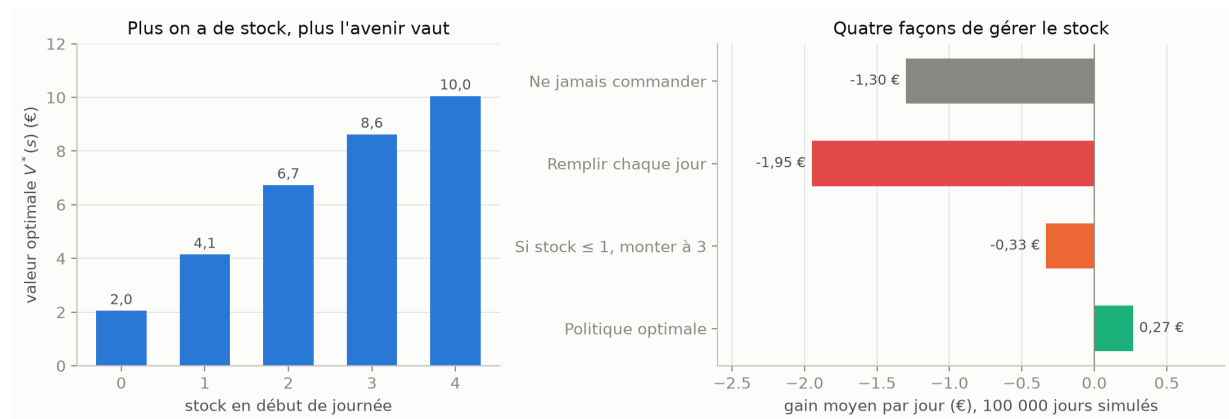


Figure 91: À gauche : valeur optimale selon le stock de départ. À droite : gain moyen par jour de quatre politiques (100 000 jours simulés).

Valeur actualisée et gain moyen ne sont pas la même chose. Les barres de gauche sont des valeurs **actualisées** (avec $\gamma = 0,95$) ; celles de droite sont des gains moyens **par jour**, sans actualisation. Rien ne garantit que deux politiques soient classées dans le même ordre par les deux mesures : un γ trop petit pourrait même faire préférer une politique myope. Quand on évalue une politique, on dit toujours **quelle** mesure on utilise.

9.1.10 Un couloir d'entrepôt : la valeur comme distance au but

Un dernier exemple, en grille, rend la notion de valeur très visuelle. Un préparateur de commandes traverse un couloir de 4×10 cases, du point de départ **S** (coin bas gauche) au quai d'expédition **G** (coin bas droit). La rangée du bas, entre les deux, est une **zone de chargement de chariots** : y mettre le pied coûte -100 € (incident évité de justesse) et ramène au départ. Chaque pas coûte -1 €, il n'y a pas d'actualisation, et les mouvements sont déterministes (haut, droite, bas, gauche).

L'itération de la valeur donne, pour chaque case, la valeur optimale : c'est tout simplement **moins le nombre de pas** du meilleur chemin jusqu'à G. Depuis le départ, il faut 11 pas (monter d'une case, avancer de 9 cases, redescendre d'une case), donc $V^*(S) = -11$. La figure montre les valeurs

par la couleur et la politique optimale par les flèches : **longer la zone dangereuse**, d'aussi près que possible, est optimal puisque le mouvement est sans aléa.

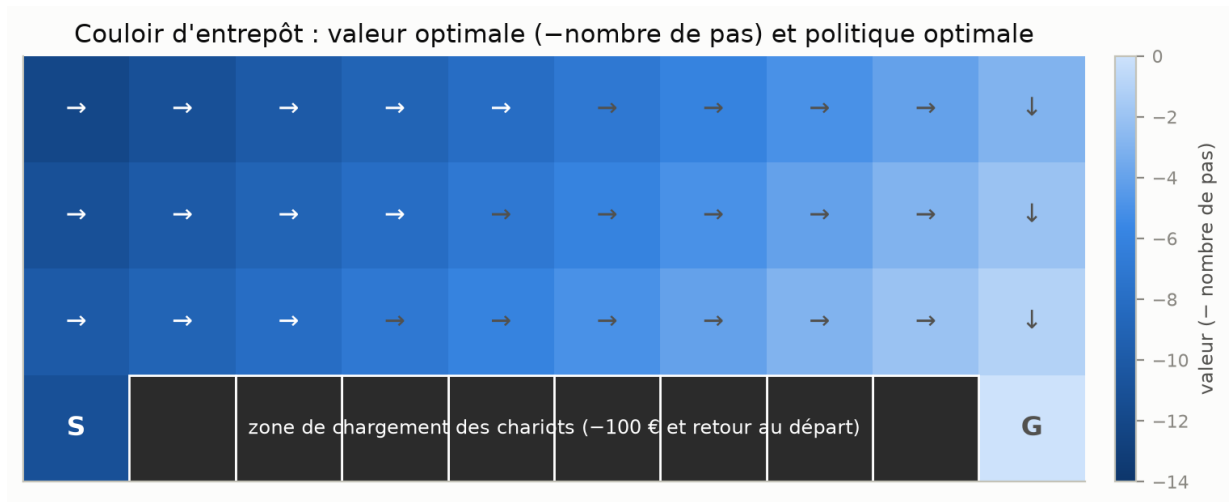


Figure 92: Le couloir d'entrepôt : valeur optimale de chaque case (plus la case est sombre, plus elle est loin du quai G) et politique optimale (flèches). La zone noire est la zone de chargement dangereuse.

En 9.3, nous réutiliserons ce couloir pour une raison précise : lorsque l'agent doit **apprendre** sa route en se trompant parfois, longer le précipice n'est plus aussi innocent.

À retenir.

- Un MDP est un état, des actions, des transitions et des récompenses, avec la propriété de Markov ; l'objectif est de maximiser le **retour actualisé** $G_t = \sum_k \gamma^k R_{t+k+1}$.
- La **valeur** $V^\pi(s)$ et la valeur d'action $Q^\pi(s, a)$ résument l'avenir ; les **équations de Bellman** les relient à la valeur de l'état suivant.
- La **politique optimale** choisit $\arg \max_a Q^*(s, a)$; l'**itération de la valeur** converge vers V^* à vitesse γ^k parce que l'opérateur de Bellman est une contraction.
- Avec un MDP **connu** et petit, on résout exactement. Tout le reste du chapitre répond à la question suivante : *que faire quand on ne connaît pas les probabilités de transition ?*

Pour s'entraîner. Cahier, chapitre 9 : application 9.1 (résoudre le problème de stock et mesurer l'effet des frais fixes), exercices 9.1 à 9.4.

9.2 Bandits manchots

Cette section isole la difficulté la plus célèbre de l'apprentissage par renforcement : le **dilemme entre explorer et exploiter**. Pour la voir à l'état pur, on retire tout le reste : pas d'état, pas d'avenir qui dépende de l'action, une décision répétée, un résultat immédiat. On l'appelle un problème de **bandit manchot à plusieurs bras** (*multi-armed bandit*), du nom des machines à sous.

Notre terrain d'expérience : la page d'accueil de la boutique peut afficher **quatre bannières** (A, B, C, D). Chaque visiteur voit une bannière et convertit (achète) ou non. Les vrais taux de conversion, **que l'algorithme ignore**, sont :

Bannière	A	B	C	D
Taux de conversion	4,0 %	5,2 %	5,8 %	7,0 %

La meilleure bannière est D. Sur 10 000 visiteurs, la connaître d'avance rapporterait en moyenne $0,07 \times 10\,000 = 700$ conversions. Mais la gérante ne la connaît pas : elle doit la découvrir **en affichant** les bannières, et chaque affichage sur une mauvaise bannière est une conversion probablement perdue.

9.2.1 Le dilemme exploration-exploitation

À chaque visiteur, deux tentations s'opposent :

- **exploiter** : afficher la bannière qui a le mieux marché jusqu'ici, pour gagner maintenant ;
- **explorer** : afficher une autre bannière, pour savoir si elle ne serait pas meilleure, au risque de perdre maintenant.

Aucune des deux attitudes pures ne marche. Un agent qui n'exploite jamais gaspille ses visiteurs en tentatives. Un agent qui n'explore jamais peut s'enfermer sur une mauvaise bannière : si, par malchance, D convertit mal sur ses dix premiers affichages, un exploiteur pur ne lui donnera plus jamais sa chance. Toute la théorie des bandits consiste à **doser** l'exploration, et à la faire décroître à mesure que l'on en sait davantage.

9.2.2 Mesurer le coût de l'ignorance : le regret

Pour comparer des stratégies, on définit le **regret** : ce que l'on perd, en moyenne, par rapport à quelqu'un qui connaîtrait la meilleure bannière. Si μ^* est le taux de la meilleure bannière et μ_{a_t} celui de la bannière affichée au t -ième visiteur, le **regret cumulé** après T visiteurs est

$$R_T = \sum_{t=1}^T (\mu^* - \mu_{a_t}) = T\mu^* - \sum_{t=1}^T \mu_{a_t}.$$

C'est un nombre de **conversions perdues** (en espérance). Une stratégie qui n'apprend rien (affichage au hasard) a un regret qui croît **linéairement** : chaque visiteur coûte en moyenne $0,07 - 0,055 = 0,015$ conversion, soit 150 conversions perdues sur 10 000 visiteurs. Une bonne stratégie a un regret qui croît **de plus en plus lentement**, parce qu'elle finit par afficher presque toujours la bonne bannière. On sait même démontrer qu'aucune stratégie ne peut faire mieux, en général, qu'un regret qui croît comme $\ln T$ (résultat de Lai et Robbins, 1985).

Le regret ne se mesure que dans une simulation (il suppose de connaître μ^*). Nous le calculerons donc sur **200 répétitions** de 10 000 visiteurs (graine 900), pour chaque stratégie.

9.2.3 Le test A/B, vu comme une stratégie

La méthode classique est le **test A/B** (volume I, section 3.4.5) : répartir les visiteurs à **égalité** entre les bannières pendant une période de test, puis adopter la gagnante. Combien de visiteurs faut-il ? Pour distinguer D (7,0 %) de C (5,8 %) avec un risque de 5 % et une puissance de 80 %, la formule du volume I donne environ **6 530 visiteurs par bannière**, donc **26 121 visiteurs au total** pour les quatre. C'est plus de deux fois notre budget de 10 000 visiteurs : un test rigoureux à quatre bannières **n'est pas possible** dans ce cadre, et l'on aurait payé très cher son exploration, puisque trois visiteurs sur quatre auraient vu une bannière moins bonne.

On peut quand même transformer le test A/B en stratégie, avec un budget d'exploration réduit : **tester 2 000 visiteurs** (500 par bannière), puis **s'engager** sur la bannière qui a le mieux converti (on dit *explore-then-commit*). Le regret moyen est alors d'environ 48 conversions, mais le résultat est très inégal : dans 14,5 % des répétitions, **la mauvaise bannière est choisie**, et dans 10 % des répétitions le regret dépasse 125. Le test est trop court pour distinguer des bannières aussi proches.

Le test A/B n'est pas « mauvais », il répond à une autre question. Il est conçu pour **conclure** (« D est meilleure que C, avec telle confiance »), pas pour **gagner pendant qu'on apprend**. Si le but est une décision définitive appuyée sur des preuves, c'est l'outil adapté ; si le but est de maximiser les ventes pendant que l'on cherche, les bandits sont plus efficaces. Les deux objectifs ne sont pas compatibles à 100 %, comme nous le verrons en 9.2.9.

9.2.4 La stratégie ϵ -glouton

La règle la plus simple pour mélanger exploration et exploitation : avec une probabilité ϵ (ici 0,1), afficher une bannière **au hasard** ; sinon, afficher celle dont le taux observé est le meilleur. L'idée est séduisante, mais elle a deux défauts. D'abord, l'exploration est **constante** : même quand D est évidemment la meilleure, 10 % des visiteurs voient une bannière tirée au hasard, ce qui coûte, par visiteur, $0,1 \times 0,015 = 0,0015$ conversion, soit 15 conversions perdues sur 10 000, **pour toujours**. Ensuite, elle est **aveugle** : elle explore autant les bannières qui semblent mauvaises que celles qui sont prometteuses.

Résultat : un regret moyen d'environ 65 conversions, avec une grande dispersion (écart-type 51), et dans 28 % des répétitions la bannière la plus affichée **n'est pas D** : le glouton s'est enfermé sur une mauvaise bannière après un démarrage malchanceux.

9.2.5 UCB : l'optimisme face à l'incertitude

L'idée d'**UCB** (*upper confidence bound*) est de choisir **la bannière qui pourrait être la meilleure**, en lui accordant le bénéfice du doute proportionnellement à notre incertitude. À chaque visiteur t , on calcule pour chaque bannière un **indice** :

$$\text{indice}_a = \underbrace{\hat{\mu}_a}_{\text{taux observé}} + \underbrace{\sqrt{\frac{2 \ln t}{n_a}}}_{\text{bonus d'incertitude}},$$

où n_a est le nombre d'affichages de a . On affiche la bannière d'indice maximal. Une bannière rarement affichée a un grand bonus et sera donc essayée ; au fur et à mesure qu'on la connaît mieux, le bonus s'effondre et seul le taux observé compte. L'exploration **se règle toute seule**, bannière par bannière.

D'où vient le bonus ? Il vient de l'inégalité de **Hoeffding** : pour un taux observé sur n essais, $P(\mu \geq \hat{\mu} + \epsilon) \leq e^{-2n\epsilon^2}$ (valable pour des résultats compris entre 0 et 1). Le bonus est la valeur de ϵ pour laquelle cette probabilité d'erreur vaut t^{-4} , c'est-à-dire $e^{-2n\epsilon^2} = t^{-4}$, soit $\epsilon = \sqrt{2 \ln t / n}$. L'indice est donc une **borne supérieure plausible** du vrai taux : la probabilité qu'on le sous-estime est minuscule. Auer, Cesa-Bianchi et Fischer (2002) ont démontré que cette stratégie, appelée UCB1, a un regret qui croît comme $\ln T$, avec la borne $\sum_{a \neq a^*} (8 \ln T / \Delta_a + (1 + \pi^2/3) \Delta_a)$, où $\Delta_a = \mu^* - \mu_a$ est l'écart avec la meilleure bannière.

Un exemple à la main. Au visiteur numéro $t = 1\,000$, la bannière A a été affichée 400 fois pour 20 conversions (5,0 %), la bannière B 100 fois pour 6 conversions (6,0 %). Les bonus valent

$$\sqrt{\frac{2\ln 1000}{400}} \approx 0,186 \quad (\text{A}), \quad \sqrt{\frac{2\ln 1000}{100}} \approx 0,372 \quad (\text{B}),$$

donc les indices valent $0,050 + 0,186 = 0,236$ pour A et $0,060 + 0,372 = 0,432$ pour B. On affiche **B** : peu connue, elle bénéficie d'un grand doute.

Mais remarquez l'**ordre de grandeur** : le bonus (0,19 à 0,37) est **plusieurs fois supérieur** aux taux eux-mêmes (5 à 6 %). La formule suppose des résultats répartis sur tout l'intervalle $[0, 1]$, alors que nos conversions sont rares. Le résultat est un **excès d'exploration** : UCB1 affiche D seulement 35 % du temps, et son regret moyen (**123 conversions**) est le **pire** de toutes les stratégies, bien qu'il soit très régulier (écart-type 5). La borne théorique de 12 690 est vraie, mais très pessimiste.

Le remède classique est de **réduire le bonus** : remplacer le 2 par une constante c à régler. Le regret moyen vaut 123,4 pour $c = 2$, 99,2 pour $c = 0,5$, **56,6** pour $c = 0,1$ et 38,8 pour $c = 0,05$. Dans le tableau qui suit, nous retenons $c = 0,1$. Attention toutefois : choisir c en regardant le regret sur *la même simulation* est une forme de triche (on sait déjà que D gagne). En pratique, c est un **hyperparamètre**, à régler avec la rigueur du chapitre 1 (section 1.5).

9.2.6 L'échantillonnage de Thompson

L'autre grande idée est bayésienne, et elle est plus élégante. Pour chaque bannière, on entretient une **croyance** sur son taux de conversion, sous forme d'une loi de probabilité. Avec une loi *a priori* uniforme et des résultats « conversion / pas de conversion », cette croyance est une **loi Bêta** (volume II, section 6.1.3) : après s conversions en n affichages, le taux suit une loi Bêta($1 + s$, $1 + n - s$). L'**échantillonnage de Thompson** (1933) procède ainsi, à chaque visiteur :

1. pour chaque bannière, **tirer un taux au hasard** dans sa loi Bêta ;
2. afficher la bannière dont le taux tiré est le plus élevé ;
3. observer le résultat et mettre à jour la loi de cette bannière.

```
# Un tour de Thompson (succes et essais : tableaux de taille 4 ; taux_vrais : inconnus de l'algorithme)
theta = rng.beta(1 + succes, 1 + essais - succes) # un taux tiré dans chaque loi Bêta
a = int(np.argmax(theta)) # on affiche la bannière au taux tiré le plus haut
x = rng.random() < taux_vrais[a] # le visiteur convertit-il ?
essais[a] += 1 # mise à jour : une conversion de plus ou non
succes[a] += x
```

L'astuce est que la **probabilité d'afficher une bannière égale la probabilité qu'elle soit la meilleure**, d'après nos croyances (c'est le *probability matching*). **Exemple à la main** : à $t = 1\,000$, A (20 conversions sur 400) suit une loi Bêta(21, 381) et B (6 sur 100) une loi Bêta(7, 95). En tirant de nombreux couples de taux, on trouve que le taux de B dépasse celui de A dans **71 % des cas** : B sera donc affichée 71 % du temps, et A 29 %. Contrairement à UCB, la décision est **aléatoire**, et l'exploration est naturellement concentrée sur les bannières encore plausibles.

La figure montre l'évolution des croyances pour une répétition (graine 4). Après 100 visiteurs, tout est flou ; après 1 000, D émerge ; après 10 000, la croyance sur D est étroite (8 816 affichages) alors que celles sur les trois autres sont larges : on **ne les a plus regardées**, et c'est parfaitement raisonnable, puisqu'on a compris qu'elles étaient moins bonnes.

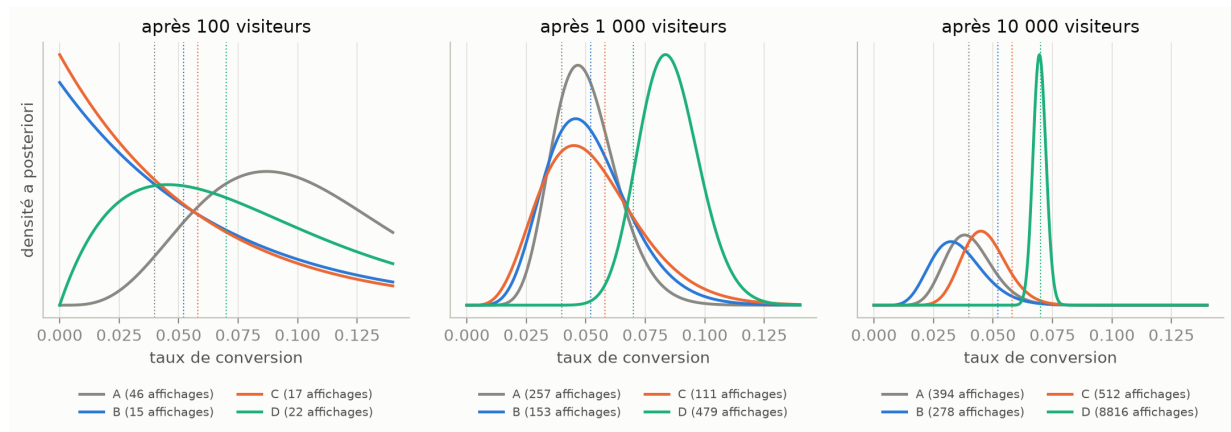


Figure 93: Croyances de l'algorithme de Thompson sur le taux de chaque bannière, après 100, 1 000 et 10 000 visiteurs (une seule répétition). Les traits pointillés verticaux marquent les vrais taux, inconnus de l'algorithme.

9.2.7 Comparaison des stratégies

Voici les cinq stratégies sur les mêmes 200 répétitions (graine 900) :

Stratégie	Regret moyen	Écart-type	90e centile	Part des affichages sur D	Répétitions où la bannière la plus affichée n'est pas D
A/B test (2 000 visiteurs, puis la meilleure)	47,9	36,6	125,0	71,8 %	14,5 %
ϵ -glouton ($\epsilon = 0,1$)	64,6	50,9	138,6	61,8 %	28,0 %
UCB1 (bonus classique)	123,4	5,3	130,3	34,7 %	1,0 %
UCB à bonus réduit ($c = 0,1$)	56,6	14,6	74,9	66,6 %	1,0 %
Thompson	46,2	23,2	74,1	71,5 %	7,0 %

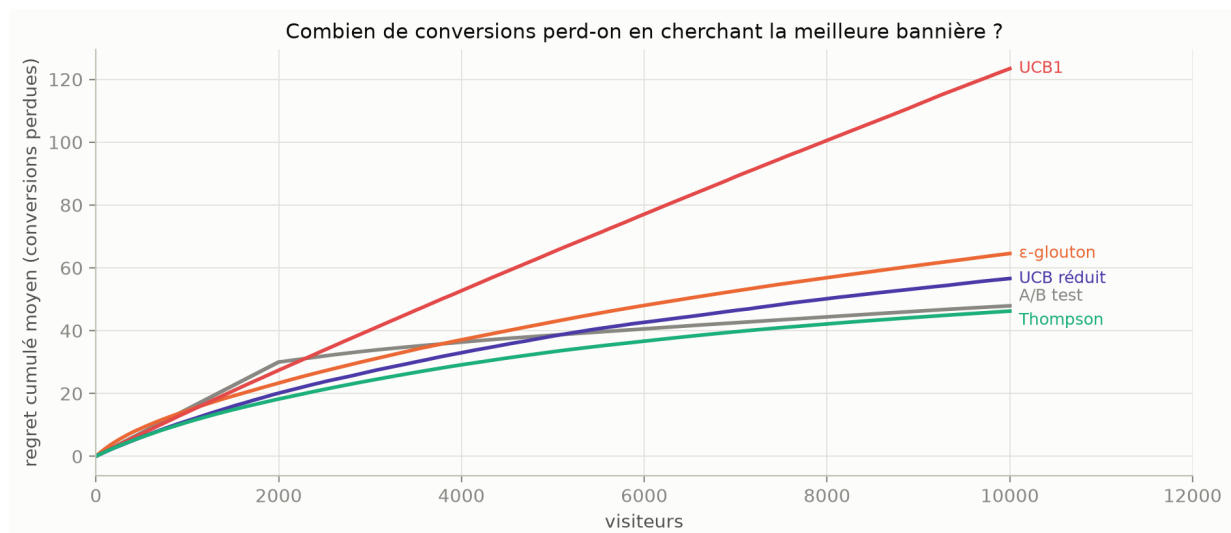


Figure 94: Regret cumulé moyen (conversions perdues) de cinq stratégies d'affichage sur 10 000 visiteurs, moyenne de 200 répétitions.

- **ϵ -glouton** est dominé par les trois meilleures stratégies (Thompson, A/B, UCB réduit) : regret moyen plus élevé, plus forte dispersion, et 28 % de répétitions enfermées sur une mauvaise bannière. Il cumule exploration constante *et* risque d'enfermement.

Ne généralisez pas ce classement. Il dépend de l'écart entre les bannières, de leur nombre, du nombre de visiteurs et de la valeur de c . Dans cette expérience précise (peu de visiteurs, taux faibles), Thompson et A/B sont proches en moyenne ; avec 100 000 visiteurs, l'avantage des méthodes adaptatives serait bien plus net. Le message n'est pas « Thompson gagne », mais « **une stratégie adaptative réduit le coût et la variance de l'exploration** ».

9.2.8 Quand le meilleur choix dépend du contexte

Jusqu'ici, la même bannière était la meilleure pour tous les visiteurs. En réalité, le meilleur choix dépend souvent de **qui est le visiteur**. Un **bandit contextuel** observe, avant de choisir, un **contexte** (le canal d'arrivée, l'appareil, la ville) et apprend une politique par contexte. Imaginons que trois canaux d'arrivée (Boutique, Site, Réseaux ; 30, 40 et 30 % des visiteurs) préfèrent des bannières différentes :

Canal d'arrivée	A	B	C	D	Meilleure
Boutique	6,0 %	4,0 %	3,0 %	4,5 %	A
Site	4,0 %	4,5 %	7,0 %	5,0 %	C
Réseaux	3,5 %	4,0 %	4,5 %	7,5 %	D

Si l'on ignore le canal, la meilleure bannière **unique** est D, avec un taux moyen de 5,6 % ; en choisissant la bonne bannière **par canal**, on atteindrait 6,85 %. En simulant Thompson (graine 77, 200 répétitions de 10 000 visiteurs), l'algorithme **aveugle au contexte** accumule un regret moyen de 164,5 conversions (par rapport à la politique idéale par canal), tandis que l'algorithme qui entretient **une croyance par canal** n'en accumule que 80,1. Savoir à qui l'on s'adresse divise le regret par deux.

La version complète des bandits contextuels remplace la table « un canal = une ligne » par un **modèle** (régression logistique, arbre) qui prédit la probabilité de conversion à partir de variables nombreuses ; les méthodes les plus connues (LinUCB, Thompson avec régression) en dérivent. Elles ne sont pas exécutées ici.

9.2.9 Précautions

Les bandits sont puissants, et dangereux quand on oublie leurs hypothèses.

- **Les taux changent.** Un taux de conversion varie avec la saison, les promotions, la météo (volume II, chapitre 4). Un bandit qui a « fini d'explorer » ne s'adapte plus ; il faut oublier le passé (fenêtre glissante, facteur d'oubli) ou garder une exploration résiduelle.
- **Les résultats arrivent en retard.** Si l'on ne sait qu'au bout de 10 jours qu'une commande est retournée, le bandit apprend sur des signaux incomplets.
- **On optimise ce qu'on mesure.** Si la récompense est le clic, la bannière « pièges à clics » gagnera, même si elle déçoit ensuite le client.
- **L'allocation adaptative biaise les estimations.** Une bannière peu affichée l'a été **parce qu'elle a mal débuté** : son taux observé est donc, en moyenne, **sous-estimé**. Dans nos 200 répétitions de Thompson, le taux estimé moyen des bannières A, B, C et D vaut 3,43 %, 4,64 %, 5,31 % et 6,92 %, contre des vrais taux de 4,0 ; 5,2 ; 5,8 et 7,0 %. Les trois bannières abandonnées sont toutes **sous-estimées**.

- **La randomisation devient un choix algorithmique.** Un test A/B randomisé donne une comparaison **causale** propre (volume II, chapitre 7, section 7.1). Les probabilités d'un bandit changent avec le temps, ce qui complique l'inférence : on peut la rétablir en gardant les probabilités d'affichage et en pondérant (volume II, section 7.2), mais ce n'est plus un simple calcul de moyennes.

	Test A/B	Bandit
Objectif	conclure avec confiance	gagner en apprenant
Allocation	fixe, à égalité	adaptative
Coût de l'exploration	élevé et connu d'avance	faible, mais variable
Qualité de l'inférence	excellente	dégradée (biais d'estimation)
À utiliser quand	la décision est définitive et doit être justifiée	le contexte change vite et chaque affichage compte

À retenir.

- Un **bandit** est le problème de décision le plus simple : pas d'état, un résultat immédiat, et un dilemme **explorer / exploiter**.
- Le **regret** mesure les conversions perdues par rapport à une connaissance parfaite ; une bonne stratégie a un regret qui croît lentement (en $\ln T$).
- **ϵ -glouton** explore à taux constant ; **UCB** explore là où l'incertitude est grande (attention aux constantes) ; **Thompson** tire au sort selon ses croyances bayésiennes. Les deux derniers explorent de façon **dirigée**.
- Le **contexte** divise le regret quand le meilleur choix dépend du visiteur ; un **test A/B** reste préférable pour **conclure**.

Pour s'entraîner. Cahier, chapitre 9 : applications 9.2 et 9.3, exercices 9.5 à 9.8.

9.3 Q-learning : apprendre sans connaître les règles

En 9.1, nous avons **résolu** le problème de stock parce que nous connaissions les probabilités de demande. Dans une vraie boutique, personne ne les connaît : on observe seulement ce qui se passe quand on agit. Cette section montre comment apprendre une bonne politique **à partir de l'expérience seule**, c'est-à-dire de suites (s, a, r, s') : « j'étais dans l'état s , j'ai fait a , j'ai gagné r , et je me suis retrouvé en s' ».

9.3.1 Apprendre sans modèle

Deux familles de méthodes se distinguent :

- les méthodes **avec modèle** (*model-based*) estiment d'abord les transitions et les récompenses à partir de l'expérience, puis résolvent le MDP estimé (comme en 9.1). Elles demandent beaucoup d'observations pour bien estimer le modèle entier ;
- les méthodes **sans modèle** (*model-free*) apprennent directement la valeur des actions, sans jamais écrire le modèle. C'est le cas du Q-learning.

Une première idée sans modèle serait de jouer des journées entières, de calculer le retour de chaque épisode, puis de **moyenner** les retours observés depuis chaque état (méthode de Monte-Carlo). Elle est correcte, mais lente : il faut attendre la fin de l'épisode, et la variance des retours est grande. La **différence temporelle** fait mieux, en s'appuyant sur l'équation de Bellman.

9.3.2 La différence temporelle

L'équation de Bellman (9.1.5) dit que $V^\pi(s) = \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s]$. À chaque transition observée, la quantité $R_{t+1} + \gamma V(S_{t+1})$ est donc **un tirage aléatoire dont l'espérance est la bonne valeur**. L'idée consiste à rapprocher notre estimation actuelle $V(S_t)$ de ce tirage, d'un petit pas α :

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\delta_t : \text{erreur de différence temporelle}}.$$

L'**erreur de différence temporelle** δ_t est la **surprise** : la différence entre ce que l'on pensait (la valeur de S_t) et ce que l'on vient d'observer (la récompense, plus la valeur de la suite *telle qu'on l'estime*). Si $\delta_t > 0$, l'état était meilleur que prévu : on relève sa valeur. On dit que la méthode **amorce** (*bootstrap*) : elle corrige une estimation à partir d'une autre estimation, sans attendre la fin de l'histoire.

Pourquoi ça marche. C'est une **moyenne mobile** : si l'on remplaçait α par $1/n$ et la cible par un tirage indépendant, on retrouverait exactement la moyenne arithmétique des tirages. La cible $R + \gamma V(S')$ n'est pas tout à fait un tirage indépendant (elle contient notre estimation V), mais la contraction de Bellman (9.1.8) garantit que cette boucle de rétroaction s'améliore au lieu de s'emballer.

9.3.3 Le Q-learning

Pour *choisir* une action, on a besoin de la valeur des **actions**, pas seulement des états. Le **Q-learning** (Watkins, 1989) applique la différence temporelle à l'équation d'**optimalité** (9.1.6) : après avoir observé (s, a, r, s') ,

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)].$$

C'est la même idée, avec un max : la cible est « la récompense observée, plus la valeur de la **meilleure** action possible dans l'état suivant ». Comme la cible est un tirage dont l'espérance est $(TQ)(s, a)$, où T est l'opérateur de Bellman d'optimalité, et que Q^* est son point fixe, la règle pousse Q vers Q^* .

```
# Une mise à jour de Q-learning après l'observation (s, a, r, s2)
cible = r + gamma * Q[s2].max()          # ce que l'on croit valoir cette action, d'après la suite
Q[s, a] += alpha * (cible - Q[s, a])     # on rapproche l'estimation de la cible, d'un pas alpha
```

Un calcul à la main. Dans le problème de stock, supposons que l'on soit en $s = 0$ (étagère vide), que l'on commande 4 articles, et que la demande du jour soit de 2 clients. La récompense vaut 2×3 (ventes) $- 4 \times 1$ (achats) $- 4$ (frais fixes) $- 0,5 \times 2$ (deux articles invendus) $- 0 = -3$ €, et le stock suivant vaut $4 - 2 = 2$. Admettons que l'estimation courante soit $Q(0, 4) = 3,0$ et que la meilleure valeur connue en $s' = 2$ soit $\max_{a'} Q(2, a') = 6,0$ (valeurs choisies pour l'illustration). Avec $\gamma = 0,95$ et $\alpha = 0,1$:

$$\text{cible} = -3 + 0,95 \times 6,0 = 2,7, \quad \delta = 2,7 - 3,0 = -0,3, \quad Q(0, 4) \leftarrow 3,0 + 0,1 \times (-0,3) = 2,97.$$

La surprise est légèrement négative (la journée a été un peu moins bonne que prévu), et l'estimation baisse d'un dixième de cet écart.

Le Q-learning est « hors politique » (*off-policy*). Il apprend la valeur de la politique **gloutonne** (à cause du max), alors que les actions *jouées* pendant l'apprentissage sont choisies par une autre politique, qui explore. Cette séparation est précieuse : on peut apprendre la politique optimale en agissant différemment, ou même à partir de données collectées par quelqu'un d'autre.

9.3.4 SARSA, ou apprendre en tenant compte de ses propres erreurs

Une variante remplace le max par l'action effectivement choisie ensuite, a' :

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)].$$

Le nom vient de la suite (S, A, R, S', A') qu'elle utilise. SARSA est **sur la politique** (*on-policy*) : elle évalue la politique qu'elle suit réellement, **exploration comprise**. La différence, qui paraît minuscule, a des conséquences visibles sur le couloir d'entrepôt de 9.1.10. Cette fois, **l'agent ne connaît pas la carte** : il l'apprend par essais, avec $\alpha = 0,5$ et une exploration $\varepsilon = 0,1$ (10 % des pas sont tirés au hasard), pendant 500 épisodes. On répète l'expérience pour 10 graines.

Deux observations :

- **Pendant l'apprentissage**, SARSA obtient un retour moyen de $-26,6$ par épisode (sur les 100 derniers), contre $-39,5$ pour le Q-learning : le Q-learning tombe plus souvent dans la zone dangereuse.
- **Les chemins appris**, si l'on suit ensuite la politique gloutonne, sont différents : le Q-learning trouve le chemin **le plus court**, qui longe la zone dangereuse (11 pas, rangée 2) ; SARSA trouve un chemin **plus long mais plus sûr**, qui passe par la rangée du haut (15 pas).

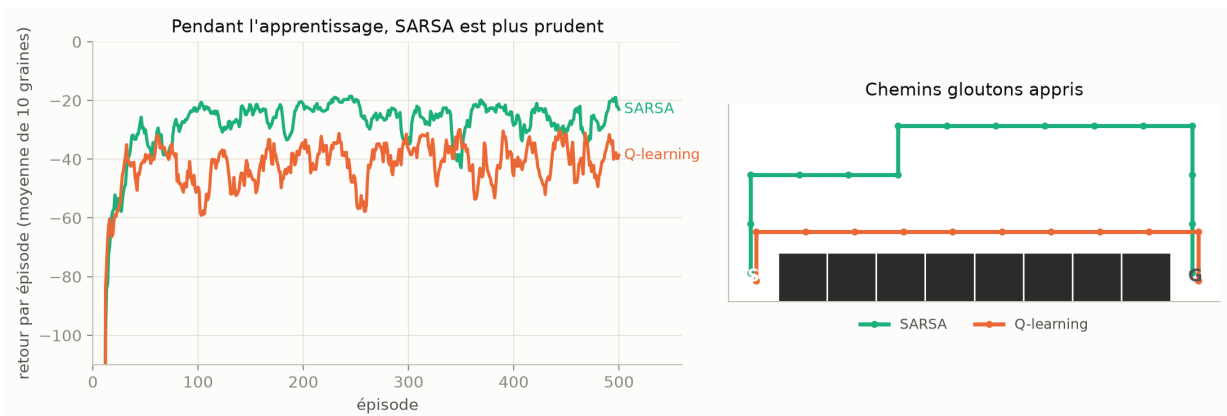


Figure 95: À gauche : retour par épisode pendant l'apprentissage (moyenne de 10 graines, lissée sur 10 épisodes). À droite : chemins gloutons appris par SARSA et par le Q-learning ; la zone noire est dangereuse.

L'explication tient à ce que chacune **estime**. Le Q-learning suppose qu'à partir de la case suivante, il jouera au mieux : le bord du précipice lui semble donc sans danger. Mais **pendant l'apprentissage**, il joue avec $\varepsilon = 0,1$: sur le bord, un pas tiré au hasard sur dix peut le précipiter. SARSA, lui, apprend la valeur de la politique **réellement suivie**, dérapages compris ; il en déduit que le bord est risqué, et s'en écarte. Quand ε tend vers zéro, les deux méthodes convergent vers le même chemin optimal.

On-policy ou off-policy : une question de sécurité. Si les erreurs d'exploration sont **coûteuses ou irréversibles** (un robot près d'un escalier, une promotion qui fâche un client pour toujours), la prudence de SARSA est souhaitable : l'agent apprend en tenant compte du fait qu'il se trompera. Si l'on apprend dans un **simulateur** où les erreurs ne coûtent rien, le Q-learning, qui vise directement la politique optimale, est préférable.

9.3.5 Explorer pendant l'apprentissage

Nous avons déjà rencontré le dilemme en 9.2 ; il revient ici, état par état. Dans nos expériences, l'exploration est de type **ϵ -glouton à décroissance** : avec la probabilité ϵ_t , l'agent choisit une action légale au hasard, sinon il choisit l'action de plus grand Q . Pour le stock, ϵ décroît linéairement de 1 à 0,05 pendant la première moitié de l'apprentissage, puis reste à 0,05. Au début, l'agent explore donc presque toujours ; à la fin, il exploite presque toujours.

D'autres techniques existent : l'**initialisation optimiste** (on part de valeurs Q très élevées, de sorte que toute action essayée « déçoit » et que les autres paraissent plus attrayantes) ; les **bonus d'exploration** inspirés d'UCB (9.2.5), qui valorisent les couples (s, a) peu visités ; l'exploration par **bruit** sur les paramètres, dans les méthodes profondes (9.4).

9.3.6 Quand le Q-learning converge

Le Q-learning ne converge pas toujours. Un théorème de **Watkins et Dayan (1992)** donne des conditions suffisantes : si chaque couple (s, a) est visité **une infinité de fois**, et si le pas d'apprentissage α_n utilisé à la n -ième mise à jour de ce couple vérifie les conditions de **Robbins et Monro**,

$$\sum_n \alpha_n = \infty \quad \text{et} \quad \sum_n \alpha_n^2 < \infty,$$

alors Q converge vers Q^* avec probabilité 1. La première condition garantit que l'on peut **aller aussi loin** que nécessaire ; la seconde que le **bruit finit par s'éteindre**. Un pas $\alpha_n = 1/n$ les satisfait (la série harmonique diverge, la série des carrés converge) ; un pas **constant** ne satisfait pas la seconde : la table continue de bouger au gré du hasard et ne se fixe jamais.

Mesurons-le sur le problème de stock (graine 5, 300 000 jours), en comparant un pas constant $\alpha = 0,1$ à un pas décroissant $\alpha = 1/(1 + N/50)$, où N est le nombre de fois que le couple a été visité. L'erreur maximale entre la table apprise et la vraie fonction Q^* (calculée en 9.1) vaut, après 10 000, 50 000, 150 000 et 300 000 jours :

Pas d'apprentissage	10 000	50 000	150 000	300 000
Constant ($\alpha = 0,1$)	0,55 €	0,71 €	1,22 €	0,79 €
Décroissant	1,02 €	0,24 €	0,16 €	0,25 €

Le pas constant **n'améliore plus rien** : l'erreur oscille autour de 1 €. Le pas décroissant, lent au départ, **descend** ensuite d'un facteur 4 environ. Sur 10 graines, le pas décroissant retrouve la politique optimale **10 fois sur 10** avec une erreur maximale moyenne de 0,20 €, contre **8 fois sur 10** et 1,22 € pour le pas constant.

Dans la pratique, on utilise souvent un pas constant. Il s'adapte si le monde change (un pas décroissant « s'endort »). C'est le bon choix quand les données évoluent, au prix d'une table qui reste bruitée : comme toujours, le réglage dépend de la situation.

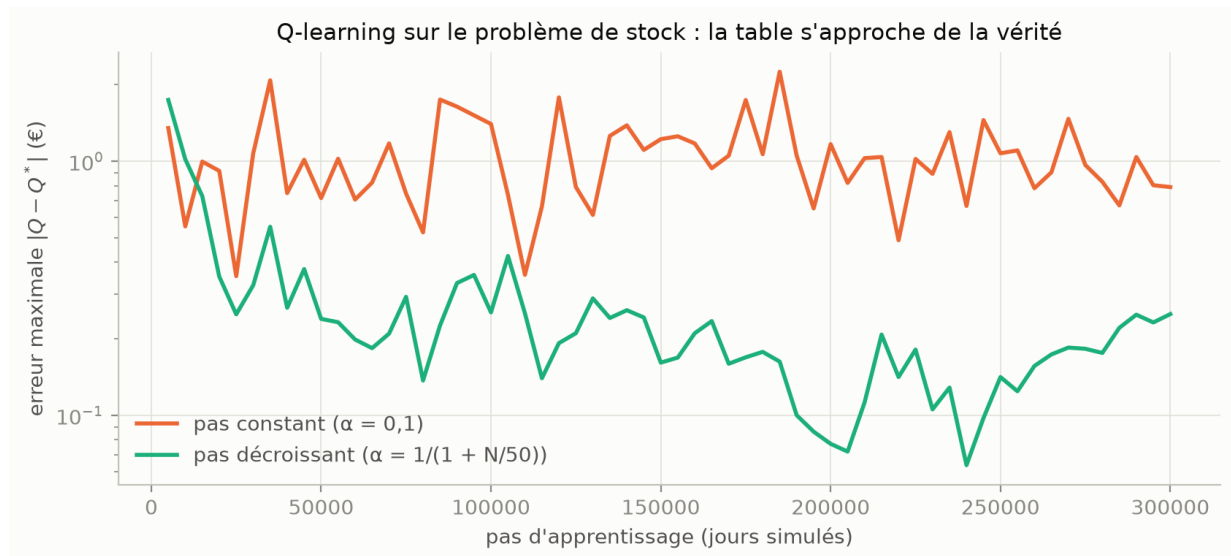


Figure 96: Erreur maximale entre la table Q apprise et la vraie fonction Q^* au fil de l'apprentissage, pour un pas constant et un pas décroissant (échelle logarithmique).

9.3.7 Sur le problème de stock

Le Q-learning, sans connaître les probabilités de demande, retrouve ici la politique optimale : **attendre la rupture, puis remplir** (commander 4 si le stock vaut 0, rien sinon). Son gain moyen simulé vaut 0,27 € par jour, identique à celui de la solution exacte, alors que la règle de bon sens de 9.1.9 (« remonter à 3 si le stock vaut 0 ou 1 ») perd 0,33 € par jour. L'apprentissage a donc découvert, **par l'expérience seule**, qu'il fallait accepter des ruptures pour amortir les frais de livraison, ce qu'aucune règle intuitive n'aurait suggéré.

9.3.8 Les limites de la table

Ce succès est trompeur, pour trois raisons.

- **L'expérience coûte cher.** Nos 300 000 jours simulés représentent environ **822 ans** de vie de la boutique. L'apprentissage ne fonctionne que dans un **simulateur** (ou avec d'énormes volumes de données historiques) : on ne peut pas laisser une vraie boutique faire 300 000 essais.
- **La table est minuscule** : 15 couples (s, a) ici. Dès que l'état contient plusieurs variables (le stock de dix produits, le jour de la semaine, la météo), le nombre d'états explose : c'est le sujet de la section suivante.
- **Le Q-learning est instable en dehors de la table.** Les garanties de convergence ci-dessus supposent une table exacte ; elles disparaissent quand on approxime Q par une fonction (9.4).

À retenir.

- L'apprentissage **par différence temporelle** corrige une estimation à partir d'une autre : $\delta = r + \gamma V(s') - V(s)$ mesure la surprise.
- Le **Q-learning** vise Q^* avec la cible $r + \gamma \max_{a'} Q(s', a')$; il est **hors politique**. **SARSA** utilise l'action réellement jouée ; elle est **sur la politique** et plus prudente quand l'exploration est risquée.
- Il converge si tous les couples sont visités infiniment et si le pas vérifie $\sum \alpha = \infty, \sum \alpha^2 < \infty$; un pas constant laisse un bruit résiduel.

- Sur un petit problème, il retrouve la solution exacte, mais au prix de **beaucoup d'expérience** : c'est la raison d'être des simulateurs.

Pour s'entraîner. Cahier, chapitre 9 : applications 9.4 et 9.5, exercices 9.9 à 9.12.

9.4 Du Q-learning à l'apprentissage profond

Cette section est une **lecture** : elle explique ce qui change quand on quitte la table, sans rien exécuter (ni PyTorch ni TensorFlow n'est installé pour ce livre). Les méthodes décrites ici sont celles des grandes réussites médiatiques (jeux vidéo, Go, robots) ; elles sont aussi celles dont il faut le plus se méfier, et nous en profitons pour discuter les précautions d'emploi.

9.4.1 Quand la table explose

Le Q-learning stocke une valeur par couple (état, action). Pour le stock d'un seul article, cela représente 15 valeurs. Imaginons maintenant que la gérante gère **dix articles**, chacun avec un stock de 0 à 20 unités : l'état est la liste des dix stocks, et il y a $21^{10} = 16\,679\,880\,978\,201$ états, soit près de **17 000 milliards**. Ajoutons le jour de la semaine (7 valeurs) : un nombre de l'ordre de 10^{14} . Aucune table ne tient en mémoire, et surtout **aucun agent ne peut visiter chaque état** : la plupart des situations ne se rencontreront jamais deux fois.

Il faut **généraliser** : estimer la valeur d'une situation jamais vue à partir de situations voisines déjà vues. C'est exactement ce que fait l'apprentissage supervisé (chapitres 1 et 2). On remplace donc la table par une **fonction paramétrée** $Q_\theta(s, a)$ (une régression, un arbre, un réseau de neurones), que l'on ajuste pour qu'elle respecte l'équation de Bellman.

9.4.2 Approximer la fonction de valeur : le principe du DQN

L'idée du **DQN** (*deep Q-network*, Mnih et al., 2015) est d'entraîner un réseau de neurones Q_θ avec la même cible que le Q-learning. Après une transition (s, a, r, s') , on cherche à réduire l'erreur

$$L(\theta) = \left(r + \gamma \max_{a'} Q_\theta(s', a') - Q_\theta(s, a) \right)^2.$$

Deux ingrédients rendent l'entraînement praticable :

- le **tampon de rejeu** (*replay buffer*) : on stocke les transitions passées et on entraîne sur des **mini-lots tirés au hasard** dans ce tampon. Sans cela, les exemples successifs sont très corrélés (la journée $t + 1$ ressemble à la journée t), ce qui viole l'hypothèse « exemples indépendants » de l'apprentissage supervisé (chapitre 1) ;
- le **réseau cible** θ^- : une copie *figée* du réseau, mise à jour rarement, qui sert à calculer la cible. Sans lui, la cible bouge à chaque pas puisqu'elle dépend des paramètres que l'on est en train de modifier.

```
# Non exécuté : PyTorch n'est pas installé dans l'environnement de ce livre.
q_reseau, q_cible = ReseauQ(), ReseauQ() # deux réseaux de même architecture
for etape in range(n_etapes):
    a = epsilon_glouton(q_reseau, s) # on explore parfois
    s2, r, fin = env.pas(a) # l'environnement répond
    tampon.ajouter((s, a, r, s2, fin)) # on garde la transition en mémoire
    s_b, a_b, r_b, s2_b, fin_b = tampon.echantillon(64) # mini-lot tiré au hasard
    cible = r_b + gamma * (1 - fin_b) * q_cible(s2_b).max(dim=1).values
```

```

perte = ((q_reseau(s_b).gather(1, a_b) - cible.detach()) ** 2).mean()
optimiseur.zero_grad(); perte.backward(); optimiseur.step()
if etape % 1000 == 0: q_cible.load_state_dict(q_reseau.state_dict()) # copie périodique

```

9.4.3 Optimiser directement la politique

Une autre famille ne passe pas par Q : elle **paramètre la politique** $\pi_\theta(a | s)$ elle-même (par exemple un réseau qui renvoie des probabilités d'actions) et monte le long du gradient de la récompense espérée $J(\theta)$. Le **théorème du gradient de politique** donne l'expression de ce gradient :

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \ln \pi_\theta(a | s) Q^{\pi_\theta}(s, a)].$$

L'intuition est simple : **augmenter la probabilité des actions qui ont mené à un bon retour, diminuer celle des autres**. L'algorithme **REINFORCE** estime cette espérance avec des retours observés, au prix d'une variance élevée ; on la réduit en soustrayant une **valeur de référence** (*baseline*) à Q . Quand cette référence est elle-même une fonction de valeur apprise, on obtient les méthodes **acteur-critique** : un « acteur » (la politique) et un « critique » (la valeur) apprennent ensemble. Les algorithmes modernes les plus répandus (PPO, SAC) en sont des variantes.

9.4.4 Pourquoi l'apprentissage devient instable

Avec une table, la convergence du Q-learning est garantie (9.3.6). Avec une fonction approchée, elle ne l'est plus. Sutton et Barto parlent de la **triade mortelle** (*deadly triad*) : trois ingrédients dont la **combinaison** peut faire diverger l'apprentissage, alors que chacun, isolément, est inoffensif.

1. l'**approximation de fonction** (le réseau), qui fait que mettre à jour un état modifie la valeur d'états voisins ;
2. l'**amorçage** (*bootstrapping*) : la cible dépend de l'estimation elle-même ;
3. l'apprentissage **hors politique** (*off-policy*) : on apprend sur des données produites par une autre politique.

Le tampon de rejeu et le réseau cible du DQN sont des **rustines** destinées à contenir ce risque, pas des garanties. En pratique, l'apprentissage par renforcement profond est connu pour être **sensible** : de petits changements d'hyperparamètres, ou de graine, produisent des résultats très différents, et les comparaisons d'algorithmes nécessitent de nombreuses répétitions (la rigueur expérimentale de la section 1.4 s'applique ici avec une force particulière).

9.4.5 Apprendre à partir de données déjà collectées

Une boutique a des **années d'historique** : quelles offres ont été envoyées à quels clients, avec quels résultats. Peut-on apprendre une politique à partir de ce journal, **sans** expérimenter sur de vrais clients ? C'est l'**apprentissage par renforcement hors ligne** (*offline RL*). C'est séduisant et difficile, pour une raison de fond : le journal ne contient que les décisions qui **ont été prises**. Pour une action jamais tentée dans un état donné, aucune donnée ne dit ce qui serait arrivé, et un algorithme naïf aura tendance à **surestimer** précisément ces actions inconnues (il en choisit alors de fausses « bonnes » idées).

Les remèdes sont de deux types : **rester proche de la politique historique** (méthodes dites conservatrices), et **évaluer d'abord, déployer ensuite** : on estime la valeur d'une nouvelle politique à partir des données de l'ancienne par **pondération par l'inverse des probabilités** (volume II, section 7.2), à condition d'avoir enregistré les probabilités avec lesquelles les actions ont été choisies. C'est le même problème que celui de l'inférence causale en données observationnelles (volume II, chapitre 7) : on veut connaître l'effet d'une action que l'on n'a pas toujours faite.

9.4.6 La récompense, ou ce que l'on demande vraiment

L'agent ne maximise pas ce que nous **voulons**, mais ce que nous **mesurons**. Si la récompense est mal spécifiée, l'agent trouvera la faille. C'est un phénomène bien documenté, sous des noms divers (*reward hacking*, loi de Goodhart : « quand une mesure devient un objectif, elle cesse d'être une bonne mesure »).

Dans notre petit monde, il suffirait que la récompense du problème de stock compte seulement le **nombre d'articles vendus**, en oubliant les coûts : l'agent apprendrait à **réapprovisionner chaque jour** pour ne jamais manquer une vente, quel qu'en soit le prix, et la boutique perdrait de l'argent (1,45 € par jour au lieu d'en gagner 0,27). Le cahier le vérifie par le calcul (application 9.6). Quelques situations réelles du même type :

- une recommandation récompensée par le **clic** apprend à fabriquer des titres racoleurs ;
- une politique de remises récompensée par le **chiffre d'affaires** brade les marges ;
- un agent récompensé par la **satisfaction déclarée** apprend à ne solliciter que les clients satisfaits.

9.4.7 Sécurité, éthique et responsabilité

Un agent qui **apprend en agissant** agit sur de vraies personnes pendant qu'il apprend. Trois points à garder en tête :

- **Explorer a un coût humain.** Tester une mauvaise offre sur un client est un coût réel, parfois irréversible (un client perdu). L'exploration doit être bornée, supervisée, et limitée à des actions dont les pires conséquences sont acceptables. C'est la même prudence que celle de SARSA en 9.3.4.
- **L'équité ne vient pas gratuitement.** Un agent qui maximise le profit peut proposer des prix ou des offres **différents selon des groupes** de clients, sans que personne ne l'ait demandé. Les outils d'audit de la section 5.4 s'appliquent aussi aux politiques apprises.
- **Manipulation.** Une politique optimisée sur l'attention ou les achats impulsifs peut exploiter les biais cognitifs des clients. Qu'un algorithme y parvienne ne signifie pas qu'on doive l'autoriser.

9.4.8 Quand utiliser l'apprentissage par renforcement ?

La plupart des problèmes de boutique **n'ont pas besoin** d'apprentissage par renforcement. Voici un guide de décision :

Votre situation	Outil adapté
Prédire une réponse à partir de données déjà collectées (churn, dépense)	apprentissage supervisé (chapitres 1 à 5)
Choisir parmi quelques options sans effet à long terme (bannière, objet d'un courriel)	bandit (9.2) ou test A/B
Même chose, avec des contextes (profil du visiteur)	bandit contextuel
Enchaîner des décisions dont l' effet se prolonge (stock, cycle de vie, tarification dynamique), avec un simulateur ou un modèle fiable	apprentissage par renforcement (9.1 à 9.3 ; profond si l'état est riche)
Mêmes décisions, sans simulateur , avec seulement un journal	RL hors ligne : avec une extrême prudence (9.4.5)
Problème de petite taille et modèle connu	programmation dynamique exacte (9.1), souvent suffisante

À retenir.

- Quand l'espace d'états explose, on remplace la table par une **fonction approchée** (réseau de neurones) : c'est l'idée du **DQN**, qui ajoute un tampon de rejeu et un réseau cible. Les **gradients de politique** et les méthodes **acteur-critique** optimisent directement la politique.
- Fonction approchée + amorçage + hors politique = **instabilité** possible ; l'apprentissage par renforcement profond est sensible aux hyperparamètres et aux graines.
- L'**apprentissage hors ligne** apprend d'un journal, mais ne peut pas juger les actions jamais essayées.
- **La récompense est le cahier des charges** : une récompense mal posée est exploitée à la lettre ; apprendre en agissant engage une **responsabilité** envers les personnes concernées.

Pour s'entraîner. Cahier, chapitre 9 : application 9.6 (une récompense mal posée).

Bilan du chapitre 9

Vous savez maintenant :

- **formuler** un problème de décision séquentielle comme un **processus de décision markovien** (états, actions, transitions, récompenses, facteur d'actualisation), et expliquer pourquoi la propriété de Markov est un choix de modélisation ;
- **démontrer** les **équations de Bellman** (d'espérance et d'optimalité), calculer à la main quelques itérations de l'**itération de la valeur** et justifier sa convergence par un argument de **contraction** ;
- lire une **politique optimale** dans Q^* , et comparer une politique apprise ou calculée à des règles simples avec un **gain moyen simulé** ;
- distinguer le **test A/B**, qui sert à **conclure**, des **bandits**, qui servent à **gagner en apprenant**, et mesurer un **regret** ; mettre en œuvre ϵ -**glouton**, **UCB** (et comprendre pourquoi ses constantes comptent) et l'**échantillonnage de Thompson** (lien avec l'inférence bayésienne du volume II) ;
- reconnaître l'intérêt d'un **contexte** (bandit contextuel) et les **biais** que crée une allocation adaptative ;
- écrire la règle du **Q-learning** et de **SARSA**, expliquer la différence entre **hors politique** et **sur la politique**, et énoncer les conditions de convergence de **Robbins et Monro** ;
- expliquer pourquoi la table ne suffit plus, ce que changent le **DQN** et les **gradients de politique**, et pourquoi l'apprentissage par renforcement profond est **fragile** ;
- poser les bonnes questions avant de déployer un agent : **la récompense mesure-t-elle ce que l'on veut ? Qui paie le coût de l'exploration ? Existe-t-il un simulateur ?**

Trois idées à emporter. **D'abord, agir et apprendre sont indissociables** : les données dépendent des décisions, donc l'évaluation doit se faire en conditions réelles de décision (le jeu de test mis de côté du chapitre 1 n'existe plus). **Ensuite, explorer a un prix** : c'est lui que mesurent le regret et l'écart entre les stratégies. **Enfin, la récompense est le cahier des charges** : l'algorithme optimise ce qu'on lui demande à la lettre.

Pour s'entraîner. Cahier, chapitre 9 : applications 9.1 à 9.6 et exercices 9.1 à 9.12.

Ce chapitre était le dernier des chapitres complémentaires. Le **projet du volume**, dans le cahier, rassemble les chapitres 1 à 5 en un pipeline complet sur un jeu de données réel, du modèle de référence à l'interprétation.

Points clés

« Un modèle vaut ce que vaut la façon dont on l'a évalué. »

Ce court chapitre fixe l'essentiel du volume, chapitre par chapitre, puis les idées qui les relient. Le **projet du volume** (un pipeline complet sur un jeu de données réel) et une **auto-évaluation** de quarante questions se trouvent dans le cahier.

Pour s'entraîner. Cahier, projet du volume et auto-évaluation : onze étapes, du cadrage à l'équité, puis quarante questions avec corrigés.

Chapitre	Ce que vous devez emporter
----------	----------------------------

Les points clés, chapitre par chapitre

Chapitre	Ce que vous devez emporter
1. La démarche	Un problème d'apprentissage se formule par la ligne du tableau , la date de prédiction et la cible . On sépare entraînement, validation et test ; on évite la fuite d'information (une variable connue après la prédiction, un prétraitement hors pipeline). La validation croisée estime la performance, mais l'écart-type entre plis n'est pas une barre d'erreur. L'erreur se décompose en bruit, biais² et variance ; on se compare à une référence , avec des différences appariées , et on n'ouvre le test qu' une fois . ⊕ Le réglage des hyperparamètres demande une validation croisée imbriquée .
2. Apprentissage supervisé	Un modèle = une famille de fonctions , une perte , un algorithme . Un arbre écrit des seuils et des interactions mais il est instable ; une forêt moyenne des arbres décorrélés (variance $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$) ; le gradient boosting est une descente de gradient dans l'espace des fonctions. Sur nos données, les modèles à arbres battent la logistique parce que le problème contient des seuils et des interactions : pas parce qu'ils sont plus sophistiqués .
3. Non supervisé	Une méthode non supervisée rend toujours un résultat . On le valide par plusieurs épreuves (critères internes, référence sans structure, stabilité, vérité externe si elle existe). L'échelle et le choix des variables décident des groupes. Réduire n'est pas neutre : l'erreur de reconstruction se mesure. ⊕ t-SNE et UMAP préservent le voisinage, pas les distances globales.
4. Variables et déséquilibre	Ce qui apprend, apprend sur l'entraînement, et rien que lui : encodage par la cible, imputation, mise à l'échelle, sélection, rééchantillonnage vivent dans un Pipeline. Un modèle simple muni de bonnes variables peut égaler un modèle flexible. Face au déséquilibre, poids et rééchantillonnage déplacent les probabilités sans améliorer le classement ; le seuil se choisit par les coûts.
5. Évaluation, calibration, interprétabilité	Une métrique ne vaut que par la décision qu'elle sert : AUC pour classer, précision moyenne quand la classe est rare, Brier pour les probabilités, coûts pour décider. On vérifie la calibration et on la répare sur un jeu séparé. On explique (permutation, PDP/ICE, LIME, SHAP) sans confondre description du modèle et causalité. ⊕ L'équité obéit à des critères qui ne peuvent pas tous être satisfaits ; la prédiction conforme donne une garantie de couverture.

Chapitre	Ce que vous devez emporter
⊗ 6. Anomalies	Sous déséquilibre extrême, l'exactitude ne veut rien dire ; on évalue par précision moyenne et budget d'alertes. Distances, densités, forêt d'isolement, autoencodeurs détectent des fraudes différentes ; un détecteur supervisé reste le meilleur quand les étiquettes existent.
⊗ 7. Recommandation	On classe plus qu'on ne prédit des notes. La popularité est une référence redoutable ; la personnalisation (voisinage, factorisation) ajoute quelques points. Protocole d'évaluation, démarrage à froid et boucles de rétroaction comptent plus que l'algorithme.
⊗ 8. Semi-supervisé et actif	Ni l'un ni l'autre n'est une baguette magique : chaque méthode repose sur une hypothèse sur les données qu'il faut mesurer à budget d'étiquettes égal. Un jeu étiqueté par apprentissage actif n'est pas représentatif .
⊗ 9. Renforcement	Agir et apprendre sont indissociables : les données dépendent des décisions. Explorer à un prix (le regret) ; la récompense est le cahier des charges de l'agent.
Projet (cahier)	Cadrer → auditer → séparer et fixer des références → variables → comparer avec incertitude → régler → décider par les coûts → calibrer → évaluer une fois → expliquer et auditer l'équité → rapporter ses limites.

Six idées qui traversent tout le volume

Les six idées.

1. **L'erreur d'entraînement est optimiste.** Seule l'erreur sur des données que le modèle n'a jamais vues dit quelque chose de l'avenir.
2. **Le jeu de test est sacré.** Chaque fois qu'il influence une décision, il devient un jeu de validation.
3. **La fuite d'information est l'erreur la plus fréquente et la plus coûteuse.** Elle se détecte par la question : « *quand cette valeur est-elle connue ?* »
4. **On bat d'abord une référence simple.** Un modèle plus riche doit justifier son surcroît de complexité par un gain *mesuré, avec son incertitude*.
5. **La métrique doit correspondre à la décision.** L'AUC, l'exactitude et le seuil de 0,5 répondent à trois questions différentes ; la bonne est souvent en euros.
6. **Interprétabilité et équité font partie de l'évaluation.** Un modèle que l'on ne sait pas expliquer, ou qui traite mal certains groupes, n'est pas terminé.

Et maintenant ?

Vous savez maintenant **construire, évaluer et expliquer** un modèle prédictif de façon rigoureuse. Le **volume IV : sujets avancés et modernes** prolonge ce travail vers les réseaux de neurones profonds, le traitement du langage et les modèles de langage, et le calcul distribué sur de gros volumes de données. Ces méthodes changent d'échelle et de matière première, pas d'exigence : tout ce que vous avez appris ici sur la **validation**, la **fuite d'information**, la **calibration** et l'**interprétabilité** y reste indispensable, souvent plus difficile à appliquer.

À retenir, tout simplement. Le modèle n'est qu'une étape. Ce qui fait la qualité d'un projet d'apprentissage automatique, c'est la rigueur avec laquelle on a posé la question, séparé les données, mesuré l'erreur et dit honnêtement ce que l'on ne sait pas.